

Documentation Quantum de Microsoft

Découvrez l'informatique quantique avec Microsoft Quantum et le service Azure Quantum. Utilisez Python et Q#, un langage pour la programmation quantique, pour créer et envoyer des programmes quantiques dans le portail Azure, ou configurer votre propre environnement de développement local avec le Kit de développement Quantique (QDK) Microsoft Quantum pour écrire des programmes quantiques.

À propos d'Azure Quantum

VUE D'ENSEMBLE

[Présentation d'Azure Quantum](#)

[Plans tarifaires Azure Quantum](#)

DÉPLOYER

[Créer un espace de travail quantique](#)

RÉFÉRENCE

[Liste des fournisseurs de matériel quantique](#)

ENTRAÎNEMENT

[Parcours d'apprentissage - Prise en main d'Azure Quantum](#)

Bien démarrer avec Q# et le Kit de développement Microsoft Quantum

TÉLÉCHARGER

[Configurer le Kit de développement Microsoft Quantum](#)

VUE D'ENSEMBLE

[Qu'est-ce que Q# ?](#)

DÉMARRAGE RAPIDE

[Créer votre premier programme Q#](#)

DIDACTICIEL

[Créer un générateur de nombres aléatoires quantiques](#)

[Explorer l'intrication quantique](#)

Développement avancé avec le Kit de développement Microsoft Quantum

DÉMARRAGE

[Différentes façons d'exécuter des programmes Q#](#)

VUE D'ENSEMBLE

[Présentation de l'estimateur de ressources Microsoft Quantum](#)

[Introduction à l'informatique quantique hybride](#)

En savoir plus sur l'informatique quantique

CONCEPT

[Qu'est-ce que l'informatique quantique ?](#)

[Le qubit](#)

[Vecteurs et matrices](#)

[Notation de Dirac](#)

[Circuits quantiques](#)

[Correction des erreurs quantiques](#)

Qu'est-ce que Azure Quantum ?

Azure Quantum est le service de calcul quantique en cloud de Microsoft Azure. Azure Quantum offre une voie ouverte, flexible et à l'épreuve du temps vers l'informatique quantique qui s'adapte à votre mode de travail.

Azure Quantum offre une gamme de solutions d'informatique quantique, y compris du matériel quantique provenant de fournisseurs de pointe, de logiciels quantiques et de services quantiques. Avec Azure Quantum, vous pouvez exécuter des programmes quantiques sur du matériel quantique réel, simuler des algorithmes quantiques et estimer les ressources nécessaires pour exécuter vos programmes quantiques sur des machines quantiques à l'échelle ultérieure.

Pour en savoir plus sur la façon dont vous pouvez utiliser l'informatique quantique et les algorithmes quantiques, consultez [Qu'est-ce que l'informatique quantique ?](#)

Comment se lancer avec Azure Quantum

Pour utiliser Azure Quantum, vous devez disposer d'un compte Azure et d'un espace de travail Azure Quantum. Pour développer des programmes quantiques et envoyer des travaux pour exécuter vos programmes sur Azure Quantum, utilisez le Microsoft Quantum Development Kit (QDK).

Pour obtenir un compte Azure, [inscrivez-vous gratuitement et inscrivez-vous à un abonnement de paiement à l'utilisation](#) [↗](#). Si vous êtes étudiant, vous pouvez tirer parti d'un [compte gratuit Azure pour les étudiants](#) [↗](#).

ⓘ Remarque

Vous n'avez pas besoin d'avoir un compte Azure pour utiliser le QDK.

Le Microsoft site web Quantum

Le site web [Microsoft Quantum](#) [↗](#) est une ressource centrale où vous pouvez explorer l'informatique quantique à Microsoft. Vous pouvez obtenir les dernières actualités et informations de Microsoft Quantum. Vous pouvez apprendre des experts et des passionnés par le biais de blogs, d'articles et de vidéos.

Vous n'avez pas besoin d'un compte Azure pour accéder aux ressources et informations sur le site web Microsoft Quantum.

Microsoft Quantum Development Kit.

Le Microsoft Quantum Development Kit (QDK) est un kit de développement logiciel conçu spécifiquement pour le développement quantique. Avec le QDK, vous pouvez écrire des programmes dans différents langages de programmation quantique, déboguer votre code, visualiser les circuits quantiques et les résultats, et envoyer des travaux aux fournisseurs de matériel quantique sur Azure Quantum. Le QDK prend en charge Microsoftle langage de programmation Q# ainsi que d'autres langages tels que Qiskit, Cirq et OpenQASM.

Le QDK est gratuit et open source. Pour commencer, installez l'extension QDK dans Visual Studio Code (VS Code) ou installez la bibliothèque QDK Python. Pour plus d'informations, consultez [Configurer le Microsoft Quantum Development Kit](#).

ⓘ Remarque

Pour exécuter des programmes sur Azure Quantum, vous devez disposer d'un espace de travail Azure Quantum. Pour plus d'informations, consultez [Créer un Azure Quantum espace de travail](#).

Le Azure portail

Si vous avez un compte Azure, utilisez le portail [Azure](#) pour créer un espace de travail Azure Quantum. Un Azure Quantum espace de travail est une collection de ressources associées à l'exécution de programmes quantiques. Pour plus d'informations, consultez [Créer un Azure Quantum espace de travail](#).

Avec le Azure portail, vous pouvez envoyer vos programmes quantiques à du matériel quantique réel, gérer votre Azure Quantum espace de travail, afficher des informations sur vos travaux quantiques et surveiller vos programmes quantiques.

Qu'est-ce que Q# ?

Q# est un langage de programmation quantique open source créé par Microsoft pour développer et exécuter vos programmes quantiques.

Vous pouvez considérer un programme quantique comme un ensemble de sous-routines classiques qui interagissent avec un système quantique pour effectuer un calcul. Un programme Q# ne modélise pas directement l'état quantique, mais décrit plutôt comment un ordinateur de contrôle classique interagit avec des qubits. Q# est indépendant du matériel. Vous n'avez donc pas besoin de prendre en compte les technologies qubit réelles lorsque vous écrivez des programmes Q#. Votre code Q# peut s'exécuter sur n'importe quelle technologie matérielle quantique.

Q# est un langage autonome qui offre un niveau élevé d'abstraction. Il n'y a aucune notion d'état quantique ou de circuit. Au lieu de cela, Q# implémente des programmes en termes d'instructions et d'expressions, comme des langages de programmation classiques tels que Python. Vous pouvez intégrer en toute transparence des structures de calcul quantique et classiques dans votre code Q#.

Pour plus d'informations, consultez [Présentation de Q#](#). Pour commencer à écrire du code Q#, consultez [Créer votre premier programme Q#](#).

Que puis-je faire avec Azure Quantum?

Azure Quantum offre un large éventail de services et d'outils pour vous aider à développer des solutions quantiques.

Pour plus d'informations sur Microsoft la recherche en informatique quantique, consultez la [Microsoft page Recherche en informatique quantique](#) .

Informatique quantique hybride

L'informatique quantique hybride fait référence aux processus et à l'architecture d'un ordinateur classique et d'un ordinateur quantique travaillant ensemble pour résoudre un problème. Avec la dernière génération d'architecture d'ordinateur quantique hybride disponible dans Azure Quantum, vous pouvez commencer à utiliser une approche hybride classique-quantique pour la programmation.

Pour plus [d'informations](#), consultez [Qu'est-ce que l'informatique quantique hybride ?](#)

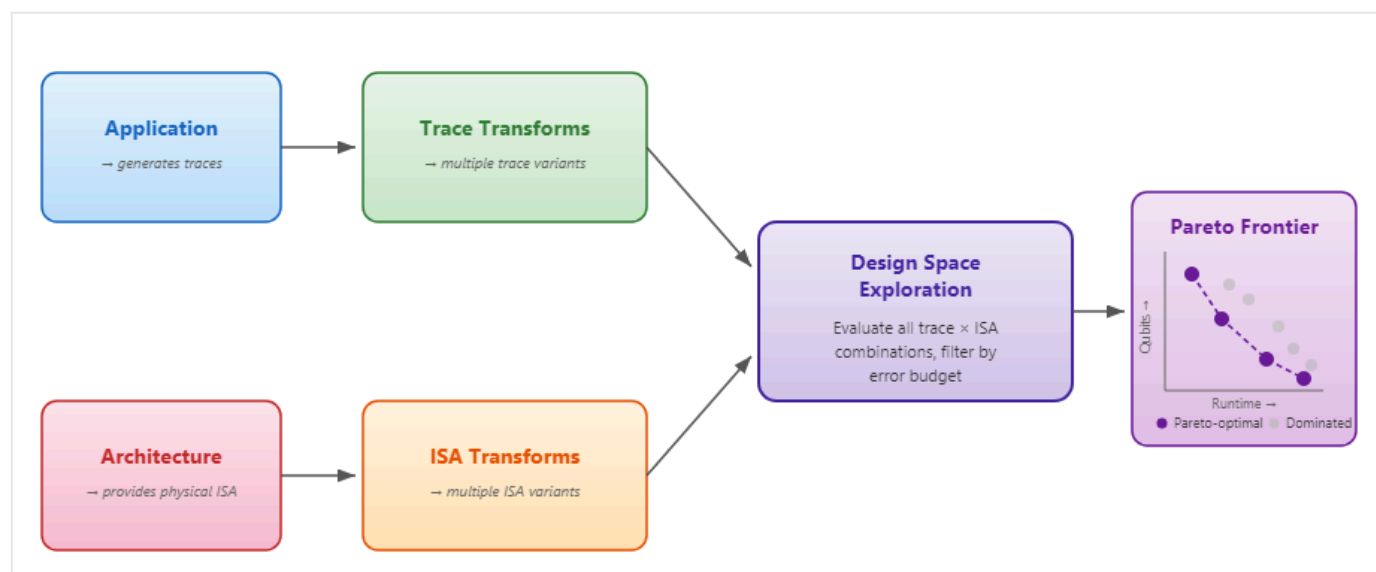
Estimation des ressources dans l'informatique quantique

Dans l'informatique quantique, l'estimation des ressources est une technique permettant de comprendre les ressources requises pour exécuter un algorithme sur un ordinateur quantique. Lorsque vous comprenez les besoins en ressources pour exécuter vos programmes sur différents

types de matériel quantique, vous pouvez préparer et affiner vos solutions quantiques pour qu'elles s'exécutent sur des machines quantiques à l'échelle ultérieure. Par exemple, l'estimation des ressources peut vous aider à déterminer la faisabilité de briser un algorithme de chiffrement particulier sur un type spécifique d'ordinateur quantique.

L'estimateur de ressources [Microsoft Quantum](#) vous permet d'évaluer les décisions architecturales, de comparer les technologies qubit et de déterminer les ressources dont vous avez besoin pour exécuter un algorithme quantique spécifique. Vous pouvez choisir parmi les protocoles prédéfinis à tolérance de panne ou concevoir vos propres modèles. L'estimateur de ressources calcule les estimations de ressources physiques post-disposition à partir d'un ensemble d'entrées telles que le budget d'erreur et les modèles de l'application quantique, l'architecture matérielle et le code QEC (Quantum Error Correction).

Pour commencer, consultez [Comment installer et utiliser l'estimateur Microsoft de ressources Quantum](#).



Simulations de chimie quantique avec Azure Quantum et QDK

Vous pouvez considérer la mécanique quantique comme le système d'exploitation sous-jacent de notre univers qui décrit comment les blocs de construction fondamentaux de la nature se comportent. Les réactions chimiques, les processus cellulaires et les propriétés matérielles sont toutes mécaniques quantiques dans la nature, et impliquent souvent des interactions entre un grand nombre de particules quantiques. Les ordinateurs quantiques ont promis de simuler des systèmes mécaniques quantiques, tels que des molécules, car les qubits peuvent être utilisés pour représenter les états quantiques naturels dans ces systèmes. Des exemples de systèmes quantiques que les qubits peuvent modéliser incluent la photosynthèse, la superconductivité et les formations moléculaires complexes.

Le QDK et Azure Quantum est conçu pour accélérer la découverte scientifique. Réinventez votre productivité de recherche et de développement avec des flux de travail de simulation optimisés pour la mise à l'échelle sur Azure des clusters HPC (High-Performance Computing), l'informatique accélérée par l'IA, l'intégration avec les outils quantiques et le matériel quantique, ainsi que l'accès futur à Microsoft l'ordinateur quantique.

Pour plus d'informations, consultez [Déverrouiller la puissance de Azure la dynamique moléculaire](#) .

Accélération quantique

Les ordinateurs quantiques font exceptionnellement bien avec des problèmes qui nécessitent des calculs d'un grand nombre de combinaisons possibles. Ces types de problèmes surviennent dans de nombreux domaines, tels que la simulation quantique, le chiffrement, le Machine Learning quantique et les problèmes de recherche.

L'un des objectifs de la recherche en informatique quantique est d'étudier les types de problèmes qu'un ordinateur quantique peut résoudre plus rapidement qu'un ordinateur classique, et combien plus rapide. Un exemple connu est l'algorithme de Grover, qui génère une accélération polynomiale sur les algorithmes classiques.

[L'algorithme de Grover](#) accélère la solution pour effectuer des recherches de données non structurées en exécutant la recherche en moins d'étapes que n'importe quel algorithme classique. En général, les problèmes qui vous permettent de vérifier si une valeur donnée est une solution valide (un « oui ou aucun problème ») peuvent être formulés en termes de problème de recherche.

Pour une implémentation de l'algorithme de Grover, consultez [Tutoriel : Implémenter l'algorithme de recherche de Grover dans Q#](#).

Fournisseurs quantiques disponibles sur Azure Quantum

Azure Quantum offre aujourd'hui quelques-unes des ressources quantiques les plus attrayantes et diversifiées disponibles auprès des leaders de l'industrie. Azure Quantum partenaires actuellement avec les fournisseurs suivants pour vous permettre d'exécuter vos programmes quantiques sur du matériel réel ou sur des simulateurs matériels.

Choisissez le fournisseur qui correspond le mieux aux caractéristiques de votre problème et à vos besoins.

- [IonQ](#) : Ordinateurs quantiques à ions piégés dynamiquement reconfigurables, comportant jusqu'à 36 qubits entièrement interconnectés, permettant d'exécuter une porte à deux qubits entre n'importe quelle paire de qubits.
- [Pasqal](#) : processeurs quantiques neutres basés sur des atomes qui fonctionnent à température ambiante, avec des temps de cohérence longs et une connectivité qubit impressionnante.
- [Quantinuum](#) : Systèmes d'ions piégés avec qubits de haute fidélité, entièrement connectés, faibles taux d'erreur, réutilisation des qubits, et la possibilité de prendre une mesure à mi-circuit.
- [Rigetti](#) : alimenté par des processeurs quantiques basés sur des qubits superconducteurs, ces systèmes offrent des temps de porte rapides, une logique conditionnelle à faible latence et des temps d'exécution de programme rapides.

Pour plus d'informations sur les spécifications de chaque fournisseur, consultez la liste complète de l'informatique quantique.

Pour plus d'informations sur le coût du travail, consultez [tarification dans Azure Quantum](#) et [FAQ : Présentation des coûts et de la facturation des travaux dans Azure Quantum](#).

Contenu connexe

Pour commencer à utiliser Azure Quantum, explorez les liens suivants :

- [Créer un Azure Quantum espace de travail](#)
- [Bien démarrer avec Q# et Visual Studio Code](#)
- [Installer le Microsoft Quantum Development Kit](#)
- [Exécuter un circuit Qiskit dans Azure Quantum](#)

Last updated on 17/06/2026

Qu'est-ce que l'informatique quantique ?

29/09/2025

L'informatique quantique est la promesse de résoudre certains des plus grands défis de notre planète - dans les domaines de l'environnement, de l'agriculture, de la santé, de l'énergie, du climat, de la science des matériaux, etc. Pour certains de ces problèmes, l'informatique classique peine de plus en plus à mesure que la taille du système augmente. Lorsqu'ils sont conçus pour être mis à l'échelle, les systèmes quantiques auront probablement des capacités qui dépassent celles des supercalculateurs les plus puissants d'aujourd'hui.

Cet article explique les principes de l'informatique quantique, comment il compare à l'informatique classique et comment il utilise les principes de la mécanique quantique.

Histoire de l'informatique quantique

Les systèmes quantiques, tels que les atomes et les molécules, peuvent être difficiles ou impossibles à simuler sur un ordinateur classique. Dans les années 1980, Richard Feynman et Yuri Manin ont suggéré qu'un matériel basé sur des phénomènes quantiques pourrait être plus efficace pour la simulation des systèmes quantiques que les ordinateurs conventionnels.

Il existe plusieurs raisons pour lesquelles les systèmes quantiques sont difficiles à simuler sur des ordinateurs réguliers. L'une des principales raisons est que la question, au niveau quantique, est décrite comme une combinaison de plusieurs configurations (appelées états) en même temps.

Les états quantiques augmentent de façon exponentielle

Considérez un système de particules et 40 emplacements possibles où ces particules peuvent exister. Le système peut se trouver dans un état unique de 2^{40} car chaque emplacement peut avoir ou non une particule. S'il s'agit de particules classiques, le système est toujours dans l'un des 2^{40} états, donc un ordinateur classique n'a besoin que de 40 bits pour décrire l'état du système. Mais s'il s'agit de particules quantiques, le système existe dans une combinaison de tous les états 2^{40} . Un ordinateur classique doit stocker 2^{40} nombres pour décrire le système quantique, qui nécessite plus de 130 Go de mémoire. Toutefois, un ordinateur quantique n'a besoin que de 40 bits quantiques pour décrire ce système quantique.

Si nous ajoutons un autre emplacement au système afin que les électrons puissent exister dans 41 emplacements, le nombre de configurations uniques du système double à 2^{41} . Il faudrait plus de 260 Go de mémoire pour stocker cet état quantique sur un ordinateur classique. Nous ne pouvons pas jouer à ce jeu d'augmenter le nombre d'emplacements pour toujours. Pour stocker un état quantique sur un ordinateur conventionnel, vous dépassez rapidement les

capacités de mémoire des machines les plus puissantes du monde. À quelques centaines d'électrons, la mémoire nécessaire pour stocker le système dépasse le nombre de particules dans l'univers. Il n'y a pas d'espoir avec nos ordinateurs conventionnels pour simuler complètement la dynamique quantique pour les systèmes plus grands !

Transformer la difficulté en opportunité

L'observation de cette croissance exponentielle pose une question puissante : est-il possible de transformer cette difficulté en opportunité ? Si les systèmes quantiques sont difficiles à simuler sur des ordinateurs réguliers, que se passe-t-il si nous créons une machine qui utilise des effets quantiques pour ses opérations fondamentales ? Pouvons-nous simuler des systèmes quantiques avec une machine qui exploite exactement les mêmes lois de physique ? Et pourrions-nous utiliser cette machine pour examiner d'autres problèmes importants en dehors de la mécanique quantique ? Il s'agit des types de questions qui ont donné lieu aux champs de l'information quantique et de l'informatique quantique.

En 1985, David Deutsch a montré qu'un ordinateur quantique pouvait simuler efficacement le comportement de n'importe quel système physique. Cette découverte a été la première indication que les ordinateurs quantiques pouvaient être utilisés pour résoudre les problèmes trop difficiles à résoudre sur les ordinateurs classiques.

En 1994, Peter Shor a découvert un algorithme quantique pour trouver les principaux facteurs de grands entiers. L'algorithme de Shor s'exécute de manière exponentielle plus rapide que l'algorithme classique le plus connu pour ce problème de factoring. Un tel algorithme rapide pourrait potentiellement briser la plupart de nos systèmes de chiffrement de clés publiques modernes que nous utilisons pour sécuriser les transactions dans le commerce électronique, telles que Rivest-Shamir-Adleman (RSA) et le chiffrement de courbe elliptique. Cette découverte a suscité un énorme intérêt pour l'informatique quantique et a conduit au développement d'algorithmes quantiques pour de nombreux autres problèmes.

Depuis ce temps, des algorithmes d'ordinateur quantique rapides et efficaces ont été développés pour d'autres problèmes difficiles à résoudre sur les ordinateurs classiques. Par exemple, nous avons maintenant des algorithmes quantiques pour rechercher une base de données non ordonnée, pour résoudre des systèmes d'équations linéaires, pour effectuer le Machine Learning et simuler des systèmes physiques dans la chimie, la physique et la science des matériaux.

Présentation d'un qubit

Tout comme les bits sont l'objet fondamental des informations dans l'informatique classique, les qubits (bits quantiques) sont l'objet fondamental des informations dans l'informatique

quantique.

Les qubits jouent un rôle similaire dans l'informatique quantique à mesure que les bits jouent dans l'informatique classique, mais les qubits se comportent différemment des bits. Les bits classiques sont binaires et, à tout moment, peuvent se trouver dans un seul état, 0 ou 1. Mais les qubits peuvent être dans une superposition des états 0 et 1 en même temps. En fait, il existe des superpositions infinies possibles de 0 et 1, et chacun d'eux est un état qubit valide.

Dans l'informatique quantique, les informations sont encodées dans des superpositions des états 0 et 1. Par exemple, 8 bits réguliers peuvent encoder jusqu'à 256 valeurs uniques, mais ces 8 bits ne peuvent représenter qu'une des 256 valeurs à la fois. Avec 8 qubits, nous pourrions encoder toutes les 256 valeurs en même temps, car les qubits peuvent être dans une superposition de tous les 256 états possibles.

Pour plus d'informations, consultez [Le qubit dans l'informatique](#) quantique.

Quelles sont les conditions requises pour créer un ordinateur quantique ?

Un ordinateur quantique utilise des systèmes quantiques et les propriétés de la mécanique quantique pour résoudre les problèmes de calcul. Les systèmes d'un ordinateur quantique se composent des qubits, des interactions entre les qubits et des opérations sur les qubits pour stocker et calculer des informations. Nous pouvons utiliser des ordinateurs quantiques pour programmer des effets tels que l'intrication quantique et l'interférence quantique pour résoudre avec précision certains problèmes plus rapidement que sur les ordinateurs classiques.

Pour créer un ordinateur quantique, nous devons déterminer comment créer et stocker les qubits. Nous devons également réfléchir à la manipulation des qubits et à la mesure des résultats de nos calculs.

Les technologies qubit populaires incluent des qubits d'ion piégés, des qubits superconducteurs et des qubits topologiques. Pour certaines méthodes de stockage qubit, l'unité qui héberge les qubits doit être conservée à une température proche de zéro absolu pour maximiser leur cohérence et réduire les interférences. D'autres types d'hébergements des qubits utilisent une chambre à vide pour réduire au maximum les vibrations et stabiliser les qubits. Les signaux peuvent être envoyés aux qubits par le biais de différentes méthodes, telles que les micro-ondes, les lasers ou les tensions.

Les cinq critères d'un ordinateur quantique

Un bon ordinateur quantique doit avoir ces cinq fonctionnalités :

1. **Évolutif** : Il peut avoir de nombreux qubits.
2. **Initialisable** : Il peut définir les qubits sur un état spécifique (généralement l'état 0).
3. **Résilient** : Il peut conserver les qubits dans l'état de superposition pendant une longue période.
4. **Universel** : Un ordinateur quantique n'a pas besoin d'effectuer toutes les opérations possibles, seulement un ensemble d'opérations appelé *jeu universel*. Un [ensemble d'opérations](#) quantiques universelles est tel que toute autre opération peut être décomposée en une séquence d'opérations.
5. **Fiable** : Il peut mesurer les qubits avec précision.

Ces cinq critères sont souvent appelés [critères](#)  Di Vincenzo pour le calcul quantique.

La création d'appareils qui répondent à ces cinq critères est l'un des défis d'ingénierie les plus exigeants jamais rencontrés par l'humanité. Azure Quantum offre une variété de solutions d'informatique quantique avec différentes technologies qubit. Pour plus d'informations, consultez [la liste complète des fournisseurs](#) Azure Quantum.

Comprendre les phénomènes quantiques

Les phénomènes quantiques sont les principes fondamentaux qui différencient l'informatique quantique de l'informatique classique. Comprendre ces phénomènes est essentiel pour comprendre comment les ordinateurs quantiques fonctionnent et pourquoi ils détiennent ce potentiel. Les deux phénomènes quantiques les plus importants sont la superposition et l'intrication.

Superposition

Imaginez que vous êtes en train de faire de l'exercice dans votre salon. Vous vous tournez entièrement vers la gauche, puis entièrement vers la droite. À présent, vous essayez de vous tourner vers la gauche et vers la droite en même temps. C'est impossible à faire, à moins de vous couper en deux ! Vous ne pouvez évidemment pas être dans ces deux états simultanément : être face au côté gauche et face au côté droit au même moment.

En revanche, si vous étiez une particule quantique, vous pourriez avoir une certaine probabilité d'être *face au côté gauche* ET une certaine probabilité d'être *face au côté droit* grâce à un phénomène connu sous le nom de *superposition* (ou *cohérence*).

Seuls les systèmes quantiques tels que les ions, les électrons ou les circuits superconducteurs peuvent exister dans les états de superposition qui permettent la puissance du calcul quantique. Par exemple, les électrons sont des particules quantiques qui ont leur propre propriété d'« orientation gauche ou droite » appelée spin. Les deux états de spin sont appelés

spin up et spin down, et l'état quantique d'un électron est une superposition des états spin up et spin down.

Si vous souhaitez en savoir plus et pratiquer la superposition, consultez [le module d'entraînement : Explorer la superposition avec Q#](#).

Intrication

Entanglement est une corrélation quantique entre deux systèmes quantiques ou plus. Lorsque deux qubits sont enchevêtrés, ils sont corrélés et partagent les informations de leurs états afin que l'état quantique des qubits individuels ne puisse pas être décrit indépendamment. Avec l'inanglement quantique, vous ne pouvez connaître que l'état quantique du système global, et non les états individuels.

Les systèmes quantiques enchevêtrés maintiennent cette corrélation même lorsqu'ils sont séparés sur de grandes distances. En d'autres termes, quelle que soit l'opération ou le processus que vous appliquez à un sous-système, il est corrélé sur l'autre sous-système également. Ainsi, la mesure de l'état d'un qubit fournit des informations sur l'état de l'autre qubit : cette propriété particulière est très utile dans l'informatique quantique.

Si vous souhaitez en savoir plus, consultez [Tutoriel : Explorer l'inanglement quantique avec Q#](#) et, pour une implémentation pratique, consultez le [module d'entraînement : Teleport a qubit à l'aide d'un entanglement](#).

Contenu connexe

- [Vecteurs et matrices dans l'informatique quantique](#)
- [Qubit](#)
- [Conventions de diagramme de circuit quantique](#)
- [Inanglement dans l'informatique quantique](#)
- [Présentation du langage de programmation quantique Q#](#)

📄 **Remarque** : L'auteur a créé cet article avec l'aide de l'IA. [En savoir plus](#)

Machine quantique de Microsoft

Microsoft est en bonne voie pour fournir une machine quantique dans le cadre du service Azure Quantum. Vous pouvez en savoir plus sur ce parcours dans [ce blog](#). Cela fait partie d'un plan complet visant à donner aux innovateurs des solutions quantiques pour un impact révolutionnaire.

En 2022, Microsoft a eu une percée scientifique majeure qui a effacé un obstacle important à la création d'une machine quantique mise à l'échelle avec des qubits topologiques. Bien que les défis techniques restent, cette nouvelle découverte illustre un bloc de construction fondamental pour son approche de l'informatique quantique mise à l'échelle. Pour plus d'informations sur cette découverte scientifique, lisez le [blog](#), regardez une [vidéo](#) ou lisez la [prépublication](#) de l'article.

Afficher un notebook Jupyter contenant les résultats

L'équipe Azure Quantum a créé un ensemble de notebooks Jupyter sur le [site GitHub azure-quantum-tgp](#), qui affichent toutes les analyses et les graphiques figurant dans l'article. À partir de la page GitHub, vous pouvez également afficher les résultats du bloc-notes exécutés avec [nbviewer](#).

Étapes suivantes

- [Qu'est-ce qu'Azure Quantum ?](#)
- [Introduction à Q#](#)
- [Créer des solutions quantiques avec la Microsoft Quantum Development Kit](#)

Créer un espace de travail Azure Quantum

Découvrez comment créer un espace de travail [Azure Quantum](#) dans le portail Azure. Une ressource d'espace de travail Azure Quantum, ou simplement appelée espace de travail, est un ensemble d'éléments associés à l'exécution d'applications quantiques.

Vous avez besoin d'un espace de travail pour envoyer des programmes quantiques au matériel quantique.

Conseil

Vous pouvez également créer un espace de travail Azure Quantum à l'aide de l'interface de ligne de commande (CLI) Azure. Pour plus d'informations, consultez [Gérer les espaces de travail quantiques avec Azure CLI](#).

Prerequisites

Vous devez disposer d'un compte Azure avec un abonnement actif pour créer un espace de travail Azure Quantum. Si vous n'en avez pas, vous pouvez choisir parmi l'un des abonnements suivants, ou pour obtenir une liste complète, consultez [les détails de l'offre Microsoft Azure](#) .

 Agrandir le tableau

Abonnement Azure	Descriptif
Païement à l'utilisation (recommandé)	Vous payez les services que vous utilisez et vous pouvez annuler à tout moment.
Enterprise Agreement	Si votre organisation dispose d'un contrat d'achat Entreprise (EA) avec Microsoft, les propriétaires de compte de votre organisation peuvent créer des abonnements Enterprise Dev/Test  pour les abonnés Visual Studio actifs sous l'EA.

Si vous avez des questions ou rencontrez des problèmes lorsque vous utilisez Azure Quantum, contactez AzureQuantumInfo@microsoft.com.

Créer l'espace de travail

Pour créer un espace de travail Azure Quantum, procédez comme suit.

1. Connectez-vous au [portail Azure](#) à l'aide des informations d'identification de votre abonnement Azure.
2. Dans la page d'accueil du portail Azure, sélectionnez **Créer une ressource**, puis, dans **La Place de marché**, entrez **Azure Quantum**. Dans la page de résultats, vous devez voir une vignette pour le service **Azure Quantum**.
3. Sélectionnez **Azure Quantum**, puis **Create**. Le formulaire de création de l'espace de travail s'ouvre.
4. Sélectionnez un abonnement à associer au nouvel espace de travail.
5. Sélectionnez **Création rapide** ou **Création avancée**.

Création rapide

Cette option est le chemin le plus simple pour créer un espace de travail. Il crée automatiquement le groupe de ressources et le compte de stockage nécessaires, ajoute les fournisseurs IonQ, Quantinuum, Rigetti et Microsoft Quantum Computing. Votre espace de travail peut toujours être personnalisé après la création, si nécessaire.

ⓘ Remarque

Pour utiliser la **création rapide**, vous devez être **propriétaire** de l'abonnement que vous avez sélectionné à l'étape précédente. Pour afficher la liste de vos abonnements et de vos accès, consultez [Vérifier vos attributions de rôles](#).

1. Entrez un nom pour l'espace de travail.
2. Sélectionnez la région de l'espace de travail.
3. Cliquez sur **Créer**.

ⓘ Remarque

Si vous rencontrez des problèmes pendant le déploiement, consultez [Les problèmes courants liés à Azure Quantum : création d'un espace de travail Azure Quantum](#).

Le déploiement de votre espace de travail peut prendre quelques minutes. Le portail met à jour les détails de l'état et du déploiement. Une fois le message **complet de votre déploiement affiché**, affichez les **détails du déploiement** ou sélectionnez **Accéder à la ressource**.

ⓘ Remarque

Si vous rencontrez des problèmes, consultez [les problèmes courants d'Azure Quantum : création d'un espace de travail Azure Quantum](#).

Last updated on 14/08/2026

Se connecter à votre espace de travail Azure Quantum avec le QDK ou Azure CLI

Si vous disposez d'un espace de travail Azure Quantum, vous pouvez vous connecter à votre espace de travail et envoyer votre code avec le `qdk.azure` module Python. Le `qdk.azure` module fournit une [Workspace classe](#) qui représente un espace de travail Azure Quantum.

Prérequis

Pour vous connecter à votre espace de travail avec le `qdk.azure` module, vous avez besoin des éléments suivants :

- Compte Azure avec un abonnement actif. Si vous n'avez pas de compte Azure, inscrivez-vous gratuitement et inscrivez-vous à un abonnement [pay-as-you-go](#) [↗](#).
- Un espace de travail Azure Quantum. Si vous n'avez pas d'espace de travail, consultez [Créer un espace de travail Azure Quantum](#).
- La dernière version du `qdk` paquet Python avec l'option supplémentaire `azure`.

Bash

```
pip install --upgrade "qdk[azure]"
```

- Si vous utilisez Azure CLI, mettez à jour vers la dernière version. Pour obtenir des instructions d'installation, consultez :
- [Installer Azure CLI sur Windows](#)
- [Installer Azure CLI sur Linux](#)
- [Installer Azure CLI sur macOS](#)

Se connecter avec une chaîne de connexion

Utilisez un chaîne de connexion pour spécifier les paramètres de connexion à un espace de travail Azure Quantum. Les chaînes de connexion sont utiles dans les scénarios suivants :

- Vous souhaitez partager l'accès à votre espace de travail avec d'autres personnes qui n'ont pas de compte Azure.
- Vous souhaitez partager l'accès à votre espace de travail avec d'autres personnes pendant une durée limitée.
- Vous ne pouvez pas utiliser l'ID Microsoft Entra en raison de stratégies d'entreprise.

Conseil

Chaque espace de travail Azure Quantum a une clé primaire et une clé secondaire, et chaque clé a sa propre chaîne de connexion. Pour permettre aux autres utilisateurs d'accéder à votre espace de travail, partagez la clé secondaire et utilisez la clé primaire uniquement pour vos propres services. Vous pouvez remplacer la clé secondaire sans provoquer de temps d'arrêt dans vos propres services. Pour plus d'informations sur le partage d'accès à votre espace de travail, consultez [Partager l'accès à votre espace de travail](#).

Copier la chaîne de connexion

1. Connectez-vous au portail [Azure](#).
2. Accédez à votre espace de travail Azure Quantum.
3. Dans le panneau de l'espace de travail, développez la liste déroulante **Opérations** et sélectionnez **Clés d'accès**.
4. Activez les clés d'accès pour votre espace de travail. Si le curseur **Touches d'accès** est défini sur **Désactivé**, définissez le curseur sur **Activé**.
5. Sélectionnez l'icône **Copier** pour l'chaîne de connexion que vous souhaitez copier. Vous pouvez choisir la chaîne de connexion primaire ou secondaire.

Avertissement

Il s'agit d'un risque de sécurité pour stocker vos clés d'accès de compte ou les chaînes de connexion en texte clair. Stockez vos clés de compte dans un format chiffré ou migrez vos applications pour utiliser l'autorisation Microsoft Entra pour accéder à votre espace de travail Azure Quantum.

Utilisez le chaîne de connexion pour accéder à votre espace de travail Azure Quantum

Utilisez le chaîne de connexion que vous avez copié pour vous connecter à votre espace de travail Azure Quantum avec le `qdk.azure` module ou l'extension QDK dans Visual Studio Code.

Python

Pour vous connecter à votre espace de travail Azure Quantum, choisissez l'une des méthodes suivantes.

- Utilisez la `from_connection_string` fonction pour créer un `Workspace` objet.

Python

```
from qdk.azure import Workspace

connection_string = "" # Add your connection string
workspace = Workspace.from_connection_string(connection_string)

print(workspace.get_targets())
```

- Stockez votre chaîne de connexion dans une variable d'environnement.

Python

```
import os
from qdk.azure import Workspace

connection_string = "" # Add your connection string
os.environ["AZURE_QUANTUM_CONNECTION_STRING"] = connection_string

workspace = Workspace()
print(workspace.get_targets())
```

Pour plus d'informations sur l'utilisation des clés, consultez [Gérer vos clés d'accès](#).

❗ Important

Si vous désactivez les clés d'accès pour votre espace de travail, vous ne pouvez pas utiliser de chaînes de connexion pour vous connecter à votre espace de travail. Utilisez plutôt des paramètres d'espace de travail pour vous connecter.

Se connecter à votre espace de travail avec des paramètres d'espace de travail

Chaque espace de travail Azure Quantum a un ensemble unique de paramètres que vous pouvez utiliser pour vous connecter à l'espace de travail.

 Agrandir le tableau

Paramètre	Description	Utiliser quand vous vous connectez avec
<code>subscription_id</code>	ID de l'abonnement Azure.	Azure CLI
<code>resource_group</code>	Le nom du groupe de ressources Azure.	Azure CLI
<code>name</code>	Nom de votre espace de travail Azure Quantum.	Azure CLI
<code>resource_id</code>	ID de ressource Azure de l'espace de travail Azure Quantum.	Python

Pour rechercher les paramètres de votre espace de travail, procédez comme suit :

1. Connectez-vous au portail [Azure](#).
2. Accédez à votre espace de travail Azure Quantum.
3. Dans le panneau de votre espace de travail, choisissez **Vue d'ensemble**.
4. Développez la liste déroulante **Essentials**.
5. Copiez les paramètres dans leurs champs correspondants.

ⓘ Remarque

Vérifiez que vous vous connectez au locataire approprié avant de vous connecter à votre espace de travail. Pour plus d'informations sur les locataires, consultez [Comment gérer les abonnements Azure avec Azure CLI](#).

Utiliser des paramètres d'espace de travail pour vous connecter à votre espace de travail Azure Quantum

Pour rester connecté lorsque vous exécutez vos scripts Python, ouvrez un terminal et exécutez la commande Azure CLI suivante. Cette commande définit l'abonnement pour votre espace de travail. Remplacez `<subscriptionId>` par votre ID d'abonnement.

Azure CLI

```
az account set --subscription <subscriptionId>
```

Vous pouvez utiliser les paramètres de votre espace de travail pour vous connecter à votre espace de travail Azure Quantum via le `qdk.azure` module ou via Azure CLI.

Python

Pour vous connecter à votre espace de travail Azure Quantum, choisissez l'une des options suivantes.

- Spécifiez l'ID de ressource (recommandé) :

Python

```
from qdk.azure import Workspace

workspace = Workspace(resource_id="") # Add the resource ID of your
workspace
```

- Spécifiez l'ID d'abonnement, le groupe de ressources et le nom de l'espace de travail :

Python

```
from qdk.azure import Workspace

workspace = Workspace(
    subscription_id="", # Add the subscription ID of your workspace
    resource_group="", # Add the resource group of your workspace
    name="" # Add the name of your workspace
)
```

- Spécifiez uniquement le nom de l'espace de travail. Cette option peut échouer lorsque vous avez plusieurs espaces de travail portant le même nom dans le même locataire.

Python

```
from qdk.azure import Workspace

workspace = Workspace(name="") # Add the name of your workspace
```

Contenu connexe

- [Gérer des espaces de travail quantiques avec Azure CLI](#)
- [Partager l'accès à votre espace de travail Azure Quantum](#)
- [Gérer des espaces de travail quantiques avec Azure Resource Manager](#)

- [S'authentifier à l'aide d'un principal de service](#)
 - [S'authentifier à l'aide d'une identité managée](#)
-

Last updated on 21/08/2026

Présentation de l'accès en fonction du rôle à votre espace de travail Azure Quantum

Article • 13/12/2024

Découvrez les différents principaux et rôles de sécurité que vous pouvez utiliser pour gérer l'accès à votre espace de travail Azure Quantum.

Contrôle d'accès en fonction du rôle Azure (RBAC)

Le [contrôle d'accès en fonction du rôle Azure \(Azure RBAC\)](#) est le système d'autorisation que vous utilisez pour gérer l'accès aux ressources Azure, comme un espace de travail. Pour accorder l'accès, vous attribuez des rôles à un principal de sécurité.

Principal de sécurité

Un principal de sécurité est un objet qui représente un utilisateur, un groupe, un principal de service ou une identité managée.

 Agrandir le tableau

Principal de sécurité	Définition
Utilisateur	Un compte d'utilisateur qui se connecte à Azure pour créer, gérer et utiliser des ressources.
Groupe	Groupe d'utilisateurs. Permet de gérer les utilisateurs qui ont besoin des mêmes accès et autorisations aux ressources.
Principal du service	Identité utilisateur pour une application, un service ou une plateforme qui doit accéder aux ressources.
Identité gérée	Une identité managée automatiquement dans Azure Active Directory (Azure AD) pour les applications à utiliser lors de la connexion aux ressources qui prennent en charge l'authentification Azure AD.

Rôle

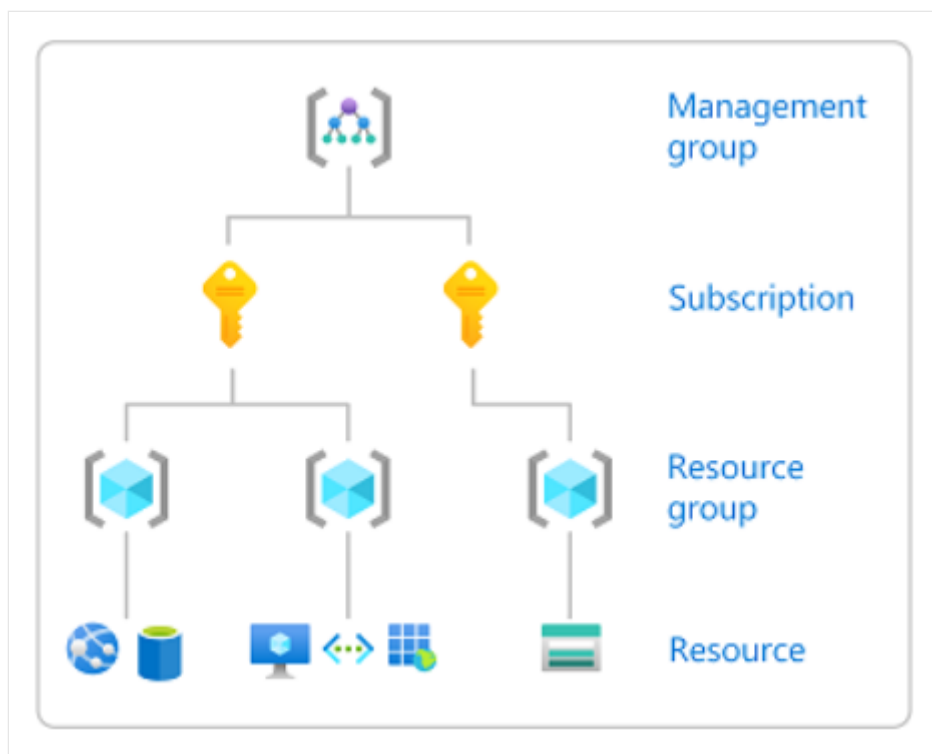
Lorsque vous accordez l'accès à un principal de sécurité, vous attribuez un **rôle** intégré ou créez un **rôle** personnalisé. Les rôles intégrés les plus couramment utilisés sont **Propriétaire, Contributeur, Contributeur de données Quantum Workspace** et **Lecteur** .

 Agrandir le tableau

Rôle	Niveau d'accès
Owner	Octroie un accès total pour gérer toutes les ressources, notamment la possibilité d'attribuer des rôles dans Azure RBAC.
Contributeur	Octroie un accès complet pour gérer toutes les ressources, mais ne vous permet pas d'attribuer des rôles dans Azure RBAC.
Contributeur aux données de l'espace de travail Quantum	Accorde l'accès aux travaux d'envoi et d'affichage dans l'espace de travail, mais ne vous permet pas de créer, de supprimer ou de modifier un espace de travail.
Lecteur	Voir toutes les ressources, mais ne vous autorise pas à apporter des modifications.

Étendue

Les rôles sont attribués à une étendue particulière. Étendue représente l'ensemble des ressources auxquelles l'accès s'applique. Les étendues sont structurées dans une relation parent-enfant. Chaque niveau de hiérarchie rend l'étendue plus spécifique. Le niveau que vous sélectionnez détermine la portée d'application du rôle. Les niveaux inférieurs héritent des autorisations de rôle des niveaux supérieurs. Vous pouvez attribuer des rôles à quatre niveaux d'étendue : groupe d'administration, abonnement, groupe de ressources ou ressource.



[Agrandir le tableau](#)

Étendue	Description
Groupe d'administration	Vous aide à gérer l'accès, la stratégie et la conformité pour plusieurs abonnements. Tous les abonnements dans un groupe d'administration héritent automatiquement des conditions appliquées à ce groupe d'administration. Vous aurez peut-être besoin d'un groupe d'administration si votre organisation dispose de plusieurs abonnements.
Abonnement	Associe logiquement les comptes d'utilisateur aux ressources qu'ils créent. Un compte d'utilisateur est une identité d'utilisateur et un ou plusieurs abonnements. Un abonnement représente un regroupement de ressources Azure. La facture est générée dans l'étendue de l'abonnement. Vous devez disposer d'un compte avec un abonnement actif pour créer des ressources Azure. Pour connaître les options d'abonnement, consultez Créer un espace de travail Azure Quantum .
Groupe de ressources	Conteneur qui contient les ressources associées d'une solution Azure. Le groupe de ressources inclut les ressources que vous voulez gérer en tant que groupe. Par exemple, les ressources suivantes sont requises pour exécuter des applications dans Azure Quantum : • Compte de stockage Azure : stocke les données d'entrée et de sortie pour les travaux quantiques. • Espace de travail Azure Quantum : collection de ressources associées à l'exécution d'applications quantiques. Ces ressources vivent dans un seul groupe de ressources.

Étendue	Description
Ressource	Instance d'un service que vous pouvez créer, comme un espace de travail ou un compte de stockage.

ⓘ Notes

Étant donné que l'accès peut être étendu à plusieurs niveaux dans Azure, un utilisateur peut avoir des rôles différents à chaque niveau. Par exemple, un utilisateur disposant d'un accès propriétaire à un espace de travail peut ne pas disposer d'un accès propriétaire à un groupe de ressources contenu dans cet espace de travail.

Conditions requises pour la création d'un espace de travail

Lorsque vous [créez un espace de travail](#), vous sélectionnez d'abord un abonnement, un groupe de ressources et un compte de stockage à associer à l'espace de travail. Votre capacité à créer un espace de travail dépend des niveaux d'accès dont vous disposez, en commençant par l'étendue de l'abonnement. Pour afficher votre autorisation pour différentes ressources, consultez [Vérifier vos attributions de rôles](#).

Propriétaire de l'abonnement

Les propriétaires d'abonnements peuvent créer des espaces de travail à l'aide des **options de création rapide** ou **de création avancée**. Vous pouvez choisir un groupe de ressources et un compte de stockage qui existe déjà sous l'abonnement ou en créer de nouveaux. Vous avez également la possibilité d'attribuer [des rôles](#) à d'autres utilisateurs.

Collaborateur de l'abonnement

Les contributeurs d'abonnement peuvent créer des espaces de travail à l'aide de l'option **De création avancée**.

- Pour créer un compte de stockage, vous devez sélectionner un groupe de ressources existant dont vous êtes propriétaire.
- Pour sélectionner un compte de stockage existant, vous devez être propriétaire du compte de stockage. Vous devez également sélectionner le groupe de ressources existant auquel appartient le compte de stockage.

Les contributeurs d'abonnement ne peuvent pas attribuer de rôles à d'autres utilisateurs.

Lecteur de l'abonnement

Les lecteurs d'abonnement ne peuvent pas créer d'espaces de travail. Vous pouvez afficher toutes les ressources créées sous l'abonnement, mais ne peut pas apporter de modifications ni attribuer de rôles.

Vérifier vos attributions de rôle

Vérifier vos abonnements

Pour afficher la liste de vos abonnements et rôles associés :

1. Connectez-vous au [portail Azure](#).
2. Sous l'en-tête des services Azure, sélectionnez **Abonnements**. Si vous ne voyez pas **Abonnements** ici, utilisez la zone de recherche pour la trouver.
3. Le filtre Abonnements en regard de la zone de recherche peut être défini par défaut sur Abonnements == **filtre** global. Pour afficher la liste de tous vos abonnements, sélectionnez le filtre Abonnements et **désélectionnez** « Sélectionner uniquement les abonnements sélectionnés dans le... » boîte. Ensuite, sélectionnez

Appliquer. Le filtre doit ensuite afficher les abonnements == all.

The screenshot shows the Azure portal's 'Subscriptions' page. At the top, there's a breadcrumb 'Home >' and the title 'Subscriptions' with a pin icon and a menu icon. Below the title, it says 'Default Directory ([redacted])'. A navigation bar contains links: '+ Add', 'Manage Policies', 'View Requests', and 'View eligible subscriptions'. A search bar is labeled 'Search for any field...'. Below this, there are filter buttons: 'Subscriptions == all', 'My role == all', 'Status == all', and an 'Add filter' button. A table is partially visible with columns 'Subscription name' and 'Status'. The first two rows are 'Azure subscription 1' with status '5f' and '4c'. A modal window titled 'subscription' is open in the foreground. It has a search bar and a list of checkboxes: 'All', 'Azure subscription 1', and 'Azure subscription 1'. At the bottom of the modal, there is a checkbox labeled 'Show only subscriptions selected in the' followed by a link 'global subscriptions filter'. Below this is a blue 'Apply' button. Red rectangles highlight the 'Show only subscriptions selected in the' checkbox and the 'Apply' button.

Vérifier vos ressources

Pour vérifier l'attribution de rôle que vous ou un autre utilisateur dispose d'une ressource particulière, consultez [Vérifier l'accès d'un utilisateur aux ressources](#) Azure.

Affecter des rôles

Pour ajouter de nouveaux utilisateurs à un espace de travail, vous devez être propriétaire de l'espace de travail. Pour accorder l'accès à 10 utilisateurs ou moins à votre espace de travail, consultez [Partager l'accès à votre espace de travail](#) Azure Quantum. Pour accorder l'accès à plus de 10 utilisateurs, consultez [Ajouter un groupe à votre espace de travail](#) Azure Quantum.

Pour attribuer des rôles pour n'importe quelle ressource à n'importe quelle étendue, y compris au niveau de l'abonnement, consultez [Affecter des rôles Azure à l'aide du Portail Azure](#).

Dépannage

Pour obtenir des solutions aux problèmes courants, consultez [Résoudre les problèmes liés à Azure Quantum : Création d'un espace de travail](#) Azure Quantum.

- Lorsque vous créez une ressource dans Azure, par exemple un espace de travail, vous n'êtes pas directement le propriétaire de la ressource. Votre rôle est hérité du rôle d'étendue le plus élevé auquel vous êtes autorisé dans cet abonnement.
- Lorsque vous créez des attributions de rôles, elles peuvent parfois prendre jusqu'à une heure pour prendre effet sur les autorisations mises en cache sur la pile.

Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

Ajouter des utilisateurs en bloc à votre espace de travail Azure Quantum

Découvrez comment accorder à un groupe d'utilisateurs l'accès à votre espace de travail Azure Quantum. Par exemple, vous devrez peut-être accorder aux membres de votre équipe ou à vos étudiants l'accès à votre espace de travail.

Nous vous recommandons d'utiliser les instructions de cet article si vous devez accorder l'accès à plus de 10 utilisateurs. Pour un plus petit nombre d'utilisateurs, consultez [Partager l'accès à votre espace de travail Azure Quantum](#).

Dans cet article, vous allez :

1. [Créez un groupe dans le portail Microsoft Entra ID](#).
2. [ajouter le groupe en tant que contributeur à votre espace de travail Quantum](#) ;
3. [Invitez en bloc les utilisateurs à l'ID Microsoft Entra](#).
4. [Importez en bloc des membres dans votre groupe](#).

Conditions préalables

Pour ajouter des utilisateurs en bloc à un espace de travail Azure Quantum, vous devez remplir les conditions préalables suivantes :

- Compte Azure avec un abonnement actif. Si vous ne disposez pas d'un compte Azure, inscrivez-vous gratuitement et souscrivez un [abonnement de paiement à l'utilisation](#) [↗].
- Un espace de travail Azure Quantum. Consultez [Créer un espace de travail Azure Quantum](#).
- Lien pour votre espace de travail Azure Quantum. Pour obtenir le lien de votre espace de travail Quantum, accédez à votre espace de travail Azure Quantum dans le portail Azure et copiez l'URL à partir de la barre d'adresse de votre navigateur.

Créer un groupe dans le portail Microsoft Entra ID

1. Connectez-vous au [portail Azure](#) [↗]. Vous devez être **propriétaire** de l'espace de travail ou disposer de privilèges d'attribution de rôles pour pouvoir ajouter le groupe dans la section suivante.
2. Recherchez et sélectionnez **Microsoft Entra ID**.
3. Dans la page **MICROSOFT Entra ID** , sélectionnez **Groupe**s dans le menu de gauche, puis sélectionnez **Nouveau groupe**.

4. Renseignez les informations obligatoires sur la page **Nouveau groupe**.

- Sélectionnez **Microsoft 365** comme **Type de groupe**.
- Créez et ajoutez un **nom de groupe**.
- Ajoutez une **adresse e-mail de groupe** pour le groupe ou conservez l'adresse e-mail renseignée automatiquement.
- **Description du groupe**. Ajoutez une description facultative à votre groupe.
- Sélectionnez **Affectée** comme **Type d'appartenance**.

Home > Groups | All groups >

New Group ...

Got feedback?

Group type
Microsoft 365

Group name * ⓘ
Quantum students - Fabrikam ✓

Group email address * ⓘ
<group-email-name> ✓ @microsoft.onmicrosoft.com

Group description ⓘ
Quantum student users at Fabrikam ✓

Membership type * ⓘ
Assigned

Sensitivity label ⓘ

Owners
No owners selected

Members
No members selected

Create

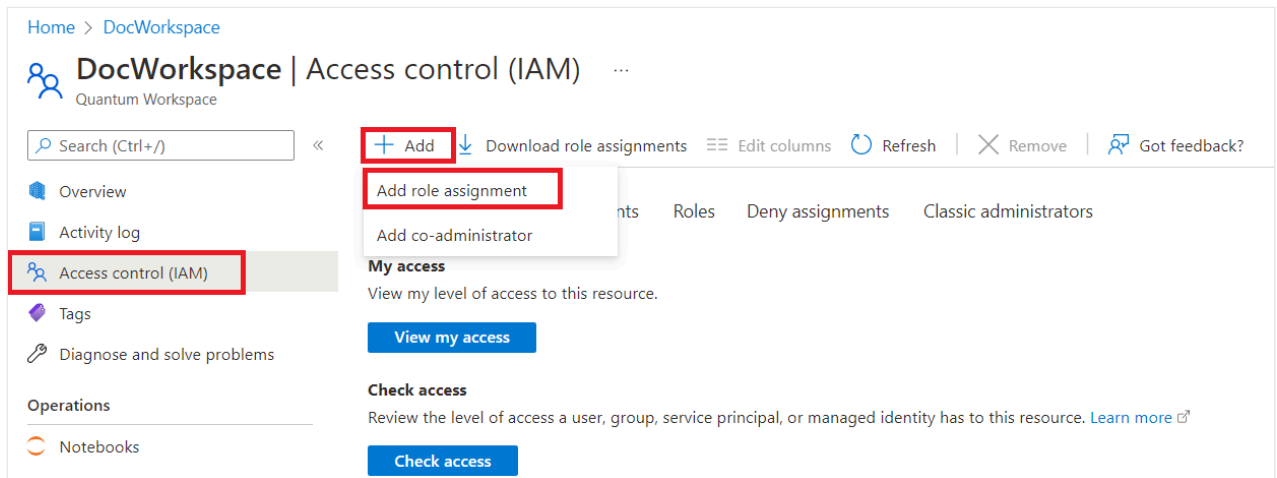
5. Cliquez sur **Créer**. Vous devriez recevoir une notification indiquant que vous avez correctement créé votre groupe.

Ajouter le groupe en tant que contributeur à votre espace de travail Quantum

ⓘ Remarque

Vous pouvez ajouter votre groupe au rôle de **Contributeur** ou au rôle de **Contributeur de données d'espace de travail Quantum**. Le rôle Contributeur permet aux utilisateurs de gérer les propriétés de l'espace de travail, tandis que le rôle Contributeur aux données de l'espace de travail Quantum permet uniquement aux utilisateurs d'envoyer et d'afficher des travaux dans l'espace de travail. Pour plus d'informations, consultez [Gérer l'accès à votre espace de travail Azure Quantum](#).

1. Dans le portail Azure, accédez à votre espace de travail Azure Quantum.
2. Autorisez votre groupe à accéder à votre espace de travail. Sélectionnez **Contrôle d'accès (IAM)** dans le menu gauche. Cliquez sur **Ajouter**, puis sélectionnez **Ajouter une affectation de rôle**.



3. La page **Ajouter une affectation de rôle** s'ouvre alors. Dans le volet **Rôle**, sélectionnez **Contributeur** ou **Quantum Workspace Data Contributor**, puis cliquez sur **Suivant**.

Home > DocWorkspace | Access control (IAM) >

Add role assignment

Got feedback?

Role **Members** Review + assign

A role definition is a collection of permissions. You can use the built-in roles or you can create your own custom roles. [Learn more](#)

Search by role name, description, or ID Type: All Category: All

Name ↑↓	Description ↑↓
Owner	Grants full access to manage all resources, including the ability to assign roles in Azure RBAC.
Contributor	Grants full access to manage all resources, but does not allow you to assign roles in Azure RBAC, manag
Reader	View all resources, but does not allow you to make any changes.
Azure Service Deploy Release Management Contributor	Contributor role for services deploying through Azure Service Deploy.
Azure Service Deploy Release Management Restricted O...	Restricted owner role for services deploying through Azure Service Deploy.
.....	
Monitoring Reader	Can read all monitoring data.
Quantum Workspace Data Contributor	Create, read, and modify jobs and other Workspace data. This role is in preview and subject to change.
Resource Policy Contributor	Users with rights to create/modify resource policy, create support ticket and read resources/hierarchy.

Review + assign Previous **Next**

4. Dans le volet **Membres**, sélectionnez Affecter l'accès à **Utilisateur, groupe ou principal de service**. Ensuite, cliquez sur **+ Sélectionner des membres**. Le volet **Sélectionner des membres** s'ouvre alors. Recherchez le nom de votre groupe et sélectionnez votre groupe. Ensuite, cliquez sur **Sélectionner**.

Home > DocWorkspace | Access control (IAM) >

Add role assignment

Got feedback?

Role **Members** Review + assign

Selected role
Contributor

Assign access to

☒ User, group, or service principal

☐ Managed identity

Members

+ Select members

Name	Object ID	Type
No members selected		

Description

Optional

Review + assign Previous Next

Select members

Select ①

Quantumstudents

No users, groups, or service principals found.

Selected members:

QS Quantum students-San Fransokyo Uni
Quantumstudents-SanFransokyoUni... [Remove](#)

Select Close

5. Le nom de votre groupe s'affiche sous **Membres**. Cliquez sur **Examiner + affecter**. Dans le volet **Examiner + affecter**, recliquez sur **Examiner + affecter**. Vous devriez recevoir une notification indiquant que votre groupe a été ajouté en tant que Contributeur à votre espace de travail.

Home > DocWorkspace | Access control (IAM) >

Add role assignment ...

Got feedback?

Role **Members** Review + assign

Selected role Contributor

Assign access to ☒ User, group, or service principal ☐ Managed identity

Members [+ Select members](#)

Name	Object ID	Type
Quantum students-San Fransokyo Uni	eeeeeeee-4444-5555-6666-fffff...	Group

Description

Review + assign Previous Next

Inviter des utilisateurs en bloc à l'ID Microsoft Entra

1. Accédez à [Utilisateurs - Microsoft Azure](#). Dans le menu gauche, accédez à **Tous les utilisateurs**.
2. Sélectionnez **Opérations en bloc**, puis **Invitation en bloc**.
3. Dans le volet **Inviter des utilisateurs en bloc**, sélectionnez **Télécharger** pour obtenir un modèle CSV valide avec les propriétés de l'invitation.

Bulk invite users ✕

1. Download csv template (optional)

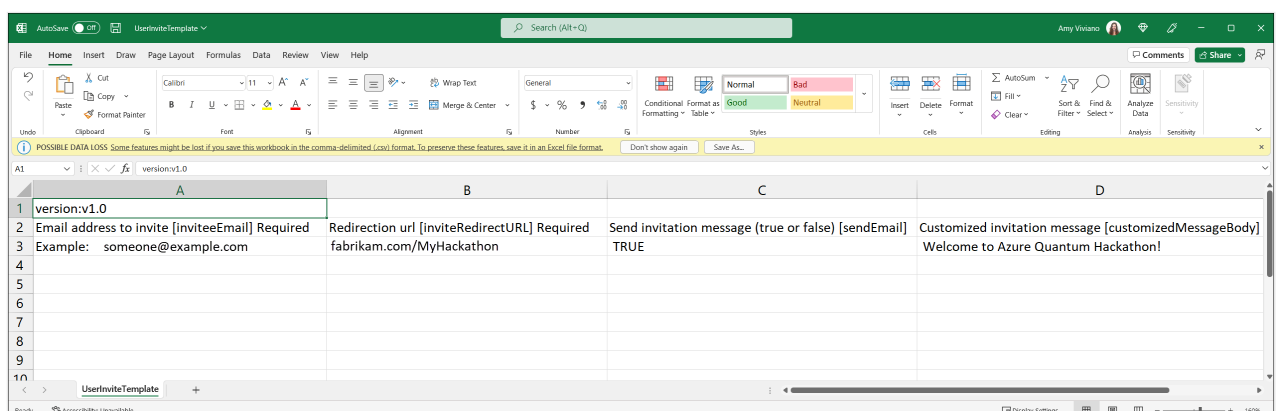
Download
2. Edit your csv file
3. Upload your csv file

Select a file 

[Learn more about bulk invite guest users](#)

4. Ouvrez le modèle CSV et ajoutez une ligne pour chaque utilisateur. Les deux premières lignes du modèle chargé ne doivent pas être supprimées ni modifiées, sinon, le chargement ne pourra pas être traité. La troisième ligne fournit des exemples de valeurs pour chaque colonne. Vous devez supprimer la ligne des exemples et la remplacer par vos propres entrées. Les valeurs obligatoires sont les suivantes :

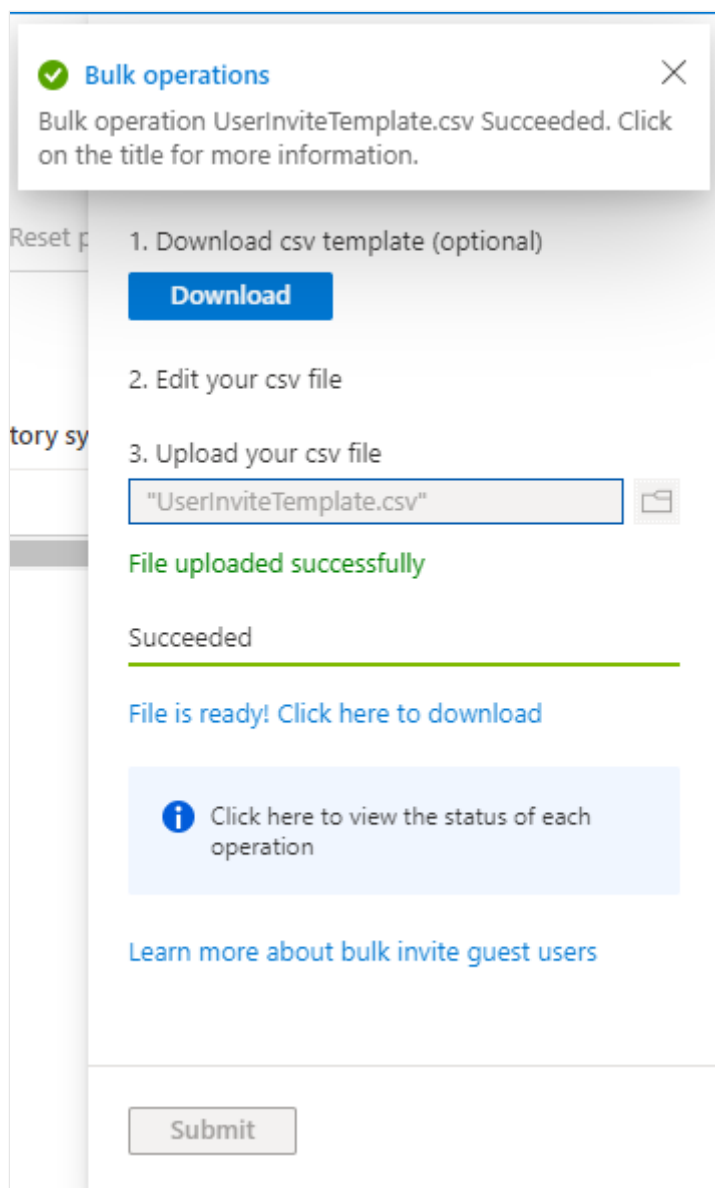
- **Adresse e-mail à inviter** : utilisateur qui recevra une invitation
- **URL de redirection** : lien vers votre espace de travail Azure Quantum. L'utilisateur invité est transféré à ce lien lorsqu'il accepte l'invitation



	A	B	C	D
1	version:v1.0			
2	Email address to invite [inviteeEmail] Required	Redirection url [inviteRedirectURL] Required	Send invitation message (true or false) [sendEmail]	Customized invitation message [customizedMessageBody]
3	Example: someone@example.com	fabrikam.com/MyHackathon	TRUE	Welcome to Azure Quantum Hackathon!
4				
5				
6				
7				
8				
9				
10				

5. Enregistrez le fichier.

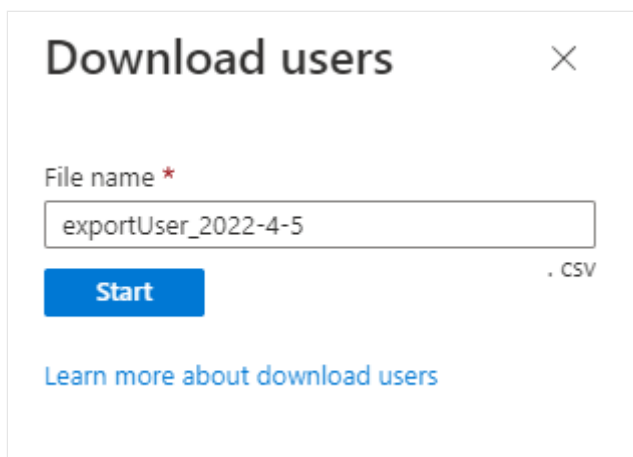
6. Dans le volet **Inviter des utilisateurs en bloc**, sous **Chargez votre fichier .csv**, accédez au fichier. Quand vous sélectionnez le fichier, la validation du fichier CSV démarre. Une fois le message **Fichier chargé avec succès** affiché, sélectionnez **Envoyer** pour lancer l'opération d'invitation en bloc.



7. Une fois l'opération d'invitation des utilisateurs en bloc terminée, vos utilisateurs reçoivent un e-mail d'invitation. Ils doivent accepter l'invitation.
8. Dans la page **Révision des autorisations**, les utilisateurs doivent sélectionner **Accepter** pour pouvoir continuer.
9. Une fois les autorisations acceptées, vos utilisateurs seront ajoutés à l'ID Microsoft Entra.

Importer en bloc des membres dans votre groupe

1. Une fois l'invitation en bloc terminée, téléchargez tous les utilisateurs microsoft Entra ID dans un fichier CSV. Accédez à **Tous les utilisateurs**, puis cliquez sur **Télécharger des utilisateurs**.
2. Dans le volet **Télécharger des utilisateurs**, cliquez sur **Démarrer**.



Download users [X]

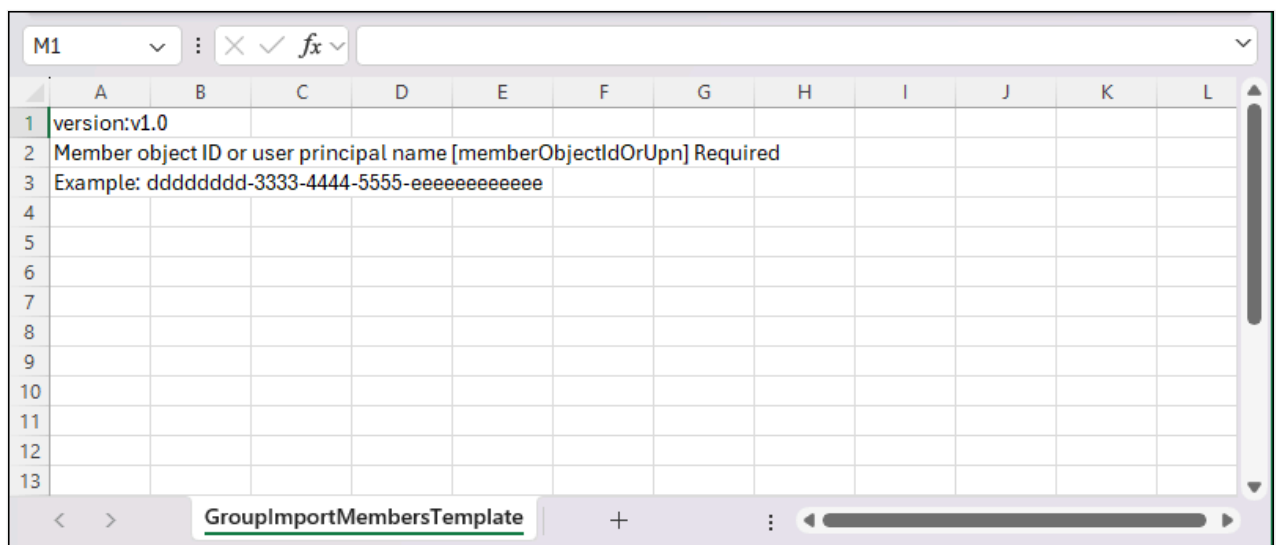
File name *

exportUser_2022-4-5

[Start] . CSV

[Learn more about download users](#)

- Une fois l'exportation des utilisateurs en bloc terminée, cliquez sur **Télécharger les résultats** pour télécharger le fichier CSV avec tous vos utilisateurs microsoft Entra ID.
- Une fois le téléchargement terminé, importez les membres de votre groupe. Accédez à [Groupes – Microsoft Azure](#). Sélectionnez votre groupe puis, dans le menu de gauche, accédez à **Membres**. Dans ce panneau, sélectionnez **Opérations en bloc**, puis **Importez des membres**.
- Dans le volet **Importer en bloc un groupe**, sélectionnez **Télécharger** pour obtenir un modèle CSV valide.
- Ouvrez le modèle CSV et ajoutez une ligne pour chaque utilisateur à inviter dans le groupe. Copiez-collez les noms d'utilisateurs principaux de vos utilisateurs à partir du fichier CSV que vous avez téléchargé à l'étape 1. La troisième ligne fournit un exemple de valeur. Vous devez supprimer la ligne d'exemple et la remplacer par votre propre entrée.



	A	B	C	D	E	F	G	H	I	J	K	L
1	version:v1.0											
2	Member object ID or user principal name [memberObjectIdOrUpn] Required											
3	Example: dddddd-3333-4444-5555-eeeeeeeeeeee											
4												
5												
6												
7												
8												
9												
10												
11												
12												
13												

GroupImportMembersTemplate

- Enregistrez le fichier.
- Dans le volet **Importer en bloc un groupe**, sous **Chargez votre fichier .csv**, accédez au fichier et chargez-le.

9. Une fois le message **Fichier chargé avec succès** affiché, sélectionnez **Envoyer** pour lancer l'opération d'importation en bloc.
 10. Une fois l'opération d'importation en bloc d'un groupe terminée, vos membres de groupe sont ajoutés correctement au groupe.
-

Last updated on 14/03/2026

Partager l'accès à votre espace de travail Azure Quantum

Découvrez comment partager l'accès à votre [espace de travail](#) Azure Quantum. Par exemple, vous devrez peut-être accorder aux membres de votre équipe ou aux étudiants l'accès à votre espace de travail. Vous pouvez attribuer des rôles aux utilisateurs pour contrôler leur accès à votre espace de travail. Par exemple, un rôle **Contributeur** peut créer, supprimer ou modifier un espace de travail, tandis qu'un **Contributeur de données Quantum Workspace** rôle dispose d'autorisations limitées et peut principalement envoyer et afficher des travaux.

Nous vous recommandons d'utiliser les instructions de cet article si vous devez accorder l'accès à 10 utilisateurs ou moins. Pour un plus grand nombre d'utilisateurs, il peut être plus facile pour vous ou votre service informatique de créer un groupe d'utilisateurs, puis de lui accorder l'accès à votre espace de travail. Pour obtenir des instructions, consultez [Ajouter un groupe à votre espace de travail](#) Azure Quantum.

Prérequis

Vous avez besoin des conditions préalables suivantes pour partager l'accès à votre espace de travail Azure Quantum :

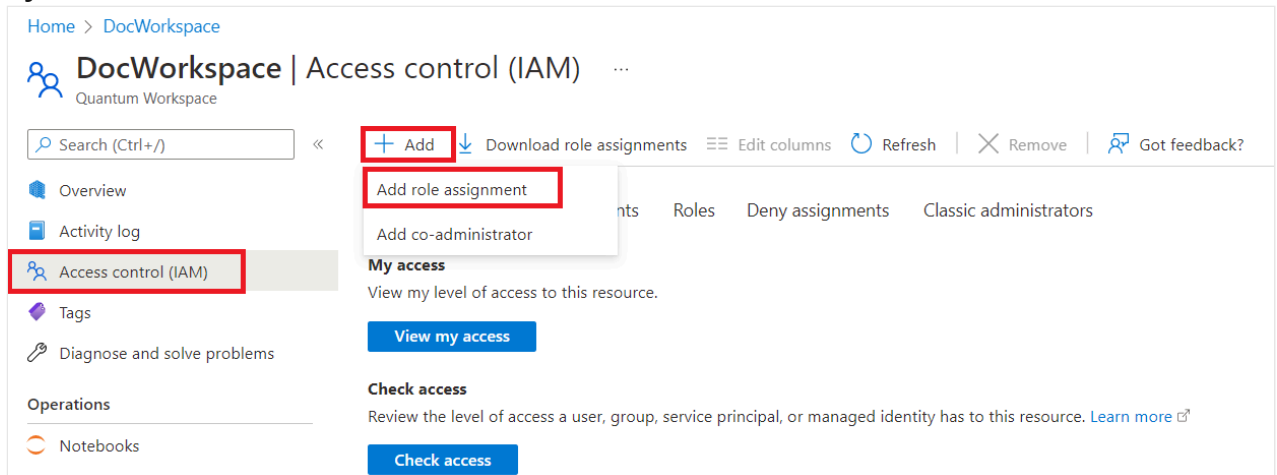
- Un compte Azure avec un abonnement actif. Si vous n'avez pas de compte Azure, inscrivez-vous gratuitement et inscrivez-vous à un [abonnement](#) de paiement à l'utilisation.
- Un espace de travail Azure Quantum. Consultez [Créer un espace de travail Azure Quantum](#).

Microsoft Entra ID (système d'identification de Microsoft)

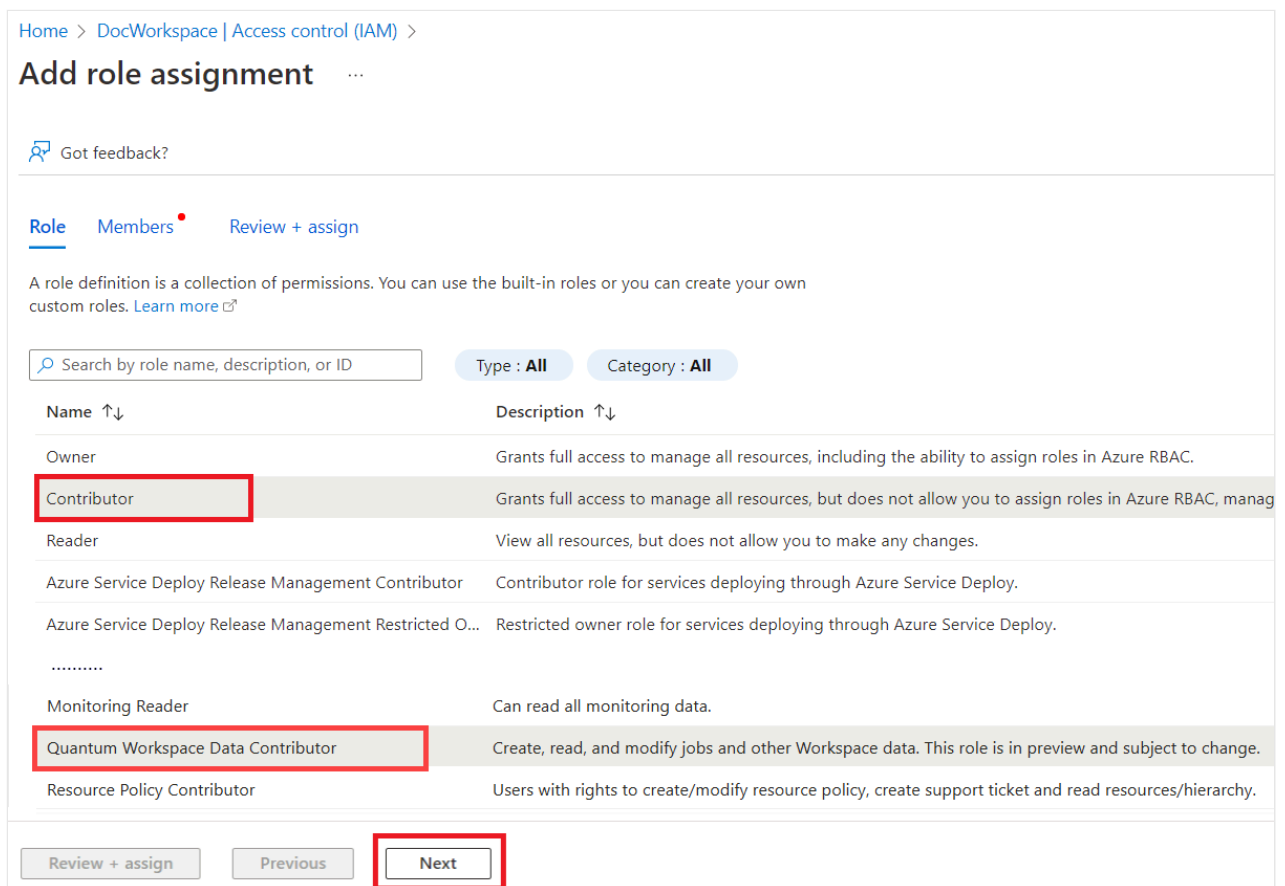
Chaque utilisateur doit disposer d'un compte dans l'ID Microsoft Entra de votre organisation avant de pouvoir lui accorder l'accès à votre espace de travail. Pour ajouter de nouveaux utilisateurs, vous devez être au moins administrateur **d'utilisateurs**. Pour obtenir des instructions, consultez [Ajouter un nouvel utilisateur](#).

Ajouter des utilisateurs à votre espace de travail Azure Quantum

1. Connectez-vous au [Portail Azure](#) et accédez à votre espace de travail Azure Quantum. Vous devez être propriétaire de l'espace de travail pour ajouter des utilisateurs.
2. Sélectionnez **Contrôle d'accès (IAM)** dans le menu de gauche. Sélectionnez **Ajouter**, puis **Ajouter une attribution de rôle**.



3. La page **Ajouter une attribution de rôle** s'ouvre. Dans le volet **Rôle**, sélectionnez **Contributeur** ou **Quantum Workspace Data Contributor**, puis cliquez sur **Suivant**.



4. Dans le volet **Membres**, sélectionnez **Attribuer l'accès à l'utilisateur, au groupe ou au principal** du service. Sélectionnez ensuite **+Sélectionner des membres**. Le panneau **Sélectionner des membres** s'ouvre. Recherchez et sélectionnez chacun de vos utilisateurs. Sélectionnez ensuite le bouton **Sélectionner bleu**.

Home > DocWorkspace | Access control (IAM) >

Add role assignment

Got feedback?

Role
Members
Review + assign

Selected role Contributor

Assign access to
☒ User, group, or service principal
☐ Managed identity

Members

+ Select members

Name	Object ID	Type
No members selected		

Description

Optional

Review + assign
Previous
Next

Select
Search by name or email address

Searching...

Selected members:

user-1@contoso.com
Remove

user-2@contoso.com
Remove

user-3@contoso.com
Remove

Select
Close

5. Une liste de vos utilisateurs s’affiche sous **Membres**. Sélectionnez **Vérifier + attribuer**. Dans le **volet Révision + affectation**, sélectionnez **Vérifier + affecter** à nouveau. Vous devez recevoir une notification indiquant que vos utilisateurs ont été ajoutés à votre espace de travail.

Home > DocWorkspace | Access control (IAM) >

Add role assignment

Got feedback?

Role
Members
Review + assign

Selected role Contributor

Assign access to
☒ User, group, or service principal
☐ Managed identity

Members

+ Select members

Name	Object ID	Type
		User
		User
		User

Description

Optional

Review + assign

Previous

Next

Partager l'accès en utilisant une chaîne de connexion

Vous pouvez partager l'accès à votre espace de travail Azure Quantum à l'aide d'un [chaîne de connexion](#). Le chaîne de connexion contient les informations nécessaires pour se connecter à votre espace de travail, notamment l'ID d'abonnement, le groupe de ressources, le nom de l'espace de travail et l'emplacement.

Chaque espace de travail Azure Quantum possède **des clés primaires et secondaires**, ainsi que leurs chaîne de connexion correspondantes. Si vous souhaitez autoriser l'accès à votre espace de travail à d'autres personnes, vous pouvez partager votre clé secondaire et utiliser votre principal pour vos propres services. De cette manière, vous pouvez remplacer la clé secondaire si nécessaire sans occasionner d'interruption dans vos propres services.

Last updated on 14/03/2026

Migrer vos Azure Quantum données de travail

Si vous disposez d'un espace de travail existant Azure Quantum et que vous créez un espace de travail, vous pouvez migrer vos données de travail de l'ancien espace de travail vers le nouvel espace de travail. Lorsque vous effectuez la migration, vos données d'entrée et de sortie de travail sont disponibles dans votre nouvel espace de travail. Toutefois, vous ne pouvez plus voir l'historique des travaux pour ces travaux dans le Azure portail, le Azure Quantum SDK (Kit de développement logiciel) ou la ligne de commande Azure CLI.

Les instructions de migration varient selon que le compte de stockage de votre ancien espace de travail est géré ou non managé.

Important

Ne supprimez pas votre ancien espace de travail tant que vous n'avez pas migré vos données vers votre nouvel espace de travail. Avant de migrer vos données de travail, assurez-vous que vous n'avez pas de travaux ouverts dans la file d'attente. Pour supprimer un travail ouvert de la file d'attente, attendez que le travail se termine ou annulez le travail.

Stockage non managé

Si votre ancien espace de travail utilise un compte de stockage non managé, liez ce compte de stockage à votre nouvel espace de travail. Le compte de stockage lié contient vos anciennes données de travail ainsi que les données des travaux que vous envoyez dans votre nouvel espace de travail.

Notes

Pour lier un compte de stockage à un espace de travail Quantum, vous devez disposer d'une attribution de rôle qui vous permet d'effectuer des attributions de rôles sur ce compte de stockage, telles que Propriétaire ou Administrateur de l'accès utilisateur.

Pour créer un espace de travail Quantum et lier votre compte de stockage, procédez comme suit :

1. Connectez-vous au Azure portail, accédez à **Quantum Workspaces**, puis choisissez **Créer**.

2. Choisissez votre abonnement dans la liste déroulante **Abonnement** , puis sélectionnez **Création avancée**.
3. Dans la section **Détails du projet** , ouvrez la liste déroulante **Groupe de ressources** et choisissez le groupe de ressources qui contient le compte de stockage de votre ancien espace de travail.
4. Dans la section **Détails de l'instance** , entrez un nom pour votre nouvel espace de travail dans le champ **Nom** de l'espace de travail, puis choisissez l'une des régions prises en charge dans la liste déroulante **Région** (USA Est, USA Ouest, Europe Nord ou Europe Ouest).
5. Pour **le stockage de données**, choisissez **Personnaliser les paramètres du compte de stockage (avancé)**. Une liste déroulante **Compte de stockage** s'affiche qui contient tous les comptes de stockage dans votre groupe de ressources sélectionné. Choisissez l'ID du compte de stockage de votre ancien espace de travail.
6. Pour passer en revue les paramètres de votre espace de travail, choisissez **Vérifier + créer**. Ou choisissez **Suivant : Fournisseurs** > pour personnaliser vos options de fournisseur et de plan, puis choisissez **Suivant : Balises pour ajouter des balises** > à votre espace de travail.
7. Vérifiez que tous les paramètres de votre espace de travail sont corrects, puis choisissez **Créer**.

Le compte de stockage de votre ancien espace de travail est désormais associé à votre nouvel espace de travail et l'historique des travaux est conservé dans ce compte de stockage.

ⓘ Notes

Pour des questions sur les problèmes de migration, e-mail azurequantuminfo@microsoft.com.

Utiliser des travaux Quantum Azure

Article • 16/09/2024

Lorsque vous exécutez un programme quantique dans Azure Quantum, vous créez et exécutez un **travail**. Les étapes de création et d'exécution d'un travail dépendent du type de travail et du fournisseur et target que vous configurez pour l'espace de travail.

Propriétés du travail

Toutes les tâches ont en commun les propriétés suivantes :

 Agrandir le tableau

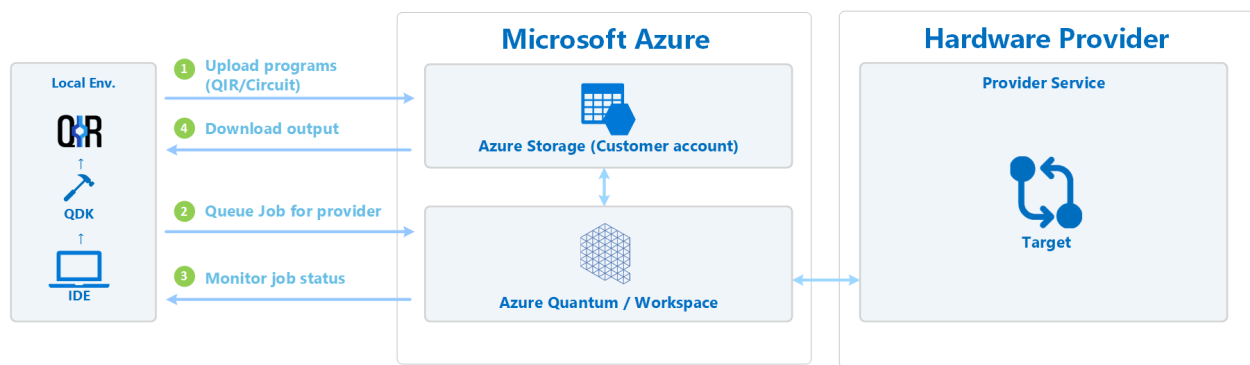
Propriété	Description
Identifiant	Identificateur unique du travail. Il doit être unique dans l'espace de travail.
Fournisseur	<i>Qui</i> devra exécuter votre tâche.
Cible	<i>Sur quoi</i> souhaitez-vous exécuter votre tâche. Par exemple, le matériel quantique lui-même ou le simulateur Quantum proposé par le fournisseur.
Nom	Nom défini par l'utilisateur pour vous aider à organiser vos tâches.
Paramètres	Paramètres d'entrée facultatifs pour targets. Consultez la documentation relative à la sélection target d'une définition de paramètres disponibles.

Une fois que vous avez créé un travail, diverses métadonnées sont disponibles sur son état et son historique des exécutions.

Cycle de vie de tâche

Une fois que vous avez écrit votre programme quantique, vous pouvez sélectionner une target tâche et l'envoyer.

Ce diagramme illustre le workflow de base après l'envoi de votre travail :



Tout d’abord, Azure Quantum charge le travail dans le compte de stockage Azure que vous avez configuré dans l’espace de travail. Ensuite, le travail est ajouté à la file d’attente des travaux pour le fournisseur que vous avez spécifié dans le travail. Azure Quantum télécharge ensuite votre programme et le traduit pour le fournisseur. Le fournisseur traite le travail et retourne la sortie au stockage Azure, où il devient disponible au téléchargement.

Surveillance des travaux

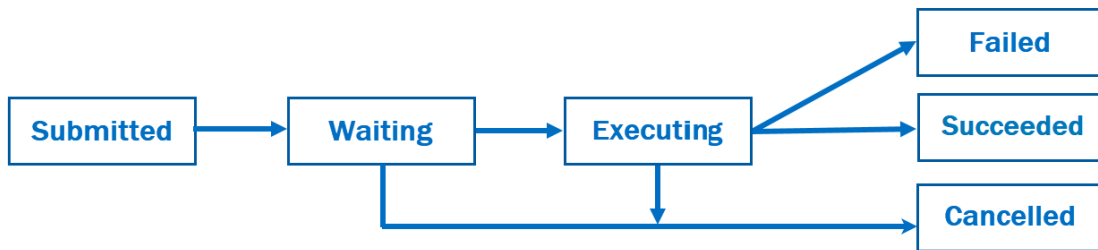
Une fois que vous avez envoyé un travail, vous pouvez surveiller l’état du travail. Les états possibles des travaux sont les suivants :

[🔍](#) Agrandir le tableau

Statut	Description
<i>en attente</i>	Le travail est en attente d’exécution. Certains travaux effectuent des tâches de prétraitement lorsqu’ils sont en attente. <code>waiting</code> est toujours le premier état. Toutefois, un travail peut passer à l’état <code>executing</code> avant que vous ne puissiez l’observer dans l’état <code>waiting</code> .
<i>en cours d’exécution</i>	Le target travail est en cours d’exécution.
<i>succeeded</i>	Le travail a abouti et la sortie est disponible. Il s’agit d’un état <i>final</i> .
<i>échec</i>	Le travail a échoué et des informations d’erreur sont disponibles. Il s’agit d’un état <i>final</i> .
<i>annulé</i>	L’utilisateur a demandé l’annulation de l’exécution du travail. Il s’agit d’un état <i>final</i> . Pour plus d’informations, consultez Annulation des travaux dans cet article.

Les états `succeeded`, `failed` et `cancelled` sont considérés comme des **états finaux**. Une fois qu’un travail est dans l’un de ces états, aucune autre mise à jour ne se produit et les données de sortie du travail correspondant ne changent pas.

Ce diagramme illustre les transitions d’état de travail possibles :



Une fois qu'un travail est correctement terminé, il présente un lien vers les données de sortie dans votre compte de stockage Azure. La façon dont vous accédez à ces données dépend du SDK ou de l'outil que vous avez utilisé pour [soumettre le travail](#).

Guide pratique pour surveiller les travaux

Vous pouvez surveiller des travaux via Python, les Portail Azure et Azure CLI.

Python

Toutes les propriétés du travail sont accessibles dans `job.details`. Par exemple, vous pouvez accéder au nom du travail, à l'état et à l'ID comme suit :

Python

```
print(job.details)
print("\nJob name:", job.details.name)
print("Job status:", job.details.status)
print("Job ID:", job.details.id)
```

Output

```
{'additional_properties': {'isCancelling': False}, 'id': '0fc396d2-97dd-11ee-9958-6ca1004ff31f', 'name': 'MyPythonJob', 'provider_id': 'rigetti'...}
Job name: MyPythonJob
Job status: Succeeded
Job ID: fc396d2-97dd-11ee-9958-6ca1004ff31f
```

Nombres de travaux

Pour obtenir des nombres résultant d'un grand nombre de travaux, vous pouvez effectuer une [installation locale des Quantum Development Kit outils](#). Avec une installation locale, vous pouvez stocker les ID de travail localement.

Vous pouvez copier le code suivant pour obtenir la liste des travaux et leurs résultats :

Python

```
for job in workspace.list_jobs():  
    print(job.id, job.details.name, job.details.output_data_uri)
```

Annulation des travaux

Quand un travail n'est pas encore dans un état final (par exemple, `succeeded`, `failed` ou `cancelled`), vous pouvez demander son annulation. Tous les fournisseurs annulent votre travail si son état est `waiting`. En revanche, tous les fournisseurs ne prennent pas en charge l'annulation si votre travail est dans l'état `executing`.

ⓘ Notes

Si vous annulez un travail alors que son exécution a commencé, votre compte risque d'être quand même facturé partiellement ou intégralement pour ce travail. Consultez la documentation sur la facturation du fournisseur que vous avez sélectionné.

Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

Envoyer des travaux à Azure Quantum avec Azure CLI

Utilisez Azure CLI lorsque vous disposez déjà d'un fichier d'entrée compilé QIR et que vous souhaitez envoyer le travail à partir d'un terminal, d'un script shell ou d'un flux de travail CI/CD. Azure CLI gère l'espace de travail, target, les métadonnées de la tâche et le fichier d'entrée, mais ne compile pas les programmes en QIR pour vous.

La méthode exacte pour envoyer un travail avec Azure CLI dépend du Azure Quantumtarget. Les étapes suivantes constituent un exemple général de la façon de soumettre un travail QIR avec Azure CLI.

Installer la prise en charge d'Azure CLI

1. Installez [Azure CLI](#).
2. Installez ou mettez à jour l'extension Azure Quantum .

Azure CLI

```
az extension add --upgrade --name quantum
```

Se connecter à un espace de travail

1. Connectez-vous à Azure.

Azure CLI

```
az login
```

2. Définissez l'abonnement qui contient votre Azure Quantum espace de travail.

Azure CLI

```
az account set --subscription <subscription-id>
```

3. Définissez l'espace de travail par défaut pour les commandes suivantes.

Azure CLI

```
az quantum workspace set \
  --resource-group <resource-group-name> \
  --workspace-name <workspace-name>
```

4. Listez les éléments targets disponibles dans l'espace de travail.

Azure CLI

```
az quantum target list --output table
```

Envoyer un fichier d'entrée compilé

Pour un QIR job, spécifiez le QIR, le nom du job, le target format d'entrée, le fichier d'entrée et le QIR point d'entrée.

Azure CLI

```
az quantum job submit \
  --target-id <target-id> \
  --job-name <job-name> \
  --job-input-format qir.v1 \
  --job-input-file <path-to-qir-file> \
  --entry-point ENTRYPOINT__main
```

La commande retourne l'ID de travail. Enregistrez l'identifiant afin de pouvoir suivre la tâche et récupérer son résultat.

⚠ Remarque

Cet exemple montre les paramètres courants QIR . Un travail natif du fournisseur peut nécessiter différents formats d'entrée et de sortie ou des paramètres supplémentaires. Consultez la documentation du target sélectionné ainsi que la [référence de commande az quantum job](#).

Suivre la tâche et obtenir le résultat

Pour vérifier l'état du travail, exécutez la commande suivante.

Azure CLI

```
az quantum job show --job-id <job-id> --output table
```

Pour obtenir la sortie d'un travail réussi, exécutez la commande suivante :

Azure CLI

```
az quantum job output --job-id <job-id> --output table
```

Pour connaître la configuration de l'espace de travail et d'autres Azure Quantum commandes, consultez [Gérer les espaces de travail quantiques avec Azure CLI](#).

Contenu connexe

- [Comment envoyer des travaux à Azure Quantum](#)
- [Envoyer des travaux avec l'extension QDK pour VS Code](#)
- [Envoyer des travaux avec le QDK package Python](#)
- [Travailler avec des Azure Quantum jobs](#)

Last updated on 29/08/2026

Gérer des espaces de travail quantiques avec Azure CLI

Dans ce guide, vous allez apprendre à utiliser Azure Command-Line Interface (Azure CLI) pour créer des espaces de travail Azure Quantum et les groupes de ressources et comptes de stockage requis, et commencer à exécuter vos applications quantiques dans Azure Quantum.

Conditions préalables

Pour utiliser le service Azure Quantum, vous avez besoin des éléments suivants :

- Un compte Azure avec un abonnement actif. Si vous n'avez pas de compte Azure, inscrivez-vous gratuitement et inscrivez-vous à un [abonnement](#) de paiement à l'utilisation.
- Groupe de ressources Azure où réside l'espace de travail quantique.
- Un compte de stockage dans le groupe de ressources à associer à l'espace de travail quantique. Plusieurs espaces de travail peuvent être associés au même compte.
- [Interface de ligne de commande Azure](#).
- [Microsoft Quantum Development Kit](#).

Configuration de l'environnement

1. Installez l'extension Azure CLI `quantum` . Ouvrez une invite de commandes et exécutez la commande suivante, qui met également à niveau l'extension si une version précédente est déjà installée :

Azure CLI

```
az extension add --upgrade -n quantum
```

2. Connectez-vous à Azure à l'aide de vos informations d'identification. Vous voyez la liste des abonnements associés à votre compte.

Azure CLI

```
az login
```

3. Spécifiez l'abonnement que vous souhaitez utiliser.

Azure CLI

```
az account set -s <Your subscription ID>
```

4. Si c'est la première fois que vous créez des espaces de travail quantiques dans votre abonnement, inscrivez le fournisseur de ressources avec cette commande :

Azure CLI

```
az provider register --namespace Microsoft.Quantum
```

Créer un espace de travail Azure Quantum

Pour créer un espace de travail Azure Quantum, vous devez savoir :

- Nom de la région Azure ou de l'emplacement pour créer la ressource. Vous pouvez utiliser la [liste des régions et leurs codes resource manager](#) pris en charge par l'outil Azure CLI, par exemple, **westus**.
- Groupe de ressources associé au nouvel espace de travail, par exemple , **MyResourceGroup**.
- Un compte de stockage dans le même groupe de ressources et le même abonnement que l'espace de travail quantique. Il est possible de [créer un compte de stockage à partir de l'outil Az CLI](#), par exemple **MyStorageAccount**.
- Nom de l'espace de travail quantique à créer, par exemple , **MyQuantumWorkspace**.
- Liste des fournisseurs Azure Quantum à utiliser dans l'espace de travail. Un fournisseur propose un ensemble de plans, chacun représentant un plan avec des conditions générales, des coûts et des quotas associés. Pour créer des espaces de travail, vous devez spécifier le plan correspondant avec les fournisseurs.

Si vous connaissez déjà le fournisseur et les noms de plan à utiliser dans votre espace de travail, vous pouvez passer à l'étape quatre, ci-dessous. Si vous souhaitez commencer avec les fournisseurs qui offrent un crédit gratuit, vous pouvez entrer la commande suivante :

Azure CLI

```
az quantum workspace create \  
-l MyLocation \  
-g MyResourceGroup \  
-w MyQuantumWorkspace \  
-a MyStorageAccount
```

Vous serez peut-être invité à accepter les conditions d'utilisation. Entrez **Y** pour accepter les conditions. Notez que le paramètre indiqué à l'étape **-r 4**, ci-dessous, n'a pas été requis.

Si vous devez déterminer les fournisseurs et les plans à utiliser, procédez comme suit :

1. Pour récupérer la liste des fournisseurs quantiques disponibles, utilisez la `list` commande (cet exemple utilise **westus** comme emplacement) :

Azure CLI

```
az quantum offerings list \  
  -l westus \  
  -o table
```

💡 Conseil

Si vous souhaitez voir quels fournisseurs donnent un crédit gratuit, utilisez le `--autoadd-only` paramètre, par exemple :

```
az quantum offerings list --autoadd-only -l westus -o table
```

Comme mentionné précédemment, ces fournisseurs sont automatiquement ajoutés à votre espace de travail. Vous n'avez pas besoin de les spécifier avec le `-r` paramètre.

2. Une fois que vous avez déterminé le fournisseur et que vous prévoyez d'inclure dans votre espace de travail, vous pouvez passer en revue les termes à l'aide de la `show-terms` commande (en ajoutant votre **MyProviderID** et **MyPlan** comme exemple de valeurs) :

Azure CLI

```
az quantum offerings show-terms \  
  -l westus \  
  -p MyProviderId \  
  -k MyPlan
```

3. La sortie de la `show-terms` commande inclut un champ `accepted` booléen qui indique si les termes de ce fournisseur ont déjà été acceptés ou non, ainsi qu'un lien vers les termes du contrat de licence à examiner. Si vous décidez d'accepter ces termes, utilisez la `accept-terms` commande pour enregistrer votre acceptation.

Azure CLI

```
az quantum offerings accept-terms \  
  -l westus \  
  -p MyProviderId \  
  -k MyPlan
```

4. Une fois que vous avez examiné et accepté toutes les conditions générales requises, vous pouvez créer votre espace de travail à l'aide de la `create` commande, en spécifiant une

liste de combinaisons de fournisseurs et de plans séparées par des virgules, comme dans l'exemple suivant :

Azure CLI

```
az quantum workspace create \  
  -l westus \  
  -g MyResourceGroup \  
  -w MyQuantumWorkspace \  
  -a MyStorageAccount \  
  -r "MyProvider1/MyPlan1, MyProvider2/MyPlan2"
```

Une fois que vous avez créé un espace de travail, vous pouvez toujours ajouter ou supprimer des fournisseurs à l'aide du portail Azure.

Modifier le compte de stockage par défaut pour un espace de travail quantique

Si vous devez modifier le compte de stockage par défaut pour un espace de travail existant, vous pouvez utiliser la `create` commande, en spécifiant toutes les propriétés actuelles ainsi que le nouveau compte de stockage. L'exemple suivant utilise les mêmes paramètres que l'espace de travail créé dans l'exemple précédent :

Azure CLI

```
az quantum workspace create \  
  -l westus \  
  -g MyResourceGroup \  
  -w MyQuantumWorkspace \  
  -a MyNEWStorageAccount \  
  -r "MyProvider1/MyPlan1, MyProvider2/MyPlan2"
```

Important

Cette procédure recrée réellement l'espace de travail avec le nouveau compte de stockage. Vérifiez que toutes les propriétés autres que le compte de stockage sont exactement identiques à l'original, sinon un deuxième espace de travail est créé.

Supprimer un espace de travail quantique

Si vous connaissez le nom et le groupe de ressources d'un espace de travail quantique que vous souhaitez supprimer, vous pouvez le faire avec la `delete` commande (en utilisant les

mêmes noms que l'exemple précédent) :

Azure CLI

```
az quantum workspace delete \  
  -g MyResourceGroup \  
  -w MyQuantumWorkspace
```

💡 Conseil

Si vous ne vous souvenez pas du nom exact, vous pouvez afficher la liste complète des espaces de travail quantiques dans votre abonnement à l'aide `az quantum workspace list -o table` de .

Après avoir supprimé un espace de travail, il apparaît encore pendant que sa suppression est en cours dans le cloud. Toutefois, la `provisioningState` propriété de l'espace de travail change immédiatement pour indiquer qu'elle est supprimée. Vous pouvez voir ces informations à l'aide de la `show` commande :

Azure CLI

```
az quantum workspace show \  
  -g MyResourceGroup \  
  -w MyQuantumWorkspace
```

⚠ Remarque

Si vous avez utilisé la `az quantum workspace set` commande précédemment pour spécifier un espace de travail quantique par défaut, vous pouvez appeler la `delete` commande sans paramètres pour supprimer (et effacer) l'espace de travail par défaut.

Azure CLI

```
az quantum workspace delete
```

Étapes suivantes

Maintenant que vous pouvez créer et supprimer des espaces de travail, découvrez les différents [targets pour exécuter des algorithmes quantiques dans Azure Quantum](#).

az quantum

Préversion

ⓘ Remarque

Cette référence fait partie de l'extension **quantum** pour la Azure CLI (version 2.73.0 ou ultérieure). L'extension installe automatiquement la première fois que vous exécutez une commande **az quantum** . [Apprenez-en davantage](#) sur les extensions.

Ce groupe de commandes est en préversion et en cours de développement. Niveaux de référence et de support : https://aka.ms/CLI_refstatus ↗

Gérez Azure Quantum espaces de travail et envoyez des travaux à des fournisseurs Azure Quantum.

Commandes

 Agrandir le tableau

Nom	Description	Type	État
az quantum execute	Envoyez un travail à exécuter sur Azure Quantum et attendez le résultat. Équivalent à <code>az quantum run</code> .	Extension	Preview
az quantum job	Gérez les travaux pour Azure Quantum.	Extension	Preview
az quantum job cancel	Demande d'annulation d'un travail sur Azure Quantum s'il n'est pas terminé.	Extension	Preview
az quantum job delete	Supprimez un travail d'un espace de travail Azure Quantum.	Extension	Preview
az quantum job file	Gérez les fichiers associés à un travail quantique.	Extension	Preview
az quantum job file download	Téléchargez un fichier à partir du conteneur de stockage de sortie d'un travail.	Extension	Preview
az quantum job file list	Répertoriez les fichiers stockés dans le conteneur de stockage de sortie d'un travail.	Extension	Preview
az quantum job list	Obtenez la liste des travaux dans un espace de travail Quantum.	Extension	Preview

Nom	Description	Type	État
az quantum job output	Obtenez les résultats de l'exécution d'un travail.	Extension	Preview
az quantum job show	Obtenez l'état et les détails du travail.	Extension	Preview
az quantum job submit	Envoyez un programme ou un circuit à exécuter sur Azure Quantum.	Extension	Preview
az quantum job update	Mettez à jour le nom, la priorité et/ou les balises d'un travail soumis.	Extension	Preview
az quantum job wait	Placez l'interface CLI dans un état d'attente jusqu'à ce que le travail se termine en cours d'exécution.	Extension	Preview
az quantum offerings	Gérer les offres de fournisseurs pour Azure Quantum.	Extension	Preview
az quantum offerings accept-terms	Acceptez les termes d'une combinaison de fournisseurs et de référence SKU pour l'activer pour la création de l'espace de travail.	Extension	Preview
az quantum offerings list	Obtenez la liste de toutes les offres de fournisseur disponibles sur l'emplacement donné.	Extension	Preview
az quantum offerings show-terms	Affichez les conditions d'une combinaison fournisseur et référence SKU, y compris l'URL de licence et l'état d'acceptation.	Extension	Preview
az quantum run	Envoyez un travail à exécuter sur Azure Quantum et attendez le résultat. Équivalent à <code>az quantum execute</code> .	Extension	Preview
az quantum target	Gérer les cibles pour les espaces de travail Azure Quantum.	Extension	Preview
az quantum target clear	Effacez l'ID cible par défaut.	Extension	Preview
az quantum target list	Obtenez la liste des fournisseurs et de leurs cibles dans un espace de travail Azure Quantum.	Extension	Preview
az quantum target set	Sélectionnez la cible par défaut à utiliser lors de l'envoi de travaux à Azure Quantum.	Extension	Preview
az quantum target show	Obtenez l'ID cible de la cible par défaut actuelle à utiliser lors de l'envoi de travaux à Azure Quantum.	Extension	Preview
az quantum workspace	Gérer Azure Quantum espaces de travail.	Extension	Preview
az quantum workspace clear	Effacez l'espace de travail Azure Quantum par défaut.	Extension	Preview

Nom	Description	Type	État
az quantum workspace create	Créez un espace de travail Azure Quantum.	Extension	Preview
az quantum workspace delete	Supprimez l'espace de travail donné (ou actuel) Azure Quantum.	Extension	Preview
az quantum workspace keys	Gérez Azure Quantum clés api d'espace de travail.	Extension	Preview
az quantum workspace keys list	Répertoriez les clés API pour l'espace de travail donné (ou actuel) Azure Quantum.	Extension	Preview
az quantum workspace keys regenerate	Régénérez la clé API pour l'espace de travail Azure Quantum donné (ou actuel).	Extension	Preview
az quantum workspace list	Obtenez la liste des espaces de travail Azure Quantum disponibles.	Extension	Preview
az quantum workspace quotas	Répertoriez les quotas pour l'espace de travail donné (ou actuel) Azure Quantum.	Extension	Preview
az quantum workspace set	Sélectionnez un espace de travail Azure Quantum par défaut pour les commandes futures.	Extension	Preview
az quantum workspace show	Obtenez les détails de l'espace de travail donné (ou actuel) Azure Quantum.	Extension	Preview
az quantum workspace update	Mettez à jour l'espace de travail Azure Quantum donné (ou actuel).	Extension	Preview
az quantum workspace user	Gérer les utilisateurs d'un espace de travail Azure Quantum.	Extension	Preview
az quantum workspace user create	Accordez à un utilisateur, un groupe ou un principal de service un accès à un espace de travail Azure Quantum.	Extension	Preview
az quantum workspace user delete	Supprimez l'accès d'un utilisateur, d'un groupe ou d'un principal de service à un espace de travail Azure Quantum.	Extension	Preview
az quantum workspace user list	Répertoriez les utilisateurs ayant accès à un espace de travail Azure Quantum.	Extension	Preview

az quantum execute

Préversion

Le groupe de commandes « quantum » est en préversion et en cours de développement.
Niveaux de référence et de support : https://aka.ms/CLI_refstatus 

Envoyez un travail à exécuter sur Azure Quantum et attendez le résultat. Équivalent à `az quantum run`.

Azure CLI

```
az quantum execute --job-input-file
                    --job-input-format
                    --resource-group
                    --target-id
                    --workspace-name
                    [--acquire-policy-token]
                    [--change-reference]
                    [--entry-point]
                    [--job-name]
                    [--job-output-format]
                    [--job-params]
                    [--shots]
                    [--storage]
                    [--target-capability]
```

Exemples

Exécutez le code binaire QIR à partir d'un fichier dans le dossier actif et attendez le résultat.

Azure CLI

```
az quantum execute -g MyResourceGroup -w MyWorkspace -t MyTarget \
  --job-name MyJob --job-input-format qir.v1 --job-input-file MyQirBitcode.bc \
  --entry-point MyQirEntryPoint
```

Exécutez un travail pass-through Quil sur le simulateur Rigetti et attendez le résultat.

Azure CLI

```
az quantum execute -g MyResourceGroup -w MyWorkspace \
  -t rigetti.sim.qvm --job-name MyJob --job-input-file MyProgram.quil \
  --job-input-format rigetti.quil.v1 --job-output-format rigetti.quil-results.v1
```

Envoyez un circuit Qiskit au simulateur IonQ avec des paramètres de travail et attendez les résultats.

```
az quantum execute -g MyResourceGroup -w MyWorkspace \  
  -t ionq.simulator --job-name MyJobName --job-input-file MyCircuit.json \  
  --job-input-format ionq.circuit.v1 --job-output-format ionq.quantum-results.v1 \  
  --job-params count=100 content-type=application/json
```

Paramètres obligatoires

--job-input-file

Emplacement du fichier d'entrée à envoyer.

--job-input-format

Format du fichier à envoyer.

--resource-group -g

Nom du groupe de ressources. Vous pouvez configurer le groupe par défaut à l'aide de `az configure --defaults group=<name>`.

--target-id -t

Moteur d'exécution pour les travaux de calcul quantique. Lorsqu'un espace de travail est configuré avec un ensemble de fournisseurs, ils activent chacun une ou plusieurs cibles.

Vous pouvez configurer la cible par défaut à l'aide `az quantum target set` de .

--workspace-name -w

Nom de l'espace de travail Quantum. Vous pouvez configurer l'espace de travail par défaut à l'aide `az quantum workspace set` de .

Paramètres facultatifs

Les paramètres suivants sont facultatifs, mais en fonction du contexte, un ou plusieurs peuvent être nécessaires pour que la commande s'exécute correctement.

--acquire-policy-token

Acquisition automatique d'un jeton Azure Policy pour cette opération de ressource.

Propriété	Valeur
Groupe de paramètres:	Global Policy Arguments

--change-reference

ID de référence de modification associé pour cette opération de ressource.

Propriété	Valeur
Groupe de paramètres:	Global Policy Arguments

--entry-point

Point d'entrée du programme ou du circuit QIR. Requis pour certains travaux QIR de fournisseur.

--job-name

Nom convivial à donner à cette exécution du programme.

--job-output-format

Format de sortie du travail attendu.

--job-params

Paramètres de travail passés à la cible sous la forme d'une liste de paires clé=valeur, de chaîne json ou `@{file}` avec du contenu json.

--shots

Nombre de fois où exécuter le programme sur la cible donnée.

--storage

Si elle est spécifiée, connectionString d'un stockage Azure est utilisée pour stocker les données et les résultats des travaux.

--target-capability

Paramètre de capacité cible transmis au compilateur.

Paramètres globaux

--debug

Augmentez la verbosité de la journalisation pour afficher tous les logs de débogage.

 Agrandir le tableau

Propriété	Valeur
Valeur par défaut:	False

--help -h

Affichez ce message d'aide et quittez.

--only-show-errors

Afficher uniquement les erreurs, en supprimant les avertissements.

 Agrandir le tableau

Propriété	Valeur
Valeur par défaut:	False

--output -o

Format de sortie.

 Agrandir le tableau

Propriété	Valeur
Valeur par défaut:	json
Valeurs acceptées:	json, jsonc, none, table, tsv, yaml, yamlc

--query

Chaîne de requête JMESPath. Pour plus d'informations et d'exemples, consultez <http://jmespath.org/>.

--subscription

Nom ou ID de l'abonnement. Vous pouvez configurer l'abonnement par défaut à l'aide de `az account set -s NAME_OR_ID`.

--verbose

Augmentez le niveau de verbosité de la journalisation. Utilisez `--debug` pour les journaux de débogage complets.

[🔗](#) Agrandir le tableau

Propriété	Valeur
Valeur par défaut:	False

az quantum run

Préversion

Le groupe de commandes « quantum » est en préversion et en cours de développement.
Niveaux de référence et de support : https://aka.ms/CLI_refstatus

Envoyez un travail à exécuter sur Azure Quantum et attendez le résultat. Équivalent à `az quantum execute`.

Azure CLI

```
az quantum run --job-input-file
               --job-input-format
               --resource-group
               --target-id
               --workspace-name
               [--acquire-policy-token]
               [--change-reference]
               [--entry-point]
               [--job-name]
               [--job-output-format]
               [--job-params]
```

```
[--shots]  
[--storage]  
[--target-capability]
```

Exemples

Exécutez le code binaire QIR à partir d'un fichier dans le dossier actif et attendez le résultat.

Azure CLI

```
az quantum run -g MyResourceGroup -w MyWorkspace -t MyTarget \  
  --job-name MyJob --job-input-format qir.v1 --job-input-file MyQirBitcode.bc \  
  --entry-point MyQirEntryPoint
```

Exécutez un travail pass-through Quil sur le simulateur Rigetti et attendez le résultat.

Azure CLI

```
az quantum run -g MyResourceGroup -w MyWorkspace \  
  -t rigetti.sim.qvm --job-name MyJob --job-input-file MyProgram.quil \  
  --job-input-format rigetti.quil.v1 --job-output-format rigetti.quil-results.v1
```

Envoyez un circuit Qiskit au simulateur IonQ avec des paramètres de travail et attendez les résultats.

Azure CLI

```
az quantum run -g MyResourceGroup -w MyWorkspace \  
  -t ionq.simulator --job-name MyJobName --job-input-file MyCircuit.json \  
  --job-input-format ionq.circuit.v1 --job-output-format ionq.quantum-results.v1 \  
  --job-params count=100 content-type=application/json
```

Paramètres obligatoires

--job-input-file

Emplacement du fichier d'entrée à envoyer.

--job-input-format

Format du fichier à envoyer.

--resource-group -g

Nom du groupe de ressources. Vous pouvez configurer le groupe par défaut à l'aide de `az configure --defaults group=<name>`.

`--target-id -t`

Moteur d'exécution pour les travaux de calcul quantique. Lorsqu'un espace de travail est configuré avec un ensemble de fournisseurs, ils activent chacun une ou plusieurs cibles. Vous pouvez configurer la cible par défaut à l'aide `az quantum target set` de .

`--workspace-name -w`

Nom de l'espace de travail Quantum. Vous pouvez configurer l'espace de travail par défaut à l'aide `az quantum workspace set` de .

Paramètres facultatifs

Les paramètres suivants sont facultatifs, mais en fonction du contexte, un ou plusieurs peuvent être nécessaires pour que la commande s'exécute correctement.

`--acquire-policy-token`

Acquisition automatique d'un jeton Azure Policy pour cette opération de ressource.

[🔗](#) Agrandir le tableau

Propriété	Valeur
Groupe de paramètres:	Global Policy Arguments

`--change-reference`

ID de référence de modification associé pour cette opération de ressource.

[🔗](#) Agrandir le tableau

Propriété	Valeur
Groupe de paramètres:	Global Policy Arguments

`--entry-point`

Point d'entrée du programme ou du circuit QIR. Requis pour certains travaux QIR de fournisseur.

--job-name

Nom convivial à donner à cette exécution du programme.

--job-output-format

Format de sortie du travail attendu.

--job-params

Paramètres de travail passés à la cible sous la forme d'une liste de paires clé=valeur, de chaîne json ou `@{file}` avec du contenu json.

--shots

Nombre de fois où exécuter le programme sur la cible donnée.

--storage

Si elle est spécifiée, connectionString d'un stockage Azure est utilisée pour stocker les données et les résultats des travaux.

--target-capability

Paramètre de capacité cible transmis au compilateur.

Paramètres globaux

--debug

Augmentez la verbosité de la journalisation pour afficher tous les logs de débogage.

 Agrandir le tableau

Propriété	Valeur
Valeur par défaut:	False

--help -h

Affichez ce message d'aide et quittez.

--only-show-errors

Afficher uniquement les erreurs, en supprimant les avertissements.

[🔗](#) Agrandir le tableau

Propriété	Valeur
Valeur par défaut:	False

--output -o

Format de sortie.

[🔗](#) Agrandir le tableau

Propriété	Valeur
Valeur par défaut:	json
Valeurs acceptées:	json, jsonc, none, table, tsv, yaml, yamlc

--query

Chaîne de requête JMESPath. Pour plus d'informations et d'exemples, consultez <http://jmespath.org/> [🔗](#).

--subscription

Nom ou ID de l'abonnement. Vous pouvez configurer l'abonnement par défaut à l'aide de `az account set -s NAME_OR_ID`.

--verbose

Augmentez le niveau de verbosité de la journalisation. Utilisez `--debug` pour les journaux de débogage complets.

[🔗](#) Agrandir le tableau

Propriété	Valeur
Valeur par défaut:	False

Gérer des espaces de travail quantiques à l'aide de Bicep

Dans ce guide, apprenez à utiliser un modèle Bicep pour créer des espaces de travail Azure Quantum et les groupes de ressources et les comptes de stockage requis. Après le déploiement du modèle, vous pouvez commencer à exécuter vos applications quantiques dans Azure Quantum. En traitant votre infrastructure en tant que code, vous pouvez suivre les changements de vos besoins en infrastructure et rendre vos déploiements plus cohérents et reproductibles.

Bicep utilise une syntaxe déclarative que vous traitez comme du code d'application. Si vous connaissez la syntaxe JSON pour écrire des modèles Azure Resource Manager (ARM), vous trouverez que Bicep fournit une syntaxe plus concise et une sécurité de type améliorée.

Prerequisites

compte Azure

Avant de commencer, vous devez disposer d'un compte Azure doté d'un abonnement actif. Si vous n'avez pas de compte Azure, inscrivez-vous gratuitement et inscrivez-vous à un abonnement [pay-as-you-go](#).

Rédacteur

Pour créer des modèles Bicep, vous avez besoin d'un bon éditeur. Nous vous recommandons la dernière version de [VS Code](#) avec l'extension [Bicep](#).

Déploiement en ligne de commande

Vous avez également besoin d'Azure PowerShell ou d'Azure CLI pour déployer le modèle. Si vous utilisez Azure CLI, vous devez disposer de la dernière version. Pour les instructions d'installation, consultez :

- [Installer Azure PowerShell](#)
- [Installer Azure CLI sur Windows](#)
- [Installer Azure CLI sur Linux](#)
- [Installer Azure CLI sur macOS](#)

Connectez-vous à Azure

Après avoir installé Azure PowerShell ou Azure CLI, veuillez à vous connecter pour la première fois. Choisissez l'un des onglets suivants et exécutez les commandes de ligne de commande correspondantes pour vous connecter à Azure :

Service de contrôle d'accès Azure (CLI)

Azure CLI

```
az login
```

Si vous avez plusieurs abonnements Azure, sélectionnez celui que vous souhaitez utiliser. Remplacez `SubscriptionName` par votre nom d'abonnement. Vous pouvez également utiliser un ID d'abonnement au lieu du nom d'abonnement.

Azure CLI

```
az account set --subscription SubscriptionName
```

Créer un groupe de ressources vide

Lorsque vous déployez un modèle, spécifiez un groupe de ressources qui contient l'espace de travail quantique avec ses ressources associées. Avant d'exécuter la commande de déploiement, créez le groupe de ressources avec Azure CLI ou Azure PowerShell.

Service de contrôle d'accès Azure (CLI)

Azure CLI

```
az group create --name myResourceGroup --location "East US"
```

Passer en revue le modèle de Azure Bicep

Azure CLI

```
@description('Application name used as prefix for the Azure Quantum workspace and its associated Storage account.')
```

```

param appName string

@description('Location of the Azure Quantum workspace and its associated Storage
account.')
@allowed([
    'westus'
    'eastus'
    'northeurope'
    'westeurope'
    'canary'
])
param location string

var quantumWorkspaceName = '${appName}-ws'
var storageAccountName = '${appName}${substring(uniqueString(resourceGroup().id), 0,
5)}'

resource storageAccount 'Microsoft.Storage/storageAccounts@2021-06-01' = {
    name: storageAccountName
    location: location
    sku: {
        name: 'Standard_LRS'
    }
    kind: 'StorageV2'
}

resource quantumWorkspace 'Microsoft.Quantum/Workspaces@2019-11-04-preview' = {
    name: quantumWorkspaceName
    location: location
    identity: {
        type: 'SystemAssigned'
    }
    properties: {
        providers: [
            {
                providerId: 'Microsoft'
                providerSku: 'DZH3178M639F'
                applicationName: '${quantumWorkspaceName}-Microsoft'
            }
        ]
        storageAccount: storageAccount.id
    }
}

resource roleAssignment 'Microsoft.Authorization/roleAssignments@2020-04-01-preview' =
{
    scope: storageAccount
    name: guid(quantumWorkspace.id,
'/subscriptions/${subscription().subscriptionId}/providers/Microsoft.Authorization/rol
eDefinitions/b24988ac-6180-42a0-ab88-20f7382dd24c', storageAccount.id)
    properties: {
        roleDefinitionId:
'/subscriptions/${subscription().subscriptionId}/providers/Microsoft.Authorization/rol
eDefinitions/b24988ac-6180-42a0-ab88-20f7382dd24c'
    }
}

```

```
principalId: reference(quantumWorkspace.id, '2019-11-04-preview',  
'full').identity.principalId  
}  
}
```

```
output subscription_id string = subscription().subscriptionId  
output resource_group string = resourceGroup().name  
output name string = quantumWorkspace.name  
output location string = quantumWorkspace.location  
output tenant_id string = subscription().tenantId
```

Les ressources Azure suivantes sont créées par le modèle :

- **Compte de stockage Azure : compte de stockage** pour le stockage des données d'entrée et de sortie pour les travaux quantiques.
- **Espace de travail Azure Quantum** : collection de ressources associées à l'exécution d'applications quantiques.

Le modèle accorde également l'autorisation **Contributeur** d'espace de travail quantique au compte de stockage. Cette étape est nécessaire pour que l'espace de travail puisse lire et écrire des données de travail.

Le modèle génère la sortie suivante. Vous pouvez utiliser ces valeurs ultérieurement pour identifier l'espace de travail quantique généré et l'authentifier auprès de celui-ci :

- **ID d'abonnement** hébergeant toutes les ressources déployées.
- **Groupe de ressources** contenant toutes les ressources déployées.
- **Nom** de l'espace de travail quantique.
- **Emplacement** du centre de données qui héberge l'espace de travail.
- **ID de locataire** contenant les informations d'identification utilisées lors du déploiement.

Déployer le modèle

Pour déployer le modèle, utilisez Azure CLI ou Azure PowerShell. Utilisez le groupe de ressources que vous avez créé. Donnez un nom au déploiement afin de pouvoir l'identifier facilement dans l'historique du déploiement. Remplacez le `{provide-the-path-to-the-template-file}` et les accolades `{ }` par le chemin d'accès de votre fichier de modèle. En outre, remplacez `{provide-app-name}` et `{provide-location}` par des valeurs pour le nom global de l'application et l'emplacement où l'espace de travail doit résider. Le nom de l'application ne doit contenir que des lettres.

Pour exécuter cette commande de déploiement, vous devez disposer de la [dernière version](#) d'Azure CLI.

Azure CLI

```
templateFile="{provide-the-path-to-the-template-file}"
az deployment group create \
  --name myDeployment \
  --resource-group myResourceGroup \
  --template-file $templateFile \
  --parameters appName="{provide-app-name}" location="{provide-location}"
```

La commande de déploiement retourne les résultats. Cherchez `ProvisioningState` pour voir si le déploiement a réussi.

Important

Dans certains cas, vous pouvez obtenir une erreur de déploiement (Code : `PrincipalNotFound`). Pour cette raison, le principal de l'espace de travail n'a pas encore été créé lorsque le gestionnaire de ressources a essayé de configurer l'attribution de rôle. Si c'est le cas, répétez simplement le déploiement. Il devrait réussir à la deuxième tentative.

Valider le déploiement

Vous pouvez vérifier le déploiement en explorant le groupe de ressources à partir du portail Azure.

1. Connectez-vous au portail [Azure](#).
2. Dans le menu de gauche, sélectionnez **Groupe de ressources**.
3. Sélectionnez le groupe de ressources déployer dans la dernière procédure. Le nom par défaut est **myResourceGroup**. Vous devez voir deux ressources déployées dans le groupe de ressources : le compte de stockage et l'espace de travail quantique.
4. Vérifiez que l'espace de travail quantique dispose des droits d'accès nécessaires pour le compte de stockage. Sélectionnez le compte de stockage. Dans le volet de menu de gauche, sélectionnez **Contrôle d'accès (IAM)** et vérifiez que, sous **Attributions de rôles**, la ressource d'espace de travail quantique est répertoriée sous **Contributeur**.

Nettoyer les ressources

Si vous n'avez plus besoin de l'espace de travail quantique, vous pouvez supprimer le groupe de ressources.

Service de contrôle d'accès Azure (CLI)

Azure CLI

```
az group delete --name myResourceGroup
```

Étapes suivantes

Maintenant que vous pouvez créer et supprimer des espaces de travail, découvrez les différents [targets pour exécuter des algorithmes quantiques dans Azure Quantum](#). Vous disposez désormais également des outils permettant d'effectuer des déploiements d'espaces de travail à partir d'[Azure Pipelines](#) ou [gitHub Actions](#).

Last updated on 14/08/2026

Plans tarifaires pour les fournisseurs de Azure Quantum

Dans Azure Quantum, les fournisseurs de matériel et de logiciels définissent et contrôlent la tarification de leurs offres. Les informations contenues dans cet article sont susceptibles d'être modifiées par les fournisseurs et certains retards dans la réflexion sur les dernières informations de tarification peuvent exister. Veuillez à vérifier les dernières informations de tarification de l'espace de travail Azure Quantum que vous utilisez.

IonQ

IonQ facture des frais selon un modèle de tarification basé sur les jetons, où l'unité de facturation est le *Azure Quantum Token (AQT)*. Ce modèle est spécifique à Azure Quantum. Le nombre de jetons Azure Quantum est calculé par la formule suivante :

$$AQT = m + 0,000220 \cdot (N_{1q} \cdot C) + 0.000975 \cdot (N_{2q} \cdot C)$$

where:

- AQT correspond au nombre de jetons de Azure Quantum consommés par le programme
- m est le prix minimum par exécution de programme, qui est de 97,50 USD si la mitigation d'erreur est activée et de 12,4166 USD si la mitigation d'erreur est désactivée
- Les unités $N_{1q} \cdot C$ et $N_{2q} \cdot C$ sont appelées *tirages de portes*, où N_{1q} et N_{2q} sont le nombre de portes à un et deux qubits soumises, respectivement, et C est le nombre de tirages d'exécution

Les portes à deux qubits avec contrôles multiples sont facturées comme $6 * (N - 2)$ portes à deux qubits, où N est le nombre de qubits impliqués dans la porte. Par exemple, une porte NOT avec trois contrôles serait facturée comme $(6 * (4 - 2))$ ou 12 portes à deux qubits. Les portes d'un qubit sont facturées sous la forme 0,225 d'une porte à deux qubits (arrondie au chiffre du dessous). Pour en savoir plus sur IonQ, consultez la [page du fournisseur IonQ](#).

IonQ offre un plan de **paiement à l'utilisation** et un plan **d'abonnement mensuel** avec accès au simulateur quantique, aux ordinateurs quantiques Aria 1 25 qubits et aux ordinateurs quantiques IonQ Forte 1 et Forte Enterprise 1 36 qubits.

Le plan paiement à l'utilisation se compose d'un accès à la carte aux ordinateurs quantiques IonQ Aria 1 à 25 qubits, IonQ Forte 1 et Forte Enterprise 1 à 36 qubits, ainsi que le simulateur IonQ. L'utilisation des ordinateurs quantiques est facturée en fonction du nombre d'AQTs + coûts d'infrastructure Azure.

[Agrandir le tableau](#)

Inclut l'accès à	Pricing
Simulateur IonQ	Gratuit
IonQ Aria 1	• USD0.000220 / tirage de porte à un qubit • USD0.000975 / tirage de porte à deux qubits • Prix minimal par exécution du programme : ◦ USD97.50 - paramètre par défaut, l'atténuation des erreurs est activée ◦ USD12.4166 si l'atténuation des erreurs est désactivée
IonQ Forte 1	• USD0.0001645 / opération de porte à 1 qubit • 0,001121 USD / essai de 2 portes à qubit • Prix minimal par exécution du programme : ◦ USD168.195 - paramètre par défaut, l'atténuation des erreurs est activée ◦ USD25.7899 si l'atténuation des erreurs est désactivée
IonQ Forte Enterprise 1	• USD0.0001645 / opération de porte à 1 qubit • 0,001121 USD / essai de 2 portes à qubit • Prix minimal par exécution du programme : ◦ USD168.195 - paramètre par défaut, l'atténuation des erreurs est activée ◦ USD25.7899 si l'atténuation des erreurs est désactivée

Pour plus d'informations sur les coûts d'infrastructure Azure, consultez [Stockage Blob Azure tarification](#).

ⓘ Remarque

Si vous disposez d'une subvention de recherche existante ou un client avec paiement à l'utilisation, vous pouvez voir une autre unité de facturation en plus d'AQT, appelée *Quantum Gate-Shots (QGS)*. Les unités QGS sont équivalentes aux unités AQT. Le nombre de QGS est calculé par la formule suivante :

$$QGS = N \cdot C$$

où :

- QGS est le nombre de tirages de portes quantiques
- N est le nombre de portes d'un ou deux qubits soumis
- C est le nombre de tirs d'exécution

PASQAL

PASQAL facture le temps d'exécution des travaux sur son processeur quantique, le Fresnel à 100 qubits, et sur son émulateur de réseaux tensoriels de pointe, EMU-TN.

PASQAL propose un plan de facturation : **Paiement à l'utilisation**.

Paiement à l'utilisation

Dans le plan paiement à l'utilisation, l'utilisation est facturée uniquement en fonction du temps d'exécution du travail.

 Agrandir le tableau

Pricing	Inclut l'accès à
• 300 USD/heure QPU + coûts d'infrastructure Azure • 15 USD/ heure EMU-TN + coûts d'infrastructure Azure	• PASQAL Fresnel QPU • EMU-TN PASQAL

Pour plus d'informations sur les coûts d'infrastructure Azure, consultez [Stockage Blob Azure tarification](#).

Quantinuum

Quantinuum utilise un système dans lequel chaque travail est facturé en fonction du nombre d'opérations qu'il contient et du nombre d'exécutions que vous effectuez. Les unités d'utilisation définies par Quantinuum sont des *crédits quantiques matériels (HQC)* pour les travaux soumis aux ordinateurs quantiques et aux HQC (eHQCs) pour les travaux soumis aux émulateurs.

Les crédits HQC et eHQC servent à calculer le coût d'exécution d'un travail, et ils sont calculés selon la formule suivante :

$$HQC = 5 + C(N_{1q} + 10N_{2q} + 5N_m)/5000$$

where:

- N_{1q} est le nombre d'opérations à un seul qubit dans un circuit.
- N_{2q} est le nombre d'opérations natives à deux qubit dans un circuit. La porte native est équivalente à CNOT jusqu'à plusieurs portes à un seul qubit.
- N_m est le nombre d'opérations de préparation et de mesure (SPAM) de l'état dans un circuit, y compris la préparation de l'état implicite initial et toutes les mesures intermédiaires et finales et les réinitialisations d'état.
- C est le nombre de captures.

Pour en savoir plus sur Quantinuum, consultez la [page du fournisseur Quantinuum](#).

Quantinuum fournit deux plans d'abonnement : **Standard** et **Premium**. Quantinuum propose également une offre de **paiement à l'utilisation**.

Subscriptions

Les abonnements sont mensuels et accessibles par accès en file d'attente.

Agrandir le tableau

Plans et tarifs	Inclut l'accès à
Plan Standard : 125 000 USD/Mois + coûts d'infrastructure Azure	• 10 000 HQC à utiliser avec le modèle de système Quantinuum H2 • 100 000 eHQC pour les émulateurs du modèle de système Quantinuum H2
Premium Plan : USD175 000/Mois + coûts d'infrastructure Azure	• 17k HQC à utiliser sur le modèle de système Quantinuum H2 • 170 000 eHQC à utiliser sur les émulateurs du modèle de système H2 Quantinuum

Pour plus d'informations sur les coûts d'infrastructure Azure, consultez [Stockage Blob Azure tarification](#) .

[Rigetti](#) facture le temps d'exécution des travaux sur leurs processeurs quantiques. Il n'y a pas de frais supplémentaires par travail, par coup ou par porte. Le simulateur [QVM \(Quantum Virtual Machine\)](#) [↗](#) est gratuit pour tous les utilisateurs sous tous les plans.

Rigetti offre un plan de facturation : **Paielement à l'utilisation**.

Paielement à l'utilisation

Le plan Paielement à l'utilisation consiste en un accès à *la carte* aux QPU Rigetti. L'utilisation est facturée en fonction du temps d'exécution du travail uniquement.

[🔍](#) Agrandir le tableau

Pricing	Inclut l'accès à
Incrément de 0,02 USD par incrément de 10 millisecondes du temps d'exécution du travail	Rigetti Cepheus-1-108Q

ⓘ Remarque

Si vous avez des questions ou rencontrez un problème à l'aide de Azure Quantum, vous pouvez contacter AzureQuantumInfo@microsoft.com.

Contenu connexe

- quotas [Azure Quantum](#)
- [Liste d'informatique target quantique](#)

Last updated on 10/04/2026

FAQ : Limites et quotas dans Azure Quantum

Dans cet article, vous trouverez des réponses aux questions sur les limites et les quotas d'utilisation des fournisseurs de Azure Quantum.

Quels sont les quotas dans Azure Quantum ?

Les quotas sont des limites sur l'utilisation de l'unité de traitement quantique (QPU). targets Chaque fournisseur Azure Quantum définit ses propres quotas. Les quotas permettent d'éviter les dépassements de coûts accidentels pour l'utilisateur et de préserver l'intégrité des systèmes du fournisseur.

Les quotas sont basés sur la sélection de votre plan de fournisseur. Si vous souhaitez augmenter vos quotas, envoyez un ticket de support. L'utilisation suivie par les quotas n'est pas nécessairement liée à un coût ou un crédit, mais peut être corrélée.

Comment les quotas sont-ils définis dans Azure Quantum ?

Dans Azure Quantum, les fournisseurs de matériel et de logiciels définissent et contrôlent les quotas de leurs offres. Pour obtenir des informations détaillées sur le quota, consultez chaque page de référence du fournisseur.

- [Quota IonQ](#)
- [Quota PASQAL](#)
- [Quota Quantinuum](#)

⚠ Remarque

Les fournisseurs qui ne figurent pas dans cette liste ne définissent pas de quotas pour leur targets.

Comment puis-je afficher mon quota restant ?

Azure Quantum, l'utilisation et les quotas sont mesurés en termes d'unités d'utilisation de chaque fournisseur. Certains fournisseurs ne définissent pas de quotas et n'affichent pas d'informations d'utilisation.

Suivre les quotas dans le portail Azure

1. Connectez-vous au portail [Azure](#) avec les informations d'identification de votre abonnement Azure.
2. Accédez à votre espace de travail Azure Quantum.
3. Dans le volet de navigation de l'espace de travail, développez la liste déroulante **Opérations** et choisissez **Quotas**.
4. Consultez les quotas consommés et restants pour chacun des fournisseurs de votre espace de travail.

Les informations de quota sont affichées dans trois colonnes :

- **Utilisation de l'espace de travail** : limite d'utilisation de l'espace de travail actif. Chaque espace de travail Azure Quantum a une limite d'utilisation.
- **Utilisation de l'abonnement Azure** : l'utilisation de tous les espaces de travail dans la région et l'abonnement actuel. Certains quotas ne sont pas suivis à ce niveau.
- **Cadence** : période de renouvellement de votre quota. Si elle est mensuelle, l'utilisation est réinitialisée le 1er de chaque mois. Si elle est unique, l'utilisation n'est jamais réinitialisée.

Suivre les quotas avec Azure CLI

Vous pouvez voir vos quotas à l'aide de l'interface Azure Command-Line (Azure CLI). Pour plus d'informations, consultez [Manage des espaces de travail quantiques avec le Azure CLI](#).

1. Installez l'extension Azure CLI `quantum`. Ouvrez une invite de commandes et exécutez la commande suivante, qui met également à niveau l'extension lorsque vous avez déjà installé une version précédente.

Azure CLI

```
az extension add --upgrade -n quantum
```

2. Connectez-vous à Azure à l'aide de vos informations d'identification. Vous voyez la liste des abonnements associés à votre compte.

Azure CLI


```
az login
```

3. Si vous êtes invité à sélectionner un abonnement, entrez le numéro correspondant à votre abonnement. Ou appuyez sur **Entrée**, puis définissez l'abonnement que vous souhaitez utiliser. Remplacez `MySubscription` par votre ID d'abonnement.

Azure CLI

```
az account set -s MySubscription
```

4. Définissez l'espace de travail que vous souhaitez utiliser. Remplacez `MyResourceGroup` et `MyWorkspace` par le nom de votre groupe de ressources et de votre espace de travail.

Azure CLI

```
az quantum workspace set -g MyResourceGroup -w MyWorkspace -o table
```

5. Affichez vos informations de quota dans une table pour l'espace de travail sélectionné.

Azure CLI

```
az quantum workspace quotas -o table
```

Votre sortie ressemble à l'exemple suivant :

Sortie

Dimension	Utilization	Holds	Limit	Period	ProviderId	Scope
qgs	0.0	8333334.0	Infinite	ionq	Subscription	33334.0
hqc	0.0	800.0	Infinite	quantinuum	Subscription	0.0

Dans cet exemple, la ligne `qgs` indique que le compte a une limite avec IonQ de `8333334 qgs`, dont `33334 qgs` ont déjà été utilisés. Le compte a également une limite de `800` HQC avec Quantinuum, dont `0` ont été utilisés.

La `Scope` colonne indique si le quota fait référence à l'espace de travail actuel ou à l'abonnement.

- **Workspace**: Le quota est suivi pour un espace de travail individuel.
- **Subscription**: Le quota est suivi ensemble pour tous les espaces de travail au sein du même abonnement/région.

La **Period** colonne indique la période de renouvellement de votre quota.

- **Monthly**: l'utilisation est réinitialisée le 1er de chaque mois.
- **Infinite**: L'utilisation ne se réinitialise jamais (également appelée **One-Time** dans la vue du [Azure](#) portal).

Suivre les quotas avec la bibliothèque de Python QDK

1. Installez la dernière version de la bibliothèque [qdk Python](#).
2. Ouvrez un nouveau fichier Python.
3. Créez un objet **Workspace** pour vous connecter à l'espace de travail que vous avez déployé précédemment dans Azure.

Python

```
from qdk.azure import Workspace

# Add your resource_id
workspace = Workspace (resource_id = "")
```

4. Utilisez la méthode **get_quotas** pour afficher les informations de quotas pour l'espace de travail sélectionné.

Python

```
quotas = workspace.get_quotas()

for quota in quotas:
    print(quota)
```

Comment demander une augmentation de quota ?

Pour demander une augmentation de quota, envoyez un ticket de support.

Créez une demande de support

1. Connectez-vous au portail [Azure](#) avec les informations d'identification de votre abonnement Azure.
2. Accédez à votre espace de travail Azure Quantum.
3. Dans le volet de navigation de l'espace de travail, sous le menu déroulant **Opérations**, accédez au panneau **Quotas**.
4. Choisissez le bouton **Augmenter** sur la page de quota.

Ou, sous la liste déroulante **Aide** , accédez au panneau **Support + Résolution des problèmes** .
5. Dans la zone de description du problème, entrez **Azure Quantum demande de dérogation de quota**.
6. Pour le problème de service, choisissez **Aucun des points ci-dessus**.
7. Dans le menu **Sélectionner un service** , choisissez **les limites de service et d'abonnement (quotas)**.
8. Choisissez le bouton **Suivant**.
9. Choisissez le bouton **Créer une demande de support** . Un nouveau ticket de demande de support s'ouvre sur le **1. Onglet Description du problème** .

Remplir la demande de support

Pour remplir la demande de support, procédez comme suit :

1. Pour le **type de problème**, choisissez **les limites de service et d'abonnement (quotas)**.
2. Pour l'**abonnement**, choisissez l'abonnement qui contient l'espace de travail pour lequel vous souhaitez augmenter le quota.
3. Pour **Quota type**, choisissez **Azure Quantum**.
4. Choisissez le bouton **Suivant** pour accéder au **3. Onglet Détails supplémentaires** .
5. Dans la zone **Description** de la section **Détails du problème** , entrez les informations suivantes :
 - Nom du fournisseur pour lequel vous souhaitez augmenter les quotas

- Que vous souhaitiez augmenter les quotas au niveau de l'abonnement ou de l'espace de travail
- Les quotas que vous souhaitez augmenter et à quel niveau vous souhaitez les augmenter.
- Raisons pour lesquelles vous souhaitez augmenter les quotas

6. Choisissez vos options préférées pour les **sections informations de diagnostic avancées** et **méthodes de support** .

7. Renseignez la section **Informations de contact** .

8. Choisissez le bouton **Suivant** pour accéder à l'onglet **4. Vérifier + créer**.

9. Si toutes les informations sont correctes, choisissez le bouton **Créer** .

ⓘ Remarque

Pour envoyer un ticket de support pour une augmentation de quota dans un espace de travail, vous devez être le propriétaire de l'espace de travail.

Contenu connexe

- [FAQ : Présentation des coûts et de la facturation des travaux dans Azure Quantum](#)

Last updated on 13/04/2026

FAQ : Présentation des coûts et de la facturation des travaux dans Azure Quantum

Cet article explique les instructions pour comprendre le coût d'exécution de programmes quantiques dans Azure Quantum et comment gérer vos factures.

Combien de plans d'utilisation sont disponibles dans Azure Quantum ?

Azure Quantum fournit des services de Microsoft et de nos sociétés partenaires, de sorte que les détails de facturation dépendent du fournisseur et du plan tarifaire que vous sélectionnez.

La plupart des fournisseurs proposent un plan de facturation basé sur les ressources que vous consommez lorsque vous exécutez une tâche. Certains fournisseurs proposent également des plans d'abonnement. Pour plus d'informations sur la façon dont chaque fournisseur facture, consultez [Azure Quantum tarification](#).

Comment modifier le plan d'utilisation de mon espace de travail Azure Quantum ?

Pour modifier vos plans d'utilisation actuels pour vos fournisseurs et voir les différents plans de facturation disponibles dans votre devise locale, procédez comme suit :

1. Connectez-vous au portail [Azure](#) avec les informations d'identification de votre abonnement Azure.
2. Accédez à votre espace de travail Azure Quantum.
3. Dans le volet de navigation de l'espace de travail, développez la liste déroulante **Opérations** et choisissez **Fournisseurs**.
4. Affichez le plan d'utilisation actuel pour chaque fournisseur dans la colonne **Plan**.
5. Choisissez **Modifier** pour afficher et modifier les différents plans de facturation disponibles pour ce fournisseur dans votre devise locale.

Les plans de facturation sont définis par les fournisseurs de matériel quantique. Pour plus d'informations sur la tarification de chaque fournisseur, consultez [Azure Quantum tarification](#).

Comment afficher le rapport de coût du travail après l'exécution d'un travail ?

Après avoir exécuté un travail, vous pouvez vérifier les estimations de coûts détaillées et utiliser ces informations pour comprendre le coût des travaux individuels. Ce coût est le montant facturé par le fournisseur. Reportez-vous à votre facture finale pour connaître les frais exacts, y compris les taxes pertinentes. Pour plus d'informations, consultez [Comment recevoir mes factures ?](#)

Pour passer en revue les coûts du travail, procédez comme suit :

1. Connectez-vous au portail [Azure](#) en utilisant les informations d'identification de votre abonnement Azure.
2. Accédez à votre espace de travail Azure Quantum.
3. Dans le volet de navigation de l'espace de travail, développez la liste déroulante **Opérations** et choisissez **Gestion des travaux**.
4. Affichez les coûts estimés pour chaque travail dans la colonne **Estimation des coûts**.
5. Pour ouvrir le volet **Détails** du travail pour un travail, choisissez le travail dans la colonne **Nom**.
6. Pour afficher des informations détaillées sur l'estimation des coûts, choisissez l'onglet **Estimation des coûts**. Le tableau affiche les dimensions de facturation et les coûts associés pour l'exécution du travail.

Le tableau d'estimation des coûts contient les colonnes suivantes :

- **Dimension** : élément de facturation pour lequel le fournisseur vous facture.
- **Prix unitaire** : coût par unité de la dimension.
- **Unités consommées** : nombre d'unités de la dimension consommée par le travail.
- **Unités facturées** : le nombre d'unités dont on vous facture. Ce nombre peut être inférieur aux unités consommées lorsque, par exemple, le plan de facturation du fournisseur offre une quantité d'utilisation gratuite ou est basé sur des crédits.
- **Coût estimé** : coût estimé pour cette dimension, calculé en tant que $\text{Unités facturées} \times \text{Prix unitaire}$.

❗ Remarque

IonQ a un coût minimal de 1,00 USD pour exécuter un travail sur le QPU IonQ, de sorte que le **Consumed Units** rapport sur la table d'estimation des coûts du travail peut être inférieur à celui des **Billed Units** petits travaux.

Pourquoi ne vois-je pas le coût de mon travail ?

Certains fournisseurs Azure Quantum ne prennent pas en charge les rapports pour les coûts par travail, mais votre facture est toujours disponible dans la page **Cost Management** dans le portail Azure. Pour plus d'informations, consultez [Comment puis-je afficher mes factures ?](#)

Comment puis-je recevoir mes factures ?

Les factures sont envoyées à l'adresse e-mail que vous avez utilisée pour vous inscrire à Azure. Si vous devez modifier l'adresse e-mail, contactez [Azure support](#) [↗](#).

Si vous utilisez un abonnement personnalisé, les factures peuvent être envoyées à l'adresse e-mail de la personne qui a configuré l'abonnement.

Les factures de Azure Quantum fournisseurs tiers ont le type de facturation **Place de marché Azure et réservations**. Vous recevez une seule facture de Place de marché Azure et réservations, qui fournit une répartition des frais de chaque fournisseur quantique.

Que comprend ma facture Azure Quantum ?

Les factures du Place de marché Azure et des réservations incluent les dépenses auprès des fournisseurs tiers d'Azure. Azure Quantum propose actuellement du matériel quantique à partir d'IonQ, Quantinuum et Rigetti. Pour plus d'informations, consultez [Azure Quantum liste des fournisseurs](#).

Les factures Place de marché Azure et réservations n'incluent pas les dépenses par rapport aux services Azure internes, tels que les comptes de stockage attachés à l'espace de travail Azure Quantum. Les frais d'Azure de première partie sont généralement très faibles pour les travaux de QPU.

Pour plus d'informations sur les coûts d'infrastructure Azure, consultez [Stockage Blob Azure tarification](#) [↗](#).

Comment les factures sont-elles facturées ?

Les factures sont facturées en retard. Par exemple, la facture que vous recevez en février correspond à l'utilisation que vous avez engagée en janvier.

Vous payez vos factures avec le même mode de paiement que celui que vous utilisez pour l'abonnement Azure de votre espace de travail Azure Quantum. Vous pouvez avoir différents modes de paiement pour différents abonnements.

Comment puis-je afficher mes factures ?

Pour afficher vos factures passées dans le portail Azure, procédez comme suit :

1. Connectez-vous au portail [Azure](#) avec les informations d'identification de votre abonnement Azure.
2. Dans la barre de recherche supérieure, entrez **Cost Management + Facturation**.
3. Si vous avez plusieurs étendues de facturation, sélectionnez l'étendue que vous souhaitez afficher dans le volet **Étendues de facturation**.
4. Dans le volet de navigation, choisissez **Cost Management**.
5. Dans le volet de navigation, développez la liste déroulante **Facturation** et choisissez **Factures**.
6. Pour afficher les détails de la facture des fournisseurs tiers d'Azure Quantum, choisissez l'ID de la **facture** dans les lignes qui ont **Place de marché Azure et Réservations** dans la colonne **Type**.

Vous pouvez filtrer les factures par date, abonnement et état. Vous pouvez également télécharger les factures sous forme de fichiers PDF. Vous pouvez voir la date de facturation et la période de facturation. La période de facturation n'est pas la même que le mois dans lequel vous recevez la facture.

Contenu connexe

- quotas [Azure Quantum](#)

Comment utiliser un principal de service pour s'authentifier dans votre espace de travail Azure Quantum

Article • 18/06/2024

Il n'est pas toujours pertinent d'utiliser l'authentification interactive ni de s'authentifier en tant que compte d'utilisateur, par exemple pour envoyer des travaux à partir d'un service web, d'un autre rôle de travail ou d'un système automatisé. Une option consiste à configurer une identité managée. Une autre option consiste à utiliser un [principal de service](#), comme l'explique cet article.

Prérequis : Création d'un principal de service et d'un secret d'application

Pour pouvoir vous authentifier en tant que principal de service, vous devez commencer par [créer un principal de service](#).

Pour créer un principal du service, attribuer l'accès et générer des informations d'identification, procédez comme suit :

1. Créer une application Azure AD :

ⓘ Notes

Il n'est pas nécessaire de définir un URI de redirection.

a. Ensuite, écrivez les valeurs *ID d'application (client)* et *ID de répertoire (locataire)*.

2. [Créez des informations d'identification](#) pour vous connecter sous l'identité de l'application :

a. Dans les paramètres de votre application, sélectionnez **Certificats et secrets**.

b. Sous **Clés secrètes client**, sélectionnez **Créer un secret**.

c. Indiquez une description et une durée, puis sélectionnez **Ajouter**.

d. Copiez immédiatement la valeur du secret dans un endroit sûr. Vous ne la verrez plus ensuite.

3. Accordez au principal de service les autorisations nécessaires pour accéder à votre espace de travail :

a. Ouvrez le portail Azure.

- b. Dans la barre de recherche, entrez le nom du groupe de ressources dans lequel vous avez créé votre espace de travail. Sélectionnez le groupe de ressources lorsqu'il apparaît dans les résultats.
- c. Dans la vue d'ensemble du groupe de ressources, sélectionnez **Contrôle d'accès (IAM)**.
- d. Sélectionnez **Ajouter une attribution de rôle**.
- e. Recherchez, puis sélectionnez le principal de service.
- f. Attribuez-lui le rôle **Contributeur** ou **Propriétaire**.

⚠ Notes

Pour créer une attribution de rôle sur le groupe de ressources ou l'espace de travail, vous devez être un *propriétaire* ou un *administrateur de l'accès utilisateur* au niveau de l'attribution de rôle. Si vous ne disposez pas des autorisations nécessaires pour créer le principal de service dans votre abonnement, vous devez demander l'autorisation du *propriétaire* ou de l'*administrateur* de l'abonnement Azure.

Si vous disposez d'autorisations uniquement au niveau du groupe de ressources ou de l'espace de travail, vous pouvez essayer de créer le principal de service dans le rôle de contributeur en utilisant :

```
az ad sp create-for-rbac --role Contributor --scopes  
/subscriptions/<SUBSCRIPTION-ID>
```

Authentification comme principal de service

Option 1 : utilisation de variables d'environnement : [DefaultAzureCredential](#) [🔗] comprend les informations d'identification par défaut utilisées lors de la création de l'objet `Workspace` et tente plusieurs types d'authentification. En premier, vous avez [EnvironmentCredential](#) [🔗]. Passez les informations d'identification du principal de service par le biais des variables d'environnement suivantes :

- **AZURE_TENANT_ID** : ID du locataire du principal de service. Également appelé ID d'annuaire.
- **AZURE_CLIENT_ID** : ID client du principal de service.
- **AZURE_CLIENT_SECRET** : l'un des secrets client du principal de service.

Option 2 : utilisation de ClientSecretCredential : passez [ClientSecretCredential](#) pendant l'instanciation de l'objet `Workspace` ou définissez la propriété `credentials`.

Python

```
from azure.identity import ClientSecretCredential

tenant_id = os.environ["AZURE_TENANT_ID"]
client_id = os.environ["AZURE_CLIENT_ID"]
client_secret = os.environ["AZURE_CLIENT_SECRET"]
credential = ClientSecretCredential(tenant_id=tenant_id,
client_id=client_id, client_secret=client_secret)

workspace.credentials = credential
```

ⓘ Notes

La méthode `workspace.login()` a été dépréciée et n'est plus nécessaire. Lors du premier appel au service, une authentification est tentée à l'aide des informations d'identification passées dans le constructeur `Workspace` ou sa propriété `credentials`. En l'absence d'informations d'identification passées, plusieurs méthodes d'authentification sont tentées par [DefaultAzureCredential](#) .

Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

Comment utiliser une identité managée pour s'authentifier dans votre espace de travail Azure Quantum

Article • 18/06/2024

Dans certains cas, il est préférable de ne pas utiliser l'authentification interactive ni de s'authentifier à l'aide d'un compte d'utilisateur. C'est notamment le cas pour soumettre des travaux à partir d'une machine virtuelle ou d'une application de fonction. Une des options consiste à [s'authentifier à l'aide d'un principal de service](#). Une autre consiste à configurer une identité managée, comme l'explique cet article.

Configurer une identité managée

Une identité managée permet à une application d'accéder à d'autres ressources Azure (comme votre espace de travail Azure Quantum) et de s'authentifier auprès de ces ressources.

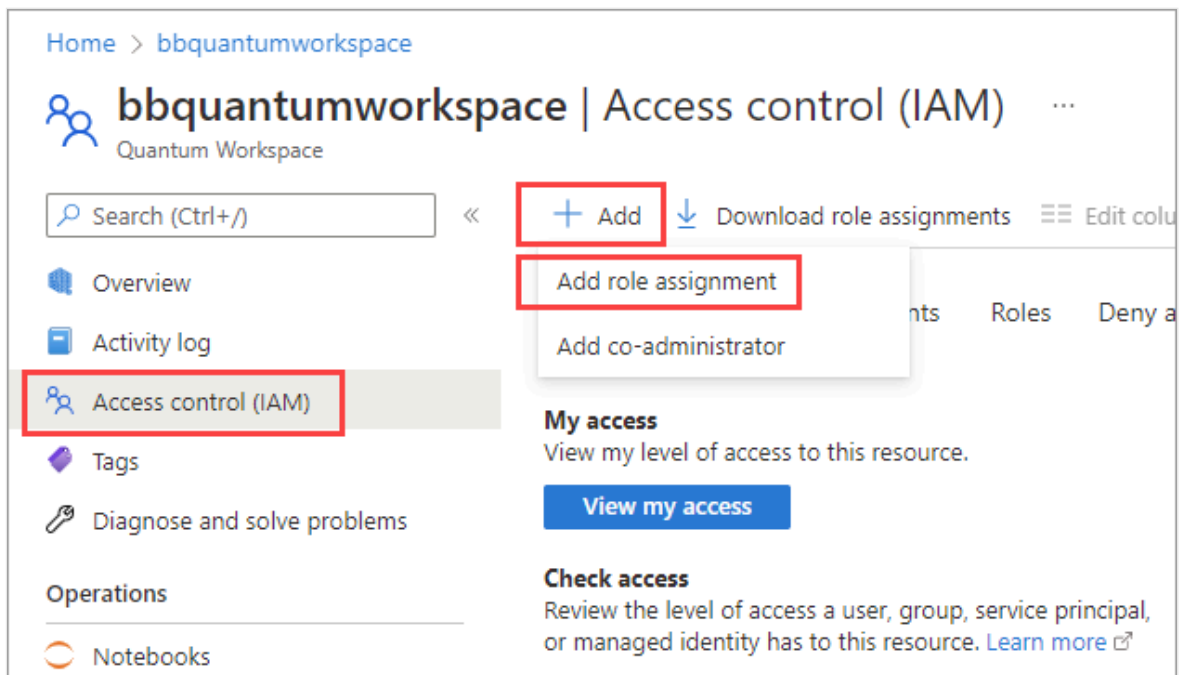
Pour configurer une identité managée :

1. À l'aide du portail Azure, localisez la ressource à laquelle vous souhaitez accorder l'accès. Cette ressource peut être une machine virtuelle, une application de fonction ou une autre application.
2. Sélectionnez la ressource, puis affichez la page de présentation.
3. Sous **Paramètres**, sélectionnez **Identité**.
4. Configurez le paramètre **État** sur **Activé**.
5. Sélectionnez **Enregistrer** pour conserver votre configuration et confirmez ce choix dans la boîte de dialogue qui s'ouvre en sélectionnant **Oui**.

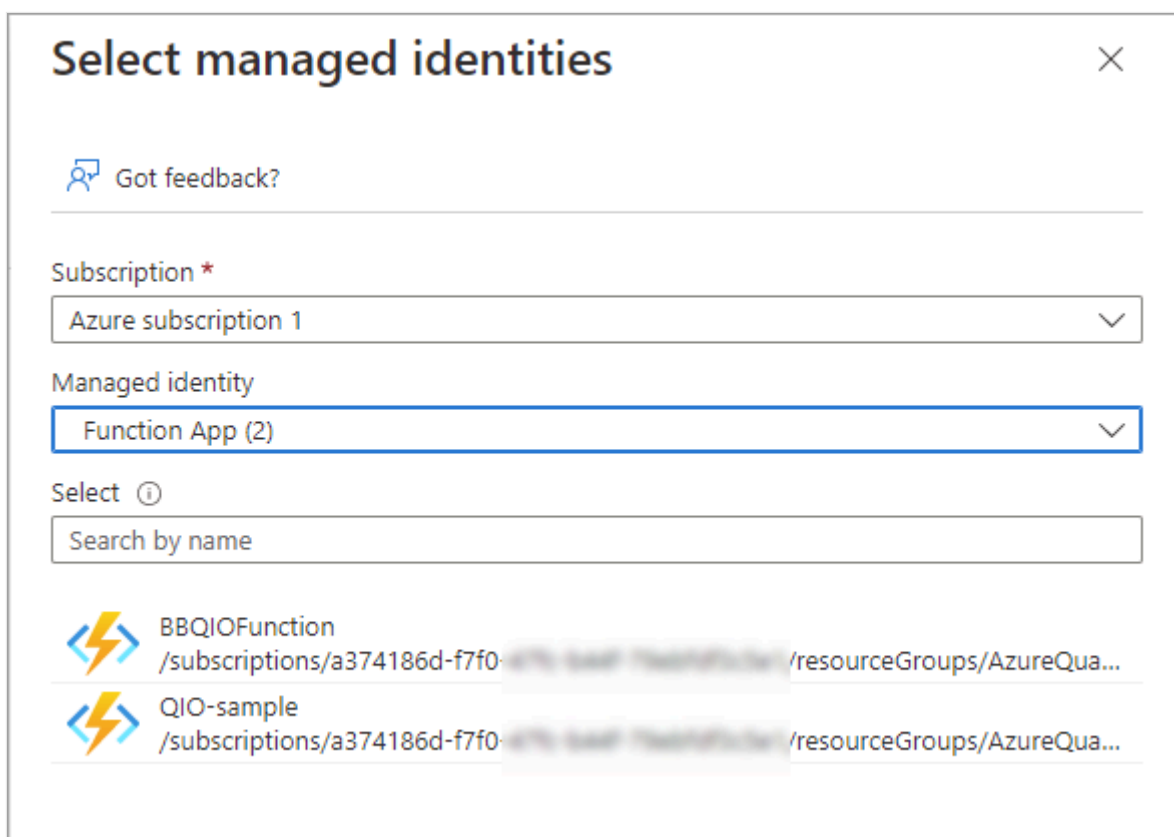
Accorder l'accès à votre espace de travail Azure Quantum

Pour autoriser la ressource à accéder à votre espace de travail Azure Quantum :

1. Accédez à votre espace de travail Azure Quantum, puis sélectionnez **Contrôle d'accès (IAM)** dans le menu situé à gauche.
2. Sélectionnez **Ajouter**, puis **Ajouter une attribution de rôle**.



3. Dans la page **Ajouter une attribution de rôle**, sélectionnez **Contributeur** et **Suivant**.
4. Dans l'onglet **Membres**, sous **Affecter l'accès à**, sélectionnez **Identité managée**, puis **+ Sélectionner des membres**.
5. Dans la **fenêtre contextuelle Sélectionner des identités managées**, sélectionnez une catégorie dans la **liste déroulante Identité managée**.
6. Sélectionnez la ressource souhaitée dans la liste, puis cliquez sur **Sélectionner**.



7. Sélectionnez **Suivant**, puis **Vérifier et attribuer**.

Connexion à votre espace de travail Azure Quantum

Vous devez maintenant être en mesure d'utiliser votre espace de travail Quantum à partir de la ressource que vous avez choisie. Par exemple, pour utiliser votre espace de travail à partir d'une machine virtuelle, vous n'avez plus besoin de vous authentifier à chaque fois.

Dans certains cas, vous pouvez également spécifier explicitement dans le code d'utiliser une identité gérée en guise d'informations d'identification :

Python

```
from azure.identity import ManagedIdentityCredential

from azure.quantum import Workspace
workspace = Workspace (
    resource_id = "",
    location = "" ,
    credential=ManagedIdentityCredential()
)
```

Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

S'authentifier dans votre espace de travail à l'aide d'une clé d'accès

Les clés d'accès sont utilisées pour authentifier et autoriser l'accès à votre espace de travail Azure Quantum. Vous pouvez utiliser des clés d'accès pour vous connecter et accorder l'accès à votre espace de travail à l'aide de chaîne de connexion s.

Dans cet article, vous allez apprendre à activer ou désactiver les clés d'accès pour votre espace de travail Azure Quantum. Vous pouvez également régénérer de nouvelles clés pour garantir la sécurité de votre espace de travail.

Avertissement

Le stockage de vos clés d'accès de compte ou chaîne de connexion en texte clair présente un risque de sécurité et n'est pas recommandé. Stockez vos clés de compte dans un format chiffré ou migrez vos applications pour utiliser l'autorisation Microsoft Entra pour accéder à votre espace de travail Azure Quantum.

Prérequis

- Compte Azure avec un abonnement actif. Si vous n'avez pas de compte Azure, inscrivez-vous gratuitement et inscrivez-vous à un [abonnement](#) de paiement à l'utilisation.
- Un espace de travail Azure Quantum. Consultez [Créer un espace de travail Azure Quantum](#).
- La dernière version de la `qdk` bibliothèque Python avec l'option `azure` en supplément.

Bash

```
pip install --upgrade "qdk[azure]"
```

Si vous utilisez Azure CLI, vous devez disposer de la dernière version. Pour obtenir des instructions d'installation, consultez :

- [Installer Azure CLI sur Windows](#)
- [Installer Azure CLI sur Linux](#)
- [Installer Azure CLI sur macOS](#)

Connectez-vous à votre espace de travail Azure Quantum avec un chaîne de connexion

Le `qdk.azure` module fournit une [Workspace classe](#) qui représente un espace de travail Azure Quantum. Pour vous connecter à votre espace de travail Azure Quantum, créez `Workspace` un objet à l'aide de la chaîne de connexion en tant qu'authentificateur. Pour plus d'informations, consultez [comment copier un chaîne de connexion](#).

Lorsque vous créez un `Workspace` objet, vous avez deux options pour identifier votre espace de travail Azure Quantum.

- Créez un `Workspace` objet en appelant `from_connection_string`.

Python

```
# Creating a new Workspace object from a connection string
from qdk.azure import Workspace

connection_string = "[Copy connection string]"
workspace = Workspace.from_connection_string(connection_string)

print(workspace.get_targets())
```

- Si vous ne souhaitez pas copier votre chaîne de connexion dans le code, stockez votre chaîne de connexion dans une variable d'environnement et utilisez `Workspace`.

Python

```
# Using environment variable to connect with connection string

connection_string = "[Copy connection string]"

import os

os.environ["AZURE_QUANTUM_CONNECTION_STRING"] = connection_string

from qdk.azure import Workspace

workspace = Workspace()
print(workspace.get_targets())
```

Gérer vos clés d'accès et vos chaîne de connexion

 **Conseil**

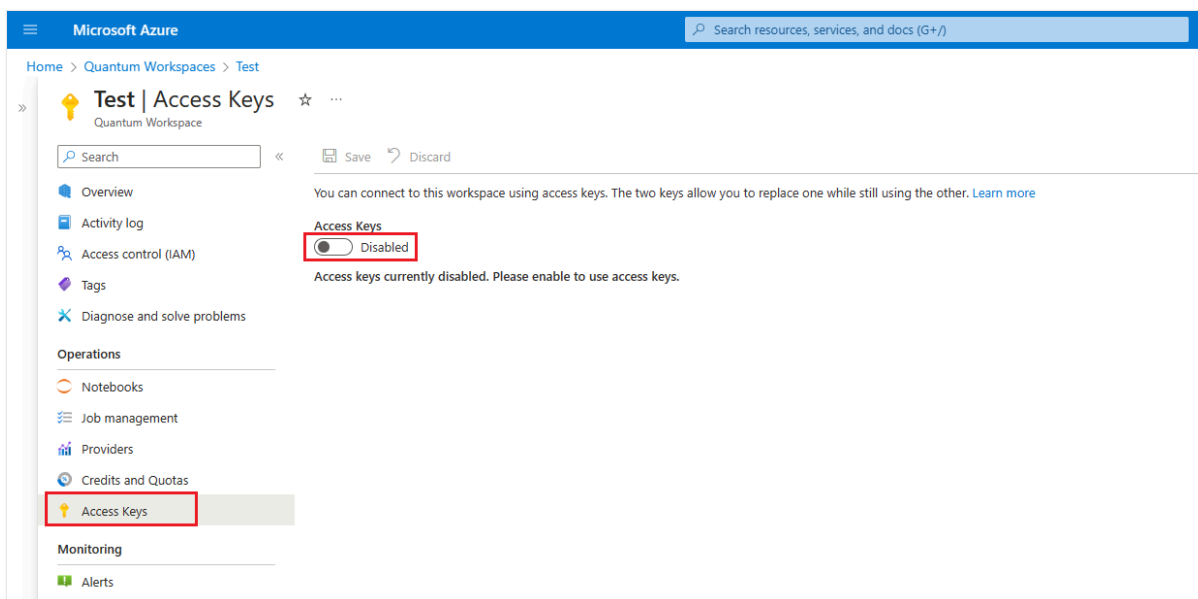
Chaque espace de travail Azure Quantum possède **des clés primaires et secondaires**, ainsi que leurs chaîne de connexion correspondantes. Si vous souhaitez autoriser l'accès à votre espace de travail à d'autres personnes, vous pouvez partager votre clé secondaire et utiliser votre principal pour vos propres services. De cette façon, vous pouvez remplacer la clé secondaire si nécessaire sans avoir de temps d'arrêt dans vos propres services. Pour plus d'informations sur le partage de votre accès à l'espace de travail, consultez [Partager l'accès à](#) votre espace de travail.

portail Azure

Vous pouvez gérer les clés d'accès et les chaîne de connexion de votre espace de travail Azure Quantum dans le Portail Azure.

Activer et désactiver les clés d'accès

1. Connectez-vous au [Portail Azure](#) et sélectionnez votre espace de travail Quantum Azure.
2. Dans le volet gauche, accédez aux **touches d'accès aux opérations**.
3. Basculez le bouton bascule sous Clés d'accès pour **activé** ou **désactivé**.
4. Cliquez sur **Enregistrer** pour enregistrer les modifications.



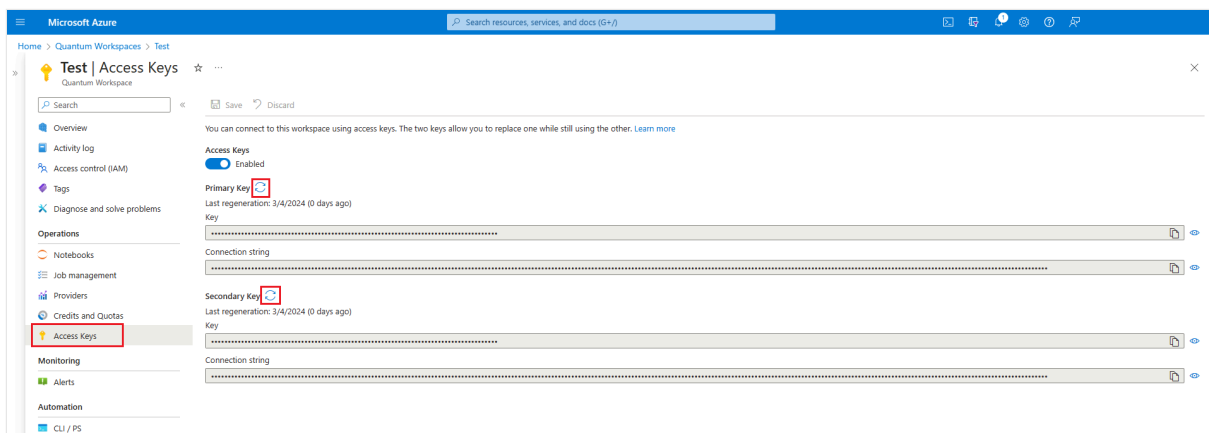
Important

Lorsque les clés d'accès sont **désactivées**, toutes les requêtes utilisant des chaîne de connexion ou des clés d'accès ne sont pas autorisées. Vous pouvez toujours utiliser les paramètres de l'espace de travail pour vous connecter à votre espace de travail.

Régénérer de nouvelles clés d'accès

Si vous pensez que vos clés d'accès ont été compromises ou que vous souhaitez arrêter de partager l'accès à votre espace de travail avec d'autres personnes, vous pouvez régénérer les clés d'accès primaires ou secondaires, ou les deux, pour garantir la sécurité de votre espace de travail.

1. Connectez-vous au [Portail Azure](#) et sélectionnez votre espace de travail Quantum Azure.
2. Dans le volet gauche, accédez aux **touches d'accès aux opérations**.
3. **Les clés d'accès** doivent être activées pour régénérer de nouvelles clés. Si les clés d'accès sont désactivées, vous devez d'abord les activer.
4. Cliquez sur l'icône de flèche circulaire pour régénérer la clé primaire ou secondaire.



Protéger les ressources Azure Quantum avec des verrous Azure Resource Manager (ARM)

Microsoft recommande de verrouiller tous vos espaces de travail Azure Quantum et comptes de stockage liés avec un verrou de ressource Azure Resource Manager (ARM) pour empêcher la suppression accidentelle ou malveillante. Par exemple, les professeurs peuvent vouloir empêcher les étudiants de modifier des références SKU de fournisseur, mais ils peuvent toujours soumettre des travaux.

Il existe deux types de verrous de ressources ARM :

- Un verrou **CannotDelete** empêche les utilisateurs de supprimer une ressource, mais autorise la lecture et la modification de sa configuration.
- Un verrou **ReadOnly** empêche les utilisateurs de modifier la configuration d'une ressource (y compris sa suppression), mais autorise la lecture de sa configuration. Pour plus d'informations sur les verrous de ressources, consultez [Verrouiller les ressources pour empêcher les modifications inattendues](#).

⚠ Notes

Si vous utilisez déjà un [modèle ARM ou Bicep pour gérer vos espaces de travail Azure Quantum](#), vous pouvez ajouter les procédures décrites dans cet article à vos modèles existants.

Configurations de verrou recommandées

Le tableau suivant présente les configurations de verrou de ressources recommandées à déployer pour un espace de travail Azure Quantum.

[🔗](#) Agrandir le tableau

Resource	Type de verrou	Remarques
Workspace	Supprimer	Empêche la suppression de l'espace de travail.
Workspace	Read-only	Empêche les modifications apportées à l'espace de travail, y compris les ajouts ou suppressions de fournisseurs, tout en permettant aux utilisateurs d'envoyer des travaux. Pour modifier les fournisseurs lorsque ce verrou est

Resource	Type de verrou	Remarques
		défini, vous devez supprimer le verrou de ressource , apporter vos modifications, puis redéployer le verrou.
Compte de stockage	Supprimer	Empêche la suppression du compte de stockage.

Les configurations suivantes doivent être évitées :

Important

Si vous définissez les verrous ARM suivants, votre espace de travail peut fonctionner de manière incorrecte.

 Agrandir le tableau

Resource	Type de verrou	Remarques
Compte de stockage	Read-only	La définition d'un verrou de ressource en lecture seule sur le compte de stockage peut entraîner des échecs lors de la création de l'espace de travail, de la soumission de travaux et de l'extraction de travaux.
Abonnement parent de l'espace de travail ou du groupe de ressources parent de l'espace de travail ou du compte de stockage	Read-only	Lorsqu'un verrou de ressource est appliqué à une ressource parente, toutes les ressources sous ce parent héritent du même verrou, y compris les ressources créées à une date ultérieure. Pour un contrôle plus précis, les verrous de ressources doivent être appliqués directement au niveau de la ressource.

Prerequisites

Vous devez être **propriétaire** ou **administrateur d'accès utilisateur** d'une ressource pour appliquer des verrous de ressources ARM. Pour plus d'informations, voir [Rôles intégrés Azure](#).

Déploiement en ligne de commande

Vous aurez besoin d'Azure PowerShell ou d'Azure CLI pour déployer le verrou. Si vous utilisez Azure CLI, vous devez disposer de la dernière version. Pour les instructions d'installation, consultez :

- Installer Azure PowerShell
- Installation de l'interface de ligne de commande Azure

Important

Si vous n'avez pas déjà utilisé Azure CLI avec Azure Quantum, suivez les étapes décrites dans la section [Configuration de l'environnement](#) pour ajouter l'extension `quantum` et inscrire l'espace de noms Azure Quantum.

Connexion à Azure

Après avoir installé Azure CLI ou Azure PowerShell, veuillez à vous connecter pour la première fois. Choisissez l'un des onglets suivants et exécutez les commandes de ligne de commande correspondantes pour vous connecter à Azure :

Azure CLI

Azure CLI

```
az login
```

Si vous avez plusieurs abonnements Azure, sélectionnez l'abonnement avec les ressources que vous souhaitez verrouiller. Remplacez `SubscriptionName` par votre nom d'abonnement ou votre ID d'abonnement. Par exemple,

Azure CLI

```
az account set --subscription "Azure subscription 1"
```

Créer un verrou de ressource ARM

Lorsque vous déployez un verrou de ressource, vous spécifiez un nom pour le verrou, le type de verrou et des informations supplémentaires sur la ressource. Ces informations peuvent être copiées et collées à partir de la page d'accueil de la ressource dans le portail Azure Quantum.

Azure CLI

Azure CLI

```
az lock create \  
  --name <lock> \  
  --resource-group <resource-group> \  
  --resource <workspace> \  
  --lock-type CanNotDelete \  
  --resource-type Microsoft.Quantum/workspaces
```

- nom : nom descriptif du verrou
- groupe de ressources : nom du groupe de ressources parent.
- ressource : nom de la ressource à laquelle appliquer le verrou.
- type de verrou : type de verrou à appliquer, **CanNotDelete** ou **ReadOnly**.
- type de ressource : type de la target ressource.

Par exemple, pour créer un verrou **CanNotDelete** sur un espace de travail :

Azure CLI

```
az lock create \  
  --name ArmLockWkspDelete \  
  --resource-group armlocks-resgrp \  
  --resource armlocks-wksp \  
  --lock-type CanNotDelete \  
  --resource-type Microsoft.Quantum/workspaces
```

Si elle réussit, Azure retourne la configuration de verrou au format JSON :

JSON

```
{  
  "id": "/subscriptions/<ID>/resourcegroups/armlocks-resgrp/providers/Microsoft.Quantum/workspaces/armlocks-wksp/providers/Microsoft.Authorization/locks/ArmLockWkspDelete",  
  "level": "CanNotDelete",  
  "name": "ArmLockWkspDelete",  
  "notes": null,  
  "owners": null,  
  "resourceGroup": "armlocks-resgrp",  
  "type": "Microsoft.Authorization/locks"  
}
```

Pour créer un verrou **ReadOnly** sur un espace de travail :

Azure CLI

```
az lock create \  
  --name ArmLockWkspRead \  
  --resource-group armlocks-resgrp \  
  --resource armlocks-wksp \  
  --lock-type ReadOnly
```

```
--lock-type ReadOnly \  
--resource-type Microsoft.Quantum/workspaces
```

JSON

```
{  
  "id": "/subscriptions/<ID>/resourcegroups/armlocks-  
resgrp/providers/Microsoft.Quantum/workspaces/armlocks-  
wksp/providers/Microsoft.Authorization/locks/ArmLockWkspRead",  
  "level": "ReadOnly",  
  "name": "ArmLockWkspRead",  
  "notes": null,  
  "owners": null,  
  "resourceGroup": "armlocks-resgrp",  
  "type": "Microsoft.Authorization/locks"  
}
```

Pour créer un verrou **CanNotDelete** sur un compte de stockage :

Azure CLI

```
az lock create \  
  --name ArmLockStoreDelete \  
  --resource-group armlocks-resgrp \  
  --resource armlocksstorage --lock-type CanNotDelete \  
  --resource-type Microsoft.Storage/storageAccounts
```

JSON

```
{  
  "id": "/subscriptions/<ID>/resourcegroups/armlocks-  
resgrp/providers/Microsoft.Storage/storageAccounts/armlocksstorage/providers/Mic  
rosoft.Authorization/locks/ArmLockStoreDelete",  
  "level": "CanNotDelete",  
  "name": "ArmLockStoreDelete",  
  "notes": null,  
  "owners": null,  
  "resourceGroup": "armlocks-resgrp",  
  "type": "Microsoft.Authorization/locks"  
}
```

Affichage et suppression de verrous

Pour afficher ou supprimer des verrous :

Azure CLI

Pour plus d'informations, consultez la [référence az lock](#).

Afficher tous les verrous dans un abonnement

Azure CLI

```
az lock list
```

Afficher tous les verrous dans un espace de travail

Azure CLI

```
az lock list \  
  --resource-group armlocks-resgrp \  
  --resource-name armlocks-wksp \  
  --resource-type Microsoft.Quantum/workspaces
```

Afficher tous les verrous pour toutes les ressources d'un groupe de ressources

Azure CLI

```
az lock list --resource-group armlocks-resgrp
```

Afficher les propriétés d'un seul verrou

azcli

```
az lock show \  
  --name ArmLockStoreDelete \  
  --resource-group armlocks-resgrp \  
  --resource-name armlocksstorage \  
  --resource-type Microsoft.Storage/storageAccounts
```

Supprimer un verrou

azcli

```
az lock delete \  
  --name ArmLockStoreDelete \  
  --resource-group armlocks-resgrp \  
  --resource-name armlocksstorage \  
  --resource-type Microsoft.Storage/storageAccounts
```

Si la suppression réussit, Azure ne retourne pas de message. Pour vérifier la suppression, vous pouvez exécuter `az lock list`.

Étapes suivantes

- [Gérer des espaces de travail quantiques avec Azure CLI](#)
 - [Gérer des espaces de travail quantiques avec Azure Resource Manager](#)
-

Last updated on 17/10/2025

Mettre à jour Microsoft Quantum Development Kit vers la version la plus récente

Découvrez comment mettre à jour le Microsoft Quantum Development Kit (QDK) vers la dernière version.

Prérequis

- Cet article part du principe que l'extension QDK est déjà installée dans Visual Studio Code (VS Code). Si vous installez pour la première fois, reportez-vous à [Installer l'extension QDK](#).

Mettre à jour l'extension VS Code

Par défaut, VS Code met automatiquement à jour les extensions. Une fois qu'une extension est mise à jour, vous êtes invité à recharger VS Code. Pour désactiver les mises à jour automatiques et mettre à jour vos extensions manuellement, consultez la [mise à jour automatique de l'extension](#) dans la documentation VS Code.

Mettre à jour les QDK bibliothèques Python

ⓘ Important

Si vous effectuez une mise à jour à partir d'un Qiskit environnement précédent, consultez [Mettre à jour le module qdk.azure avec le support de Qiskit dans un environnement Python virtuel \(recommandé\)](#).

1. Mettez à jour vers la dernière version du paquet Python `qdk` avec l'extra `azure` :

Bash

```
pip install --upgrade "qdk[azure]"
```

2. Pour prendre en charge l'analyse, la transformation, la génération de code et la simulation de circuits Qiskit, installez les extras `qiskit` et `jupyter`.

Bash

```
pip install --upgrade "qdk[qiskit,jupyter]"
```

⚠ Remarque

Le QDK prend en charge les versions 1 et 2 de Qiskit.

Mettre à jour le module `qdk.azure` avec la prise en charge de Qiskit dans un environnement Python virtuel (recommandé)

Les modules Python `qdk.azure` et `qdk.qiskit` comprennent une prise en charge facultative pour créer et soumettre des circuits Qiskit à Azure Quantum. Lorsque vous installez les modules `qdk.azure` et `qdk.qiskit`, la dernière version de Qiskit est installée par défaut, ce qui peut entraîner des problèmes avec un environnement Qiskit existant. Pour garantir un environnement de développement stable, créez un environnement Python virtuel et installez-le `qdk.azure` et `qdk.qiskit` dans l'environnement virtuel.

⚠ Remarque

Si vous utilisez Qiskit la version 1, consultez les [Qiskit modifications apportées à l'empaquetage 1.0](#) pour plus d'informations sur la compatibilité des packages.

Pour créer un environnement Python virtuel et l'installer `qdk.azure`, `qdk.qiskit` procédez comme suit :

1. Créez un dossier local, par exemple `~/qiskit10-env`.
2. Exécutez `venv` avec le chemin d'accès au dossier.

Bash

```
python -m venv ~/qiskit10-env
```

3. Activez l'environnement.

Bash

```
~/qiskit10-env/bin/activate
```

4. Exécutez `pip list` pour voir que seuls les packages principaux sont installés dans le nouvel environnement.
5. Pour installer les modules `qdk.azure` et `qdk.qiskit`, exécutez la commande suivante :

Bash

```
pip install "qdk[azure,qiskit]"
```

6. Installez d'autres packages que vous avez utilisés dans votre environnement précédent en fonction des besoins. Exécutez la `pip list` commande dans chaque environnement pour comparer les packages et les versions.

❗ Remarque

Vous pouvez également ouvrir votre environnement virtuel dans VS Code. Dans le menu **Affichage**, choisissez **Palette de commandes**, puis entrez **Python : Créer un environnement** et choisir **venv**. Ensuite, choisissez **Ouvrir le dossier...** et sélectionnez le dossier d'environnement que vous avez créé précédemment. Pour plus d'informations sur l'utilisation d'environnements dans VS Code, consultez [les environnements Python dans VS Code](#).

Mettre à jour les modules `qdk.azure` et `qdk.qiskit` dans l'environnement actuel

Vous pouvez également mettre à jour les modules `qdk.azure` et `qdk.qiskit` sans utiliser d'environnement virtuel. Toutefois, les mises à jour des `qiskit` packages dans un environnement existant peuvent entraîner des conflits de dépendances avec d'autres packages. Pour plus d'informations sur la compatibilité des packages, consultez [Qiskit les changements de packaging 1.0](#).

Pour mettre à jour les modules `qdk.azure` et `qdk.qiskit` dans votre environnement actuel, procédez comme suit :

1. Désinstallez les modules existants `qdk` et d'autres Qiskit dépendances :

Bash

```
pip uninstall -y "qdk[azure,qiskit]" qiskit qiskit-terra qiskit-qir
```

2. Réinstallez le `qdk` package avec l'élément `qiskit` supplémentaire :

Bash

```
pip install "qdk[qiskit]"
```

Mettre à jour l'extension Azure CLI `quantum`

Mettez à jour ou installez la dernière Azure extension CLI `quantum` .

1. Ouvrez une invite de commandes Windows.
2. À partir de l'invite de commandes, exécutez la commande suivante :

Azure CLI

```
az extension add --upgrade -n quantum
```

Last updated on 14/08/2026

Ajouter ou supprimer un fournisseur dans un espace de travail existant

Lorsque vous créez un espace de travail Azure Quantum, vous choisissez les fournisseurs et les plans disponibles dans l'espace de travail. Toutefois, vous pouvez ajouter de nouveaux fournisseurs ou supprimer des fournisseurs existants à tout moment. Si vous envoyez un travail quantique à un fournisseur qui n'est pas dans votre espace de travail, vous recevez un message d'erreur qui vous invite à ajouter ce fournisseur à votre espace de travail.

Ajouter un fournisseur ou un plan

Pour ajouter un fournisseur à votre espace de travail, procédez comme suit :

1. Connectez-vous au portail [Azure](#) avec les informations d'identification de votre abonnement Azure.
2. Accédez à votre espace de travail Azure Quantum.
3. Dans le volet de navigation de l'espace de travail, développez la liste déroulante **Opérations** et choisissez **Fournisseurs**.
4. Au-dessus de la liste de vos fournisseurs existants, choisissez + **Ajouter un fournisseur**. Le volet **Ajouter des fournisseurs supplémentaires** s'ouvre et affiche tous les fournisseurs que vous pouvez ajouter à votre espace de travail.

ⓘ Remarque

Si le fournisseur que vous souhaitez ajouter ne figure pas dans la liste des fournisseurs disponibles, votre compte de facturation peut se trouver dans un pays ou une région que le fournisseur ne prend pas en charge. Pour plus d'informations, consultez [Disponibilité globale des fournisseurs Azure Quantum](#).

5. Choisissez + **Ajouter** en regard du fournisseur que vous souhaitez ajouter, puis choisissez un plan tarifaire.
6. Sélectionnez la zone **Conditions générales de ce fournisseur**, puis choisissez le bouton **Ajouter** pour commencer le déploiement.

Une fois le déploiement terminé, le fournisseur figure dans la liste sur la page **Fournisseurs**. Si le déploiement réussit, vous verrez une coche verte pour le **statut** du fournisseur. Si le déploiement échoue, passez en revue la notification ou vérifiez le **journal d'activité**.

Supprimer un fournisseur ou un plan

Pour supprimer un fournisseur de votre espace de travail, procédez comme suit :

1. Connectez-vous au portail [Azure](#) avec les informations d'identification de votre abonnement Azure.
2. Accédez à votre espace de travail Azure Quantum.
3. Dans le volet de navigation de l'espace de travail, développez la liste déroulante **Opérations** et choisissez **Fournisseurs**.
4. Pour le fournisseur que vous souhaitez supprimer, choisissez **Supprimer** dans la dernière colonne de la liste des fournisseurs.
5. Choisissez le bouton **Oui** pour supprimer le fournisseur.

Last updated on 13/04/2026

Prise en main de la conception de l'architecture DevOps

DevOps intègre le développement, l'assurance qualité (QA) et les opérations informatiques dans une culture unifiée et un ensemble de processus pour la livraison de logiciels. En adoptant des pratiques DevOps, les équipes peuvent fournir des solutions de qualité supérieure plus rapidement et répondre aux besoins de l'entreprise changeant avec une plus grande agilité.

services Azure pour DevOps

Azure fournit une gamme d'opérations et de services pour DevOps.

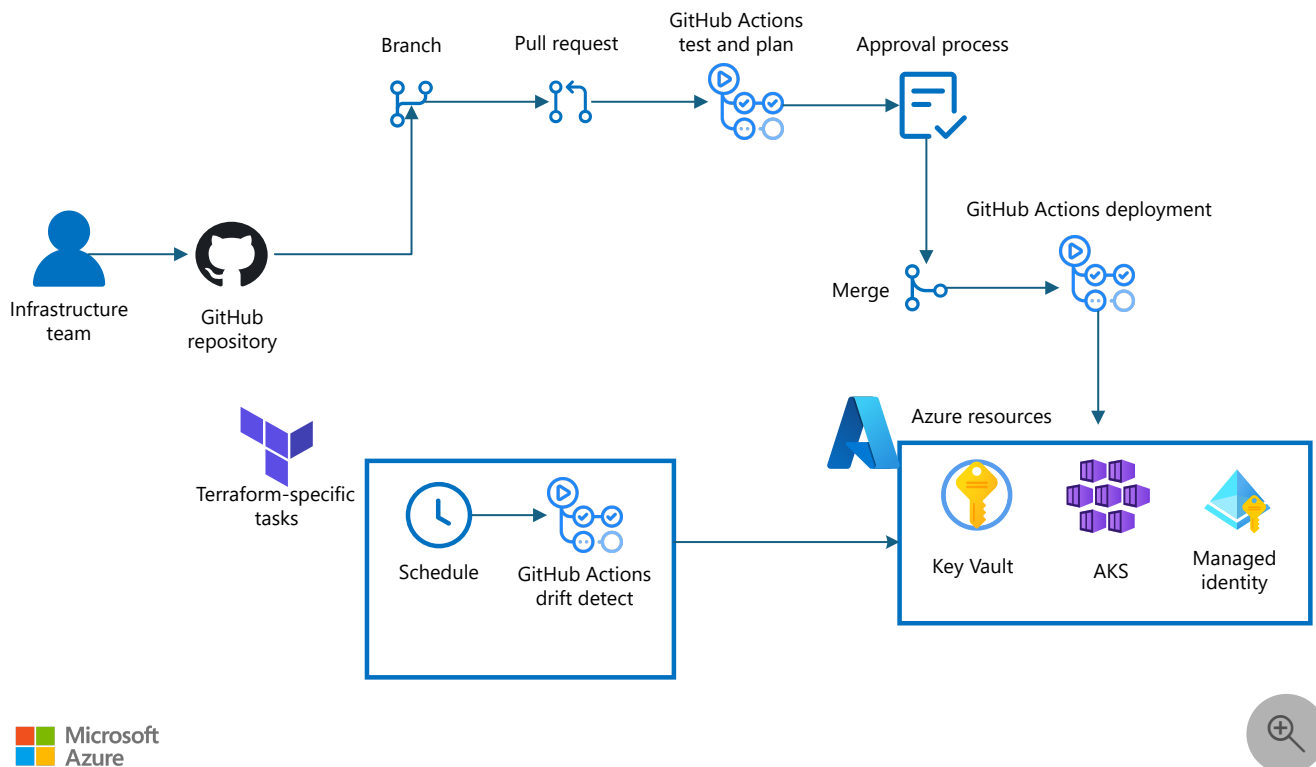
Opérations DevOps

- **Intégration continue (CI)** : pratique de fusionner fréquemment tout le code du développeur en base de code central et d'effectuer des processus de génération et de test automatisés. Les objectifs sont de détecter et corriger rapidement les problèmes de code, de simplifier le déploiement et de garantir la qualité du code.
- **Livraison continue (CD)** : pratique de créer, tester et déployer automatiquement du code dans des environnements de type production. L'objectif est de faire en sorte que le code soit toujours prêt à être déployé. L'ajout de CD pour créer un pipeline CI/CD complet vous permet de détecter les défauts de code dès que possible. Cela garantit également que des mises à jour correctement testées peuvent être publiées dans un délai court.
- **Déploiement continu** : processus qui prend automatiquement toutes les mises à jour qui passent par le pipeline CI/CD et les déploie en production. Le déploiement continu nécessite des tests automatiques robustes et une planification avancée du processus. Il peut ne pas convenir à toutes les équipes.
- **Supervision continue** : processus et technologie nécessaires pour intégrer la supervision à chaque phase du cycle de vie des opérations informatiques et DevOps. La supervision permet de garantir l'intégrité, les performances et la fiabilité de votre application et de votre infrastructure à mesure que l'application passe du développement à la production. La supervision continue repose sur les concepts de CI et de CD.

Services DevOps

- [Azure DevOps](#) : planifiez, développez et fournissez des logiciels plus rapidement avec les services DevOps intégrés, notamment le contrôle de version, les pipelines CI/CD et la gestion des packages.
- [Azure Pipelines](#) : Automatisez les processus de génération, de test et de déploiement avec des pipelines CI/CD hébergés dans le cloud pour n'importe quelle plateforme ou langage.
- [GitHub Actions](#) [↗](#) : Automatisez les flux de travail logiciels directement à partir de votre référentiel GitHub avec des fonctionnalités natives ci/CD, de test et de déploiement.
- [Azure Monitor](#) : surveillez, analysez et optimisez les performances de l'application et de l'infrastructure avec une observabilité et des diagnostics complets.

Architecture



Téléchargez un [fichier Visio](#) [↗](#) de cette architecture.

Le diagramme précédent illustre une implémentation DevOps de base ou de base classique. Pour obtenir des solutions réelles que vous pouvez créer dans Azure, consultez [les architectures DevOps](#).

Explorer les guides, les architectures et les idées de solution DevOps

Les articles de cette section incluent des guides et des architectures entièrement développées que vous pouvez déployer dans Azure et développer vers des solutions de niveau production. Les idées de solution illustrent des modèles d'implémentation et des possibilités à prendre en compte lorsque vous planifiez votre développement de preuve de concept (POC) DevOps. Ces articles peuvent vous aider à décider comment utiliser les technologies DevOps dans Azure.

Idées de solution DevOps

Les idées de solution DevOps suivantes illustrent des modèles d'implémentation et des possibilités à explorer :

- [DevSecOps pour l'infrastructure en tant que code \(IaC\)](#) : implémentez un pipeline DevSecOps à l'aide de GitHub pour IaC avec une gouvernance pour l'excellence opérationnelle, la sécurité et l'optimisation des coûts.

Architectures DevOps

Les architectures prêtes pour la production suivantes illustrent les solutions DevOps de bout en bout que vous pouvez déployer et personnaliser.

- [Architecture de référence CI/CD avec Azure Pipelines](#) : créez un pipeline CI/CD à l'aide de Azure Pipelines pour déployer des modifications d'application dans des environnements intermédiaires et de production.
- [Automatisez les déploiements d'API avec APIOps](#) : appliquez des techniques GitOps et DevOps au déploiement d'API à l'aide de Gestion des API Azure et de Azure DevOps.
- [Applications cloud évolutives et ingénierie de fiabilité de site \(SRE\)](#) : découvrez comment modéliser les performances des applications et appliquer les pratiques SRE pour mettre à l'échelle les applications cloud de manière fiable.
- [Gérez Microsoft 365 configuration de locataire à l'aide de Microsoft365DSC et de Azure DevOps](#) : suivez et automatisez les modifications apportées aux configurations de locataire Microsoft 365 à l'aide de Azure DevOps et de PowerShell DSC.

Les guides DevOps

- [Azure bac à sable](#) : découvrez comment utiliser Azure bac à sable pour fournir des environnements de Azure isolés et en libre-service pour le développement et le test.

- [DevSecOps sur Azure Kubernetes Service \(AKS\)](#) : incorporer des contrôles de sécurité et des bonnes pratiques dans les phases de cycle de vie devOps pour les charges de travail hébergées Azure Kubernetes Service (AKS).
- [Utilisez des scripts de déploiement pour vérifier les propriétés des ressources](#) : découvrez comment utiliser des scripts de déploiement pour valider Azure propriétés de ressource dans le cadre de votre pipeline DevOps.

Préparation de l'organisation

Les organisations au début du processus d'adoption du cloud peuvent utiliser le [Cloud Adoption Framework pour Azure](#) pour accéder à des conseils éprouvés qui accélèrent l'adoption du cloud.

- [Considérations relatives à DevOps pour Azure zones d'atterrissage](#) : définissez votre framework DevOps, établissez des métriques, planifiez votre parcours d'implémentation et sélectionnez votre chaîne d'outils DevOps.

Pour garantir la qualité de votre solution DevOps sur Azure, suivez les instructions de [Azure Well-Architected Framework](#). Le framework Well-Architected fournit des conseils prescriptifs pour les organisations qui recherchent l'excellence architecturale et expliquent comment concevoir, provisionner et surveiller des solutions Azure optimisées pour les coûts.

Pratiques recommandées

Suivez ces bonnes pratiques pour améliorer la sécurité, la fiabilité, les performances et la qualité opérationnelle de vos charges de travail DevOps sur Azure.

DevOps

- [Sécurisez votre Azure DevOps](#) : conseils de sécurité complets pour Azure DevOps, notamment la protection réseau, l'implémentation Confiance nulle, le contrôle d'accès et les mesures de sécurité au niveau du service.
- [Passez à gauche pour rendre les tests rapides et fiables](#) : conseils pour le changement de test plus tôt dans le cycle de développement, y compris les pyramides de test, les boucles de rétroaction rapides et les pratiques de test fiables.
- [Déplacer les tests vers la production](#) : recommandations sur les pratiques de test en production, notamment les mises en production Canary, les indicateurs de fonctionnalité et l'injection de pannes.

- [Migrer de Team Foundation Version Control \(TFVC\) vers Git](#) : Conseils pour la migration du contrôle de code source de Team Foundation Version Control vers des référentiels Git dans Azure DevOps.
- [Guides de fiabilité par service](#) : conseils spécifiques au service pour la création de solutions fiables sur Azure.
- [Surveillez votre patrimoine cloud Azure](#) : planifiez, configurez et optimisez la surveillance dans les environnements Azure à l'aide de Azure Monitor, de Microsoft Defender for Cloud, de Microsoft Sentinel et d'autres outils.
- [Meilleures pratiques en matière de fiabilité dans Azure Monitor](#) : Conseils pour la surveillance de la fiabilité des applications à l'aide de Azure Monitor.
- [Vue d'ensemble du benchmark de sécurité cloud Microsoft](#) : recommandations pour la sécurisation des solutions cloud sur Azure, alignées sur les infrastructures du secteur telles que NIST et PCI.
- [Sécurité DevOps](#) : contrôles de sécurité pour les processus DevOps, notamment les tests statiques de sécurité des applications, la gestion des vulnérabilités, la modélisation des menaces et la sécurité de la chaîne logistique logicielle.
- [Azure meilleures pratiques en matière de sécurité de gestion des identités et de contrôle d'accès](#) : meilleures pratiques pour implémenter la gestion des identités et le contrôle d'accès à l'aide de Microsoft Entra ID.
- [Azure meilleures pratiques et modèles de sécurité](#) : collection de bonnes pratiques de sécurité pour la conception, le déploiement et la gestion des solutions cloud sur Azure.
- [Azure liste de contrôle de sécurité opérationnelle](#) : liste de contrôle qui couvre les pratiques de sécurité opérationnelle pour les déploiements Azure.
- [Meilleures pratiques de développement sécurisées sur Azure](#) : meilleures pratiques pour la création d'applications sécurisées sur Azure, y compris le cycle de vie du développement de la sécurité.

Azure Artifacts

- [Meilleures pratiques pour Azure Artifacts](#) : recommandations en matière de création, de publication et de consommation de packages dans Azure Artifacts, notamment la gestion des flux et les stratégies de rétention.

Azure Resource Manager

- [bonnes pratiques de modèle \(modèle ARM\) Azure Resource Manager](#) : pratiques recommandées pour la construction de modèles Azure Resource Manager (modèles ARM), notamment la conception de paramètres, les dépendances de ressources et le contrôle de version d'API.
- [Meilleures pratiques pour Bicep](#) : recommandations pour le développement de fichiers Bicep, notamment les conventions d'affectation de noms, les définitions de ressources et la sécurité de sortie.

Se tenir informé des nouveautés de DevOps

Azure DevOps services évoluent pour relever les défis modernes liés au développement et aux opérations. Restez informé des dernières [mises à jour et fonctionnalités](#) [↗].

Pour rester à jour avec les principaux services DevOps, consultez les articles suivants :

- [feuille de route Azure DevOps](#)
- documentation [Azure DevOps - nouveautés ?](#)
- [Nouveautés dans la documentation Azure Monitor](#)

Autres ressources

DevOps est une catégorie étendue et couvre une gamme de solutions. Les ressources suivantes peuvent vous aider à en savoir plus sur les services Azure qui prennent en charge les implémentations DevOps.

Microsoft Fabric CI/CD

Dans Microsoft Fabric, vous mettez en place CI/CD en connectant votre espace de travail Fabric à un dépôt Git (Azure DevOps ou GitHub) pour le contrôle de version et les flux de travail basés sur les branches. Vous gérez le déploiement continu à l'aide de pipelines de déploiement.

- [Intégration Git de Microsoft Fabric](#) : Connectez votre espace de travail Fabric à Azure DevOps ou GitHub pour la gestion des versions et l'intégration continue (CI).
- [Introduction aux pipelines de déploiement](#) : promouvoir le contenu dans les environnements de développement, de test et de production à l'aide de pipelines de déploiement.

Professionnels Amazon Web Services (AWS) ou Google Cloud

Pour vous aider à commencer rapidement, les articles suivants comparent Azure DevOps options à d'autres services cloud et fournissent des conseils de migration.

Comparaison des services

- [Azure pour les professionnels AWS - DevOps et surveillance des applications](#)
- Comparaison des services Google Cloud et Azure – DevOps et surveillance des applications

Recommandations en matière de migration

Si vous migrez à partir d'une autre plateforme cloud, consultez l'article suivant :

- [Migrer une charge de travail d'Amazon Web Services \(AWS\) vers Azure](#)

📌 **Remarque** : L'auteur a créé cet article avec l'aide de l'IA. [En savoir plus](#)

Last updated on 26/08/2026

Intégration de l'informatique quantique à des applications classiques

Azure Quantum

Certains problèmes de calcul sont impraticables ou inductibles à résoudre sur les ordinateurs classiques, même sur les supercalculateurs volumineux. Pour certains de ces problèmes, un algorithme quantique peut atteindre une solution en utilisant beaucoup moins de ressources que l'approche classique la plus connue. Un ordinateur quantique utilise des effets mécaniques quantiques, tels que la superposition et l'enchevêtrement, pour représenter et traiter des informations de manière à ce qu'un ordinateur classique ne puisse pas.

Les programmes quantiques s'exécutent sur [fournisseurs d'informatique quantique](#) auxquels vous accédez en soumettant des tâches. Les cibles quantiques exposent différents [profils cibles](#). Certains profils autorisent uniquement les opérations quantiques, et la logique classique que vous pouvez exécuter sur eux est limitée. D'autres profils permettent à la fois aux opérations quantiques et classiques de s'exécuter ensemble sur le fournisseur.

Quel que soit le profil cible, les composants de calcul classiques gèrent l'intégration d'applications environnantes. Même lorsqu'une cible quantique accepte des opérations classiques, vous atteignez la cible en envoyant un travail et en attendant les résultats. Cet article décrit et compare deux modèles d'orchestration pour l'intégration d'un travail quantique à des applications classiques.

En pratique, l'exécution d'un programme quantique est un appel de service. Votre application classique ou votre code client envoie un travail à une cible, attend qu'elle s'exécute et récupère les résultats. Un ou plusieurs composants de calcul classiques orchestrent chaque travail quantique en effectuant les activités suivantes :

- Préparation des données d'entrée
- Envoi de [travaux](#) d'informatique quantique à un environnement quantique cible
- Surveillance de l'exécution du travail
- Résultats de la tâche post-traitement

Modèles d'intégration quantique

Vous intégrez le travail quantique à une application classique à l'aide de l'un des deux modèles d'orchestration :

- **Intégration quantique directe.** Une application cliente ou un harnais classique léger interagit directement avec l'espace de travail Azure Quantum. Le client possède la préparation des entrées, la soumission de travaux, la surveillance, la gestion des résultats et la logique classique qui entoure l'exécution quantique.
- **Intégration quantique orchestrée par flux de travail.** Un orchestrateur de workflow gère l'état global et les transitions. L'exécution quantique a lieu dans des étapes d'un flux de travail qui s'exécutent parallèlement à des étapes s'exécutant sur des ressources de calcul haute performance (HPC) ou sur processeur graphique (GPU).

Pour une limite d'intégration d'application donnée, ces modèles sont des alternatives. Soit le client s'intègre directement à l'espace de travail, soit un workflow plus large possède l'étape quantique. Cet article décrit l'implémentation de chaque modèle.

⚠ Remarque

Les architectures de cet article exécutent une partie d'une tâche de calcul sur une cible quantique. Pour certains défis de calcul, les services existants conçus pour effectuer un [calcul hautes performances](#) ou fournir des [fonctionnalités IA](#) peuvent être des alternatives.

Le choix d'intégration est indépendant de l'endroit où le calcul classique s'exécute. Selon le profil cible et la charge de travail, la logique classique peut s'exécuter dans le programme quantique, dans le client harnais entre les exécutions quantiques, dans l'orchestrateur de flux de travail ou dans une étape de flux de travail classique sur le calcul HPC ou GPU.

Matrice décisionnelle

Utilisez la matrice de décision suivante pour guider votre choix de modèles d'intégration :

 Agrandir le tableau

Si votre charge de travail présente ces caractéristiques	Utiliser cette approche
Une application ou un script léger gère l'ensemble du cycle de vie de la tâche quantique, et la préparation des données d'entrée ainsi que le traitement des résultats sont pris en charge par ce client.	Intégration quantique directe

Si votre charge de travail présente ces caractéristiques	Utiliser cette approche
Vous explorez les propriétés du matériel quantique avec du code quantique manuscrit, et une application ou un petit ensemble d'applications associées consomme les résultats.	Intégration quantique directe
La partie classique est lourde et fait appel au calcul quantique comme à une étape parmi d'autres, dans un pipeline à plusieurs étapes dont chacune s'exécute sur le back-end qui lui convient, par exemple un système HPC, un GPU ou une cible quantique.	Intégration quantique orchestrée par flux de travail
Les exécutions quantiques se répètent et le résultat d'une exécution génère le programme quantique suivant ou détermine ses paramètres.	Intégration quantique orchestrée par flux de travail

Détails du scénario

Les deux flux de travail implémentent le [modèle de Request-Reply asynchrone](#) et les étapes définies pour le [cycle de vie des travaux Azure Quantum](#).

Les cibles quantiques, en particulier le matériel quantique, sont des ressources limitées. Azure Quantum alloue ces ressources via une file d'attente de travaux. Lorsque vous soumettez une tâche, elle est mise dans la file d'attente de la cible que vous sélectionnez et s'exécute une fois que cette cible a terminé les entrées précédentes. Pour afficher le temps d'attente attendu, [répertoriez les cibles disponibles](#). Calculez votre temps de réponse total comme la somme du temps d'attente dans la file et du temps d'exécution de la tâche.

Les fonctionnalités d'une cible quantique varient également. Certaines cibles acceptent uniquement les opérations quantiques, tandis que d'autres exécutent une logique classique avec des opérations quantiques dans un seul travail. Ce modèle prend en charge les algorithmes qui s'adaptent pendant l'exécution. Avant de valider une cible, vérifiez qu'elle prend en charge les opérations dont votre algorithme a besoin. Pour plus d'informations sur la façon dont les instructions classiques et quantiques s'exécutent ensemble, consultez [Présentation de l'informatique quantique hybride](#).

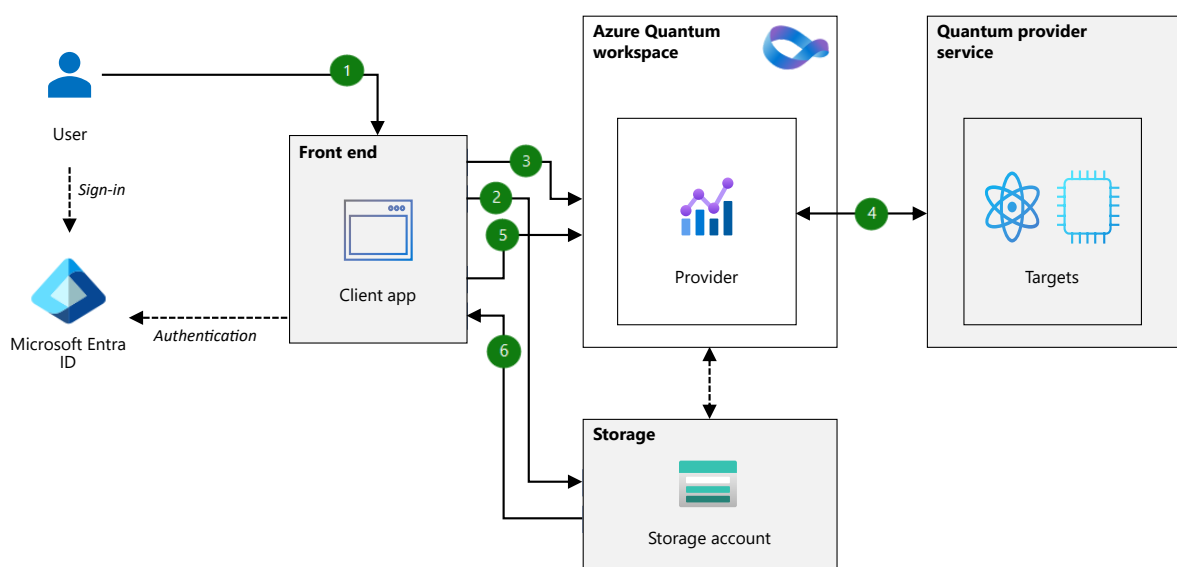
❗ Remarque

Ces modèles décrivent comment intégrer le travail quantique à votre architecture d'application. Ils sont distincts de Azure Quantum *modèles d'informatique hybride*, qui décrivent comment les calculs classiques et quantiques interagissent dans les travaux quantiques. Les profils cibles déterminent ce qui peut être exécuté dans ces tâches. Pour plus d'informations, consultez [Présentation de l'informatique quantique hybride](#). Les modèles d'intégration d'applications sont indépendants du comportement au sein du travail et contrôlent si un client ou un flux de travail exécute les travaux quantiques.

Intégration quantique directe

Les sections suivantes décrivent le modèle d'intégration directe pour intégrer le travail quantique à une application classique.

Architecture



Téléchargez un [fichier PowerPoint](#) de cette architecture.

Flux de données

Le flux de données suivant correspond au diagramme précédent :

1. Un utilisateur connecté déclenche l'exécution d'un travail quantique via une application cliente classique.

2. L'application cliente envoie les données dans le stockage Azure.
3. L'application cliente envoie le travail à un espace de travail Azure Quantum, en spécifiant la cible ou les cibles d'exécution.

Le client identifie l'espace de travail à partir de sa configuration et s'authentifie auprès de l'espace de travail à l'aide d'une identité Microsoft Entra. Un client qui s'exécute sur une ressource hébergée Azure peut utiliser une [identité managée](#). Une application cliente locale s'authentifie avec une autre identité Microsoft Entra, comme un principal de service ou une connexion utilisateur interactive.

4. Un fournisseur quantique exécute le travail sur un environnement cible.
5. L'application cliente surveille l'exécution du travail en interrogeant l'état du travail.
6. Dès que le travail quantique se termine, l'application cliente obtient le résultat de calcul à partir du stockage.

Composants

- [Azure Quantum](#) fournit un [workspace](#), accessible à partir du portail Azure, pour les ressources associées à l'exécution de travaux quantiques sur différentes cibles. Les travaux s'exécutent sur des simulateurs quantiques ou du matériel quantique, selon le fournisseur que vous choisissez.
- [Microsoft Entra ID coordonne l'authentification](#) des utilisateurs et permet de protéger l'accès à l'espace de travail Azure Quantum.
- [Le stockage](#) assure le stockage des données d'entrée et des résultats provenant du fournisseur quantique.

Cas d'usage potentiels d'intégration directe

Le modèle d'intégration quantique directe correspond aux cas d'usage suivants :

- Une application cliente ou un harnais classique léger possède le cycle de vie complet du travail quantique sans flux de travail plus large.
- Le client peut exécuter des travaux classiques, tels que la préparation des entrées et le traitement des résultats.
- Vous explorez les propriétés du matériel quantique, vous écrivez donc généralement le code quantique à la main plutôt que de le générer dynamiquement.
- L'utilisation des composants quantiques est limitée à une seule application ou à un petit ensemble d'applications associées.

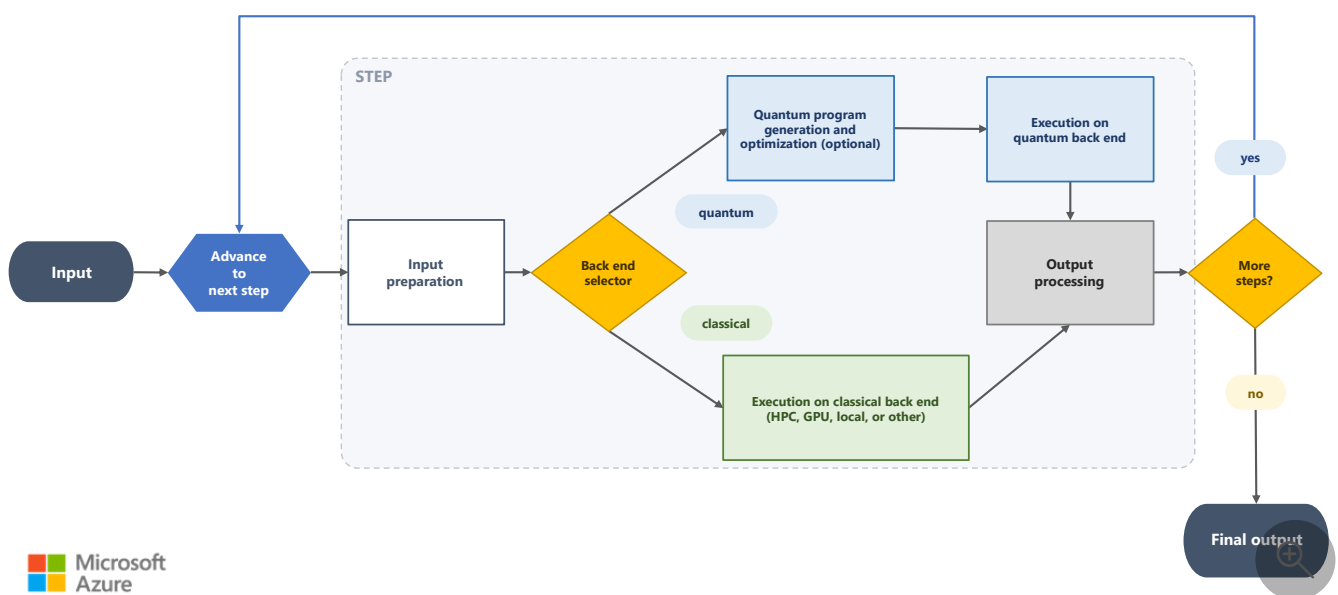
- Le travail quantique représente une solution spécialisée, telle qu'une simulation moléculaire, qu'une seule application classique spécialisée utilise.

Intégration quantique orchestrée par flux de travail

Les sections suivantes décrivent le modèle orchestré par le flux de travail pour intégrer le travail quantique à une application classique.

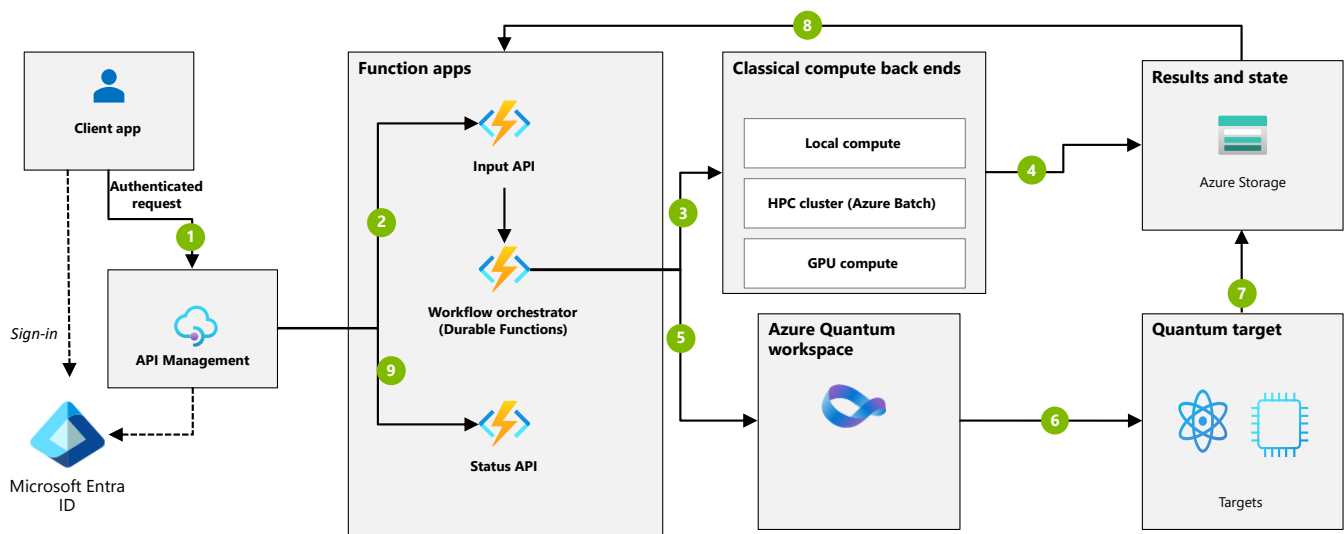
Architecture

La logique de flux de travail ne dépend d'aucun hôte ou fournisseur spécifique. L'organigramme suivant montre l'ordinateur d'état : une seule entrée, une boucle sur les étapes et les phases par étape. La branche quantique peut inclure une phase facultative de génération de programme et d'optimisation que les étapes classiques n'utilisent pas.



Le sélecteur principal dans l'organigramme précédent est une phase logique plutôt qu'un composant requis et peut être une recherche de configuration qui achemine chaque étape vers un type back-end fixe.

Vous pouvez héberger ce flux de travail sur Azure avec un ensemble de services classiques qui envoient et surveillent le travail quantique. Le diagramme suivant illustre un exemple de topologie :



Téléchargez un [fichier PowerPoint](#) de cette architecture.

Flux de données

Le flux de données suivant correspond à la topologie de déploiement :

1. Une application cliente envoie une demande via Gestion des API, qui authentifie l'appelant avec Microsoft Entra ID et applique la limitation avant que la demande atteigne le niveau de calcul.
2. Gestion des API transfère la requête à l'API d'entrée, une fonction déclenchée par HTTP qui la valide et démarre l'orchestrateur de flux de travail.
3. Pour une étape qui s'exécute classiquement, l'orchestrateur achemine l'exécution vers un serveur principal classique, tel qu'un cluster HPC ou un calcul GPU.
4. Le back-end classique écrit ses résultats et l'état mis à jour du pipeline dans l'espace de stockage.
5. Pour une étape qui s'exécute sur quantum, l'orchestrateur achemine l'exécution via le chemin quantum. Le chemin quantique prépare le programme de l'étape, en le générant et en l'optimisant à partir de l'entrée de l'étape ou à l'aide d'une implémentation fixe, et envoie le travail à l'espace de travail Azure Quantum. La soumission s'authentifie via [une identité managée](#).
6. L'espace de travail exécute le travail sur la cible quantique sélectionnée.
7. La cible quantum écrit ses résultats dans Stockage.

8. L'orchestrateur lit le stockage d'état mis à jour et évalue la logique de transition du pipeline. Si une condition de convergence ou d'itération nécessite une autre exécution quantique, l'orchestrateur utilise le résultat pour déterminer les paramètres du programme quantique suivant ou pour fournir des entrées qui génèrent le programme suivant. Le pipeline effectue ensuite une boucle vers l'étape appropriée.
9. Le client interroge l'API Status via Gestion des API pour suivre la progression et récupérer les résultats finaux lorsque le pipeline atteint un état terminal.

Composants

- [Durable Functions](#) agit en tant qu'orchestrateur de flux de travail pour exécuter le pipeline en tant qu'ordinateur d'état, coordonner les étapes, puis sélectionner le serveur principal de type de calcul ou quantique pour l'exécution de chaque étape. Vous pouvez implémenter l'orchestrateur avec Durable Functions ou un autre moteur de flux de travail.
- [Azure serveur principal de calcul HPC et GPU classique](#) exécutent les étapes non quantiques.
- [Azure Functions](#) héberge les API HTTP qui démarrent et surveillent le flux de travail, ainsi que l'orchestrateur qui l'exécute.
- [Gestion des API](#) est le point d'entrée pour les demandes clientes. Il authentifie les appelants et applique une limitation du débit. Pour empêcher l'application de fonction d'être appelée directement, utilisez [Azure Functions options de mise en réseau](#), telles que les restrictions d'accès entrantes ou un point de terminaison privé, afin qu'elle accepte uniquement le trafic à partir de Gestion des API.
- [Azure Quantum](#) fournit un [espace de travail](#) pour les ressources associées aux travaux quantiques en cours d'exécution. Les travaux s'exécutent sur des simulateurs quantiques ou du matériel quantique, selon la cible que vous choisissez.
- [Microsoft Entra ID coordonne l'authentification](#) et protège l'accès à l'espace de travail Azure Quantum.
- [Le stockage](#) stocke les données d'entrée, l'état du pipeline intermédiaire et les résultats.

Alternatives

Les architectures de cet article exécutent une partie d'une tâche de calcul sur une cible quantique. Pour certains défis de calcul, les services existants conçus pour effectuer un [calcul hautes performances](#) [↗](#) ou fournir des [fonctionnalités IA](#) [↗](#) peuvent être des alternatives.

Pour les charges de travail de R&D scientifique, Microsoft Discovery utilise l'IA pour orchestrer des tâches complexes à travers les modèles, les outils et les ressources de calcul. Microsoft Discovery peut coordonner divers outils dans des pipelines en plusieurs étapes et est extensible, ce qui vous permet de connecter vos propres outils et agents au lieu de développer et gérer

vous-même l'orchestration. L'intégration avec les fonctionnalités quantiques, y compris l'exécution d'étapes quantiques, peut utiliser l'IA pour effectuer l'orchestration que ce modèle génère manuellement.

Détails du scénario orchestré par flux de travail

Dans ce modèle, un orchestrateur de workflow exécute la charge de travail en tant que pipeline d'étapes. L'orchestrateur se comporte comme une machine d'état dans laquelle les étapes se répètent et se bouclent jusqu'à ce qu'une condition d'arrêt ou de convergence soit remplie. Chaque étape suit la même forme : elle prépare son entrée, s'exécute et traite la sortie.

L'orchestrateur sélectionne le système back-end qui exécute chaque étape. Cette sélection fait partie de la logique d'orchestration, et non d'un service distinct, et peut être aussi simple que la lecture du back-end cible pour chaque étape de la configuration. Une étape s'exécute sur un back-end quantique ou sur un calcul classique, tel qu'un cluster HPC ou un calcul GPU. Une étape classique légère peut s'exécuter dans l'orchestrateur lui-même. Lorsque les étapes quantiques se répètent, le flux de travail traite le résultat d'une exécution quantique pour déterminer les paramètres du programme quantique suivant ou pour fournir des entrées qui génèrent le programme suivant.

Une étape qui s'exécute sur un back-end quantique peut ajouter une phase que les étapes classiques n'ont pas. Cette phase génère et optimise le programme quantique avant l'exécution. Cette phase est facultative. Le programme peut être généré dynamiquement à partir de l'entrée de l'étape, ou provenir d'une implémentation fixe qui saute cette phase. Une étape éligible au traitement quantique peut néanmoins être exécutée sur un backend classique si cela est mieux adapté aux données d'entrée.

La chimie quantique est un exemple représentatif. Un pipeline classique utilise des étapes classiques pour préparer un système moléculaire : optimisation de la géométrie, calcul de champ auto-cohérent et sélection d'espace actif. Le pipeline calcule ensuite une propriété cible, telle que l'énergie d'état terrestre d'une molécule. L'étape d'informatique énergétique conserve la même intention qu'elle s'exécute sur une approximation classique ou sur un algorithme quantique. Vous choisissez le back-end en fonction de la précision dont vous avez besoin et de l'ampleur du problème.

La bibliothèque [QDK/Chimie](#) prend en charge les pipelines comme ceux-ci. La bibliothèque fournit des composants modulaires pour les étapes de préparation classiques et pour générer un circuit de préparation d'état à partir de la fonction d'onde calculée classiquement, qu'un back-end quantique utilise ensuite pour estimer l'énergie avec un algorithme tel que l'estimation de

phase quantique. Cette fonctionnalité de génération de circuits est un exemple concret de génération dynamique du programme quantique à partir de l'entrée d'une étape.

Cas d'usage potentiels

Le modèle d'intégration quantique orchestrée par flux de travail correspond aux cas d'usage suivants :

- Le côté classique est lourd et orienté vers une logique d'états et de problèmes, et il exploite la capacité de quantum comme une ou plusieurs étapes parmi tant d'autres.
- La charge de travail est un pipeline à plusieurs étapes, souvent itératif, comme une machine à états. Chaque étape s'exécute sur le back-end de calcul le mieux adapté : HPC, GPU ou cible quantique. L'informatique quantique est une option pour une ou plusieurs étapes.
- Une étape quantique représente un bloc de construction bien défini, tel qu'un calcul scientifique qui calcule une propriété moléculaire. Le code quantique précis peut même être généré et optimisé dynamiquement pour s'adapter aux entrées réelles.
- L'orchestrateur de flux de travail assure la gestion de plusieurs exécutions quantiques autonomes et du traitement classique entre elles. Le résultat d'une exécution détermine les paramètres du programme quantique suivant ou fournit des entrées qui génèrent le programme suivant. [L'estimation de phase itérative](#) correspond à cette structure, comme les algorithmes variationnels tels que l'Eigensolver (VQE) variational quantum et l'algorithme d'optimisation approximative quantique (QAOA).

Considérations

Ces considérations implémentent les piliers d'Azure Well-Architected Framework, un ensemble de principes directeurs que vous pouvez utiliser pour améliorer la qualité d'une charge de travail. Pour plus d'informations, consultez [Well-Architected Framework](#).

Fiabilité

La fiabilité permet de s'assurer que votre application peut respecter les engagements que vous prenez à vos clients. Pour plus d'informations, veuillez consulter [la liste de vérification de la conception pour la fiabilité](#).

Les tâches quantiques s'exécutent sur des cibles distantes et partagées, de sorte que leur exécution peut échouer en raison d'erreurs transitoires, telles que l'expiration du délai d'attente de la cible. Quel que soit le modèle d'intégration que vous choisissiez, surveillez l'exécution du travail afin de pouvoir exposer l'état du travail à l'utilisateur. Lorsqu'un travail échoue en raison

d'une erreur temporaire, appliquez le [modèle Nouvelle tentative](#). Envoyez des travaux au moyen d'appels asynchrones et interrogez régulièrement le résultat afin de ne pas bloquer le client appelant.

La disponibilité de la fonctionnalité de calcul quantique dépend fortement de la disponibilité et des caractéristiques de capacité du [fournisseur de calcul quantique](#). Selon la cible de calcul, l'application cliente classique peut rencontrer des retards longs ou une indisponibilité de la cible.

Pour les services Azure environnants, les considérations de disponibilité habituelles s'appliquent. Si nécessaire, envisagez d'utiliser les options de réplication dans [stockage Azure redondance](#).

Fiabilité de l'intégration orchestrée par flux de travail

- Pour la haute disponibilité dans l'intégration orchestrée par le flux de travail, déployez gestion des API dans plusieurs zones de [disponibilité](#) ou [plusieurs régions](#). La redondance de zone nécessite le niveau Premium ou Premium v2, et le déploiement multirégion nécessite le niveau Premium.
- Si vous implémentez l'orchestrateur avec Durable Functions, planifiez sa reprise après sinistre comme un tout, au lieu de considérer l'application de fonction et son état comme des services indépendants. Les fonctions durables conservent l'état d'orchestration dans un hub de tâches dans un serveur principal de stockage, qui est stockage Azure par défaut. Étant donné que l'état d'exécution et le calcul sont couplés via ce hub de tâches, l'approvisionnement de l'application de fonction dans une deuxième région et la réplication du stockage séparément ne définit pas de basculement sécurisé. Les orchestrations peuvent interrompre, perdre des transactions récentes ou lire un hub de tâches entre des régions, en fonction de la topologie.

Pour un basculement sécurisé, utilisez une configuration active-passive qui bascule vers une région secondaire, placée derrière un service mondial d'équilibrage de charge tel que [Azure Front Door](#) ou [Azure Traffic Manager](#). Vérifiez que les sondes d'intégrité du service peuvent accéder à l'application de fonction avec vos restrictions réseau, car les contrôles qui limitent l'application au trafic d'API Management peuvent également bloquer ces sondes. Choisissez la topologie qui correspond à votre tolérance pour la perte de données et la latence entre régions. Pour connaître les options coordonnées et leurs compromis, consultez [la récupération d'urgence et la géo-distribution dans Durable Functions](#).

Sécurité

La sécurité offre des garanties contre les attaques délibérées et l'utilisation abusive de vos données et systèmes précieux. Pour plus d'informations, consultez [liste de vérification pour la révision de conception concernant la sécurité](#).

Appliquez les pratiques de renforcement suivantes aux services classiques qui entourent le travail quantique, quel que soit le modèle d'intégration que vous choisissiez :

- Authentifiez-vous pour Azure Quantum et les services environnants à l'aide de Microsoft Entra identités, puis désactivez l'authentification locale où le service le prend en charge. Utilisez [des identités managées](#) partout où l'environnement d'hébergement les prend en charge. Pour un client qui ne peut pas utiliser une identité managée, authentifiez-vous avec une autre identité Microsoft Entra, telle qu'un principal de service ou une connexion utilisateur interactive.
- Accordez à chaque composant qui accède à Stockage l'accès au plan de données dont il a besoin via le [contrôle d'accès en fonction des rôles Azure \(Azure RBAC\) avec Microsoft Entra ID](#), limité à ce composant. N'incorporez pas de clés de compte de stockage dans le code d'application.

En général, appliquez les [recommandations de sécurité du Well-Architected Framework](#) le cas échéant.

Sécurité pour l'intégration quantique directe

Contrairement à l'intégration quantique orchestrée par le flux de travail, ce modèle suppose qu'un seul client accède à l'espace de travail Azure Quantum. Le client est généralement une plateforme classique légère qui se concentre sur l'envoi et l'opération des travaux plutôt que sur un état de workflow plus large. Ce scénario entraîne les configurations suivantes :

- Étant donné que le client est connu, vous pouvez lui donner une identité fixe. Lorsque le client s'exécute sur une ressource hébergée Azure, associez une [identité managée](#) à celle-ci. Lorsque le client s'exécute en dehors de Azure, utilisez un principal de service ou une connexion utilisateur interactive.
- Vous pouvez implémenter la limitation des requêtes et la mise en cache des résultats dans le client lui-même.

Sécurité de l'intégration orchestrée par flux de travail

Contrairement à l'intégration quantique directe, ce modèle place un niveau de service classique devant le travail quantique. Gestion des API est la porte d'entrée de ce niveau, de sorte que les

configurations de sécurité mettent l'accent sur la protection du point d'entrée et du chemin d'accès à l'espace de travail quantique.

- Les clients doivent s'authentifier auprès de l'API. Implémentez cette authentification à l'aide de [stratégies d'authentification](#).
- Vous pouvez implémenter l'authentification des fonctions Azure via [identités managées](#) associées aux fonctions. Vous utilisez ces identités pour authentifier les appels sortants vers l'espace de travail Azure Quantum.
- Gestion des API peut appliquer la limitation des demandes pour protéger le back-end quantique et limiter l'utilisation des ressources quantiques. Pour plus d'informations, consultez [la limitation du débit des requêtes dans API Management](#).
- Selon le modèle de requête, vous pouvez implémenter la mise en cache des résultats de l'informatique quantique à l'aide de [stratégies de mise en cache gestion des API](#).

Optimisation des coûts

L'optimisation des coûts se concentre sur les moyens de réduire les dépenses inutiles et d'améliorer l'efficacité opérationnelle. Pour plus d'informations, consultez la [liste de vérification de conception pour l'optimisation des coûts](#).

Le coût global de cette solution dépend de la cible d'informatique quantique que vous sélectionnez pour exécuter le travail quantique. Les composants classiques sont simples à estimer. Pour un déploiement représentatif du modèle orchestré par le flux de travail, consultez cet [exemple d'estimation des coûts](#), qui couvre les composants classiques, notamment Gestion des API, Azure Functions et Stockage. Le modèle d'intégration directe est plus léger, mais ses coûts classiques incluent toujours stockage et tout hébergement, mise en réseau et surveillance que l'application cliente utilise.

Vous pouvez consommer des fournisseurs d'informatique quantique pour Azure Quantum via une offre [de place de marché Microsoft](#). La tarification dépend du type de ressource (simulateur ou matériel), de la référence SKU et de votre utilisation. Pour plus d'informations, accédez à la page de référence du fournisseur de votre scénario à partir de [fournisseurs d'informatique quantique sur Azure Quantum](#).

Contributeurs

Microsoft conserve cet article. Le contributeur suivant a écrit cet article.

Auteur principal :

- [Zander Chocron](#) | Ingénieur logiciel principal, Microsoft Quantum

Pour afficher les profils LinkedIn non publics, connectez-vous à LinkedIn.

Étapes suivantes

- Pour obtenir une vue d'ensemble de l'écosystème d'informatique quantique Microsoft, consultez le site web [Microsoft Quantum](#) et suivez le parcours d'apprentissage [des bases de l'informatique quantique](#).
- Pour plus d'informations sur le service Azure Quantum, consultez [Qu'est-ce que Azure Quantum ?](#).
- Pour obtenir des informations générales sur la gestion des travaux Azure Quantum, consultez [Travailler avec des travaux Azure Quantum](#).
- Pour plus d'informations sur la combinaison d'instructions classiques et quantiques dans un travail, consultez [Présentation de l'informatique quantique hybride](#).

Ressources associées

- [Principes de conception de l'excellence opérationnelle](#)
- [modèle de Request-Reply asynchrone](#)

Prise en main des sessions

Les sessions sont une fonctionnalité clé de l'informatique quantique hybride. Une session est un regroupement logique d'un ou plusieurs travaux que vous soumettez à un seul target. Chaque session a un ID unique attaché à chaque travail de cette session. Les sessions sont utiles lorsque vous souhaitez exécuter plusieurs travaux de calcul quantique en séquence et exécuter du code classique entre les travaux quantiques.

❗ Remarque

Les travaux de groupe de sessions pour la soumission d Azure Quantum 'un fournisseur, mais les travaux d'une session ne sont pas traités par lots côté fournisseur.

Prérequis

Pour créer une session, vous avez besoin des prérequis suivants :

- Un compte Azure avec un abonnement actif. Si vous n'avez pas de compte Azure, inscrivez-vous gratuitement et [inscrivez-vous à un abonnement de paiement à l'utilisation](#) .
- Un Azure Quantum espace de travail. Pour plus d'informations, consultez [Créer un Azure Quantum espace de travail](#).
- Un Python environnement avec [Python et Pip](#) installé.
- Dernière version de [Visual Studio Code \(VS Code\)](#) , avec l'extension [QDK](#) , l'extension [Python](#) et l'extension [Jupyter](#) installées.
- Le package `qdk` Python. Pour soumettre les Qiskit et Cirq programmes, installez les `azure` , `qiskit` et `cirq` extras.

Bash

```
pip install --upgrade "qdk[azure,qiskit,cirq]"
```

Qu'est-ce qu'une session ?

Dans les sessions, vous pouvez déplacer la ressource de calcul du client vers le cloud pour une latence inférieure et la possibilité d'exécuter votre programme quantique plusieurs fois avec différents paramètres. Vous pouvez regrouper logiquement des travaux dans une session, et vous pouvez hiérarchiser les travaux de cette session par rapport aux travaux non-session. Les états Qubit ne persistent pas entre les travaux, mais les travaux d'une session ont des temps de file d'attente plus courts. Des temps de file d'attente plus courts vous permettent d'exécuter des algorithmes complexes pour mieux organiser et suivre vos travaux d'informatique quantique individuels.

Les sessions sont utiles pour les algorithmes quantiques paramétrés, où la sortie d'un travail d'informatique quantique est utilisée pour définir des paramètres d'entrée pour le prochain travail de calcul quantique. Les exemples les plus courants de ce type d'algorithme sont Variational Quantum Eigensolvers (VQE) et Quantum Approximate Optimization Algorithms (QAOA).

Matériel pris en charge

Les sessions sont prises en charge sur tous les fournisseurs de matériel informatique quantique. Dans certains cas, les travaux soumis dans une session sont classés par ordre de priorité dans la file d'attente de ce target. Pour plus d'informations, consultez [Comportement cible](#).

Guide pratique pour créer une session

Pour créer une session, procédez comme suit :

Q# + Python

Cet exemple montre comment créer une session avec Q# du code inline dans un Jupyter bloc-notes dans VS Code.

ⓘ Remarque

Les sessions sont gérées avec Python, même lors de l'exécution Q# de code inline.

1. Dans VS Code, ouvrez le menu **Affichage** et choisissez **Palette de commandes**.
2. Entrez et sélectionnez **Créer : Nouveau Jupyter Notebook**.

3. En haut à droite, VS Code détectera et affichera la version de Python ainsi que l'environnement virtuel Python sélectionné pour le notebook. Si vous avez plusieurs Python environnements, vous devrez peut-être sélectionner un noyau à l'aide du sélecteur de noyau en haut à droite. Si aucun environnement n'a été détecté, consultez [Jupyter Notebook informations d'installation dans VS Code](#).

4. Dans la première cellule du notebook, exécutez le code suivant :

Python

```
from qdk.azure import Workspace

workspace = Workspace(resource_id="") # add your resource ID
```

5. Ajoutez une nouvelle cellule dans le notebook et importez le `qsharp` Python package :

Python

```
from qdk import qsharp
```

6. Choisissez votre [quantum target](#). Dans cet exemple, il s'agit target du [simulateur IonQ](#).

Python

```
target = workspace.get_targets("ionq.simulator")
```

7. Sélectionnez les configurations du [target profil](#), soit `Base`, `Adaptive_RI` soit `Unrestricted`.

Python

```
qsharp.init(target_profile=qsharp.TargetProfile.Base)
```

⚠ Remarque

`Adaptive_RI` target les tâches de profil sont actuellement prises en charge sur Quantinuum targets. Pour plus d'informations, consultez [l'informatique](#) quantique hybride intégrée.

8. Écrivez votre Q# programme. Par exemple, le programme suivant Q# génère un bit aléatoire. Pour illustrer l'utilisation des arguments d'entrée, ce programme prend un

entier, `n` et un tableau d'angles, `angle` comme entrée.

Q#

```
%%qsharp
import Std.Measurement.*;
import Std.Arrays.*;

operation GenerateRandomBits(n: Int, angle: Double[]) : Result[] {
    use qubits = Qubit[n]; // n parameter as the size of the qubit array
    for q in qubits {
        H(q);
    }
    R(PauliZ, angle[0], qubits[0]); // arrays as entry-points parameters
    R(PauliZ, angle[1], qubits[1]);
    let results = MeasureEachZ(qubits);
    ResetAll(qubits);
    return results;
}
```

9. Ensuite, vous créez une session. Supposons que vous souhaitez exécuter

`GenerateRandomBit` l'opération trois fois, de sorte que vous utilisez `target.submit` pour envoyer l'opération Q# avec les `target` données et répéter le code trois fois - dans un scénario réel, vous pouvez soumettre différents programmes au lieu du même code.

Python

```
angle = [0.0, 0.0]
with target.open_session(name="Q# session of three jobs") as session:
    target.submit(input_data=qsharp.compile(f"GenerateRandomBits(2, {angle}")), name="Job 1", shots=100) # First job submission
    angle[0] += 1
    target.submit(input_data=qsharp.compile(f"GenerateRandomBits(2, {angle}")), name="Job 2", shots=100) # Second job submission
    angle[1] += 1
    target.submit(input_data=qsharp.compile(f"GenerateRandomBits(2, {angle}")), name="Job 3", shots=100) # Third job submission

session_jobs = session.list_jobs()
[session_job.details.name for session_job in session_jobs]
```

❗ Important

Lorsque vous passez des arguments en tant que paramètres à la tâche, les arguments sont mis en forme dans l'expression Q# lorsque `qsharp.compile` est appelé. Cela signifie que vous devez mettre en forme vos arguments en tant qu'objets Q#. Dans cet exemple, comme les tableaux dans Python sont déjà

imprimés selon le format `[item0, item1, ...]`, les arguments d'entrée correspondent à la mise en forme Q#. Pour d'autres Python structures de données, vous devrez peut-être effectuer davantage de traitement pour obtenir les valeurs de chaîne insérées d'une manière compatible avec Q#.

10. Une fois que vous avez créé une session, vous pouvez utiliser

`workspace.list_session_jobs` pour récupérer une liste de tous les travaux de la session.

Pour plus d'informations, consultez [Comment gérer les sessions](#).

Comportement cible

Chaque fournisseur de matériel quantique définit leurs propres heuristiques pour mieux gérer la hiérarchisation des travaux au sein d'une session.

Quantinuum

Si vous choisissez d'envoyer des travaux au sein d'une session à un [Quantinuum target](#), votre session a un accès exclusif au matériel tant que vous placez les travaux en file d'attente dans un délai maximal d'une minute entre chaque soumission. Après cela, vos travaux sont acceptés et gérés avec la logique de mise en file d'attente et de hiérarchisation standard.

Contenu connexe

- [Comment gérer vos sessions](#)
- [Exécuter l'informatique quantique hybride](#)

Comment gérer vos sessions

Dans Azure Quantum, vous pouvez regrouper plusieurs travaux sur une cible unique pour gérer efficacement vos travaux. Il s'agit d'une session. Pour plus d'informations, consultez [Commencer avec les sessions](#).

Dans cet article, vous allez apprendre à utiliser des sessions pour gérer manuellement vos travaux. Vous découvrirez également les politiques d'échec de tâches et comment éviter les timeouts de session.

Conditions préalables

- Un compte Azure avec un abonnement actif. Si vous n'avez pas de compte Azure, inscrivez-vous gratuitement et inscrivez-vous à un [abonnement](#) de paiement à l'utilisation.
- Un Azure Quantum espace de travail. Pour plus d'informations, consultez [Créer un Azure Quantum espace de travail](#).
- Un Python environnement avec [Python et Pip](#) installé.
- Dernière version du `qdk` Python package avec la `azure` version supplémentaire.

Bash

```
pip install --upgrade "qdk[azure]"
```

Si vous souhaitez utiliser Qiskit et Cirq, vous devez installer les `qiskit` éléments supplémentaires. `cirq`

Bash

```
pip install --upgrade "qdk[azure,qiskit,cirq]"
```

⚠ Remarque

Les sessions sont gérées avec Python, même lorsque vous exécutez Q# du code inline.

Surveiller vos sessions

Pour afficher et gérer vos sessions soumises, procédez comme suit.

Dans le [portail Azure](#), utilisez le panneau **Gestion des travaux** dans votre espace de travail Quantum pour afficher tous les éléments envoyés de niveau supérieur, y compris les sessions et les travaux individuels qui ne sont associés à aucune session.

1. Connectez-vous au portail [Azure](#).
2. Accédez à votre espace de travail Quantum.
3. Développez la liste déroulante **Opérations** et sélectionnez **Gestion des travaux** pour afficher une table de vos travaux et sessions soumis. Les sessions ont une valeur de **Session** dans la colonne **Type**.
4. Pour afficher des détails sur une session, y compris une liste de **tous les travaux** de la session, sélectionnez le **nom** de la session.

Chaque session a un ID et un état uniques. Les sessions peuvent avoir les états suivants.

- **En attente** : les travaux au sein de la session sont en cours d'exécution.
- **Réussite** : la session s'est terminée avec succès.
- **Délai d'expiration** : si aucun nouveau travail n'est envoyé dans la session pendant 10 minutes, cette session expire. Pour plus d'informations, consultez [Délais d'expiration de session](#).
- **Échec** : si un travail au sein d'une session échoue, cette session se termine et signale un état d'échec. Pour plus d'informations, voir la [stratégie d'échec des travaux au sein des sessions](#).

Récupérer et répertorier les sessions

Le tableau suivant montre les Python commandes permettant d'obtenir la liste de toutes les sessions et de tous les travaux d'une session donnée. Pour vous connecter à votre espace de travail via le `qdk.azure` module, consultez [Se connecter à votre Azure Quantum espace de travail avec l'Azure CLI ou le QDK Azure CLI](#).

 Agrandir le tableau

Commande	Description
<code>workspace.list_sessions()</code> ou <code>session.list_sessions()</code>	Récupérez la liste de toutes les sessions d'un espace de travail Quantum.
<code>workspace.get_session(sessionId)</code> ou <code>session.get_session(sessionId)</code>	Récupérez la session avec l'ID <code>sessionId</code> . Chaque session a un ID unique.

Commande	Description
<code>workspace.list_session_jobs(sessionId)</code> ou <code>session.list_session_jobs(sessionId)</code>	Récupérez une liste de tous les travaux de la session avec l'ID <code>sessionId</code> . Chaque session a un ID unique.

Par exemple, le code suivant définit une fonction qui obtient une session avec un nombre minimal de travaux. Ensuite, pour cette session, elle répertorie tous les travaux, le nombre total de travaux et les 10 premiers travaux.

```

Python

def get_a_session_with_jobs(min_jobs):
    all_sessions = workspace.list_sessions() # list of all sessions
    for session in all_sessions:
        if len(workspace.list_session_jobs(session.id)) >= min_jobs:
            return session

session = get_a_session_with_jobs(min_jobs=3) # Get a Session with at least 3 jobs

session_jobs = workspace.list_session_jobs(session.id) # List of all jobs within Session ID

print(f"Job count: {len(session_jobs)} \n")
print(f"First 10 jobs for session {session.id}:")
for job in session_jobs[0:10]:
    print(f"Id: {job.id}, Name={job.details.name}")

```

Ouvrir ou fermer manuellement des sessions

Pour créer une session, il est recommandé de suivre les étapes de [prise en main des sessions](#). Pour créer manuellement des sessions à la place, choisissez l'onglet de votre environnement de développement.

Q# + Python

1. Créez un objet Session.

```

Python

from qdk.azure.job import Session, SessionDetails, SessionJobFailurePolicy
import uuid

session = Session(
    workspace=workspace, # required
    id=f"{uuid.uuid1()}", # optional, if not passed will use uuid.uuid1()

```

```

name="", # optional, will be blank if not passed
provider_id="ionq", # optional, if not passed will try to parse from the
target
target="ionq.simulator", # required
job_failure_policy=SessionJobFailurePolicy.ABORT # optional, defaults to
abort
)

print(f"Session status: {session.details.status}")

```

❗ Remarque

À ce stade, la session existe uniquement sur le client et vous pouvez voir que l'état est **None**. Pour afficher l'état de la session, vous devez également créer la session dans le service.

2. Pour **créer** une session dans le service, vous pouvez utiliser `workspace.open_session(session)` ou `session.open()`.
3. Vous pouvez actualiser l'état et les détails de la session avec `session.refresh()`, ou en obtenant un nouvel objet de session à partir d'un ID de session.

Python

```

same_session = workspace.get_session(session.id)
print(f"Session: {session.details} \n")
print(f"Session: {same_session.details} \n")

```

4. Vous pouvez **fermer** une session avec `session.close()` ou `workspace.close_session(session)`.
5. Pour **attacher la session** à une cible, vous pouvez utiliser `target.latest_session`.
6. Vous pouvez **attendre** la fin d'une session :

Python

```

session_jobs = session.list_jobs()
[session_job.id for session_job in session_jobs]

import time
while (session.details.status != "Succeeded" and session.details.status !=
"Failed" and session.details.status != "TimedOut"):
    session.refresh()
    time.sleep(5)

```

Transmettre des arguments dans Q#

Si votre Q# opération prend des arguments d'entrée, transmettez ces arguments en tant qu'objets Q# lorsque vous envoyez des travaux via Python.

Lorsque vous passez des arguments en tant que paramètres à la tâche, ils sont mis en forme en tant que Q# code lorsqu'ils `qsharp.compile` sont appelés, de sorte que les valeurs de Python doivent être mises en forme dans une chaîne avec une syntaxe valide Q#.

Considérez le programme suivant Q# , qui prend un entier, `n` et un tableau d'angles, `angle` comme entrée.

Q#

```
import Std.Measurement.*;
import Std.Arrays.*;

operation GenerateRandomBits(n: Int, angle: Double[]) : Result[] {
    use qubits = Qubit[n]; // n parameter as the size of the qubit array
    for q in qubits {
        H(q);
    }
    R(PauliZ, angle[0], qubits[0]); // arrays as entry-points parameters
    R(PauliZ, angle[1], qubits[1]);
    let results = MeasureEachZ(qubits);
    ResetAll(qubits);
    return results;
}
```

Vous souhaitez exécuter l'opération `GenerateRandomBits` trois fois avec `n=2` et différents angles. Vous pouvez utiliser le code suivant Python pour envoyer trois travaux avec des angles différents.

Python

```
angle = [0.0, 0.0]
with target.open_session(name="Q# session of three jobs") as session:
    target.submit(input_data=qsharp.compile(f"GenerateRandomBits(2, {angle})"),
name="Job 1", shots=100) # First job submission
    angle[0] += 1
    target.submit(input_data=qsharp.compile(f"GenerateRandomBits(2, {angle})"),
name="Job 2", shots=100) # Second job submission
    angle[1] += 1
    target.submit(input_data=qsharp.compile(f"GenerateRandomBits(2, {angle})"),
name="Job 3", shots=100) # Third job submission
```

```
session_jobs = session.list_jobs()
[session_job.details.name for session_job in session_jobs]
```

Dans cet exemple, étant donné que les tableaux sont déjà affichés sous la forme [item0, item1, ...], les arguments d'entrée correspondent à la mise en forme de Python. Pour d'autres Python structures de données, vous devrez peut-être effectuer plus de traitements pour insérer les valeurs de chaîne dans le Q# de manière compatible. Par exemple, un Q# tuple doit être entre parenthèses avec des valeurs séparées par des virgules.

Délais d'expiration de la session

Une session expire si aucun nouveau travail n'est envoyé dans la session pendant 10 minutes. La session affiche le statut **TimedOut**. Pour éviter cette situation, ajoutez un `with` bloc à l'aide de `backend.open_session(name="Name")`, de sorte que la session `close()` soit appelée par le service à la fin du bloc de code.

⚠ Remarque

S'il existe des erreurs ou des bogues dans votre programme, la soumission d'un nouveau travail peut prendre plus de 10 minutes après la fin des travaux précédents de la session.

Les extraits de code suivants montrent un exemple de session qui expire après 10 minutes, car aucun nouveau travail n'est envoyé. Pour éviter cela, l'extrait de code suivant montre comment utiliser un `with` bloc pour créer une session.

Python

```
# Example of a session that times out

session = backend.open_session(name="Qiskit circuit session") # Session times out
because only contains one job
backend.run(circuit=circuit, shots=100, job_name="Job 1")
```

Python

```
# Example of a session that includes a with block to avoid timeout

with backend.open_session(name="Qiskit circuit session") as session: # Use a with
block to submit multiple jobs within a session
    job1 = backend.run(circuit=circuit, shots=100, job_name="Job 1") # First job
submission
    job1.wait_for_final_state()
    job2 = backend.run(circuit=circuit, shots=100, job_name="Job 2") # Second job
```

```
submission
    job2.wait_for_final_state()
    job3 = backend.run(circuit=circuit, shots=100, job_name="Job 3") # Third job
submission
    job3.wait_for_final_state()
```

Stratégie d'échec des travaux au sein des sessions

La stratégie par défaut d'une session en cas d'échec d'un travail consiste à mettre fin à cette session. Si vous envoyez un travail supplémentaire au sein de la même session, le service le rejette et la session signale un état **d'échec**. Tous les travaux en cours sont annulés.

Toutefois, ce comportement peut être modifié en spécifiant une stratégie de gestion des échecs de tâches `job_failure_policy=SessionJobFailurePolicy.CONTINUE`, au lieu de la stratégie par défaut `SessionJobFailurePolicy.ABORT`, pendant la création de la session. Lorsque la stratégie d'échec des travaux est `CONTINUE`, le service continue d'accepter des travaux. La session signale l'état des échecs dans ce cas, ce qui passera à **Échec** une fois la session fermée.

Si la session n'est jamais fermée et arrive à expiration, le statut est **TimedOut** même si les travaux ont échoué.

Par exemple, le programme suivant crée une session avec trois tâches. Le premier travail échoue, car il spécifie `"garbage"` comme données d'entrée. Pour éviter la fin de la session à ce stade, le programme montre comment ajouter `job_failure_policy=SessionJobFailurePolicy.CONTINUE` lorsque vous créez la session.

Python

```
#Example of a session that does not close but reports Failure(s) when a jobs fails

with target.open_session(name="JobFailurePolicy Continue",
    job_failure_policy=SessionJobFailurePolicy.CONTINUE) as session:
    target.submit(input_data="garbage", name="Job 1") #Input data is missing, this job
fails
    target.submit(input_data=quil_program, name="Job 2") #Subsequent jobs are accepted
because of CONTINUE policy
    target.submit(input_data=quil_program, name="Job 3")
```

Contenu connexe

- [Exécution de l'informatique quantique hybride](#)
- [Commencez avec les sessions](#)

Fournisseurs de calcul quantique sur Azure Quantum

Azure Quantum propose différentes solutions quantiques, telles que différents appareils matériels quantiques et simulateurs quantiques que vous pouvez utiliser pour exécuter vos programmes d'informatique quantique. Cet article répertorie les fournisseurs auxquels vous pouvez accéder avec Azure Quantum et fournit une description de ce que chaque fournisseur propose.

[🔗 Agrandir le tableau](#)

Provider	Description
 IONQ	Les ordinateurs quantiques basés sur la porte d'ion piégée d'IonQ sont universels et reconfigurables dynamiquement dans les logiciels, fournissant jusqu'à 25 qubits dans les processeurs QPU IonQ Aria et 36 qubits dans le QPU IonQ Forte 1 et Forte Enterprise 1 QPU. Tous les qubits sont entièrement connectés, ce qui signifie que vous pouvez exécuter une porte à deux qubits entre n'importe quelle paire. L'implémentation des opérations de porte quantique passe par la manipulation d'ions Ytterbium avec des impulsions laser. IonQ fournit un simulateur quantique avec accélération par GPU prenant en charge jusqu'à 36 qubits, à l'aide du même ensemble de portes qu'IonQ fournit sur son matériel quantique. Pour plus d'informations, consultez la page du fournisseur IonQ .
 PASQAL	Les processeurs quantiques neutres basés sur les atomes de PASQAL qui fonctionnent à température ambiante ont des temps de cohérence longs et une connectivité qubit impressionnante. Les opérations sont effectuées avec des pinces optiques, à l'aide de la lumière laser pour manipuler des registres quantiques 1D et 2D avec jusqu'à une centaine de qubits. Pour plus d'informations, consultez la page du fournisseur PASQAL .
 QUANTINUUM	Les ordinateurs quantiques à ions piégés de Quantinuum ont des qubits de haute fidélité, entièrement connectés, avec la possibilité de les réutiliser. Les opérations quantiques sont des portes laser avec de faibles taux d'erreur et peuvent effectuer des mesures à mi-circuit. La génération de matériel System Model H2, propulsé par Honeywell, utilise une architecture de Dispositif à Couplage de Charge Quantique (QCCD). Quantinuum fournit des outils d'émulation, deux émulateurs de modèle système H2, qui contiennent des modèles physiques détaillés et des modèles de bruit du matériel quantique réel. Pour plus d'informations, consultez la page du fournisseur Quantinuum .
 rigetti	Les systèmes de Rigetti sont alimentés par des processeurs quantiques basés sur des qubits superconducteurs. Ils offrent des temps de contrôle rapides, une logique conditionnelle à faible latence et des temps d'exécution de programme rapides. Au niveau de la puce, chaque qubit superconducteur se compose d'une inductance Josephson non linéaire en parallèle avec un condensateur ultra-faible perte pour créer une structure résonante dans la plage de 3 à 6GHz. Les qubits sont couplés à un résonateur de superconducteur linéaire

Provider	Description
	pour la lecture. La combinaison du qubit, du résonateur de lecture linéaire et du câblage associé fournit un élément de circuit quantique à usage général capable d'encodage fiable, de manipulation et de lecture d'informations quantiques. Les processeurs de Rigetti utilisent des tableaux de qubits couplés les uns aux autres avec des condensances à puce. Les opérations logiques uniques et multi-qubits sont implémentées par le biais de l'application d'impulsions micro-ondes ou DC. Pour plus d'informations, consultez la page du fournisseur Rigetti .

Important

Les appareils matériels quantiques sont encore une technologie émergente. Ces appareils présentent des limitations et des exigences pour les programmes quantiques qui s'exécutent dessus. Pour plus d'informations, consultez les types de profils [target dans Azure Quantum](#).

Pour plus d'informations sur les fournisseurs de calcul quantique disponibles dans votre région, consultez [Disponibilité globale des fournisseurs de Azure Quantum](#).

Disponibilité de Qubit pour les fournisseurs de calcul quantique

Les partenaires fournisseurs de Microsoft offrent un large éventail de disponibilités de qubits pour leurs processeurs matériels et simulateurs.

 Agrandir le tableau

Nom de la cible	Nombre de qubits
Simulateur IonQ Quantum	29 qubits
IonQ Aria 1	25 qubits
IonQ Forte 1	36 qubits
IonQ Forte Enterprise 1	36 qubits
PASQAL Emu-TN	100 qubits
PASQAL Fresnel1	100 qubits
Vérificateur de syntaxe Quantinuum H2-1	32 qubits
Vérificateur de syntaxe Quantinuum H2-2	32 qubits

Nom de la cible	Nombre de qubits
Émulateur Quantinuum H2-1	20 qubits
Émulateur Quantinuum H2-2	20 qubits
Quantinuum H2-1	20 qubits
Quantinuum H2-2	32 qubits
Rigetti Quantum Virtual Machine (QVM)	30 qubits
Rigetti Cepheus-1-108Q	108 qubits

Bientôt disponible pour Azure Quantum

Azure Quantum est une plateforme d'innovation. À mesure que les partenaires de matériel quantique dans l'écosystème Azure Quantum continuent de croître, vous pouvez explorer ces solutions matérielles quantiques à venir.

[🔍 Agrandir le tableau](#)

Provider	Description
	Les circuits supraconducteurs complets de Quantum Circuits offrent une rétroaction en temps réel qui permettent une correction d'erreur et des portes d'intrication indépendantes de tout encodage. Vous pouvez vous inscrire dès aujourd'hui pour la préversion privée d'Azure Quantum de QCI.

Last updated on 10/04/2026

Disponibilité globale des fournisseurs

Azure Quantum

La disponibilité des fournisseurs de calcul quantique IonQ, Pasqal, Quantinuum et Rigetti varie en fonction du pays/de la région où votre compte de facturation est inscrit. Sélectionnez l'onglet de chaque fournisseur pour afficher sa disponibilité par pays/région.

IonQ

IonQ offre un plan de paiement à l'utilisation via Azure Quantum. Pour plus d'informations sur les ressources IonQ sur Azure Quantum, consultez [le fournisseur IonQ et targets](#).

IonQ est disponible dans les pays/régions suivants :

- Armenia
- Australia
- Austria
- Belarus
- Belgium
- Bulgaria
- Canada
- Chile
- Colombia
- Croatia
- Cyprus
- Czechia
- Denmark
- Estonia
- Finland
- France
- Germany
- Greece
- Hungary
- Iceland
- India
- Indonesia

- Ireland
- Israel
- Italy
- Kenya
- Korea
- Latvia
- Liechtenstein
- Lithuania
- Luxembourg
- Malaysia
- Malta
- Monaco
- Netherlands
- Nouvelle-Zélande
- Nigeria
- Norway
- Poland
- Portugal
- Porto Rico
- Romania
- Arabie Saoudite
- Serbia
- Singapore
- Slovakia
- Slovenia
- Afrique du Sud
- Spain
- Sweden
- Switzerland
- Taiwan
- Thailand
- Türkiye
- Émirats arabes unis
- Royaume-Uni
- United States
- Vietnam

IonQ n'est pas disponible dans les pays/régions suivants :

- Afghanistan
- Albania
- Algeria
- Andorra
- Angola
- Argentina
- Azerbaijan
- Bahrain
- Bangladesh
- Barbados
- Belize
- Bermuda
- Bolivia
- Bosnie-Herzégovine
- Botswana
- Brazil
- Brunei
- Cap-Vert
- Cameroon
- Îles Caïmanes
- Costa Rica
- Côte d'Ivoire
- Curaçao
- République dominicaine
- Ecuador
- Egypt
- Salvador
- Ethiopia
- Féroé (îles)
- Fiji
- Georgia
- Ghana
- Guatemala
- Honduras
- Hong Kong, Région administrative spéciale

- Iraq
- Jamaica
- Japan
- Jordan
- Kazakhstan
- Kuwait
- Kyrgyzstan
- Lebanon
- Libya
- Macao, Région administrative spéciale
- Mauritius
- Mexico
- Moldova
- Mongolia
- Montenegro
- Morocco
- Namibia
- Nepal
- Nicaragua
- Macédoine du Nord
- Oman
- Pakistan
- Autorité palestinienne
- Panama
- Paraguay
- Peru
- Philippines
- Qatar
- Russia
- Rwanda
- Saint-Christophe et Nevis
- Senegal
- Sri Lanka
- Tajikistan
- Tanzania
- Trinité-et-Tobago
- Tunisia

- Turkmenistan
- Îles Vierges américaines
- Uganda
- Ukraine
- Uruguay
- Uzbekistan
- État de la Cité du Vatican
- Venezuela
- Yemen
- Zambia
- Zimbabwe

Last updated on 23/04/2026

Simulateurs quantiques backend provenant de fournisseurs quantiques

Cet article décrit les simulateurs principaux disponibles à partir de fournisseurs quantiques. Ces simulateurs sont disponibles pour tous les utilisateurs Azure Quantum et constituent un excellent moyen de tester vos Q# programmes avant de les exécuter sur un ordinateur quantique réel.

IonQ

IonQ fournit un simulateur idéalisé avec accélération GPU prenant en charge jusqu'à 29 qubits, en utilisant le même ensemble de portes que IonQ fournit sur son matériel quantique. Le simulateur est un excellent endroit pour prédéfinir les travaux avant de les exécuter sur un ordinateur quantique réel.

- Type de tâche : `Simulation`
- Format de données : `ionq.circuit.v1`
- ID cible : `ionq.simulator`
- Profil d'exécution cible : base QIR (représentation intermédiaire quantique)

Pour plus d'informations, consultez la page [du fournisseur IonQ](#) .

Pasqal

L'EMU_MPS de Pasqal est un backend Pulser qui simule cette dynamique avec des états de produit de matrice (MPS). Les Matrix Product States (MPS) ou Tensor Train (TT) sont une classe spécifique de réseaux de tenseurs qui fournissent une paramétrisation gérable des états quantiques.

EMU_MPS émulateur s'exécute sur un cluster de nœuds NVIDIA DGX, chacun équipé de GPU NVIDIA A100, ce qui permet l'émulation des processeurs quantiques de Pasqal. Il s'agit d'un outil clé pour prototyper et valider des programmes quantiques avant de les exécuter sur le QPU (unité de traitement quantique). Jusqu'à 80 qubits dans des tableaux 2D peuvent être émulés pour développer des applications industrielles et faire progresser la découverte scientifique.

- Type de tâche : `Simulation`
- Format de données : `application/json`
- ID cible : `pasqal.sim.emu-mps`

- Profil d'exécution cible : N/A

Pour plus d'informations, consultez la page [du fournisseur Pasqal](#) .

Quantinuum

Quantinuum fournit deux outils d'émulateur :

Vérificateurs de syntaxe : ces outils vérifient la syntaxe correcte, l'achèvement de la compilation et la compatibilité des machines en utilisant le même compilateur que l'ordinateur quantique auquel ils sont destinés. Il existe des vérificateurs de syntaxe pour les deux machines du modèle système H2.

- Type de tâche : `Simulation`
- Formats de données : `honeywell.openqasm.v1`, `honeywell.qir.v1`
- ID cible :
 - Vérificateur de syntaxe H2-1 : `quantinuum.sim.h2-1sc`
 - Vérificateur de syntaxe H2-2 : `quantinuum.sim.h2-2sc`
- Profil d'exécution cible : QIR Adaptive RI
- Tarification : Gratuit (\$0)

Émulateurs : ces outils contiennent un modèle physique détaillé et un modèle de bruit réaliste du matériel système H2 réel. Il existe des émulateurs pour les deux machines H2, ainsi qu'un émulateur Quantinuum basé sur le cloud.

- Type de tâche : `Simulation`
- Format de données : `honeywell.openqasm.v1`, `honeywell.qir.v1`
- ID cible :
 - Émulateur H2-1 : `quantinuum.sim.h2-1e`
 - H2-2 Emulator : `quantinuum.sim.h2-2e`
- Profil d'exécution cible : QIR Adaptive RI

L'émulateur Quantinuum est un émulateur basé sur la série H de modèle système disponible gratuitement sur la page [Code avec Microsoft Quantum](#) [↗] . Pour plus d'informations, consultez la page [Quantinuum Emulator](#) .

Pour plus d'informations sur tous les émulateurs Quantinuum, consultez la page [du fournisseur Quantinuum](#) .

Rigetti

Rigetti fournit leur machine virtuelle Quantum (QVM), un simulateur open source pour Quil. La cible QVM accepte un programme Quil en tant que texte et exécute ce programme sur QVM hébergé dans le cloud, retournant des résultats simulés.

- Type de tâche : `Simulation`
- Formats de données : `rigetti.quil.v1`, `rigetti.qir.v1`
- ID cible : `rigetti.sim.qvm`
- Profil d'exécution cible : base QIR
- Tarification : Gratuit (\$0)

Pour plus d'informations, consultez la [page du fournisseur Rigetti](#).

Last updated on 23/04/2026

Fournisseur IonQ

Les ordinateurs quantiques de IonQ effectuent des calculs en manipulant les états d'énergie très précis des ions Ytterbium avec des lasers. Les atomes sont des qubits de la nature : chaque qubit est identique dans et entre les programmes. Les opérations logiques peuvent également être effectuées sur n'importe quelle paire arbitraire de qubits, ce qui permet aux programmes quantiques complexes de rester inaltérés par la connectivité physique. Vous souhaitez en savoir plus ? Consultez [Présentation de la technologie des ordinateurs quantiques à ions piégés](#) de IonQ.

- Éditeur : [IonQ](#)
- ID du fournisseur : `ionq`

Les éléments suivants targets sont disponibles auprès de ce fournisseur :

[Agrandir le tableau](#)

Nom de la cible	ID de la cible	Nombre de qubits	Descriptif
Simulateur quantique	ionq.simulator	29 qubits	Simulateur idéalisé basé sur le cloud d'IonQ. Gratuit.
IonQ Aria 1	ionq.qpu.aria-1	25 qubits	IonQ's Aria 1 trapped-ion quantum computer.
IonQ Forte 1	ionq.qpu.forte-1	36 qubits	L'ordinateur quantique Forte 1 d'IonQ.
IonQ Forte Enterprise 1	ionq.qpu.forte-enterprise-1	36 qubits	L'ordinateur quantique à ions piégés Forte Enterprise 1 d'IonQ.

Les targets de IonQ correspondent à un profil **QIR Base**. Pour plus d'informations sur ce target profil et ses limitations, consultez [Présentation des target types de profils dans Azure Quantum](#).

Simulateur de quantum

Simulateur idéalisé accéléré par GPU prenant en charge jusqu'à 29 qubits, utilisant le même ensemble de portes qu'IonQ fournit sur son matériel quantique, ce qui constitue un excellent point de départ pour les tâches préliminaires avant leur exécution sur un ordinateur quantique réel.

- Type de tâche : `Simulation`
- Format de données : `ionq.circuit.v1`
- ID cible : `ionq.simulator`
- Profil d'exécution cible : [QIR Base](#)

Ordinateur quantum IonQ Aria

IonQ Aria est le fleuron des ordinateurs quantiques à ions piégés d’IonQ, avec un système de 25 qubits à reconfiguration dynamique. Pour plus d’informations, consultez [IonQ Aria \(ionq.com\)](https://ionq.com).

Important

Le dé-biaisage est activé par défaut sur les systèmes Aria, et les travaux soumis sont soumis à des tarifs basés sur le dé-biaisage. Pour plus d’informations sur la débiasie et la façon de désactiver/activer le service, consultez [Atténuation des](#) erreurs.

- Type de tâche : `Quantum Program`
- Format de données : `ionq.circuit.v1`
- ID cible : `ionq.qpu.aria-1`
- Profil d'exécution cible : [QIR Base](#)

 [Agrandir le tableau](#)

Nom du paramètre	Catégorie	Obligatoire	Descriptif
<code>shots</code>	Int	Non	Nombre de prises de vue expérimentales.

Synchronisation du système

 [Agrandir le tableau](#)

Mesure	Durée moyenne
T1	10-100 secondes
T2	1 s
Porte à un seul qubit	135 μ s
Porte à deux qubits	600 μ s

Fidélité du système

Opération	Fidélité moyenne
Porte à un seul qubit	99,95 % (courrier indésirable corrigé)
Porte à deux qubits	99,6 % (sans correction du spam)
POURRIEL*	99,61%

* Préparation de l'état et mesure (SPAM) : cette mesure détermine la précision avec laquelle un ordinateur quantique peut définir un qubit dans son état initial, puis mesurer le résultat à la fin.

IonQ Aria est disponible via le plan Paiement à l'utilisation. Pour plus d'informations, consultez la [tarification Azure Quantum](#).

Ordinateur quantique IonQ Forte

IonQ Forte est l'ordinateur quantique à ions piégés commercialisé par IonQ le plus performant. Avec un système configurable par logiciel de 36 qubits. Pour plus d'informations, consultez [IonQ Forte \(ionq.com\)](#) [↗](#).

📘 Important

Le débiaisement est activé par défaut sur le système Forte, et les travaux envoyés font l'objet de tarifs basés sur le débiaisement. Pour plus d'informations sur la débiasie et la façon de désactiver/activer le service, consultez [Atténuation des](#) erreurs.

- Type de tâche : `Quantum Program`
- Format de données : `ionq.circuit.v1`
- ID cible : `ionq.qpu.forte-1`
- Profil d'exécution cible : [QIR Base](#)

Nom du paramètre	Catégorie	Obligatoire	Descriptif
<code>shots</code>	Int	Non	Nombre d'essais expérimentaux.

Ordinateur quantique IonQ Forte Enterprise

IonQ Forte Enterprise est l'ordinateur quantique à ions piégés le plus performant d'IonQ disponible sur le marché. Avec un système configurable par logiciel de 36 qubits. Pour plus d'informations, consultez [IonQ Forte Enterprise \(ionq.com\)](https://ionq.com).

Forte Enterprise est une version des systèmes de classe Forte qui a été adapté et renforcé pour le déploiement dans un environnement de centre de données standard, ce qui le rend plus adapté aux tâches orientées entreprise et orientées production. Le matériel et les performances de calcul quantique de base sont les mêmes. La principale différence entre IonQ Forte et IonQ Forte Enterprise réside dans leur déploiement et leurs cas d'usage prévus, et non dans leurs spécifications de performances principales. Bien que les deux systèmes disposent des mêmes métriques hautes performances, Forte Enterprise est spécifiquement conçu pour l'intégration dans un environnement de centre de données.

 Important

Le débiaisage est activé par défaut sur le système Forte Enterprise, et les tâches envoyées sont soumises à une tarification basée sur le débiaisage. Pour plus d'informations sur la débiasie et la façon de désactiver/activer le service, consultez [Atténuation des erreurs](#).

- Type de tâche : `Quantum Program`
- Format de données : `ionq.circuit.v1`
- ID cible : `ionq.qpu.forte-enterprise-1`
- Profil d'exécution cible : [QIR Base](#)

 [Agrandir le tableau](#)

Nom du paramètre	Catégorie	Obligatoire	Descriptif
<code>shots</code>	Int	Non	Nombre de prises de vue expérimentales.

Format d'entrée

En Q#, la sortie d'une mesure quantique est une valeur de type `Result`, qui ne peut prendre que les valeurs `Zero` et `One`. Lorsque vous définissez une opération Q#, elle ne peut être envoyée qu'au matériel IonQ si le type de retour est une collection de `Result`s, autrement dit, si la sortie de l'opération est le résultat d'une mesure quantique. La raison en est que IonQ génère un histogramme à partir des valeurs retournées, de sorte qu'il limite le type de retour pour `Result` simplifier la création de cet histogramme.

IonQ targets correspond au [QIR Base profile](#). Ce profil ne peut pas exécuter d'opérations quantiques qui nécessitent l'utilisation des résultats des mesures qubit pour contrôler le flux du programme.

Format de sortie

Lorsque vous soumettez un programme quantique au simulateur IonQ, il retourne l'histogramme créé par les mesures. Le simulateur IonQ n'effectue pas d'échantillonnage de la distribution de probabilité créée par un programme quantique, mais renvoie à la place la distribution mise à l'échelle en fonction du nombre de tirs. Cela est le plus évident lorsque vous soumettez un circuit de tir unique. Vous verrez plusieurs résultats de mesure dans l'histogramme pour une seule capture. Ce comportement est inhérent au simulateur IonQ, tandis que le QPU IonQ exécute réellement le programme et agrège les résultats.

Fonctionnalités supplémentaires

Des fonctionnalités supplémentaires prises en charge par le matériel IonQ sont répertoriées ici.

 Agrandir le tableau

Capacité	Descriptif
Atténuation des erreurs	Utiliser la débiaison pour réduire le bruit et optimiser les performances algorithmiques sur le matériel IonQ
Prise en charge des portes natives	Définir et exécuter des circuits directement sur les portes natives du matériel IonQ
Simulation de modèle de bruit	Simulez le profil de bruit que les circuits rencontreront lorsque vous les exécutez sur différents matériels IonQ.

Les utilisateurs peuvent tirer parti de ces capacités supplémentaires via des paramètres de transmission dans les fournisseurs Azure Quantum pour Q# et Qiskit.

Atténuation des erreurs

IonQ offre la possibilité d'activer l'atténuation des erreurs quantiques lorsque vous envoyez des travaux au matériel IonQ. L'atténuation des erreurs est un processus au niveau du compilateur qui s'exécute et exécute plusieurs variantes symétriques d'un circuit, puis agrège les résultats tout en réduisant l'impact des erreurs matérielles et de la décohérence qubit. Contrairement aux techniques de correction d'erreurs quantiques, l'atténuation des erreurs ne nécessite pas une surcharge importante en portes et qubits.

Le débiaisage est le processus de création de légères variations d'un circuit donné qui doit être identique sur une machine sans bruit idéale, à l'aide de techniques telles que différentes affectations de qubits, de décompositions de porte et de solutions d'impulsions, puis d'exécution de ces variations.

La netteté et la moyenne sont des options permettant d'agréger les résultats des variations. Le calcul de la moyenne repose également sur tous les résultats des variations, tandis que l'accentuation filtre les résultats erronés et peut être plus fiable selon certains types d'algorithmes.

Pour plus d'informations, consultez [Réduction des biais et accentuation](#). Pour connaître les tarifs de l'atténuation des erreurs, consultez [les tarifs d'IonQ](#).

Activation de l'atténuation des erreurs

ⓘ Remarque

Le dé-biaisage est activé par défaut sur les systèmes Aria et Forte.

Sur Azure Quantum, l'atténuation des erreurs peut être activée ou désactivée pour les travaux envoyés avec Q# ou Qiskit.

Pour activer l'atténuation des erreurs, ajoutez un paramètre facultatif pour l'ordinateur target :

Python

```
option_params = {
    "error-mitigation": {
        "debias": True
    }
}
```

Pour désactiver l'atténuation des erreurs, définissez le paramètre sur `False`:

Python

```
option_params = {
    "error-mitigation": {
        "debias": False
    }
}
```

ⓘ Remarque

Si vous utilisez également la simulation de modèle de bruit d'IonQ, ces paramètres peuvent être inclus ici, par exemple :

Python

```
option_params = {
    "error-mitigation": {
        "debias": False
    },
    "noise": {
        "model": "aria-1",
        "seed": 100
    }
}
```

Pour plus d'informations, consultez [simulation](#) de modèle de bruit.

Exécution d'un travail sur Azure Quantum avec atténuation des erreurs

Cet exemple utilise un générateur de nombres aléatoires simple.

Tout d'abord, importez les packages requis et lancez le profil de base :

Python

```
from qdk import qsharp
from qdk.azure import Workspace
qsharp.init(target_profile=qsharp.TargetProfile.Base)
```

Ensuite, définissez la fonction.

Q#

```
%%qsharp
import Std.Measurement.*;
import Std.Arrays.*;
import Std.Convert.*;

operation GenerateRandomBit() : Result {
    use target = Qubit();

    // Apply an H-gate and measure.
    H(target);
    return M(target);
}
```

and compile the `operation`:

```
```python
MyProgram = qsharp.compile("GenerateRandomBit()")
```

Connectez-vous à Azure Quantum, sélectionnez la target machine et configurez les paramètres de bruit pour l'émulateur :

#### Python

```
MyWorkspace = Workspace(resource_id = "")

MyTarget = MyWorkspace.get_targets("ionq.qpu.aria-1")
```

Spécifier la `error-mitigation` configuration

#### Python

```
option_params = {
 "error-mitigation": {
 "debias": True
 }
}
```

Transmettez la configuration d'atténuation des erreurs lors de l'envoi du travail :

#### Python

```
job = MyTarget.submit(MyProgram, "Experiment with error mitigation", shots = 10,
input_params = option_params)
job.get_results()
```

Dans Qiskit, vous transmettez les paramètres facultatifs à la configuration de l'ordinateur target avant d'envoyer le travail :

#### Python

```
circuit.name = "Single qubit random - Debias: True"
backend.options.update_options(**option_params)
job = backend.run(circuit, shots=500)
```

### ⓘ Remarque

Si vous ne passez pas le `error-mitigation` paramètre, l'ordinateur target utilise son paramètre par défaut, qui est activé pour les systèmes Aria et Forte.

## Prise en charge et utilisation des portes natives

Par défaut, IonQ vous permet de spécifier un circuit quantique utilisant un ensemble abstrait de portes quantiques appelées `qis`, ce qui apporte flexibilité et portabilité pendant l'écriture d'un algorithme sans devoir se soucier de l'optimisation du matériel.

Cependant, dans certains cas d'usage avancés, vous pouvez souhaiter définir un circuit directement sur des portes natives de façon à vous rapprocher du matériel et contourner l'optimisation. L'ensemble de portes natives est l'ensemble de portes quantiques qui sont exécutées physiquement dans le processeur quantique, et elles mappent le circuit à celles-ci pendant l'exécution.

Pour plus d'informations, consultez [Bien démarrer avec les portes natives \(ionq.com\)](https://ionq.com).

Pour utiliser l'ensemble de portes natives lors de l'envoi de travaux Qiskit à Azure Quantum, spécifiez le paramètre `gateset` au moment d'initialiser le back-end, comme dans l'exemple ci-dessous :

### Python

```
Here 'provider' is an instance of AzureQuantumProvider
backend = provider.get_backend("ionq.qpu.aria-1", gateset="native")
```

[🔗](#) Agrandir le tableau

Nom du paramètre	Catégorie	Obligatoire	Descriptif
<code>gateset</code>	ficelle	Non	Spécifie l'ensemble de portes qui servira à définir un circuit. La valeur <code>qis</code> correspond aux portes abstraites (comportement par défaut) et <code>native</code> aux <a href="#">portes natives du matériel IonQ</a> .

Pour plus d'informations sur les travaux Qiskit, consultez [Envoyer des travaux avec le package QDK Python](#).

## Simulation de modèle de bruit

Même le meilleur du matériel quantique d’aujourd’hui a un bruit inhérent. Connaître les caractéristiques du bruit de votre target système peut vous aider à affiner vos algorithmes et à obtenir une prédiction plus réaliste des résultats lors de l’exécution du circuit sur du matériel réel. IonQ fournit une simulation de modèle de bruit qui introduit le bruit dans le circuit à l’aide d’une « empreinte digitale sonore » spécifique au target matériel. Pour plus d’informations, consultez [Prise en main de la simulation](#) de modèle de bruit matériel.

## Paramètres du modèle de bruit

 Agrandir le tableau

Nom du paramètre	Valeurs	Descriptif
noise	model, seed	Active la simulation du modèle de bruit
model	ideal, aria-1	Spécifie le modèle de bruit pour le target matériel.    - ideal - Aucun bruit n’est introduit dans le circuit. Il s’agit de la même chose que de ne pas activer la simulation de bruit.   - aria-1 - Utilise le modèle de bruit pour l’ordinateur quantique IonQ Aria.
seed	Entier compris entre 1 et $2^{31}$ (2 147 483 648)	Vous permet de spécifier une valeur de départ pour le bruit pseudo-aléatoire et l’échantillonnage par tir, ce qui crée des résultats bruyants reproductibles. Si le paramètre n’est pas spécifié, une valeur aléatoire seed est créée.

## Prise de conscience des coups

La simulation de modèle de bruit est sensible aux tirs, c’est-à-dire qu’elle échantillonne les mesures de l’état de sortie en fonction du nombre de tirs fournis. Dans Azure Quantum, le shots paramètre est envoyé avec le travail et est requis pour aria-1 les modèles de bruit. Si aucune valeur n’est shot spécifiée, une valeur par défaut est 1000 utilisée. Si le ideal modèle de bruit est utilisé, le shots paramètre est ignoré.

## Capacité qubit

Bien que le ideal modèle de bruit vous permet de simuler jusqu’à 29 qubits avec le simulateur quantique IonQ, les modèles de bruit spécifiques au matériel sont limités à la capacité qubit réelle du target matériel, qui est de 25 qubits pour le aria-1 modèle de bruit.

## Activer la simulation de modèle de bruit

Sur Azure Quantum, la simulation de modèle de bruit peut être activée ou désactivée pour les travaux soumis avec Q# ou qiskit.

Pour activer la simulation de modèle de bruit, ajoutez un paramètre facultatif pour la target machine, par exemple :

#### Python

```
option_params = {
 "noise": {
 "model": "aria-1", # targets the Aria quantum computer
 "seed" : 1000 # If seed isn't specified, a random value is used
 }
}
```

#### ❗ Remarque

Si vous utilisez également l'atténuation des erreurs d'IonQ, ces paramètres peuvent être inclus ici, par exemple :

#### Python

```
option_params = {
 "error-mitigation": {
 "debias": False
 },
 "noise": {
 "model": "aria-1",
 "seed": 1000
 }
}
```

Pour plus d'informations, consultez [Atténuation des erreurs](#).

## Exécuter un travail avec la simulation de modèle de bruit

Vous pouvez utiliser le même exemple de programme présenté précédemment dans l'atténuation [des](#) erreurs et ajouter ou remplacer la configuration du modèle de bruit dans `option_params`;

#### Python

```
option_params = {
 "error-mitigation": {
```

```

 "debias": True
 },
 "noise": {
 "model": "aria",
 "seed": 1000
 }
}

```

Ensuite, transmettez les paramètres facultatifs lorsque vous envoyez le travail :

#### Python

```

job = MyTarget.submit(MyProgram, "Experiment with noise model simulation", shots = 10,
input_params = option_params)
job.get_results()

```

Dans Qiskit, vous transmettez les paramètres facultatifs à la configuration de l'ordinateur target avant d'envoyer le travail :

#### Python

```

circuit.name = "Single qubit random - Debias: True"
backend.options.update_options(**option_params)
job = backend.run(circuit, shots=500)

```

## Tarification

Pour voir le plan de facturation d'IonQ, visitez la [page des prix d'Azure Quantum](#).

## Limites et quotas

Les quotas IonQ sont suivis en fonction de l'unité d'utilisation du QPU, appelée qubit-gate-shot (QGS). L'utilisation de ressources est créditée sur votre compte.

Chaque programme quantique, composé de  $N$  portes logiques quantiques d'un ou plusieurs qubits, est exécuté un certain nombre de fois. Le nombre de tirs de grille se calcule à l'aide de la formule suivante :

$$QGS = N \cdot C$$

où :

- $N$  est le nombre de portes à un ou deux qubits soumises
- $C$  est le nombre de shots d'exécution demandés



# Statut d'IonQ

Pour plus d'informations sur les délais de traitement des travaux QPU IonQ, consultez [la page](#) d'état IonQ.

## Meilleurs pratiques et graphique de connectivité IonQ

Pour voir les bonnes pratiques recommandées pour le QPU IonQ, consultez les [bonnes pratiques IonQ \(ionq.com\)](#).

---

Last updated on 29/08/2026

# Stratégie de support pour IonQ dans Azure Quantum

18/09/2025

Cet article décrit la politique de support Microsoft qui s'applique lorsque vous utilisez le fournisseur IonQ dans Azure Quantum. L'article s'applique à l'un targets des sous-fournisseurs.

Si vous utilisez le fournisseur IonQ et si vous avez rencontré des problèmes inattendus que vous ne pouvez pas résoudre vous-même, vous pouvez contacter l'équipe du support technique Azure pour obtenir de l'aide en [créant un dossier de support Azure](#).

Toutefois, dans certaines situations, l'équipe du support Azure devra vous rediriger vers l'équipe du support technique de IonQ. Vous pouvez également recevoir une réponse plus rapide en contactant IonQ directement sur le site web [du support IonQ](#).

## Forum aux questions

### Q : Quelle est la stratégie de support pour l'utilisation des offres IonQ via Azure Quantum ?

Microsoft fournit une prise en charge de la plateforme Azure et du service Azure Quantum uniquement. Prise en charge du matériel, des simulateurs et d'autres produits IonQ et targets sera fournie directement par IonQ. Pour plus d'informations sur le support Azure, [consultez les plans de support Azure](#). Pour plus d'informations sur le support IonQ, [consultez le site web de support IonQ](#).

### Q : Que se passe-t-il si je soulève un problème de support avec l'équipe du support Azure et qu'il s'avère qu'un fournisseur tiers (comme IonQ) doit encore se pencher sur le problème ?

L'ingénieur du support crée un package de redirection pour vous. Il s'agit d'un document PDF qui contient des informations sur votre cas que vous pouvez fournir à l'équipe de support IonQ. L'ingénieur du support vous apportera également une assistance et des conseils sur la façon d'accéder à IonQ pour poursuivre la résolution des problèmes. Pour plus d'informations, consultez [la page d'état IonQ](#).

## **Q : Que se passe-t-il si je soulève un problème de support avec l'équipe IonQ et qu'il y a un problème avec le service Azure Quantum ?**

L'équipe de support Honeywell vous aidera à contacter Microsoft et vous fournira un package de redirection. Ce package est un document PDF que vous pouvez utiliser pour poursuivre votre demande de support auprès de l'équipe de support Azure.

## **Exclusion de responsabilité de tiers**

Les produits tiers mentionnés dans le présent article sont fabriqués par des sociétés indépendantes de Microsoft. Microsoft exclut toute garantie, implicite ou autre, concernant les performances ou la fiabilité de ces produits.

## **Exclusion de responsabilité sur les coordonnées externes**

Microsoft fournit des informations de contacts externes afin de vous aider à obtenir un support technique sur ce sujet. Ces informations de contact peuvent changer sans préavis. Microsoft ne garantit pas l'exactitude des informations de contact tierces.

# Fournisseur Pasqal

Les ordinateurs quantiques de Pasqal contrôlent les atomes neutres avec des pinces optiques, en utilisant la lumière de laser pour manipuler des registres quantiques avec jusqu'à cent qubits.

- Éditeur : [Pasqal](#)
- ID du fournisseur : pasqal

Les cibles suivantes sont disponibles auprès de ce fournisseur :

 Agrandir le tableau

Nom de la cible	ID de la cible	Nombre de qubits	Descriptif
<a href="#">EMU_SV</a>	pasqal.sim.emu-sv	25 qubits 1D et réseaux 2D	Les émulateurs sont des back-ends conçus pour émuler la dynamique des tableaux programmables d'atomes neutres.
<a href="#">EMU_MPS</a>	pasqal.sim.emu-mps	80 qubits 1D et réseaux 2D	Les émulateurs sont des back-ends conçus pour émuler la dynamique des tableaux programmables d'atomes neutres.
<a href="#">EMU_FREE</a>	pasqal.sim.emu-free	12 qubits 1D et réseaux 2D	Les émulateurs sont des back-ends conçus pour émuler la dynamique des tableaux programmables d'atomes neutres.
<a href="#">FRESNEL</a>	pasqal.qpu.fresnel	100 qubits	FRESNEL est une unité de traitement quantique à atomes neutres - génération Orion Beta.
<a href="#">FRESNEL_CAN1</a>	pasqal.qpu.fresnel-can1	100 qubits	FRESNEL_CAN1 est un QPU à atomes neutres en matériel - de la génération Orion Beta.

## EMU\_SV

Les émulateurs sont des back-ends conçus pour émuler la dynamique des tableaux programmables d'atomes neutres.

EMU\_SV est un back-end Pulser qui émule ces dynamiques à l'aide de vecteurs d'état (SV). La représentation vectorielle d'état fournit une description complète de l'état quantique, ce qui permet des simulations hautement précises avec l'accélération GPU si elle est activée.

Pour plus d'informations, consultez la [documentation pasqal EMU\\_MPS](#)

- Type de tâche : `Simulation`
- Format de données : `application/json`
- ID cible : `pasqal.sim.emu-sv`

## EMU\_MPS

Les émulateurs sont des back-ends conçus pour émuler la dynamique des tableaux programmables d'atomes neutres.

EMU\_MPS est un module back-end Pulser qui émule cette dynamique avec des états de produit de matrice (MPS). Les états de produit de matrice (MPS) ou trains de tenseurs (TT) sont une classe spécifique de réseaux de tenseurs qui fournissent une paramétrisation praticable d'états quantiques.

Pour plus d'informations, consultez la [documentation pasqal EMU\\_MPS](#) ↗

- Type de tâche : `Simulation`
- Format de données : `application/json`
- ID cible : `pasqal.sim.emu-mps`

## EMU\_FREE

Les émulateurs sont des back-ends conçus pour émuler la dynamique des tableaux programmables d'atomes neutres.

EMU\_FREE est un petit back-end Pulser sur lequel vous pouvez émuler de petits systèmes (pas plus de 12 qubits).

- Type de tâche : `Simulation`
- Format de données : `application/json`
- ID cible : `pasqal.sim.emu-free`

## FRESNEL

FRESNEL est un QPU (Unité de traitement quantique) à atomes neutres, indépendant du matériel - Génération Orion Beta. Il s'agit essentiellement d'une machine optique, utilisant la lumière pour capturer et manipuler des tableaux d'atomes de rubidium.

En utilisant des tweezers optiques, nous pouvons assembler un registre quantique réglable pour les atomes qui serviront de base de calcul. Pour la machine Pasqal, un seul atome piégé correspond à un qubit.

- Type de tâche : Quantum program
- Format de données : application/json
- ID cible : pasqal.qpu.fresnel

## FRESNEL\_CAN1

FRESNEL\_CAN1 est un QPU de atomes neutres matériels (unité de traitement quantique) - Génération orion bêta.

Il s'agit essentiellement d'une machine optique, utilisant la lumière pour piéger et manipuler des ensembles d'atomes de rubidium.

En utilisant des tweezers optiques, nous pouvons assembler un registre quantique réglable pour les atomes qui serviront de base de calcul. Pour la machine Pasqal, un seul atome piégé correspond à un qubit.

- Type de tâche : Quantum program
- Format de données : application/json
- ID cible : pasqal.qpu.fresnel-can1

## Pulser SDK

Dans pasqal QPU, les atomes individuels sont piégés à des positions bien définies dans des lattiques 1D ou 2D. [Pulser](#) est un framework pour la composition, la simulation et l'exécution de séquences d'impulsions sur des atomes neutres quantiques. Pour plus d'informations, consultez [la documentation](#) Pulser.

Pour installer les packages du Kit de développement logiciel (SDK) Pulser, exécutez le code suivant :

### Python

```
!pip -q install pulser-simulation #Only for using the local Qutip emulator included in Pulser
!pip -q install pulser-core
```

## Format des données d'entrée

Les cibles pasqal acceptent les fichiers JSON comme format de données d'entrée. Pour soumettre les séquences d'impulsions, vous devez convertir les objets Pulser en une chaîne JSON qui peut être utilisée comme données d'entrée.

## Python

```
Convert the sequence to a JSON string
def prepare_input_data(seq):
 input_data = {}
 input_data["sequence_builder"] = json.loads(seq.to_abstract_repr())
 to_send = json.dumps(input_data)
 #print(json.dumps(input_data, indent=4, sort_keys=True))
 return to_send
```

Avant d'envoyer votre travail quantique à Pasqal, vous devez définir des paramètres de format de données d'entrée et de sortie appropriés. Par exemple, le code suivant définit le format de données d'entrée sur `pasqal.pulser.v1` et le format `pasqal.pulser-results.v1` de données de sortie sur .

## Python

```
Submit the job with proper input and output data formats
def submit_job(target, seq):
 job = target.submit(
 input_data=prepare_input_data(seq), # Take the JSON string previously defined
 as input data
 input_data_format="pasqal.pulser.v1",
 output_data_format="pasqal.pulser-results.v1",
 name="Pasqal sequence",
 shots=100 # Number of shots
)
```

Pour plus d'informations sur la façon d'envoyer des travaux au fournisseur Pasqal, consultez [Envoyer un circuit à Pasqal à l'aide du Kit de développement logiciel \(SDK\) Pulser](#).

## Tarification

Pour voir le plan de facturation Pasqal, consultez la [tarification d'Azure Quantum](#).

## Limites et quotas

Les quotas pasqal s'appliquent à l'utilisation de l'émulateur et du QPU et peuvent être augmentés avec un ticket de support. Pour afficher vos limites et quotas actuels, accédez à la section **Opérations** et sélectionnez le panneau **Quotas** de votre espace de travail sur le [portail Azure](#)  . Pour plus d'informations, consultez les quotas Azure Quantum .





# Stratégie de support pour Pasqal dans Azure Quantum

Cet article décrit la stratégie de support Microsoft qui s'applique lorsque vous utilisez le fournisseur Pasqal dans Azure Quantum. L'article s'applique à l'une des [cibles](#) proposées par Pasqal.

## Support Azure

En général, les problèmes liés à l'utilisation de Pasqal sur Microsoft Azure Quantum sont mieux traités avec l'équipe de support Azure Quantum directement en [créant un cas de support Azure](#). Cette équipe dispose de tous les détails nécessaires pour isoler le problème que vous rencontrez et diriger le rapport vers le bon point de contact. L'équipe de support Azure a également les moyens d'atteindre le support Pasqal en dehors des heures d'ouverture pour les problèmes les plus urgents.

## Prise en charge de Pasqal

Dans certains cas, l'équipe de support Azure doit vous rediriger vers l'équipe de support technique de Pasqal. Vous pouvez recevoir une réponse plus rapide en accédant directement à [Pasqal](#).

## Chronologie de prise en charge

Pasqal tentera toujours de résoudre les problèmes liés à l'intégration de Microsoft Azure aussi rapidement que pratique. Certains problèmes prendront plus de temps à résoudre que d'autres, mais les tableaux ci-dessous indiquent le temps que vous pouvez attendre pour une réponse initiale de Pasqal une fois qu'un problème est signalé.

[🔍](#) Agrandir le tableau

Sévérité	Définition	Réponse SLA
Élevé	Défaillance de la connectivité, dégradation complète des performances	Un jour ouvré
Moyenne	Dégradation des performances	Deux jours ouvrables
Faible	Problèmes et questions liés à l'application	Trois jours ouvrables

---

Last updated on 23/04/2026

# Fournisseur Quantinuum

02/10/2025

Quantinuum donne accès à des systèmes à ions piégés offrant des qubits haute fidélité entièrement connectés et la possibilité d’effectuer des mesures à mi-circuit.

- Éditeur : [Quantinuum](#)
- ID du fournisseur : `quantinuum`

## Targets

### Avertissement

Quantinuum mettra hors service le matériel H1-1 le 15 octobre 2025. Si vous utilisez un plan qui utilise le système H1, basculez vers un plan qui prend en charge le matériel H2.

Les éléments suivants targets sont disponibles auprès de ce fournisseur :

 Agrandir le tableau

Nom de la cible	ID de la cible	Nombre de qubits	Description
<a href="#">Vérificateur de syntaxe H2-1</a>	quantinuum.sim.h2-1sc	56 qubits	Utilisez-le pour valider les programmes quantiques par rapport au compilateur H2-1 avant de les soumettre à du matériel ou des émulateurs sur la plateforme de Quantinuum. Gratuit.
<a href="#">Vérificateur de syntaxe H2-2</a>	quantinuum.sim.h2-2sc	56 qubits	Utilisez cette option pour valider les programmes quantiques par rapport au compilateur H2-2 avant de les soumettre à du matériel ou des émulateurs sur la plateforme de Quantinuum. Gratuit.
<a href="#">Émulateur H2-1</a>	quantinuum.sim.h2-1e	56/32 qubits	Utilise un modèle physique réaliste et un modèle de bruit de H2-1. La simulation à 56 qubits n’est disponible que comme simulation du stabilisateur
<a href="#">Émulateur H2-2</a>	quantinuum.sim.h2-2e	56/32 qubits	Utilise un modèle physique réaliste et un modèle de bruit H2-2. La simulation à 56 qubits n’est disponible que comme simulation du stabilisateur
<a href="#">H2-1</a>	quantinuum.qpu.h2-1	56 qubits	Quantinuum H2-1 appareil à ions piégés.

Nom de la cible	ID de la cible	Nombre de qubits	Description
H2-2	quantinuum.qpu.h2-1	56 qubits	Appareil d'ion piégé H2-2 de Quantinuum.

Les targets de Quantinuum correspondent à un profil **QIR Adaptive RI**. Pour plus d'informations sur ce target profil et ses limitations, consultez [Présentation des target types de profils dans Azure Quantum](#).

Tous les targets de Quantinuum supportent désormais les circuits hybrides intégrés. Pour plus d'informations sur l'envoi de travaux hybrides intégrés, consultez [l'informatique](#) hybride intégrée.

Pour commencer à utiliser le fournisseur Quantinuum sur Azure Quantum, consultez [Prise en main de Q# et d'un notebook Azure Quantum](#).

### Conseil

Les travaux quantiques soumis sous une session ont **un accès exclusif** au matériel Quantinuum tant que vous placez les travaux dans la file d'attente à chaque délai d'une minute entre eux. Après cela, tout travail est accepté et géré avec la logique de mise en file d'attente et de hiérarchisation standard. Pour plus d'informations, consultez [les sessions dans Azure Quantum](#).

## Vérificateurs de syntaxe

Nous recommandons aux utilisateurs de commencer par valider leur code à l'aide de notre vérificateur de syntaxe. Il s'agit d'un outil qui permet de vérifier la syntaxe, l'achèvement de la compilation et la compatibilité de la machine. Les vérificateurs de syntaxe utilisent la même capacité calcul que l'ordinateur quantique qu'ils target. Par exemple, le vérificateur de syntaxe H2-1 utilise le même compilateur que H2-1. La pile de compilation complète est exécutée à l'exception des opérations quantiques réelles. Si le code est compilé, le vérificateur syntaxique renvoie un statut `success` et un résultat de tous les 0. Si le code ne se compile pas, le vérificateur syntaxique renvoie un état d'échec et donne l'erreur renvoyée pour aider les utilisateurs à déboguer la syntaxe de leur circuit. Le vérificateur de syntaxe permet aux développeurs de valider leur code à tout moment, même lorsque les machines sont hors connexion.

- Type de tâche : `Simulation`
- Formats de données : `honeywell.openqasm.v1`, `honeywell.qir.v1`

- ID cible :
  - Vérificateur de syntaxe H2-1 : `quantinuum.sim.h2-1sc`
  - Vérificateur de syntaxe H2-2 : `quantinuum.sim.h2-2sc`
- Profil d'exécution cible : [QIR Adaptive RI](#)

L'utilisation des vérificateurs de syntaxe est gratuite.

## Émulateur quantinuum (basé sur le cloud)

L'émulateur Quantinuum est disponible gratuitement sur la page [Code avec Azure Quantum](#) du site web Azure Quantum, où vous pouvez écrire du code Q# et envoyer vos travaux à l'émulateur Quantinuum sans compte Azure. L'émulateur Quantinuum est un émulateur quantique basé sur un vecteur d'état qui utilise un modèle de bruit physique réaliste et des paramètres d'erreur généralisés basés sur les performances typiques d'un [ordinateur quantique de modèle système H2](#). La simulation quantique effectuée est la même que l'[émulateur System Model H2](#), mais la routine d'optimisation du circuit classique est réduite pour augmenter le débit. La prise en charge de [l'informatique](#) hybride intégrée est prévue pour une date ultérieure.

## Émulateur du modèle de système H2

Après avoir validé la syntaxe de son code avec le vérificateur de syntaxe H2-1, les utilisateurs peuvent tirer parti de l'émulateur System Model H2 de Quantinuum, un outil d'émulation qui contient un modèle physique détaillé et un modèle de bruit réaliste du matériel H2 du modèle système réel. Vous trouverez plus d'informations sur le modèle de bruit dans la feuille *de données produit de l'émulateur System Model H2* dans la [page System Model H2](#). L'émulateur System Model H2 utilise une API identique pour la soumission de travaux comme le matériel System Model H2, facilitant une transition fluide de l'émulation au matériel. Pour optimiser la productivité et raccourcir le temps de développement, l'émulateur H2 est disponible même si le matériel est hors connexion.

- Type de tâche : `Simulation`
- Format de données : `quantinuum.openqasm.v1`
- ID cible :
  - Émulateur H2-1 : `quantinuum.sim.h2-1e`
  - H2-2 Emulator : `quantinuum.sim.h2-2e`
- Profil d'exécution cible : [QIR Adaptive RI](#)

L'utilisation de l'émulateur H2 du modèle système est proposée gratuitement avec un abonnement matériel. Pour plus de détails, consultez [Tarifs Azure Quantum](#).

# Modèle système H2

Le quantinuum System Model H2 génération d’ordinateurs quantiques, alimenté par Honeywell, est composé d’un appareil à couplage de charge quantique (QCCD) avec deux sections linéaires connectées et possède actuellement 1 machine, le H2-1. Vous trouverez plus d’informations dans la *fiche technique produit du modèle Système H2* sur la [page du modèle Système H2](#) . Les utilisateurs sont encouragés à tester la compatibilité de leur code en envoyant des travaux à un [vérificateur de syntaxe](#) et à [l’émulateur System Model H2](#) avant de les soumettre aux target machines.

Si un utilisateur envoie un travail à l’ordinateur H2-1 et que l’ordinateur H2-1 n’est pas disponible, le travail reste dans la file d’attente de cet ordinateur jusqu’à ce que l’ordinateur soit disponible.

Le matériel System Model H2 est mis à niveau en continu tout au long du cycle de vie du produit. Les utilisateurs ont accès au matériel le plus récent, le plus avancé et le plus performant qui soit.

- Type de tâche : Quantum Program
- Format de données : quantinuum.openqasm.v1
- ID cible :
  - H2-1 : quantinuum.qpu.h2-1
  - H2-2 : quantinuum.qpu.h2-2
- Profil d’exécution cible : [QIR Adaptive RI](#)

## Spécifications techniques du modèle système H2

Vous trouverez des détails techniques pour le modèle système H2 dans les feuilles de données de produit de Quantinuum sur la page [System Model H2](#)  , ainsi que des liens vers la spécification Quantinuum et les référentiels de données de volume quantique et comment citer l’utilisation des systèmes Quantinuum.

## Fonctionnalités supplémentaires

Des fonctionnalités supplémentaires disponibles via l’API Quantinuum sont répertoriées ici.

 [Agrandir le tableau](#)

Capability	Description
<a href="#">Mesure et réinitialisation à mi-</a>	Mesurer les qubits au milieu d’un circuit et les réutiliser

Capability	Description
circuit (MCMR)	
Portes ZZ à angle arbitraire	Effectuer directement des rotations de portes à angle arbitraire de 2 qubits
Porte d'intrication SU(4) générale	Effectuer directement des rotations de portes à angle arbitraire de 2 qubits
Paramètres de bruit de l'émulateur	Expérimenter avec les paramètres de bruit utilisés dans les émulateurs Quantinuum
Optimisations TKET dans la pile Quantinuum	Expérimentez avec l'activation de différents niveaux d'optimisations TKET dans la pile Quantinuum.

Les utilisateurs peuvent tirer parti de ces fonctionnalités supplémentaires via des fonctions de circuit ou des paramètres pass-through dans les fournisseurs Azure Quantum Q# et Qiskit.

## Mesure et réinitialisation du circuit intermédiaire

La mesure et la réinitialisation du circuit intermédiaire (MCMR) permettent aux utilisateurs de mesurer les qubits au milieu d'un circuit et de les réinitialiser. Cela permet la fonctionnalité de correction des erreurs quantiques ainsi que la possibilité de réutiliser des qubits au sein du circuit.

En raison de la structure interne des qubits à ions piégés, une mesure à mi-circuit peut laisser le qubit dans un état non calculable. Toutes les mesures de circuit intermédiaire doivent être suivies d'une réinitialisation si le qubit doit être utilisé à nouveau dans ce circuit. Les exemples de code suivants illustrent cela.

Lorsqu'un sous-ensemble de qubits est mesuré au milieu du circuit, les informations classiques de ces mesures peuvent être utilisées pour conditionner les éléments futurs du circuit. Les exemples mettent également en évidence cette utilisation.

Pour plus d'informations sur MCMR dans les systèmes Quantinuum, consultez la page [MCMR](#) sur [Quantinuum Docs](#).

### MCMR avec le fournisseur Q#

Dans Q#, la `MResetZ` fonction peut être utilisée à la fois pour mesurer un qubit et la réinitialiser. Pour plus d'informations sur cette fonction, consultez [MResetZ](#) dans la documentation Q#.

Q#

```

%%qsharp
import Std.Measurement.*;

operation ContinueComputationAfterReset() : Result[] {
 // Set up circuit with 2 qubits
 use qubits = Qubit[2];

 // Perform Bell Test
 H(qubits[0]);
 CNOT(qubits[0], qubits[1]);

 // Measure Qubit 1 and reset it
 let res1 = MResetZ(qubits[1]);

 // Continue additional computation, conditioned on qubits[1] measurement
 outcome
 if res1 == One {
 X(qubits[0]);
 }
 CNOT(qubits[0], qubits[1]);

 // Measure qubits and return results
 let res2 = Measure([PauliZ, PauliZ], qubits);
 return [res1, res2];
}

```

## Portes ZZ à angle arbitraire

L'ensemble de portes natives de Quantinuum inclut des portes ZZ avec un angle arbitraire. Cela permet de réduire le nombre de portes à 2 qubits pour de nombreux algorithmes quantiques et séquences de portes. Pour plus d'informations sur les portes ZZ à angle arbitraire dans les systèmes Quantinuum, consultez la page [Portes ZZ à angle paramétrable](#) sur [Quantinuum Docs](#).

Portes ZZ à angle arbitraire avec le fournisseur Q#

Dans Q#, la porte ZZ d'angle arbitraire est implémentée avec l'opération `Rzz`.

Q#

```

%%qsharp
import Std.Intrinsic.*;
import Std.Measurement.*;
import Std.Arrays.*;

operation ArbitraryAngleZZExample(theta : Double) : Result[] {

```



```
// Set up circuit with 2 qubits
use qubits = Qubit[2];

// Create array for measurement results
mutable resultArray = [Zero, size = 2];

H(qubits[0]);
Rz(theta, qubits[0]);
Rz(theta, qubits[1]);
X(qubits[1]);

// Add Arbitrary Angle ZZ gate
Rzz(theta, qubits[0], qubits[1]);

// Measure qubits and return results
for i in IndexRange(qubits) {
 resultArray w/= i <- M(qubits[i]);
}

return resultArray;
}
```

## Porte d'intrication SU(4) générale

Le jeu de portes natif de Quantinuum comprend une porte d'enchevêtrement SU(4) générale. Notez que les circuits quantiques soumis au matériel sont rebasés sur la porte ZZ à enchevêtrement complet et sur la porte RZZ à angle arbitraire. Les circuits ne sont rebasés vers la porte d'intrication SU(4) générale que si les utilisateurs choisissent de le faire. Pour plus d'informations sur l'Entangler SU(4) Général dans les systèmes Quantinuum, consultez la page [Portes d'angle paramétré SU\(4\)](#) dans [Quantinuum Docs](#).

### Porte d'intrication SU(4) générale avec fournisseur Q#

Dans Q#, la porte d'intrication générale SU(4) est implémentée via le profil QIR de Quantinuum. Pour l'utiliser, définissez une fonction avec une intrinsèque personnalisée correspondant à la signature de profil QIR et utilisez cette fonction dans l'opération

`SU4Example`.

Pour vous assurer que le circuit s'exécute avec la porte d'intrication SU(4) générale, passez les options suivantes dans la pile Quantinuum :

- `nativetq: Rxxyyzz` afin d'éviter le rebasage vers d'autres portes natives.
- `noreduce: True` pour éviter d'autres optimisations du compilateur (facultatives).

Q#

```

%%qsharp
import Std.Math.*;

operation __quantum__qis__rxxxyzz__body(a1 : Double, a2 : Double, a3 : Double,
q1 : Qubit, q2 : Qubit) : Unit {
 body intrinsic;
}

operation SU4Example() : Result[] {
 use qs = Qubit[2];

 // Add SU(4) gate
 __quantum__qis__rxxxyzz__body(PI(), PI(), PI(), qs[0], qs[1]);

 MResetEachZ(qs)
}

```

Compilez maintenant l'opération :

Python

```
MyProgram = qsharp.compile("GenerateRandomBit()")
```

Connectez-vous à Azure Quantum, sélectionnez la target machine et configurez les paramètres de bruit pour l'émulateur :

Python

```

MyWorkspace = azure.quantum.Workspace(
 resource_id = "",
 location = ""
)

MyTarget = MyWorkspace.get_targets("quantinuum.sim.h2-1e")

Update TKET optimization level desired
option_params = {
 "nativetq": `Rxxxyzz`,
 "noreduce": True
}

```

Indiquez l'option `noreduce` lorsque vous soumettez le travail :

Python

```

job = MyTarget.submit(MyProgram, "Submit a program with SU(4) gate", shots =
10, input_params = option_params)

```

```
job.get_results()
```

## Paramètres de bruit de l'émulateur

Les utilisateurs ont la possibilité d'expérimenter les paramètres de bruit des émulateurs Quantinuum. **Seuls quelques-uns des paramètres de bruit disponibles sont mis en surbrillance** ici, montrant comment passer les paramètres dans les fournisseurs Azure Quantum.

Pour plus d'informations sur l'ensemble complet des paramètres de bruit disponibles, consultez la page [Émulateurs Quantinuum](#) dans [la documentation Quantinuum](#).

Paramètres de bruit de l'émulateur avec le fournisseur Q#

Tout d'abord, importez les packages requis et lancez le profil de base :

Python

```
import qsharp
import azure.quantum
qsharp.init(target_profile=qsharp.TargetProfile.Base)
```

Ensuite, définissez la fonction.

Q#

```
%%qsharp
import Std.Measurement.*;
import Std.Arrays.*;
import Std.Convert.*;

operation GenerateRandomBit() : Result {
 use target = Qubit();

 // Apply an H-gate and measure.
 H(target);
 return M(target);
}
```

et compilez l'opération :

Python

```
MyProgram = qsharp.compile("GenerateRandomBit()")
```

Connectez-vous à Azure Quantum, sélectionnez la target machine et configurez les paramètres de bruit pour l'émulateur :

Python

```
MyWorkspace = azure.quantum.Workspace(
 resource_id = "",
 location = ""
)

MyTarget = MyWorkspace.get_targets("quantinuum.sim.h2-1e")

Update the parameter names desired
Note: This is not the full set of options available.
For the full set, see the Quantinuum Emulators page
option_params = {
 "error-params": {
 "p1": 4e-5,
 "p2": 3e-3,
 "p_meas": [3e-3, 3e-3],
 "p_init": 4e-5,
 "p_crosstalk_meas": 1e-5,
 "p_crosstalk_init": 3e-5,
 "p1_emission_ratio": 6e-6,
 "p2_emission_ratio": 2e-4
 }
}
```

Transmettez les options de bruit de l'émulateur lors de l'envoi du travail :

Python

```
job = MyTarget.submit(MyProgram, "Experiment with Emulator Noise Parameters",
 shots = 10,
 input_params = option_params)

job.get_results()
```

Pour désactiver le modèle de bruit de l'émulateur, définissez l'option `error-model` sur `False`. Par défaut, elle est définie sur `True`.

Python

```
option_params = {
 "error-model": False
```

```
}
```

Pour utiliser l'émulateur de stabilisateur, définissez l'option `simulator` sur `stabilizer`. Par défaut, elle est définie sur `state-vector`.

Python

```
option_params = {
 "simulator": "stabilizer"
}
```

## Compilation TKET dans la pile Quantinuum

Les circuits soumis aux systèmes Quantinuum Quantinuum, à l'**exception des soumissions hybrides intégrées**, sont exécutés automatiquement via des passes de compilation TKET pour le matériel Quantinuum. Cela permet aux circuits d'être automatiquement optimisés pour les systèmes Quantinuum et de s'exécuter plus efficacement.

Vous trouverez plus d'informations sur les passes de compilation spécifiques appliquées dans la [pytket-quantinuum](#) documentation, notamment dans la section [pytket-quantinuum Passes de compilation](#).

Dans la pile logicielle Quantinuum, le niveau d'optimisation appliqué est défini avec le paramètre `tket-opt-level`. *Le paramètre de compilation par défaut pour tous les circuits soumis aux systèmes Quantinuum est le niveau d'optimisation 2.*

Les utilisateurs qui souhaitent expérimenter avec les passes de compilation TKET et voir les optimisations qui pourraient être appliquées à leurs circuits *avant* d'envoyer des travaux peuvent consulter le notebook `Quantinuum_compile_without_api.ipynb` dans le dossier [pytket-quantinuum Exemples](#).

Pour désactiver la compilation TKET dans la pile, une autre option `no-opt` peut être définie sur `True` à l'intérieur de `option_params`. Par exemple : `"no-opt": True`.

Pour plus d'informations sur `pytket`, consultez les liens suivants :

- [pytket](#)
- [pytket Manuel de l'utilisateur](#)

Compilation TKET avec le fournisseur Q#

Tout d'abord, importez les packages requis et lancez le profil de base :

Python

```
import qsharp
import azure.quantum
qsharp.init(target_profile=qsharp.TargetProfile.Base)
```

Ensuite, définissez la fonction.

Q#

```
%%qsharp
import Std.Measurement.*;
import Std.Arrays.*;
import Std.Convert.*;

operation GenerateRandomBit() : Result {
 use target = Qubit();

 // Apply an H-gate and measure.
 H(target);
 return M(target);
}
```

et compilez l'opération :

Python

```
MyProgram = qsharp.compile("GenerateRandomBit()")
```

Connectez-vous à Azure Quantum, sélectionnez la target machine et configurez les paramètres de bruit pour l'émulateur :

Python

```
MyWorkspace = azure.quantum.Workspace(
 resource_id = "",
 location = ""
)

MyTarget = MyWorkspace.get_targets("quantinuum.sim.h2-1e")

Update TKET optimization level desired
option_params = {
 "tket-opt-level": 1
}
```

Introduisez l'option d'optimisation lors de la soumission du travail :

Python

```
job = MyTarget.submit(MyProgram, "Experiment with TKET Compilation", shots =
10, input_params = option_params)
job.get_results()
```

## Spécifications techniques

Vous trouverez des détails techniques pour le modèle de système H2 et les émulateurs du modèle de système H2 sur la page [Guide de l'utilisateur matériel](#) <sup>↗</sup>, en plus de la spécification Quantinuum, des référentiels de données de volume quantique et de [Comment citer les systèmes Quantinuum](#) <sup>↗</sup>.

## Disponibilité cible

Les ordinateurs quantiques Quantinuum sont conçus pour être mis à niveau en continu, ce qui permet aux clients d'avoir accès aux dernières fonctionnalités matérielles à mesure que Quantinuum améliore les fidélités de porte, les erreurs de mémoire et la vitesse du système.

Le matériel Quantinuum passe par des périodes commerciales et des périodes de développement. Pendant les périodes commerciales, le matériel est disponible pour traiter des travaux par le biais d'un système de file d'attente. Pendant les périodes de développement, le matériel est hors connexion, car des mises à niveau sont appliquées.

Chaque mois, un calendrier est envoyé aux utilisateurs Quantinuum avec des informations sur les périodes commerciales et de développement. Si vous n'avez pas reçu ce calendrier, veuillez envoyer un e-mail à l'adresse [QCSupport@quantinuum.com](mailto:QCSupport@quantinuum.com).

L'état d'un target indique sa capacité actuelle à traiter les tâches. Les états possibles d'un target élément sont les suivants :

- **Disponible** : target est en ligne, traitant les travaux soumis et acceptant de nouveaux.
- **Détérioré** : le target accepte des travaux, mais ne les traite pas actuellement.
- **Non disponible** : l'objet target est hors connexion et n'accepte pas de nouvelles soumissions de travaux.

Pour l'ordinateur targetsquantique Quantinuum, **Disponible** et **détérioré** correspondent aux périodes commerciales, tandis que **Non disponible** correspond aux périodes de développement où la machine est hors connexion pour les mises à niveau.

Les informations d'état actuelles peuvent être *récupérées* à partir de l'onglet Fournisseurs d'un espace de travail sur le [portail Azure](#) .

## Pricing

Pour voir les plans de facturation de Quantinuum, visitez la [page des tarifs Azure Quantum](#).

## Limites et quotas

Les quotas de Quantinuum sont suivis sur la base de l'unité de crédit d'utilisation QPU, *crédit Hardware Quantum (HQC)*, pour les travaux soumis aux ordinateurs quantiques Quantinuum et eHQC (HQC d'émulateur) pour les travaux soumis aux émulateurs.

Les crédits HQC et eHQC servent à calculer le coût d'exécution d'un travail, et ils sont calculés selon la formule suivante :

$$HQC = 5 + C(N_{1q} + 10N_{2q} + 5N_m)/5000$$

where:

- $N_{1q}$  est le nombre d'opérations à un qubit dans un circuit.
- $N_{2q}$  est le nombre d'opérations natives à deux qubit dans un circuit. La porte native est équivalente à CNOT jusqu'à plusieurs portes à un qubit.
- $N_m$  est le nombre d'opérations de préparation et de mesure (SPAM) de l'état dans un circuit, y compris la préparation de l'état implicite initial et toutes les mesures intermédiaires et finales et les réinitialisations d'état.
- $C$  est le nombre de captures.

### ⓘ Notes

Le coût total en HQC inclut toutes les portes et mesures à travers toutes les branches conditionnelles ou flux de contrôle. Cela peut avoir un impact plus élevé sur les travaux hybrides intégrés.

Les quotas dépendent du plan sélectionné et peuvent être augmentés avec un ticket de support. Pour afficher vos limites et quotas actuels, accédez à la section **Opérations** et sélectionnez le panneau **Quotas** de votre espace de travail sur le [portail Azure](#) . Pour plus d'informations, consultez [Quotas Azure Quantum](#).



# Stratégie de support pour Quantinuum dans Azure Quantum

Cet article décrit la stratégie de support Microsoft qui s'applique lorsque vous utilisez le fournisseur Quantinuum dans Azure Quantum. L'article s'applique à l'un targets des sous-fournisseurs.

Si vous utilisez le fournisseur Quantinuum et que vous rencontrez des problèmes inattendus, vous pouvez contacter l'équipe du support Azure pour obtenir de l'aide, en [créant un cas de support Azure](#).

Dans certains cas, l'équipe de support Azure doit vous rediriger vers l'équipe de support technique de Quantinuum. Vous pouvez recevoir une réponse plus rapide en accédant directement à Quantinuum.[QCSupport@quantinuum.com](mailto:QCSupport@quantinuum.com)

## Stratégie de prise en charge quantinuum - Préversion publique

L'objectif du support est d'identifier et de remédier aux défauts ou aux dysfonctionnements provoquant l'échec de l'exécution du matériel target Quantinuum conformément aux spécifications et à la documentation convenus (« Problèmes »). Le support couvre uniquement la version publiée en cours mise à la disposition générale des clients. Bien que nos experts quantiques puissent vous aider à résoudre les problèmes d'algorithme, nous ne sommes pas responsables des problèmes, erreurs ou défaillances de votre algorithme. Pour réduire les problèmes de programmation, les utilisateurs peuvent exécuter leurs algorithmes à l'aide du vérificateur de syntaxe avant l'exécution sur du matériel.

## Contacter le support technique au sein de Quantinuum

Pour créer une demande de support avec Quantinuum, envoyez un rapport d'incident par e-mail à [QCSupport@quantinuum.com](mailto:QCSupport@quantinuum.com). L'ingénieur contacté répondra dans le respect du contrat de niveau de service approprié (tableau 2). L'équipe Quantinuum reçoit uniquement les rapports d'incident qui contiennent toutes les informations suivantes et ont la fenêtre de résolution du contrat SLA appropriée. Les rapports sans les informations nécessaires ne déclenchent pas cette réponse. Toutes les demandes doivent être fournies en anglais et sont répondues en anglais.

*Tableau 1 : Informations de contact du support et heures de fonctionnement standard*

Email	Heures de fonctionnement standard
<a href="mailto:QCsupport@quantinum.com">QCsupport@quantinum.com</a>	8h à 17h00 MST les jours ouvrables <sup>1</sup>

<sup>1</sup> « Jours ouvrés » signifie lundi à vendredi, mais à l'exclusion des jours fériés, juridiques ou nationaux.

Le rapport d'incident nécessite les informations suivantes :

- **Description** : description détaillée du problème que vous rencontrez, de ce que vous observez et des étapes que vous avez déjà suivies pour effectuer le triage/comprendre le problème
- **Contact principal** : le nom et le numéro de téléphone du contact principal au cas où nous aurions besoin d'informations supplémentaires
- **Gravité de l'incident** : gravité de l'incident, selon nos définitions de gravité

Une fois qu'un rapport est déposé, l'équipe Quantinum est avertie et répond dans la fenêtre sla requise. Vous recevrez un accusé de réception du problème une fois qu'il a été reçu et que l'ingénieur à l'appel a été averti. L'ingénieur contacté peut demander plus d'informations ou vous appeler à l'aide du numéro indiqué dans le rapport en fonction de la gravité.

Pendant les sessions réservées (dédiées), Quantinum fournit au moins un spécialiste quantique disponible pour le support client pour l'intégralité de votre session dédiée. Quantinum se réserve le droit de changer de spécialiste au cours de votre session, mais recourra aux efforts commerciaux raisonnables pour s'assurer que le nouveau spécialiste est prêt pour poursuivre le support.

Pendant les exécutions en file d'attente, si votre travail s'exécute dans la file d'attente générale et que vous rencontrez un problème, vous pouvez soumettre votre demande de problème en ligne à l'adresse suivante : [QCsupport@quantinum.com](mailto:QCsupport@quantinum.com). Vous pouvez vous attendre à recevoir une réponse dans les trois (3) jours ouvrables.

## Indice de gravité Quantinum et accord de niveau de service (SLA) de réponse

Si l'ingénieur Quantinum détermine que le problème ne se situe pas dans les limites de nos engagements de réponse aux incidents ou n'a pas la bonne classification de gravité, il peut choisir à sa seule discrétion d'ajuster la gravité et/ou la prise en charge de fin si la nouvelle gravité ne l'exige pas.

Certains problèmes peuvent être plus faciles à gérer que d'autres, et il est possible que nous ne puissions pas résoudre complètement le problème avec notre réponse initiale. Si nous ne

pouvons pas résoudre le problème dans un délai raisonnable, nous faisons notre possible pour évaluer le problème et en estimer la durée de résolution.

Tableau 2 : Définition de l'index de gravité

 Agrandir le tableau

Niveau de gravité	Signification	Description
Gravité 1	Impact significatif	Performances système abaissées en-deçà du niveau minimal accepté
Gravité 2	Impact urgent ou élevé	L'état du travail n'a pas été mis à jour ; problèmes de récupération de données
Gravité 3	Non urgent	Divers

Le tableau suivant présente le contrat SLA de réponse qui est respecté par Quantinuum pour les rapports d'incidents de la gravité correspondante. Notez que ceux-ci définissent la rapidité à laquelle nous répondrons aux problèmes : les temps de résolution ne sont pas garantis.

Tableau 3 : Contrat sla de réponse pour différentes gravités

 Agrandir le tableau

Niveau de gravité	Contrat de niveau de service pour les réponses
Gravité 1	Réponse dans l'heure pendant les heures d'ouverture
Gravité 2	Réponse d'ici un jour ouvrable pendant les heures d'ouverture
Gravité 3	Réponse d'ici trois jours ouvrables pendant les heures d'ouverture

## Forum aux questions

**Q : Que se passe-t-il si je soulève un problème de support avec l'équipe du support Azure et qu'il s'avère qu'un fournisseur tiers (comme Quantinuum) doit encore se pencher sur le problème ?**

L'ingénieur du support technique crée un package de redirection pour vous. Il s'agit d'un document PDF qui contient des informations sur votre cas que vous pouvez fournir à l'équipe

de support Quantinuum. L'ingénieur du support vous donne également des conseils et des conseils sur la façon de contacter Quantinuum pour continuer à résoudre les problèmes.

## **Q : Que se passe-t-il si je soulève un problème de support avec l'équipe Quantinuum et qu'il y a un problème avec le service Azure Quantum ?**

L'équipe de support quantinuum vous aide à contacter Microsoft et à vous fournir un package de redirection. Il s'agit d'un document PDF que vous pouvez utiliser pour poursuivre votre demande de support auprès de l'équipe de support Azure.

## **Exclusion de responsabilité de tiers**

Les produits tiers mentionnés dans le présent article sont fabriqués par des sociétés indépendantes de Microsoft. Microsoft exclut toute garantie, implicite ou autre, concernant les performances ou la fiabilité de ces produits.

## **Exclusion de responsabilité sur les coordonnées externes**

Microsoft fournit des informations de contacts externes afin de vous aider à obtenir un support technique sur ce sujet. Ces informations de contact peuvent changer sans préavis. Microsoft ne garantit pas la précision des informations de contact tierces.

---

Last updated on 29/01/2026

# Fournisseur Rigetti

Les [processeurs quantum Rigetti](#) sont des machines universelles fondées sur un modèle de porte basées sur des qubits superconducteurs paramétrables. Les caractéristiques système et les caractéristiques des appareils incluent des fonctionnalités de lecture améliorées, une accélération des temps de traitement quantique, des temps de basculement rapides pour plusieurs familles de portes d'intrication, un échantillonnage rapide via la réinitialisation active du registre et le contrôle paramétrique.

- Publisher : [Rigetti](#)
- ID du fournisseur : `rigetti`

Le fournisseur Rigetti met à disposition les éléments suivants targets :

[Agrandir le tableau](#)

Nom de la cible	ID de la cible	Nombre de qubits	Descriptif
<a href="#">Machine virtuelle quantique (QVM)</a>	rigetti.sim.qvm	-	Simulateur open source pour les programmes Quil, Q# et Qiskit. Gratuit.
<a href="#">Cépheus-1-108Q</a>	rigetti.qpu.cepheus-1-108q	108 qubits	

## ⓘ Remarque

Les simulateurs rigetti et le matériel targets ne prennent pas en charge les programmes Cirq.

Rigetti targets correspond à un **QIR Base** profil. Pour plus d'informations sur ce profil target et ses limitations, consultez [Understanding target types de profils dans Azure Quantum](#).

## Ordinateurs Quantum

Tous les [QPU](#) disponibles publiquement de Rigetti sont accessibles via Azure Quantum. Cette liste est susceptible de changer sans préavis.

### Cépheus-1-108Q

Un processeur quantique de 108 qubits créé à partir d'un réseau de puces de 9-qubits assemblées en mosaïque.

- Type de tâche : `Quantum Program`
- Format de données : `rigetti.quil.v1`, `rigetti.qir.v1`
- ID cible : `rigetti.qpu.cepheus-1-108q`
- Profil d'exécution cible : [QIR Base](#)

## Simulateurs

La [machine virtuelle quantique \(QVM\)](#) est un simulateur open source pour [Quil](#). Il `rigetti.sim.qvm` target accepte un [programme Quil](#) en tant que texte et exécute ce programme sur QVM hébergé dans le cloud, en retournant des résultats simulés.

- Type de tâche : `Simulation`
- Formats de données : `rigetti.quil.v1`, `rigetti.qir.v1`
- ID cible : `rigetti.sim.qvm`
- Profil d'exécution cible : [QIR Base](#)
- Tarification : gratuit (0 \$)

## Tarification

Pour voir le plan de facturation de Rigetti, visitez [Azure Quantum tarification](#).

## Format d'entrée

Tous les Rigetti targets acceptent actuellement deux formats :

- `rigetti.quil.v1`, qui est le texte d'un programme [Quil](#).
- `rigetti.qir.v1`, qui est le bitcode QIR.

Tous targets prennent également le paramètre entier facultatif `count` pour définir le nombre de tirs à exécuter. S'il est omis, le programme s'exécute une seule fois.

## Quil

Tous les Rigetti targets acceptent le `rigetti.quil.v1` format d'entrée, qui est le texte d'un [programme Quil](#), qui représente le langage d'instruction quantique. Par défaut, les programmes sont compilés à l'aide de [quilk](#) avant l'exécution. Toutefois, `quilk` ne prend pas en charge les fonctionnalités de contrôle au niveau de l'impulsion ([Quil-T](#)), donc si vous

souhaitez utiliser ces fonctionnalités, vous devez fournir un programme [Quil natif](#) (contenant également Quil-T) comme entrée et spécifier le paramètre d'entrée `skipQuilc: true`.

Pour faciliter la construction d'un programme Quil, vous pouvez utiliser [pyQuil](#) avec le package [pyquil-for-azure-quantum](#). Sans ce package, `pyQuil` peut être utilisé pour *construct* programmes Quil, mais pas pour les soumettre à Azure Quantum.

## QIR

Tout le matériel Rigetti prend en charge l'exécution de Quantum Intermediate Representation travaux conformes au QIR avec le [QIR Base Profil, v1](#) en tant que `rigetti.qir.v1`. QIR fournit une interface commune qui prend en charge de nombreux langages et target plateformes quantiques pour le calcul quantique et permet la communication entre les langages de haut niveau et les machines. Par exemple, vous pouvez envoyer des tâches Q#, Quil ou Qiskit au matériel Rigetti, et Azure Quantum gère automatiquement l'entrée pour vous. Pour plus d'informations, consultez [Quantum Intermediate Representation](#).

## Sélection du format d'entrée approprié

Devez-vous utiliser Quil ou un autre langage conforme QIR ? Cela dépend de votre cas d'utilisation final. QIR est plus accessible pour de nombreux utilisateurs, tandis que Quil est plus puissant aujourd'hui.

Si vous utilisez Qiskit, Q# ou un autre kit de ressources qui prend en charge la génération QIR, et que votre application fonctionne sur Rigetti targets via Azure Quantum, alors QIR vous convient ! QIR a une spécification en évolution rapide, et Rigetti continue d'augmenter la prise en charge des programmes QIR plus avancés à mesure que le temps passe - ce qui ne peut pas être compilé aujourd'hui peut bien compiler demain.

En revanche, les programmes Quil expriment l'ensemble complet de fonctionnalités disponibles pour les utilisateurs de systèmes Rigetti à partir de n'importe quelle plateforme, y compris Azure Quantum. Si vous souhaitez adapter la décomposition de vos portes quantiques ou écrire des programmes au niveau de l'impulsion, vous voudrez travailler dans Quil, car ces fonctionnalités ne sont pas encore disponibles via QIR.

## Exemples

Le moyen le plus simple d'envoyer des travaux Quil consiste à utiliser le package [pyquil-for-azure-quantum](#), car il vous permet d'utiliser les outils et la documentation de la bibliothèque [pyQuil](#).

Vous pouvez également construire manuellement des programmes Quil et les envoyer directement à l'aide du package `azure-quantum`.

Utiliser pyquil-for-azure-quantum

#### Python

```
from pyquil.gates import CNOT, MEASURE, H
from pyquil.quil import Program
from pyquil.quilbase import Declare
from pyquil_for_azure_quantum import get_qpu, get_qvm

Note that some environment variables must be set to authenticate with Azure
Quantum
qc = get_qvm() # For simulation

program = Program(
 Declare("ro", "BIT", 2),
 H(0),
 CNOT(0, 1),
 MEASURE(0, ("ro", 0)),
 MEASURE(1, ("ro", 1)),
).wrap_in_numshots_loop(5)

Optionally pass to_native_gates=False to .compile() to skip the compilation
stage
result = qc.run(qc.compile(program))
data_per_shot = result.readout_data["ro"]
Here, data_per_shot is a numpy array, so you can use numpy methods
assert data_per_shot.shape == (5, 2)
ro_data_first_shot = data_per_shot[0]
assert ro_data_first_shot[0] == 1 or ro_data_first_shot[0] == 0

Let's print out all the data
print("Data from 'ro' register:")
for i, shot in enumerate(data_per_shot):
 print(f"Shot {i}: {shot}")
```



# Stratégie de support pour Rigetti dans Azure Quantum

Article • 16/10/2024

Cet article décrit la stratégie de support Microsoft qui s'applique lorsque vous utilisez le fournisseur Rigetti dans Azure Quantum. L'article s'applique à l'une [targets](#) des offres offertes par le fournisseur Rigetti.

## Points de contact du support

### Support Azure

En général, les problèmes liés à l'utilisation du fournisseur Rigetti sur Microsoft Azure Quantum sont mieux traités par l'équipe de support Azure Quantum directement via la [création d'un incident de support Azure](#). Cette équipe dispose de tous les détails nécessaires pour isoler le problème que vous rencontrez et diriger le rapport vers le bon point de contact. L'équipe de support Azure a également les moyens de joindre le support Rigetti en dehors des heures ouvrables pour les problèmes les plus urgents.

### Prise en charge de Rigetti

Dans certaines situations, l'équipe du support Azure devra vous rediriger vers l'équipe du support technique de Rigetti. Vous pouvez recevoir une réponse plus rapide en accédant directement à Rigetti. <https://rigetti.zendesk.com/hc/> 

## Chronologie de prise en charge

Rigetti tente toujours de résoudre les problèmes liés à l'intégration de Microsoft Azure de manière aussi rapide que pratique. Certains problèmes prendront plus de temps pour être résolus que d'autres, mais les tableaux ci-dessous indiquent le temps que vous pouvez attendre pour une réponse initiale de Rigetti après le signalement d'un problème.

 [Agrandir le tableau](#)

Niveau de gravité	Définition	Contrat de niveau de service pour les réponses
Élevé	Échec de la connectivité ; dégradation complète des performances	1 jour ouvrable
Moyenne	Détérioration des performances	2 jours ouvrables
Faible	Questions et préoccupations liées aux applications	5 jours ouvrables

Les *jours ouvrables* sont ceux du lundi au vendredi, sans compter les jours fériés américains. Par exemple, si un incident de gravité moyenne est signalé le mardi, une réponse sera retournée à la fin du jour ouvrable le jeudi si les jours intermédiaires sont également des jours ouvrables.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Résolution des problèmes dans Azure Quantum

Lorsque vous travaillez avec le service Azure Quantum, vous pouvez rencontrer des problèmes de connexion ou de travail. Cet article explique comment résoudre ces problèmes.

## Problèmes de connexion à l'espace de travail

### Problème : je ne peux pas m'authentifier auprès de Azure Quantum avec `pytket-azure`

Lorsque vous essayez de vous authentifier auprès de Azure Quantum avec le package `pytket-azure` dans un environnement CI à l'aide des variables d'environnement `"AZURE_TENANT_ID"`, `"AZURE_CLIENT_ID"` et `"AZURE_CLIENT_SECRET"`, vous pouvez rencontrer l'erreur suivante :

#### Invite de commandes Windows

```
Code: InsufficientPermissions
Message: There are not enough permissions to perform this operation.
```

Pour résoudre ce problème, utilisez une chaîne de connexion et la variable d'environnement `"AZURE_QUANTUM_CONNECTION_STRING"` pour vous authentifier à la place. Pour plus d'informations, consultez [Connexion avec une chaîne de connexion](#).

#### Python

```
connection_string = "" # Add your connection string

import os

os.environ["AZURE_QUANTUM_CONNECTION_STRING"] = connection_string
```

## Problèmes de soumission de travaux

### Problème : je ne trouve pas ce target que je veux soumettre mon travail à

Si le Azure Quantum target sur lequel vous souhaitez exécuter votre travail n'est pas dans la liste target disponible, effectuez une mise à jour vers la dernière version du [Quantum Development](#)

Kit (QDK) pour Visual Studio Code (VS Code) [↗](#) . Pour plus d'informations, consultez [Mettre à jour le QDK](#).

## Problème : l'opération renvoie un code d'état non valide « non autorisé »

Pour résoudre ce problème, procédez comme suit :

1. Connectez-vous au portail [Azure](#) [↗](#) et authentifiez votre compte.
2. Accédez à l'espace de travail Quantum auquel vous essayez d'envoyer un travail.
3. Dans le volet de navigation de l'espace de travail, sélectionnez **Contrôle d'accès (IAM)** .
4. Sélectionnez le bouton **Afficher mon accès** . Le volet **affectations** s'ouvre.
5. Dans la colonne **Rôle** de la table **Attributions** de rôles, vérifiez si vous avez le rôle **Propriétaire** ou **Contributeur** .
6. Si vous n'avez pas de rôle, demandez à votre administrateur d'abonnement de vous attribuer le rôle **Propriétaire** ou **Contributeur** pour cet espace de travail.

## Problème : « AuthorizationFailure - Cette demande n'est pas autorisée à effectuer cette opération »

Si vous recevez ce message même si vous disposez d'une connexion valide au service Azure Quantum, le compte de stockage peut être configuré pour bloquer l'accès au réseau public. Le service Azure Quantum prend uniquement en charge les comptes de stockage disposant d'un accès Internet public.

Pour vérifier les paramètres du compte de stockage, procédez comme suit :

1. Connectez-vous au portail [Azure](#) [↗](#).
2. Accédez à votre espace de travail Quantum.
3. Dans la page **Vue d'ensemble** , sélectionnez le **compte de stockage**.
4. Dans le volet de navigation, développez la liste déroulante **Sécurité + mise en réseau** , puis sélectionnez **Mise en réseau**.
5. Dans la section **Accès réseau public** de l'onglet **Accès public** , sélectionnez le bouton **Gérer** .
6. Pour le paramètre **d'accès au réseau public** , sélectionnez **Activer**. Pour le paramètre **d'étendue d'accès au réseau public** , sélectionnez **Activer à partir de tous les réseaux**.
7. Sélectionnez le bouton **Enregistrer**.

## Problème : « Échec de la compilation du programme » lorsque vous envoyez un programme Q# à partir de Azure CLI

Lorsque vous envoyez un travail avec la commande `az quantum submit` Azure CLI, vous pouvez rencontrer le message d'erreur suivant :

### Azure CLI

```
az quantum job submit ...
Failed to compile program.
Command ran in 21.181 seconds (init: 0.457, invoke: 20.724)
```

Cette erreur se produit lorsqu'il existe un problème avec le programme Q# qui provoque l'échec de la compilation. Assurez-vous que votre code Q# a une syntaxe appropriée.

## Problème : Erreur du compilateur « Nombre incorrect de paramètres de porte »

Lorsque vous envoyez un travail à Quantinuum à partir d'un environnement local Jupyter Notebook ou CLI, et que vous utilisez le traducteur QASM hérité (OPENQASM 2.0), vous pouvez rencontrer cette erreur :

### Invite de commandes Windows

```
Job ID <jobId> failed or was cancelled with the message: 1000: Compile error: [<file,
line>] Wrong number of gate parameters
```

Cette erreur se produit lorsqu'une virgule « , » ou un autre caractère non pointé est utilisé comme séparateur décimal. Remplacez tous les séparateurs décimaux autres que des points par des points « . ». Par exemple:

### Q#

```
// replace this line:
rx(1,5707963267948966) q[0];

// with this:
rx(1.5707963267948966) q[0];
```

## Problème : Erreur du compilateur « non disponible pour la configuration de compilation actuelle »

Lorsque vous exécutez une cellule de code Q# dans un Jupyter Notebook dans VS Code, vous pouvez rencontrer l'erreur :

#### Invite de commandes Windows

```
<function name> not found. Found a matching item '<function name>' that is not available for the current compilation configuration
```

Cette erreur indique que vous définissez le profil QIR (représentation intermédiaire quantique) target sur **Base** lorsque la fonction requiert le profil **Illimité**target . Si vous ne spécifiez pas de target type de profil, le compilateur définit automatiquement la target valeur **Non restreint**.

## Problème : l'opération a retourné un code d'état non valide : « Interdit ».

Lorsque vous envoyez votre premier travail, vous pouvez obtenir un code d'erreur **'forbidden'** .

Ce problème se produit lorsque vous créez un espace de travail dans le portail Azure et que Azure Quantum ne parvient pas à terminer l'attribution de rôle qui lie l'espace de travail au compte de stockage spécifié. Cela peut se produire lorsque vous fermez l'onglet ou le navigateur web avant la création de l'espace de travail.

Pour vérifier que vous rencontrez ce problème d'attribution de rôle, procédez comme suit :

1. Accédez à votre espace de travail Quantum dans le portail Azure.
2. Dans la page **Vue d'ensemble** , sélectionnez le **compte de stockage**.
3. Dans le volet de navigation, sélectionnez **Access Control (IAM)**.
4. Choisissez l'onglet **Attributions de rôles** .
5. Dans la colonne **Rôle** , vérifiez si le compte de stockage de votre espace de travail a le rôle **Contributeur de compte de stockage** et le rôle **Contributeur aux données Blob de stockage** .

Si l'espace de travail n'a pas ces deux rôles sur le compte de stockage, effectuez l'une des opérations suivantes :

- Créez un espace de travail et assurez-vous que la création de l'espace de travail se termine avant de fermer la fenêtre ou l'onglet du navigateur web.
- Attribuez les rôles **Contributeur de compte de stockage** et **Contributeur aux données Blob de stockage** à votre espace de travail sur le compte de stockage.

## Problème : Échec du travail avec le code d'erreur : QIRPreProcessingFailed

Lorsque vous envoyez un travail à un Rigetti target et que le travail échoue, vous pouvez voir le message d'erreur suivant dans la **gestion des tâches** console de votre espace de travail Quantum dans le portail Azure :

### Sortie

```
Error code: QIRPreProcessingFailed
Error message: No match found for output recording set converter from
outputrecordingset.v2.labeled to outputrecordingset.v1.nonlabeled
```

Cette erreur peut être due à un conflit de dépendances avec une version antérieure de `pyqir` ou `qiskit-qir`. Désinstallez toutes les versions de `pyqir`, `pyqir-*` et `qiskit-qir` sur votre ordinateur local, puis installez ou mettez à jour la bibliothèque `qdk` Python avec les `azure` et `qiskit` extras :

### Shell

```
pip install --upgrade "qdk[azure,qiskit]"
```

## Problème : récupération d'informations de base sur les travaux ayant échoué

Lorsque vous soumettez un travail à un périphérique target, votre travail peut rester dans la file d'attente pendant plusieurs heures ou jours avant d'échouer.

Pour récupérer plus d'informations sur l'échec du travail, effectuez l'une des opérations suivantes :

- Pour afficher la sortie du travail ou le message d'erreur, utilisez la méthode `get_results()` à partir du module `qdk.azure` Python :

### Python

```
job.get_results()
```

- Dans votre espace de travail Quantum dans le portail Azure, allez au volet **Gestion des tâches** à partir de la liste déroulante **Opérations**, puis choisissez le nom de tâche **Nom** pour ouvrir le volet **Détails de la tâche**.

- Dans votre espace de travail Quantum dans le portail Azure, accédez au volet **Providers** dans le volet **Operations**. Vérifiez que le target matériel est disponible. Si l'état target est **détérioré**, les tâches peuvent rester dans la file d'attente plus longtemps qu'habituellement. Parfois, les travaux sont traités, mais parfois ils expirent et renvoient une erreur de **target indisponible**.

## Problème : Azure Quantum m'invite à m'authentifier lorsque je me connecte par programmation à mon espace de travail

Si vous utilisez le Kit de développement logiciel (SDK) Azure Quantum Python et que vous vous connectez à votre espace de travail à l'aide de la classe `AzureQuantumProvider`, vous pouvez rencontrer une fenêtre contextuelle pour vous authentifier auprès de Azure chaque fois que vous exécutez votre script.

Cette fenêtre contextuelle se produit, car votre jeton de sécurité est réinitialisé chaque fois que vous exécutez le script.

Pour résoudre ce problème, exécutez `az login` à partir du Azure CLI. Pour plus d'informations, consultez [az login](#).

## problèmes de création d'espace de travail Azure Quantum

Vous pouvez rencontrer les problèmes suivants lorsque vous créez un espace de travail Quantum dans le portail Azure.

### Problème : vous ne pouvez pas accéder au formulaire de création de l'espace de travail dans le portail Azure et vous êtes invité à vous inscrire à un abonnement à la place

Ce problème se produit parce que vous n'avez pas d'abonnement actif Azure.

Lorsque vous vous inscrivez à une version d'essai gratuite de 30 jours pour un abonnement Azure, vous bénéficiez de crédits Azure gratuits. Après avoir utilisé tous vos crédits gratuits, ou 30 jours après l'inscription, vous devez effectuer une mise à niveau vers un abonnement [pay-as-you-go](#) pour continuer à utiliser les services Azure Quantum. Lorsque vous disposez d'un abonnement actif, le portail Azure vous permet d'accéder au formulaire de création de l'espace de travail.



Pour afficher la liste de vos abonnements et rôles associés, consultez [Vérifier vos abonnements](#).

ⓘ Remarque

Les crédits Azure de l'abonnement d'essai gratuit de 30 jours ne sont pas éligibles pour être utilisés chez les fournisseurs de matériel quantique.

## Problème : l'option Création rapide n'est pas disponible

Vous devez être **propriétaire** d'un abonnement pour utiliser l'option **Création rapide**. Pour afficher la liste de vos abonnements et rôles associés, consultez [Vérifier vos abonnements](#). Si vous êtes contributeur d'abonnement, **vous pouvez utiliser l'option De création avancée pour créer un espace de travail**.

## Problème : vous ne pouvez pas créer ou sélectionner un groupe de ressources ou un compte de stockage

Ce problème se produit, car vous n'avez pas l'autorisation requise au niveau de l'abonnement, du groupe de ressources ou du compte de stockage. Pour plus d'informations sur les niveaux d'accès requis, consultez [Les exigences de rôle pour la création d'un espace de travail](#).

## Problème : un message d'erreur « Échec de la validation du déploiement » s'affiche lorsque vous choisissez Créer

Ce message d'erreur peut inclure plus de détails, tels que « Le client n'a pas l'autorisation d'effectuer une action ».

Ce problème se produit, car vous n'avez pas l'autorisation requise au niveau de l'abonnement, du groupe de ressources ou du compte de stockage. Pour plus d'informations sur les niveaux d'accès requis, consultez [Les exigences de rôle pour la création d'un espace de travail](#).

Si on vous a récemment accordé l'accès, vous devrez peut-être actualiser la page. Les nouvelles attributions de rôles peuvent prendre jusqu'à une heure pour prendre effet sur les autorisations mises en cache sur la pile.

## Problème : vous ne voyez pas de fournisseur de matériel quantique spécifique sous l'onglet Fournisseurs

Ce problème se produit parce que le fournisseur ne prend pas en charge la région de facturation dans laquelle votre abonnement est défini. Pour obtenir la liste des fournisseurs et leur disponibilité par pays/région, consultez [Disponibilité globale des fournisseurs Azure Quantum](#).

## Problème : la création ou la suppression de fournisseurs d'espace de travail échoue avec « ResourceDeploymentFailure » ou « ProviderDeploymentFailure »

Ce problème peut inclure plus de détails tels que « ResourceDeploymentFailure - L'opération de ressource 'AzureAsyncOperationWaiting' s'est terminée avec l'état de fourniture terminal 'Failed' », ou « ProviderDeploymentFailure - Échec de la création du plan pour le fournisseur : <Name of the provider> ».

Cette défaillance se produit parce que le locataire n'a pas activé Place de marché Azure achats. Suivez les étapes décrites dans [Activer les achats Place de marché Azure](#) pour activer les achats Place de marché Azure.

## Problème : le déploiement d'un espace de travail quantique ou d'un compte de stockage échoue

Lorsque vous essayez de déployer un espace de travail Quantum ou un compte de stockage, vous pouvez obtenir l'une des erreurs suivantes :

- **Espace de travail** : « L'opération d'écriture de ressource ne s'est pas exécutée correctement, car elle a atteint l'état d'approvisionnement du terminal « Échec ». »
- **Compte de stockage** : « Échec du déploiement du modèle en raison d'une violation de stratégie ».

Ce problème peut se produire si votre stratégie de sécurité d'abonnement bloque la création de comptes de stockage dont l'accès public est activé. Le service Azure Quantum prend uniquement en charge les comptes de stockage disposant d'un accès Internet public.

Pour résoudre ce problème, collaborez avec votre administrateur d'abonnement pour obtenir une exception pour le compte de stockage que vous souhaitez utiliser.

# Notes de publication pour le Microsoft Quantum Development Kit (QDK) et Azure Quantum

Les notes de publication décrivent les mises à jour apportées [Microsoft Quantum Development Kit au \(QDK\)](#) et au [Azure Quantum service](#).

Pour obtenir des instructions sur la [mise en Azure Quantum](#) route, consultez le guide de configuration. Pour obtenir des instructions sur la mise à jour de votre QDK version la plus récente, consultez [Mettre à jour la Microsoft Quantum Development Kit version la plus récente](#).

- Pour obtenir les dernières notes de publication et , consultez le référentiel GitHub qsharp .
- Pour obtenir les dernières notes de publication du package Python azure-quantum, consultez le [dépôt](#) [GitHub azure-quantum-python](#).

---

Last updated on 29/01/2026

# Créer des solutions quantiques avec la Microsoft Quantum Development Kit

👉 Explorez les dernières QDK mises à jour à [quantum.microsoft.com](https://quantum.microsoft.com) ↗

👉 Regardez des didacticiels pratiques QDK ↗

Le Microsoft Quantum Development Kit (QDK) est un kit de développement logiciel open source gratuit conçu spécifiquement pour le développement de programmes quantiques et l'informatique quantique. La QDK connexion se connecte directement à Azure Quantum. Vous pouvez donc exécuter des programmes sur des ordinateurs quantiques réels dans le cloud.

## QDK Microsoft Quantum Development Kit

Streamline and accelerate quantum development with AI-powered tools fully integrated with VS Code and GitHub Copilot, and compatible with multiple quantum languages and SDKs.



### Core Foundation

The core layer that drives all quantum programs

Languages & Frameworks  
Resource Estimator  
Libraries  
Compilers



### Productivity & Tools

Everything you need to build, debug, and optimize quantum programs

VS Code and Copilot  
Debugging & Testing  
Visualization  
Simulation



### Domain Libraries

Use quantum methods for domain-specific problems without deep expertise

QDK for Chemistry  
QDK for Error Correction  
More to come

Avec le QDK, vous avez tout ce dont vous avez besoin pour écrire, simuler, déboguer et exécuter des programmes quantiques dans un seul kit de ressources.

## Fonctionnalités du QDK

Explorez la documentation suivante pour en savoir plus sur les principales fonctionnalités du QDK.

- [Prise en charge du langage quantique](#)
- [Simulateurs quantiques et modèles de bruit](#)
- [Éditeur de circuit](#)
- [Estimateur de ressources quantiques](#)

- [Correction des erreurs quantiques](#)
- [Applications scientifiques de la chimie et des matériaux avec QDK/Chemistry](#)
- [Découvrir l'informatique quantique avec QDK Learning](#)
- [Copilot intégration dans VS Code](#)

## Commencez avec le QDK

Se QDK compose des composants suivants.

 Agrandir le tableau

Composant	Utilisé pour	Où installer
Extension QDK pour Visual Studio Code	Développement quantique général	<a href="#">VS Code Marché</a>
Package <code>qdk</code> Python	Développement quantique général	<a href="#">Python Package Index</a>
Bibliothèque <code>qdk-chemistry</code> Python	Applications de chimie et de science des matériaux	<a href="#">Python Package Index</a>
La QDKsuite logicielle -EC	Recherche sur la correction des erreurs quantiques	<a href="#">GitHub</a>

Pour commencer à utiliser , QDKinstallez l'extension et Python les VS Code bibliothèques. Vous pouvez utiliser chaque composant individuellement ou les utiliser ensemble pour le même projet. Par exemple, vous pouvez rapidement écrire et exécuter ou OpenQASM exécuter Q# des programmes dansVS Code, puis importer les programmes dans Python pour effectuer l'estimation des ressources pour votre programme.

Pour obtenir des instructions sur l'installation de l'extension et le package, consultez [Configurer l'extensionQDK](#).PythonVS CodeQDK Pour installer la QDKbibliothèque /Chemistry, consultez [Comment installer QDK pour la chimie](#).

 [Explorez les dernières QDK mises à jour à \[quantum.microsoft.com\]\(https://quantum.microsoft.com\)](#)

 [Regardez des didacticiels pratiques QDK](#)

# Configurer le Microsoft Quantum Development Kit

Dans cet article, vous allez apprendre à installer l'extension Microsoft Quantum Development Kit (QDK) pour Visual Studio Code (VS Code), le package QDK Python avec Jupyter Notebook prise en charge et le Azure CLI.

## Prérequis

- Dernière version de [VS Code](#) <sup>↗</sup>.
- Si vous souhaitez envoyer des travaux à Azure Quantum, vous devez disposer d'un compte Azure avec un espace de travail quantique. Pour plus d'informations, consultez [Créer un espace de travail Azure Quantum](#).

## Installer l'extension QDK

Pour utiliser le QDK dans VS Code, installez [l'extension QDK](#) <sup>↗</sup>. Vous pouvez également utiliser le QDK dans [VS Code pour le web](#) <sup>↗</sup> sans installer l'extension, mais vous n'aurez pas toutes les fonctionnalités de VS Code Desktop. Pour plus d'informations, consultez [Différentes façons d'exécuter Q# des programmes](#).

Vous pouvez maintenant écrire, déboguer et exécuter Q# des programmes sur le simulateur quantique intégré. Ou, si vous disposez d'un compte Azure, vous pouvez connecter et envoyer Q# des programmes au matériel quantique, tous à partir de VS Code.

Pour tester votre configuration, consultez [Envoyer Q# des travaux à Azure Quantum](#).

## Installer le package de Python QDK

Avec Python prise en charge dans VS Code, vous pouvez incorporer ou appeler Q# du code à partir de vos programmes Python ou notebooks Jupyter, puis les exécuter sur les simulateurs quantiques intégrés. Vous pouvez également vous connecter à votre espace de travail Azure Quantum et soumettre vos travaux à exécuter sur du matériel quantique réel.

## Prérequis

- Installez un environnement Python (3.10 ou version ultérieure, 3.11 recommandé) avec [Python et Pip](#).
- Installez l'extension QDK dans VS Code.

## Installer les packages requis

Pour ajouter le support de Python et Jupyter Notebook :

1. Installez les [extensions Python](#) et [Jupyter](#) pour VS Code.
2. Ouvrez la ligne de commande.
3. Installez le `qdk` package Python avec les `azure` informations supplémentaires :

Invite de commandes Windows

```
python -m pip install "qdk[azure]"
```

4. Pour la prise en charge de Qiskit version 1 et version 2, installez l'élément `qiskit` supplémentaire.

Invite de commandes Windows

```
python -m pip install "qdk[qiskit]"
```

### Important

Si vous effectuez une mise à jour à partir d'un environnement Qiskit précédent, consultez [Mettre à jour le module avec la qdk.azure prise en charge de Qiskit](#).

5. Pour travailler dans Jupyter Notebook et afficher des visualisations, installez les packages Python suivants :

Invite de commandes Windows

```
python -m pip install "qdk[jupyter]" ipykernel ipynb jupyterlab
```

Pour tester votre configuration, consultez [Envoyer Q# des tâches avec Python](#) ou [Envoyer Q# des tâches avec les notebooks Jupyter](#).

## Ajouter la prise en charge d'Azure CLI

Vous avez la possibilité d'utiliser Azure CLI pour envoyer des travaux quantiques à partir d'une fenêtre de terminal dans VS Code.

1. Installez [Azure CLI](#).
2. Ouvrez une invite de commandes Windows ou un terminal dans VS Code.
3. Dans l'invite de commandes, exécutez la commande suivante pour effectuer une mise à jour vers la dernière extension Azure CLI `quantum` :

**Invite de commandes Windows**

```
az extension add --upgrade -n quantum
```

Pour tester votre configuration, consultez [Envoyer Q# des travaux à Azure Quantum](#).

## Contenu connexe

- [Différentes façons d'exécuter Q# des programmes](#)
- [Démarrage rapide : Créer votre premier Q# programme](#)
- [Mettre à jour le QDK vers la dernière version](#)

---

Last updated on 13/08/2026



# Comment utiliser le mode agent dans VS CodeQDK

Utilisez le mode agent dans Visual Studio Code (VS Code), alimenté par GitHubCopilot, pour améliorer votre expérience de générateur avec le MicrosoftQuantum Development Kit (QDK).

Le mode agent est une expérience de développement assistée par l'IA qui vous aide à écrire et déboguer du code, et à effectuer d'autres tâches de développement dans VS Code. L'extension QDK inclut un fichier `SKILL.md` que l'agent GitHubCopilot lit pour apprendre Agent Skills dans le but de réaliser des tâches spécifiques liées au développement dans le QDK. Les compétences QDK aident Copilot pour effectuer les tâches suivantes en mode agent :

- Écrire, déboguer et expliquer le code Q# et OpenQASM
- Développer avec la bibliothèque `qdk` Python
- Expliquer les concepts de l'informatique quantique
- Fournissez des informations sur vos travaux et espaces de travail Azure Quantum

Pour plus d'informations sur Agent Skills pour GitHubCopilot dans VS Code, voir [Utilisez Agent Skills dans VS Code](#) sur le site web VS Code.

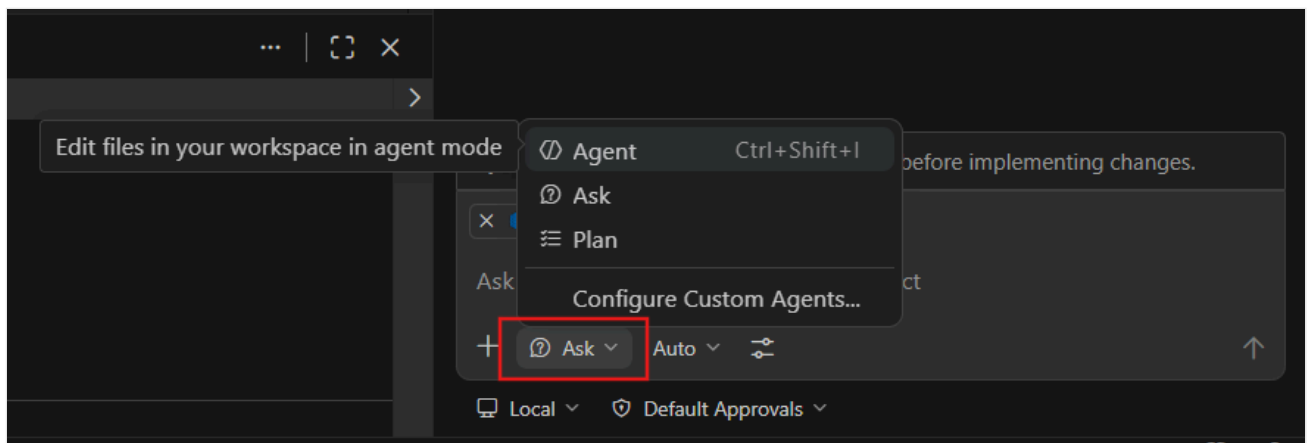
## Prerequisites

- Installez la dernière version de [VS Code](#).
- Installez l'extension [QDK](#) dans VS Code.

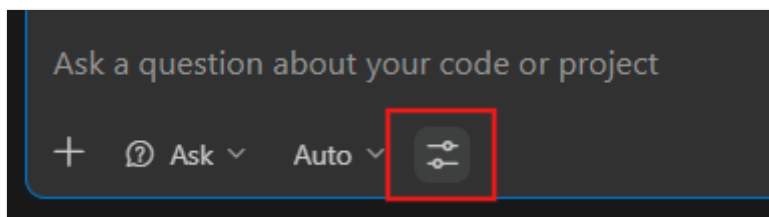
## Configurer Agent Skills à partir de QDK

Pour commencer à utiliser le QDKAgent Skills mode agent dans VS Code, procédez comme suit :

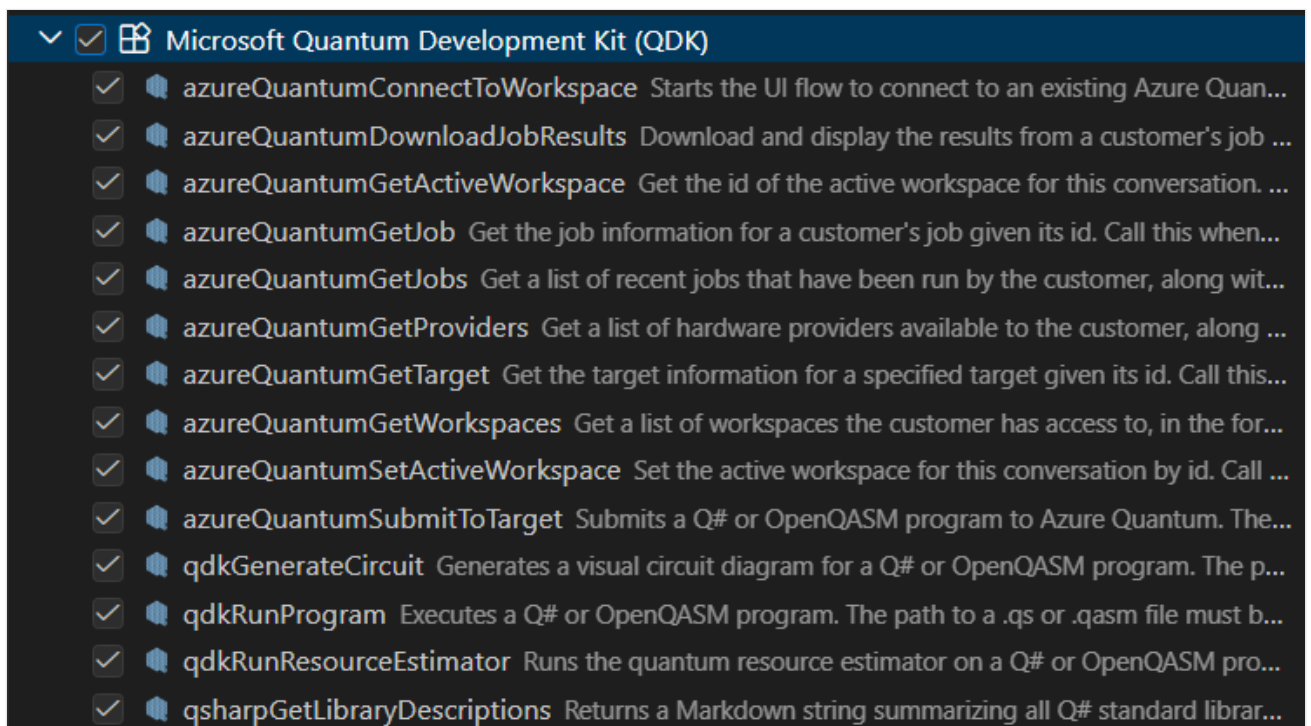
1. Ouvrez un fichier Q# (`.qs`) ou (`.qasm`) dans VS Code.
2. Ouvrez la fenêtre de conversation Copilot dans VS Code.
3. Dans la zone de conversation, sélectionnez l'icône **Définir l'agent**, puis choisissez **Agent**.



4. Dans la zone de conversation, sélectionnez l'icône **Configurer les outils** . La fenêtre **Configurer les outils** s'ouvre.



5. Sélectionnez tous les outils dans la section déroulante **MicrosoftQuantum Development Kit (QDK)**, puis sélectionnez le bouton **OK**.



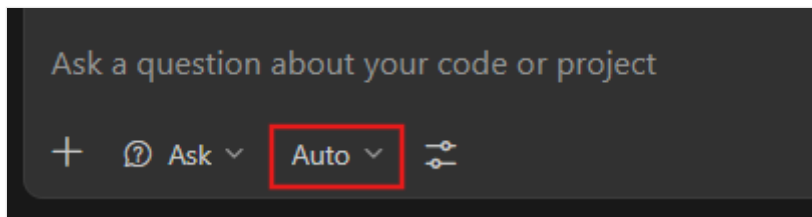
6. Entrez une demande pour l'agent Copilot. Par exemple, si votre fichier contient déjà un programme, demandez ce que fait le code. Ou, si votre fichier est vide, indiquez Copilot pour écrire un algorithme de téléportation.

Copilot lit le Agent Skills du QDK et répond à vos invites avec des connaissances spécialisées sur le QDK, Azure Quantum et l'informatique quantique.

## Essayer différents modèles d'agent

Copilot vous permet de choisir parmi un ensemble de différents modèles de langage à utiliser en mode agent. Différents modèles ont des forces différentes, donc le meilleur modèle pour vous dépend de votre cas d'usage.

Pour explorer différents modèles, sélectionnez l'icône **Pick Model** dans la zone de conversation Copilot et choisissez l'un des modèles disponibles dans la liste.



## Essayez quelques requêtes.

Maintenant que vous êtes configuré pour tirer parti de l'IA dans vos QDK projets, essayez quelques commandes dans le Copilot chat.

Pour commencer, voici quelques exemples de suggestions :

Simulez ce programme pour 1 000 coups et montrez-moi un histogramme.

Envoyez ce programme à Azure Quantum.

Ajoutez des tests pour ce code.

Montrez-moi mes travaux récents sur Azure Quantum.

Salut, j'ai l'intention d'apprendre un peu sur l'informatique quantique, mais je n'ai aucune idée par où commencer. Pouvez-vous m'aider à écrire un programme très simple, à le comprendre, et peut-être même à l'exécuter sur un ordinateur quantique réel ?

# Référence : Extension du Kit de développement Microsoft Quantum pour Visual Studio Code

Le Microsoft Quantum Development Kit (QDK) utilise les fonctionnalités standard de Visual Studio Code (VS Code), ainsi que des fonctionnalités spécifiques au langage lorsque vous utilisez des fichiers `.qs` ou `.qasm`. Ce guide de référence décrit toutes les commandes QDK auxquelles vous pouvez accéder dans la palette de commandes VS Code, ainsi que d'autres fonctionnalités et liens vers du contenu supplémentaire. Pour obtenir des conseils généraux sur VS Code, consultez la [documentation](#) VS Code.

## Conseil

La plupart des commandes de l'extension QDK sont accessibles à partir du menu **Afficher > la palette de commandes**. Dans un fichier `.qs` ou `.qasm`, ouvrez la palette de commandes et entrez **QDK** : pour afficher une liste filtrée de commandes.

## Commands

La plupart des commandes QDK sont liées à l'écriture et à l'exécution de programmes Q# et OpenQASM et sont disponibles uniquement lorsqu'un fichier `.qs` ou `.qasm` est actif. Les autres commandes fonctionnent en arrière-plan et ne sont pas accessibles dans la palette de commandes.

Le tableau suivant décrit les commandes de l'extension QDK qui apparaissent dans la palette de commandes.

## Conseil

Vous pouvez créer des raccourcis clavier personnalisés pour les commandes VS Code à l'aide de **raccourcis clavier**, ou **Ctrl + K + S**. Pour plus d'informations, consultez [Liaisons de clés pour VS Code](#).

Command	Action	Remarques	Autre action de l'utilisateur
QDK : Créer un projet Q#	Crée un projet Q# dans le dossier actif, y compris un <code>qsharp.json</code> fichier manifeste et un <code>src</code> sous-dossier avec un fichier par défaut <code>main.qs</code> .	Pour plus d'informations sur les projets Q#, consultez <a href="#">Utilisation des projets Q#</a> .	Dans Explorateur de fichiers, cliquez avec le bouton droit sur le dossier cible et sélectionnez <b>Créer un projet Q#</b> .
QDK : Créer un notebook Microsoft Quantum	Ouvre un exemple de notebook Jupyter qui exécute un programme Q# + Python et envoie un travail à Azure Quantum.	Pour plus d'informations sur le QDK dans Jupyter Notebook, consultez <a href="#">Envoyer des travaux à Azure Quantum avec le package QDK Python</a> .	N/A
QDK : Se connecter à un espace de travail Azure Quantum	Connectez-vous à un espace de travail Azure Quantum à l'aide de votre compte Azure ou d'un chaîne de connexion. Une fois que vous êtes authentifié, sélectionnez l'icône <b>Microsoft panneau Quantum</b> (en bas à gauche).	Pour plus d'informations sur les connexions Azure Quantum, consultez <a href="#">Se connecter à votre espace de travail Azure Quantum</a> .	Pointez sur <b>les espaces de travail Quantum</b> et sélectionnez l'icône + .
QDK : Ouvrir l'aire de jeu QDK	Ouvre un dossier en ligne des exemples de programmes Q# et OpenQASM dans l'Explorateur de fichiers. Vous pouvez modifier et exécuter les programmes dans le simulateur quantique local, ainsi que définir des points d'arrêt et parcourir le code avec le débogueur intégré.	Pour plus d'informations, consultez le <b>fichier README</b> dans l'exemple de dossier.	N/A
QDK : Actualiser les espaces de travail Azure Quantum	Synchronise les dernières modifications de vos espaces de travail quantiques connectés.	En cas de problème de connexion, une icône d'alerte apparaît en regard du nom de l'espace de travail.	Pointez sur <b>Les espaces de travail Quantum</b> et sélectionnez l'icône d'actualisation.
Explorateur : Focus sur l'affichage Espaces de travail Quantum	Ouvre Explorateur de fichiers et se concentre sur vos espaces de travail quantiques connectés, s'il est configuré. Si aucun espace de travail n'est configuré, vous êtes invité à ajouter un espace de travail existant.	Pour plus d'informations sur les connexions Azure Quantum, consultez <a href="#">Se connecter à votre espace de travail Azure Quantum</a> .	N/A

Les commandes suivantes sont disponibles lorsqu'un fichier `.qs` ou `.qasm` est actif.

Command	Action	Remarques	Autre action de l'utilisateur
QDK : Afficher le circuit	Affiche un diagramme de circuit pour le programme avant son exécution.	Pour plus d'informations, consultez <a href="#">Visualiser les diagrammes de circuit quantique</a> .	Sélectionnez l'option <b>code lens Circuit</b> dans le menu à côté de l'opération du point d'entrée ou au-dessus de chaque opération définie par l'utilisateur dans le programme.
QDK : Exécuter le fichier et afficher l'histogramme	Exécute le programme actuel et affiche un histogramme des résultats dans un nouveau volet.	Pour accéder aux options de tri et de filtre de l'affichage de l'histogramme, sélectionnez l'icône de filtre dans le volet histogramme.	Sélectionnez l'option Code Lens <b>Histogramme</b> dans le menu en regard de l'opération de point d'entrée.
QDK : Obtenir QIR pour le programme QDK actuel	Ouvre la source QIR pour le code Q# ou OpenQASM actuel dans une nouvelle fenêtre d'édition. Votre programme doit utiliser le profil cible Base, Ri adaptatif ou Adaptive RIF pour exporter la source QIR.	Pour obtenir plus d'informations sur QIR, consultez la <a href="#">représentation intermédiaire de Quantum</a> et le <a href="#">blog du développeur du QDK</a> .	N/A
QDK : Calculer les estimations des ressources	Appelle la version intégrée de l'estimateur de ressource.	Pour plus d'informations, consultez <a href="#">Présentation de l'estimateur de ressources Microsoft Quantum</a> .	N/A
QDK : Aide	Vue d'ensemble de l'extension QDK dans VS Code.	Pour obtenir la documentation Complète de Microsoft Quantum, consultez la <a href="#">documentation Microsoft Quantum</a> .	N/A
QDK : Exécuter un fichier et afficher le diagramme de circuit	Exécute le programme actuel et affiche un circuit du programme avec des sorties.	Pour plus d'informations, consultez <a href="#">Visualiser les diagrammes de circuit quantique</a> .	N/A

Command	Action	Remarques	Autre action de l'utilisateur
QDK : Afficher la documentation de l'API	Ouvre la documentation de l'API dans un nouveau volet. Pour rechercher dans ce volet, appuyez sur Ctrl+F.	Pour plus d'informations, consultez la référence de l'API Quantum <a href="#">Microsoft</a> .	N/A
QDK : Afficher le journal des modifications	Ouvre un journal des modifications dans un nouvel onglet qui affiche les mises à jour QDK pour les versions actuelles et toutes les versions de version précédentes.	Le journal des modifications est également disponible dans le <a href="#">référentiel GitHub QDK</a> <a href="#">open source</a> .	N/A
Déboguer : Démarrer le débogage	Ouvre le programme actuel dans le débogueur.	Pour plus d'informations, consultez <a href="#">Débogage et test de votre code</a> quantique.	Appuyez sur <b>F5</b> , ou sélectionnez l'option Objectif de code <b>Déboguer</b> dans le menu en regard de l'opération de point d'entrée, ou sélectionnez l'icône <b>Exécuter</b> en haut à droite, puis choisissez <b>Démarrer le débogage</b> .
Déboguer : Exécuter	Exécute le programme actuel dans le simulateur quantique par défaut.	Pour plus d'informations, consultez <a href="#">Commencez avec les programmes Q#</a> .	Appuyez sur <b>Ctrl + F5</b> , ou sélectionnez l'option <b>Run de Code Lens</b> dans le menu à côté de l'opération de point d'entrée, ou cliquez sur l'icône <b>Exécuter</b> en haut à droite, puis choisissez <b>Exécuter</b> .

## Bornes

Les programmes Q# et OpenQASM utilisent deux fenêtres de terminal dans VS Code :

[🔍](#) Agrandir le tableau

Terminal	Action
Console de débogage	Affiche la sortie d'exécution ou la sortie de débogage
Problèmes	Affiche les vérifications des erreurs de précompilation

# Modifier le code Q# et OpenQASM

La plupart des fonctionnalités courantes de modification de code dans VS Code sont également disponibles lorsque vous utilisez des programmes Q# et OpenQASM :

- Vérification des erreurs de précompilation
- Accéder à la définition
- Références
- Signatures de fonction
- Informations sur les paramètres
- Suggestions de saisie automatique, y compris les saisies automatiques sensibles au contexte, les membres d'espace de noms et les membres de type.
- Linting : dans les fichiers Q#, vous configurez le linting par projet dans le fichier manifeste. Pour plus d'informations, consultez [Travail avec des projets Q#](#).

Pour plus d'informations, consultez [IntelliSense](#) dans la documentation VS Code.

## Tâches courantes

### Utiliser des fichiers et des projets Q#

 Agrandir le tableau

Tâche	Action	Remarques
Nouveau fichier Q#	Sélectionnez <b>Fichier &gt; Nouveau fichier texte</b> . Enregistrez le fichier avec une extension <code>.qs</code> .	Si vous définissez <code>Files: Default Language = qsharp</code> , un nouveau fichier est automatiquement défini par défaut sur la mise en forme Q#.
Créer un projet Q#	Dans un dossier ouvert dans Explorateur de fichiers, sélectionnez <b>Créer un projet Q#</b> dans la palette de commandes, ou cliquez avec le bouton droit sur le dossier dans Explorateur de fichiers, puis sélectionnez <b>Créer un projet Q#</b> .	Pour plus d'informations sur les projets Q#, consultez <a href="#">Utilisation des projets Q#</a> .
Exemples de fichiers	Dans un fichier <code>.qs</code> vide ou <code>.qasm</code> , entrez <b>exemples</b> , puis choisissez un programme exemple dans la liste des options.	Vous pouvez également sélectionner <b>Ouvrir le terrain de jeu QDK</b> dans la palette de commandes pour ouvrir un dossier en ligne des exemples de programmes Q# et OpenQASM dans l'Explorateur de fichiers.



# Se connecter à Azure Quantum

 Agrandir le tableau

Tâche	Action	Remarques	Autre action de l'utilisateur
Se connecter à un espace de travail Azure Quantum	Sélectionnez l'icône <b>Microsoft panneau Quantum</b> (en bas à gauche). Dans <b>Quantum Workspaces</b> , sélectionnez <b>Ajouter un espace de travail existant</b> . Suivez les instructions pour sélectionner un abonnement et un espace de travail.	Vous pouvez vous connecter à plusieurs espaces de travail. Sélectionnez <b>+</b> à côté de <b>espaces de travail Quantum</b> pour connecter un autre espace de travail. Les connexions d'espace de travail persistent entre vos sessions VS Code.	Dans la palette de commandes, sélectionnez <b>QDK : Se connecter à un espace de travail Azure Quantum</b> .
Se connecter par programmation avec un programme Python	Cliquez avec le bouton droit sur une connexion d'espace de travail existante, puis sélectionnez <b>Copier le code Python pour vous connecter à l'espace de travail</b> . Collez le code résultant dans votre programme Python.	Pour plus d'informations, consultez <a href="#">Envoyer des travaux à Azure Quantum avec le package QDK Python</a> .	N/A

## Exécuter des programmes

 Agrandir le tableau

Tâche	Action	Remarques	Autre action de l'utilisateur
Exécuter un programme Q# ou OpenQASM sur le simulateur quantique local	Dans un fichier <b>.qs</b> ou <b>.qasm</b> , sélectionnez l'icône <b>Exécuter</b> en haut à droite, puis sélectionnez <b>Exécuter</b> .	Pour plus d'informations sur le simulateur quantique, consultez le <a href="#">simulateur quantique épars</a> .	Appuyez sur <b>Ctrl + F5</b> , ou choisissez <b>QDK : Exécuter le fichier et afficher l'histogramme</b> ou <b>QDK : Exécuter le fichier et afficher le diagramme de circuit</b> dans la palette de commandes, ou choisissez l'option <b>Exécuter</b> l'objectif de code au-dessus de l'opération de point d'entrée.
Déboguer un programme	Dans un programme Q# ou OpenQASM, sélectionnez l'icône <b>Exécuter</b> en haut à	Pour plus d'informations sur le débogueur Q# dans VS Code, consultez	Appuyez sur <b>F5</b> ou choisissez le <b>CodeLens Débogage</b> dans le menu au-dessus de l'opération de point d'entrée.

Tâche	Action	Remarques	Autre action de l'utilisateur
	droite, puis choisissez <b>Démarrer le débogage</b> .	<a href="#">Débogage et test de votre code</a> quantique.	
<b>Afficher les fournisseurs et les cibles dans vos espaces de travail</b>	Sélectionnez l'icône <b>Microsoft panneau Quantum</b> (en bas à gauche). À partir <b>des espaces de travail Quantum</b> , développez l'espace de travail, puis développez <b>Fournisseurs</b> pour afficher les fournisseurs disponibles dans l'espace de travail. Développez un prestataire individuel pour afficher les cibles disponibles.	Pointez sur un nom de cible pour afficher son <b>statut</b> et son <b>temps d'attente dans la file d'attente</b> avant de soumettre un emploi.	N/A
<b>Envoyer un travail à Azure Quantum</b>	Dans un programme Q# ou OpenQASM, choisissez un espace de travail, un fournisseur et une cible. Pour envoyer le programme actuel, sélectionnez la flèche en regard de la cible.	Pour plus d'informations, consultez <a href="#">Envoyer des travaux à Azure Quantum avec l'extension QDK pour VS Code</a> .	N/A
<b>Afficher les résultats du travail</b>	Développez l'espace de travail, puis développez <b>Travaux</b> . Pour ouvrir le résultat du travail depuis stockage Azure, cliquez sur l'icône cloud à côté du nom du travail.	Les travaux sont répertoriés du plus récent au plus ancien.	N/A

# Prise en charge du langage de programmation quantique dans le QDK

QDK(Microsoft Quantum Development Kit) prend en charge le développement dans plusieurs langages de programmation quantique avec l'extension QDK pour Visual Studio Code (VS Code) et le QDKPython package. Avec le QDK, vous pouvez écrire des programmes quantiques et les soumettre à Azure Quantum dans votre langue de choix, ou utiliser plusieurs langages dans votre flux de travail de développement.

## Support linguistique dans l'extension QDK pour VS Code

L'[extension QDK pour VS Code](#) soutient directement le développement dans les langages de programmation quantique Q# et OpenQASM. Lorsque vous ouvrez un fichier Q# `.qs` ou OpenQASM `.qasm` dans VS Code, vous pouvez utiliser toutes les fonctionnalités de l'extension QDK, telles que l'estimation des ressources, la visualisation de circuit et l'envoi de travaux à Azure Quantum cibles du fournisseur. Vous pouvez également utiliser d'autres fonctionnalités VS Code, telles que la complétion de code, IntelliSense et le débogage.

L'extension VS Code inclut également QDK, une interface graphique pour créer des circuits quantiques que vous pouvez utiliser dans les programmes Q#.

## Prise en charge linguistique dans le QDKPython package

Le [QDKPython package](#) prend en charge le développement dans les langages et packages de programmation quantique suivants :

- Q#
- OpenQASM
- Qiskit
- Cirq
- PennyLane

Le QDK package inclut des modules qui vous permettent de :

- Utilisez les fonctionnalités Jupyter Notebook, telles que les outils de visualisation et une instruction magique pour écrire du code Q#
- Convertir Q# et OpenQASM structs et callables en objets Python à utiliser dans d'autres langages et frameworks
- Connectez-vous à votre espace de travail Azure Quantum et envoyez des travaux

## Comparer les QDK options de langue

Le tableau suivant compare la disponibilité des langues et des frameworks pour l'extension Python et le package VS Code.

 Agrandir le tableau

Language	QDK VS Code Extension	QDK Python Paquet
Q#	Disponible directement	Disponible via le <code>qdk.qsharp</code> module et la <code>%%qsharp</code> commande magic
OpenQASM	Disponible directement	Disponible via le <code>qdk.openqasm</code> module
Qiskit	Non disponible	Disponible via le <code>qdk.qiskit</code> module
Cirq	Non disponible	Disponible via le <code>qdk.cirq</code> module
PennyLane	Non disponible	Disponible via la conversion en QIR

## Étapes suivantes

Pour commencer avec les langages de programmation quantique dans le QDK, installez l'extension PythonQDK et le package VS Code. Pour plus d'informations, consultez [Configurer le QDK](#).

Pour soumettre des programmes quantiques à Azure Quantum dans différentes langues, consultez les guides suivants :

- [Comment envoyer des travaux à Azure Quantum](#)
- [Envoyer des travaux à Azure Quantum avec l'extension QDK pourVS Code](#)
- [Envoyer des travaux à Azure Quantum avec le QDKPython package](#)

# Présentation du langage de programmation quantique Q#

Q# est un langage de programmation [open source](#) de haut niveau développé par Microsoft pour écrire des programmes quantiques. Q# est inclus dans le Kit de développement Quantum (QDK). Pour plus d'informations, consultez [Configurer le Kit](#) de développement Quantum.

En tant que langage de programmation quantique, Q# répond aux exigences suivantes pour le langage, le compilateur et le runtime :

- **Indépendant du matériel** : Les qubits dans les algorithmes quantiques ne sont pas liés à un matériel ou une disposition quantique spécifique. Le Q# compilateur et le runtime gèrent le mappage entre les qubits de programme et les qubits physiques, ce qui permet au même code de s'exécuter sur différents processeurs quantiques.
- **Intégration de l'informatique quantique et classique** : Q# permet l'intégration de calculs quantiques et classiques, qui est essentiel pour l'informatique quantique universelle.
- **Gestion des qubits** : Q# fournit des opérations et des fonctions intégrées pour la gestion des qubits, notamment la création d'états de superposition, des qubits d'enchèvement et l'exécution de mesures quantiques.
- **Respectez les lois de la physique** : Q# et les algorithmes quantiques doivent suivre les règles de la physique quantique. Par exemple, vous ne pouvez pas copier ou accéder directement à l'état qubit dans Q#.

Pour plus d'informations sur les origines de Q#, consultez le billet de blog [Pourquoi avons-nous besoin de Q#?](#).

## Structure d'un Q# programme

Avant de commencer à écrire Q# des programmes, il est important de comprendre leur structure et leurs composants. Considérez le programme suivant Q# , nommé `Superposition`, qui crée un état de superposition :

Q#

```
namespace Superposition {
 @EntryPoint()
 operation MeasureOneQubit() : Result {
 // Allocate a qubit. By default, it's in the 0 state.
 use q = Qubit();
 // Apply the Hadamard operation, H, to the state.
 // It now has a 50% chance of being measured as 0 or 1.
 H(q);
 // Measure the qubit in the Z-basis.
 let result = M(q);
 }
}
```

```

 // Reset the qubit before releasing it.
 Reset(q);
 // Return the result of the measurement.
 return result;
}
}

```

En fonction des commentaires (`//`), le Q# programme alloue d'abord un qubit, applique une opération pour placer le qubit en superposition, mesure l'état du qubit, réinitialise le qubit et retourne enfin le résultat.

Nous allons décomposer ce Q# programme en ses composants.

## Espaces de noms utilisateur

Q# les programmes peuvent éventuellement commencer par un espace de noms défini par l'utilisateur, par exemple :

Q#

```

namespace Superposition {
 // Your code goes here.
}

```

Les espaces de noms peuvent vous aider à organiser les fonctionnalités associées. Les espaces de noms sont facultatifs dans Q# les programmes, ce qui signifie que vous pouvez écrire un programme sans définir d'espace de noms.

Par exemple, le `Superposition` programme de l'exemple peut également être écrit sans espace de noms comme suit :

Q#

```

@EntryPoint()
operation MeasureOneQubit() : Result {
 // Allocate a qubit. By default, it's in the 0 state.
 use q = Qubit();
 // Apply the Hadamard operation, H, to the state.
 // It now has a 50% chance of being measured as 0 or 1.
 H(q);
 // Measure the qubit in the Z-basis.
 let result = M(q);
 // Reset the qubit before releasing it.
 Reset(q);
 // Return the result of the measurement.
 return result;
}

```

### ❗ Remarque

Chaque Q# programme ne peut avoir qu'un `namespace` seul . Si vous ne spécifiez pas d'espace de noms, le Q# compilateur utilise le nom de fichier comme espace de noms.

## Points d'entrée

Chaque Q# programme doit avoir un point d'entrée, qui est le point de départ du programme. Par défaut, le Q# compilateur démarre l'exécution d'un programme à partir de l'opération `Main()` , le cas échéant, qui peut se trouver n'importe où dans le programme. Si vous le souhaitez, vous pouvez utiliser l'attribut `@EntryPoint()` pour spécifier n'importe quelle opération dans le programme comme point d'exécution.

Par exemple, dans le `Superposition` programme, l'opération `MeasureOneQubit()` est le point d'entrée du programme, car elle a l'attribut `@EntryPoint()` avant la définition de l'opération :

Q#

```
@EntryPoint()
operation MeasureOneQubit() : Result {
 ...
}
```

Toutefois, le programme peut également être écrit sans l'attribut `@EntryPoint()` en renommant l'opération `MeasureOneQubit()` `Main()` , par exemple :

Q#

```
// The Q# compiler automatically detects the Main() operation as the entry point.

operation Main() : Result {
 // Allocate a qubit. By default, it's in the 0 state.
 use q = Qubit();
 // Apply the Hadamard operation, H, to the state.
 // It now has a 50% chance of being measured as 0 or 1.
 H(q);
 // Measure the qubit in the Z-basis.
 let result = M(q);
 // Reset the qubit before releasing it.
 Reset(q);
 // Return the result of the measurement.
 return result;
}
```

# Types

Les types sont essentiels dans n'importe quel langage de programmation, car ils définissent les données qu'un programme peut utiliser. Q# fournit des types intégrés qui sont communs à la plupart des langages, notamment `Int`, `Double`, `Bool`, et `String` des types qui définissent des plages, des tableaux et des tuples.

Q# fournit également des types spécifiques à l'informatique quantique. Par exemple, le `Result` type représente le résultat d'une mesure qubit et peut avoir deux valeurs : `Zero` ou `One`.

Dans le programme `Superposition`, l'opération `MeasureOneQubit()` renvoie un type `Result`, qui correspond au type de retour de l'opération `M`. Le résultat de la mesure est stocké dans une nouvelle variable définie à l'aide de l'instruction `let` :

Q#

```
// The operation definition returns a Result type.
operation MeasureOneQubit() : Result {
 ...
 // Measure the qubit in the Z-basis, returning a Result type.
 let result = M(q);
 ...
}
```

Un autre exemple de type spécifique au quantum est le `Qubit` type, qui représente un bit quantique.

Q# vous permet également de définir vos propres types personnalisés. Pour plus d'informations, consultez [Déclarations de type](#).

## Allocation de qubits

Dans Q#, vous allouez des qubits à l'aide du `use` mot clé et du `Qubit` type. Les qubits sont toujours alloués dans l'état  $|0\rangle$ .

Par exemple, le `Superposition` programme définit un qubit unique et le stocke dans la variable `q`:

Q#

```
// Allocate a qubit.
use q = Qubit();
```

Vous pouvez également allouer plusieurs qubits et accéder à chacun par le biais de son index :



Q#

```
use qubits = Qubit[2]; // Allocate two qubits.
H(qubits[0]); // Apply H to the first qubit.
X(qubits[1]); // Apply X to the second qubit.
```

## Opérations quantiques

Après avoir alloué un qubit, vous pouvez le transmettre aux opérations et aux fonctions. Les opérations sont les blocs de construction de base d'un Q# programme. Une Q# opération est une sous-routine quantique ou une routine appelante qui contient des opérations quantiques qui modifient l'état du registre qubit.

Pour définir une Q# opération, vous spécifiez un nom pour l'opération, ses entrées et sa sortie. Dans le programme `Superposition`, l'opération `MeasureOneQubit()` ne prend aucun paramètre et retourne un type `Result`.

Q#

```
operation MeasureOneQubit() : Result {
 ...
}
```

Voici un exemple de base qui ne prend aucun paramètre et attend aucune valeur de retour. La valeur `Unit` est équivalente à `NULL` dans d'autres langues.

Q#

```
operation SayHelloQ() : Unit {
 Message("Hello quantum world!");
}
```

La Q# bibliothèque standard fournit également des opérations que vous pouvez utiliser dans des programmes quantiques, tels que l'opération Hadamard, `H` dans le `Superposition` programme. Étant donné un qubit sur la base Z, `H` place le qubit dans une superposition même, où il a une probabilité de 50 % d'être mesurée en tant que `Zero` ou `One`.

## Mesure des qubits

Bien qu'il existe de nombreux types de mesures quantiques, Q# se concentre sur les mesures projectives sur des qubits uniques, également appelées mesures Pauli.

Dans Q#, l'opération `Measure` mesure un ou plusieurs qubits dans la base Pauli spécifiée, qui peut être `PauliX`, `PauliY` ou `PauliZ`. `Measure` renvoie un type `Result` soit `Zero` soit `One`.

Pour implémenter une mesure dans la base computationnelle  $\{|0\rangle, |1\rangle\}$ , vous pouvez également utiliser l'opération `M`, qui mesure un qubit dans la base de Pauli Z. Cela équivaut `M` à `Measure([PauliZ], [qubit])`.

Par exemple, le `Superposition` programme utilise l'opération `M` :

```
Q#

// Measure the qubit in the Z-basis.
let result = M(q);
```

## Réinitialisation de qubits

Dans Q#, les qubits **doivent** être dans l'état  $|0\rangle$  lorsqu'ils sont libérés pour éviter les erreurs dans le matériel quantique. Vous pouvez réinitialiser un qubit à l'état  $|0\rangle$  à l'aide de l'opération `Reset` à la fin du programme. L'échec de la réinitialisation d'un qubit entraîne une erreur d'exécution.

```
Q#

// Reset a qubit.
Reset(q);
```

## Espaces de noms de bibliothèque standard

La Q# bibliothèque standard a des espaces de noms intégrés qui contiennent des fonctions et des opérations que vous pouvez utiliser dans les programmes quantiques. Par exemple, l'espace `Std.Intrinsic` de noms contient des opérations et des fonctions couramment utilisées, telles que `M` la mesure des résultats et `Message` l'affichage des messages utilisateur n'importe où dans le programme.

Pour appeler une fonction ou une opération, vous pouvez spécifier l'espace de noms complet ou utiliser une `import` instruction, ce qui rend toutes les fonctions et opérations de cet espace de noms disponibles et rend votre code plus lisible. Les exemples suivants appellent la même opération :

```
Q#

Std.Intrinsic.Message("Hello quantum world!");
```

```
Q#
```

```
// imports all functions and operations from the Std.Intrinsic namespace.
import Std.Intrinsic.*;
Message("Hello quantum world!");

// imports just the `Message` function from the Std.Intrinsic namespace.
import Std.Intrinsic.Message;
Message("Hello quantum world!");
```

### ❗ Remarque

Le `Superposition` programme n'a `import` pas d'instructions ni d'appels avec des espaces de noms complets. Cela est dû au fait que l'environnement Q# de développement charge automatiquement deux espaces de noms et `Std.Core Std.Intrinsic`, qui contiennent des fonctions et des opérations couramment utilisées.

Vous pouvez tirer parti de l'espace de noms `Std.Measurement`, en utilisant l'opération `MResetZ` pour optimiser le programme `Superposition`. `MResetZ` combine les opérations de mesure et de réinitialisation en une seule étape, comme dans l'exemple suivant :

Q#

```
// Import the namespace for the MResetZ operation.
import Std.Measurement.*;

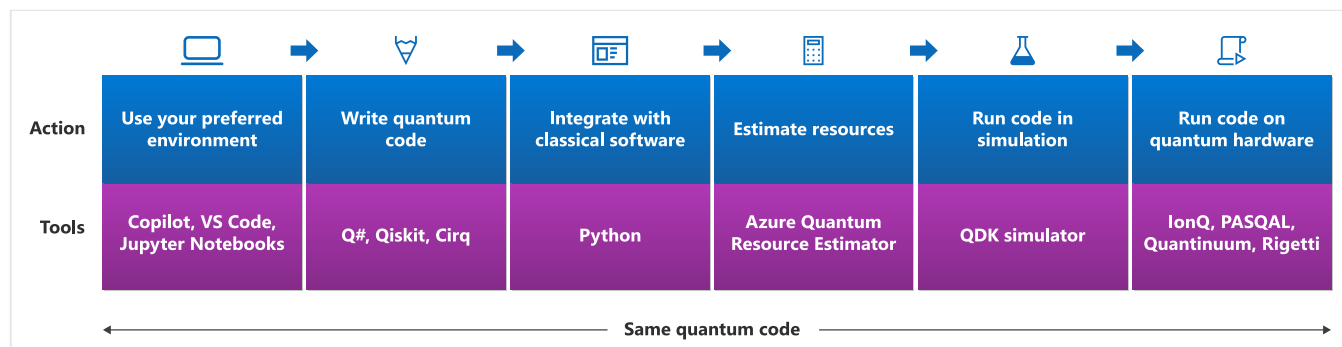
@EntryPoint()
operation MeasureOneQubit() : Result {
 // Allocate a qubit. By default, it's in the 0 state.
 use q = Qubit();
 // Apply the Hadamard operation, H, to the state.
 // It now has a 50% chance of being measured as 0 or 1.
 H(q);
 // Measure and reset the qubit, and then return the result value.
 return MResetZ(q);
}
```

## Apprendre à développer des programmes quantiques avec Q# et Azure Quantum

Q# et Azure Quantum sont une combinaison puissante pour le développement et l'exécution de programmes quantiques. Avec Q# et Azure Quantum, vous pouvez écrire des programmes quantiques, simuler leur comportement, estimer les besoins en ressources et les exécuter sur du matériel quantique réel. Cette intégration vous permet d'explorer le potentiel de l'informatique quantique et de développer des solutions innovantes pour des problèmes complexes. Qu'il s'agisse d'un débutant ou d'un développeur quantique expérimenté, Q# azure

Quantum fournit les outils et les ressources dont vous avez besoin pour déverrouiller la puissance du calcul quantique.

Le diagramme suivant montre les étapes à travers lesquelles un programme quantique passe lorsque vous le développez avec Q# et Azure Quantum. Votre programme commence par l'environnement de développement et se termine par la soumission du travail à du matériel quantique réel.



Nous allons décomposer les étapes du diagramme.

## Choisir votre environnement de développement

Exécutez vos programmes quantiques dans votre environnement de développement préféré. Vous pouvez utiliser l'éditeur de code en ligne dans le site web Microsoft Quantum, un environnement de développement local avec Visual Studio Code ou combiner Q# du code avec du code Python dans jupyter Notebooks. Pour plus d'informations, consultez [Différentes façons d'exécuter Q# des programmes](#).

## Écrire votre programme quantique

Vous pouvez écrire des programmes quantiques dans Q# en utilisant le Kit de développement quantique (QDK). Pour commencer, consultez [Démarrage rapide : Créer votre premier Q# programme](#).

En outre, le QDK offre une prise en charge d'autres langages Q# pour l'informatique quantique, comme [Qiskit](#) et [Cirq](#).

## Intégrer à Python

Vous pouvez utiliser Q# lui-même ou avec Python dans différents IDE. Par exemple, vous pouvez utiliser un Q# projet avec un programme hôte Python pour appeler Q# des opérations ou l'intégrer Q# à Python dans Jupyter Notebooks. Pour plus [d'informations, consultez Intégration de Q# et Python](#).

## La commande %%qsharp

Par défaut, Q# les programmes dans Jupyter Notebooks utilisent le `ipykernel` package Python. Pour ajouter du code Q# à une cellule de bloc-notes, utilisez la commande `%%qsharp`, qui est activée avec le paquet Python `qsharp`, suivie de votre code Q#.

Lorsque vous utilisez `%%qsharp`, gardez à l'esprit ce qui suit :

- Vous devez d'abord exécuter `from qdk import qsharp` pour activer `%%qsharp`.
- `%%qsharp` se limite à la cellule de notebook dans laquelle il apparaît et change le type de cellule de Python à Q#.
- Vous ne pouvez pas placer d'instruction Python avant ou après `%%qsharp`.
- Q# le code qui suit `%%qsharp` doit respecter la Q# syntaxe. Par exemple, utilisez `//` plutôt que de `#` désigner des commentaires et `;` pour mettre fin aux lignes de code.

## Estimer les ressources

Avant d'exécuter sur du matériel quantique réel, vous devez déterminer si votre programme peut s'exécuter sur du matériel existant et le nombre de ressources qu'il consommera.

L'estimateur de ressources Azure Quantum vous permet d'évaluer les décisions architecturales, de comparer les technologies qubit et de déterminer les ressources nécessaires pour exécuter un algorithme quantique donné. Vous pouvez choisir parmi les protocoles prédéfinis à tolérance de panne et spécifier des hypothèses du modèle qubit physique sous-jacent.

Pour plus d'informations, consultez [Exécuter votre première estimation](#) de ressource.

### ⚠ Remarque

L'estimateur de ressources Azure Quantum est gratuit et ne nécessite pas de compte Azure.

## Exécuter votre programme dans la simulation

Lorsque vous compilez et exécutez un programme quantique, le QDK crée une instance du simulateur quantique et lui transmet le Q# code. Le simulateur utilise le code Q# pour créer des qubits (simulations de particules quantiques) et appliquer des transformations afin de changer leur état. Les résultats des opérations quantiques dans le simulateur sont ensuite retournés au programme. L'isolation du code Q# dans le simulateur garantit que les algorithmes suivent les lois de la physique quantique et s'exécutent correctement sur les ordinateurs quantiques.

# Soumettre votre programme à du matériel quantique réel

Une fois que vous avez testé votre programme en simulation, vous pouvez l'exécuter sur du matériel quantique réel. Lorsque vous exécutez un programme quantique dans Azure Quantum, vous créez et exécutez un **travail**. Pour envoyer un travail aux fournisseurs Azure Quantum, vous avez besoin d'un compte Azure et d'un espace de travail quantique. Si vous n'avez pas d'espace de travail quantique, consultez [Créer un espace de travail](#) Azure Quantum.

Azure Quantum offre certains des matériels quantiques les plus attrayants et diversifiés. Pour connaître la liste actuelle des fournisseurs de matériel pris en charge, consultez [Fournisseurs d'informatique quantique](#).

Une fois que vous avez envoyé votre travail, Azure Quantum gère le cycle de vie des travaux, notamment la planification, l'exécution et la supervision des travaux. Vous pouvez suivre l'état de votre travail et afficher les résultats dans le portail Azure Quantum.

## Contenu connexe

- [Différentes façons d'exécuter Q# des programmes](#)
- [Configurer le Kit de développement Quantum](#)
- [Démarrage rapide : Créer votre premier Q# programme](#)

---

Last updated on 12/11/2025

# Développer des programmes OpenQASM dans le Microsoft Quantum Development Kit

Le Microsoft Quantum Development Kit (QDK) fournit différents environnements de développement pour les programmes OpenQASM avec l'intégration Azure Quantum. Dans cet article, vous allez apprendre à développer et exécuter OpenQASM du code dans :

- Extension QDK dans Visual Studio Code (VS Code)
- Package QDK Python dans Jupyter Notebook.

Les outils de développement comme CodeLends, IntelliSense, la saisie semi-automatique de code et le débogage de point d'arrêt sont disponibles uniquement dans l'extension VS Code. D'autres fonctionnalités sont disponibles uniquement dans le package Python. Par exemple, vous pouvez passer des éléments appelables Q# et OpenQASM sous forme d'objets Python, et compiler en QIR des programmes OpenQASM qui acceptent des entrées.

## Conditions préalables

- Installer la dernière version de [VS Code](#)
- Pour travailler directement avec les fichiers OpenQASM, installez la dernière version de l'extension [QDK](#) dans VS Code.
- Pour utiliser OpenQASM dans Python et Jupyter Notebook, installez la dernière version de l'extension [Python](#) et l'extension [Jupyter](#) dans VS Code.
- Installez la dernière version du `qdk` package Python avec les `jupyter` éléments supplémentaires :

Bash

```
pip install --upgrade "qdk[jupyter]"
```

## Travaillez avec OpenQASM dans VS Code en utilisant l'extension QDK

Pour écrire OpenQASM code dans VS Code, ouvrez un projet Q# et créez un fichier OpenQASM avec une extension `.qasm`. Le système QDK reconnaît automatiquement le fichier `.qasm` comme

OpenQASM, et les fonctionnalités QDK pour OpenQASM sont disponibles au fur et à mesure que vous rédigez votre code.

Les fonctionnalités suivantes QDK sont disponibles pour les programmes OpenQASM dans VS Code:

- Complétion de code
- Lentilles de code (**Run**, **Histogram**, **Estimate**, **Debug** et **Circuit**)
- Mise en surbrillance de la syntaxe et fonctions de base de syntaxe, telles que l'appariement des accolades
- Vérification des erreurs dans les OpenQASM fichiers sources
- Débogage par points d'arrêt et exécution de scripts pour les fichiers source OpenQASM
- Débogage avec génération de circuits en direct
- Intégration avec Azure Quantum pour la soumission de tâches quantiques

Pour exécuter votre programme OpenQASM à partir de VS Code, vous pouvez utiliser le simulateur local QDK ou envoyer le code en tant que travail à Azure Quantum.

## Exécuter votre code sur le QDK simulateur local

Pour exécuter votre code OpenQASM sur le simulateur quantique local sparse dans VS Code, ouvrez un fichier `.qasm`, puis sélectionnez la commande **Exécuter** dans l'indicateur CodeLens au début du fichier. La sortie de votre programme s'affiche dans le VS Code terminal.

### ⓘ Remarque

Le QDK tente automatiquement de définir un profil QIR target qui prend en charge votre programme. Si le profil QIR target ne prend pas en charge les opérations dans votre programme, vous obtenez des erreurs de compilateur lorsque vous exécutez votre OpenQASM code. Pour plus d'informations sur les profils target, consultez [Types de profils target dans Azure Quantum](#).

## Exemples de programmes préinstallés

Le QDK est fourni avec plusieurs exemples intégrés d'algorithmes quantiques pour OpenQASM. Pour essayer les exemples d'algorithmes, entrez `sample` dans un fichier `.qasm` vide. Une liste d'achèvement s'affiche qui contient les exemples d'algorithmes. Choisissez un exemple, puis exécutez le code généré.



## Débuguer localement dans VS Code

Le QDK inclut un débogueur pour OpenQASM. Le débogueur prend en charge le débogage de style point d'arrêt et peut afficher l'état du programme.

Pour démarrer le débogueur, sélectionnez la commande **Débuguer** dans le code lens.

En mode débogage, vous pouvez définir des points d'arrêt dans votre code pour vous aider à déboguer. L'état de votre programme s'affiche dans le volet **RUN AND DEBUG**. Par exemple, vous pouvez afficher les états de vos variables locales et les états qubits.

## Exécutez votre code sur Azure Quantum

Pour exécuter des OpenQASM programmes sur Azure Quantum targets, vous devez disposer d'un Azure Quantum espace de travail. Pour créer un Azure Quantum espace de travail, consultez [Créer un Azure Quantum espace de travail](#).

Pour soumettre des travaux OpenQASM à Azure Quantum à partir de VS Code, vous devez vous connecter à l'espace de travail Azure Quantum. Pour vous connecter à votre espace de travail à partir de VS Code, consultez [Se connecter à l'aide d'une chaîne de connexion](#).

Pour soumettre une OpenQASM tâche à Azure Quantum depuis VS Code, procédez comme suit :

1. Ouvrez le `qasm` fichier qui contient votre programme dans VS Code.
2. Sélectionnez l'icône **Microsoft Quantum** pour ouvrir le panneau Microsoft Quantum.
3. Développez la liste déroulante **QUANTUM WORKSPACES** et sélectionnez votre espace de travail.
4. Développez la liste déroulante **Fournisseurs** et choisissez un fournisseur, puis choisissez celui target sur lequel vous souhaitez exécuter votre travail. Par exemple, soumettez votre tâche au `rigetti.svm.sim` target pour exécuter votre programme sur le simulateur backend de Rigetti. Pour plus d'informations sur le fournisseur et target la disponibilité, consultez [Ajouter ou supprimer un fournisseur dans un espace de travail existant](#).
5. Sélectionnez l'icône de lecture en regard du nom de target.
6. Pour soumettre la tâche, saisissez des valeurs pour les requêtes **Nom de la tâche** et **Nombre de prises**.

Pour afficher les heures et l'état d'un travail, développez la liste déroulante **Travaux** et pointez sur le nom du travail. Pour télécharger les résultats du travail, sélectionnez l'histogramme ou les icônes de texte en regard du nom du travail.

# Utiliser OpenQASM dans Python et Jupyter Notebook

Avec le package Python QDK, vous pouvez exécuter directement du code OpenQASM et transmettre des circuits OpenQASM aux programmes Q# sous forme d'opérations. Écrivez votre code OpenQASM en tant que chaîne Python, puis utilisez cette chaîne pour exécuter, importer ou compiler votre code OpenQASM dans l'environnement Python.

## Exécutez OpenQASM directement dans Python

Pour exécuter OpenQASM du code directement, utilisez la fonction `run` du module `qdk.openqasm`. Par exemple, le code suivant exécute 10 tirs sur le simulateur local d'un programme simple OpenQASM avec bruit :

### Python

```
from qdk import BitFlipNoise
from qdk.openqasm import run

qasm_code = """
 include "stdgates.inc";
 qubit[2] q;
 reset q;
 h q[0];
 cx q[0], q[1];
 bit c = measure q[1];
 """

results = run(qasm_code, shots=10, noise=BitFlipNoise(0.1))
print(results)
```

## Appeler un OpenQASM programme dans Q#

Vous pouvez utiliser la fonction `import_openqasm` pour stocker un programme OpenQASM en tant qu'objet Python, puis appeler cet objet plus loin dans votre code de Python. Par exemple, la cellule suivante stocke un circuit OpenQASM dans un objet Python appelé `bell` :

### Python

```
from qdk import qsharp
from qdk.openqasm import import_openqasm

qasm_code = """
 include "stdgates.inc";
 qubit[2] q;
 reset q;
 h q[0];
```

```

cx q[0], q[1];
bit c = measure q[1];
"""

import_openqasm(qasm_code, name="bell")

```

Quand `bell` il fait partie de votre environnement, vous pouvez l'exécuter directement à partir d'une cellule Q# :

```

Q#

%%qsharp

use qs = Qubit[2];
bell(qs)

```

Vous pouvez également utiliser la fonctionnalité du package QDK, telle que la simulation bruyante, le rendu de circuit `bell` et la génération de code.

## Programmes OpenQASM paramétrables

La cellule suivante crée un circuit OpenQASM qui prend un angle `theta` comme entrée, puis appelle le circuit en tant que fonction Python :

```

Python

from qdk.openqasm import import_openqasm, ProgramType

qasm_code = """
include "stdgates.inc";
input float theta;
qubit[2] q;
rx(theta) q[0];
rx(-theta) q[1];
bit[2] c;
c = measure q;
"""

import_openqasm(qasm_code, name="parameterized_circuit",
program_type=ProgramType.File)

from qdk.code.qasm_import import import parameterized_circuit

parameterized_circuit(1.57)

```

## Passer un programme OpenQASM à Q#

Avec le QDK package Python, vous pouvez passer directement un OpenQASM programme à un programme Q#. Par exemple, le code suivant crée un OpenQASM programme appelé `Entangle` , puis utilise `Entangle` comme opération dans un programme Q# :

#### Python

```
from qdk.openqasm import import_openqasm
from qdk import qsharp

qasm_code = """
 include "stdgates.inc";
 qubit[2] qs;
 h qs[0];
 cx qs[0], qs[1];
 """

import_openqasm(qasm_code, name="Entangle")

qsharp.eval("{ use qs = Qubit[2]; Entangle(qs); MResetEachZ(qs) }")
```

Lorsque vous importez Q# et importez des programmes OpenQASM dans un environnement Python avec `qsharp.eval` et `import_openqasm`, ces programmes deviennent des objets Python que vous pouvez appeler dans votre code Python. Par exemple, le code suivant crée un programme Q# appelé `TestAntiCorrelation` et transmet le `Entangle` OpenQASM programme à `TestAntiCorrelation`:

#### Python

```
qsharp.eval("""
 operation TestAntiCorrelation(entangler : Qubit[] => Unit) : Result[] {
 use qs = Qubit[2];
 X(qs[1]);
 entangler(qs);
 MResetEachZ(qs)
 }
 """)

from qdk.code import Entangle, TestAntiCorrelation

TestAntiCorrelation(Entangle)
```

## Compiler des programmes OpenQASM en QIR

Vous pouvez compiler du OpenQASM code en représentation intermédiaire quantique (QIR) avec la fonction `compile`.

Par exemple, la cellule suivante compile le OpenQASM code d'un circuit paramétré en QIR :

## Python

```
from qdk import TargetProfile
from qdk.openqasm import import_openqasm, ProgramType
from qdk import qsharp

qsharp.init(target_profile=TargetProfile.Base)

qasm_code = """
 include "stdgates.inc";
 input float theta;
 qubit[2] q;
 rx(theta) q[0];
 rx(-theta) q[1];
 bit[2] c;
 c = measure q;
 """

import_openqasm(qasm_code, name="parameterized_circuit",
program_type=ProgramType.File)

from qdk.code.qasm_import import import parameterized_circuit

bound_compilation = qsharp.compile(parameterized_circuit, 1.57)
print(bound_compilation)
```

### ⚠ Remarque

Pour compiler OpenQASM des programmes qui prennent des variables d'entrée dans QIR, vous devez utiliser le QDK package Python. Vous ne pouvez pas compiler de programmes paramétrables OpenQASM en QIR à l'aide de l'extension QDK dans VS Code.

## Envoyer des OpenQASM programmes à Azure Quantum

Pour soumettre un programme OpenQASM pour qu'il s'exécute sur un Azure Quantum target, compilez le code OpenQASM en QIR, puis soumettez le QIR pour exécution sur Azure Quantum.

Par exemple, compilez un circuit d'intrication simple en QIR.

## Python

```
from qdk.openqasm import compile

qasm_code = """
 include "stdgates.inc";
 bit[2] c;
 qubit[2] q;
 h q[0];
```

```
cx q[0], q[1];
c = measure q;
"""

qir_compilation = compile(qasm_code)
```

Pour exécuter une tâche QIR vers Azure Quantum, connectez-vous à votre espace de travail quantique et soumettez la tâche à l'une des ressources disponibles targets dans votre espace de travail. Par exemple, soumettez le QIR d'enchevêtrement pour l'exécuter sur le simulateur backend d'IonQ.

#### Python

```
from qdk.azure import Workspace

workspace = Workspace(resource_id="") # Add the resource ID of your workspace

target = workspace.get_targets('ionq.simulator')
job = target.submit(qir_compilation, shots=100)

print("Job submitted. Waiting for results...")

results = job.get_results()
print(results)
```

## Développement continu OpenQASM dans le QDK

La prise en charge de OpenQASM dans le QDK est en cours.

Pour obtenir la liste des fonctionnalités OpenQASM en cours de développement dans le QDK, consultez [Limitations et problèmes connus](#) sur le wiki QDK GitHub.

Last updated on 14/08/2026

# Vue d'ensemble des simulateurs quantiques dans le QDK

Le Microsoft Quantum Development Kit (QDK) inclut un ensemble de simulateurs quantiques qui vous permettent d'exécuter des programmes quantiques sur votre ordinateur local. Utilisez les simulateurs locaux pour itérer sur vos programmes avant de les soumettre à des cibles Azure Quantum.

Le QDK possède quatre simulateurs.

- Le simulateur épars
- Simulateur Clifford
- Simulateur GPU
- Simulateur CPU

## Le simulateur épars

Le simulateur épars est le simulateur par défaut dans le QDK. Ce simulateur représente des états qubits en tant que vecteurs épars pour tirer parti de l'algèbre de matrice épars pour les simulations rapides, en particulier pour les programmes qui ne génèrent pas de nombreux états de superposition. Utilisez le simulateur épars lorsque vous souhaitez effectuer des simulations rapides de programmes Q#, OpenQASM ou Qiskit, ou pour afficher l'état quantique de votre programme pendant la simulation.

Le simulateur sparse est disponible à la fois dans l'extension QDK pour Visual Studio Code (VS Code) et dans le package QDKPython. Ce simulateur est le seul simulateur disponible dans l'extension QDK, donc le compilateur appelle automatiquement le simulateur parcimonieux lorsque vous exécutez les programmes Q# et OpenQASM dans VS Code. Avec le simulateur épars dans VS Code, vous pouvez utiliser la fonction Q# `DumpMachine` ou le débogueur pour afficher des vecteurs d'état qubit à différents points de la simulation.

L'extension VS Code et le package Python prennent toutes deux en charge les modèles de bruit pour les simulations éparses de programmes OpenQASM et Q#. Seul le package Python prend en charge la simulation épars pour les programmes Qiskit, mais sans prise en charge du modèle de bruit.

Pour plus d'informations sur le simulateur creux, consultez [le simulateur creux](#).

# Simulateur Clifford

Le simulateur Clifford est rapide et efficace pour les programmes comportant jusqu'à plusieurs milliers de qubits, mais ne peut simuler que des programmes contenant uniquement des portes Clifford. Les portes Clifford sont des composants courants des circuits de correction d'erreurs. L'ensemble de Clifford portes se compose des portes  $H$ ,  $S$  et  $CX$ , et toutes les portes qui peuvent être construites à partir de ces trois portes. Les portes suivantes Clifford sont des composants courants des circuits quantiques :

- $X$ ,  $Y$  et  $Z$
- $S$  et  $S^\dagger$
- $H$ , ou Hadamard
- $CX$ ,  $CY$  et  $CZ$
- SWAP et iSWAP

Le simulateur Clifford est disponible dans le package QDKPython pour les programmes Q#, OpenQASM, Qiskit et QIR, mais n'est pas disponible dans l'extension QDK pour VS Code.

# Simulateur GPU

Le simulateur GPU est un simulateur à état complet qui utilise les commandes GPU de votre machine pour exécuter de nombreuses exécutions de votre programme quantique en parallèle. Ce simulateur peut exécuter des programmes qui contiennent n'importe quel type de porte, mais est coûteux à exécuter. Le simulateur peut modéliser jusqu'à 27 qubits. La puissance de votre machine GPU détermine les performances, mais celles-ci sont optimales pour les programmes comportant environ 20 qubits et un grand nombre d'exécutions.

Le simulateur GPU prend Qiskit ou QIR en entrée, et est disponible via les API de certains packages QDKPython, mais pas dans l'extension VS Code. Il existe également un support étendu pour les modèles de bruit sur n'importe quel type de porte quantique ou d'opération.

# Simulateur CPU

Comme le GPU simulateur, le CPU simulateur est un simulateur à état complet qui peut exécuter des programmes qui contiennent n'importe quel type de porte quantique. Toutefois, le CPU simulateur est plus lent, car il ne peut pas exécuter plusieurs tirs en parallèle. Ce simulateur effectue un scale-up jusqu'à environ 25 qubits, mais ralentit de façon exponentielle avec plus de qubits en raison de problèmes de mémoire. Utilisez le CPU simulateur lorsque votre machine n'a



pas de GPU, ou lorsque votre programme a peu de qubits et que vous n'exécutez pas un grand nombre d'exécutions.

Le simulateur CPU prend Qiskit ou QIR en entrée, et est disponible via les API de certains packages QDKPython, mais pas dans l'extension VS Code. Il existe également un support étendu pour les modèles de bruit sur n'importe quel type de porte quantique ou d'opération.

## Quel simulateur dois-je utiliser ?

Le meilleur simulateur à utiliser dépend de plusieurs facteurs, tels que :

- Votre environnement de développement (VS Code ou Python)
- Infrastructure de programmation quantique dans laquelle vous développez
- La complexité de votre programme et le nombre de tirs
- Fonctionnalités matérielles de votre ordinateur local
- Les cibles Azure Quantum ou tout autre matériel quantique sur lequel vous souhaitez exécuter vos programmes
- Modèles de bruit que vous souhaitez créer

Le tableau suivant récapitule les QDK options et les frameworks de programmation quantique que vous pouvez utiliser pour chaque simulateur :

 Agrandir le tableau

Simulateur	Availability	Frameworks pris en charge
Partiellement alloué	Extension VS Code et package Python	Q#, OpenQASM, Qiskit
Clifford	Python paquet	Q#, OpenQASM, Qiskit, QIR
GPU	Python paquet	Qiskit, QIR
CPU	Python paquet	Qiskit, QIR

## Simulations dans l'extension QDK pour VS Code

Le simulateur sparse est le seul simulateur disponible dans l'extension QDK pour VS Code. Pour utiliser le simulateur parcimonieux, exécutez votre fichier Q# ou OpenQASM dans VS Code. Vous pouvez ajouter des modèles de bruit limités au simulateur clairsemé pour les programmes Q# dans VS Code, mais pas pour les programmes OpenQASM.

## Simulations dans le QDKPython package

L'environnement de développement QDKPython prend en charge plusieurs frameworks quantiques et les quatre simulateurs QDK, mais tous les simulateurs ne sont pas compatibles avec tous les frameworks et API.

Ce tableau présente les simulateurs que vous pouvez utiliser pour chaque Python API qui appelle un QDK simulateur et l'infrastructure de programmation prise en charge par chaque API :

[🔗](#) Agrandir le tableau

Python Interface de Programmation d'Applications (API)	Partiellement alloué	Clifford	GPU	CPU	Framework de programmation
<code>qsharp.run</code>	✓	✓			Q#
<code>openqasm.run</code>	✓	✓			OpenQASM
<code>qiskit.QSharpBackend</code>	✓				Qiskit
<code>simulation.NeutralAtomDevice</code>		✓	✓	✓	QIR
<code>simulation.NeutralAtomBackend</code>		✓	✓	✓	Qiskit
<code>simulation.run_qir</code>		✓	✓	✓	QIR

## Contenu connexe

- [Comment installer et exécuter les QDK simulateurs quantiques](#)
- [Comment créer des modèles de bruit pour les simulations quantiques dans le QDK](#)
- [Simulateurs quantiques backend de fournisseurs quantiques](#)

Last updated on 14/08/2026

# Comment installer et exécuter les QDK simulateurs quantiques

Le Microsoft Quantum Development Kit (QDK) inclut un ensemble de simulateurs quantiques auxquels vous pouvez accéder à partir de l'extension Visual Studio Code (VS Code) et du QDKPython package. Ces simulateurs vous permettent de tester la façon dont vos programmes quantiques s'exécutent sur du matériel quantique réel. Cet article explique comment installer les simulateurs et comment exécuter des simulations de base dans Jupyter Notebook et VS Code.

Il QDK comprend quatre simulateurs quantiques :

- Simulateur de parcimonie
- Simulateur Clifford
- Simulateur GPU
- Simulateur de CPU

Pour plus d'informations sur les QDK simulateurs quantiques, consultez [Vue d'ensemble des simulateurs quantiques dans le QDK](#).

## Prerequisites

Pour suivre les étapes décrites dans cet article, vous devez installer les outils suivants :

- Un environnement Python (version 3.10 ou ultérieure), avec Python et Pip
- La dernière version de [Visual Studio Code \(VS Code\)](#) [↗](#)
- Les dernières versions de l'extension [Python](#) [↗](#) et [Jupyter](#) [↗](#) dans VS Code

## Installer les simulateurs

Les quatre simulateurs sont disponibles dans le QDKpackage, mais seul le simulateur partiellement alloué est disponible dans l'extensionQDKVS Code.Python

Pour utiliser le simulateur sparse dans l'extension VS Code, installez l'extension [QDK pour VS Code](#) [↗](#).

Pour utiliser les simulateurs quantiques dans Python et Jupyter Notebookinstaller la dernière version duPython `qdk` package avec les `jupyter` éléments supplémentaires :

Bash

```
pip install --upgrade "qdk[jupyter]"
```

L'extra `jupyter` n'est pas nécessaire pour utiliser les simulateurs, mais installe le `qdk.widgets` module. Le module `widgets` vous permet de créer des visualisations à partir de vos résultats de simulation dans Jupyter Notebook.

## Exécuter des simulations à l'aide de l'extension QDK pour VS Code

L'extension QDK pour VS Code n'offre que le simulateur sparse. Pour exécuter votre programme sur le simulateur sparse dans VS Code, ouvrez un programme Q# (`.qs`) ou OpenQASM (`.qasm`) dans VS Code et exécutez le fichier.

Par exemple, suivez ces étapes pour exécuter l'un des exemples de programmes Q# fournis avec l'extension QDK :

1. Dans VS Code, ouvrez le menu **Fichier** et sélectionnez **Nouveau fichier texte**.
2. Donnez au fichier un nom qui se termine par l'extension `.qs`.
3. Dans le fichier vide, entrez **l'exemple**. Une liste d'exemples de code inclus s'ouvre.
4. Choisissez un exemple de code, par exemple **Bell Pair sample**.
5. Pour exécuter le programme sur le simulateur sparse, sélectionnez la commande **Exécuter** dans la lentille de code au-dessus de l'opération **Main**.

Le VS Code terminal affiche la sortie de la simulation.

### ❗ Remarque

Vous pouvez ajouter des modèles de bruit limités aux simulations éparées des programmes Q# dans l'extension QDK . Pour plus d'informations, consultez [Ajouter du bruit de Pauli au simulateur creux dans VS Code](#).

## Exécuter des simulations à l'aide du QDKPython package

Le QDKPython package offre les quatre simulateurs. L'appel des simulateurs dépend du simulateur et de l'infrastructure de programmation quantique.

# Appeler le simulateur partiellement alloué

Vous pouvez utiliser le simulateur sparse pour les programmes Q#, OpenQASM et Qiskit. Le tableau suivant montre comment appeler le simulateur partiellement alloué à partir du `qdk` package pour 10 captures dans chaque infrastructure de programmation quantique prise en charge.

[Agrandir le tableau](#)

Framework de programmation	Python API	Exemple d'appel
Q#	<code>qsharp.run</code>	<code>qsharp.run("MyQsharpProgram()", shots=10)</code>
OpenQASM	<code>openqasm.run</code>	<code>openqasm.run(my_qasm_program, shots=10)</code>
Qiskit	<code>qiskit.QSharpBackend</code>	<code>QSharpBackend().run(my_qiskit_program, shots=10)</code>

## ⓘ Remarque

Vous pouvez ajouter des modèles de bruit au simulateur sparse à l'aide du paramètre `noise` pour les programmes Q# et OpenQASM, mais vous ne pouvez pas ajouter de modèles de bruit pour les programmes Qiskit.

Par exemple, suivez ces étapes pour exécuter le programme Q# de paire Bell dans un Jupyter bloc-notes dans VS Code:

1. Dans VS Code, ouvrez le menu **Affichage** et sélectionnez **Palette de commandes**.
2. Entrez **Créer : Nouveau Jupyter Notebook**. Un fichier de notebook vide s'ouvre dans un nouvel onglet.
3. Dans la première cellule, importez le `qsharp` module à partir du `qdk` package.

Python

```
from qdk import qsharp
```

4. Dans une nouvelle cellule, utilisez la `%qsharp` commande magique pour écrire le code Q#.

Q#

```
%%qsharp

operation Main() : (Result, Result) {
 use (q1, q2) = (Qubit(), Qubit());
 PrepareBellPair(q1, q2);
 (MResetZ(q1), MResetZ(q2))
}

operation PrepareBellPair(q1 : Qubit, q2 : Qubit) : Unit {
 H(q1);
 CNOT(q1, q2);
}
```

5. Créez une cellule. Pour exécuter le programme Q# sur le simulateur creux, appelez la fonction `run` du module `qsharp` et spécifiez le nombre d'exécutions.

#### Python

```
qsharp.run("Main()", shots=10)
```

## Appeler le simulateur Clifford

Vous pouvez appeler le simulateur Clifford pour les programmes Q#, OpenQASM, Qiskit et QIR. Le tableau suivant montre comment appeler le simulateur Clifford à partir du `qdk` package pour 10 captures dans chaque infrastructure de programmation quantique prise en charge.

[🔗](#) Agrandir le tableau

Framework de programmation	Python API	Exemple d'appel
Q#	<code>qsharp.run</code>	<code>qsharp.run("MyQsharpProgram()", type='clifford', shots=10)</code>
OpenQASM	<code>openqasm.run</code>	<code>openqasm.run(my_qasm_program, type='clifford', shots=10)</code>
Qiskit	<code>simulation.NeutralAtomBackend</code>	<code>simulation.NeutralAtomBackend().run(my_qiskit_program, simulator_type='clifford', shots=10)</code>
QIR	<code>simulation.NeutralAtomDevice</code>	<code>simulation.NeutralAtomDevice().simulate(my_qir, type='clifford', shots=10)</code>
QIR	<code>simulation.run_qir</code>	<code>simulation.run_qir(my_qir, type='clifford', shots=10)</code>

ⓘ Remarque

Les API `NeutralAtomDevice` et `NeutralAtomBackend` sont spécifiquement destinées aux simulations sur des ordinateurs quantiques à atomes neutres. Pour plus d'informations, consultez [vue d'ensemble du simulateur d'appareil atom neutre](#).

Par exemple, le code Python suivant crée un programme OpenQASM simple et exécute le programme sur le simulateur Clifford :

Python

```
from qdk.openqasm import run

qasm_code = """
 include "stdgates.inc";
 qubit[2] q;
 reset q;
 h q[0];
 cx q[0], q[1];
 bit c = measure q[1];
 """

run(qasm_code, type='clifford', shots=10)
```

## Appeler les simulateurs GPU et CPU

Vous pouvez utiliser les simulateurs GPU et CPU pour les programmes Qiskit et QIR. Le tableau suivant montre comment appeler le simulateur GPU à partir du `qdk` package pour 10 captures dans chacune des infrastructures de programmation quantique prises en charge. Pour appeler le simulateur de processeur, remplacez `'gpu'` par `'cpu'` dans les paramètres de type du simulateur.

 Agrandir le tableau

Framework de programmation	Python API	Exemple d'appel
Qiskit	<code>simulation.NeutralAtomBackend</code>	<code>simulation.NeutralAtomBackend().run(my_qiskit_program, simulator_type='gpu', shots=10)</code>
QIR	<code>simulation.NeutralAtomDevice</code>	<code>simulation.NeutralAtomDevice().simulate(my_qir, type='gpu', shots=10)</code>
QIR	<code>simulation.run_qir</code>	<code>simulation.run_qir(my_qir, type='gpu', shots=10)</code>

Par exemple, le code Python suivant convertit un programme OpenQASM simple en QIR et exécute le QIR sur le simulateur GPU :

## Python

```
from qdk.openqasm import compile#, ProgramType
from qdk.simulation import run_qir

qasm_code = """
 include "stdgates.inc";
 qubit[2] q;
 reset q;
 h q[0];
 cx q[0], q[1];
 bit c = measure q[1];
 """

qir = compile(qasm_code)
run_qir(qir, type='gpu', shots=10)
```

---

Last updated on 31/07/2026



# Simulateur quantique parcimonieux

Le simulateur épars utilise une représentation épars des vecteurs d'état quantique. Un état quantique parcimonieux est un état où la plupart des coefficients d'amplitude sont des zéros. Les représentations éparss permettent au simulateur épars de réduire l'empreinte mémoire requise pour représenter les états quantiques, ce qui permet des simulations sur un plus grand nombre de qubits. Le simulateur clairsemé est efficace pour les programmes avec des états quantiques clairsemés dans la base de calcul. Le simulateur épars permet aux utilisateurs d'explorer de plus grandes applications qu'un simulateur à état complet, car les simulateurs à état complet gaspillent de la mémoire et du temps sur un nombre exponentiellement élevé d'amplitudes nulles.

Pour plus d'informations sur le simulateur de parcimonie, consultez [Jaques et Häner \(arXiv:2105.01533\)](#).

## Appeler le simulateur de parcimonie

Le simulateur épars est le simulateur local par défaut dans l'extension MicrosoftQuantum Development Kit (QDK) pour Visual Studio Code (VS Code). L'utilisation du simulateur de parcimonie dépend de votre environnement de développement.

 Agrandir le tableau

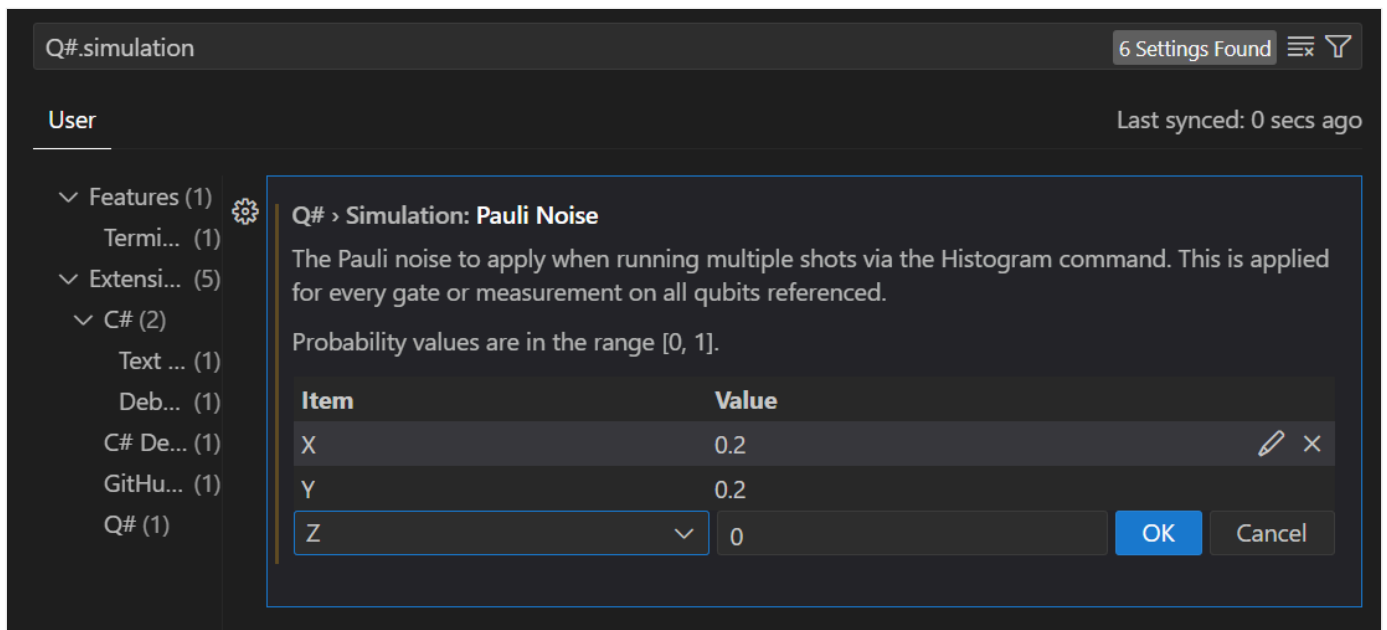
Environnement de développement	Comment appeler le simulateur de parcimonie
Dans un programme Q# ou OpenQASM dans VS Code	Exécuter le fichier Q# ou OpenQASM
Dans un programme Python avec le paquet <code>qdk</code>	<code>qsharp.run</code> OU <code>openqasm.run</code> OU <code>qiskit.QSharpBackend</code>
Dans une cellule de notebook <code>%%qsharp</code>	Appelez l'opération d'entrée de programme, par exemple : <code>Main()</code>

## Ajouter du bruit Pauli au simulateur de parcimonie dans VS Code

Le simulateur de parcimonie prend en charge l'ajout de bruit Pauli aux simulations de vos programmes Q# avec l'extension VS Code. Cette fonctionnalité vous permet de simuler les effets du bruit sur les opérations quantiques et les mesures. Pour spécifier un modèle de bruit dans votre Q# programme, utilisez la `ConfigurePauliNoise` fonction. La fonction définit les probabilités pour les opérateurs Pauli `X`, `Y` et `Z`, que le bruit se produise. Vous pouvez également configurer des modèles de bruit globaux dans les paramètres d'extension.

## Ajouter du bruit Pauli à l'aide des paramètres VS Code

Pour définir le bruit Pauli global pour les programmes Q# dans VS Code, configurez le paramètre utilisateur **Q#> Simulation :Pauli Bruit** pour l'extension QDK.



Les paramètres de bruit s'appliquent aux résultats de l'histogramme pour toutes les portes, mesures et qubits dans tous les Q# programmes dans VS Code.

Par exemple, un histogramme pour l'exemple de programme GHz suivant sans bruit configuré montre un résultat de  $|00000\rangle$  pour environ la moitié des mesures et  $|11111\rangle$  pour l'autre moitié.

```
Q#

import Std.Diagnostics.*;
import Std.Measurement.*;

operation Main() : Result []{
 let num = 5;
 return GHzSample(num);
}

operation GHzSample(n: Int) : Result[] {
 use qs = Qubit[n];
 H(qs[0]);
```

```

ApplyCNOTChain(qs);
let results = MeasureEachZ(qs);
ResetAll(qs);
return results;
}

```



Si vous ajoutez même un taux de bruit de bit-flip de 1 %, les résultats commencent à se brouiller. Avec un bruit d'inversion de bits de 25 %, l'histogramme est indistinct du bruit pur.



Ajouter du bruit Pauli aux programmes Q# individuels

Utilisez la `ConfigurePauliNoise` fonction pour définir ou modifier le modèle de bruit pour des programmes individuels Q#. La `ConfigurePauliNoise` fonction vous permet de contrôler quand et où le bruit se produit dans vos Q# programmes.

#### ⚠ Remarque

Si vous configurez le bruit dans les VS Code paramètres, le bruit est appliqué à tous les Q# programmes. Toutefois, la `ConfigurePauliNoise` fonction remplace les VS Code paramètres de bruit du programme qui appelle la fonction.

Par exemple, dans le programme précédent, vous pouvez ajouter du bruit immédiatement après l'allocation de qubit.

Q#

```
operation GHZSample(n: Int) : Result[] {
 use qs = Qubit[n];

 // 5% bit-flip noise applies to all operations
 ConfigurePauliNoise(0.05, 0.0, 0.0);

 H(qs[0]);
 ApplyCNOTChain(qs);
 let results = MeasureEachZ(qs);
 ResetAll(qs);
 return results;
}
```

Total shots: 100



[Zero, Zero, Zero, Zero, Zero]  
[Zero, Zero, Zero, Zero, One]  
[Zero, Zero, Zero, One, Zero]  
[Zero, Zero, Zero, One, One]  
[Zero, Zero, One, Zero, Zero]  
[Zero, One, Zero, Zero, Zero]  
[Zero, One, Zero, Zero, One]  
[Zero, One, One, Zero, Zero]  
[Zero, One, One, Zero, One]  
[Zero, One, One, One, Zero]  
[Zero, One, One, One, One]  
[Zero, One, One, One, Zero]  
[One, Zero, One, One, One]  
[One, Zero, Zero, Zero, Zero]  
[One, Zero, Zero, One, One]  
[One, Zero, One, One, Zero]  
[One, Zero, One, One, One]  
[One, One, Zero, Zero, Zero]  
[One, One, Zero, Zero, One]  
[One, One, Zero, One, One]  
[One, One, One, Zero, Zero]  
[One, One, One, Zero, One]  
[One, One, One, One, Zero]  
[One, One, One, One, One]

22 unique results

Note: If a [noise model](#) has been configured, this may impact results

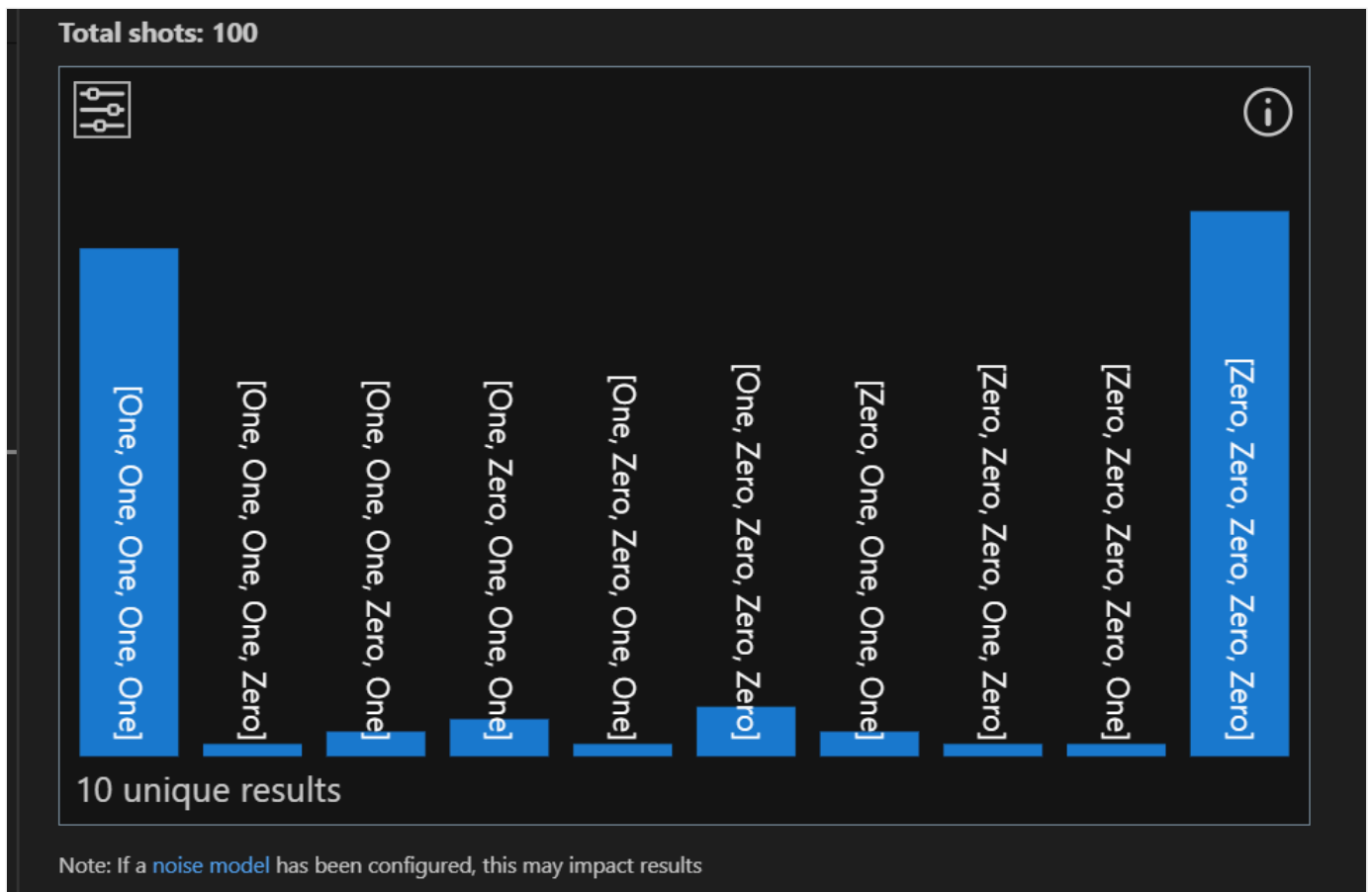
Vous pouvez également ajouter du bruit à l'opération de mesure uniquement.

Q#

```
operation GHZSample(n: Int) : Result[] {
 use qs = Qubit[n];
 H(qs[0]);
 ApplyCNOTChain(qs);

 // Noise applies to only the measurement operation
 ConfigurePauliNoise(0.05, 0.0, 0.0);

 let results = MeasureEachZ(qs);
 ResetAll(qs);
 return results;
}
```



Pour modifier ou effacer les configurations de bruit à différents points de votre programme, appelez `ConfigurePauliNoise` plusieurs fois. Par exemple, vous pouvez définir un bruit d'inversion de bits de 5 % sur la porte de Hadamard et ne pas définir de bruit pour le reste du programme.

Q#

```
operation GHZSample(n: Int) : Result[] {
 use qs = Qubit[n];

 // Noise applies to the H operation
 ConfigurePauliNoise(0.05, 0.0, 0.0);

 H(qs[0]);

 // Clear the noise settings
 ConfigurePauliNoise(0.0, 0.0, 0.0);

 ApplyCNOTChain(qs);
 let results = MeasureEachZ(qs);
 ResetAll(qs);
 return results;
}
```

**Autres Q# fonctions de bruit**

La `ConfigureNoiseFunction` fonction est suffisante pour modéliser n'importe quel type de Pauli bruit dans votre programme, mais Q# a d'autres fonctions de bruit que vous pouvez également utiliser. Les fonctions suivantes sont disponibles dans la `Std.Diagnostics` bibliothèque pour configurer le bruit dans les Q# programmes :

[🔗 Agrandir le tableau](#)

Fonction	Descriptif	Exemple
<code>ConfigurePauliNoise</code>	Configure le bruit Pauli pour une exécution de simulateur. Les paramètres représentent les probabilités de bruit de porte Pauli X, Y et Z. La configuration du bruit s'applique à toutes les portes, mesures et qubits suivants dans un programme Q#. Remplace les paramètres de bruit de l'extension VS Code . Les appels suivants à <code>ConfigurePauliNoise</code> écrasent le bruit défini par les appels précédents.	<code>ConfigurePauliNoise(0.1, 0.0, 0.5)</code> ou <code>ConfigurePauliNoise(BitFlipNoise(0.1))</code>
<code>BitFlipNoise</code>	Configure uniquement le bruit de porte X avec la probabilité spécifiée.	Bruit d'inversion de bits de 10 % : <code>ConfigurePauliNoise(BitFlipNoise(0.1))</code> $\equiv$ <code>ConfigurePauliNoise(0.1, 0.0, 0.0)</code>
<code>PhaseFlipNoise</code>	Configure uniquement le bruit de porte Z avec la probabilité spécifiée.	10 % de bruit de renversement de phase : <code>ConfigurePauliNoise(PhaseFlipNoise(0.1))</code> $\equiv$ <code>ConfigurePauliNoise(0.0, 0.0, 0.1)</code>
<code>DepolarizingNoise</code>	Configure le bruit pour être une porte X, Y ou Z avec des probabilités égales.	6% de bruit de dépolarisation <code>ConfigurePauliNoise(DepolarizingNoise(0.06))</code> $\equiv$ <code>ConfigurePauliNoise(0.2, 0.2, 0.2)</code>
<code>NoNoise</code>	Réinitialise le modèle de bruit pour qu'il n'y ait aucun bruit.	<code>ConfigurePauliNoise(NoNoise())</code> $\equiv$ <code>ConfigurePauliNoise(0.0, 0.0, 0.0)</code>
<code>ApplyIdleNoise</code>	Applique le bruit configuré à un qubit unique pendant la simulation.	<code>...</code> <code>use q = Qubit[2];</code> <code>ConfigurePauliNoise(0.1, 0.0, 0.0);</code> <code>ApplyIdleNoise(q[0]);</code> <code>...</code>

# Comment créer des modèles de bruit pour les simulations quantiques dans le QDK

Le Microsoft Quantum Development Kit (QDK) inclut un ensemble de simulateurs quantiques qui modélisent la façon dont votre programme s'exécute sur un ordinateur quantique. Les programmes que vous exécutez sur un ordinateur quantique incluent toujours un certain type et un certain degré de bruit. Le QDKPython package vous permet de créer des modèles de bruit personnalisés à utiliser dans vos simulations via l'API `NoiseConfig` .

Pour obtenir des instructions sur l'installation et l'utilisation QDK des simulateurs, consultez [Comment installer et exécuter les QDK simulateurs quantiques](#).

## Types de bruit

Chaque opération ou instruction dans un programme quantique peut introduire du bruit. Le tableau suivant répertorie toutes les opérations et instructions pour lesquelles vous pouvez définir le bruit.

 Agrandir le tableau

Source de bruit	Paramètre de modèle de bruit	Description de la source
porte $X$	<code>x</code>	Porte de Pauli sur un seul qubit, inversion de bit
porte $Y$	<code>y</code>	Porte de Pauli à un qubit, inversion de bit et inversion de phase
porte $Z$	<code>z</code>	Porte de Pauli à un seul qubit, inversion de phase
porte $H$	<code>h</code>	Porte Hadamard à qubit unique, crée un état de superposition égal
porte $S$	<code>s</code>	Porte à qubit unique, inversion de phase de demi- $\pi$
porte $S^\dagger$	<code>s_adj</code>	Porte à qubit unique, adjoint de $S$
porte $T$	<code>t</code>	Porte à qubit unique, inversion de phase de quart de $\pi$
porte $T^\dagger$	<code>t_adj</code>	Porte à qubit unique, adjoint de $T$
porte $S_X$	<code>sx</code>	Porte à un qubit, demi-renversement de bit



Source de bruit	Paramètre de modèle de bruit	Description de la source
porte $S_X^\dagger$	<code>sx_adj</code>	Porte à qubit unique, adjointe de $S_X$
porte $R_X$	<code>rx</code>	Porte à qubit unique, rotation de phase générale sur l'axe $x$
porte $R_Y$	<code>ry</code>	Porte à qubit unique, rotation de phase générale sur l'axe $y$
porte $R_Z$	<code>rz</code>	Porte à qubit unique, rotation de phase générale sur l'axe $z$
porte $CX$	<code>cx</code>	Porte à deux qubits, porte $X$ contrôlée
porte $CY$	<code>cy</code>	Porte à deux qubits, porte $Y$ contrôlée
porte $CZ$	<code>cz</code>	Porte à deux qubits, porte $Z$ contrôlée
porte $R_{XX}$	<code>rxx</code>	Porte à deux qubits, analogue à $R_X$
porte $R_{YY}$	<code>ryy</code>	Porte à deux qubits, analogue à $R_Y$
porte $R_{ZZ}$	<code>rzz</code>	Porte à deux qubits, analogue à $R_Z$
porte $SWAP$	<code>swap</code>	Porte à deux qubits, échange les états des qubits
porte $CCX$	<code>ccx</code>	Porte à trois qubits, porte contrôlée à deux qubits $X$
Le mouvement du qubit	<code>mov</code>	Déplacement de qubits entre les zones du dispositif (pour la simulation d'un dispositif à atomes neutres)
Mesure de qubit	<code>mz</code>	Mesure à qubit unique dans la base pauli- $Z$
Mesure et réinitialisation du qubit	<code>mresetz</code>	Mesure à qubit unique et réinitialisation à 0 état

Avec `NoiseConfig`, vous pouvez appliquer quatre types de bruit différents aux opérations précédentes avec des probabilités spécifiques. Le tableau suivant répertorie les paramètres de type de bruit que vous pouvez définir avec `NoiseConfig`, y compris un paramètre pour aucun bruit.

[🔍 Agrandir le tableau](#)

Type de bruit	Paramètre de modèle de bruit	Description du bruit	Exemple d'utilisation	Exemple de description
Bruit quantique de Pauli $X$	<code>x</code>	Retournement de bit	<code>noise.z.x = 0.03</code>	Une inversion de bit se produit dans 3 % des opérations $Z$
Bruit de Pauli $Y$	<code>y</code>	Retournement de bit et retournement de phase	<code>noise.sx.y = 0.01</code>	Le retournement de bit et le retournement de phase se produisent dans 1% des opérations $S_X$
Bruit de Pauli $Z$	<code>z</code>	Inversion de phase	<code>noise.h.z = 0.02</code>	Une inversion de phase se produit dans 2 % des opérations $H$
Aucun bruit	<code>i</code>	Opération d'identité, aucun effet	<code>noise.cz.ix = 0.02</code>	Le retournement de bit se produit uniquement sur le qubit cible dans 2 % des opérations $CZ$ .
Perte de qubit	<code>1</code>	Qubit est perdu de l'appareil	<code>noise.mov.1 = 0.03</code>	Le qubit est perdu dans 3 % des déplacements entre les zones de l'appareil sur un ordinateur quantique à atomes neutres

Le bruit se produit après l'opération source, et non à la place de l'opération source. Par exemple, `noise.z.x` signifie que le programme applique la porte  $Z$  prévue au qubit, puis applique une porte  $X$  involontaire au qubit. Étant donné que le bruit s'applique après la source, vous pouvez configurer le bruit qui a le même effet que la source. Par exemple, `noise.x.x` applique une inversion de bit involontaire après l'inversion de bit prévue.

### ⓘ Remarque

Les API de simulation d'appareil atom neutre prennent en charge le bruit provenant d'un nombre limité de sources. Pour plus d'informations, consultez [Comment élaborer des modèles de bruit pour les simulations de dispositifs à atomes neutres dans le QDK](#).

## Créer un modèle de bruit

Pour créer un modèle de bruit pour une simulation et afficher les effets de ce bruit sur les résultats de votre programme quantique, procédez comme suit.

1. Dans VS Code, ouvrez le menu **Affichage** et choisissez **Palette de commandes**.

- Entrez **Créer : Nouveau Jupyter Notebook**. Un fichier vide Jupyter Notebook s'ouvre dans un nouvel onglet.
- Dans la première cellule du notebook, importez les objets requis Python .

#### Python

```
from qdk import init, TargetProfile
from qdk.openqasm import compile
from qdk.simulation import NoiseConfig, run_qir
from qdk.widgets import Histogram
```

- Dans une nouvelle cellule, définissez le profil cible QIR de l'appareil et compilez votre circuit OpenQASM en QIR.

#### Python

```
init(target_profile=TargetProfile.Base)

qasm_src = """
include "stdgates.inc";
qubit[2] qs;
bit[2] r;

h qs[0];
cx qs[0], qs[1];
r = measure qs;
"""

qir = compile(qasm_src)
```

- Créez un `NoiseConfig` objet et générez votre modèle de bruit.

#### Python

```
noise = NoiseConfig()

noise.h.x = 0.01
noise.cx.zi = 0.02
```

Ce code produit le modèle de bruit suivant, où le taux de bruit est la probabilité que la source provoque le type de bruit correspondant.

Source de bruit	Type de bruit	Taux de bruit
porte $H$	Retournement de bit	1 %
porte $CX$	Inversion de phase sur le qubit de contrôle	2 %

6. Exécutez le simulateur avec le modèle de bruit et affichez un histogramme des résultats de mesure. Par exemple, exécutez le code suivant pour simuler 1 000 captures de votre programme sur le simulateur Clifford.

#### Python

```
results = run_qir(qir, shots=1000, noise=noise, type="clifford")
Histogram(results, labels="kets")
```

7. Pour comparer les résultats bruyants à une simulation sans bruit, réexécutez la simulation sans modèle de bruit.

#### Python

```
results = run_qir(qir, shots=1000, type="clifford")
Histogram(results, labels="kets")
```

## Définir plusieurs types de bruit sur la même source

Vous pouvez modéliser différents types de bruit sur la même source, avec des probabilités différentes pour chaque type de bruit. Par exemple, le code suivant définit une probabilité de 1 % qu'un retournement de bit se produise et une probabilité de 3 % qu'un retournement de phase se produise après une porte de Hadamard.

#### Python

```
noise.h.x = 0.01
noise.h.z = 0.03
```

Lorsque vous configurez plusieurs types de bruit pour la même opération, un seul type de bruit peut s'appliquer à chaque opération de votre programme. Par exemple, chaque porte Hadamard peut avoir un bruit  $X$  ou un bruit  $Z$ , mais pas les deux.

## Fonctions de modèle de bruit

Au lieu des paramètres de modèle de bruit, vous pouvez utiliser l'ensemble suivant de fonctions de bruit pour créer votre modèle de bruit.

## Configurer le bruit de Pauli

Pour inclure le bruit Pauli dans votre modèle, appelez la fonction `set_pauli_noise` sur une porte ou une opération de mouvement.

Pour les opérations à qubit unique, passez une chaîne Pauli d'un caractère et une fréquence de bruit. Par exemple, le code suivant définit une probabilité de 1 % qu'une inversion de bit se produise lors du déplacement du qubit.

### Python

```
Equivalent to: noise.mov.x = 0.01
noise.mov.set_pauli_noise('X', 0.01)
```

Pour les opérations à deux qubits, spécifiez une chaîne de Pauli de deux caractères et un taux de bruit. Le premier caractère de la chaîne Pauli correspond au bruit sur le qubit de contrôle et le deuxième caractère correspond au bruit sur le qubit cible. Par exemple, le code suivant définit des retournements de phase corrélés après 1% d'opérations  $CX$ .

### Python

```
Equivalent to: noise.cx.zz = 0.01
noise.cx.set_pauli_noise('ZZ', 0.01)
```

## Définir le bruit dépolarisant

La `set_depolarizing` fonction définit des taux de bruit égaux mais non corrélés pour les trois types de bruit Pauli. Par exemple, le code suivant définit une probabilité de 3 % que du bruit de Pauli se produise après une opération  $H$ , répartie uniformément en une probabilité de 1 % pour chacun des trois types de bruit de Pauli.

### Python

```
Equivalent to:
noise.h.x = 0.01
noise.h.y = 0.01
noise.h.z = 0.01
noise.h.set_depolarizing(0.03)
```

# Définir le bruit de retournement du bit

Pour définir le taux de bruit pour les retournements de bits, utilisez la fonction `set_bitflip` sur une porte ou une opération de mouvement. Par exemple, le code suivant définit une chance de 1% qu'un retournement de phase se produit après une opération  $R_Z$ .

Python

```
Equivalent to: noise.rz.x = 0.01
noise.rz.set_bitflip(0.01)
```

## Paramétrer le bruit de retournement de phase

Pour définir le taux de bruit pour les retournements de phase dans une opération, utilisez la `set_phaseflip` fonction sur une porte ou une opération de déplacement. Par exemple, le code suivant définit une chance de 1% qu'un retournement de phase se produit après une opération  $R_Y$ .

Python

```
Equivalent to: noise.ry.z = 0.01
noise.ry.set_phaseflip(0.01)
```

## Contenu connexe

- [Créer des modèles de bruit pour les portes multi-qubit](#)
- [Comment construire des modèles de bruit pour les simulations de dispositifs à atomes neutres dans le QDK](#)

# Créer des modèles de bruit pour les portes multi-qubit

Le package Microsoft Quantum Development Kit (QDK) Python prend en charge des modèles de bruit flexibles pour les portes à plusieurs qubits dans les simulations de programmes quantiques. Vous pouvez modéliser des erreurs corrélées et d'autres effets sonores sur plusieurs qubits pour représenter plus précisément le comportement du matériel quantique.

Pour plus d'informations sur les modèles de bruit dans le QDK, consultez [Comment créer des modèles de bruit pour les simulations quantiques dans le QDK](#).

## Prerequisites

Pour générer des modèles de bruit dans le QDK, installez les outils suivants.

- Visual Studio Code (VS Code) avec [l'extension QDK](#) et [l'extension Jupyter](#) installées.
- La dernière version du package `qdk` Python avec l'extra `jupyter`.

Bash

```
pip install --upgrade "qdk[jupyter]"
```

## Bruit corrélé

Les portes multi-qubits peuvent produire un bruit corrélé, où le même modèle de bruit s'applique à tous les qubits sur lesquels la porte fonctionne. Pour définir le bruit corrélé sur les portes multi-qubits, spécifiez un paramètre de bruit pour chaque qubit.

Par exemple, le code Python suivant définit des inversions de bits corrélées sur les portes  $CX$  avec une probabilité de 2 %. Lorsque le bruit se produit sur une porte  $CX$ , une porte  $X$  s'applique à la fois au qubit de contrôle et au qubit cible. Le bruit est corrélé car le bruit s'applique toujours aux deux qubits.

Python

```
from qdk.simulation import NoiseConfig

noise = NoiseConfig()
```

```
noise.cx.xx = 0.02
```

Pour rendre le bruit non corrélé, définissez plusieurs types de bruit avec le paramètre d'identité.

Python

```
noise.cx.xi = 0.02 # Bit flip on control qubit, do nothing to target qubit
noise.cx.ix = 0.02 # Do nothing to control qubit, bit flip on target qubit
```

Dans le modèle sans corrélation, chaque paramètre de bruit se produit indépendamment avec 2% probabilité. Étant donné qu'une seule configuration de bruit peut s'appliquer à une porte donnée, ce modèle de bruit ne peut pas appliquer un bruit  $X$  aux deux qubits d'une même porte.

Pour modéliser la possibilité de bruit sur les deux qubits, configurez un autre paramètre de bruit qui applique le bruit aux deux qubits.

Python

```
noise.cx.xi = 0.02 # Bit flip on control qubit, do nothing to target qubit
noise.cx.ix = 0.02 # Do nothing to control qubit, bit flip on target qubit
noise.cx.xx = 0.02 # Bit flip on both qubits
```

# Politiques de perte de qubits

La perte de qubit est un type de bruit où les qubits sont perdus de l'appareil. Les politiques de perte définissent le comportement d'une porte multi-qubit lorsqu'un ou plusieurs qubits sont absents au début de l'opération. Le QDK prend en charge les stratégies de perte suivantes.

[🔗 Agrandir le tableau](#)

Politique de perte	Portes auxquelles la stratégie peut s'appliquer	Effet sur les qubits restants
SKIP	Toutes les portes	La porte n'a aucun effet sur les qubits restants
PROPAGATE	Toutes les portes	Les autres qubits sont également perdus
DEGRADE	<code>rxx</code> , <code>ryy</code> et <code>rzz</code>	Appliquer comme porte à un qubit au qubit restant
RESIDUAL_S_DAGGER	Toutes les portes	Appliquer la porte <code>s_sdj</code> aux qubits restants
APPLY_ANYWAY	<code>swap</code>	Le qubit restant continue d'échanger son état avec le qubit perdu



Pour que les stratégies de perte affectent votre simulation, vous devez inclure la perte de qubits dans votre modèle de bruit. Les qubits perdus sont suivis pendant la simulation. Les stratégies de perte s'appliquent lorsqu'une porte agit sur au moins un qubit perdu.

Pour inclure des politiques de perte dans votre modèle de bruit, utilisez `LossPolicy` du module `qdk.simulation`. Le code suivant montre des exemples pour définir chaque stratégie.

#### Python

```
from qdk.simulation import NoiseConfig, LossPolicy

noise = NoiseConfig()

noise.x.l = 0.01 # 1% chance of qubit loss after X gates

noise.cz.on_loss = LossPolicy.SKIP # Don't apply CZ to remaining qubit
noise.cx.on_loss = LossPolicy.PROPAGATE # Both qubits are lost from CX gate
noise.rxx.on_loss = LossPolicy.DEGRADE # Apply Rx gate to remaining qubit
noise.ryy.on_loss = LossPolicy.RESIDUAL_S_DAGGER # Apply S-adjoint to remaining
qubit
noise.swap.on_loss = LossPolicy.APPLY_ANYWAY # Perform swap anyway
```

Pour toutes les stratégies de perte, le bruit configuré peut encore affecter les qubits restants. Par exemple, le code suivant peut toujours appliquer un bruit d'inversion de bit aux portes CZ même lorsqu'un des qubits d'une porte CZ est manquant.

#### Python

```
noise.z.l = 0.01 # Introduce qubit loss

noise.cz.xx = 0.02 # Set correlated bit flip noise on CZ gates
noise.cz.on_loss = LossPolicy.SKIP
```

## Intrinsèques de bruit personnalisées

Pour créer des modèles de bruit plus complexes, le QDK dispose d'instructions intrinsèques de bruit personnalisées pour les programmes Q# et OpenQASM. Les intrinsèques de bruit se comportent comme des portes personnalisées que vous insérez dans votre programme pour modéliser le bruit corrélé. Vous pouvez utiliser des fonctions intrinsèques personnalisées pour modéliser le bruit sur les portes prises en charge par `NoiseConfig` ou sur des portes personnalisées.

Les exemples suivants montrent comment construire une fonction intrinsèque de bruit personnalisée qui modélise la diaphonie entre trois qubits. L'intrinsèque de bruit applique des

retournements de bits corrélés à deux des qubits après l'application d'une porte *CNOT*.

## Ajouter des fonctions intrinsèques de bruit à un programme Q#

Dans les programmes Q#, utilisez `@NoiseIntrinsic()` pour déclarer une primitive intrinsèque de bruit. Ensuite, utilisez la méthode `intrinsic` de `NoiseConfig` pour configurer l'intrinsèque de bruit.

Pour configurer et utiliser l'exemple d'intrinsèque de bruit, suivez ces étapes dans un Jupyter Notebook.

1. Importez les objets requis et définissez le profil cible QIR.

### Python

```
from qdk import init, TargetProfile
from qdk import qsharp
from qdk.simulation import run_qir, NoiseConfig

init(target_profile=TargetProfile.Adaptive_RIF)
```

2. Écrivez un programme Q# appelé `GHZ` qui appelle une intrinsèque de bruit appelée `Crosstalk3Q` après chaque porte *CNOT*.

### Q#

```
%%qsharp

// A noise intrinsic representing crosstalk on 3 qubits.
// In the ideal circuit this is a no-op; the simulator injects
// Pauli errors according to the NoiseConfig.
@NoiseIntrinsic()
operation Crosstalk3Q(q0: Qubit, q1: Qubit, q2: Qubit) : Unit {
 body intrinsic;
}

// Prepare a GHZ state on 3 qubits, with crosstalk after each CNOT.
operation GHZ() : Result[] {
 use qs = Qubit[3];
 H(qs[0]);
 CNOT(qs[0], qs[1]);
 Crosstalk3Q(qs[0], qs[1], qs[2]); // crosstalk hits all 3 qubits
 CNOT(qs[1], qs[2]);
 Crosstalk3Q(qs[0], qs[1], qs[2]); // crosstalk again
 MResetEachZ(qs)
}
```

3. Configurez la table de bruit pour le paramètre intrinsèque. Définissez le nombre de qubits, les types de bruit et la probabilité de chaque type de bruit.

#### Python

```
noise = NoiseConfig()
table = noise.intrinsic("Crosstalk3Q", num_qubits=3)
table.ixx = 0.10 # 10% XX on qubits 1-2
table.xxi = 0.05 # 5% XX on qubits 0-1
```

4. Compilez le programme en QIR.

#### Python

```
qir = qsharp.compile("GHZ()")
```

5. Exécutez la simulation et le tracé de l'histogramme des résultats.

#### Python

```
result = run_qir(qir, shots=1000, noise=noise)
Histogram(result)
```

6. Pour comparer le résultat à une simulation sans bruit, réexécutez la simulation sans modèle de bruit.

#### Python

```
result = run_qir(qir, shots=1000)
Histogram(result)
```

## Ajouter des intrinsèques de bruit à un programme OpenQASM

Dans les programmes OpenQASM, utilisez `@qdk.qir.noise_intrinsic` pour créer un bruit intrinsèque en tant que définition de porte personnalisée. Ensuite, utilisez la méthode `intrinsic` de `NoiseConfig` pour configurer l'intrinsèque de bruit.

Pour écrire un programme OpenQASM avec un intrinsèque de bruit appelé `crosstalk_3q` et compiler le programme en QIR, exécutez le code suivant dans un notebook Jupyter.

#### Python

```
from qdk.openqasm import compile, OutputSemantics
from qdk import TargetProfile
```

```

from qdk.simulation import run_qir, NoiseConfig
from qdk.widgets import Histogram

qasm_source = """
OPENQASM 3.0;
include "stdgates.inc";

// A noise intrinsic representing crosstalk on 3 qubits.
// In the ideal circuit this is a no-op; the simulator injects
// Pauli errors according to the NoiseConfig.
@qdk.qir.noise_intrinsic
gate crosstalk_3q q0, q1, q2 {}

qubit[3] qs;

// Prepare a GHZ state on 3 qubits, with crosstalk after each CNOT.
h qs[0];
cx qs[0], qs[1];
crosstalk_3q qs[0], qs[1], qs[2]; // crosstalk hits all 3 qubits
cx qs[1], qs[2];
crosstalk_3q qs[0], qs[1], qs[2]; // crosstalk again

bit[3] res = measure qs;
"""

qir_qasm = compile(
 qasm_source,
 output_semantics=OutputSemantics.OpenQasm,
 target_profile=TargetProfile.Base,
)

```

Pour configurer l'intrinsèque du bruit et exécuter la simulation, exécutez le code suivant.

#### Python

```

noise = NoiseConfig()
table = noise.intrinsic("crosstalk_3q", num_qubits=3)
table.ixx = 0.10 # 10% XX on qubits 1-2
table.xxi = 0.05 # 5% XX on qubits 0-1

result = run_qir(qir_qasm, shots=1000, noise=noise)
Histogram(result)

```

# Explorer les diagrammes de circuits dans le QDK

Les algorithmes quantiques sont un mélange de programmation quantique et de programmation classique. La partie quantique comprend des qubits, des portes quantiques et des mesures probabilistes, tandis que la partie classique inclut des constructions de flux de contrôle telles que la logique conditionnelle et les boucles. Les diagrammes de circuits quantiques doivent incorporer des éléments de la programmation quantique et classique.

Le Microsoft Quantum Development Kit (QDK) crée des diagrammes de circuit interactifs pour les programmes Q# et OpenQASM qui montrent comment les groupes de contrôle classiques affectent la structure de votre circuit.

Pour plus d'informations sur la création de diagrammes de circuits avec l'élément QDK in VS Code, voir [Comment visualiser des diagrammes de circuit quantiques avec le QDK](#)

## Groupes de contrôles classiques dans les diagrammes de circuits quantiques

Dans certains algorithmes quantiques, le circuit change en fonction du résultat d'une mesure de circuit intermédiaire. Par exemple, le circuit applique une porte à un qubit spécifique uniquement si le résultat de mesure d'un autre qubit est 1. Ou, le circuit itère sur une boucle jusqu'à ce que le résultat de mesure d'un qubit spécifique soit 0.

Par exemple, le programme Q# à deux qubits suivant place un qubit dans un état de superposition, mesure le qubit, puis applique une autre porte à l'autre qubit en fonction du résultat de mesure du premier qubit.

Q#

```
import Std.Measurement.*;

operation Main() : Result[] {
 use q0 = Qubit();
 use q1 = Qubit();

 H(q0);

 let r = M(q0);

 if r == One {
 X(q1);
 } else {
 Y(q1);
 }
}
```

```

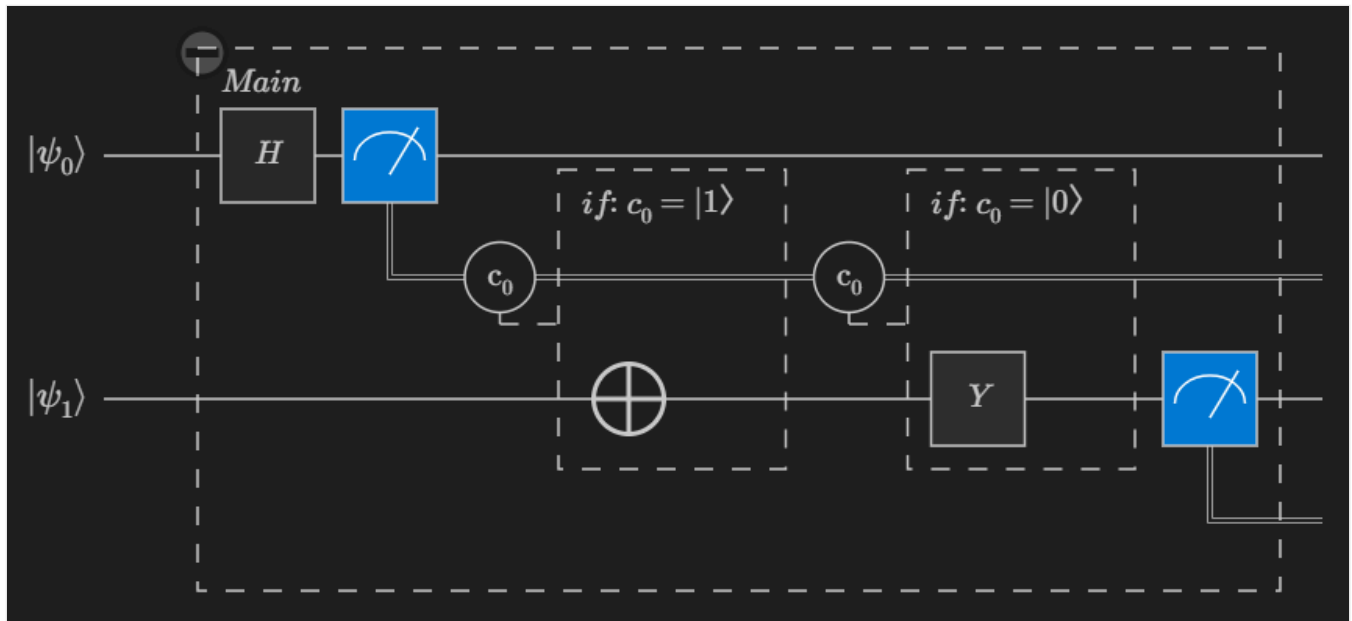
}

let r1 = M(q1);

[r, r1]
}

```

Le diagramme de circuit de ce programme ressemble à ceci :



### 💡 Conseil

Sélectionnez un élément dans le diagramme de circuit pour mettre en surbrillance le code qui crée l'élément de circuit.

## Types de blocs de diagramme de circuit

QDK les diagrammes de circuit ont deux types de blocs, de blocs d'opérations quantiques et de blocs de groupe de contrôle classiques. Certains blocs sont repliables. Pour réduire ou développer un bloc, sélectionnez l'icône +/- en haut à gauche du bloc.



## Blocs d'opérations quantiques

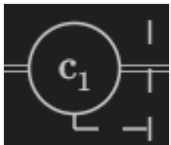
Chaque objet de votre programme qui applique des portes quantiques ou des mesures à qubits a son propre bloc dans le diagramme de circuit. Pour les programmes Q#, ces objets incluent des opérations, des foncteurs et des lambdas. Chaque bloc d'opération affiche le nom de l'objet Q# ou OpenQASM qui correspond à ce bloc.

## Blocs de flux de contrôle classique

Chaque instruction conditionnelle et boucle dans votre programme a son propre bloc dans le diagramme de circuit.

### Blocs conditionnels

Les blocs conditionnels sont précédés d'une icône de condition  $C_n$ , où chaque instruction conditionnelle du programme est identifiée par un numéro d'indice unique.



Chaque bloc conditionnel est étiqueté avec la condition qui, le cas échéant, entraîne l'exécution de cette partie du circuit pendant le programme.

### Blocs de boucles

Les blocs de boucles sont étiquetés avec **une boucle : 1.. N**, où **N** correspond au nombre d'itérations de la boucle. Pour afficher le circuit pour chaque itération de la boucle, sélectionnez l'icône d'expansion dans le bloc de boucle.

## Contenu connexe

- [Comment visualiser des diagrammes de circuits quantiques avec le QDK](#)
- [Conventions de diagramme de circuit quantique](#)

# Créer et visualiser des circuits avec le circuit editor

circuit editor est une fonctionnalité du Kit de développement Quantum (QDK) qui fournit une interface graphique dans laquelle vous pouvez créer, modifier et visualiser des diagrammes de circuits quantiques à l'intérieur de vos projets Q#. Vous pouvez utiliser les circuits que vous générez directement dans vos programmes Q# en tant qu'opérations appelables.

## Comment accéder à l'application circuit editor

Pour commencer à utiliser , circuit editor procédez comme suit :

- 1. Créez un Q# projet dans VS Code ou ouvrez un projet existant.
- 2. Créez un fichier de circuit avec l'extension `.qsc`.
- 3. Ouvrez le fichier de circuit dans VS Code.

Le circuit editor est la la vue par défaut pour les fichiers `.qsc`.

## Fonctionnalités circuit editor

Voici ce que vous pouvez faire avec le circuit editor :

 [Agrandir le tableau](#)

Fonctionnalité	Guide pratique pour utiliser
Agrandir le panneau Visualiseur d'état	Cliquez sur <b>Visualisation d'état</b> pour développer le panneau Visualiseur d'état pour afficher l'état quantique résultant de l'exécution du circuit. Le panneau affiche la densité et la phase de probabilité pour chaque état de base. Vous pouvez utiliser la boîte à outils pour modifier le circuit et afficher les mises à jour du panneau en direct.
Insérer un élément de circuit	Cliquez et faites glisser l'élément de la boîte à outils dans le diagramme du circuit.
Supprimer un élément de circuit	Cliquez et faites glisser l'élément hors du diagramme de circuit. Vous pouvez également cliquer avec le bouton droit sur l'élément et choisir <b>Supprimer</b> dans le menu contextuel.
Déplacer un élément de circuit	Cliquez et faites glisser l'élément vers un nouvel emplacement dans le diagramme du circuit.
Copier un élément de circuit	Maintenez la <b>touche Ctrl</b> enfoncée pendant que vous cliquez et faites glisser l'élément vers un nouvel emplacement dans le diagramme du circuit.



Fonctionnalité	Guide pratique pour utiliser
Ajouter un qubit	Placez un élément de la boîte à outils sur un nouveau câble dans le diagramme de circuit. Ou, déplacez un élément existant vers un nouveau câble dans le diagramme.
Supprimer un qubit	Cliquez et faites glisser l'icône qubit hors du diagramme de circuit. Le qubit le plus bas est automatiquement supprimé lorsque vous supprimez tous les éléments de circuit de ce qubit.
Réorganiser l'ordre du qubit	Cliquez et faites glisser les icônes qubit dans le diagramme de circuit.
Ajouter un contrôle à une porte	Cliquez avec le bouton droit sur l'icône de porte, choisissez <b>Ajouter un contrôle</b> dans le menu contextuel, puis choisissez le fil qubit de contrôle.
Supprimer un contrôle d'une porte	Cliquez avec le bouton droit sur l'icône de porte, choisissez <b>Supprimer le contrôle</b> dans le menu contextuel, puis sélectionnez l'icône du contrôle que vous souhaitez supprimer. Vous pouvez également cliquer avec le bouton droit sur l'icône de contrôle directement et choisir <b>Supprimer le contrôle</b> , ou faire glisser l'icône de contrôle hors du diagramme.
Convertir une porte en son adjoint	Cliquez avec le bouton droit sur l'icône en forme porte et choisissez <b>Activer/Désactiver adjoint</b> dans le menu contextuel.
Définir un argument pour une porte	Lorsque vous placez une porte qui nécessite un argument, une zone d'invite s'affiche. Saisissez un nombre ou une expression dans la zone d'invite. Pour mettre à jour l'argument, cliquez avec le bouton droit sur la porte et choisissez <b>Modifier l'argument</b> . Vous pouvez également cliquer sur le texte de l'argument sur l'icône en forme de porte.

## Comment utiliser des circuits circuit editor dans vos projets Q#

Les fichiers de circuit avec l'extension `.qsc` définissent les opérations que vous pouvez référencer à partir du Q# code dans le même Q# projet. Les circuits des fichiers `.qsc` apparaissent comme n'importe quelle autre opération Q# et sont pris en charge par le service de langage Q# avec des fonctionnalités telles que la saisie semi-automatique, l'aide à la signature et l'accès aux définitions.

Le fichier circuit définit un espace de noms qui provient du nom de fichier, similaire au fonctionnement des fichiers Q#. Le circuit de votre `.qsc` fichier définit une Q# opération portant le même nom que votre fichier sous ce même espace de noms. Par exemple, pour référencer un circuit à partir du fichier `Foo.qsc` dans votre Q# code, utilisez l'instruction import suivante : `import Foo.Foo;`

Le programme suivant Q# importe un circuit à partir du fichier `JointMeasurement.qsc`, puis appelle l'opération `JointMeasurement()` pour appliquer le circuit à un système de qubits.

Q#

```
import JointMeasurement.JointMeasurement;

operation Main() : Result[] {
 use qs = Qubit[4];
 ApplyToEach(H, qs[0..2]);
 let results = JointMeasurement(qs);
 ResetAll(qs);
 results
}
```

## Contenu connexe

- [Visualiser des diagrammes de circuits quantiques avec Q#](#)
- [Conventions de diagramme de circuit quantique](#)

---

Last updated on 12/03/2026

# Simuler des programmes quantiques sur le matériel d'un dispositif à atomes neutres

Le Microsoft Quantum Development Kit (QDK) fournit un ensemble d'outils de simulation qui vous permettent d'évaluer et d'itérer sur vos programmes quantiques avant de les exécuter sur du matériel quantique réel. Les API de simulation d'appareil atomique neutre modélisent les types de traitement du bruit et des qubits qui se produisent lorsque les programmes s'exécutent sur des ordinateurs quantiques atom neutres, tels que la perte de qubit et le mouvement qubit. Si vous envisagez d'exécuter vos programmes quantiques sur du matériel atom neutre, utilisez les API de simulation d'appareil atom neutre pour tester et affiner votre code.

## Fonctionnement des ordinateurs quantiques atom neutres

Les appareils atom neutres sont l'une des nombreuses technologies actuelles du matériel informatique quantique. D'autres technologies incluent des qubits superconducteurs, des qubits d'ion piégés et des qubits topologiques. Chaque technologie a ses propres atouts et inconvénients. Les ordinateurs quantiques à atomes neutres offrent une bonne évolutivité, une connectivité flexible entre les qubits et de longs temps de cohérence, mais présentent toutefois des difficultés en matière de fidélité des portes et de complexité de la commande des lasers.

La technologie de qubit exacte utilisée dans un dispositif à atomes neutres dépend de l'architecture spécifique. Mais en général, chaque qubit est un seul atome neutre électriquement. Les états qubit 0 et 1 correspondent à différents états d'énergie des atomes. Les atomes sont organisés dans des tableaux 2D ou 3D sur l'appareil. Les lasers piègent les atomes en place et les déplacent entre différentes zones dans le dispositif pour le stockage, la mesure et les interactions qui réalisent des opérations logiques quantiques.

## Outils de simulation de dispositifs à atomes neutres dans le QDK

La simulation de dispositif à atomes neutres dans le QDK modélise les propriétés suivantes du matériel quantique à atomes neutres :

- Les appareils contiennent uniquement des portes  $S_X$ ,  $R_Z$  et  $CZ$ .

- Les qubits se déplacent physiquement entre les zones de stockage, d'interaction et de mesure sur l'appareil.

La bibliothèque QDKPython a deux API pour la simulation d'appareil atom neutre, selon le format de votre programme :

[Agrandir le tableau](#)

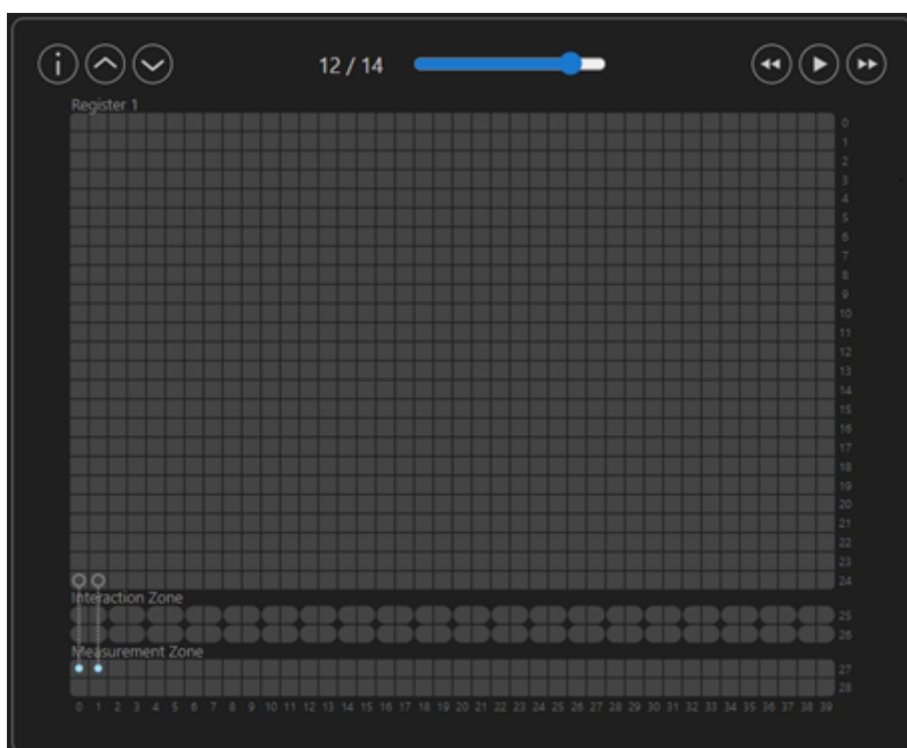
API	QDK Module	Format d'entrée	Simulateurs pris en charge
<code>NeutralAtomDevice</code>	<code>qdk.simulation</code>	QIR	Clifford, GPU et CPU
<code>NeutralAtomBackend</code>	<code>qdk.qiskit</code>	Qiskit	Clifford, GPU et CPU

Les deux API compilent votre programme en QIR, qui comporte des instructions de déplacement de qubits et contient uniquement les portes prises en charge par les dispositifs à atomes neutres.

Pour plus d'informations sur la simulation quantique dans le QDK, consultez [Vue d'ensemble des simulateurs quantiques dans le QDK](#).

## Visualiseur de dispositif à atomes neutres

Le `NeutralAtomDevice` API inclut un visualiseur de dispositif à atomes neutres pour Jupyter Notebook grâce à la méthode `show_trace`. Le visualiseur crée un diagramme interactif qui montre comment les qubits passent par un appareil atom neutre de base à mesure que votre programme s'exécute. Le visualiseur n'inclut pas les effets de la perte de qubits ni d'autres sources de bruit.



Pour plus d'informations sur le visualiseur de l'appareil à atomes neutres, consultez [Comment utiliser le visualiseur de l'appareil à atomes neutres](#).

## Modèles de bruit pour les simulations de dispositifs à atomes neutres

Les simulateurs quantiques de la bibliothèque QDKPython utilisent la classe `NoiseConfig` du module `qdk.simulation` pour ajouter des modèles de bruit aux simulations. Lorsque vous effectuez des simulations avec les API `NeutralAtomDevice` ou `NeutralAtomBackend`, vos modèles de bruit ne prennent en compte que le bruit des portes du dispositif à atomes neutres, le déplacement des qubits et la mesure.

Pour plus d'informations sur les modèles de bruit pour les dispositifs à atomes neutres dans le QDK, consultez [Comment créer des modèles de bruit pour les simulations de dispositifs à atomes neutres dans le QDK](#).

## Prise en main de la simulation d'appareil à atomes neutres

Pour commencer à utiliser la simulation quantique dans le QDK, consultez [Comment installer et exécuter les QDK simulateurs quantiques](#).

Pour savoir comment utiliser le visualiseur de périphérique à atomes neutres, consultez [Comment utiliser le visualiseur de périphérique à atomes neutres](#).

---

Last updated on 30/06/2026

# Comment utiliser le visuel de l'appareil à atomes neutres

Le Microsoft Quantum Development Kit (QDK) offre plusieurs simulateurs quantiques et un visualiseur pour les ordinateurs quantiques atomiques neutres. Le visualiseur d'appareil atom neutre produit un diagramme interactif qui vous permet de suivre le déplacement et le traitement des qubits lorsque votre programme s'exécute sur un appareil atom neutre de base. Cet article explique comment accéder au visualiseur et l'utiliser dans Jupyter Notebook.

Pour obtenir des instructions sur l'installation des simulateurs et du visualiseur, consultez [Comment installer et exécuter les QDK simulateurs quantiques](#).

## Comment créer un diagramme de qubit atom neutre

Pour créer un diagramme de qubit avec le visualiseur d'atomes neutres, procédez comme suit :

1. Dans VS Code, ouvrez le menu **Affichage** et choisissez **Palette de commandes**.
2. Entrez et sélectionnez **Créer : Nouveau Jupyter Notebook**. Un nouvel onglet s'ouvre avec un fichier vide Jupyter Notebook .
3. Importez les packages et objets nécessaires. Dans la première cellule, entrez et exécutez le code suivant :

Python

```
from qdk.openqasm import compile
from qdk.simulation import NeutralAtomDevice
```

4. Écrivez votre circuit OpenQASM et compilez le circuit dans QIR. Par exemple, le circuit suivant entangle deux qubits :

Python

```
qasm_src = """
include "stdgates.inc";
qubit[2] qs;
bit[2] r;

h qs[0];
cx qs[0], qs[1];
```

```
r = measure qs;
"""

qir = compile(qasm_src)
```

5. Créez un objet de simulateur à l'aide de `NeutralAtomDevice`.

Python

```
simulator = NeutralAtomDevice()
```

6. Pour accéder au visualiseur, appelez la `show_trace` méthode et transmettez le QIR pour votre programme.

Python

```
simulator.show_trace(qir)
```

Le visualiseur s'affiche dans la cellule de sortie.

#### ⚠ Remarque

Le visualiseur n'inclut pas la perte de qubits ni d'autres sources de bruit.

## Comment interagir avec le visualiseur

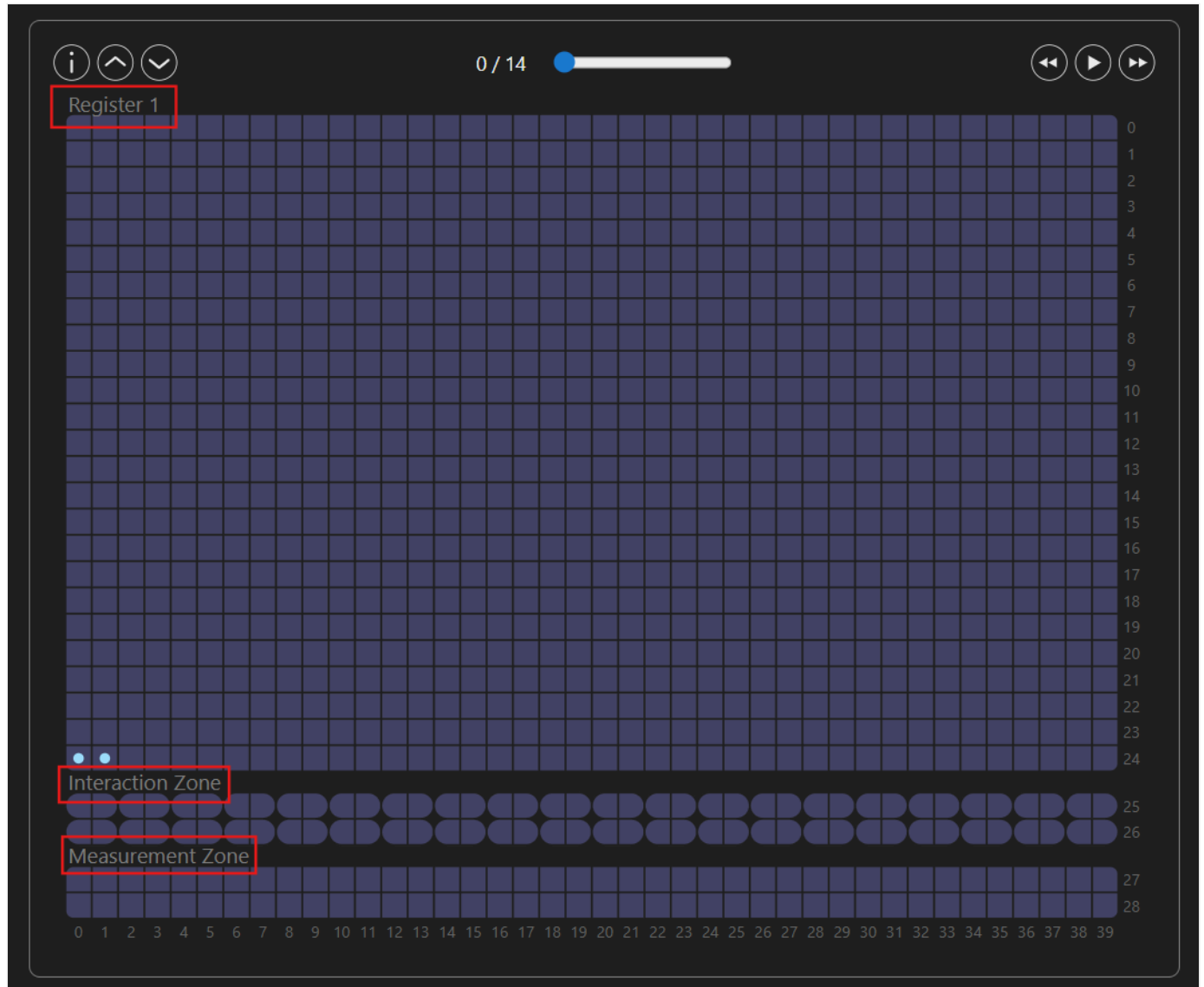
Le visualiseur a des éléments interactifs qui vous permettent d'explorer une simulation de la façon dont les qubits se comportent comme votre programme s'exécute sur un ordinateur quantique atom neutre de base. Le diagramme d'appareil est une grille 2D avec des lignes et des colonnes étiquetées. Chaque point à une position de la grille représente un qubit à atome neutre du dispositif.

Le diagramme contient trois zones :

[🔍 Agrandir le tableau](#)

Zone	Descriptif
Zone de stockage	Cette zone est étiquetée <b>Registre 1</b> . Les qubits commencent dans la zone de stockage et restent là jusqu'à ce qu'ils soient prêts pour le traitement ou la mesure. Les qubits reviennent toujours à la zone de stockage après les opérations et les mesures.

Zone	Descriptif
Zone d'interaction	Cette zone est l'endroit où les portes quantiques sont appliquées aux qubits pour le traitement.
Zone de mesure	Cette zone est l'endroit où les qubits sont mesurés.



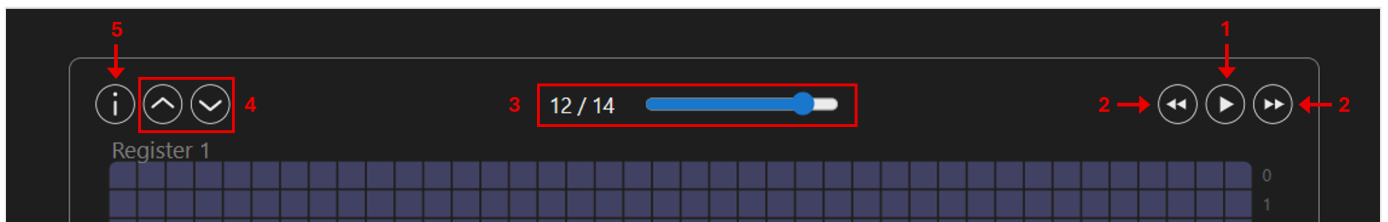
## Éléments interactifs dans le visualiseur

Pour interagir avec le visualiseur, utilisez les éléments en haut du diagramme. Le diagramme contient les éléments suivants :

1. **Bouton Lecture** : Sélectionnez ce bouton pour lancer une animation montrant l'exécution de votre programme. L'animation passe par chaque étape du programme jusqu'à ce que le programme se termine. Pendant l'animation, sélectionnez à nouveau ce bouton pour suspendre l'animation à l'étape actuelle. Lorsque l'animation se termine, sélectionnez à nouveau ce bouton pour démarrer l'animation à partir du début.



2. **Boutons avant et arrière** : Sélectionnez ces boutons pour parcourir le programme une étape à la fois sans lire l'animation complète.
3. **Curseur de progression** : Cet élément montre l'étape actuelle du programme. Déplacez le curseur pour parcourir le programme et choisissez une étape spécifique. À chaque étape, pointez sur un qubit pour voir où le qubit a été déplacé à partir de l'étape précédente.
4. **Redimensionner les boutons** : Sélectionnez le bouton flèche vers le haut pour augmenter la taille du diagramme, puis sélectionnez le bouton flèche bas pour réduire la taille du diagramme.
5.  **Icône d'informations** : Pointez sur cette icône pour afficher une liste de raccourcis clavier qui vous permettent d'interagir avec le diagramme. Le raccourci clavier **F** accélère l'animation et **S** ralentit l'animation lorsque vous sélectionnez le bouton **Lecture** .



# Comment construire des modèles de bruit pour les simulations de dispositifs à atomes neutres dans le QDK

Le Microsoft Quantum Development Kit (QDK) inclut un ensemble d'API de simulation d'appareil atom neutre que vous pouvez utiliser pour modéliser la façon dont votre programme s'exécute sur un ordinateur quantique atom neutre. Ces API modélisent les types de bruit qui se produisent spécifiquement sur le matériel atomique neutre. La bibliothèque QDKPython vous permet d'ajouter du bruit à des modèles pour les simulations avec la classe `NoiseConfig`.

Pour plus d'informations sur les modèles de bruit dans QDK les simulations, consultez [Comment créer des modèles de bruit pour les simulations quantiques dans la QDKPython bibliothèque](#).

Pour obtenir des instructions sur l'installation et l'utilisation QDK des simulateurs, consultez [Comment installer et exécuter les QDK simulateurs quantiques](#).

## Les API de simulation de dispositif à atomes neutres

Le QDK inclut deux API de simulation de dispositifs à atomes neutres, `NeutralAtomDevice` et `NeutralAtomBackend`. Ces API modélisent le bruit qui se produit spécifiquement sur les ordinateurs quantiques atom neutres. Pour plus d'informations, consultez [Simuler des programmes quantiques sur du matériel d'appareil atom neutre](#).

## Types de bruit dans les simulateurs d'appareils atom neutres

Dans les appareils atomiques neutres, les lasers déplacent physiquement les qubits entre différentes zones de l'appareil. Le bruit peut se produire dans les dispositifs atomiques neutres dans les situations suivantes :

- Mouvements de qubits entre les zones
- Opérations de porte quantique sur des qubits dans la zone d'interaction
- Mesures de qubit dans la zone de détection

Les dispositifs à atomes neutres disposent d'un nombre limité de portes quantiques. Les simulations de dispositifs à atomes neutres dans le QDK prennent en charge le bruit provenant

des sources suivantes.

 Agrandir le tableau

Source de bruit	Paramètre de modèle de bruit	Description de la source
porte quantique	<code>sx</code>	Passe unitaire à qubit unique, demi-renversement de bit
porte quantique	<code>rz</code>	Porte à qubit unique, rotation de phase générale
porte quantique	<code>cz</code>	Porte à deux qubits, bascule de phase contrôlée-
Mesure et réinitialisation du qubit	<code>mresetz</code>	Mesure à qubit unique et réinitialisation à 0 état
Le mouvement du qubit	<code>mov</code>	Le mouvement du qubit entre les zones de l'appareil

Pour ces sources de bruit, vous pouvez configurer tous les types de bruit pris en charge par les QDK simulateurs.

Votre programme quantique peut contenir n'importe quel type de porte prise en charge par le profil cible QIR. Les API de simulation sur atomes neutres compilent le QIR en entrée et décomposent les portes de votre programme en un ensemble de portes disponibles sur les dispositifs à atomes neutres : `cx`, `cz` et `rz`. Lorsque vous utilisez `NeutralAtomDevice` et `NeutralAtomBackend` pour vos simulations, ne configurez pas le bruit sur d'autres portes, car ce bruit n'affecte pas la simulation. Par exemple, si votre programme contient des portes Hadamard et que vous configurez le bruit pour les portes Hadamard, ce bruit ne s'affiche pas dans la simulation, car les portes Hadamard sont décomposées en portes d'appareil atomique neutre.

Les API de simulation d'atomes neutres convertissent l'ensemble des opérations de mesure de votre programme en instructions de mesure suivie de réinitialisation. Pour inclure le bruit de mesure, configurez le bruit sur `mresetz`. Le bruit sur `mz` n'est pas inclus dans la simulation.

## Créer un modèle de bruit pour la simulation d'un dispositif à atomes neutres

Pour créer un modèle de bruit pour une simulation d'appareil atom neutre, procédez comme suit.

1. Dans VS Code, ouvrez le menu **Affichage** et choisissez **Palette de commandes**.

- Entrez **Créer : Nouveau Jupyter Notebook**. Un fichier vide Jupyter Notebook s'ouvre dans un nouvel onglet.
- Dans la première cellule du notebook, importez les objets requis Python .

#### Python

```
from qdk import init, TargetProfile
from qdk.openqasm import compile
from qdk.simulation import NeutralAtomDevice, NoiseConfig
from qdk.widgets import Histogram
```

- Dans une nouvelle cellule, définissez le profil cible QIR de l'appareil et compilez un programme OpenQASM en QIR.

#### Python

```
init(target_profile=TargetProfile.Base)

qasm_src = """
include "stdgates.inc";
qubit[2] qs;
bit[2] r;

h qs[0];
cx qs[0], qs[1];
r = measure qs;
"""

qir = compile(qasm_src)
```

- Créez un `NoiseConfig` objet et générez votre modèle de bruit. Par exemple, exécutez le code suivant dans une nouvelle cellule.

#### Python

```
noise = NoiseConfig()

noise.sx.x = 0.01
noise.rz.z = 0.01
noise.cz.iy = 0.02
noise.mov.loss = 0.005
```

Ce code produit le modèle de bruit suivant, où le taux de bruit est la probabilité que la source provoque le type de bruit correspondant.

Source de bruit	Type de bruit	Taux de bruit
porte	Retournement de bit	1 %
porte	Inversion de phase	1 %
porte	Retournement de bit et retournement de phase sur le qubit cible	2 %
Le mouvement du qubit	Perte de qubit	0.5%

6. Exécutez le simulateur avec le modèle de bruit et affichez un histogramme des résultats de mesure. Par exemple, exécutez le code suivant dans une nouvelle cellule pour exécuter 1 000 captures de votre programme sur le simulateur Clifford.

#### Python

```
device = NeutralAtomDevice()

results = device.simulate(qir, shots=1000, noise=noise, type="clifford")
Histogram(results, labels="kets")
```

# Bienvenue à QDK pour la chimie

[Bien démarrer ici](#)

QDK pour la chimie (QDK/Chemistry) est un package open source Python dans le Microsoft Quantum Development Kit (QDK). Il fournit un ensemble d'outils et de bibliothèques pour permettre des flux de travail de chimie quantique de pointe sur des ordinateurs quantiques en réduisant l'écart entre les calculs de structure électronique classiques et l'exécution de circuits quantiques.

Les pipelines d'applications quantiques dépendent de la préparation et de la post-traitement des données classiques robustes et efficaces. QDK/Chemistry répond à ces besoins par le biais d'une architecture modulaire qui sépare les représentations de données des méthodes de calcul. Cette modularité permet aux chercheurs de composer des flux de travail flexibles à partir de composants interchangeables. Le kit de ressources fournit des implémentations natives d'algorithmes classiques quantiques clés et s'intègre en toute transparence aux packages de chimie quantique open source largement utilisés et aux infrastructures de calcul quantique via un système de plug-in. En offrant une interface unifiée à diverses méthodes et outils, QDK/Chemistry sert de plateforme pour l'innovation et une base pour les expériences de chimie quantique reproductibles et évolutives.

## Pourquoi utiliser QDK pour la chimie ?

### Intégration / Familiarité

QDK/Chemistry est intégré en toute transparence aux outils de chimie quantique et aux langages de programmation quantique tiers, ce qui permet l'utilisation flexible d'outils et d'environnements préférés. Le codage et le débogage assistés par l'IA peuvent être effectués en VS Code, tandis que le développement avec Python peut être effectué en Jupyter Notebook.

### Accessibilité

QDK/Chemistry est accessible aux chercheurs en chimie quantique de chaque niveau. Pour les débutants, le kit de ressources propose des outils et des algorithmes avec des paramètres par défaut robustes que vous pouvez utiliser pour obtenir des résultats pour une grande variété de systèmes moléculaires dès le départ. Les utilisateurs avancés peuvent tirer parti des

fonctionnalités étendues et de la personnalisation des QDK/Chemistry outils pour aborder des questions de recherche plus complexes.

## Modularity

QDK/Chemistry simplifie le processus de développement avec un ensemble d'outils modulaire. Chaque partie du flux de travail de chimie quantique fonctionne indépendamment, de sorte que les mêmes outils peuvent être réutilisés entre différents flux de travail et molécules.

## Implementation

QDK/Chemistry offre une solution de bout en bout rapide, intuitive et transparente pour chaque étape du flux de travail de chimie quantique. L'ensemble d'outils tire parti des techniques d'optimisation de pointe qui optimisent les performances, réduisent les besoins en ressources et les échelles pour exploiter la puissance du matériel quantique de nouvelle génération.

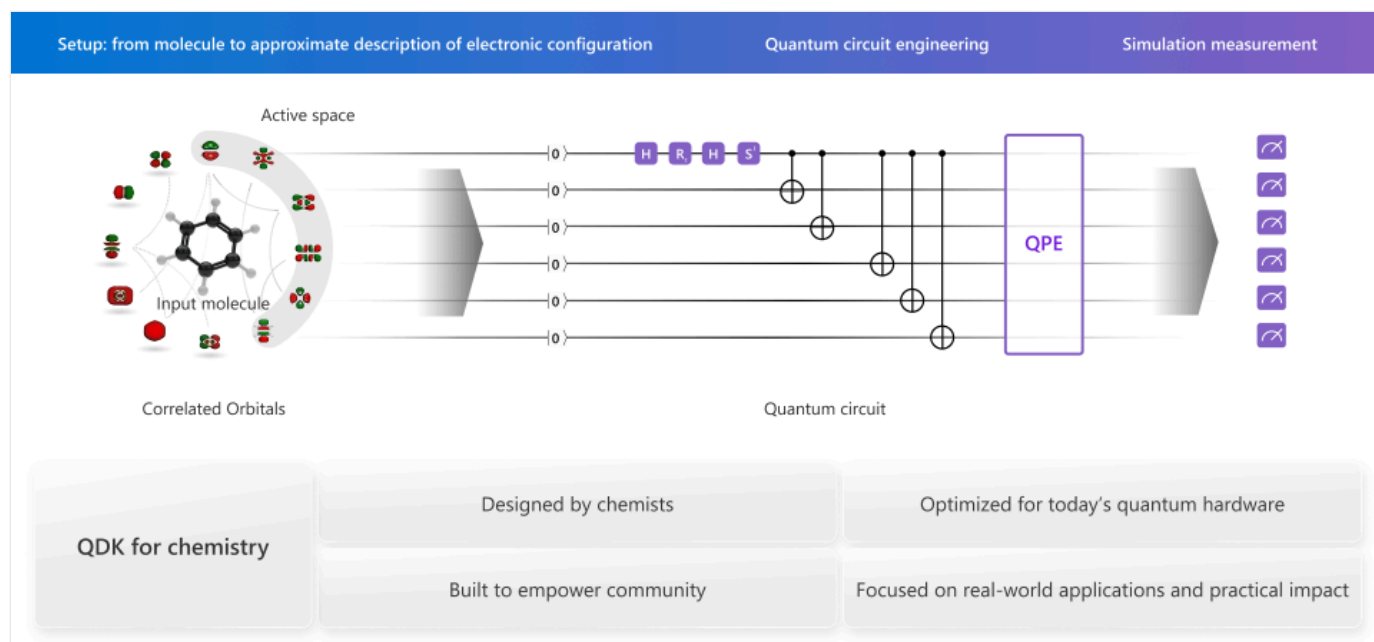
## Documentation et exemples

QDK/Chemistry est fourni avec une suite d'exemples de code pour vous aider à bien démarrer avec le développement. Les exemples incluent un large éventail de systèmes moléculaires, de langages de programmation quantique et de didacticiels qui parcourent les flux de travail de bout en bout de la structure moléculaire jusqu'à l'exécution d'algorithmes quantiques. La documentation comprend également une référence complète API qui explique le fonctionnement de chaque outil et algorithme, ainsi que des exemples d'utilisation des outils.

## QDK/Chemistry principales fonctionnalités et technologies

- [Calculs SCF robustes et sélection fiable de l'espace actif automatique](#)
- [Préparation efficace de circuits quantiques avec des isométries clairsemées GF2+X](#)
- Intégration transparente avec QDK [des simulateurs natifs hautes performances et estimateur de ressources](#)
- VS Code et des extensions Python pour QDK/Chemistry, [alimentés par des capacités de codage assisté par l'IA](#) pour vous aider à écrire et à comprendre les charges de travail en informatique quantique

## QDK: Mappage de la chimie à un ordinateur quantique



## Ressources QDK/Chemistry

Pour commencer avec QDK/Chemistry, consultez [Comment installer QDK pour la chimie](#).

Pour obtenir un exemple de bout en bout d'un flux de travail de préparation de l'état de chimie quantique, consultez le [guide de démarrage rapide](#) dans la QDK/Chemistry documentation sur GitHub.

Pour plus d'informations sur les outils disponibles dans QDK/Chemistry, consultez les références [Python API](#) et [C++ API](#).

Last updated on 29/05/2026



# Effectuer des calculs et une sélection d'espace actif pour construire des orbitales moléculaires

Dans la chimie quantique, la théorie de la structure électronique est un cadre qui décrit l'organisation des électrons dans les orbitales moléculaires (). Les méthodes de champ auto-cohérent () sont une technique de structure électronique que vous utilisez pour calculer les électrons.

Dans la chimie quantique, la méthode est connue sous le nom de méthode (HF), qui représente la fonction d'onde électronique de la molécule comme un déterminant unique qui satisfait au principe d'exclusion de Pauli. Chaque terme dans le déterminant est un produit d'orbitales de spin. La plupart des autres méthodes sont basées sur la méthode HF.

## Fonctionnement des méthodes

Il est difficile de décrire la structure électronique exacte d'une molécule, car tous les électrons sont corrélés entre eux. Lorsqu'une molécule a de nombreux électrons, il devient coûteux de calculer les interactions entre tous les électrons individuels. Les méthodes traitent les interactions plusieurs corps comme une interaction de champ moyenne unique plutôt que des interactions individuelles entre les particules. Les calculs trouvent un champ moyen dû à tous les électrons, et chaque électron individuel interagit avec ce champ moyen.

Un calcul est une procédure itérative pour rechercher les coefficients des OM qui sont cohérents avec le champ moyen générés par . Vous faites une estimation initiale des coefficients MO, puis la méthode génère une densité d'électrons et calcule l'énergie de la molécule à partir de l'estimation initiale. Ensuite, la méthode utilise la densité d'électrons pour construire un nouvel ensemble de coefficients MO, qui sont utilisés pour calculer une nouvelle densité d'électrons et l'énergie.

Le processus se répète jusqu'à ce que les modifications de la densité d'électrons et de l'énergie soient inférieures aux critères de convergence que vous spécifiez. Cette méthode pour calculer l'énergie est appelée méthode variationnelle. Pour les méthodes variationnelles, l'énergie calculée est garantie d'être supérieure ou égale à l'énergie exacte de l'état terrestre électronique de la molécule.

## Sélection d'espace actif

Pour certains systèmes moléculaires, une fonction d'onde déterminant unique ne modélise pas correctement la structure électronique et la corrélation, même dans la limite définie de base.

Dans ces cas, une fonction d'onde multifacteur est utilisée pour modéliser les orbitales moléculaires () qui sont les plus impliquées dans l'activité chimique ou qui ont la plus de corrélation.

La sélection d'espace actif est le processus de choix des issus du calcul à inclure dans la fonction d'onde multidéterminante. La chimie () dans le Kit de développement Quantum () propose des outils qui choisissent automatiquement le meilleur espace actif pour votre problème en fonction d'un ensemble de paramètres que vous spécifiez, tels que le nombre d'électrons et le nombre d'orbitales.

## Commencez les calculs SCF et la sélection de l'espace actif dans QDK les bibliothèques de chimie

Pour plus d'informations sur les calculs dans le , consultez le solveur de champ auto-consistant () .

Pour en savoir plus sur la sélection d'espace actif dans le QDK/Chemistry, consultez [Sélection d'espace actif](#) .

Pour obtenir des exemples de code de calculs SCF et de sélection d'espace actif dans le QDK/Chemistry, consultez [Exécuter un champ auto-cohérent \(SCF\)](#) et [Select an active space](#) dans le guide de démarrage rapide QDK/Chemistry.

# Générer des circuits de préparation d'état pour les calculs de chimie quantique avec des isométrie éparses

La préparation de l'état est une sous-routine centrale dans de nombreux algorithmes quantiques pour les calculs de chimie quantique, comme l'estimation de phase quantique (QPE).

Le circuit de préparation de l'état place le système qubit sur l'ordinateur quantique dans un état qui représente l'état quantique réel de votre molécule. Les techniques d'isométrie sont utilisées pour créer le circuit de préparation de l'état à partir de la fonction d'onde que vous calculez pour votre molécule. Les approches isométrie classiques entraînent généralement des circuits profonds qui contiennent beaucoup de portes quantiques. Chaque porte d'un circuit a le potentiel d'introduire le bruit dans le calcul sur l'ordinateur quantique, de sorte que le circuit idéal est un circuit qui effectue un calcul précis avec les portes les plus rares.

## Génération de circuit d'isométrie éparses avec les QDK bibliothèques de chimie

QDK pour la chimie (QDK/Chemistry) dans le Microsoft Quantum Development Kit (QDK) comprend une nouvelle technique d'isométrie éparses qui utilise un petit nombre de qubits denses pour créer efficacement un circuit optimal de préparation d'état avec moins de profondeur et de portes que les techniques traditionnelles d'encodage isométrique. Le circuit de l'isométrie éparses prépare le même état initial que le circuit plus grand de l'isométrie régulière, mais le circuit d'isométrie éparses est moins coûteux en termes de calcul et introduit moins de bruit dans le calcul sur l'ordinateur quantique.

L'implémentation d'isométrie éparses dans QDK/Chemistry vous permet d'encoder des fonctions d'onde ab initio dans un flux de travail linéaire et cohérent, facilitant la création de circuits de préparation d'état optimaux pour une grande variété de molécules et de systèmes. Pour commencer, consultez [Comment installer QDK pour la chimie](#).

## Learn more

Pour en savoir plus sur les circuits de préparation de l'état et les isométrie éparses dans QDK/Chemistry, consultez [la préparation de l'état](#) dans la QDK/Chemistry documentation sur GitHub.

---



# Comment installer QDK pour la chimie

Dans cet article, vous allez apprendre à installer QDK pour la chimie (QDK/Chemistry), une Python bibliothèque pour les calculs de chimie quantique dans le Microsoft Quantum Development Kit (QDK).

## Prerequisites

- Installez la dernière version de [Visual Studio \(VS\) Code](#).
- Installez les extensions [Python](#) et [Jupyter](#) dans VS Code.
- Installez un Python interpréteur (version 3.11, 3.12 ou 3.13).

### ❗ Important

Windows La prise en charge de QDK/Chemistry est assurée via le Windows sous-système pour Linux (WSL). Pour l'utiliser QDK/Chemistry sur Windows des machines, vous devez [installer WSL](#). Pour l'utiliser VS Code dans WSL, [installez l'extension WSL](#).

## Installer la `qdk-chemistry` bibliothèque

QDK/Chemistry est distribué en tant que `qdk-chemistry` Python bibliothèque via PyPI. Pour installer le package, procédez comme suit :

1. Créez un Python environnement virtuel. Par exemple, `myenv` :

Bash

```
python3 -m venv myenv
```

2. Activez l'environnement virtuel :

Bash

```
source myenv/bin/activate
```

3. Installez la `qdk-chemistry` bibliothèque avec tous les extras :

#### Bash

```
python3 -m pip install "qdk-chemistry[all]"
```

#### ⚠ Remarque

Pour utiliser QDK/Chemistry avec Jupyter Notebook dans VS Code, assurez-vous que votre notebook utilise l'interpréteur Python dans l'environnement virtuel où vous avez installé `qdk-chemistry`.

Pour optimiser votre build, consultez [Instructions d'installation pour QDK/Chemistry](#) sur GitHub pour des instructions détaillées sur l'installation manuelle.

## Contenu connexe

Pour en savoir plus sur les simulations de chimie quantique et sur l'utilisation QDK/Chemistry, consultez les articles suivants :

- [Générer des circuits de préparation d'état pour les calculs de chimie quantique avec des isométrie éparses](#)
- [Effectuer des calculs SCF et une sélection d'espace actif pour construire des orbitales moléculaires](#)

---

Last updated on 03/04/2026

# Comment utiliser le visualiseur de molécules avec QDK pour la chimie

Le Microsoft Quantum Development Kit (QDK) inclut un visualiseur de molécules à utiliser avec QDK pour la chimie (QDK/Chemistry). Vous pouvez utiliser le visualiseur pour interagir avec la structure 3D de votre molécule et superposer les orbitales moléculaires (MOs) dans un Jupyter notebook.

## Prerequisites

Pour utiliser le visualiseur de molécules, installez les outils suivants :

- Python environnement (version 3.11, 3.12 ou 3.13) avec Python et Pip
- Visual Studio Code (VS Code) avec l'extension Jupyter Notebook ou ouvrez VS Code pour le Web
- La dernière version de la bibliothèque `qdk` Python avec l'extra `jupyter`, et la bibliothèque `qdk-chemistry` avec tous les extras :

Bash

```
pip install --upgrade "qdk[jupyter]" "qdk-chemistry[all]"
```

## Créer un objet `Structure`

Le visualiseur de molécules nécessite un `Structure` objet de la `qdk-chemistry` bibliothèque. Un `Structure` objet contient les coordonnées tridimensionnelles et le type d'élément de chaque atome dans la molécule. Pour créer un objet `Structure`, vous pouvez charger des données à partir d'un fichier de structure `.xyz` ou `.json`, ou spécifier directement les coordonnées et les éléments dans votre code.

Par exemple, pour charger un `Structure` objet pour *p-benzyne* à partir d'un `.xyz` fichier, procédez comme suit :

1. Dans VS Code, ouvrez le dossier dans lequel vous souhaitez enregistrer vos fichiers.
2. Créez un fichier texte vide nommé `para_benzyne.structure.xyz`.

3. Copiez le texte suivant dans `para_benzyne.structure.xyz` et enregistrez le fichier.

plaintext

```
10
para benzyne
C 0.000000 1.396000 0.000000
C 1.209077 0.698000 0.000000
C 1.209077 -0.698000 0.000000
C 0.000000 -1.396000 0.000000
C -1.209077 -0.698000 0.000000
C -1.209077 0.698000 0.000000
H 2.151000 1.242000 0.000000
H 2.151000 -1.242000 0.000000
H -2.151000 -1.242000 0.000000
H -2.151000 1.242000 0.000000
```

4. Dans VS Code, ouvrez le menu **Affichage** et choisissez **Palette de commandes**.

5. Entrez et sélectionnez **Créer : Nouveau Jupyter Notebook**. Un nouvel onglet s'ouvre avec un fichier vide Jupyter Notebook .

6. Dans la première cellule de votre bloc-notes, copiez et exécutez le code suivant pour charger la structure :

Python

```
from qdk_chemistry.data import Structure

structure = Structure.from_xyz_file('para-benzyne.structure.xyz')

structure
```

Pour plus d'informations sur les objets `Structure` dans QDK/Chemistry, consultez [Structure](#) dans la documentation QDK/Chemistry sur GitHub.

## Afficher la structure moléculaire

Pour ouvrir le visualiseur de molécules, utilisez la `to_xyz` méthode pour passer une chaîne XYZ au `MoleculeViewer` widget. Copiez et exécutez le code suivant dans une nouvelle cellule :

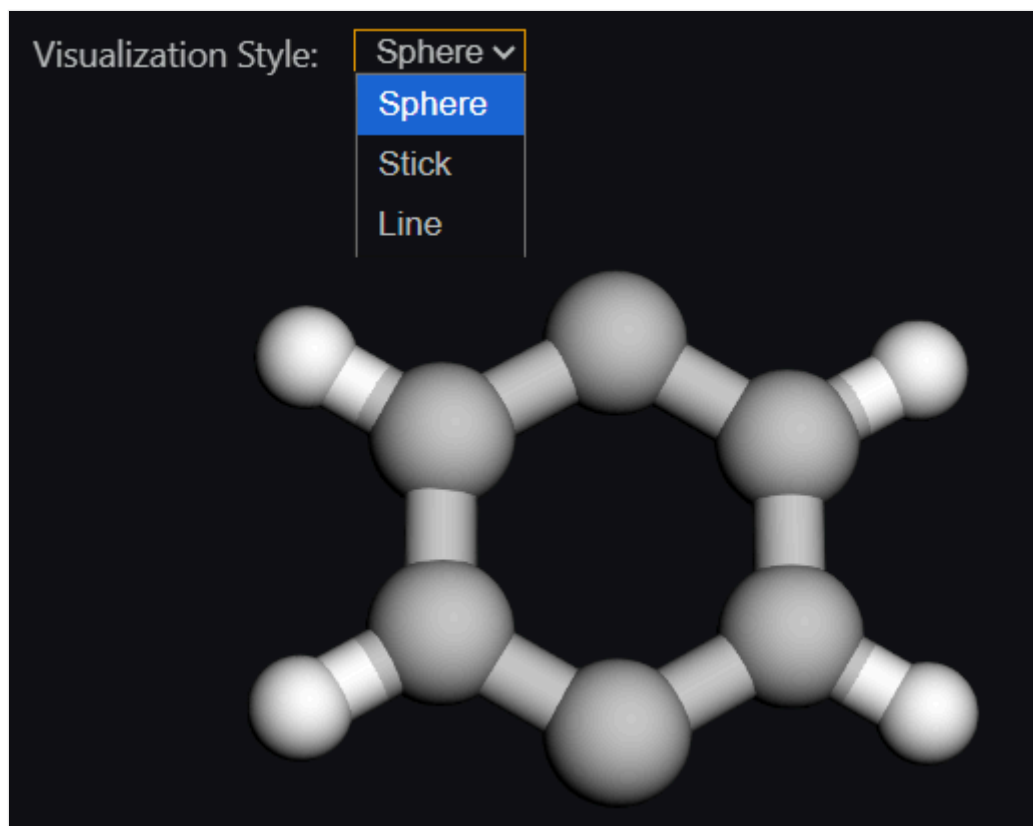
Python

```
from qdk.widgets import MoleculeViewer
```



```
viewer = MoleculeViewer(molecule_data=structure.to_xyz())
viewer
```

Le visualiseur de molécules s'ouvre dans la cellule de sortie et restitue une représentation 3D de la structure moléculaire. Le visualiseur dessine automatiquement des liaisons entre les atomes qui se trouvent dans un seuil de distance chimiquement raisonnable.



Pour modifier la représentation visuelle, ouvrez la liste déroulante **Style de visualisation** et choisissez l'une des options suivantes :

[🔍 Agrandir le tableau](#)

Style de visualisation	Descriptif
Sphère	Représentation de remplissage d'espace qui montre des atomes sous forme de sphères avec une taille et une couleur spécifiques à l'élément
Clé USB	Modèle bâton qui montre des liaisons explicites entre les atomes et des couleurs spécifiques à chaque élément
Ligne	Représentation filaire montrant la connectivité de liaison, mais sans couleur

## Interagir avec le visualiseur de molécules

Utilisez les contrôles de souris et de clavier suivants pour interagir avec le visualiseur de molécules

 Agrandir le tableau

Input	Action
Cliquez avec le bouton gauche et faites glisser	Faire pivoter la molécule dans l'espace 3D
Maj + clic gauche et glisser	Zoom avant (glisser vers le haut) ou zoom arrière (glisser vers le bas)
Ctrl + clic gauche et glisser	Traduire la molécule

## Afficher les orbitales moléculaires

Si vous avez `.cube` des fichiers qui stockent les données MO de votre molécule, vous pouvez charger ces fichiers dans le visualiseur de molécules pour afficher les MOs superpositions sur la structure de la molécule. Stockez les données de cube dans un Python dictionnaire avec des étiquettes MO en tant que clés, puis passez le dictionnaire à `MoleculeViewer`.

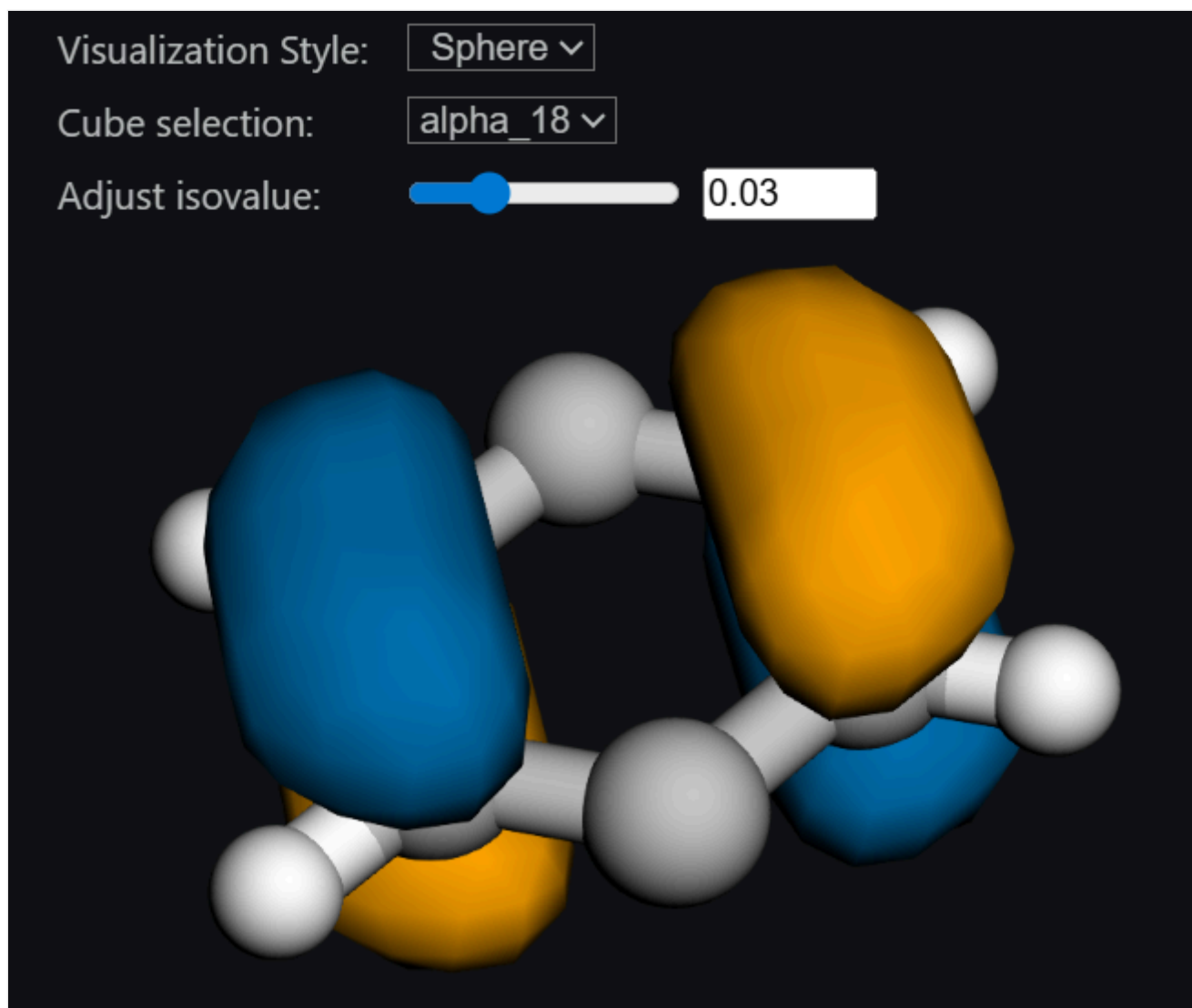
Par exemple, le code suivant s'affiche pour deux orbitales spatiales MOs actives dans *p-benzynes* lorsque vous avez les fichiers de cube dans votre répertoire de travail :

### Python

```
from pathlib import Path

cube_data = {
 "alpha_18": Path("MO_alpha_18.cube").read_text(),
 "alpha_19": Path("MO_alpha_19.cube").read_text(),
}

MoleculeViewer(molecule_data=structure.to_xyz(), cube_data=cube_data, isoval=0.03)
```



Lorsque vous transmettez des données de cube au `MoleculeViewer` widget, le visualiseur possède les autres éléments d'interface utilisateur suivants :

[🔍 Agrandir le tableau](#)

Élément d'interface utilisateur	Descriptif
Sélection du cube	Choisissez le MO que le visualiseur affiche. Vous ne pouvez afficher qu'un seul MO à la fois.
Ajuster isovalue	Définissez la valeur isovalue des coefficients MO. La valeur isovalue détermine la façon dont les MOs sont rendus.

Pour plus d'informations sur comment générer des `.cube` fichiers, consultez [`qdk\\_chemistry.utils.cubegen` module](#) dans la QDK/ChemistryAPI référence sur GitHub.

## Afficher d'autres informations MO

Le visualiseur de molécules peut également afficher des informations personnalisées sur les MOs. Pour ajouter des informations personnalisées à la sortie du visualiseur, utilisez des dictionnaires pour définir les données du cube et inclure la "info" clé. La valeur de la clé "info" est un autre dictionnaire qui contient les informations personnalisées.

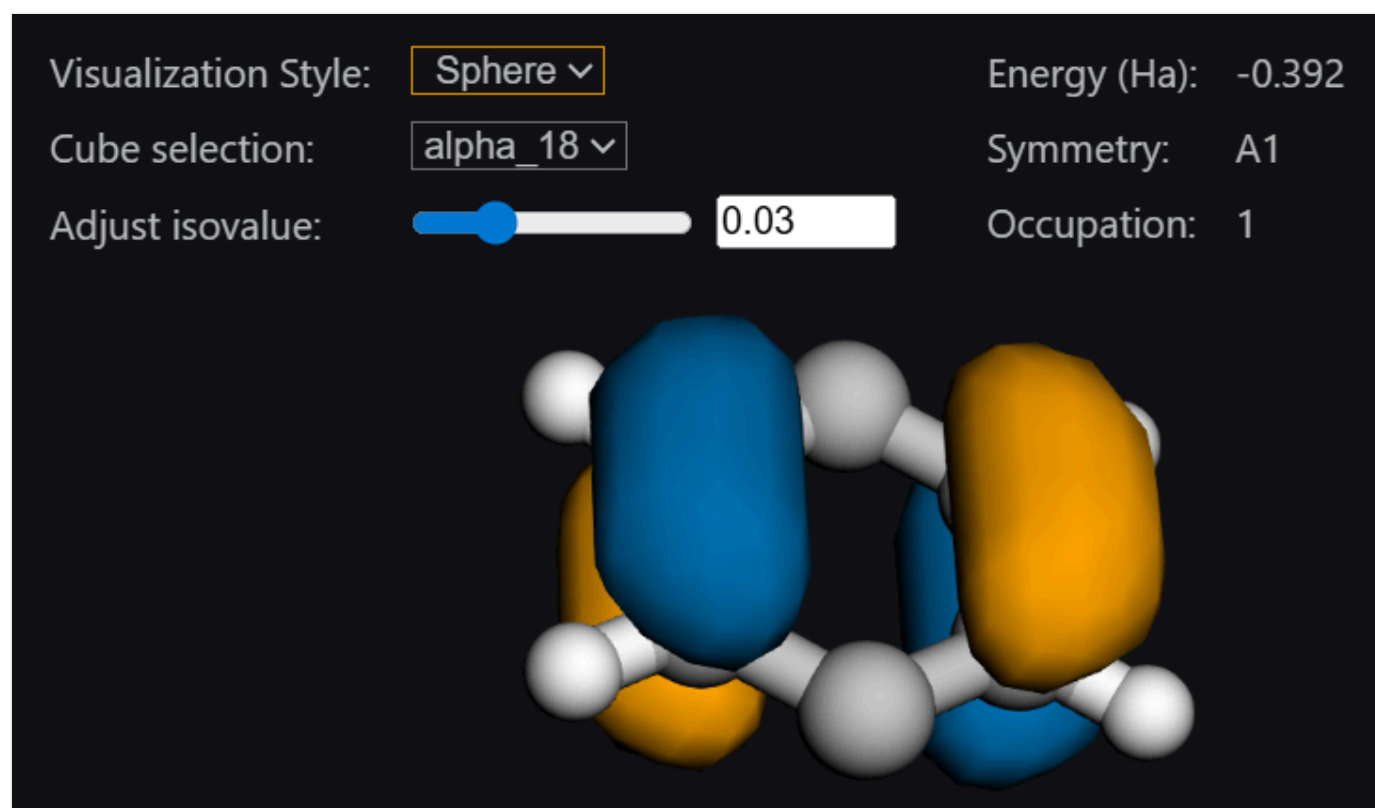
Par exemple, le code suivant affiche l'énergie, la symétrie et l'occupation électronique de chaque MO :

#### Python

```
from pathlib import Path

cube_data = {
 "alpha_18": {"data": Path("MO_alpha_18.cube").read_text(),
 "info": {"Energy (Ha)": -0.392, "Symmetry": "A1", "Occupation":
1.0}},
 "alpha_19": {"data": Path("MO_alpha_19.cube").read_text(),
 "info": {"Energy (Ha)": 0.581, "Symmetry": "B2", "Occupation":
0.0}}
}

MoleculeViewer(molecule_data=structure.to_xyz(), cube_data=cube_data, isoval=0.03)
```



# Qu'est-ce que l'estimateur de ressource Microsoft Quantum ?

Les ordinateurs quantiques sont sujets au bruit et aux erreurs, car les qubits individuels sont très sensibles aux changements dans les conditions environnementales. L'informatique quantique tolérante aux pannes nécessite des algorithmes qui utilisent des codes de correction d'erreur quantique (QEC). Ces codes créent et utilisent des qubits logiques à partir de collections de qubits physiques individuels. Les qubits logiques corrigent les erreurs pendant qu'un programme s'exécute sur un ordinateur quantique afin que les calculs quantiques puissent fournir des résultats fiables.

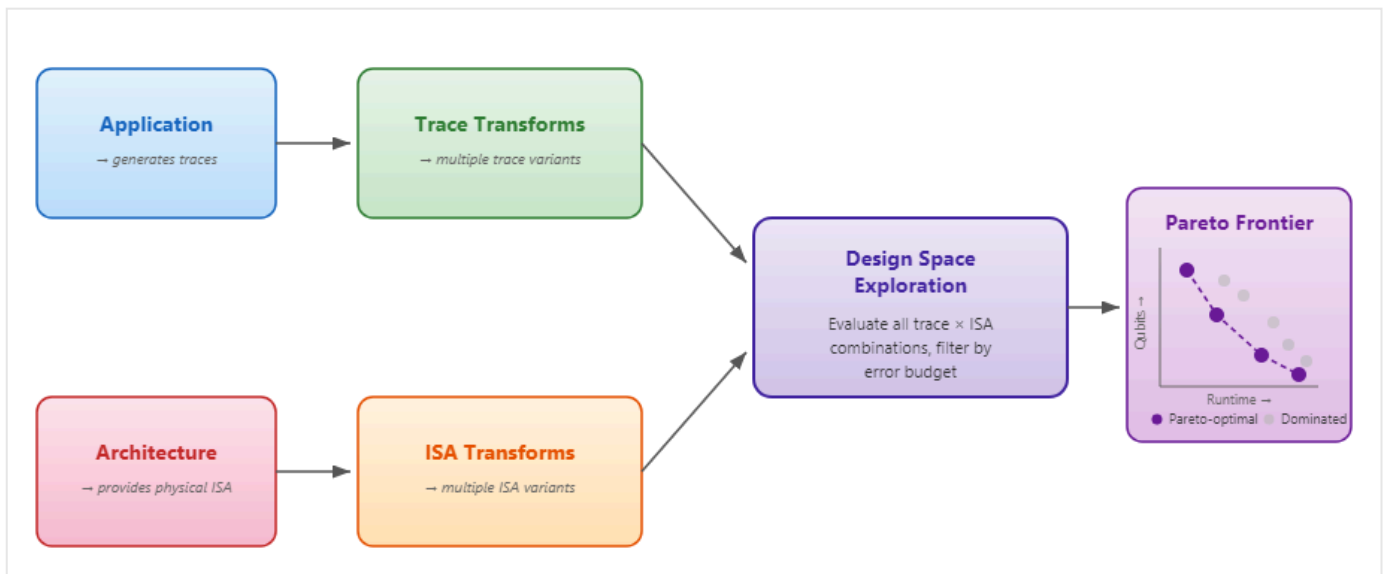
L'estimateur de ressources Microsoft Quantum est un outil [open source](#) dans la bibliothèque Microsoft Quantum Development Kit (QDK) Python qui vous permet d'estimer les ressources nécessaires pour exécuter un programme quantique sur un ordinateur quantique tolérant aux pannes. Les codes de correction d'erreur augmentent le nombre de qubits physiques et le temps d'exécution d'un algorithme quantique. L'estimateur de ressources détermine le nombre de qubits physiques et le temps nécessaire pour qu'une application quantique s'exécute sur un matériel spécifique avec un schéma de correction d'erreur donné.

Avec l'estimateur de ressources quantiques, vous pouvez comparer les technologies qubit, les schémas de correction d'erreurs quantiques et les technologies matérielles pour comprendre l'impact sur les ressources nécessaires pour exécuter un programme quantique.

## Comment fonctionne l'estimateur de ressources quantiques ?

L'estimateur de ressources est conçu autour d'une approche de modélisation en couches pour être flexible et composable. Vous construisez des modèles pour l'application, le matériel et le schéma de correction des erreurs. Vous spécifiez également comment ces modèles interagissent. Tous les modèles et instructions sont indépendants les uns des autres. Vous pouvez donc combiner et comparer différentes combinaisons de modèles.

L'estimateur de ressources utilise des représentations intermédiaires pour connecter les couches. Ces représentations font partie de l'implémentation de l'estimateur et ne sont pas requises pour une utilisation classique. Vous interagissez avec l'estimateur de ressources via des langages de programmation standard et des modèles de configuration de haut niveau.



L'estimateur de ressources quantiques nécessite quatre entrées :

- Modèle d'application qui décrit le calcul quantique, tel qu'un programme Q#
- Modèle d'architecture qui décrit le matériel quantique target, par exemple des qubits supraconducteurs à portes logiques avec des temps de porte et des taux d'erreur spécifiques
- Un modèle de correction des erreurs et de distillation en usine correspondant au modèle d'architecture matérielle
- Un budget d'erreur, qui est le taux d'erreur maximal autorisé pour les opérations sur des qubits logiques

À partir du modèle d'application, l'estimateur de ressource génère une ou plusieurs traces d'applications. Ces traces sont des représentations compactes des séquences d'instructions appliquées aux qubits. À partir du modèle d'architecture et du modèle de correction d'erreurs, l'estimateur de ressources dérive un jeu d'instructions physiques (ISA) qui spécifie les temps d'opération, les coûts de qubit et les taux d'erreur.

La couche application et la couche d'architecture sont converties en transformations configurables. Les transformations de traçage issues de la couche applicative incluent des décompositions de portes et des routines d'agencement. Les transformations ISA de la couche d'architecture incluent des codes de correction d'erreurs quantiques et des fabriques d'état magiques pour créer des ensembles d'instructions logiques haute fidélité à partir des primitives physiques de l'architecture.

L'estimateur de ressources explore un espace de conception combinatoire volumineux, car il existe de nombreux choix de conception valides à chaque couche. Par exemple, différentes distances de code QEC, différents protocoles d'usine et différents paramètres de décomposition. L'estimateur de ressources explore chaque combinaison de trace d'application et d'architecture

ISA pour évaluer les nombres de qubits physiques, les temps d'exécution et les taux d'erreur cumulés pour chaque combinaison. Les résultats sont filtrés jusqu'à une frontière pareto-optimale, qui est l'ensemble optimal de configurations où aucun autre résultat n'est simultanément mieux dans les qubits et le runtime tout en restant dans le budget d'erreur spécifié.

## Prise en charge du langage d'application

L'estimateur de ressources accepte les applications quantiques dans plusieurs langages de programmation et formats intermédiaires suivants :

- Q#
- Cirq
- OpenQASM
- QIR
- Comptages logiques
- Applications personnalisées

Avec QIR et la possibilité de créer des applications personnalisées, l'estimateur de ressources est indépendant du langage et prend en charge un large éventail d'outils de programmation quantique. Les applications dans différents frameworks sont analysées à l'aide du même pipeline d'estimation. Vous pouvez également créer des applications personnalisées en dehors de ces frameworks pris en charge.

## Prise en charge matérielle de l'architecture

Les estimations de ressources dépendent des hypothèses relatives au target matériel et au code de correction des erreurs qui protègent les opérations logiques contre les erreurs. L'estimateur de ressources inclut des modèles intégrés pour les architectures courantes et les schémas de correction des erreurs. Vous pouvez également définir vos propres modèles de correction de matériel et d'erreur personnalisés pour explorer des architectures hypothétiques, évaluer les technologies émergentes ou étudier la façon dont les modifications apportées aux paramètres physiques affectent les exigences en matière de ressources pour exécuter un programme.

## Comparer les estimations et visualiser les résultats

L'estimateur de ressources vous permet d'estimer les ressources nécessaires pour exécuter le même algorithme quantique pour différentes configurations de modèles d'architecture target . Vous pouvez tracer les résultats de chaque configuration pour comparer les résultats. Lorsque

vous comparez des estimations, vous pouvez comprendre comment l'architecture qubit, le schéma QEC et d'autres paramètres matériels ont un impact sur les ressources globales requises pour exécuter votre programme.

## Prise en main de l'estimateur de ressources

Pour commencer, consultez [Comment installer et utiliser l'estimateur de ressource Quantum Microsoft](#).

Pour plus d'informations et d'exemples de code, consultez les [exemples d'estimateur de ressources quantiques dans le référentiel QDK sur GitHub](#) [↗](#).

---

Last updated on 17/06/2026



# Comment installer et utiliser l'estimateur de ressources Microsoft Quantum

L'estimateur de ressources Microsoft Quantum fait partie de l'Microsoft Quantum Development Kit (QDK). Dans cet article, vous allez apprendre à installer l'estimateur de ressources quantiques et à exécuter une estimation pour un programme Q# simple avec un modèle d'architecture par défaut.

## Avertissement

L'estimateur de ressources dans l'extension QDK pour VS Code sera bientôt déconseillé. Utilisez le module pour effectuer l'estimation `qdk.qre` Python des ressources.

## Prerequisites

Pour suivre les étapes décrites dans cet article, vous devez installer les éléments suivants :

- La dernière version de [Visual Studio Code \(VS Code\)](#) 
- Dernière version de [l'extension QDK](#)  installée dans VS Code
- Les versions les plus récentes de [l'Python extension](#)  et de [l'extension Jupyter](#)  dans VS Code

## Installer l'estimateur de ressources quantiques

L'estimateur de ressources est disponible via le `qdk.qre` Python module. Pour utiliser l'estimateur de ressources, installez la dernière version de la `qdk` Python bibliothèque avec les `qre` éléments supplémentaires :

Bash

```
pip install --upgrade "qdk[qre]"
```

## Conseil

Vous n'avez pas besoin d'un compte Azure pour utiliser l'estimateur de ressource.

# Exécuter une estimation simple des ressources avec les paramètres par défaut

Pour effectuer une estimation de ressource de base, passez un modèle d'application, un modèle d'architecture matérielle et un modèle de correction d'erreur à l'estimateur de ressource. Ensuite, exécutez l'estimateur de ressource pour obtenir les résultats. L'exemple suivant exécute l'estimateur de ressources à partir de Jupyter Notebook dans VS Code.

## Créer un modèle d'application

Pour cet exemple, utilisez l'algorithme du modèle d'Ising 1D issu des exemples Q# dans l'extension QDK pour VS Code. Vous pouvez également utiliser l'un de vos programmes Q# existants.

## Écrire un programme Q#

1. Dans VS Code, créez un fichier texte et enregistrez le fichier sous `ising-1d-sample.qs`.
2. Sur la première ligne du fichier, entrez **l'exemple** pour afficher la liste d'exemples de programmes Q#.
3. Choisissez **l'exemple Ising Model (Simple 1D)**, puis enregistrez le fichier.

## Créer une application

Utilisez le programme Q# pour créer un modèle d'application pour l'estimateur de ressources.

1. Pour créer un fichier Jupyter Notebook dans VS Code, ouvrez la **palette de commandes** et entrez **Create : New Jupyter Notebook**.
2. Dans la première cellule du bloc-notes, évaluez le programme Q#.

Python

```
from pathlib import Path
from qdk import qsharp

qsharp.eval(Path("ising-1d-sample.qs").read_text(encoding="utf-8"))
```

Ce code crée un `Main` objet dans l'espace `qdk.code` de noms qui contient le programme Q#.

3. Dans une nouvelle cellule, créez une application estimateur de ressources à partir du programme Q#.

### Python

```
import qdk
from qdk.qre.application import QSharpApplication

app = QSharpApplication(qdk.code.Main)
```

## Créer un modèle d'architecture

Pour l'architecture matérielle, utilisez le modèle par défaut `GateBased` avec lequel l'estimateur de ressource est fourni. Le modèle `GateBased` nécessite des entrées pour le taux d'erreur, le temps de porte et le temps de mesure.

Dans une nouvelle cellule, créez une architecture basée sur une porte avec un taux d'erreur de  $10^{-4}$ , une heure de porte de 100 ns et une durée de mesure de 500 ns.

### Python

```
from qdk.qre.models import GateBased

Times are in units of nanoseconds
arch = GateBased(error_rate=1e-4, gate_time=100, measurement_time=500)
```

## Exécuter une estimation

Pour exécuter une estimation, utilisez la `estimate` fonction et transmettez les entrées suivantes :

- Application
- Architecture
- Requête ISA (Physical Instruction Set)
- Taux d'erreur maximal

La requête ISA indique à l'estimateur de ressource quelles combinaisons de code de correction d'erreur et de protocoles de distillation T factory à évaluer. L'estimateur de ressources détermine l'ensemble de résultats optimisé sur la frontière de Pareto à partir de ces combinaisons. Pour cet exemple, utilisez le `SurfaceCode` QEC et le protocole de distillation `RoundBasedFactory` par défaut fournis avec l'estimateur de ressources.

### Python

```
from qdk.qre import estimate
from qdk.qre.models import SurfaceCode, RoundBasedFactory
```

```
results = estimate(app, arch, isa_query=SurfaceCode.q() * RoundBasedFactory.q(),
max_error=0.01)``
```

## Afficher les résultats

Pour afficher les combinaisons optimales du nombre de qubits physiques et du temps d'exécution du programme, utilisez la `as_frame` méthode pour afficher les résultats d'estimation dans un DataFrame pandas.

### Python

```
results.as_frame()
```

Pour tracer les résultats, utilisez la `plot_estimates` méthode et passez une unité pour le temps d'exécution.

### Python

```
from qdk.qre import plot_estimates

plot_estimates(results, figsize=(6, 4), runtime_unit="ms")
```

---

Last updated on 17/06/2026

# Création de modèles d'application pris en charge dans l'estimateur de ressources quantiques

L'estimateur de ressources Microsoft Quantum prend un ou plusieurs programmes quantiques comme entrée et convertit les programmes en applications et les traces des applications pour l'estimation des ressources. Vous pouvez écrire des programmes pour l'estimateur de ressources dans plusieurs langages et infrastructures de programmation quantique. Cet article explique comment importer des programmes et créer des applications pour les frameworks pris en charge par défaut.

Pour obtenir des instructions sur l'installation et l'utilisation de l'estimateur de ressources dans le Microsoft Quantum Development Kit (QDK), consultez [Comment installer et utiliser l'estimateur de ressources Microsoft Quantum](#).

## Avertissement

L'estimateur de ressources dans l'extension QDK pour VS Code sera bientôt déconseillé. Utilisez le module pour effectuer l'estimation `qdk.qre` Python des ressources.

## Créer des modèles d'application pour les frameworks pris en charge

L'estimateur de ressources fournit des outils par défaut pour importer des programmes quantiques écrits dans les frameworks suivants :

- Q#
- Cirq
- OpenQASM
- Représentation intermédiaire quantique (QIR)

Pour générer les applications suivantes et exécuter des estimations pour celles-ci, créez un notebook Jupyter dans Visual Studio Code (VS Code) :

1. Dans Visual Studio Code (VS Code), ouvrez le menu **Affichage** et sélectionnez **Palette de commandes**.

## 2. Entrez **Créer : Nouveau bloc-notes Jupyter**.

Un bloc-notes Jupyter vide s'ouvre dans un nouvel onglet.

## Créer une application Q#

Pour importer un programme Q#, vous pouvez charger un fichier Q# existant `qs` ou écrire un Q# directement dans Python.

Pour écrire directement un programme Q# dans Jupyter Notebook et le charger comme application dans l'estimateur de ressources, suivez les étapes ci-dessous dans votre notebook Jupyter vide :

1. Importez le `qsharp` module à partir de la bibliothèque QDK Python .

Python

```
from qdk import qsharp
```

2. Dans une nouvelle cellule, rédigez une opération en Q# qui effectue une addition à retenue en cascade sur deux registres de qubits.

Q#

```
%%qsharp

import Std.Arithmetic.*;

// Ripple-carry addition on two n-qubit registers
operation Program(n : Int) : Unit {
 use a = Qubit[n];
 use b = Qubit[n];

 RippleCarryCGIncByLE(a, b);
}
```

3. Pour créer un modèle d'application, encapsulez le programme `QSharpApplication` compilé et transmettez la taille du registre qubit en tant qu'argument d'exécution.

Python

```
from qdk import code
from qdk.qre.application import QSharpApplication
```

```
Wrap the compiled Q# entry point with a register size of 10 qubits
qsharp_app = QSharpApplication(code.Program, args=(10,))
```

4. L'application contient des traces que l'estimateur de ressource utilise pour évaluer différentes configurations de paramètres d'application. Pour afficher un résumé de l'ISA nécessaire pour exécuter la première trace, utilisez la méthode `enumerate_traces` de l'application.

#### Python

```
Get the first generated application trace
trace = next(app.enumerate_traces())

View the physical instruction set required to run the trace
trace.required_isa.as_frame()
```

Le résumé montre que la trace nécessite une porte **CCX** et une mesure dans la base **Z**.

## Créer une application Cirq

Pour générer une application Cirq, importez la `cirq` bibliothèque pour écrire un programme Cirq, puis passez le programme à `CirqApplication`. Par exemple, utilisez l'assistant intégré `qft` de `cirq` pour créer un programme de transformée de Fourier quantique à 10 qubits.

#### Python

```
import cirq
from qdk.qre.application import CirqApplication

Build a 10-qubit QFT circuit and make it an application
circuit = cirq.Circuit(cirq.qft(*cirq.LineQubit.range(10)))
cirq_app = CirqApplication(circuit)
```

Affichez le résumé ISA pour la première trace de l'application Cirq :

#### Python

```
trace = next(cirq_app.enumerate_traces())
trace.required_isa.as_frame()
```

## Créer une application OpenQASM

Pour générer une application OpenQASM, transmettez une chaîne source OpenQASM 3 à `OpenQASMApplication`. Par exemple, utilisez Qiskit pour générer un programme OpenQASM 3

pour un multiplicateur RGQFT de 4 qubits, puis convertissez le programme en application.

#### Python

```
from qdk.qre.application import OpenQASMApplication
from qiskit.circuit.library import RGQFTMultiplier
from qiskit.qasm3.exporter import Exporter

Export a Qiskit circuit to OpenQASM 3
qasm = Exporter().dumps(RGQFTMultiplier(num_state_qubits=4))

qasm_app = OpenQASMApplication(qasm)
```

Consultez le résumé ISA de la première trace de l'application OpenQASM :

#### Python

```
trace = next(qasm_app.enumerate_traces())
trace.required_isa.as_frame()
```

## Créer une application QIR

QIR est une représentation intermédiaire basée sur LLVM pour les programmes quantiques. Les programmes de la plupart des autres frameworks de langage quantique peuvent être compilés en QIR. Pour créer une application à partir de QIR, passez le QIR en `QIRApplication` tant que chaîne ou code binaire.

Par exemple, compilez un circuit de préparation d'état OpenQASM Bell en QIR, puis convertissez une représentation sous forme de chaîne du QIR en application d'estimateur de ressources.

#### Python

```
from qdk.openqasm import compile
from qdk.qre.application import QIRApplication

qasm_source = """
 include "stdgates.inc";
 bit[2] c;
 qubit[2] q;
 h q[0];
 cx q[0], q[1];
 c = measure q;
 """

qir = compile(qasm_source)

qir_app = QIRApplication(str(qir))
```



Consultez le résumé de l'ISA pour la première trace de l'application QIR :

#### Python

```
trace = next(qir_app.enumerate_traces())
trace.required_isa.as_frame()
```

## Exécuter des estimations de ressources pour plusieurs applications

Le code précédent a créé quatre applications d'estimateur de ressources, chacune d'une infrastructure de programmation différente. Avec l'estimateur de ressources, vous pouvez exécuter des estimations sur les quatre applications et comparer les résultats. Pour comparer les estimations, les applications n'ont pas besoin d'être du même framework de programmation.

Pour exécuter des estimations pour les quatre applications avec les mêmes modèles d'architecture, procédez comme suit :

1. Créez un modèle d'architecture basé sur une porte avec une heure de porte de 100 ns et un temps de mesure de 500 ns.

#### Python

```
from qdk.qre.models import GateBased

arch = GateBased(gate_time=100, measurement_time=500)
```

2. Créez une requête ISA pour un QEC surfacique et une distillation d'usine par cycles.

#### Python

```
from qdk.qre.models import SurfaceCode, RoundBasedFactory

isa_query = SurfaceCode.q() * RoundBasedFactory.q()
```

3. Exécutez des estimations pour chaque application.

#### Python

```
apps = [("Q# adder", qsharp_app), ("Cirq QFT", cirq_app), ("QIR Bell state",
qir_app), ("OpenQASM multiplier", qasm_app)]

estimates = []
```

```
for name, app in apps:
 estimates.append(estimate(app, arch, isa_query, max_error=0.01, name=name))
```

4. Tracez les estimations optimales de la frontière Pareto pour chaque application.

Python

```
from qdk.qre import plot_estimates

plot_estimates(estimates)
```

## Applications personnalisées

L'estimateur de ressources quantiques offre la possibilité de créer vos propres applications personnalisées à partir d'un ensemble de paramètres de programme fixes et de paramètres d'application variables. Pour plus d'informations, consultez [Comment créer des modèles d'application personnalisés dans l'estimateur de ressources quantiques](#).

---

Last updated on 18/06/2026

# Comment créer des modèles d'application personnalisés dans l'estimateur de ressources quantiques

L'estimateur de ressources Microsoft Quantum prend en charge les applications par défaut pour plusieurs frameworks de programmation quantique, tels que Q# et Cirq. Les paramètres de ces types d'applications pris en charge sont prédéfinis par l'estimateur de ressource. L'estimateur de ressources vous permet également de créer des applications personnalisées lorsque vous avez besoin d'un contrôle plus précis sur les paramètres de l'application.

## Avertissement

L'estimateur de ressources dans l'extension QDK pour VS Code sera bientôt déconseillé. Utilisez le module pour effectuer l'estimation `qdk.qre` Python des ressources.

## Paramètres de modèle d'application personnalisé

Pour générer vos propres applications, utilisez la `Application` sous-classe. Les modèles d'application personnalisés se composent de deux types de paramètres :

 Agrandir le tableau

Type de paramètre	Description	Exemples
Paramètres d'instance	Définissez le problème résolu par l'application. Ces paramètres sont corrigés pour une exécution d'estimation donnée.	Taille du bit d'un adder, longueur de clé d'un schéma de chiffrement
Paramètres de trace	Hyperparamètres algorithmiques qui contrôlent la façon dont l'application résout le problème. L'estimateur de ressources explore toutes les combinaisons de ces paramètres pendant l'estimation.	Choix d'un circuit d'ajout, ratio calcul/mémoire, stratégie d'éviction

## Exemple d'application personnalisée

En guise d'exemple d'application personnalisée, créez une application d'addition configurable avec les paramètres suivants :

- **Paramètre d'instance** : taille du bit
- **Paramètres de trace** : implémentation d'add, fraction de calcul, stratégie de gestion de la mémoire

Avec une taille de bits fixe, l'estimateur de ressources produit des traces d'application pour chaque combinaison d'implémentation d'add, de fraction de calcul et de stratégie de gestion de la mémoire. La sortie est le résultat pareto optimal qui réduit les compromis entre le nombre de qubits et le temps d'exécution total de l'algorithme.

Pour créer l'additionneur avec les paramètres personnalisés dans Q#, procédez comme suit :

1. Dans VS Code, ouvrez le menu **Affichage** et sélectionnez **Palette de commandes**.
2. Entrez **Créer : Nouveau bloc-notes Jupyter**.
3. Dans la première cellule, importez `qsharp` pour utiliser la `%%qsharp` cellule magique.

Python

```
from qdk import qsharp
```

4. Dans une nouvelle cellule, exécutez le code de la fonction adder Q#. Certaines des variables, telles `adder` que et `strategy`, sont définies dans les étapes suivantes.

Q#

```
%%qsharp
```

```
import Std.Arithmetic.*;
import Std.Convert.*;
import Std.Math.*;
import Std.ResourceEstimation.*;
```

```
// An adder operation that with custom instance and trace parameters for resource estimation
```

```
operation Adder(n : Int, adder: (Qubit[], Qubit[]) => Unit, computeFraction: Double, strategy: Int) : Unit {
 use a = Qubit[n];
 use b = Qubit[n];
```

```
 if computeFraction > 0.0 {
 // Enables an automatic memory/compute architecture with a fixed number of compute qubits
 // (computed as a fraction of the total number of qubits). The strategy controls how
 // compute qubits are deallocated if free compute qubits are needed (either least recently
 // used or least frequently used).
```

```

 EnableMemoryComputeArchitecture(Ceiling(computeFraction *
IntAsDouble(n)), strategy);
 }

 // Run the adder
 adder(a, b);

 // Reset all qubits (if code is used in simulation)
 ResetAll(a + b);
}

// Re-export adders and strategies to use in Python, together with strings to
display the results
function CGAdder(): ((Qubit[], Qubit[]) => Unit, String) { (RippleCarryCGIncByLE,
"CG") }
function TTKAdder(): ((Qubit[], Qubit[]) => Unit, String) {
(RippleCarryTTKIncByLE, "TTK") }
function LRU(): (Int, String) { (LeastRecentlyUsed(), "LRU") }
function LFU(): (Int, String) { (LeastFrequentlyUsed(), "LFU") }

```

5. Définissez les paramètres de trace comme un `dataclass` avec `kw_only=True`, car l'estimateur de ressource traite uniquement les champs de mot clé uniquement comme des paramètres de trace de variable. D'autres champs sont corrigés pendant l'estimation.

#### Python

```

from dataclasses import dataclass, field
from typing import Any
from qdk import code

@dataclass(kw_only=True)
class AdderParameters:
 adder: tuple[Any, str] = field(default = code.CGAdder(), metadata={"domain":
[code.CGAdder(), code.TTKAdder()]})
 compute_fraction: float = field(default=1.0, metadata={"domain": [0.5, 1.0,
1.5, 2.0]})
 strategy: tuple[int, str] = field(default=code.LRU(), metadata={"domain":
[code.LRU(), code.LFU()]})

```

Vous devez spécifier un `"domain"` dans le dictionnaire de métadonnées de chaque mot-clé, car le domaine indique à l'estimateur de ressources quelles valeurs de trace explorer lors de l'estimation. Vous devez également spécifier une valeur par défaut dans la liste des valeurs de domaine pour chaque paramètre. L'estimateur de ressource évalue les paramètres de trace suivants pour le programme d'ajout :

Paramètre	Description	Values
<code>adder</code>	Implémentation de l'adder	Adder à retenue en cascade de Craig Gidney (CG), additionneur à retenue en cascade de Takahashi–Tani–Kunihiro (TTK)
<code>compute_fraction</code>	Ratio des qubits alloués au calcul actif par rapport à la mémoire	0.5, 1.0, 1.5, 2.0
<code>strategy</code>	Stratégie d'éviction lorsque la capacité de calcul est dépassée	Moins récemment utilisé (LRU), le moins fréquemment utilisé (LFU)

6. Pour lier les types de paramètres de trace, définissez `Application[AdderParameters]` comme sous-classe avec `Adder` les paramètres d'instance fixe. Définissez les paramètres d'instance en tant que champs de classe de données standard (non-mot clé). Dans ce cas, `bitsize` est le seul paramètre d'instance.

#### Python

```

from qdk.qre import Application
from qdk.qre.interop import trace_from_entry_expr
from qdk.qre.property_keys import custom_properties

Property keys are integers; we can use the custom_properties function to
define custom properties that do not conflict with existing properties.
ADDER, COMPUTE_FRACTION, STRATEGY = custom_properties(3)

@dataclass
class Adder(Application[AdderParameters]):
 bitsize: int

 def __post_init__(self):
 # Disable parallel trace generation since passing the Q# adder operations
 # is not thread-safe.
 self.disable_parallel_traces()

 def get_trace(self, parameters: AdderParameters):
 # obtain the adder function and its name
 adder_fn, adder_name = parameters.adder
 # obtain the memory/compute strategy and its name
 strategy, strategy_name = parameters.strategy
 # generate a trace from the Q# entry point with the specified parameters
 trace = trace_from_entry_expr(code.Adder, self.bitsize, adder_fn,
 parameters.compute_fraction, strategy)

 # Set trace properties for later analysis and display
 trace.set_property(ADDER, adder_name)
 trace.set_property(COMPUTE_FRACTION, parameters.compute_fraction)
 trace.set_property(STRATEGY, strategy_name)

```

```
return trace
```

Le `get_trace` génère une trace de l'application pour chaque combinaison de valeurs du paramètre de trace variable, avec des valeurs de paramètre d'instance fixes pour chaque trace. La `set_property` méthode définit des métadonnées personnalisées que vous pouvez utiliser pour filtrer les résultats d'estimation.

7. Créez un modèle d'architecture et un modèle d'ensemble d'instructions physiques (ISA) pour le protocole de correction d'erreur quantique (QEC) et de validation d'usine. Pour cet exemple, utilisez un modèle matériel à portes logiques et un modèle ISA qui inclut un code de surface 2D couplé afin de prendre en compte les qubits de mémoire.

#### Python

```
from qdk.qre.models import GateBased, RoundBasedFactory, SurfaceCode,
TwoDimensionalYokedSurfaceCode

Define the target architecture: gate-based with specified timing (ns)
arch = GateBased(gate_time=100, measurement_time=500)

Explore surface code distances x round-based magic state factories x 2D yoked
surface code distances
isa_query = SurfaceCode.q() * RoundBasedFactory.q() *
TwoDimensionalYokedSurfaceCode.q(source=SurfaceCode.q())
```

8. Exécutez l'estimateur de ressource et ajoutez des colonnes de résultats personnalisées.

#### Python

```
from qdk.qre import estimate

results = estimate(Adder(64), arch, isa_query, max_error=0.01, name="Configurable
adder")

Add results columns using default resource estimator functions
results.add_factory_summary_column()
results.add_qubit_partition_column()

Add results columns for custom application parameters
results.add_property_column(ADDER, "adder")
results.add_property_column(COMPUTE_FRACTION, "compute_fraction")
results.add_property_column(STRATEGY, "strategy")

results.as_frame()
```

9. Tracez les résultats de l'estimation de la frontière pareto.

```
Python
```

```
results.plot()
```

Pour plus d'informations sur l'accès aux résultats à partir de l'estimateur de ressource, consultez [Comment utiliser les résultats de l'estimateur de ressources quantiques](#).

## Contenu connexe

- [Qu'est-ce que l'estimateur de ressources Microsoft Quantum ?](#)
- [Création de modèles d'application pris en charge dans l'estimateur de ressources quantiques](#)

---

Last updated on 18/06/2026



# Comment créer des modèles d'architecture matérielle dans l'estimateur de ressources quantiques

L'estimateur de ressources Microsoft Quantum prend un modèle d'architecture matérielle comme l'une de ses entrées pour estimer les ressources nécessaires à l'exécution d'un programme sur un ordinateur quantique. Le modèle d'architecture décrit les caractéristiques matérielles physiques de l'ordinateur. L'estimateur de ressources inclut des modèles matériels par défaut et vous permet de créer des modèles personnalisés.

## Avertissement

L'estimateur de ressources dans l'extension QDK pour VS Code sera bientôt déconseillé. Utilisez le module pour effectuer l'estimation `qdk.qre` Python des ressources.

## Modèles d'architecture par défaut

L'estimateur de ressources quantiques fournit trois modèles d'architecture par défaut :

- La classe `GateBased` pour les qubits à usage général avec des dispositifs typiques à portes quantiques
- La classe `NeutralAtom` pour les dispositifs à qubits à atomes neutres
- Classe `Majorana` pour les qubits Majorana

Les paramètres des modèles par défaut s'appliquent à l'architecture entière. Par exemple, lorsque vous définissez une heure de porte, cette heure s'applique à toutes les portes de l'application. Si vous souhaitez des heures différentes pour différents types de portes, vous devez créer un modèle d'architecture personnalisé.

## Créer une architecture basée sur une porte

Pour créer un modèle d'architecture basé sur une porte par défaut, utilisez la `GateBased` classe. Le modèle basé sur une porte par défaut nécessite quatre paramètres :

Paramètre	Unités	Valeur par défaut	Description
<code>error_rate</code>	Sans unité	$10^{-4}$	Probabilité qu'une erreur se produise après une porte ou une mesure
<code>gate_time</code>	Nanosecondes	None	Temps nécessaire pour appliquer une porte à qubit unique
<code>measurement_time</code>	Nanosecondes	None	Temps nécessaire pour mesurer l'état d'un qubit
<code>two_qubit_gate_time</code>	Nanosecondes	Valeur de <code>gate_time</code>	Temps nécessaire pour appliquer une porte à deux qubits

Le code suivant crée un modèle d'architecture à base de portes avec des temps de porte de 200 ns et des temps de mesure de 500 ns, ainsi qu'un taux d'erreur global de  $10^{-5}$  :

Python

```

from qdk.qre.models import GateBased

gate_arch = GateBased(error_rate=1e-5, gate_time=200, measurement_time=500)

```

Pour afficher les propriétés de toutes les opérations prises en charge dans l'architecture basée sur la porte par défaut, exécutez le code suivant :

Python

```

gate_arch.provided_isa(gate_arch.context()).as_frame()

```

Chaque opération de porte et de mesure possède les paramètres par défaut suivants :

[🔗 Agrandir le tableau](#)

Paramètre	Description
<code>encoding</code>	Spécifie si l'opération agit sur des qubits physiques ou logiques
<code>arity</code>	Nombre d'entrées sur laquelle l'opération agit
<code>space</code>	Nombre de qubits physiques sur lesquels l'opération agit
<code>time</code>	Temps en nanosecondes nécessaire à l'exécution de l'opération
<code>error</code>	Probabilité qu'une erreur se produise après l'opération

## Créer une architecture de dispositif à atomes neutres

Pour générer un modèle d'architecture d'appareil atom neutre par défaut, utilisez la `NeutralAtom` classe. Ce modèle intègre les caractéristiques physiques des dispositifs à atomes neutres, telles que l'espacement des qubits, le déplacement des qubits et un jeu de portes spécifique. Le modèle atom neutre par défaut nécessite dix paramètres. Pour plus d'informations, consultez la [référence de l'API](#).

Le code suivant construit un modèle d'architecture à atomes neutres avec des temps de mesure de 5 000 ns, un espacement des atomes de 5  $\mu\text{m}$  et des valeurs par défaut pour les autres paramètres :

### Python

```
from qdk.qre.models import NeutralAtom

atom_arch = NeutralAtom(measurement_time=5000, atom_spacing=5)
```

Pour afficher les propriétés de toutes les opérations prises en charge dans l'architecture atom neutre par défaut, exécutez le code suivant :

### Python

```
atom_arch.provided_isa(gate_arch.context()).as_frame()
```

## Créer une architecture Majorana

Pour générer un modèle d'architecture Majorana qubit par défaut, utilisez la `Majorana` classe. Le modèle Majorana par défaut prend un taux d'erreur comme seule entrée. Toutes les heures d'opération et de mesure sont fixes à 1 000 ns.

Le code suivant génère un modèle d'architecture Majorana par défaut avec un taux d'erreur global de  $10^{-6}$  :

### Python

```
from qdk.qre.models import Majorana

maj_arch = Majorana(error_rate=1e-6)
```

Pour afficher les propriétés de toutes les opérations prises en charge dans l'architecture Majorana par défaut, exécutez le code suivant :

## Python

```
maj_arch.provided_isa(maj_arch.context()).as_frame()
```

# Créer des modèles d'architecture personnalisée

Un modèle d'architecture définit les opérations physiques que le matériel d'un ordinateur quantique peut effectuer, avec des heures et des taux d'erreur pour chaque opération. Les modèles `GateBased` et `Majorana` par défaut ont des opérations et des horaires fixes. L'estimateur de ressources vous permet de créer des modèles d'architecture personnalisés dans lesquels vous choisissez les opérations de jeu et peut définir des heures et des taux d'erreur différents pour chaque opération.

Pour créer un modèle d'architecture personnalisé, utilisez la `Architecture` sous-classe et définissez sa `provided_isa` méthode avec le `ISAContext` paramètre. Par exemple, créez un modèle où les portes à deux qubits ont des taux d'erreur plus élevés et des temps d'opération plus longs que des portes à qubit unique, et les opérations de mesure ont un temps et un taux d'erreur différents de ceux des opérations de porte.

Le code suivant génère un modèle configurable avec des opérations qubit physiques appelées `ParameterizedArchitecture`:

## Python

```
from dataclasses import KW_ONLY, dataclass
from qdk.qre import Architecture, ISAContext, ISA
from qdk.qre.instruction_ids import H, S, S_DAG, T, T_DAG, CNOT, CZ, MEAS_X, MEAS_Y, MEAS_Z

@dataclass
class ParameterizedArchitecture(Architecture):
 """
 A configurable architecture with separate timing and error parameters for
 single-qubit gates, two-qubit gates, and measurements.
 """

 _: KW_ONLY
 single_qubit_gate_time: int
 two_qubit_gate_time: int
 measurement_time: int
 single_qubit_error_rate: float
 two_qubit_error_rate: float
 measurement_error_rate: float

 def provided_isa(self, ctx: ISAContext) -> ISA:
 instructions = []
```

```

Single-qubit gates: Clifford gates and T gates
for gate_id in [H, S, S_DAG, T, T_DAG]:
 instructions.append(
 ctx.add_instruction(
 gate_id,
 encoding=PHYSICAL,
 arity=1,
 time=self.single_qubit_gate_time,
 error_rate=self.single_qubit_error_rate,
)
)

Measurements
for meas_id in [MEAS_X, MEAS_Y, MEAS_Z]:
 instructions.append(
 ctx.add_instruction(
 meas_id,
 encoding=PHYSICAL,
 arity=1,
 time=self.measurement_time,
 error_rate=self.measurement_error_rate,
)
)

Two-qubit entangling gates
for gate_id in [CNOT, CZ]:
 instructions.append(
 ctx.add_instruction(
 gate_id,
 encoding=PHYSICAL,
 arity=2,
 time=self.two_qubit_gate_time,
 error_rate=self.two_qubit_error_rate,
)
)

return ctx.make_isa(*instructions)

```

La méthode `add_instruction` de `ISAContext` vous permet de définir des caractéristiques physiques du matériel, telles que le type de porte, l'encodage des qubits et l'arité des qubits.

Pour utiliser le modèle personnalisé, appelez `ParameterizedArchitecture` et définissez des valeurs pour les paramètres personnalisés. Par exemple, le modèle suivant décrit le matériel avec des opérations rapides mais bruyantes :

#### Python

```

fast_noisy = ParameterizedArchitecture(
 single_qubit_gate_time=50,
 two_qubit_gate_time=100,

```

```

measurement_time=200,
single_qubit_error_rate=1e-4,
two_qubit_error_rate=5e-4,
measurement_error_rate=5e-4,
)

Inspect the physical ISA provided by the fast-but-noisy architecture
fast_noisy.provided_isa(fast_noisy.context()).as_frame()

```

## Exécuter des estimations pour les architectures matérielles personnalisées

Pour exécuter une estimation sur ce modèle, transmettez le `fast_noisy` modèle à la `estimate` fonction en tant que paramètre d'architecture avec un modèle d'application et un modèle de correction d'erreur.

1. Importez les modules et objets nécessaires.

### Python

```

from qdk import qsharp
from qdk.qre import estimate, plot_estimates
from qdk.qre.application import QSharpApplication
from qdk.qre.models import SurfaceCode, RoundBasedFactory

```

2. Générez un modèle d'application. Par exemple, écrivez un programme d'additionneur sur 8 bits en Q# avec des rotations sur un seul qubit.

### Q#

```

%%qsharp

import Std.Arithmetic.*;
import Std.Math.*;
import Std.Convert.*;

/// An 8-bit adder with single-qubit rotations.
/// The rotations introduce T gates via synthesis, which exercises
/// the magic state factory during resource estimation.
operation AdderWithRotations() : Unit {
 use a = Qubit[8];
 use b = Qubit[8];
 for i in 0..7 {
 Ry(PI() / IntAsDouble(i + 2), a[i]);
 }
 RippleCarryCGIncByLE(a, b);
}

```

3. Convertissez le programme Q# en modèle d'application.

Python

```
from qdk import code

app = QSharpApplication(code.AdderWithRotations)
```

4. Créez un modèle de correction d'erreurs et de distillation en usine. Par exemple, utilisez un code de correction d'erreurs de surface et une distillation en usine par cycles.

Python

```
isa_query = SurfaceCode.q() * RoundBasedFactory.q()
```

5. Pour exécuter une estimation, passez le modèle d'application, le modèle d'architecture et le modèle de correction d'erreur à la `estimate` fonction. Définissez un taux d'erreur maximal de 1%.

Python

```
result = estimate(app, fast_noisy, isa_query, max_error=0.01)

result.plot(figsize=(8, 5))
```

6. Pour comparer des estimations pour différents ensembles de paramètres d'architecture personnalisée, créez une liste d'estimations et tracez les résultats. Par exemple, comparez le `fast_noisy` modèle à un `slow_clean` modèle qui a des temps d'opération plus longs, mais moins de bruit. Définissez le même taux d'erreur maximal pour chaque estimation.

Python

```
slow_clean = ParameterizedArchitecture(
 single_qubit_gate_time=500,
 two_qubit_gate_time=1000,
 measurement_time=1000,
 single_qubit_error_rate=1e-6,
 two_qubit_error_rate=5e-6,
 measurement_error_rate=1e-5,
)

results = [
 estimate(app, fast_noisy, isa_query, max_error=0.01, name="Fast but noisy"),
 estimate(app, slow_clean, isa_query, max_error=0.01, name="Slow but clean")
]
```

```
plot_estimates(results, figsize=(8, 5), runtime_unit="ms")
```

## Contenu connexe

- [Comment créer des modèles d'application personnalisés dans l'estimateur de ressources quantiques](#)
- [Comment générer des modèles de correction d'erreurs dans l'estimateur de ressources quantiques](#)
- [Comment utiliser les résultats de l'estimateur de ressources quantiques](#)

---

Last updated on 17/06/2026



# Comment générer des modèles de correction d'erreurs dans l'estimateur de ressources quantiques

L'estimateur de ressources Microsoft Quantum utilise comme l'une de ses principales entrées un modèle de transformation du code de correction d'erreur quantique (QEC) et un modèle de transformation de fabrique d'états magiques. Les transformations QEC et factory déterminent la façon dont le jeu d'instructions physiques (ISA) du modèle d'architecture matérielle se convertit en jeu d'instructions logiques pour la correction des erreurs.

À partir du modèle de code QEC et du modèle de fabrique d'état magique, vous générez une requête ISA que vous passez à l'estimateur de ressource. La requête ISA définit les combinaisons de codes QEC et d'usines à états magiques que l'estimateur de ressources explore. Chaque combinaison de code QEC et de fabrique d'état magique détermine comment l'estimateur de ressources convertit l'ISA physique en ISA logique. Dans cet article, vous allez apprendre à générer des requêtes ISA à partir des modèles de code QEC par défaut et des modèles de fabrique d'état magique par défaut, et comment créer vos propres modèles personnalisés.

## Avertissement

L'estimateur de ressources dans l'extension QDK pour VS Code sera bientôt déconseillé. Utilisez le module pour effectuer l'estimation `qdk.qre` Python des ressources.

## Générer des requêtes ISA à partir du code QEC par défaut et des modèles d'usine d'état magique

L'estimateur de ressource quantique comprend quatre modèles de code QEC par défaut et trois modèles de fabrique d'état magique par défaut.

### Modèles de code QEC par défaut

L'estimateur de ressources quantiques inclut les modèles QEC par défaut suivants :

Modèle QEC	Description
<a href="#">SurfaceCode</a>	Code de surface pivoté basé sur la porte
<a href="#">SurfaceCodeLowMove</a>	Code de surface tourné à portes logiques pour matériel à atomes neutres doté de qubits ancillas mobiles
<a href="#">OneDimensionalYokedSurfaceCode</a>	Code de surface Yoked 1D qui fournit des instructions génériques sur la mémoire
<a href="#">TwoDimensionalYokedSurfaceCode</a>	Code de surface Yoked 2D qui fournit des instructions génériques sur la mémoire
<a href="#">ThreeAux</a>	Code de surface à base de mesures par paires avec trois qubits auxiliaires par mesure de stabilisateur

## Modèles par défaut d’usines à états magiques

L’estimateur de ressource quantique inclut les modèles d’usine d’état magique par défaut suivants :

 Agrandir le tableau

Modèle d’usine à états magiques	Description
<a href="#">RoundBasedFactory</a>	Fabrique d’état magique qui utilise des pipelines de distillation par cycles pour produire des instructions de porte T
<a href="#">Litinski19Factory</a>	Les usines T et CCZ basées sur l’article <a href="#">arXiv:1905.06903</a> ↗
<a href="#">GSJ24CCXFactory</a>	Usines 8T-to-CCZ basées sur l’article <a href="#">arXiv:2409.17595</a> ↗
<a href="#">GSJ24Factory</a>	Préparation d’états magiques basée sur l’article <a href="#">arXiv:2409.17595</a> ↗
<a href="#">MagicUpToClifford</a>	Transformation ISA qui ajoute des représentations équivalentes de Clifford des états magiques

## Générer une requête ISA à partir de modèles par défaut

Pour créer une requête ISA, combinez un modèle QEC avec un modèle de fabrique d’états magiques à l’aide de l’opérateur `+` ou `*`. Par exemple, créez une requête ISA à partir du modèle QEC de code de surface basé sur les portes et du modèle d’usine à états magiques basé sur les cycles.

```
Python
```

```
from qdk.qre.models import SurfaceCode, RoundBasedFactory

isa_query = SurfaceCode.q() * RoundBasedFactory.q()
```

# Créer des requêtes ISA à partir de modèles personnalisés de code QEC et de fabrique d'états magiques

L'estimateur de ressources quantiques vous permet de créer des modèles personnalisés de QEC et d'usine à états magiques. Ces modèles personnalisés utilisent la `ISATransform` sous-classe pour convertir des instructions qubit physiques en instructions qubit logiques.

Pour générer ces modèles personnalisés, utilisez la `ISATransform` sous-classe et définissez deux méthodes :

- `required_isa` : méthode statique qui définit les opérations à utiliser à partir de l'ISA du modèle d'architecture. Cette méthode ne peut pas référencer des champs, tels que `error_correction_threshold`. Utilisez plutôt `ConstraintBound` avec des opérateurs de comparaison pour définir des limites ou des plages sur des valeurs.
- `provided_isa` : calcule les propriétés d'instruction logique à partir d'instructions physiques pour produire un ou plusieurs isas logiques. Cette méthode référence les champs personnalisés dans votre modèle.

## Créer un modèle QEC personnalisé

Une transformation QEC encode les qubits en code de correction d'erreur en fonction des paramètres principaux suivants :

 Agrandir le tableau

Paramètre de code QEC	Description	Dépend de
Espace	Nombre de qubits physiques par qubits logiques	Distance du code
Time	Temps nécessaire pour effectuer une opération logique	Cycles d'extraction du syndrome
Error	Probabilité qu'une erreur se produise à partir d'une opération logique	Taux d'erreur physique, taux d'erreur maximal et distance de code

Les transformations QEC personnalisées doivent définir des valeurs pour ces paramètres. Par exemple, le code suivant génère un modèle QEC personnalisé appelé `GenericQEC` qui convertit la porte H physique, la porte CNOT et les instructions de mesure en un ensemble d'instructions logiques de chirurgie de treillis.

#### Python

```
from dataclasses import KW_ONLY, dataclass, field
from typing import Generator
from qdk.qre import (
 ISAContext,
 ConstraintBound,
 ISA,
 ISARequirements,
 ISATransform,
 LOGICAL,
 constraint,
 linear_function
)
from qdk.qre.instruction_ids import H, CNOT, MEAS_Z, LATTICE_SURGERY

@dataclass
class GenericQEC(ISATransform):
 """
 A configurable QEC code model with tunable threshold, overhead,
 and error suppression.

 This transform consumes physical H, CNOT, and MEAS_Z instructions and
 produces a logical LATTICE_SURGERY instruction. The resource formulas
 are parameterized so you can model different QEC code families.

 The logical error rate follows:

 error = crossing_prefactor × (p_physical / threshold) ^ [(d+1)/2]

 Space per data qubit is:

 qubits_per_data_qubit × d²
 """
 crossing_prefactor: float = 0.03
 error_correction_threshold: float = 0.01
 qubits_per_data_qubit: int = 2
 syndrome_extraction_depth: int = 4
 _: KW_ONLY
 distance: int = field(default=3, metadata={"domain": range(3, 22, 2)})

 @staticmethod
 def required_isa() -> ISARequirements:
 # required_isa is static, so we cannot reference instance fields here.
 # Instead, we use ConstraintBound to express fixed constraints on the
 # implementation ISA. ConstraintBound supports .lt(), .le(), .eq(),
```

```

.gt(), and .ge() comparisons.
return ISARquirements(
 constraint(H, error_rate=ConstraintBound.lt(0.01)),
 constraint(CNOT, arity=2, error_rate=ConstraintBound.lt(0.01)),
 constraint(MEAS_Z, error_rate=ConstraintBound.lt(0.01)),
)

def provided_isa(
 self, impl_isa: ISA, ctx: ISAContext
) -> Generator[ISA, None, None]:
 cnot = impl_isa[CNOT]
 h = impl_isa[H]
 meas_z = impl_isa[MEAS_Z]

 # Physical error rate is the worst case across all required gates
 physical_error_rate = max(
 cnot.expect_error_rate(),
 h.expect_error_rate(),
 meas_z.expect_error_rate(),
)

 d = self.distance

 # Space: physical qubits per data qubit, scaled by distance2.
 #
 # Because LATTICE_SURGERY has variable arity (it can operate on any
 # number of logical qubits), space and error_rate must be provided as
 # functions of arity rather than as fixed numbers. The estimator
 # provides several helpers for this:
 #
 # linear_function(slope) → f(n) = slope × n
 # constant_function(value) → f(n) = value
 # block_linear_function(k, s, o) → f(n) = s × [n/k] + o
 # generic_function(callable) → f(n) = callable(n)
 #
 # For QEC codes, resources scale linearly with the number of logical
 # qubits, so linear_function is the natural choice.
 space = linear_function(self.qubits_per_data_qubit * d**2)

 # Time: syndrome extraction cycle × code distance
 code_cycle_time = (
 h.expect_time()
 + self.syndrome_extraction_depth * cnot.expect_time()
 + meas_z.expect_time()
)
 time = code_cycle_time * d

 # Error: exponential suppression below threshold
 logical_error = self.crossing_prefactor * (
 (physical_error_rate / self.error_correction_threshold)
 ** ((d + 1) // 2)
)
 error = linear_function(logical_error)

 yield ctx.make_isa(

```

```

 ctx.add_instruction(
 LATTICE_SURGERY,
 encoding=LOGICAL,
 arity=None,
 space=space,
 time=time,
 error_rate=error,
 # transform=self records that this instruction was produced by
 # this QEC transform instance, and source=[...] records which
 # physical instructions it was derived from. Together, they
 # form the provenance chain that you can inspect in the
 # estimation results (see notebook 2).
 transform=self,
 source=[cnot, h, meas_z],
 # Extra keyword arguments (like distance) are stored as
 # properties on the instruction. This lets you retrieve the
 # code distance used for each Pareto-optimal result when
 # analyzing the estimation output. Further, the property can be
 # required and read by a parent ISATransform to create new
 # logical instructions based on this one.
 distance=d,
),
)

```

Vous pouvez appeler `GenericQEC` avec les valeurs de paramètre par défaut ou définir d'autres valeurs lorsque vous appelez le modèle. Par exemple, créez un modèle QEC qui a un seuil d'erreur élevé et une surcharge de qubit faible.

#### Python

```

custom_qec = GenericQEC.q(
 crossing_prefactor=0.01,
 error_correction_threshold=0.03,
 qubits_per_data_qubit=1,
)

```

## Créer un modèle personnalisé de fabrique d'état magique

Une transformation de modèle d'usine à états magiques produit des états magiques de haute fidélité à partir d'instructions physiques de bas niveau.

Vous pouvez créer un modèle de fabrique d'état magique personnalisé ou utiliser un modèle personnalisé pour explorer le comportement des modèles établis avec des modifications apportées à des paramètres spécifiques. Par exemple, générez un modèle de fabrique personnalisé appelé `ScaledLitinskiFactory` basé sur le modèle par défaut `Litinski19Factory`. Le modèle personnalisé vous permet de modifier le facteur d'espace pour les qubits logiques et le nombre de cycles requis par la fabrique.

## Python

```
from dataclasses import dataclass
from typing import Generator
from math import ceil
from qdk.qre import (
 ISAContext,
 ConstraintBound,
 ISA,
 ISARrequirements,
 ISATransform,
 LOGICAL,
 constraint
)
from qdk.qre.instruction_ids import T, H, CNOT, MEAS_Z

Reference data from Litinski (arXiv:1905.06903), Table 1.
Assumes Clifford and T error rates at most 1e-4.
Each tuple: (output_error_rate, space_in_physical_qubits, cycles)
_LITINSKI_TABLE = [
 (4.4e-8, 810, 18.1),
 (1.5e-9, 762, 36.2),
 (9.3e-10, 1150, 18.1),
 (1.9e-11, 2070, 30.0),
 (2.4e-15, 16400, 90.3),
 (6.3e-25, 18600, 67.8),
]

@dataclass
class ScaledLitinskiFactory(ISATransform):
 """
 A factory transform based on the Litinski (2019) distillation protocols,
 with configurable scaling factors for space and cycle costs.

 The base data is taken directly from Table 1 in arXiv:1905.06903.
 Each entry specifies a distillation protocol's output error rate, space
 footprint (in physical qubits), and time (in syndrome extraction cycles).
 The space already includes the QEC overhead within the factory itself.

 Scaling factors let you explore hypothetical improvements:

 - ``space_factor=0.5`` models a factory that uses half the qubits.
 - ``cycle_factor=0.5`` models a factory that runs twice as fast.

 Note:
 The error rates are kept fixed when scaling. In practice, changing
 space or cycles would affect error rates too, but modelling that
 relationship requires protocol-specific analysis.
 """

 space_factor: float = 1.0
 cycle_factor: float = 1.0

 @staticmethod
```

```

def required_isa() -> ISARrequirements:
 return ISARrequirements(
 constraint(T, error_rate=ConstraintBound.le(1e-4)),
 constraint(H, error_rate=ConstraintBound.le(1e-4)),
 constraint(CNOT, arity=2, error_rate=ConstraintBound.le(1e-4)),
 constraint(MEAS_Z, error_rate=ConstraintBound.le(1e-4)),
)

def provided_isa(
 self, impl_isa: ISA, ctx: ISAContext
) -> Generator[ISA, None, None]:
 cnot = impl_isa[CNOT]
 h = impl_isa[H]
 meas_z = impl_isa[MEAS_Z]
 t = impl_isa[T]

 # Syndrome extraction time from the physical ISA
 syndrome_extraction_time = (
 4 * cnot.expect_time() + h.expect_time() + meas_z.expect_time()
)

 for error_rate, base_space, base_cycles in _LITINSKI_TABLE:
 # Apply scaling factors to space and time
 space = ceil(base_space * self.space_factor)
 time = ceil(syndrome_extraction_time * base_cycles * self.cycle_factor)

 yield ctx.make_isa(
 ctx.add_instruction(
 T,
 encoding=LOGICAL,
 arity=1,
 space=space,
 time=time,
 error_rate=error_rate,
 transform=self,
 source=[cnot, h, meas_z, t],
),
)

```

Avec le modèle personnalisé `ScaledLitinskiFactory`, vous pouvez réduire `space_factor` pour explorer les fabriques qui utilisent moins de qubits physiques, ou réduire `cycle_factor` pour explorer les fabriques qui nécessitent moins de cycles de distillation. Par exemple, concevez un modèle qui utilise deux fois moins de qubits physiques que `LitinskiFactory`, et un autre modèle qui nécessite deux fois moins de cycles.

#### Python

```

compact_factory = ScaledLitinskiFactory.q(space_factor=0.5)
fast_factory = ScaledLitinskiFactory.q(cycle_factor=0.5)

```



# Générer une requête ISA à partir de modèles personnalisés

Pour générer une requête ISA à partir de modèles QEC et de distillation personnalisés, définissez les valeurs des paramètres personnalisés et combinez les modèles en utilisant l'opérateur `+` ou `*`. Par exemple, générez une requête ISA à partir du modèle QEC personnalisé que vous avez déjà créé. Définissez le `GenericQEC` modèle pour avoir un seuil d'erreur plus élevé et définissez le `ScaledLitinskiFactory` modèle pour exiger moins de cycles.

## Python

```
custom_qec = GenericQEC.q(error_correction_threshold=0.02)
custom_factory = ScaledLitinskiFactory.q(cycle_factor=0.75)

custom_query = custom_qec * custom_factory
```

# Exécuter des estimations pour les modèles de QEC et de distillation personnalisés

Pour exécuter des estimations avec vos modèles personnalisés, transmettez `custom_query` à la fonction `estimate` comme paramètre de requête ISA, ainsi qu'un modèle d'application et un modèle d'architecture matérielle.

1. Importez les modules et objets nécessaires.

## Python

```
from qdk import qsharp
from qdk.qre.application import QSharpApplication
from qdk.qre.models import GateBased
from qdk.qre import estimate, plot_estimates
```

2. Générez un modèle d'application. Par exemple, écrivez un programme d'additionneur sur 8 bits en Q# avec des rotations sur un seul qubit.

## Q#

```
%%qsharp

import Std.Arithmetic.*;
import Std.Math.*;
import Std.Convert.*;

/// An 8-bit adder with single-qubit rotations.
/// The rotations introduce T gates via synthesis, which exercises
```

```

/// the magic state factory during resource estimation.
operation AdderWithRotations() : Unit {
 use a = Qubit[8];
 use b = Qubit[8];
 for i in 0..7 {
 Ry(PI() / IntAsDouble(i + 2), a[i]);
 }
 RippleCarryCGIncByLE(a, b);
}

```

3. Convertissez le programme Q# en modèle d'application.

Python

```

from qdk import code

app = QSharpApplication(code.AdderWithRotations)

```

4. Créez un modèle d'architecture matérielle. Par exemple, utilisez le modèle basé sur la porte par défaut.

Python

```

arch = GateBased(error_rate=1e-4, gate_time=100, measurement_time=500)

```

5. Pour exécuter une estimation, transmettez le modèle d'application, le modèle d'architecture et la requête ISA personnalisée à la `estimate` fonction. Définissez un taux d'erreur maximal de 1%.

Python

```

result = estimate(app, arch, custom_query, max_error=0.01)

result.plot(figsize=(8, 5))

```

6. Pour comparer des estimations pour différents ensembles de modèles de fabriques d'états magiques et de QEC, créez une liste d'estimations et tracez les résultats. Par exemple, générez une requête à partir des paramètres par défaut des modèles personnalisés et comparez-les aux modèles qui ont des paramètres différents.

Python

```

baseline = GenericQEC.q() * ScaledLitinskiFactory.q()
high_cross = GenericQEC.q(crossing_prefactor=0.05) * ScaledLitinskiFactory.q()
compact_factory = GenericQEC.q() * ScaledLitinskiFactory.q(space_factor=0.5)
high_compact = GenericQEC.q(crossing_prefactor=0.05) *

```

```
ScaledLitinskiFactory.q(space_factor=0.5)

results = [
 estimate(app, arch, baseline, max_error=0.02, name="Baseline"),
 estimate(app, arch, high_cross, max_error=0.02, name="High crossing
prefactor"),
 estimate(app, arch, compact_factory, max_error=0.02, name="Compact factory"),
 estimate(app, arch, high_compact, max_error=0.02, name="High crossing
prefactor & Compact factory"),
]

plot_estimates(results, figsize=(8, 5), runtime_unit="ms")
```

## Contenu connexe

- [Comment créer des modèles d'application personnalisés dans l'estimateur de ressources quantiques](#)
- [Comment créer des modèles d'architecture matérielle dans l'estimateur de ressources quantiques](#)
- [Comment utiliser les résultats de l'estimateur de ressources quantiques](#)

---

Last updated on 17/06/2026

# Comment utiliser les résultats de l'estimateur de ressources quantiques

L'estimateur de ressources quantiques dans le Microsoft Quantum Development Kit (QDK) produit des estimations optimales du temps d'exécution et du nombre de qubits nécessaires pour exécuter un programme quantique sur un certain type de matériel. Pour chaque résultat d'estimation, vous pouvez obtenir des statistiques, l'architecture du jeu d'instructions (ISA), les usines à états magiques et d'autres propriétés. Vous pouvez également générer et tracer des propriétés de résultats personnalisées.

Cet article montre comment accéder aux résultats et les analyser à partir de l'estimateur de ressource et comment personnaliser les résultats. Les exemples de code sont destinés à être exécutés dans un notebook Jupyter dans l'ordre dans lequel ils apparaissent dans l'article.

Pour obtenir des instructions sur l'installation et l'utilisation de l'estimateur de ressources, consultez [Comment installer et utiliser l'estimateur de ressources Microsoft Quantum](#).

## Avertissement

L'estimateur de ressources dans l'extension QDK pour VS Code sera bientôt déconseillé. Utilisez le `qdk.qre` module Python pour effectuer l'estimation des ressources.

## Exécuter l'estimateur de ressource

Pour exécuter l'estimateur de ressources et utiliser les résultats d'estimation, créez un modèle d'application, un modèle d'architecture matérielle et un modèle de correction des erreurs. Ensuite, passez les modèles à l'estimateur de ressource et exécutez une estimation pour un taux d'erreur maximal donné.

## Créer un modèle d'application à partir d'un Q# programme

Pour cet exemple, écrivez un programme Q# qui effectue huit additions à propagation de retenue sur 8 bits en parallèle, avec une rotation sur un seul qubit précédant chaque addition. Les rotations introduisent la décomposition de la porte T dans les résultats d'estimation.

Pour créer une application à partir du programme d'addition Q#, suivez ces étapes dans un carnet Jupyter :

1. Importez les objets nécessaires.

Python

```
from qdk import qsharp
from qdk.qre.application import QSharpApplication
```

2. Dans une nouvelle cellule, écrivez le Q# programme.

Q#

```
import Std.Arithmetic.*;
import Std.Math.*;
import Std.Convert.*;

operation EstimateAdder() : Unit {
 for _ in 1..8 {
 use a = Qubit[8];
 use b = Qubit[8];
 for i in 0..7 {
 Ry(PI() / IntAsDouble(i + 2), a[i]);
 }
 RippleCarryCGIncByLE(a, b);
 }
}
```

3. Générez le modèle d'application.

Python

```
from qdk.code import EstimateAdder

app = QSharpApplication(EstimateAdder)
```

## Créer un modèle d'architecture matérielle

Créez un modèle d'architecture basé sur une porte à l'aide de la classe par défaut `GateBased`. Le modèle d'architecture a un taux d'erreur maximal de , des temps de porte de 100 ns et des temps de mesure de 500 ns.

Python

```
from qdk.qre.models import GateBased

arch = GateBased(error_rate=1e-4, gate_time=100, measurement_time=500)
```

# Générer un modèle de correction d'erreur

Pour le modèle de correction d'erreur, utilisez le code de surface QEC par défaut et le modèle par défaut de distillation de fabrique basé sur les cycles. Combinez le code QEC et le modèle de distillation pour créer une requête ISA qui définit l'ensemble d'isas d'architecture que l'estimateur de ressource explore.

## Python

```
from qdk.qre.models import SurfaceCode, RoundBasedFactory

isa_q = SurfaceCode.q() * RoundBasedFactory.q()
```

# Obtenir les résultats d'estimation de base

Pour exécuter l'estimateur de ressources, transmettez un seuil d'erreur maximal, un modèle d'application, un modèle d'architecture et une requête ISA à la `estimate` fonction. Pour augmenter le nombre de points d'estimation optimaux, transmettez également une requête de trace facultative qui définit l'ensemble des traces d'application que l'estimateur de ressources explore. Pour cet exemple, utilisez la transformation de trace `PSSPC` et la transformation de trace `LatticeSurgery` avec un ensemble de facteurs de ralentissement.

Définissez un taux d'erreur maximal de 1% et exécutez l'estimateur de ressource. Pour imprimer les résultats de base de chaque point optimal Pareto, utilisez la `as_frame` méthode.

## Python

```
from qdk.qre import estimate, PSSPC, LatticeSurgery

results = estimate(
 app,
 arch,
 isa_q,
 trace_query=PSSPC.q() * LatticeSurgery.q(slow_down_factor=[1.0, 1.5, 2.0, 3.0,
4.0]),
 max_error=0.01
)

print(f"Number of Pareto-optimal results: {len(results)}")
results.as_frame()
```

La `estimate` fonction retourne une `EstimationTable` valeur qui contient les estimations de ressources suivantes pour chaque point optimal :

Paramètre de résultats	Description
<code>qubits</code>	Nombre total de qubits physiques requis pour exécuter le programme
<code>runtime</code>	Le temps nécessaire pour que le programme termine son exécution
<code>error</code>	Taux d'erreur global après correction d'erreur

Le résultat de base de l'estimateur de ressource est un ensemble optimal de points pour les exigences de temps d'exécution et de qubits physiques dans un seuil d'erreur maximal. Ces informations vous aident à choisir les meilleurs paramètres pour exécuter votre programme sur du matériel quantique réel en fonction de vos contraintes de temps et de disponibilité physique du qubit.

## Tracer les résultats pareto optimaux

Pour afficher les résultats sous la forme d'un tracé du nombre de qubits par rapport à l'heure d'exécution, utilisez la `plot` méthode. Par exemple, tracez les six points pareto optimaux avec le temps d'exécution en millisecondes.

### Python

```
results.plot(figsize=(8, 5), runtime_unit="ms")
```

## Obtenir des résultats d'estimation détaillés

L'estimateur de ressources produit des informations détaillées sur le processus d'estimation et les résultats. Pour accéder à et afficher des informations plus détaillées sur l'estimation que vous avez effectuée, consultez les exemples de code suivants.

## Obtenir des statistiques d'estimation

Le résultat d'estimation a un `stats` attribut qui contient les informations suivantes sur le processus d'estimation :

Statistique d'estimation	Procédure accès
Nombre de traces d'application	<code>stats.num_traces</code>

Statistique d'estimation	Procédure accès
Nombre d'ISAs	<code>stats.num_isas</code>
Nombre de travaux d'estimation	<code>stats.total_jobs</code>
Nombre d'estimations inférieures au seuil d'erreur	<code>stats.successful_estimates</code>
Nombre de résultats pareto optimaux	<code>stats.pareto_results</code>

Le code suivant affiche ces informations pour l'estimation des ressources que vous avez faite :

```

Python

stats = results.stats

print(f"Traces explored: {stats.num_traces}")
print(f"ISAs explored: {stats.num_isas}")
print(f"Total estimation jobs: {stats.total_jobs}")
print(f"Successful estimates: {stats.successful_estimates}")
print(f"Pareto-optimal results: {stats.pareto_results}")

```

Le nombre total de tâches d'estimation possibles est le produit du nombre de traces et du nombre d'ISA. Toutefois, le nombre réel de travaux d'estimation est inférieur, car l'estimateur de ressources utilise une stratégie d'élagage qui conserve uniquement les combinaisons les plus pertinentes.

## Obtenir des détails pour des résultats pareto optimaux individuels

Le résultat d'estimation a un `EstimationTableEntry` élément pour chaque point pareto optimal. Les entrées de table contiennent des informations détaillées sur les estimations optimales individuelles.

Pour accéder à l'entrée de table d'estimation pour un point individuel, utilisez l'index pour ce point. Par exemple, exécutez le code suivant pour obtenir les résultats d'estimation pour le troisième point optimal.

```

Python

entry = results[2]

```

## Résultats de base



Pour accéder aux résultats d'estimation de base pour un point optimal individuel, utilisez les attributs `qubits`, `runtime` et `error`. Par exemple, le code suivant affiche les résultats d'estimation de base pour le troisième point optimal.

#### Python

```
print(f"Total physical qubits: {entry.qubits}")
print(f"Runtime: {entry.runtime} ns")
print(f"Error: {entry.error:.6f}")
```

## Propriétés détaillées

Chaque entrée de table d'estimation a un `properties` attribut qui contient des informations détaillées sur les fabriques et la distribution des qubits. Pour afficher toutes les propriétés, exécutez le code suivant :

#### Python

```
from qdk.qre import property_name

props = {property_name(k): v for k, v in entry.properties.items()}

for name, value in props.items():
 print(f" {name}: {value}")
```

Vous pouvez également accéder à chaque propriété individuellement. Par exemple, obtenez uniquement le nombre de portes T par rotation.

#### Python

```
from qdk.qre.property_keys import NUM_TS_PER_ROTATION

print(f"Number of T gates per rotation: {entry.properties.get(NUM_TS_PER_ROTATION, 'N/A')}")
```

## Afficher les informations sur la partition de qubits et la fabrique de qubits

L'estimateur de ressources fournit des outils qui vous permettent d'afficher des informations plus détaillées dans la table de résultats. Par exemple, la `add_qubit_partition_column` méthode ajoute des colonnes à la table de résultats qui montrent comment les qubits physiques sont distribués. Le `add_factory_summary_column` nombre d'usines T requises pour chaque point optimal est affiché.

### Python

```
results.add_qubit_partition_column()
results.add_factory_summary_column()

results.as_frame()
```

## Analyser l'ISA d'une estimation

Chaque résultat d'estimation optimal a un graphique source d'instructions qui décrit la façon dont les instructions logiques sont implémentées en termes d'opérations de niveau inférieur. Pour accéder au graphe de la source d'instruction d'un résultat, utilisez l'attribut `source` du `EstimationTableEntry` correspondant à ce résultat.

Par exemple, imprimez le graphique source d'instructions pour le troisième résultat dans la `entry` variable que vous avez créée.

### Python

```
print("Instruction source graph for the third result:\n")
print(entry.source)
```

Le graphique source d'instruction se comporte comme un dictionnaire Python où les ID d'instruction sont les clés de dictionnaire. Par exemple, exécutez le code suivant pour obtenir des informations sur les instructions de porte T.

### Python

```
from qdk.qre import instruction_name
from qdk.qre.instruction_ids import T

if T in entry.source:
 t_node = entry.source[T]
 t_inst = t_node.instruction
 print("T instruction:")
 print(f" ID: {instruction_name(t_inst.id)}")
 print(f" Encoding: {'LOGICAL' if t_inst.encoding == 1 else 'PHYSICAL'}")
 print(f" Arity: {t_inst.arity}")
 print(f" Space: {t_inst.space()} physical qubits")
 print(f" Time: {t_inst.time()} ns")
 print(f" Error rate: {t_inst.error_rate():.2e}")
 if t_node.transform is not None:
 print(f" Transform: {t_node.transform}")
```

## Obtenir des informations sur les fabriques

Chaque entrée de résultat d'estimation comporte un attribut `factories`, qui est un dictionnaire contenant les informations suivantes sur les fabriques d'états magiques :

- Type de fabrique
- Nombre de copies de la fabrique
- Nombre de fois que la fabrique s'exécute
- Nombre d'états magiques créés par l'usine
- Taux d'erreur après la distillation

Par exemple, exécutez le code suivant pour afficher toutes les informations de fabrique pour le troisième résultat :

#### Python

```
print("Factories in the third result:")

for inst_id, factory in entry.factories.items():
 print(f"\n {instruction_name(inst_id)} factory:")
 print(f" Copies: {factory.copies}")
 print(f" Runs: {factory.runs}")
 print(f" States: {factory.states}")
 print(f" Error rate: {factory.error_rate:.2e}")
```

## Personnaliser les résultats de l'estimateur de ressources

Vous pouvez ajouter des colonnes personnalisées à la sortie de `EstimationTable` de l'estimateur de ressources à l'aide des fonctions `add_property_column` et `add_column`.

`add_property_column` prend en entrée une clé de propriété et crée une colonne pour cette propriété dans le tableau de résultats. Par exemple, utilisez la `NUM_TS_PER_ROTATION` clé de propriété pour ajouter une colonne pour cette propriété.

#### Python

```
results.add_property_column(NUM_TS_PER_ROTATION)

results.as_frame()
```

La `add_column` fonction prend un nom de colonne et une fonction comme entrée. La fonction d'entrée obtient les valeurs des `EstimationTableEntry` attributs de chaque résultat pour remplir la nouvelle colonne. Par exemple, créez des colonnes pour le taux d'erreur de porte T, le taux d'erreur d'opération logique et le nombre d'états T.

## Python

```
from qdk.qre.instruction_ids import LATTICE_SURGERY

results.add_column(
 "t_error_rate",
 lambda r: r.source[T].instruction.error_rate() if T in r.source else None,
)
results.add_column(
 "logical_error_rate",
 lambda r: r.source[LATTICE_SURGERY].instruction.error_rate(1) if LATTICE_SURGERY
in r.source else None,
)
results.add_column(
 "t_states",
 lambda r: r.factories[T].states if T in r.factories else 0,
)

results.as_frame()
```

Last updated on 18/06/2026

# Apprendre l'informatique quantique et Q# avec les Katas dans QDK Learning

L'extension Microsoft Quantum Development Kit (QDK) pour Visual Studio Code (VS Code) inclut QDK Learning, un ensemble de ressources éducatives qui vous enseignent comment utiliser le QDK pour le développement quantique. QDK Learning contient les katas quantiques, une série de leçons qui vous aident à apprendre les principes fondamentaux de l'informatique quantique.

## Que sont les Quantum Katas ?

Kata est le mot japonais pour « form », un modèle d'apprentissage et de pratique de nouvelles compétences. Les katas quantiques sont des tutoriels d'apprentissage en autonomie qui enseignent les concepts de l'informatique quantique et la programmation quantique dans le langage de programmation Q#. Chaque leçon, ou kata, comprend la théorie de l'informatique quantique et des exercices de codage interactifs qui vous permettent d'appliquer vos connaissances.

Avec l'extension QDK dans VS Code, les Katas vous permettent de :

- Suivez et enregistrez votre progression.
- Enregistrez vos exercices sous forme de fichiers Q# que vous pouvez utiliser ultérieurement.
- Obtenez de l'aide du QDK Learning assistant Copilot.

## Prerequisites

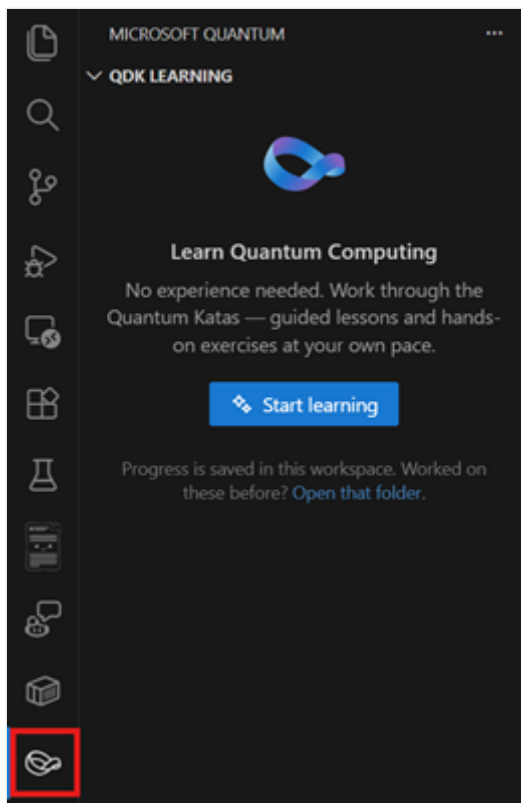
Pour bien démarrer avec QDK Learning et les katas quantiques, vous devez avoir installé les outils suivants.

- VS Code à partir du [site web VS Code](#) ou de l'application Microsoft Store.
- Extension [QDK](#) dans VS Code.

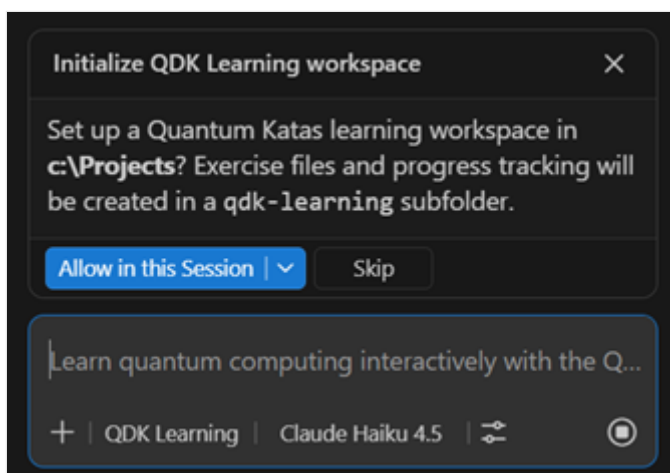
## Prise en main des Katas pour la première fois

Pour commencer à utiliser les Katas, procédez comme suit.

1. Dans VS Code, ouvrez le **Microsoft panneau Quantum** .



2. Dans la liste déroulante **QDK LEARNING** , sélectionnez le bouton **Démarrer l'apprentissage** . La fenêtre de conversation Copilot QDK Learning s'ouvre et Copilot vous invite à initialiser votre espace de travail VS Code.



3. Pour démarrer les Katas à partir du début, sélectionnez la première section, **Prise en main**. Un onglet **Leçon** s'ouvre pour la leçon **Bienvenue aux katas quantiques** .
4. Pour accéder à la leçon suivante, sélectionnez le bouton **Suivant** . La leçon suivante est un exercice de codage. Un nouvel onglet s'ouvre donc avec un `.qs` fichier Q#. Terminez la leçon, puis sélectionnez **Suivant** pour terminer le premier module.

Si vous souhaitez continuer à apprendre, poursuivez avec les leçons suivantes. QDK Learning enregistre automatiquement votre progression dans un fichier appelé `qdk-learning.json` , afin que vous puissiez revenir aux Katas à tout moment. Vous n'avez pas à faire les leçons dans

l'ordre, donc n'hésitez pas à ignorer les sujets que vous connaissez déjà ou à accéder à des sujets qui vous intéressent.

## Guide pratique pour parcourir les leçons

Les Katas ont deux types de leçons, de leçons théoriques et de leçons d'exercice. Les leçons théoriques vous enseignent les concepts et les leçons d'exercice vous permettent d'appliquer des concepts par le biais du code Q#.

Les leçons théoriques ont trois boutons :

- **Prochain:** Marquez la leçon actuelle comme terminée et passez à la leçon suivante.
- **Expliquer:** Invitez Copilot pour vous apprendre plus sur le concept.
- **Retour :** Accédez à la leçon précédente.

Les leçons d'exercice ont quatre boutons :

- **Vérifier:** Vérifiez si votre code est la solution correcte.
- **Indice:** Invitez Copilot pour vous aider à déterminer la solution appropriée.
- **Réinitialiser:** Réinitialisez votre code à l'état d'origine et réinitialisez votre progression pour les exercices terminés.
- **Retour :** Accédez à la leçon précédente.

Si votre solution est correcte, le bouton **Expliquer** devient le bouton **Suivant** . Si votre solution est incorrecte, une fenêtre contextuelle **Vérifier l'échec** s'ouvre avec plus d'informations. Sélectionnez **Qu'est-ce qui s'est passé ?** pour obtenir de l'aide de Copilot.

## Utiliser Copilot pour prendre en charge votre apprentissage

Lorsque vous ouvrez les katas dans le **Microsoft panneau Quantum**, le QDK définit Copilot être l'agent **QDK Learning**. Vous pouvez utiliser cet agent avec tous les modèles Copilot disponibles. Choisissez le modèle que vous souhaitez utiliser, puis demandez à l'agent QDK Learning sur l'informatique quantique, Q# ou les leçons Katas. Par exemple, essayez quelques-unes des instructions suivantes.

## Naviguer dans le contenu Katas

Copilot peut vous amener au contenu qui vous intéresse avec une invite comme celle-ci.

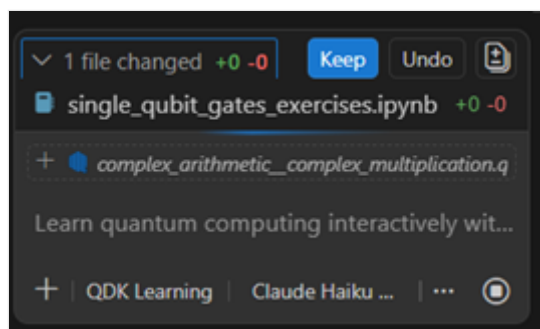
Je suis intéressé par l'apprentissage de l'algorithme de Grover. Accédez au contenu de Grover.

## Créer du contenu d'apprentissage

Copilot peut créer du nouveau contenu pour vous dans un notebook Jupyter à l'aide d'une instruction comme celle-ci.

Pouvez-vous créer de nouveaux exercices pour tester mes connaissances sur les portes quantiques à qubit de base ?

Lorsque vous demandez Copilot créer du contenu, une boîte de confirmation s'ouvre. Pour autoriser Copilot à créer le fichier et à l'enregistrer dans votre répertoire de travail actuel, sélectionnez le bouton **Conserver**.



Last updated on 30/06/2026



# Comment envoyer des travaux à Azure Quantum

Une Azure Quantum tâche exécute un programme quantique auprès d'un fournisseur target, tel qu'un ordinateur quantique ou un simulateur. Utilisez le Microsoft Quantum Development Kit (QDK) pour envoyer des travaux de manière interactive à partir Visual Studio Code (VS Code) ou par programmation avec Python.

## Avant d'envoyer un travail

Pour soumettre un travail, vous avez besoin des éléments suivants :

- Un compte Azure avec un abonnement actif.
- Un Azure Quantum espace de travail. Pour en créer un, consultez [Créer un Azure Quantum espace de travail](#).
- Un fournisseur et target dans votre espace de travail qui acceptent le format d'entrée et le profil de représentation intermédiaire quantique (Quantum Intermediate Representation, QIR) target de votre programme.

## Comparer les méthodes d'envoi de travaux

Choisissez une méthode en fonction de la façon dont vous développez votre programme et de la quantité de contrôle ou d'automatisation dont vous avez besoin.

 Agrandir le tableau

Méthode	Langues et entrées	Utilisez cette méthode quand	Envisagez une autre méthode lorsque
<a href="#">QDK extension pour VS Code</a> 	Q# et OpenQASM fichiers sources	Vous souhaitez disposer d'un flux de travail interactif dans VS Code, avec la saisie semi-automatique du code, la simulation locale, le débogage, la visualisation de circuits et la soumission de tâches dans une seule interface.	Vous utilisez une Python infrastructure quantique, vous devez générer des travaux par programme ou vous souhaitez automatiser les soumissions.
<a href="#">QDK Python package</a> 	Q#, OpenQASM, Cirq, Qiskit,	Vous travaillez dans un Python script ou un notebook Jupyter, souhaitez utiliser les API natives du	Il vous suffit de soumettre de façon interactive un fichier OpenQASM ou Q#, ou vous

Méthode	Langues et entrées	Utilisez cette méthode quand	Envisagez une autre méthode lorsque
	PennyLane, et compilé QIR	framework, ou devez générer, soumettre et traiter des tâches dans le code.	disposez déjà d'un fichier d'entrée compilé pour un workflow d'automatisation.

## Envoyer des travaux avec l'extension QDK pour VS Code

Utilisez l'extension QDK lorsque vous écrivez Q# ou OpenQASM directement dans VS Code et souhaitez un flux de travail visuel et interactif. L'extension peut :

- Exécutez et déboguez votre programme sur un simulateur local.
- Affichez des diagrammes de circuit et des histogrammes de résultat.
- Connectez-vous à un Azure Quantum espace de travail.
- Répertoire les fournisseurs, targetset les travaux envoyés.
- Soumettez le OpenQASM ou le Q# actif, puis téléchargez les résultats.

Cette méthode est bonne pour l'exploration et le développement interactif. Vous n'avez pas besoin d'écrire de code de connexion ou de soumission.

Pour obtenir des instructions, consultez [Envoyer des travaux avec l'extension QDK pour VS Code](#).

## Envoyer des travaux avec le QDKPython package

Utilisez le package QDKPython si votre flux de travail commence dans Python ou dans un notebook Jupyter. Le Python package fournit les éléments suivants :

- Module `qdk.azure` permettant de se connecter à un espace de travail et d'envoyer des travaux.
- Q# et OpenQASM compilation vers QIR.
- API de soumission de tâches pour Qiskit et Cirq.
- Chemin de soumission QIR pour les programmes PennyLane
- API pour inspecter l'état du travail et récupérer et visualiser les résultats.

Cette méthode est adaptée aux expériences paramétrables, aux soumissions répétées et à l'intégration d'applications.

Pour obtenir des instructions, consultez [Envoyer des travaux avec le QDKPython package](#).

## Utiliser des formats spécifiques au fournisseur

Les exigences relatives aux formats et paramètres d'entrée de travail peuvent varier selon le fournisseur et target peuvent avoir différents flux de travail d'envoi de travaux. Pour plus d'informations, consultez [Envoyer des circuits quantiques mis en forme à Azure Quantum](#).

Utilisez des formats spécifiques au fournisseur uniquement lorsque votre flux de travail ou target les nécessite.

## Que se passe-t-il après la soumission

Toutes les méthodes de soumission créent une Azure Quantum tâche. Le travail passe par des états tels que **l'attente**, **l'exécution** et un état final tel que **Réussite**, **Échec** ou **Annulation**. Vous pouvez surveiller et gérer tous les travaux à partir du VS CodePythonAzure CLIportail Azure.

Pour plus d'informations sur les propriétés et les résultats des tâches, consultez [Utiliser les Azure Quantum tâches](#).

## Contenu connexe

- [Se connecter à votre Azure Quantum espace de travail](#)
- [Azure Quantum fournisseur de calcul targets](#)
- [Profils d'informatique target quantique](#)
- [Soumettre des tâches à Azure Quantum avec Azure CLI](#)

---

Last updated on 29/08/2026

# Envoyer des tâches à Azure Quantum à l'aide de l'extension QDK pour VS Code

Utilisez l'extension Visual Studio Code (VS Code) pour Q# (OpenQASM) afin de soumettre les programmes Azure Quantum et QDK à Microsoft Quantum Development Kit. Vous pouvez tester votre programme localement, vous connecter à votre Azure Quantum espace de travail, sélectionner un fournisseur target, soumettre le programme actif et afficher ses résultats sans écrire de code de soumission.

Si vous utilisez Qiskit, Cirq, PennyLane ou un flux de travail OpenQASM ou Q# basé sur Python, consultez [Soumettre des travaux avec le package Python QDK](#).

## Prerequisites

- Un compte Azure avec un abonnement actif.
- Un Azure Quantum espace de travail. Pour en créer un, consultez [Créer un Azure Quantum espace de travail](#).
- La dernière version de [VS Code](#).
- Dernière version de [l'extensionQDK](#).

Pour les détails d'installation, consultez [Configurer Microsoft Quantum Development Kit](#).

### ⓘ Remarque

Les exemples de cet article envoient des travaux au [simulateur targets](#). L'option targets disponible dépend des fournisseurs que vous configurez dans votre espace de travail.

## Préparer et tester votre programme

Choisissez l'onglet pour la langue de votre fichier source.

Q#

1. Dans VS Code, sélectionnez **Fichier>nouveau fichier texte**, puis enregistrez le fichier sous `Main.qs`.

2. Ajoutez le programme suivant Q#, qui utilise le profil **BaseQIRtarget**.

Q#

```
@EntryPoint(Base)
operation Main() : Result {
 use q = Qubit();
 H(q);
 return MResetZ(q);
}
```

3. Sélectionnez **Exécuter** dans l'objectif de code au-dessus de l'opération **Main** pour tester le programme sur le simulateur local.

## Se connecter à votre Azure Quantum espace de travail

Vous pouvez vous connecter avec une chaîne de connexion d'espace de travail.

1. Dans VS Code, sélectionnez **Afficher > lapalette de commandes**.
2. Enter **QDK : se connecter à l'espace de travail Azure Quantum**.
3. Utilisez un chaîne de connexion pour [vous connecter à votre Azure Quantum espace de travail](#).
4. Vérifiez que votre espace de travail apparaît sous **Espaces de travail Quantum**.

### Important

Traitez une chaîne de connexion d'espace de travail comme un secret. Ne la stockez pas dans le contrôle de code source ou partagez-la en texte brut.

## Sélectionner un Azure Quantumtarget

1. Dans le panneau **Microsoft Quantum**, développez votre espace de travail.
2. Développez **Fournisseurs**.
3. Développez un fournisseur et consultez les targets disponibles dans votre espace de travail.
4. Sélectionnez un simulateur target qui accepte le format d'entrée et QIRtarget le profil de votre programme.

### Conseil

Testez sur un simulateur target avant d'utiliser un matériel target quantique payant. Avant votre envoi au matériel quantique, passez en revue la tarification du fournisseur et estimez le coût du travail lorsque le fournisseur prend en charge l'estimation des coûts.

## Soumettre votre programme

1. Dans VS Code, ouvrez le fichier OpenQASM ou Q# que vous souhaitez soumettre.
2. Dans le panneau **Microsoft Quantum**, recherchez celui target que vous avez sélectionné.
3. Sélectionnez l'icône de lecture à côté de target.
4. Entrez un nom qui identifie le travail.
5. Entrez le nombre d'exécutions (le nombre de fois où le programme doit être exécuté).
6. Appuyez sur **Entrée** pour envoyer le travail.

VS Code affiche une notification lorsque vous envoyez le travail. Le temps nécessaire pour effectuer la tâche dépend du target et de sa file d'attente.

## Surveiller le travail et afficher les résultats

1. Dans le panneau **Microsoft Quantum**, développez **Travaux**.
2. Recherchez la tâche que vous avez soumise. Survolez la tâche pour afficher des détails tels que son état et l'heure de soumission.
3. Une fois le travail réussi, sélectionnez l'icône d'histogramme en regard du travail pour afficher les résultats pris en charge sous forme d'histogramme. Pour afficher ou télécharger le résultat brut, sélectionnez l'icône de texte.

Les formats de résultats diffèrent selon le fournisseur et target. Si le résultat ne peut pas être affiché en tant qu'histogramme, affichez la sortie brute.

Pour en savoir plus sur les états des travaux, l'annulation et les données de sortie, consultez [Utiliser les Azure Quantum travaux](#).

## Résoudre les problèmes liés à la soumission d'un travail

Si vous rencontrez des problèmes avec la soumission d'un travail, reportez-vous aux instructions suivantes pour obtenir de l'aide.

### Le target n'apparaît pas

Confirmez que :

- Le fournisseur est configuré dans votre espace de travail.
- target est actuellement disponible.
- Vous vous êtes connecté au répertoire, à l'abonnement et à l'espace de travail prévus Azure.

Pour plus d'informations sur la gestion des fournisseurs, consultez [Ajouter ou supprimer un fournisseur dans un Azure Quantum espace de travail](#).

## Le programme ne se compile pas pour le target

Le programme doit utiliser un profil QIRtarget et des opérations pris en charge par target. Passez en revue l'erreur du compilateur, le profil du target programme et les target fonctionnalités. Pour plus d'informations, consultez [profils d'informatique target quantique](#).

## Le travail échoue après l'envoi

Ouvrez les détails du travail et passez en revue le message d'erreur du fournisseur. Les échecs de travail peuvent résulter d'entrées non prises en charge, de paramètres de travail non valides, de disponibilité du fournisseur ou de limites de quota. Pour connaître les résolutions courantes, consultez [Résoudre les problèmes Azure Quantum](#).

## Contenu connexe

- [Comment envoyer des travaux à Azure Quantum](#)
- [Envoyer des travaux avec le QDK package Python](#)
- [Soumettre des travaux à Azure Quantum avec Azure CLI](#)
- [Travailler avec des Azure Quantum jobs](#)
- [Azure Quantum fournisseur de calcul targets](#)
- [VS Code référence pour le QDK](#)

# Soumettre des tâches à Azure Quantum à l'aide du package QDKPython

Utilisez le Microsoft Quantum Development Kit package (QDK) Python pour envoyer Q#, OpenQASM, Qiskit, Cirq, et PennyLane les programmes à Azure Quantum.

Cet article fournit des exemples de soumission de travaux pour chaque langage ou infrastructure quantique pris en charge. Pour obtenir un flux de travail interactif Q# ou OpenQASM qui ne nécessite Python pas de code, consultez [Envoyer des travaux avec l' QDK extension pour VS Code](#).

## Prerequisites

- Un compte Azure avec un abonnement actif.
- Un Azure Quantum espace de travail. Pour en créer un, consultez [Créer un Azure Quantum espace de travail](#).
- Python 3.10 ou version ultérieure.
- Si vous utilisez un notebook Jupyter, un environnement Jupyter comme l'extension Jupyter pour [VS Code](#) et le package `ipykernel`.
- Installez [Azure CLI](#) avec l'extension `quantum`.

### Azure CLI

```
az extension add --upgrade --name quantum
```

- Installez la dernière version du `qdk` Python package avec les `azure` et `jupyter` extras.

### Bash

```
pip install --upgrade "qdk[azure,jupyter]"
```

Si vous souhaitez soumettre un programme Qiskit ou Cirq, installez les modules complémentaires `cirq` et `qiskit`.

### Bash



```
pip install --upgrade "qdk[qiskit,cirq]"
```

## Se connecter à votre Azure Quantum espace de travail

Avant de soumettre un travail, vous devez vous connecter à un Azure Quantum espace de travail. Le `qdk.azure` module fournit l'objet `workspace` pour se connecter aux Azure Quantum espaces de travail. Pour vous connecter à votre espace de travail, procédez comme suit.

### Se connecter avec Azure CLI

Si vous ne l'avez pas déjà fait, connectez-vous à Azure à partir d'un terminal avant d'ouvrir un bloc-notes Jupyter.

Azure CLI

```
az login
```

Si vous ne vous connectez pas à partir du terminal, vous devez vous authentifier chaque fois que vous vous connectez à un espace de travail Quantum via Python. Si votre compte a accès à plusieurs abonnements, définissez l'abonnement qui contient votre Azure Quantum espace de travail.

Azure CLI

```
az account set --subscription <subscription-id>
```

Réexécutez `az login` lorsque la session expire ou lorsque vous souhaitez utiliser un autre compte Azure.

### Obtenir l'ID de ressource de votre espace de travail

1. Connectez-vous au portail [Azure](#).
2. Accédez à l'espace de travail Quantum dans lequel vous souhaitez envoyer votre travail.
3. Dans la page **Vue d'ensemble**, recherchez et copiez l'**ID de ressource**.

### Se connecter à l'espace de travail

1. Pour vous connecter à votre espace de travail Quantum, créez un `workspace` objet avec l'ID de ressource que vous avez copié.

#### Python

```
from qdk.azure import Workspace

workspace = Workspace(resource_id="") # Add your resource ID
```

#### ⓘ Remarque

Y

2. Vérifiez la connexion et consultez le targets disponible dans l'espace de travail.

#### Python

```
for target in workspace.get_targets():
 print(target.name)
```

Pour obtenir d'autres options de connexion et d'authentification, consultez [Se connecter à votre Azure Quantum espace de travail](#).

#### ⓘ Remarque

Les exemples suivants envoient des travaux au [simulateur targets](#). Les targets disponibles dans votre espace de travail dépendent des fournisseurs que vous avez configurés.

## Préparer et soumettre votre programme

Choisissez l'onglet de votre langage ou infrastructure quantique.

#### Q#

Utilisez le module `qdk.qsharp` pour définir ou charger du code QIR, compiler un point d'entrée en QIR et soumettre le Azure Quantum à un targetQ#.

### Définir et compiler le Q# programme

1. Initialiser Q# avec un profil QIRtarget pris en charge par Azure Quantumtarget.

#### Python

```
from qdk import qsharp, TargetProfile
```

```
qsharp.init(target_profile=TargetProfile.Base)
```

Pour plus d'informations sur les QIRtarget profils, consultez [Azure QuantumQIRtarget les profils dans le QDK](#).

2. Définissez une Q# opération. Par exemple, utilisez l'opération suivante `RandomBit` .

Python

```
qsharp.eval("""
 operation RandomBit() : Result {
 use q = Qubit();
 H(q);
 return MResetZ(q);
 }
""")
```

3. Testez l'opération sur le simulateur local.

Python

```
print(qsharp.run("RandomBit()", shots=10))
```

4. Compilez l'opération sur QIR.

Python

```
from qdk.qsharp import compile

program = compile("RandomBit()")
```

## Soumettre la Q# tâche

1. Sélectionnez un(e) target compatible dans votre espace de travail.

Python

```
target = workspace.get_targets("rigetti.sim.qvm")
```

2. Soumettez le programme compilé.

Python

```
job = target.submit(program, "qsharp-job", shots=100)
print("Job ID:", job.id)
```

3. Attendez que la tâche soit terminée et récupérez ses résultats.

#### Python

```
job.wait_until_completed()
print("Status:", job.details.status)

results = job.get_results()
print(results)
```

## Contenu connexe

- [Comment envoyer des travaux à Azure Quantum](#)
- [Envoyer des travaux avec l'extension QDK pour VS Code](#)
- [Soumettre des travaux à Azure Quantum avec Azure CLI](#)
- [Travailler avec des Azure Quantum jobs](#)
- [Prise en charge du langage de programmation quantique dans QDK](#)
- [Azure Quantum fournisseur de calcul targets](#)

---

Last updated on 29/08/2026

# Azure Quantum QIR profils cibles dans le QDK

L'informatique quantique est encore une technologie émergente. Le matériel quantique ne peut pas exécuter tous les programmes quantiques. Par exemple, seul un matériel peut effectuer des mesures de circuit intermédiaire, ce qui est nécessaire pour exécuter des programmes avec des branches conditionnelles en fonction des résultats de mesure qubit.

Lorsque vous soumettez un programme à exécuter sur Azure Quantum, votre programme est converti au format de représentation intermédiaire quantique (QIR). QIR ne dépend pas du langage de programmation ou du type de matériel quantique sur lequel votre programme s'exécute. Le MicrosoftQuantum Development Kit (QDK) prend en charge plusieurs profils de cible QIR pour différentes capacités matérielles.

Pour plus d'informations sur QIR, consultez [Représentation intermédiaire Quantique](#).

## Vue d'ensemble des profils cibles

Azure Quantum et QDK prennent en charge plusieurs profils cibles QIR. Le type de profil cible que vous choisissez détermine les constructions de programmation suivantes que votre programme peut utiliser.

- Branches conditionnelles avec des instructions `if` basées sur les résultats des mesures des qubits
- Opérations arithmétiques sur les nombres à virgule flottante calculés à partir des résultats de mesure qubit
- Boucles fixes et non liées basées sur les résultats de mesure qubit

Le tableau suivant répertorie tous les QIR profils cibles dans le QDK, ainsi que les structures de programmation prises en charge par ces profils cibles, par ordre du plus restrictif au moins restrictif.

[🔍](#) Agrandir le tableau

QIR profil cible	les branches conditionnelles ;	Opérations en virgule flottante	Boucles
Base	✗	✗	✗
Adaptive RI	✓	✗	✗

QIR profil cible	les branches conditionnelles ;	Opérations en virgule flottante	Boucles
Adaptive RIF	✓	✓	✗
Adaptive	✓	✓	✓
Unrestricted	✓	✓	✓

## Définir des profils cibles dans le QDK

Pour exécuter un programme sur les simulateurs QDK ou une cible Azure Quantum, vous devez définir un QIR profil de cible. Si vous ne définissez pas manuellement un profil cible, il QDK tente de définir automatiquement le profil approprié pour la cible choisie.

lors de l'envoi de programmes à exécuter via le Azure Quantum service, il QDK tente de sélectionner automatiquement le profil approprié pour la cible d'exécution choisie.

### Base QIR profil

Le profil cible de base QIR est le profil cible le plus restrictif. Utilisez le profil de base pour des programmes plus simples qui n'utilisent pas de structures de programmation classiques telles que des branches et des boucles. Si votre cible matérielle quantique ne peut pas effectuer de mesures intermédiaires, vous devez probablement utiliser le profil de base.

Les cibles suivantes Azure Quantum peuvent exécuter des programmes qui utilisent le profil de base QIR .

[🔗](#) Agrandir le tableau

Provider	Simulateur	QPU
IonQ	<code>ionq.simulator</code>	<code>ionq.qpu.*</code>
Rigetti	<code>rigetti.sim.*</code>	<code>rigetti.qpu.*</code>

Pour en savoir plus sur ces Azure Quantum fournisseurs, consultez [le fournisseur IonQ](#) et [le fournisseur Rigetti](#).

### Définir le profil de base

Pour définir le profil cible de base QIR dans l'extension QDK pour Visual Studio Code (VS Code), choisissez l'une des options suivantes.

- Si vous configurez un projet Q#, ajoutez la commande suivante au fichier de `qsharp.json` votre projet.

#### JSON

```
{
 "targetProfile": "base"
}
```

- Si vous travaillez dans un `.qs` fichier qui ne fait pas partie d'un projet Q#, définissez le profil cible directement dans votre code Q#. Pour ce faire, ajoutez `@EntryPoint(Base)` sur la ligne précédant l'opération d'entrée dans votre programme.

#### Q#

```
@EntryPoint(Base)
operation Main() : Unit {

 ...

}
```

Pour définir le profil cible de base dans la QDK bibliothèque Python, exécutez le code suivant.

#### Python

```
from qdk import init, TargetProfile

init(target_profile=TargetProfile.Base)
```

## Limitations sur les programmes Q# avec profil de base

Le profil cible de base peut exécuter un large éventail de programmes Q#. La contrainte principale est que les programmes Q# ne peuvent pas effectuer de comparaisons logiques avec `Result` des valeurs de type à partir d'opérations de mesure.

Par exemple, vous ne pouvez pas exécuter l'opération suivante `FlipQubitOnZero` sur une cible de base, car le programme contient une `if` instruction qui utilise un résultat de mesure.

#### Q#

```
@EntryPoint(Base)
operation FlipQubitOnZero() : Unit {
 use q = Qubit();
 if M(q) == Zero {
```

```
 X(q);
 }
}
```

## Adaptive RI QIR profil

Le profil cible ri adaptatif peut exécuter un plus large éventail de programmes que le profil de base, mais présente toujours certaines limitations. Les cibles Adaptive RI prennent en charge les programmes qui utilisent des mesures en cours de circuit dans des instructions conditionnelles `if`. Si votre matériel quantique peut effectuer des mesures intermédiaires et que votre programme utilise des instructions `if` fondées sur les résultats des mesures, vous devez probablement utiliser le profil RI adaptatif.

Par exemple, le programme Q# suivant peut s'exécuter sur une cible RI adaptative.

```
Q#

@EntryPoint(Adaptive_RI)
operation MeasureQubit(q : Qubit) : Result {
 return M(q);
}

operation SetToZero(q : Qubit) : Unit {
 if MeasureQubit(q) == One { X(q); }
}
```

Les cibles suivantes Azure Quantum peuvent exécuter des programmes qui utilisent le profil de cible RI adaptatif.

[🔗 Agrandir le tableau](#)

Provider	Simulateur	QPU
Quantinuum	quantinuum.sim.h2-1e	quantinuum.qpu.h2-1
Quantinuum	quantinuum.sim.h2-2e	quantinuum.qpu.h2-2

Pour plus d'informations sur Quantinuum dans Azure Quantum, consultez [le fournisseur Quantinuum](#).

## Définir le profil RI adaptatif

Pour définir le profil cible ri QIR adaptatif dans l'extension QDK pour VS Code, choisissez l'une des options suivantes.



- Si vous configurez un projet Q#, ajoutez la commande suivante au fichier de `qsharp.json` votre projet :

#### JSON

```
{
 "targetProfile": "adaptive_ri"
}
```

- Si vous travaillez dans un `.qs` fichier qui ne fait pas partie d'un projet Q#, définissez le profil cible directement dans votre code Q#. Pour ce faire, ajoutez `@EntryPoint(Adaptive_RI)` sur la ligne précédant l'opération de point d'entrée dans votre programme.

#### Q#

```
@EntryPoint(Adaptive_RI)
operation Main() : Unit {

 ...

}
```

Pour définir le profil cible RI adaptatif dans la bibliothèque Python QDK, exécutez le code suivant.

#### Python

```
from qdk import init, TargetProfile

init(target_profile=TargetProfile.Adaptive_RI)
```

## Adaptive RIF QIR profil

Le profil cible RIF adaptatif a les mêmes fonctionnalités que le profil ri adaptatif, mais prend également en charge les programmes qui contiennent des opérations arithmétiques à virgule flottante. Si votre matériel quantique peut effectuer des mesures en cours de circuit et que votre programme utilise des instructions `if` ainsi que des nombres à virgule flottante calculés à partir des résultats des mesures, vous devez probablement utiliser le profil RIF adaptatif.

Par exemple, le programme Q# suivant peut s'exécuter sur une cible RIF adaptative.

#### Q#

```
@EntryPoint(Adaptive_RIF)
operation DynamicFloat() : Double {
```

```

use q = Qubit();
H(q);
mutable f = 0.0;
if M(q) == One {
 f = 0.5;
}
Reset(q);
return f;
}

```

Azure Quantum n'a pas de cibles RIF adaptatives pour l'instant, mais vous pouvez exécuter des programmes pour les cibles RIF adaptatives sur les simulateurs locaux QDK . Pour plus d'informations sur les simulateurs dans le QDK, consultez [Vue d'ensemble des simulateurs quantiques dans le QDK](#).

## Définir le profil RIF adaptatif

Pour définir le profil cible RIF QIR adaptatif dans l'extension QDK pour VS Code, choisissez l'une des options suivantes.

- Si vous configurez un projet Q#, ajoutez la commande suivante au fichier de `qsharp.json` votre projet :

### JSON

```

{
 "targetProfile": "adaptive_rif"
}

```

- Si vous travaillez dans un `.qs` fichier qui ne fait pas partie d'un projet Q#, définissez le profil cible directement dans votre code Q#. Pour ce faire, ajoutez `@EntryPoint(Adaptive_RIF)` sur la ligne avant l'opération du point d'entrée dans votre programme.

### Q#

```

@EntryPoint(Adaptive_RIF)
operation Main() : Unit {

 ...

}

```

Pour définir le profil cible RIF adaptatif dans la QDK bibliothèque Python, exécutez le code suivant.

## Python

```
from qdk import init, TargetProfile

init(target_profile=TargetProfile.Adaptive_RIF)
```

## Adaptive QIR profil

Le profil cible adaptatif a les mêmes fonctionnalités que le profil RIF adaptatif, mais prend également en charge les programmes qui utilisent des boucles basées sur les résultats de mesure. Adaptive les programmes peuvent utiliser des boucles avec un nombre défini d'itérations, ainsi que des boucles « répéter jusqu'au succès » (RUS).

Azure Quantum n'a pas de cibles adaptatives pour l'instant, mais vous pouvez exécuter des programmes pour des cibles adaptatives sur les simulateurs locaux QDK . Pour plus d'informations sur les simulateurs dans le QDK, consultez [Vue d'ensemble des simulateurs quantiques dans le QDK](#).

## Définir le profil adaptatif

Pour définir le profil cible adaptatif QIR dans l'extension QDK pour VS Code, choisissez l'une des options suivantes.

- Si vous configurez un projet Q#, ajoutez la commande suivante au fichier de `qsharp.json` votre projet :

### JSON

```
{
 "targetProfile": "adaptive"
}
```

- Si vous travaillez dans un `.qs` fichier qui ne fait pas partie d'un projet Q#, définissez le profil cible directement dans votre code Q#. Pour ce faire, ajoutez `@EntryPoint(Adaptive)` à la ligne précédant l'opération de point d'entrée de votre programme.

### Q#

```
@EntryPoint(Adaptive)
operation Main() : Unit {

 ...

}
```

Pour définir le profil cible adaptatif dans la QDK bibliothèque Python, exécutez le code suivant.

#### Python

```
from qdk import init, TargetProfile

init(target_profile=TargetProfile.Adaptive)
```

## Unrestricted QIR profil

Le profil cible illimité peut exécuter tous les programmes quantiques. Aucune cible quantique actuelle ne prend en charge le mode non restreint QIR, mais vous pouvez utiliser le profil non restreint pour exécuter des programmes complexes sur les simulateurs QDK pour le développement quantique.

## Définir le profil illimité

Pour définir le profil cible illimité QIR dans l'extension QDK pour VS Code, choisissez l'une des options suivantes.

- Si vous configurez un projet Q#, ajoutez la commande suivante au fichier de `qsharp.json` votre projet :

#### JSON

```
{
 "targetProfile": "unrestricted"
}
```

- Si vous travaillez dans un `.qs` fichier qui ne fait pas partie d'un projet Q#, définissez le profil cible directement dans votre code Q#. Pour ce faire, ajoutez `@EntryPoint(Unrestricted)` sur la ligne précédant l'opération de point d'entrée dans votre programme.

#### Q#

```
@EntryPoint(Unrestricted)
operation Main() : Unit {

 ...

}
```

Pour définir le profil cible illimité dans la QDK bibliothèque Python, exécutez le code suivant.

## Python

```
from qdk import init, TargetProfile

init(target_profile=TargetProfile.Unrestricted)
```

---

Last updated on 02/07/2026

# Se connecter à votre espace de travail Azure Quantum avec le QDK ou Azure CLI

Si vous disposez d'un espace de travail Azure Quantum, vous pouvez vous connecter à votre espace de travail et envoyer votre code avec le `qdk.azure` module Python. Le `qdk.azure` module fournit une [Workspace classe](#) qui représente un espace de travail Azure Quantum.

## Prérequis

Pour vous connecter à votre espace de travail avec le `qdk.azure` module, vous avez besoin des éléments suivants :

- Compte Azure avec un abonnement actif. Si vous n'avez pas de compte Azure, inscrivez-vous gratuitement et inscrivez-vous à un abonnement [pay-as-you-go](#) <sup>↗</sup>.
- Un espace de travail Azure Quantum. Si vous n'avez pas d'espace de travail, consultez [Créer un espace de travail Azure Quantum](#).
- La dernière version du `qdk` paquet Python avec l'option supplémentaire `azure`.

Bash

```
pip install --upgrade "qdk[azure]"
```

- Si vous utilisez Azure CLI, mettez à jour vers la dernière version. Pour obtenir des instructions d'installation, consultez :
- [Installer Azure CLI sur Windows](#)
- [Installer Azure CLI sur Linux](#)
- [Installer Azure CLI sur macOS](#)

## Se connecter avec une chaîne de connexion

Utilisez un chaîne de connexion pour spécifier les paramètres de connexion à un espace de travail Azure Quantum. Les chaînes de connexion sont utiles dans les scénarios suivants :

- Vous souhaitez partager l'accès à votre espace de travail avec d'autres personnes qui n'ont pas de compte Azure.
- Vous souhaitez partager l'accès à votre espace de travail avec d'autres personnes pendant une durée limitée.
- Vous ne pouvez pas utiliser l'ID Microsoft Entra en raison de stratégies d'entreprise.

#### Conseil

Chaque espace de travail Azure Quantum a une clé primaire et une clé secondaire, et chaque clé a sa propre chaîne de connexion. Pour permettre aux autres utilisateurs d'accéder à votre espace de travail, partagez la clé secondaire et utilisez la clé primaire uniquement pour vos propres services. Vous pouvez remplacer la clé secondaire sans provoquer de temps d'arrêt dans vos propres services. Pour plus d'informations sur le partage d'accès à votre espace de travail, consultez [Partager l'accès à votre espace de travail](#).

## Copier la chaîne de connexion

1. Connectez-vous au portail [Azure](#).
2. Accédez à votre espace de travail Azure Quantum.
3. Dans le panneau de l'espace de travail, développez la liste déroulante **Opérations** et sélectionnez **Clés d'accès**.
4. Activez les clés d'accès pour votre espace de travail. Si le curseur **Touches d'accès** est défini sur **Désactivé**, définissez le curseur sur **Activé**.
5. Sélectionnez l'icône **Copier** pour l'chaîne de connexion que vous souhaitez copier. Vous pouvez choisir la chaîne de connexion primaire ou secondaire.

#### Avertissement

Il s'agit d'un risque de sécurité pour stocker vos clés d'accès de compte ou les chaînes de connexion en texte clair. Stockez vos clés de compte dans un format chiffré ou migrez vos applications pour utiliser l'autorisation Microsoft Entra pour accéder à votre espace de travail Azure Quantum.

## Utilisez le chaîne de connexion pour accéder à votre espace de travail Azure Quantum

Utilisez le chaîne de connexion que vous avez copié pour vous connecter à votre espace de travail Azure Quantum avec le `qdk.azure` module ou l'extension QDK dans Visual Studio Code.

#### Python

Pour vous connecter à votre espace de travail Azure Quantum, choisissez l'une des méthodes suivantes.

- Utilisez la `from_connection_string` fonction pour créer un `Workspace` objet.

#### Python

```
from qdk.azure import Workspace

connection_string = "" # Add your connection string
workspace = Workspace.from_connection_string(connection_string)

print(workspace.get_targets())
```

- Stockez votre chaîne de connexion dans une variable d'environnement.

#### Python

```
import os
from qdk.azure import Workspace

connection_string = "" # Add your connection string
os.environ["AZURE_QUANTUM_CONNECTION_STRING"] = connection_string

workspace = Workspace()
print(workspace.get_targets())
```

Pour plus d'informations sur l'utilisation des clés, consultez [Gérer vos clés d'accès](#).

#### ❗ Important

Si vous désactivez les clés d'accès pour votre espace de travail, vous ne pouvez pas utiliser de chaînes de connexion pour vous connecter à votre espace de travail. Utilisez plutôt des paramètres d'espace de travail pour vous connecter.

## Se connecter à votre espace de travail avec des paramètres d'espace de travail



Chaque espace de travail Azure Quantum a un ensemble unique de paramètres que vous pouvez utiliser pour vous connecter à l'espace de travail.

 Agrandir le tableau

Paramètre	Description	Utiliser quand vous vous connectez avec
<code>subscription_id</code>	ID de l'abonnement Azure.	Azure CLI
<code>resource_group</code>	Le nom du groupe de ressources Azure.	Azure CLI
<code>name</code>	Nom de votre espace de travail Azure Quantum.	Azure CLI
<code>resource_id</code>	ID de ressource Azure de l'espace de travail Azure Quantum.	Python

Pour rechercher les paramètres de votre espace de travail, procédez comme suit :

1. Connectez-vous au portail [Azure](#).
2. Accédez à votre espace de travail Azure Quantum.
3. Dans le panneau de votre espace de travail, choisissez **Vue d'ensemble**.
4. Développez la liste déroulante **Essentials**.
5. Copiez les paramètres dans leurs champs correspondants.

#### Remarque

Vérifiez que vous vous connectez au locataire approprié avant de vous connecter à votre espace de travail. Pour plus d'informations sur les locataires, consultez [Comment gérer les abonnements Azure avec Azure CLI](#).

## Utiliser des paramètres d'espace de travail pour vous connecter à votre espace de travail Azure Quantum

Pour rester connecté lorsque vous exécutez vos scripts Python, ouvrez un terminal et exécutez la commande Azure CLI suivante. Cette commande définit l'abonnement pour votre espace de travail. Remplacez `<subscriptionId>` par votre ID d'abonnement.

#### Azure CLI

```
az account set --subscription <subscriptionId>
```

Vous pouvez utiliser les paramètres de votre espace de travail pour vous connecter à votre espace de travail Azure Quantum via le `qdk.azure` module ou via Azure CLI.

## Python

Pour vous connecter à votre espace de travail Azure Quantum, choisissez l'une des options suivantes.

- Spécifiez l'ID de ressource (recommandé) :

### Python

```
from qdk.azure import Workspace

workspace = Workspace(resource_id="") # Add the resource ID of your
workspace
```

- Spécifiez l'ID d'abonnement, le groupe de ressources et le nom de l'espace de travail :

### Python

```
from qdk.azure import Workspace

workspace = Workspace(
 subscription_id="", # Add the subscription ID of your workspace
 resource_group="", # Add the resource group of your workspace
 name="" # Add the name of your workspace
)
```

- Spécifiez uniquement le nom de l'espace de travail. Cette option peut échouer lorsque vous avez plusieurs espaces de travail portant le même nom dans le même locataire.

### Python

```
from qdk.azure import Workspace

workspace = Workspace(name="") # Add the name of your workspace
```

## Contenu connexe

- [Gérer des espaces de travail quantiques avec Azure CLI](#)
- [Partager l'accès à votre espace de travail Azure Quantum](#)
- [Gérer des espaces de travail quantiques avec Azure Resource Manager](#)

- [S'authentifier à l'aide d'un principal de service](#)
  - [S'authentifier à l'aide d'une identité managée](#)
- 

Last updated on 21/08/2026

# Comment envoyer des circuits mis en forme spécifiques à Azure Quantum

Découvrez comment utiliser le module `qdk.azure` Python pour envoyer des circuits dans des formats spécifiques au service Azure Quantum. Cet article vous montre comment envoyer des circuits dans les formats suivants :

- [Envoyer des circuits QIR](#)
- [Envoyer des circuits spécifiques au fournisseur](#)

Pour plus d'informations, consultez la page [Circuits quantiques](#).

## Prérequis

Pour développer et exécuter vos circuits dans Visual Studio Code (VS Code), vous devez disposer des éléments suivants :

- Un compte Azure avec un abonnement actif. Si vous n'avez pas de compte Azure, inscrivez-vous gratuitement et inscrivez-vous à un abonnement [pay-as-you-go](#) [↗](#).
- Espace de travail Azure Quantum. Pour plus d'informations, consultez [Créer un espace de travail Azure Quantum](#).
- Environnement Python avec [Python et Pip](#) [↗](#) installés.
- VS Code avec les extensions [Microsoft Quantum Development Kit \(QDK\)](#) [↗](#), [Python](#) [↗](#) et [Jupyter](#) [↗](#) installées.
- Bibliothèque `qdk` Python avec le supplément `azure` et le paquet `ipykernel`.

Bash

```
python -m pip install --upgrade "qdk[azure]" ipykernel
```

## Créez un notebook Jupyter et connectez-vous à votre espace de travail Quantum

Pour vous connecter à votre espace de travail dans un notebook Jupyter dans VS Code, procédez comme suit :

1. Dans VS Code, ouvrez le menu **Affichage** et choisissez **PaLETTE de commandes**.
2. Entrez **Créer : Nouveau cahier Jupyter**. Un fichier Jupyter Notebook vide s'ouvre dans un nouvel onglet.
3. Dans la première cellule du notebook, exécutez le code suivant. Vous trouverez l'ID de ressource dans le volet **Overview** de votre espace de travail dans le portail Azure.

#### Python

```
from qdk.azure import Workspace

workspace = Workspace (resource_id="") # Add your resource ID
```

## Soumettre des circuits au format QIR

La représentation intermédiaire quantique (QIR) est une représentation intermédiaire qui sert d'interface commune entre les langages de programmation quantique et les plateformes de calcul quantique ciblées. Pour plus d'informations, consultez [Quantum Intermediate Representation](#).

Pour soumettre un circuit au format QIR, procédez comme suit :

1. Créez le circuit QIR. Par exemple, lancez le code suivant dans une nouvelle cellule pour créer un circuit d'enchevêtrement simple.

#### Python

```
QIR_routine = """%Result = type opaque
%Qubit = type opaque

define void @ENTRYPOINT__main() #0 {
 call void @__quantum__qis__h__body(%Qubit* inttoptr (i64 0 to %Qubit*))
 call void @__quantum__qis__cx__body(%Qubit* inttoptr (i64 0 to %Qubit*),
%Qubit* inttoptr (i64 1 to %Qubit*))
 call void @__quantum__qis__h__body(%Qubit* inttoptr (i64 2 to %Qubit*))
 call void @__quantum__qis__cz__body(%Qubit* inttoptr (i64 2 to %Qubit*),
%Qubit* inttoptr (i64 0 to %Qubit*))
 call void @__quantum__qis__h__body(%Qubit* inttoptr (i64 2 to %Qubit*))
 call void @__quantum__qis__h__body(%Qubit* inttoptr (i64 3 to %Qubit*))
 call void @__quantum__qis__cz__body(%Qubit* inttoptr (i64 3 to %Qubit*),
%Qubit* inttoptr (i64 1 to %Qubit*))
 call void @__quantum__qis__h__body(%Qubit* inttoptr (i64 3 to %Qubit*))
 call void @__quantum__qis__mz__body(%Qubit* inttoptr (i64 2 to %Qubit*),
%Result* inttoptr (i64 0 to %Result*)) #1
 call void @__quantum__qis__mz__body(%Qubit* inttoptr (i64 3 to %Qubit*),
%Result* inttoptr (i64 1 to %Result*)) #1
```

```

 call void @_quantum_rt_tuple_record_output(i64 2, i8* null)
 call void @_quantum_rt_result_record_output(%Result* inttoptr (i64 0 to
%Result*), i8* null)
 call void @_quantum_rt_result_record_output(%Result* inttoptr (i64 1 to
%Result*), i8* null)
 ret void
}

declare void @_quantum_qis_ccx_body(%Qubit*, %Qubit*, %Qubit*)
declare void @_quantum_qis_cx_body(%Qubit*, %Qubit*)
declare void @_quantum_qis_cy_body(%Qubit*, %Qubit*)
declare void @_quantum_qis_cz_body(%Qubit*, %Qubit*)
declare void @_quantum_qis_rx_body(double, %Qubit*)
declare void @_quantum_qis_rxx_body(double, %Qubit*, %Qubit*)
declare void @_quantum_qis_ry_body(double, %Qubit*)
declare void @_quantum_qis_ryy_body(double, %Qubit*, %Qubit*)
declare void @_quantum_qis_rz_body(double, %Qubit*)
declare void @_quantum_qis_rzz_body(double, %Qubit*, %Qubit*)
declare void @_quantum_qis_h_body(%Qubit*)
declare void @_quantum_qis_s_body(%Qubit*)
declare void @_quantum_qis_s_adj(%Qubit*)
declare void @_quantum_qis_t_body(%Qubit*)
declare void @_quantum_qis_t_adj(%Qubit*)
declare void @_quantum_qis_x_body(%Qubit*)
declare void @_quantum_qis_y_body(%Qubit*)
declare void @_quantum_qis_z_body(%Qubit*)
declare void @_quantum_qis_swap_body(%Qubit*, %Qubit*)
declare void @_quantum_qis_mz_body(%Qubit*, %Result* writeonly) #1
declare void @_quantum_rt_result_record_output(%Result*, i8*)
declare void @_quantum_rt_array_record_output(i64, i8*)
declare void @_quantum_rt_tuple_record_output(i64, i8*)

attributes #0 = { "entry_point" "output_labeling_schema"
"qir_profiles"="base_profile" "required_num_qubits"="4"
"required_num_results"="2" }
attributes #1 = { "irreversible" }

; module flags

!llvm.module.flags = !{!0, !1, !2, !3}

!0 = !{i32 1, !"qir_major_version", i32 1}
!1 = !{i32 7, !"qir_minor_version", i32 0}
!2 = !{i32 1, !"dynamic_qubit_management", i1 false}
!3 = !{i32 1, !"dynamic_result_management", i1 false}
""

```

2. Créez une `submit_qir_job` fonction d'assistance pour envoyer le circuit QIR à un target.

Dans cet exemple, les formats de données d'entrée et de sortie sont `qir.v1` et

`microsoft.quantum-results.v1`, respectivement.

```
Submit the job with proper input and output data formats
def submit_qir_job(target, input, name, count=100):
 job = target.submit(
 input_data=input,
 input_data_format="qir.v1",
 output_data_format="microsoft.quantum-results.v1",
 name=name,
 input_params = {
 "entryPoint": "ENTRYPOINT__main",
 "arguments": [],
 "count": count
 }
)

 print(f"Queued job: {job.id}")
 job.wait_until_completed()
 print(f"Job completed with state: {job.details.status}")
 #if job.details.status == "Succeeded":
 result = job.get_results()

 return result
```

3. Envoyez le circuit QIR à un Azure Quantum target spécifique. Par exemple, pour soumettre le circuit QIR au simulateur targetIonQ, exécutez le code suivant :

#### Python

```
target = workspace.get_targets(name="ionq.simulator")
result = submit_qir_job(target, QIR_routine, "QIR routine")
result
```

## Envoyer un circuit avec un format spécifique au fournisseur pour Azure Quantum

Chaque fournisseur Azure Quantum a son propre format pour représenter des circuits quantiques. Vous pouvez envoyer des circuits à Azure Quantum dans des formats spécifiques au fournisseur au lieu de langues QIR, telles que Q# ou Qiskit.

- [IonQ](#)
- [Pasqal](#)
- [Quantinuum](#)
- [Rigetti](#)

## Envoyer un circuit à IonQ au format JSON

IonQ utilise le format JSON pour exécuter des circuits sur leur targets. Pour plus d'informations, consultez la documentation de l'API [IonQ targets](#) et [IonQ](#).

L'exemple suivant crée une superposition entre trois qubits au format JSON.

1. Dans une nouvelle cellule, créez un circuit quantique au format JSON.

Python

```
circuit = {
 "qubits": 3,
 "circuit": [
 {
 "gate": "h",
 "target": 0
 },
 {
 "gate": "cnot",
 "control": 0,
 "target": 1
 },
 {
 "gate": "cnot",
 "control": 0,
 "target": 2
 },
]
}
```

2. Envoyez le circuit à l'IonQ target. L'exemple suivant utilise le simulateur IonQ qui renvoie un objet `Job`.

Python

```
target = workspace.get_targets(name="ionq.simulator")
job = target.submit(circuit)
```

3. Une fois la tâche terminée, obtenez les résultats.

Python

```
results = job.get_results()
print(results)
```

## Envoyer un circuit à Pasqal au format SDK Pulser



Vous pouvez utiliser le SDK Pulser pour créer des séquences d'impulsions et les soumettre à Pasqal targets.

## Installer le Kit de développement logiciel (SDK) Pulser

[Pulser](#) est un framework qui vous permet de créer, simuler et exécuter des séquences d'impulsions pour les appareils quantiques neutres à atomes. Pulser est conçu par PASQAL comme pass-through pour soumettre des expériences quantiques à leurs processeurs quantiques. Pour plus d'informations, consultez la [documentation Pulser](#).

Pour soumettre les séquences d'impulsions, installez d'abord les packages du Kit de développement logiciel (SDK) Pulser :

### Python

```
try:
 import pulser
 import pulser_pasqal
except ImportError:
 !pip -q install pulser pulser-pasqal --index-url https://pypi.org/simple
```

## Créer un registre quantique

Définissez à la fois un registre et une disposition. Le registre spécifie où organiser les atomes, et la disposition spécifie les positions des pièges qui capturent et structurent les atomes dans le registre.

Pour plus d'informations sur les dispositions, consultez la [documentation Pulser](#).

Créez un `devices` objet pour importer l'ordinateur targetquantique Pasqal, [FRESNEL\\_CAN1](#).

### Python

```
from pulser_pasqal import PasqalCloud

devices = PasqalCloud().fetch_available_devices()
QPU = devices["FRESNEL_CAN1"]
```

## Configurer votre mise en page

Utilisez une disposition personnalisée lorsque les dispositions pré-étalonnées ne répondent pas aux exigences de votre expérience.

Pour un registre arbitraire donné, un QPU neutre-atome place des pièges en fonction de la disposition, qui doit ensuite être étalonné. Étant donné que chaque étalonnage prend du temps, il est recommandé de réutiliser une disposition étalonnée existante si possible.

Pour créer une disposition arbitraire, choisissez l'une des options suivantes :

- Générez automatiquement une disposition basée sur un registre spécifié. Pour les grands registres, ce processus peut produire des solutions sous-optimales. Par exemple :

#### Python

```
from pulser import Register
qubits = {
 "q0": (0, 0),
 "q1": (0, 10),
 "q2": (8, 2),
 "q3": (1, 15),
 "q4": (-10, -3),
 "q5": (-8, 5),
}

reg = Register(qubits).with_automatic_layout(device)
```

- Pour définir manuellement une disposition pour créer votre registre, consultez la [documentation Pulser](#).

## Écrire une séquence d'impulsions

Les atomes neutres sont contrôlés avec des impulsions laser. Le SDK Pulser vous permet de créer des séquences d'impulsions à appliquer au registre quantique.

1. Définissez les attributs de séquence d'impulsions en déclarant les canaux qui contrôlent les atomes. Pour créer un `Sequence`, fournissez une `Register` instance avec l'appareil sur lequel la séquence sera exécutée. Par exemple, le code suivant déclare un canal : `ch0`.

#### Python

```
from pulser import Sequence

seq = Sequence(reg, QPU)

Print the available channels for your sequence
print(seq.available_channels)

Declare a channel. For example, `rydberg_global`
seq.declare_channel("ch0", "rydberg_global")
```

### ❗ Remarque

Vous pouvez utiliser l'appareil `QPU = devices["FRESNEL_CAN1"]` ou importer un appareil virtuel à partir de Pulser pour plus de flexibilité. L'utilisation d'une `VirtualDevice` fonctionnalité permet la création de séquences moins contrainte par les spécifications de l'appareil, ce qui vous permet d'exécuter sur un émulateur. Pour plus d'informations, consultez [la documentation](#)  Pulser.

2. Ajoutez des impulsions à votre séquence. Pour ce faire, créez et ajoutez des impulsions aux canaux que vous avez déclarés. Par exemple, le code suivant crée une impulsion et l'ajoute au canal `ch0`:

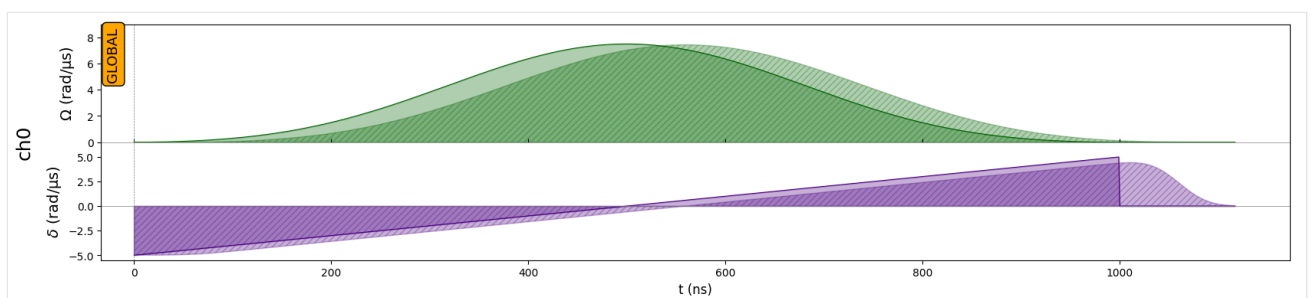
#### Python

```
from pulser import Pulse
from pulser.waveforms import RampWaveform, BlackmanWaveform
import numpy as np

amp_wf = BlackmanWaveform(1000, np.pi)
det_wf = RampWaveform(1000, -5, 5)
pulse = Pulse(amp_wf, det_wf, 0)
seq.add(pulse, "ch0")

seq.draw()
```

L'image suivante montre la séquence d'impulsions :



## Convertir la séquence en chaîne JSON

Pour soumettre les séquences d'impulsions, convertissez les objets Pulser en une chaîne JSON qui peut être utilisée comme données d'entrée.

#### Python

```
import json
```

```
Convert the sequence to a JSON string
def prepare_input_data(seq):
 input_data = {}
 input_data["sequence_builder"] = json.loads(seq.to_abstract_repr())
 to_send = json.dumps(input_data)
 return to_send
```

## Envoyer la séquence d'impulsions à Pasqal target

1. Définissez les formats de données d'entrée et de sortie appropriés. Par exemple, le code suivant définit le format de données d'entrée sur `pasqal.pulser.v1` et le format `pasqal.pulser-results.v1` de données de sortie sur .

### Python

```
Submit the job with proper input and output data formats
def submit_job(target, seq, shots):
 job = target.submit(
 input_data=prepare_input_data(seq), # Take the JSON string previously
 defined as input data
 input_data_format="pasqal.pulser.v1",
 output_data_format="pasqal.pulser-results.v1",
 name="Pasqal sequence",
 shots=shots # Number of shots
)

 print(f"Queued job: {job.id}")
 return job
```

### ⚠ Remarque

Le temps nécessaire pour exécuter un travail sur le QPU dépend des heures de file d'attente actuelles. Vous pouvez afficher le temps moyen de file d'attente d'un target dans le volet **Fournisseurs** de votre espace de travail.

2. Envoyez le programme à Pasqal. Avant de soumettre votre code à du matériel quantique réel, il est recommandé de tester votre code sur l'émulateur `pasqal.sim.emu-mps` target.

### Python

```
target = workspace.get_targets(name="pasqal.sim.emu-mps") # Change to
"pasqal.qpu.fresnel-can1" to use FRESNEL_CAN1 QPU
job = submit_job(target, seq, 10)

job.wait_until_completed()
print(f"Job completed with state: {job.details.status}")
```

```
result = job.get_results()
print(result)
```

#### Sortie

```
{
 "1000000": 3,
 "0010000": 1,
 "0010101": 1
}
```

## Envoyer un circuit OpenQASM à Quantinuum

1. Créez un circuit quantique dans la représentation [OpenQASM](#). Par exemple, le code suivant crée un circuit de téléportation :

py

```
circuit = """OPENQASM 2.0;
include "qelib1.inc";
qreg q[3];
creg c0[3];
h q[0];
cx q[0], q[1];
cx q[1], q[2];
measure q[0] -> c0[0];
measure q[1] -> c0[1];
measure q[2] -> c0[2];
"""
```

Vous pouvez également charger le circuit à partir d'un fichier OpenQASM :

py

```
with open("my_teleport.qasm", "r") as f:
 circuit = f.read()
```

2. Soumettez le circuit à Quantinuum target. L'exemple suivant soumet la tâche à l'un des simulateurs Quantinuum targets.

Python

```
target = workspace.get_targets(name="quantinuum.sim.h2-1sc")
job = target.submit(circuit, shots=500)
```

3. Attendez que le travail soit terminé, puis extrayez les résultats.

#### Python

```
results = job.get_results()
print(results)
```

#### ❗ Remarque

Ces résultats retournent 000 pour chaque coup, ce qui n'est pas aléatoire. Cela est dû au fait que le validateur d'API vérifie uniquement si votre code peut s'exécuter sur le matériel Quantinuum, mais retourne 0 pour chaque mesure quantique. Pour un générateur de vrais nombres aléatoires, vous devez exécuter votre circuit sur du matériel quantique.

## Soumettre un circuit Quil à Rigetti

Pour soumettre un travail Quil à un Rigetti target, utilisez le module `qdk.azure` Python.

1. Chargez les importations requises.

#### Python

```
from azure.quantum import Workspace
from azure.quantum.target.rigetti import Result, Rigetti, RigettiTarget,
InputParams
```

2. Créez un `target` objet et passez le nom du Rigetti target auquel vous souhaitez soumettre votre travail. Par exemple, le code suivant sélectionne le `QVM` target.

#### Python

```
target = Rigetti(workspace=workspace, name=RigettiTarget.QVM)
```

3. Créez un programme Quil. Pour que votre programme soit accepté, vous devez configurer la lecture sur `"ro"`.

#### Python

```
readout = "ro"
bell_state_quil = f"""
DECLARE {readout} BIT[2]

H 0
CNOT 0 1

MEASURE 0 {readout}[0]
```

```
MEASURE 1 {readout}[1]
"""

num_shots = 5
job = target.submit(
 input_data=bell_state_quil,
 name="bell state",
 shots=100,
 input_params=InputParams(skip_quilc=False)
)

print(f"Job completed with state: {job.details.status}")
result = Result(job) # This throws an exception if the job failed
```

4. Vous pouvez indexer un résultat avec le nom de la valeur lue. Dans le code suivant, `data_per_shot` est une liste de longueur `num_shots`, et chaque élément de la liste est une autre liste qui contient les données du registre de cette capture.

#### Python

```
data_per_shot = result[readout]

ro_data_first_shot = data_per_shot[0]
```

Dans ce cas, étant donné que le type du registre est BIT, le type est entier et la valeur 0 ou 1.

#### Python

```
assert isinstance(ro_data_first_shot[0], int)
assert ro_data_first_shot[0] == 1 or ro_data_first_shot[0] == 0
```

5. Imprimez toutes les données.

#### Python

```
print(f"Data from '{readout}' register:")
for i, shot in enumerate(data_per_shot):
 print(f"Shot {i}: {shot}")
```

### Important

Vous ne pouvez pas envoyer plusieurs circuits sur un seul travail. Pour contourner ce problème, vous pouvez appeler la `backend.run` méthode pour envoyer chaque circuit de manière asynchrone, puis extraire les résultats de chaque travail. Par exemple :

## Python

```
jobs = []
for circuit in circuits:
 jobs.append(backend.run(circuit, shots=N))

results = []
for job in jobs:
 results.append(job.result())
```

## Contenu connexe

- [Soumettre un circuit avec Qiskit à Azure Quantum.](#)
- [Submitez un circuit avec Cirq pour Azure Quantum.](#)

---

Last updated on 06/05/2026



# Qu'est-ce que l'informatique quantique hybride ?

L'informatique quantique hybride fait référence aux processus et à l'architecture d'un ordinateur classique et d'un ordinateur quantique travaillant ensemble pour résoudre un problème. Avec la dernière génération d'architecture de calcul quantique hybride disponible dans Azure Quantum, vous pouvez commencer à programmer des ordinateurs quantiques en combinant des instructions classiques et quantiques.

Azure Quantum incarne une vision prospective de l'informatique quantique hybride, où certaines architectures sont déjà opérationnelles, tandis que d'autres sont activement développées. Cet article décrit les différentes approches de l'informatique quantique hybride et comment elles peuvent être utilisées pour optimiser certains problèmes.

## Regroupement de circuits avec calcul quantique par lots

L'informatique quantique batch vous permet d'envoyer plusieurs circuits quantiques en tant que tâche unique au matériel quantique.

En règle générale, les circuits quantiques sont envoyés un à la fois en tant que travaux uniques à une cible matérielle quantique. Lorsque le client reçoit le résultat d'un circuit, le circuit suivant est ajouté en tant que nouveau travail à la file d'attente. Le traitement par lots de plusieurs circuits en un seul travail élimine toutefois l'attente entre les soumissions de travaux, ce qui vous permet d'exécuter plusieurs travaux plus rapidement. Les exemples de problèmes qui peuvent tirer parti de l'informatique quantique par lots incluent l'algorithme de Shor et l'estimation de phase quantique simple.

Avec le modèle de calcul par lots, vous pouvez également traiter plusieurs circuits prédéfinis en un seul travail. Les circuits sont soumis au matériel quantique dès que le circuit précédent est terminé, ce qui réduit l'attente entre les soumissions de travaux.

Dans cette architecture, l'état des qubits est perdu entre chaque soumission de circuit.

 **Remarque**

## Regroupement de travaux avec des sessions

Les sessions vous permettent d'organiser plusieurs travaux d'informatique quantique avec la possibilité d'exécuter du code classique entre les travaux quantiques. Vous serez en mesure d'exécuter des algorithmes complexes pour mieux organiser et suivre vos travaux d'informatique quantique individuels. De plus, les travaux regroupés dans les sessions sont hiérarchisés par rapport aux travaux non-session.

Dans ce modèle, la ressource de calcul du client est déplacée vers le cloud, ce qui entraîne une latence inférieure et une exécution répétée du circuit quantique avec différents paramètres. Bien que les sessions autorisent des temps de file d'attente plus courts et des problèmes d'exécution plus longs, les états qubits ne persistent pas entre chaque itération. Exemples de problèmes qui peuvent utiliser cette approche sont Variational Quantum Eigensolvers (VQE) et Quantum Approximate Optimization Algorithms (QAOA).

Pour plus d'informations, consultez [Commencer avec les sessions](#).

## Exécution de l'informatique quantique hybride

Avec l'informatique quantique hybride, les architectures classiques et quantiques sont étroitement couplées, ce qui permet aux calculs classiques d'être effectués alors que les qubits physiques sont cohérents. Bien que limité par la durée de vie et la correction des erreurs qubit, cela permet aux programmes quantiques de s'éloigner des circuits justes. Les programmes peuvent désormais utiliser des constructions de programmation courantes pour effectuer des mesures intermédiaires, optimiser et réutiliser des qubits, et s'adapter en temps réel au QPU. Les exemples de scénarios qui peuvent tirer parti de ce modèle sont l'estimation de phase adaptative et le Machine Learning.

Pour plus d'informations, consultez [l'informatique quantique intégrée](#).

## Exécution de l'informatique quantique distribuée

Dans cette architecture, le calcul classique fonctionne avec des qubits logiques. Avec un contrôle classique entièrement intégré et des qubits logiques de longue durée, le modèle de calcul quantique distribué permet des calculs en temps réel entre les ressources quantiques et distribuées. Les contrôles classiques ne sont plus limités aux boucles et permettent des scénarios

tels que la modélisation de matériaux complexes ou l'évaluation de réactions catalytiques complètes.

ⓘ Remarque

Azure Quantum ne prend actuellement pas en charge l'informatique quantique distribuée.

---

Last updated on 14/08/2026

# Exécuter des travaux d'informatique quantique hybride avec un profil adaptatif target

L'informatique hybride combine des processus informatiques classiques et quantiques pour résoudre des problèmes complexes.

Dans l'informatique hybride, le code classique contrôle l'exécution d'opérations quantiques basées sur des mesures de circuit moyen tandis que les qubits physiques restent actifs. Vous pouvez utiliser des techniques de programmation courantes, telles que les conditions imbriquées, les boucles et les appels de fonction, dans un seul programme quantique pour exécuter des problèmes complexes, réduisant ainsi le nombre de captures nécessaires. Avec les techniques de réutilisation qubit, les programmes plus volumineux peuvent s'exécuter sur des machines à l'aide d'un plus petit nombre de qubits.

Cet article explique comment envoyer des travaux hybrides à Azure Quantum à l'aide du [Adaptive Rtarget](#) profil. Le Adaptive Rtarget profil offre la prise en charge des mesures intermédiaires, du flux de contrôle basé sur les mesures, de la réinitialisation des qubits et du calcul entier classique.

## Prérequis

- Compte Azure avec un abonnement actif. Si vous n'avez pas de compte Azure, inscrivez-vous gratuitement et inscrivez-vous à un [abonnement](#) de paiement à l'utilisation.
- Un espace de travail Azure Quantum. Pour plus d'informations, consultez [Créer un espace de travail Azure Quantum](#).

Si vous souhaitez soumettre Q# des programmes autonomes, vous avez également besoin des conditions préalables suivantes :

- Installez [Visual Studio Code \(VS Code\)](#).
- Installez la dernière version de [l'extension Microsoft Quantum Development Kit](#).

Si vous souhaitez soumettre des programmes Python et Q#, vous avez également besoin des prérequis suivants :

- Un environnement Python avec [Python et Pip](#) installés.

- La dernière `qdk` bibliothèque Python avec l'option `azure` supplémentaire.

Bash

```
pip install --upgrade "qdk[azure]"
```

## Pris en charge targets

Pour exécuter des travaux de calcul quantique hybride, vous devez sélectionner un fournisseur quantique qui prend en charge le [Adaptive Rtarget profil](#).

Actuellement, le profil adaptatif target dans Azure Quantum est pris en charge sur [Quantinuum](#)targets.

## Envoi de travaux RI adaptatifs

Pour soumettre des travaux d'informatique quantique hybride, vous devez configurer le [target profil](#) en tant que **Adaptive RI**, où RI signifie «qubit Reset et Integer computations».

Le Adaptive Rtarget profil offre la prise en charge des mesures intermédiaires, du flux de contrôle basé sur les mesures, de la réinitialisation de qubit et du calcul classique des entiers.

Vous pouvez soumettre des travaux quantiques hybrides à Azure Quantum en tant que Q# programmes autonomes ou Python + Q# programmes. Pour configurer le target profil pour les travaux quantiques hybrides, consultez les sections suivantes.

Q# dans Visual Studio Code

Pour configurer le target profil pour les travaux hybrides dans Visual Studio Code, choisissez l'une des options suivantes :

- Si votre Q# fichier ne fait pas partie d'un Q# projet, ouvrez le fichier et entrez `@EntryPoint(Adaptive_RI)` avant l'opération de point d'entrée de votre programme.
- Si votre Q# fichier fait partie d'un Q# projet, ajoutez ce qui suit au fichier de `qsharp.json` votre projet :

JSON

```
{
 "targetProfile": "adaptive_ri"
}
```

Lorsque vous définissez votre profil sur target, vous pouvez soumettre votre programme **Adaptive RI** en tant que tâche quantique hybride à Quantinuum. Pour ce faire, procédez comme suit :

1. Ouvrez le menu **Affichage** et choisissez **Palette de commandes**, entrez **QDK : Se connecter à un espace de travail Azure Quantum**, puis appuyez sur **Entrée**.
2. Sélectionnez **un compte** Azure, puis suivez les invites pour vous connecter à votre annuaire, abonnement et espace de travail préférés.
3. Une fois connecté, dans le volet **Explorateur** , développez **Espaces de travail Quantum**.
4. Développez votre espace de travail et développez le **fournisseur Quantinuum** .
5. Sélectionnez n'importe quel quantinuum disponible target, par exemple **quantinuum.sim.h2-1e**.
6. Sélectionnez l'icône de lecture à droite du nom target afin de lancer l'envoi du programme actuel Q#.
7. Entrez un nom pour identifier la tâche et le nombre de prises.
8. Appuyez sur **Entrée** pour envoyer le travail. L'état du travail s'affiche en bas de l'écran.
9. Développez **Tâches** et passez la souris sur votre tâche pour afficher les horaires et le statut de votre tâche.

## Fonctionnalités prises en charge

Le tableau suivant répertorie les fonctionnalités prises en charge pour l'informatique quantique hybride avec Quantinuum dans Azure Quantum.

 Agrandir le tableau

Fonctionnalité prise en charge	Notes
Valeurs Dynamics	Bools et entiers dont la valeur dépend d'un résultat de mesure
Boucles	Boucles à limite classique uniquement
Flux de contrôle arbitraire	Utilisation du branchement if/else
Mesure du circuit moyen	Utilise des ressources de registres classiques
Réutilisation du Qubit	Pris en charge
Calcul classique en temps réel	Arithmétique d'entier signé 64 bits, utilise des ressources de registre classiques

Le QDK fournit des commentaires spécifiques target lorsque les fonctionnalités linguistiques Q# ne sont pas prises en charge pour target l'élément sélectionné. Si votre Q# programme contient des fonctionnalités non prises en charge lorsque vous exécutez des travaux quantiques hybrides, vous recevez un message d'erreur au moment du design. Pour plus d'informations, consultez la [QIR page wiki](#).

#### ❗ Remarque

Vous devez sélectionner le profil approprié **Adaptive RItarget** pour obtenir un retour approprié lorsque vous utilisez des fonctionnalités Q# que target ne prend pas en charge.

Pour afficher les fonctionnalités prises en charge en action, copiez le code suivant dans un Q# fichier et ajoutez les extraits de code suivants.

Q#

```
import Std.Measurement.*;
import Std.Math.*;
import Std.Convert.*;

operation Main() : Result {
 use (q0, q1) = (Qubit(), Qubit());
 H(q0);
 let r0 = MResetZ(q0);

 // Copy here the code snippets below to see the supported features
 // in action.
 // Supported features include dynamic values, classically-bounded loops,
 // arbitrary control flow, and mid-circuit measurement.

 r0
}
```

Quantinuum prend en charge les bools et entiers dynamiques, ce qui signifie des bools et des entiers qui dépendent des résultats de mesure. Notez que `r0` est un type `Result` qui peut être utilisé pour générer des valeurs de type booléen et entier dynamiques.

Q#

```
let dynamicBool = r0 != Zero;
let dynamicBool = ResultAsBool(r0);
let dynamicInt = dynamicBool ? 0 | 1;
```

Quantinuum prend en charge les bools et entiers dynamiques. Toutefois, il ne prend pas en charge les valeurs dynamiques pour d'autres types de données, tels que double. Copiez le code suivant pour voir les commentaires sur les limitations des valeurs dynamiques.

Q#

```
let dynamicDouble = r0 == One ? 1. | 0.; // cannot use a dynamic double value
let dynamicInt = r0 == One ? 1 | 0;
let dynamicDouble = IntAsDouble(dynamicInt); // cannot use a dynamic double value
let dynamicRoot = Sqrt(dynamicDouble); // cannot use a dynamic double value
```

Même si les valeurs dynamiques ne sont pas prises en charge pour certains types de données, ces types de données peuvent toujours être utilisés avec des valeurs statiques.

Q#

```
let staticRoot = Sqrt(4.0);
let staticBigInt = IntAsBigInt(2);
```

Même les valeurs dynamiques des types pris en charge ne peuvent pas être utilisées dans certaines situations. Par exemple, Quantinuum ne prend pas en charge les tableaux dynamiques, autrement dit, les tableaux dont la taille dépend d'un résultat de mesure. Quantinuum ne prend pas en charge les boucles délimitées dynamiquement non plus. Copiez le code suivant pour voir les limitations des valeurs dynamiques.

Q#

```
let dynamicInt = r0 == Zero ? 2 | 4;
let dynamicallySizedArray = [0, size = dynamicInt]; // cannot use a dynamically-
sized array
let staticallySizedArray = [0, size = 10];
// Loops with a dynamic condition are not supported by Quantinuum.
for _ in 0..dynamicInt {
 Rx(PI(), q1);
}

// Loops with a static condition are supported.
let staticInt = 3;
for _ in 0..staticInt {
 Rx(PI(), q1);
}
```

Quantinuum prend en charge le flux de contrôle, y compris `if/else` le branchement, en utilisant des conditions statiques et dynamiques. La branchement sur des conditions dynamiques est également appelé branchement basé sur les résultats de mesure.



Q#

```
let dynamicInt = r0 == Zero ? 0 | 1;
if dynamicInt > 0 {
 X(q1);
}
let staticInt = 1;
if staticInt > 5 {
 Y(q1);
} else {
 Z(q1);
}
```

Quantinuum prend en charge les boucles avec des conditions classiques ainsi que des expressions `if`.

Q#

```
for idx in 0..3 {
 if idx % 2 == 0 {
 Rx(ArcSin(1.), q0);
 Rz(IntAsDouble(idx) * PI(), q1)
 } else {
 Ry(ArcCos(-1.), q1);
 Rz(IntAsDouble(idx) * PI(), q1)
 }
}
```

Quantinuum prend en charge la mesure en milieu de circuit, c'est-à-dire la bifurcation en fonction des résultats de mesure.

Q#

```
if r0 == One {
 X(q1);
}
let r1 = MResetZ(q1);
if r0 != r1 {
 let angle = PI() + PI() + PI()* Sin(PI()/2.0);
 Rxx(angle, q0, q1);
} else {
 Rxx(PI() + PI() + 2.0 * PI() * Sin(PI()/2.0), q1, q0);
}
```

## Estimer le coût d'un travail de calcul quantique hybride

Vous pouvez estimer le coût d'exécution d'un travail de calcul quantique hybride sur le matériel Quantinuum en exécutant d'abord le travail sur un émulateur.

Après une exécution réussie sur l'émulateur :

1. Dans votre espace de travail Azure Quantum, sélectionnez **Gestion des travaux**.
2. Sélectionnez la tâche que vous avez envoyée.
3. Dans la fenêtre contextuelle Détails du travail, sélectionnez **Estimation** du coût pour afficher le nombre de crédits d'émulateur de quantinuum (eHQCs) utilisés. Ce nombre se traduit directement par le nombre de HQC (crédits quantiques Quantinuum) nécessaires pour exécuter le travail sur le matériel Quantinuum.

**Job details**  
Quantum Workspace

Cancel job Refresh

**Essentials**

Name: IterativePhaseEstimation  
Id: 338214f9-a487-430c-a93f-37add00c599b  
Provider: quantinuum  
Target: quantinuum.sim.h1-1e  
Cost estimate: --

Input/Output **Cost Estimation**

The table below shows the usage generated by this job, and an estimate of the job cost that will appear on your Azure bill. [Learn more](#)

Dimension	Unit Price	Consumed Units	Billed Units	Cost Estimate
EHQC	\$0 / hqc	11.31	11.31	\$0
				Total: \$0

### ❗ Remarque

Quantinuum déroule le circuit entier et calcule le coût sur tous les chemins de code, qu'ils soient exécutés de manière conditionnelle ou non.

## Exemples d'informatique quantique hybride

Vous trouverez les exemples suivants dans le [Q# référentiel](#) d'exemples de code. Ils illustrent l'ensemble de fonctionnalités actuel pour l'informatique quantique hybride.

### Code de répétition à trois qubits

Cet exemple montre comment créer un [code](#) de répétition à trois qubits qui peut être utilisé pour détecter et corriger les erreurs de retournement de bits.

Il tire parti des fonctionnalités de calcul hybride intégrées pour compter le nombre de fois où la correction des erreurs a été effectuée alors que l'état d'un registre qubit logique est cohérent.

Vous trouverez le code dans [cet exemple](#).

## Estimation de la phase itérative

Cet exemple de programme illustre une estimation de phase itérative dans Q#. Il utilise l'estimation de phase itérative pour calculer un produit interne entre deux vecteurs 2 dimensionnels encodés sur un target qubit et un qubit ancilla. Un qubit de contrôle supplémentaire est également initialisé, qui est le seul qubit utilisé pour la mesure.

Le circuit commence par encoder la paire de vecteurs sur le target qubit et le qubit ancilla. Il applique ensuite un opérateur Oracle à l'ensemble du registre, contrôlé sur le qubit de contrôle, qui est configuré dans l'état  $|+\rangle$ . L'opérateur Oracle contrôlé génère une phase sur l'état  $|1\rangle$  du qubit de contrôle. Cela peut ensuite être lu en appliquant une porte H au qubit de contrôle pour rendre la phase observable lors de la mesure.

Vous trouverez le code dans [cet exemple](#).

### ⓘ Remarque

Cet exemple de code a été écrit par des membres de [l'équipe KPMG](#) Quantum en Australie et fait partie d'une licence MIT. Il illustre des capacités étendues de Adaptive RTargets, et fait usage de boucles bornées, d'appels de fonctions classiques à l'exécution, d'instructions conditionnelles imbriquées, de mesures intermédiaires de circuit et de la réutilisation de qubits.

## Contenu connexe

- [Commencer avec les sessions](#)

---

Last updated on 29/04/2026

# Présentation du langage de programmation quantique Q#

Q# est un langage de programmation [open source](#) de haut niveau développé par Microsoft pour écrire des programmes quantiques. Q# est inclus dans le Kit de développement Quantum (QDK). Pour plus d'informations, consultez [Configurer le Kit](#) de développement Quantum.

En tant que langage de programmation quantique, Q# répond aux exigences suivantes pour le langage, le compilateur et le runtime :

- **Indépendant du matériel** : Les qubits dans les algorithmes quantiques ne sont pas liés à un matériel ou une disposition quantique spécifique. Le Q# compilateur et le runtime gèrent le mappage entre les qubits de programme et les qubits physiques, ce qui permet au même code de s'exécuter sur différents processeurs quantiques.
- **Intégration de l'informatique quantique et classique** : Q# permet l'intégration de calculs quantiques et classiques, qui est essentiel pour l'informatique quantique universelle.
- **Gestion des qubits** : Q# fournit des opérations et des fonctions intégrées pour la gestion des qubits, notamment la création d'états de superposition, des qubits d'enchèvement et l'exécution de mesures quantiques.
- **Respectez les lois de la physique** : Q# et les algorithmes quantiques doivent suivre les règles de la physique quantique. Par exemple, vous ne pouvez pas copier ou accéder directement à l'état qubit dans Q#.

Pour plus d'informations sur les origines de Q#, consultez le billet de blog [Pourquoi avons-nous besoin de Q#?](#).

## Structure d'un Q# programme

Avant de commencer à écrire Q# des programmes, il est important de comprendre leur structure et leurs composants. Considérez le programme suivant Q# , nommé `Superposition`, qui crée un état de superposition :

Q#

```
namespace Superposition {
 @EntryPoint()
 operation MeasureOneQubit() : Result {
 // Allocate a qubit. By default, it's in the 0 state.
 use q = Qubit();
 // Apply the Hadamard operation, H, to the state.
 // It now has a 50% chance of being measured as 0 or 1.
 H(q);
 // Measure the qubit in the Z-basis.
 let result = M(q);
 }
}
```

```

 // Reset the qubit before releasing it.
 Reset(q);
 // Return the result of the measurement.
 return result;
}
}

```

En fonction des commentaires (`//`), le Q# programme alloue d'abord un qubit, applique une opération pour placer le qubit en superposition, mesure l'état du qubit, réinitialise le qubit et retourne enfin le résultat.

Nous allons décomposer ce Q# programme en ses composants.

## Espaces de noms utilisateur

Q# les programmes peuvent éventuellement commencer par un espace de noms défini par l'utilisateur, par exemple :

Q#

```

namespace Superposition {
 // Your code goes here.
}

```

Les espaces de noms peuvent vous aider à organiser les fonctionnalités associées. Les espaces de noms sont facultatifs dans Q# les programmes, ce qui signifie que vous pouvez écrire un programme sans définir d'espace de noms.

Par exemple, le `Superposition` programme de l'exemple peut également être écrit sans espace de noms comme suit :

Q#

```

@EntryPoint()
operation MeasureOneQubit() : Result {
 // Allocate a qubit. By default, it's in the 0 state.
 use q = Qubit();
 // Apply the Hadamard operation, H, to the state.
 // It now has a 50% chance of being measured as 0 or 1.
 H(q);
 // Measure the qubit in the Z-basis.
 let result = M(q);
 // Reset the qubit before releasing it.
 Reset(q);
 // Return the result of the measurement.
 return result;
}

```

### ❗ Remarque

Chaque Q# programme ne peut avoir qu'un `namespace` seul . Si vous ne spécifiez pas d'espace de noms, le Q# compilateur utilise le nom de fichier comme espace de noms.

## Points d'entrée

Chaque Q# programme doit avoir un point d'entrée, qui est le point de départ du programme. Par défaut, le Q# compilateur démarre l'exécution d'un programme à partir de l'opération `Main()` , le cas échéant, qui peut se trouver n'importe où dans le programme. Si vous le souhaitez, vous pouvez utiliser l'attribut `@EntryPoint()` pour spécifier n'importe quelle opération dans le programme comme point d'exécution.

Par exemple, dans le `Superposition` programme, l'opération `MeasureOneQubit()` est le point d'entrée du programme, car elle a l'attribut `@EntryPoint()` avant la définition de l'opération :

Q#

```
@EntryPoint()
operation MeasureOneQubit() : Result {
 ...
}
```

Toutefois, le programme peut également être écrit sans l'attribut `@EntryPoint()` en renommant l'opération `MeasureOneQubit()` `Main()` , par exemple :

Q#

```
// The Q# compiler automatically detects the Main() operation as the entry point.

operation Main() : Result {
 // Allocate a qubit. By default, it's in the 0 state.
 use q = Qubit();
 // Apply the Hadamard operation, H, to the state.
 // It now has a 50% chance of being measured as 0 or 1.
 H(q);
 // Measure the qubit in the Z-basis.
 let result = M(q);
 // Reset the qubit before releasing it.
 Reset(q);
 // Return the result of the measurement.
 return result;
}
```

# Types

Les types sont essentiels dans n'importe quel langage de programmation, car ils définissent les données qu'un programme peut utiliser. Q# fournit des types intégrés qui sont communs à la plupart des langages, notamment `Int`, `Double`, `Bool`, et `String` des types qui définissent des plages, des tableaux et des tuples.

Q# fournit également des types spécifiques à l'informatique quantique. Par exemple, le `Result` type représente le résultat d'une mesure qubit et peut avoir deux valeurs : `Zero` ou `One`.

Dans le programme `Superposition`, l'opération `MeasureOneQubit()` renvoie un type `Result`, qui correspond au type de retour de l'opération `M`. Le résultat de la mesure est stocké dans une nouvelle variable définie à l'aide de l'instruction `let` :

Q#

```
// The operation definition returns a Result type.
operation MeasureOneQubit() : Result {
 ...
 // Measure the qubit in the Z-basis, returning a Result type.
 let result = M(q);
 ...
}
```

Un autre exemple de type spécifique au quantum est le `Qubit` type, qui représente un bit quantique.

Q# vous permet également de définir vos propres types personnalisés. Pour plus d'informations, consultez [Déclarations de type](#).

## Allocation de qubits

Dans Q#, vous allouez des qubits à l'aide du `use` mot clé et du `Qubit` type. Les qubits sont toujours alloués dans l'état  $|0\rangle$ .

Par exemple, le `Superposition` programme définit un qubit unique et le stocke dans la variable `q`:

Q#

```
// Allocate a qubit.
use q = Qubit();
```

Vous pouvez également allouer plusieurs qubits et accéder à chacun par le biais de son index :

Q#

```
use qubits = Qubit[2]; // Allocate two qubits.
H(qubits[0]); // Apply H to the first qubit.
X(qubits[1]); // Apply X to the second qubit.
```

## Opérations quantiques

Après avoir alloué un qubit, vous pouvez le transmettre aux opérations et aux fonctions. Les opérations sont les blocs de construction de base d'un Q# programme. Une Q# opération est une sous-routine quantique ou une routine appelante qui contient des opérations quantiques qui modifient l'état du registre qubit.

Pour définir une Q# opération, vous spécifiez un nom pour l'opération, ses entrées et sa sortie. Dans le programme `Superposition`, l'opération `MeasureOneQubit()` ne prend aucun paramètre et retourne un type `Result`.

Q#

```
operation MeasureOneQubit() : Result {
 ...
}
```

Voici un exemple de base qui ne prend aucun paramètre et attend aucune valeur de retour. La valeur `Unit` est équivalente à `NULL` dans d'autres langues.

Q#

```
operation SayHelloQ() : Unit {
 Message("Hello quantum world!");
}
```

La Q# bibliothèque standard fournit également des opérations que vous pouvez utiliser dans des programmes quantiques, tels que l'opération Hadamard, `H` dans le `Superposition` programme. Étant donné un qubit sur la base Z, `H` place le qubit dans une superposition même, où il a une probabilité de 50 % d'être mesurée en tant que `Zero` ou `One`.

## Mesure des qubits

Bien qu'il existe de nombreux types de mesures quantiques, Q# se concentre sur les mesures projectives sur des qubits uniques, également appelées mesures Pauli.

Dans Q#, l'opération `Measure` mesure un ou plusieurs qubits dans la base Pauli spécifiée, qui peut être `PauliX`, `PauliY` ou `PauliZ`. `Measure` renvoie un type `Result` soit `Zero` soit `One`.



Pour implémenter une mesure dans la base computationnelle  $\{|0\rangle, |1\rangle\}$ , vous pouvez également utiliser l'opération `M`, qui mesure un qubit dans la base de Pauli Z. Cela équivaut `M` à `Measure([PauliZ], [qubit])`.

Par exemple, le `Superposition` programme utilise l'opération `M` :

```
Q#

// Measure the qubit in the Z-basis.
let result = M(q);
```

## Réinitialisation de qubits

Dans Q#, les qubits **doivent** être dans l'état  $|0\rangle$  lorsqu'ils sont libérés pour éviter les erreurs dans le matériel quantique. Vous pouvez réinitialiser un qubit à l'état  $|0\rangle$  à l'aide de l'opération `Reset` à la fin du programme. L'échec de la réinitialisation d'un qubit entraîne une erreur d'exécution.

```
Q#

// Reset a qubit.
Reset(q);
```

## Espaces de noms de bibliothèque standard

La Q# bibliothèque standard a des espaces de noms intégrés qui contiennent des fonctions et des opérations que vous pouvez utiliser dans les programmes quantiques. Par exemple, l'espace `Std.Intrinsic` de noms contient des opérations et des fonctions couramment utilisées, telles que `M` la mesure des résultats et `Message` l'affichage des messages utilisateur n'importe où dans le programme.

Pour appeler une fonction ou une opération, vous pouvez spécifier l'espace de noms complet ou utiliser une `import` instruction, ce qui rend toutes les fonctions et opérations de cet espace de noms disponibles et rend votre code plus lisible. Les exemples suivants appellent la même opération :

```
Q#

Std.Intrinsic.Message("Hello quantum world!");
```

```
Q#
```

```
// imports all functions and operations from the Std.Intrinsic namespace.
import Std.Intrinsic.*;
Message("Hello quantum world!");

// imports just the `Message` function from the Std.Intrinsic namespace.
import Std.Intrinsic.Message;
Message("Hello quantum world!");
```

### ❗ Remarque

Le `Superposition` programme n'a `import` pas d'instructions ni d'appels avec des espaces de noms complets. Cela est dû au fait que l'environnement Q# de développement charge automatiquement deux espaces de noms et `Std.Core Std.Intrinsic`, qui contiennent des fonctions et des opérations couramment utilisées.

Vous pouvez tirer parti de l'espace de noms `Std.Measurement`, en utilisant l'opération `MResetZ` pour optimiser le programme `Superposition`. `MResetZ` combine les opérations de mesure et de réinitialisation en une seule étape, comme dans l'exemple suivant :

Q#

```
// Import the namespace for the MResetZ operation.
import Std.Measurement.*;

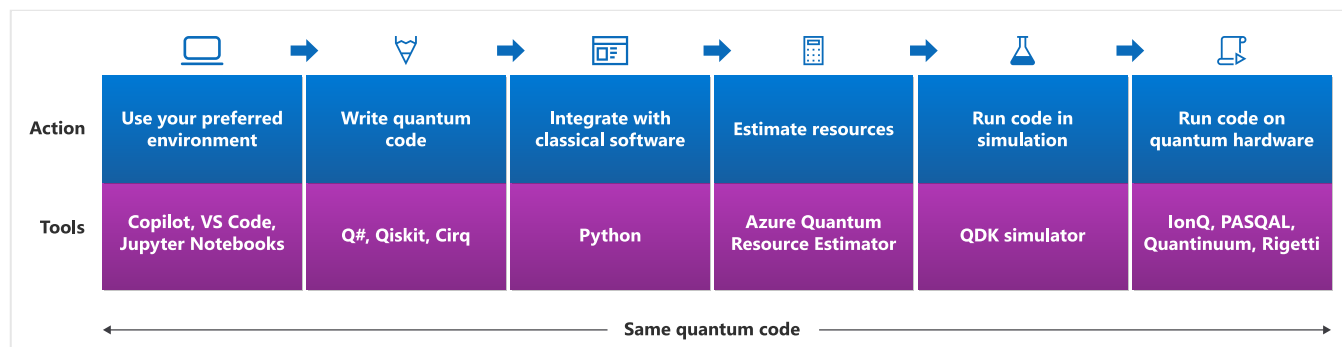
@EntryPoint()
operation MeasureOneQubit() : Result {
 // Allocate a qubit. By default, it's in the 0 state.
 use q = Qubit();
 // Apply the Hadamard operation, H, to the state.
 // It now has a 50% chance of being measured as 0 or 1.
 H(q);
 // Measure and reset the qubit, and then return the result value.
 return MResetZ(q);
}
```

## Apprendre à développer des programmes quantiques avec Q# et Azure Quantum

Q# et Azure Quantum sont une combinaison puissante pour le développement et l'exécution de programmes quantiques. Avec Q# et Azure Quantum, vous pouvez écrire des programmes quantiques, simuler leur comportement, estimer les besoins en ressources et les exécuter sur du matériel quantique réel. Cette intégration vous permet d'explorer le potentiel de l'informatique quantique et de développer des solutions innovantes pour des problèmes complexes. Qu'il s'agisse d'un débutant ou d'un développeur quantique expérimenté, Q# azure

Quantum fournit les outils et les ressources dont vous avez besoin pour déverrouiller la puissance du calcul quantique.

Le diagramme suivant montre les étapes à travers lesquelles un programme quantique passe lorsque vous le développez avec Q# et Azure Quantum. Votre programme commence par l'environnement de développement et se termine par la soumission du travail à du matériel quantique réel.



Nous allons décomposer les étapes du diagramme.

## Choisir votre environnement de développement

Exécutez vos programmes quantiques dans votre environnement de développement préféré. Vous pouvez utiliser l'éditeur de code en ligne dans le site web Microsoft Quantum, un environnement de développement local avec Visual Studio Code ou combiner Q# du code avec du code Python dans jupyter Notebooks. Pour plus d'informations, consultez [Différentes façons d'exécuter Q# des programmes](#).

## Écrire votre programme quantique

Vous pouvez écrire des programmes quantiques dans Q# en utilisant le Kit de développement quantique (QDK). Pour commencer, consultez [Démarrage rapide : Créer votre premier Q# programme](#).

En outre, le QDK offre une prise en charge d'autres langages Q# pour l'informatique quantique, comme [Qiskit](#) et [Cirq](#).

## Intégrer à Python

Vous pouvez utiliser Q# lui-même ou avec Python dans différents IDE. Par exemple, vous pouvez utiliser un Q# projet avec un programme hôte Python pour appeler Q# des opérations ou l'intégrer Q# à Python dans Jupyter Notebooks. Pour plus [d'informations, consultez Intégration de Q# et Python](#).

## La commande %%qsharp

Par défaut, Q# les programmes dans Jupyter Notebooks utilisent le `ipykernel` package Python. Pour ajouter du code Q# à une cellule de bloc-notes, utilisez la commande `%%qsharp`, qui est activée avec le paquet Python `qsharp`, suivie de votre code Q#.

Lorsque vous utilisez `%%qsharp`, gardez à l'esprit ce qui suit :

- Vous devez d'abord exécuter `from qdk import qsharp` pour activer `%%qsharp`.
- `%%qsharp` se limite à la cellule de notebook dans laquelle il apparaît et change le type de cellule de Python à Q#.
- Vous ne pouvez pas placer d'instruction Python avant ou après `%%qsharp`.
- Q# le code qui suit `%%qsharp` doit respecter la Q# syntaxe. Par exemple, utilisez `//` plutôt que de `#` désigner des commentaires et `;` pour mettre fin aux lignes de code.

## Estimer les ressources

Avant d'exécuter sur du matériel quantique réel, vous devez déterminer si votre programme peut s'exécuter sur du matériel existant et le nombre de ressources qu'il consommera.

L'estimateur de ressources Azure Quantum vous permet d'évaluer les décisions architecturales, de comparer les technologies qubit et de déterminer les ressources nécessaires pour exécuter un algorithme quantique donné. Vous pouvez choisir parmi les protocoles prédéfinis à tolérance de panne et spécifier des hypothèses du modèle qubit physique sous-jacent.

Pour plus d'informations, consultez [Exécuter votre première estimation](#) de ressource.

### ⚠ Remarque

L'estimateur de ressources Azure Quantum est gratuit et ne nécessite pas de compte Azure.

## Exécuter votre programme dans la simulation

Lorsque vous compilez et exécutez un programme quantique, le QDK crée une instance du simulateur quantique et lui transmet le Q# code. Le simulateur utilise le code Q# pour créer des qubits (simulations de particules quantiques) et appliquer des transformations afin de changer leur état. Les résultats des opérations quantiques dans le simulateur sont ensuite retournés au programme. L'isolation du code Q# dans le simulateur garantit que les algorithmes suivent les lois de la physique quantique et s'exécutent correctement sur les ordinateurs quantiques.

# Soumettre votre programme à du matériel quantique réel

Une fois que vous avez testé votre programme en simulation, vous pouvez l'exécuter sur du matériel quantique réel. Lorsque vous exécutez un programme quantique dans Azure Quantum, vous créez et exécutez un **travail**. Pour envoyer un travail aux fournisseurs Azure Quantum, vous avez besoin d'un compte Azure et d'un espace de travail quantique. Si vous n'avez pas d'espace de travail quantique, consultez [Créer un espace de travail](#) Azure Quantum.

Azure Quantum offre certains des matériels quantiques les plus attrayants et diversifiés. Pour connaître la liste actuelle des fournisseurs de matériel pris en charge, consultez [Fournisseurs d'informatique quantique](#).

Une fois que vous avez envoyé votre travail, Azure Quantum gère le cycle de vie des travaux, notamment la planification, l'exécution et la supervision des travaux. Vous pouvez suivre l'état de votre travail et afficher les résultats dans le portail Azure Quantum.

## Contenu connexe

- [Différentes façons d'exécuter Q# des programmes](#)
- [Configurer le Kit de développement Quantum](#)
- [Démarrage rapide : Créer votre premier Q# programme](#)

---

Last updated on 12/11/2025

# Démarrage rapide : Créer votre premier Q# programme

Découvrez comment écrire un programme Q# de base qui démontre l'intrication, un concept clé de l'informatique quantique.

Lorsque deux qubits ou plus [sont enchevêtrés, ils partagent des informations quantiques](#), ce qui signifie que tout ce qui arrive à un qubit arrive également à l'autre. Dans ce guide de démarrage rapide, vous créez un état enchevêtré à deux qubits particulier appelé paire Bell. Dans une paire Bell, si vous mesurez un qubit dans l'état  $|0\rangle$ , vous savez que l'autre qubit est également dans l'état  $|0\rangle$  sans la mesurer. Pour plus d'informations, consultez [l'intrication quantique](#).

Dans ce guide de démarrage rapide, vous :

- ✓ Créez un fichier Q#.
- ✓ Allouez une paire de qubits.
- ✓ Entangle les qubits.

## Prérequis

- La dernière version de [Visual Studio Code](#).
- L'extension [Microsoft Quantum Development Kit \(QDK\)](#). Pour plus d'informations sur l'installation, consultez [Configurer le Kit de développement Microsoft Quantum](#).

## Créez un fichier Q#

1. Ouvrez Visual Studio Code.
2. Sélectionnez **Fichier** > **nouveau fichier** texte.
3. Enregistrez le fichier sous le nom `Main.qs`. L'extension .qs désigne un Q# programme.

## Écrire votre Q# code

Dans votre `Main.qs` fichier, suivez ces étapes pour entangler et mesurer une paire de qubits.

## Importer une bibliothèque quantique

Le QDK inclut la Q# bibliothèque standard avec des fonctions et des opérations prédéfinies pour vos programmes quantiques. Pour les utiliser, vous devez d'abord importer la bibliothèque appropriée.

Dans votre programme, utilisez une `import` instruction pour ouvrir la `Std.Diagnostics` bibliothèque. Cela vous permet d'accéder à toutes les fonctions et opérations de la bibliothèque, notamment `DumpMachine`, que vous utilisez ultérieurement pour afficher l'état enchevêtré.

Q#

```
import Std.Diagnostics.*;
```

## Définir une opération

Après avoir importé les bibliothèques pertinentes, définissez votre opération quantique et ses valeurs d'entrée et de sortie. Pour ce guide de démarrage rapide, votre opération est `Main`. C'est là que vous allez écrire le code restant Q# pour allouer, manipuler et mesurer deux qubits.

`Main` ne prend aucun paramètre et retourne deux `Result` valeurs, soit `Zero One`, qui représentent les résultats des mesures qubit :

Q#

```
operation Main() : (Result, Result) {
 // Your entanglement code goes here.
}
```

## Allouer deux qubits

L'opération `Main` est actuellement vide, donc l'étape suivante consiste à allouer deux qubits, `q1` et `q2`. Dans Q#, vous allouez des qubits à l'aide du `use` mot clé :

Q#

```
// Allocate two qubits, q1 and q2, in the 0 state.
use (q1, q2) = (Qubit(), Qubit());
```

### ⚠ Remarque

Dans Q#, les qubits sont toujours alloués dans l'état  $|0\rangle$ .

## Placer un qubit en superposition

Les qubits `q1` et `q2` sont dans l'état  $|0\rangle$ . Pour préparer les qubits pour l'enchevêtrement, vous devez placer l'un d'entre eux dans une superposition égale, où il a une probabilité de 50 % d'être mesuré en tant que  $|0\rangle$  ou  $|1\rangle$ .

Vous placez un qubit dans une superposition en appliquant l'opération Hadamard, `H`.

Q#

```
// Put q1 into an even superposition.
H(q1);
```

L'état résultant est `q1`  $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ , qui est une superposition égale de  $|0\rangle$  et  $|1\rangle$ .

## Entangler les qubits

Vous êtes maintenant prêt à entrelacer les qubits à l'aide de l'opération CNOT, `CNOT`. `CNOT` est une opération de contrôle qui prend deux qubits, l'un jouant le rôle de contrôle et l'autre comme cible.

Pour ce guide de démarrage rapide, vous définissez `q1` en tant que qubit de contrôle et `q2` en tant que qubit cible. Cela signifie que `CNOT` inverse l'état de `q2` lorsque l'état de `q1` est  $|1\rangle$ .

Q#

```
// Entangle q1 and q2, making q2 depend on q1.
CNOT(q1, q2);
```

L'état résultant des deux qubits est la paire  $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ .

### Conseil

Si vous souhaitez découvrir comment les opérations Hadamard et CNOT transforment l'état des qubits, consultez [Création d'un entanglement avec des opérations](#) quantiques.

## Afficher l'état enchevêtré

Avant de mesurer les qubits, il est important de vérifier que votre code précédent les entasse correctement. Utilisez l'opération `DumpMachine`, qui fait partie de la `Std.Diagnostics` bibliothèque, pour générer l'état actuel de votre Q# programme :



Q#

```
// Show the entangled state of the qubits.
DumpMachine();
```

## Mesurer les qubits

Maintenant que vous avez vérifié que les qubits sont enchevêtrés, vous pouvez utiliser l'opération `M` pour les mesurer. Mesurer `q1` et `q2` effondre leurs états quantiques en `Zero` ou `One` avec une probabilité égale.

Dans Q#, vous utilisez le `let` mot clé pour déclarer une nouvelle variable. Pour stocker les résultats de mesure de `q1` et `q2`, déclarez respectivement les variables `m1` et `m2`.

Q#

```
// Measure q1 and q2 and store the results in m1 and m2.
let (m1, m2) = (M(q1), M(q2));
```

## Réinitialiser les qubits

Avant d'être libérés à la fin de chaque Q# programme, les qubits doivent être dans l'état  $|0\rangle$ . Pour ce faire, utilisez l'opération `Reset` :

Q#

```
// Reset q1 and q2 to the 0 state.
Reset(q1);
Reset(q2);
```

## Retourner les résultats de la mesure

Enfin, pour terminer l'opération `Main` et observer l'état enchevêtré, retournez les résultats de mesure de `m1` et `m2`.

Q#

```
// Return the measurement results.
return (m1, m2);
```

 **Conseil**

Si vous souhaitez en savoir plus sur une fonction ou une Q# opération, pointez dessus.

## Exécuter votre Q# code

Félicitations ! Vous avez écrit un Q# programme qui entangle deux qubits et crée une paire Bell.

Votre programme final Q# doit ressembler à ceci :

Q#

```
import Std.Diagnostics.*;

operation Main() : (Result, Result) {
 // Allocate two qubits, q1 and q2, in the 0 state.
 use (q1, q2) = (Qubit(), Qubit());

 // Put q1 into an even superposition.
 // It now has a 50% chance of being measured as 0 or 1.
 H(q1);

 // Entangle q1 and q2, making q2 depend on q1.
 CNOT(q1, q2);

 // Show the entangled state of the qubits.
 DumpMachine();

 // Measure q1 and q2 and store the results in m1 and m2.
 let (m1, m2) = (M(q1), M(q2));

 // Reset q1 and q2 to the 0 state.
 Reset(q1);
 Reset(q2);

 // Return the measurement results.
 return (m1, m2);
}
```

Pour exécuter votre programme et afficher le résultat des deux qubits, sélectionnez **Exécuter** au-dessus de l'opération `Main` ou appuyez sur **Ctrl+F5**

```

1 import Microsoft.Quantum.Diagnostics.*;
2
3 Run | Histogram | Estimate | Debug | Circuit
4 operation Main() : (Result, Result) {
5 // Allocate two qubits, q1 and q2, in the 0 state.
6 use (q1, q2) = (Qubit(), Qubit());
7
8 // Put q1 into an even superposition.
9 // It now has a 50% chance of being measured as 0 or 1.
10 H(q1);
11
12 // Entangle q1 and q2, making q2 depend on q1.
13 CNOT(q1, q2);
14
15 // Show the entangled state of the qubits.
16 DumpMachine();
17
18 // Measure q1 and q2 and store the results in m1 and m2.
19 let (m1, m2) = (M(q1), M(q2));
20
21 // Reset q1 and q2 to the 0 state.
22 Reset(q1);

```

Vous pouvez exécuter le programme plusieurs fois, chacun avec un résultat différent dans la console de débogage. Cela illustre la nature probabiliste des mesures quantiques et l'enchevêtrement des qubits.

Par exemple, si le résultat est `Zero`, votre console de débogage doit ressembler à ceci :

#### Result

DumpMachine:

Basis	Amplitude	Probability	Phase
00>	0.7071+0.0000i	50.0000%	0.0000
11>	0.7071+0.0000i	50.0000%	0.0000

Result: "(Zero, Zero)"

## Étape suivante

Pour en savoir plus sur l'enchevêtrement quantique avec Q#, consultez [Tutoriel : Explorer l'enchevêtrement quantique avec Q#](#). Ce tutoriel approfondit les concepts abordés dans ce guide de prise en main rapide et vous aide à écrire un programme d'enchevêtrement plus avancé.

---

Last updated on 18/02/2026

# Différentes façons d'exécuter des programmes Q#

Le Microsoft Quantum Development Kit (QDK) offre deux options de développement pour écrire et exécuter des programmes Q#. L'interface des deux options avec Azure Quantum vous permet d'exécuter des programmes Q# sur des simulateurs et du matériel quantique à partir de fournisseurs Azure Quantum. Le QDK inclut également plusieurs simulateurs locaux pour exécuter des programmes Q# sur votre ordinateur personnel.

## Options de développement et d'exécution de programmes Q#

Développez des programmes Q# et gérez des travaux Q# que vous envoyez à Azure Quantum via les environnements de développement suivants :

- **Visual Studio Code** : Écrire, exécuter et déboguer du code Q# dans votre environnement local et envoyer des travaux à Azure Quantum avec l'extension QDK dans Visual Studio Code (VS Code). Installation requise.
- **Jupyter Notebook** : Développer du code Q# et envoyer des travaux à Azure Quantum dans Jupyter Notebook avec le module `qdk.qsharp` Python. Installation requise.
- **Azure portail** : Gérer votre abonnement Azure et votre espace de travail Azure Quantum, ainsi que des informations sur vos fournisseurs quantiques et soumissions de travaux. Nécessite un compte Azure.

L'option que vous choisissiez dépend de votre expérience de codage, de vos connaissances quantiques et de vos objectifs. Chaque option offre des caractéristiques et des fonctionnalités différentes, vous pouvez donc les utiliser ensemble. Par exemple, écrivez des programmes Q# avec l'extension QDK dans VS Code et gérez votre espace de travail quantique dans le portail Azure.

## Visual Studio Code

[VS Code](#) est un éditeur de code open source gratuit de Microsoft. Avec l'extension QDK pour VS Code, vous pouvez créer des programmes Q# et charger des exemples Q# intégrés. Le QDK dans VS Code offre les fonctionnalités de développement locales suivantes et bien plus encore pour les programmes Q# (`.qs` fichiers) :

- Messagerie d'erreur
- Mise en surbrillance de la syntaxe
- Débogage
- CodeLens
- IntelliSense
- Estimation des ressources d'ordinateur quantique

#### ❗ Remarque

L'extension QDK fournit également la prise en charge linguistique des programmes OpenQASM ( `.qasm` fichiers).

Vous n'avez pas besoin d'un compte Azure pour utiliser le QDK dans VS Code, mais vous avez besoin d'un compte Azure pour envoyer des travaux à Azure Quantum avec le QDK. Vous pouvez utiliser le QDK pour vous connecter à votre espace de travail Azure Quantum à partir de VS Code et exécuter des programmes Q# sur les ordinateurs quantiques et les simulateurs de différents fournisseurs Azure Quantum. Pour plus d'informations, veuillez consulter la section [Comment soumettre des programmes Q# avec Visual Studio Code](#).

Pour commencer à utiliser l'extension QDK dans VS Code, consultez [Configurer le QDK](#).

#### ❗ Remarque

L'extension QDK est également disponible dans [VS Code for the Web](#) <sup>↗</sup>, qui fournit les mêmes fonctionnalités de connectivité Azure et de langage Q# que la version de bureau. Toutefois, le Web ne prend pas en charge les programmes Python, Qiskit ou Cirq.

## L'extension QDK dans VS Code est-elle adaptée à moi ?

VS Code est un environnement riche en fonctionnalités qui inclut CodeLens et IntelliSense pour vous aider à écrire, exécuter et déboguer des programmes quantiques Q# et OpenQASM. Si vous avez une expérience de codage et que vous souhaitez explorer Q# en profondeur, VS Code est pour vous.

Le tableau suivant montre ce que vous pouvez et ne pouvez pas faire dans VS Code :

[🔍](#) Agrandir le tableau

Vous pouvez :	Vous ne pouvez pas :	Ce dont vous avez besoin :
• Exécutez des programmes Q# et OpenQASM.     • Charger des exemples de code.     • Déboguez vos programmes.     • Enregistrez vos programmes et résultats.     • Afficher les messages d'erreur du compilateur.     • Connectez-vous à votre espace de travail Azure Quantum.     • Visualiser les circuits quantiques.     • Utilisez l'estimateur de ressource.	• Gérez vos abonnements et vos espaces de travail.     • Gérez vos travaux quantiques.     • Choisissez des fournisseurs et des plans d'informatique quantique.	• Pour installer <a href="#">VS Code</a> .     • Pour installer l'extension <a href="#">QDK</a> .     • Un abonnement Azure et un espace de travail quantique (si vous souhaitez exécuter des programmes sur du matériel réel).

## Jupyter Notebook

Le QDK a un package de Python riche `qdk` qui vous permet de développer des programmes Q# dans `.py` des fichiers Python ou des Jupyter Notebook. Le package QDK Python prend également en charge d'autres langages quantiques, tels que Qiskit, Cirq et PennyLane.

Le `qdk` package Python comprend plusieurs modules pour vous aider à développer des programmes quantiques et à gérer des travaux Azure Quantum. Par exemple, le module `qsharp` vous permet d'écrire du code Q# dans Jupyter Notebook, et le module `azure` vous permet de vous connecter à votre espace de travail quantique et d'envoyer des travaux à Azure Quantum.

Pour obtenir une vue d'ensemble des fonctionnalités de `qdk` package et de module Python, consultez la [description du projet QDK](#) sur le site web PyPi.

## Python et Jupyter Notebook me conviennent-ils ?

Jupyter Notebook est pratique pour écrire du code Python et visualiser la sortie dans un seul environnement de développement. Si vous préférez développer dans Python et que vous souhaitez prendre en charge plusieurs langages de programmation quantique, le package QDK Python et les Jupyter Notebook sont pour vous.

Le tableau suivant montre ce que vous pouvez et ne pouvez pas faire dans Python et Jupyter Notebook :

[🔗 Agrandir le tableau](#)

Vous pouvez :	Vous ne pouvez pas :	Ce dont vous avez besoin :
• Développez dans plusieurs langages de programmation quantique.    • Enregistrez vos programmes et résultats.    • Connectez-vous à votre espace de travail Azure Quantum.    • Visualiser les circuits quantiques.    • Utilisez l'estimateur de ressource.	• Gérez vos abonnements et vos espaces de travail.    • Gérez vos travaux quantiques.    • Choisissez des fournisseurs et des plans d'informatique quantique.	• Pour installer <a href="#">Python et Pip</a> .    • Pour installer <a href="#">Jupyter Notebook</a> .    • Pour installer le <a href="#">package QDK Python</a> .    • Un abonnement Azure et un espace de travail quantique (si vous souhaitez exécuter des programmes sur du matériel réel).

## Portail Azure

Le [Portail Azure](#) est l'interface principale de la plateforme de cloud computing de Microsoft Azure. À partir du portail, vous pouvez créer un espace de travail [Azure Quantum](#) pour exécuter des programmes quantiques, envoyer des travaux à des fournisseurs de matériel [quantum](#) et stocker des résultats de travail dans un compte de stockage Azure Quantum. Vous pouvez également gérer vos abonnements, activités, utilisation de crédit, quotas et contrôle d'accès.

## Est-ce que le Portail Azure est bon pour moi ?

À partir du Portail Azure, vous pouvez accorder à un groupe d'utilisateurs, comme les membres de votre équipe ou les étudiants, l'accès à votre espace de travail quantique. Si vous souhaitez gérer vos travaux et abonnements, passez en revue vos factures ou essayez différents fournisseurs quantiques, puis le portail Azure vous convient.

Le tableau suivant montre ce que vous pouvez et ne pouvez pas faire dans le Portail Azure :

[🔗 Agrandir le tableau](#)



Vous pouvez :	Vous ne pouvez pas :	Ce dont vous avez besoin :
• Créez des espaces de travail quantiques.     • Gérez vos abonnements et vos espaces de travail.     • Copiez les clés d'accès de l'espace de travail.     • Gérez vos travaux quantiques.     • Enregistrez vos programmes et résultats.     • Choisissez des fournisseurs et des plans d'informatique quantique.	• Développer des programmes quantiques     • Visualisez les circuits quantiques et les résultats.     • Calculez les estimations des ressources pour vos programmes.	• Un abonnement Azure.     • Un espace de travail Azure Quantum.

## Ressources d'apprentissage Q#

Pour découvrir et explorer le langage de programmation Q#, utilisez les ressources suivantes :

- [Parcours d'apprentissage Azure Quantum](#) : si vous êtes intéressé par l'informatique quantique, mais que vous ne savez pas où commencer, prenez ce parcours d'apprentissage. Grâce à une série de modules interactifs, vous allez découvrir l'informatique quantique et comment développer des solutions quantiques dans Azure Quantum à l'aide de Q# et du QDK.
- [Katas quantiques](#) : Apprenez l'informatique quantique et la programmation grâce à ces didacticiels à votre rythme, chacun accompagné d'une théorie pertinente et d'exercices en Q# pour tester vos connaissances.
- [Exemples de code](#)  Q# : Créez votre première solution quantique avec ces exemples Q# prêts à l'emploi. Ils couvrent quatre domaines : algorithmes quantiques, estimation des ressources, constructions de langage et notebooks Jupyter.
- [Terrain de jeu](#)  QDK : Explorez les algorithmes quantiques courants écrits en Q#. Le terrain de jeu est hébergé sur VS Code pour le web et est préconfiguré avec le QDK. Vous n'avez donc pas besoin d'installer quoi que ce soit.

## Contenu connexe

- [Configurer le Kit de développement Microsoft Quantum](#)
- [Démarrage rapide : Créer votre premier programme Q#](#)
- [Référence : extension QDK pour Visual Studio Code](#)



# Guide pratique pour créer et gérer Q# des projets et des bibliothèques personnalisées

Dans cet article, vous allez apprendre à créer, gérer et partager Q# des projets. Un Q# projet est une structure de dossiers avec plusieurs Q# fichiers qui peuvent accéder aux opérations et fonctions des autres. Les projets vous aident à organiser logiquement votre code source. Vous pouvez également utiliser des projets en tant que bibliothèques personnalisées auxquelles vous pouvez accéder à partir de sources externes.

## Prérequis

- Espace de travail Azure Quantum dans votre abonnement Azure. Pour créer un espace de travail, consultez [Créer un espace de travail Azure Quantum](#).
- Visual Studio Code (VS Code) avec les extensions [Microsoft Quantum Development Kit \(QDK\)](#) et [Python](#) installées.
- Pour publier votre projet externe dans un référentiel de GitHub public, vous devez disposer d'un compte GitHub.

Pour exécuter des programmes Python, vous avez également besoin des éléments suivants :

- Environnement Python avec [Python et Pip](#) installés.
- Bibliothèque `qdk` Python avec l'option `azure` supplémentaire.

Bash

```
python -m pip install --upgrade "qdk[azure]"
```

## Fonctionnement des Q# projets

Un Q# projet contient un Q# fichier manifeste nommé `qsharp.json`, et un ou plusieurs `.qs` fichiers et `.qsc` fichiers dans une structure de dossiers spécifiée. Vous pouvez créer un Q# projet manuellement ou directement dans VS Code.

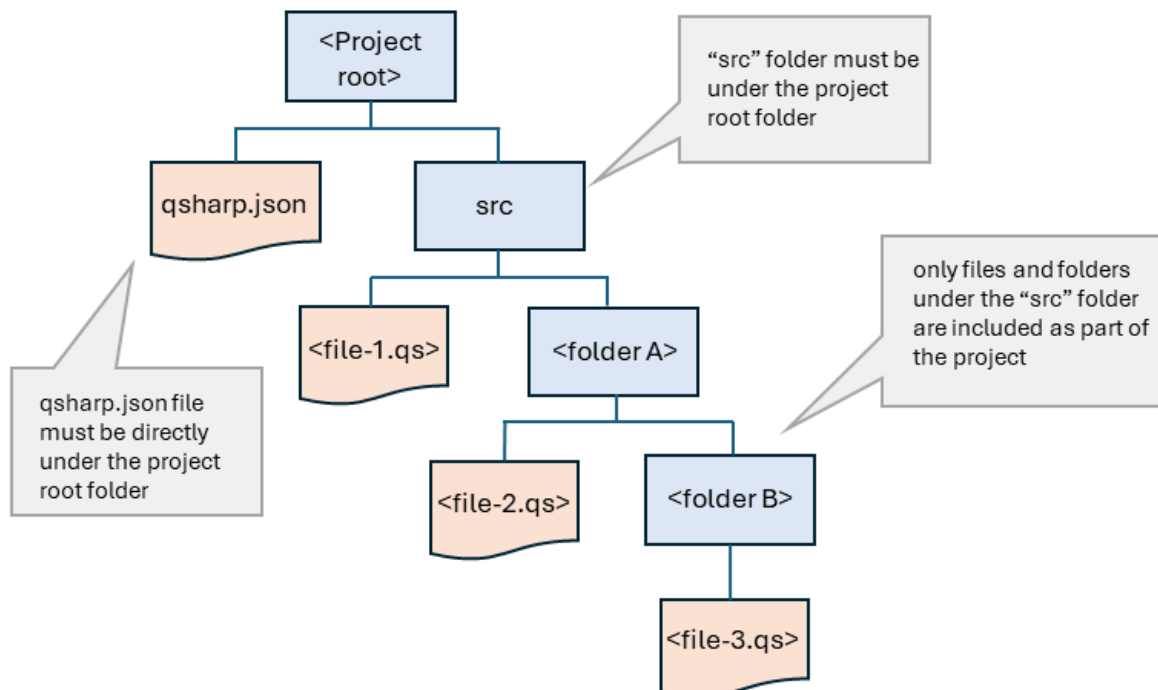
Lorsque vous ouvrez un fichier `.qs` ou `.qsc` dans VS Code, le compilateur recherche dans la hiérarchie des dossiers environnants le fichier manifeste et détermine l'étendue du projet. Si le compilateur ne trouve pas de fichier manifeste, le compilateur fonctionne en mode fichier unique.

Lorsque vous définissez le fichier `project_root` dans un fichier Jupyter Notebook ou Python, le compilateur recherche le fichier manifeste dans le dossier `project_root`.

Un projet Q# externe est un projet standard Q# situé dans un autre répertoire ou sur un référentiel de GitHub public et agit comme une bibliothèque personnalisée. Un projet externe utilise `export` des instructions pour définir les fonctions et les opérations accessibles par des programmes externes. Les programmes définissent le projet externe comme une dépendance dans leur fichier manifeste et utilisent `import` des instructions pour accéder aux éléments du projet externe, tels que les opérations, les fonctions, les structs et les espaces de noms. Pour plus d'informations, consultez [Utilisation de projets en tant que dépendances](#) externes.

## Définir un Q# projet

Un Q# projet est défini par la présence d'un `qsharp.json` fichier manifeste et d'un `src` dossier, dont les deux doivent se trouver dans le dossier racine du projet. Le `src` dossier contient les Q# fichiers sources. Pour Q# les programmes et les projets externes, le Q# compilateur détecte automatiquement le dossier du projet. Pour les programmes Python et les fichiers Jupyter Notebook, vous devez [spécifier le dossier de projet Q#](#) avec un appel `qsharp.init`. Toutefois, la structure de dossiers d'un Q# projet est la même pour tous les types de programmes.



## Définir le dossier de projet des programmes Q#

Lorsque vous ouvrez un fichier dans `.qs`, le VS Code compilateur effectue une Q# recherche vers le haut dans la structure de dossiers d'un fichier manifeste. Si le compilateur trouve un fichier manifeste, le compilateur inclut tous les Q# fichiers dans le `/src` répertoire et ses sous-répertoires. Les éléments définis dans chaque fichier deviennent disponibles pour tous les autres fichiers du projet.

Par exemple, considérez la structure de dossiers suivante :

- **Projet\_de\_Téléportation**
  - `qsharp.json`
  - `src`
    - `Main.qs`
    - **TeleportOperations**
      - `TeleportLib.qs`
      - **PrepareState**
        - `PrepareStateLib.qs`

Lorsque vous ouvrez le fichier `/src/TeleportOperation/PrepareState/PrepareStateLib.qs`, le Q# compilateur effectue les opérations suivantes :

1. Vérifie `/src/TeleportOperation/PrepareState/` pour `qsharp.json`.
2. Vérifie `/src/TeleportOperation` pour `qsharp.json`.
3. Vérifie `/src` pour `qsharp.json`.
4. Vérifie `/Teleportation_project` pour `qsharp.json` et trouve le fichier.
5. Établit `/Teleportation_project` comme répertoire racine du projet et inclut tous les fichiers `.qs` et `.qsc` du répertoire `/src` dans le projet. Si le fichier manifeste.

### ⚠ Remarque

Si vous incluez des références explicites à `.qs` des chemins d'accès aux fichiers `.qsc` dans `qsharp.json`, le compilateur charge ces fichiers et ne passe pas par le processus de découverte automatique. Les références de chemin d'accès de fichier explicites sont nécessaires uniquement lorsque vous définissez une bibliothèque à charger à partir d'une référence Git.

# Créer un fichier manifeste

Un fichier manifeste est un fichier JSON nommé `qsharp.json` qui peut inclure des champs, `author` et `license` facultatifs `lints`. Le fichier manifeste minimum viable est la chaîne `{}`. Lorsque vous créez un Q# projet dans VS Code, un fichier manifeste minimal est créé pour vous.

JSON
<pre>{}</pre>

## Exemples de fichiers manifestes

Les exemples suivants montrent comment les fichiers manifestes définissent l'étendue de votre Q# projet.

- Dans cet exemple, `author` est le seul champ spécifié. Tous les `.qs` fichiers de ce répertoire et ses sous-répertoires sont donc inclus dans le Q# projet.

JSON
<pre>{   "author": "Microsoft" }</pre>

- Dans un Q# projet, vous pouvez également utiliser le fichier manifeste pour affiner les VS Code Q# paramètres Linter. Par défaut, les trois règles Linter sont les suivantes :
  - `needlessParens`: default = `allow`
  - `divisionByZero`: default = `warn`
  - `redundantSemicolons`: default = `warn`

Vous pouvez définir chaque règle dans le fichier manifeste sur `allow`, `warn` ou `error`. Par exemple:

JSON
<pre>{   "author": "Microsoft",   "lints": [     {       "lint": "needlessParens",       "level": "allow"     }   ] }</pre>

```

 },
 {
 "lint": "redundantSemicolons",
 "level": "warn"
 },
 {
 "lint": "divisionByZero",
 "level": "error"
 }
]
}

```

- Vous pouvez également utiliser le fichier manifeste pour définir un projet externe Q# en tant que dépendance et accéder à distance aux opérations et fonctions dans ce projet externe. Pour plus d'informations, consultez [Utilisation de projets en tant que dépendances externes](#).

## Q# exigences et propriétés du projet

Les exigences et configurations suivantes s'appliquent à tous les Q# projets.

- Tous les `.qs` fichiers que vous souhaitez inclure dans le projet doivent se trouver sous un dossier nommé `src`, qui doit se trouver sous le dossier racine du Q# projet. Lorsque vous créez un Q# projet dans VS Code, le `/src` dossier est automatiquement créé.
- Le fichier manifeste doit être au même niveau que le `src` dossier. Lorsque vous créez un Q# projet dans VS Code, un fichier manifeste minimal est automatiquement créé.
- Utilisez des `import` instructions pour référencer des opérations et des fonctions à partir d'autres fichiers du projet.

**Q#**

```

import MyMathLib.*; //imports all the callables in the MyMathLib namespace

...

Multiply(x,y);

```

Vous pouvez également les référencer individuellement avec l'espace de noms.

**Q#**

```

MyMathLib.Multiply(x,y);

```

## Pour les projets Q# uniquement

- Vous pouvez définir une opération de point d'entrée dans un `.qs` seul fichier d'un Q# projet, qui est l'opération `Main()` par défaut.
- Vous devez placer le `.qs` fichier avec la définition du point d'entrée au niveau d'un répertoire de projet sous le fichier manifeste.
- Toutes les opérations et fonctions du projet Q# qui sont mises en cache et affichées en mode texte prédictif à partir d'un affichage `.qs` dans VS Code.
- Si l'espace de noms d'une opération ou d'une fonction sélectionnée n'est pas encore importé, VS Code ajoute automatiquement l'instruction nécessaire `import` .

## Guide pratique pour créer un Q# projet

Pour créer un Q# projet, procédez comme suit :

1. Dans l'Explorateur VS Code de fichiers, accédez au dossier que vous souhaitez utiliser comme dossier racine pour le Q# projet.
2. Ouvrez le menu **Affichage** et choisissez **Palette de commandes**.
3. Entrée **QDK: Créer un Q# projet**. VS Code crée un fichier manifeste minimal dans le dossier et ajoute un `/src` dossier avec un `Main.qs` fichier de modèle.
4. Modifiez le fichier manifeste de votre projet. Consultez les [exemples](#) de fichier manifeste.
5. Ajoutez et organisez vos Q# fichiers sources sous le `/src` dossier.
6. Si vous accédez au projet Q# à partir d'un programme Python ou Jupyter Notebook, définissez le chemin du dossier `root` avec `qsharp.init`. Cet exemple suppose que votre programme se trouve dans le `/src` dossier du Q# projet :

### Python

```
qsharp.init(project_root = '../Teleportation_project')
```

7. Si vous utilisez uniquement des fichiers Q# dans VS Code, alors le compilateur recherche un fichier manifeste lorsque vous ouvrez un fichier Q# et détermine le dossier racine du projet. Ensuite, le compilateur analyse `/src` et ses sous-répertoires pour `.qs` et `.qsc` fichiers.

### ❗ Remarque



Vous pouvez créer manuellement le fichier manifeste et le dossier `/src` à la place.

## Exemple de projet

Ce programme de téléportation quantique est un exemple de Q# projet qui s'exécute sur le simulateur local dans VS Code. Pour exécuter le programme sur Azure Quantum matériel ou des simulateurs tiers, consultez [Démarrer avec les programmes Q# et VS Code](#) pour obtenir les étapes nécessaires pour compiler votre programme et vous connecter à votre espace de travail Azure Quantum.

Cet exemple présente la structure de répertoires suivante :

- **Projet\_de\_Téléportation**
  - *qsharp.json*
  - **src**
    - *Main.qs*
    - **TeleportOperations**
      - *TeleportLib.qs*
      - **PrepareState**
        - *PrepareStateLib.qs*

Le fichier manifeste contient les champs d'auteur et de licence :

### JSON

```
{
 "author": "Microsoft",
 "license": "MIT"
}
```

## Q# fichiers sources

Le fichier principal `Main.qs` contient le point d'entrée et fait référence à l'espace de noms `TeleportOperations.TeleportLib` à partir de `TeleportLib.qs`.

### Q#

```
import TeleportOperations.TeleportLib.Teleport; // references the Teleport
operation from TeleportLib.qs

operation Main() : Unit {
 use msg = Qubit();
```

```

 use target = Qubit();

 H(msg);
 Teleport(msg, target); // calls the Teleport() operation from TeleportLib.qs
 H(target);

 if M(target) == Zero {
 Message("Teleported successfully!");

 Reset(msg);
 Reset(target);
 }
}

```

Le `TeleportLib.qs` fichier définit l'opération `Teleport` et appelle l'opération `PrepareBellPair` à partir du `PrepareStateLib.qs` fichier.

Q#

```

import TeleportOperations.PrepareState.PrepareStateLib.*; // references the
namespace in PrepareStateLib.qs

operation Teleport(msg : Qubit, target : Qubit) : Unit {
 use here = Qubit();

 PrepareBellPair(here, target); // calls the PrepareBellPair() operation from
PrepareStateLib.qs
 Adjoint PrepareBellPair(msg, here);

 if M(msg) == One { Z(target); }
 if M(here) == One { X(target); }

 Reset(here);
}

```

Le `PrepareStateLib.qs` fichier contient une opération réutilisable standard pour créer une paire Bell.

Q#

```

operation PrepareBellPair(left : Qubit, right : Qubit) : Unit is Adj + Ctl {
 H(left);
 CNOT(left, right);
}

```

## Exécuter les programmes

Choisissez l'onglet de l'environnement dans lequel vous exécutez votre programme.

Exécuter du Q# code dans VS Code

Pour exécuter ce programme, ouvrez le `Main.qs` fichier dans VS Code et choisissez **Exécuter**.

## Configurer des Q# projets en tant que dépendances externes

Vous pouvez configurer Q# des projets en tant que dépendance externe pour d'autres projets, similaires à une bibliothèque, pour rendre les fonctions et les opérations dans le projet externe Q# disponibles pour d'autres Q# projets. Une dépendance externe peut résider sur un partage de lecteur ou être publiée dans un référentiel de GitHub public.

Pour utiliser un Q# projet comme dépendance externe, vous devez :

- Ajoutez le projet externe en tant que dépendance dans le fichier manifeste du projet appelant.
- Si le projet externe est publié sur GitHub, ajoutez la propriété `files` au fichier manifeste du projet externe.
- Ajoutez des `export` déclarations de code au projet externe.
- Ajoutez des `import` instructions au projet appelant.

## Configurer les fichiers manifestes

Les projets Q# externes peuvent résider sur un partage de lecteur local ou réseau, ou vous pouvez les publier dans un référentiel de GitHub public.

### Le fichier manifeste du projet appelant

Pour ajouter une dépendance à un projet externe sur un partage de lecteurs, définissez la dépendance dans le fichier manifeste du projet appelant.

JSON

```
{
 "author": "Microsoft",
 "license": "MIT",
 "dependencies": {
 "MyDependency": {
 "path": "/path/to/project/folder/on/disk"
 }
 }
}
```

```
}
}
```

Dans le fichier manifeste précédent, `MyDependency` est une chaîne définie par l'utilisateur qui identifie l'espace de noms lorsque vous appelez une opération. Par exemple, si vous créez une dépendance nommée `MyMathFunctions`, vous pouvez appeler une fonction à partir de cette dépendance avec `MyMathFunctions.MyFunction()`.

Pour ajouter une dépendance à un projet publié dans un référentiel de GitHub public, utilisez l'exemple de fichier manifeste suivant :

#### JSON

```
{
 "author": "Microsoft",
 "dependencies": {
 "MyDependency": {
 "github": {
 "owner": "GitHubUser",
 "repo": "GitHubRepoName",
 "ref": "CommitHash",
 "path": "/path/to/dependency"
 }
 }
 }
}
```

#### ⓘ Remarque

Pour les dépendances GitHub, `ref` fait référence à un GitHub [refspec](#). Microsoft recommande d'utiliser toujours un hachage de validation pour pouvoir vous appuyer sur une version spécifique de votre dépendance.

## Fichier manifeste du projet externe

Si votre projet Q# externe est publié dans un référentiel de GitHub public, vous devez ajouter la propriété `files` au fichier manifeste du projet externe, y compris tous les fichiers utilisés par le projet.

#### JSON

```
{
 "author": "Microsoft",
 "license": "MIT",
 "files": {
 "src": "src",
 "test": "test",
 "build": "build",
 "doc": "doc",
 "examples": "examples",
 "tools": "tools",
 "scripts": "scripts",
 "resources": "resources",
 "images": "images",
 "sounds": "sounds",
 "videos": "videos",
 "other": "other"
 }
}
```

```
"files": ["src/MyMathFunctions.qs", "src/Strings/MyStringFunctions.qs"]
}
```

La `files` propriété est facultative pour un projet externe que vous importez avec `"path"` une importation basée sur un chemin de fichier local. La propriété `files` est requise uniquement pour les projets publiés dans GitHub.

## Utiliser l'instruction `export`

Pour rendre les fonctions et les opérations dans un projet externe accessibles aux projets appelants, utilisez l'instruction `export`. Vous pouvez exporter l'un ou l'ensemble des callables dans le fichier. Vous ne pouvez pas utiliser la syntaxe des caractères génériques, vous devez donc spécifier chaque élément callable que vous souhaitez exporter.

Q#

```
operation Operation_A() : Unit {
...
}
operation Operation_B() : Unit {
...
}

// makes just Operation_A available to calling programs
export Operation_A;

// makes Operation_A and Operation_B available to calling programs
export Operation_A, Operation_B, etc.;

// makes Operation_A available as 'OpA'
export Operation_A as OpA;
```

## Utiliser l'instruction `import`

Pour rendre les éléments d'une dépendance externe disponibles, utilisez les déclarations `import` du programme appelant. L'instruction `import` utilise l'espace de noms que vous définissez pour la dépendance dans le fichier manifeste.

Par exemple, considérez la dépendance dans le fichier manifeste suivant :

JSON

```
{
 "author": "Microsoft",
 "license": "MIT",
 "dependencies": {
```

```
"MyMathFunctions": {
 "path": "/path/to/project/folder/on/disk"
}
}
```

Importez les fonctions appelables avec le code suivant :

Q#

```
import MyMathFunctions.MyFunction; // imports "MyFunction()" from the namespace

...
```

L'instruction `import` prend également en charge la syntaxe de génériques de caractères et les alias.

Q#

```
// imports all items from the "MyMathFunctions" namespace
import MyMathFunctions.*;

// imports the namespace as "Math", all items are accessible via "Math.<callable>"
import MyMathFunctions as Math;

// imports a single item, available in the local scope as "Add"
import MyMathFunctions.MyFunction as Add;

// imports can be combined on one line
import MyMathFunctions.MyFunction, MyMathFunctions.AnotherFunction as Multiply;
```

## Exemple de projet externe

Pour cet exemple, utilisez le même programme de téléportation que l'exemple précédent, mais séparez le programme d'appel et les entités appelables dans différents projets.

1. Créez deux dossiers sur votre lecteur local, par exemple `Project_A` et `Project_B`.
2. Créez un Q# projet dans chaque dossier. Pour plus d'informations, consultez les étapes de création [d'un Q# projet](#).
3. Dans `Project_A`, le programme appelant, collez le code suivant dans le fichier manifeste, mais modifiez le chemin si nécessaire pour `Project_B`:

JSON

```
{
 "author": "Microsoft",
 "license": "MIT",
 "dependencies": {
 "MyTeleportLib": {
 "path": "/Project_B"
 }
 }
}
```

4. Dans `Project_A`, copiez le code suivant dans `Main.qs`:

Q#

```
import MyTeleportLib.Teleport; // imports the Teleport operation from the
MyTeleportLib namespace defined in the manifest file

operation Main() : Unit {
 use msg = Qubit();
 use target = Qubit();

 H(msg);
 Teleport(msg, target); // calls the Teleport() operation from the
MyTeleportLib namespace
 H(target);

 if M(target) == Zero {
 Message("Teleported successfully!");
 }

 Reset(msg);
 Reset(target);
}
```

5. Dans `Project_B`, copiez le code suivant dans `Main.qs`:

Q#

```
operation Teleport(msg : Qubit, target : Qubit) : Unit {
 use here = Qubit();

 PrepareBellPair(here, target);
 Adjoint PrepareBellPair(msg, here);

 if M(msg) == One { Z(target); }
 if M(here) == One { X(target); }

 Reset(here);
}

operation PrepareBellPair(left : Qubit, right : Qubit) : Unit is Adj + Ctl {
```

```

 H(left);
 CNOT(left, right);
 }

 export Teleport; // makes the Teleport operation available to external
 programs

```

### ❗ Remarque

Vous n'avez pas besoin d'exporter l'opération `PrepareBellPair` si votre programme dans `Project_A` n'appelle pas directement cette opération. L'opération `PrepareBellPair` est déjà accessible par l'opération `Teleport`, car `PrepareBellPair` elle se trouve dans l'étendue locale de `Project_B`.

6. Pour exécuter le programme, ouvrez `/Project_A/Main.qs` VS Code et choisissez **Exécuter**.

## Projets et espaces de noms implicites

Dans Q# les projets, si vous ne spécifiez pas d'espace de noms dans un `.qs` programme, le compilateur utilise le nom de fichier comme espace de noms. Ensuite, lorsque vous référencez un appelant à partir d'une dépendance externe, vous utilisez la syntaxe `<dependencyName>.<namespace>.<callable>`. Toutefois, si le fichier est nommé `Main.qs`, le compilateur part du principe que l'espace de noms et la syntaxe appelante sont `<dependencyName>.<callable>`. Par exemple : `import MyTeleportLib.Teleport`.

Étant donné que vous avez peut-être plusieurs fichiers projet, vous devez prendre en compte la syntaxe correcte lorsque vous référencez des fonctions appelables. Par exemple, considérez un projet avec la structure de fichiers suivante :

- `/src`
  - `Main.qs`
  - `MathFunctions.qs`

Le code suivant effectue des appels à la dépendance externe :

Q#

```

import MyTeleportLib.MyFunction; // "Main" namespace is implied

import MyTeleportLib.MathFunctions.MyFunction; // "Math" namespace must be explicit

```



Pour plus d'informations sur le comportement de l'espace de noms, consultez [Espaces de noms utilisateur](#).

---

Last updated on 29/04/2026

# Comment déboguer et tester votre code quantique dans le QDK

Les tests et le débogage sont tout aussi importants dans la programmation quantique que dans la programmation classique. Cet article explique comment déboguer et tester vos programmes quantiques avec le Microsoft Quantum Development Kit (QDK) dans Visual Studio Code (VS Code) et Jupyter Notebook.

## Déboguer votre code quantique

Le QDK fournit plusieurs outils pour déboguer votre code. Si vous écrivez Q# ou ouvrez des programmes OpenQASM dans VS Code, vous pouvez utiliser le débogueur VS Code pour définir des points d'arrêt dans vos programmes et analyser votre code. Le QDK fournit également un ensemble de fonctions de vidage que vous pouvez utiliser pour obtenir des informations à différents points de votre programme.

## Comment utiliser le débogueur VS Code

Avec l'extension QDK dans VS Code, vous pouvez utiliser le débogueur pour parcourir votre code et dans chaque fonction ou opération, suivre les valeurs des variables locales et suivre les états quantiques des qubits.

L'exemple suivant montre comment utiliser le débogueur avec un programme Q#. Pour obtenir des informations complètes sur les débogueurs VS Code, consultez [Débogage](#) sur le site web VS Code.

1. Dans VS Code, créez et enregistrez un fichier `.qs` avec le code suivant :

Q#

```
import Std.Arrays.*;
import Std.Convert.*;

operation Main() : Result {
 use qubit = Qubit();
 H(qubit);
 let result = M(qubit);
 Reset(qubit);
 return result;
}
```

2. Sur la ligne 6, `H(qubit)` cliquez à gauche du numéro de ligne pour définir un point d'arrêt. Un cercle rouge s'affiche.
3. Dans la barre latérale principale, choisissez l'icône du débogueur pour ouvrir le volet débogueur, puis sélectionnez **Exécuter et Déboguer**. La barre de contrôle du débogueur s'ouvre.
4. Appuyez sur **F5** pour démarrer le débogueur et passer au point d'arrêt. Dans le menu **Variables** du volet débogueur : développez la liste déroulante **État Quantique** pour voir que le qubit a été initialisé dans l'état  $|0\rangle$ .
5. Appuyez sur **F11** pour passer à l'opération `H`. Le code source de l'opération `H` s'affiche. Notez que l'état quantique passe à une superposition à mesure que vous parcourez l'opération `H`.
6. Appuyez sur **F10** pour passer au-dessus de l'opération `M`. Notez que l'état quantique est résolu en  $|0\rangle$  ou  $|1\rangle$  après la mesure. La `result` variable est répertoriée sous **Locals**.
7. Appuyez de nouveau sur **F10** pour sauter par-dessus l'opération `Reset`. Notez que l'état quantique est réinitialisé à  $|0\rangle$ .

Lorsque vous avez terminé d'explorer le débogueur, appuyez sur **Maj + F5** pour quitter le débogueur.

#### ⚠ Remarque

Le débogueur VS Code fonctionne uniquement avec les fichiers Q# (`.qs`) et les fichiers OpenQASM (`.qasm`). Vous ne pouvez pas utiliser le débogueur VS Code sur Q# cellules dans un Jupyter Notebook.

## Comment déboguer avec les fonctions de vidage de QDK

Le QDK fournit plusieurs Q# et fonctions Python qui enregistrent des informations sur l'état actuel de votre programme lorsque vous appelez ces fonctions. Utilisez les informations provenant de ces fonctions de vidage pour vérifier si votre programme se comporte comme prévu.

### La Q# `DumpMachine` fonction

`DumpMachine` est une Q# fonction qui vous permet d'exporter des informations sur l'état actuel du système qubit sur la console pendant l'exécution de votre programme. `DumpMachine` n'arrête pas ou n'interrompt pas votre programme pendant l'exécution.

L'exemple suivant appelle `DumpMachine` à deux points dans un Q# programme et explore la sortie.

1. Dans VS Code, créez et enregistrez un fichier `.qs` avec le code suivant :

Q#

```
import Std.Diagnostics.*;

operation Main() : Unit {
 use qubits = Qubit[2];
 X(qubits[1]);
 H(qubits[1]);
 DumpMachine();

 R1Frac(1, 2, qubits[0]);
 R1Frac(1, 3, qubits[1]);
 DumpMachine();

 ResetAll(qubits);
}
```

2. Appuyez sur **Ctrl+ Maj + Y** pour ouvrir la **console de débogage**.
3. Appuyez sur **Ctrl + F5** pour exécuter votre programme. La sortie `DumpMachine` suivante s'affiche dans la **console de débogage** :

Sortie

Basis	Amplitude	Probability	Phase
00>	0.7071+0.0000i	50.0000%	0.0000
01>	-0.7071+0.0000i	50.0000%	-3.1416

Basis	Amplitude	Probability	Phase
00>	0.7071+0.0000i	50.0000%	0.0000
01>	-0.6533-0.2706i	50.0000%	-2.7489

La sortie de `DumpMachine` montre comment l'état des systèmes de qubits change après chaque ensemble de portes.

❗ Remarque

La sortie de `DumpMachine` utilise l'ordre des octets big-endian.

## La fonction Python `dump_machine`

La `dump_machine` fonction est une fonction du `qsharp` package Python. Cette fonction retourne le nombre actuel de qubits alloués et un dictionnaire qui contient les amplitudes d'état éparses du système qubit.

L'exemple suivant exécute le même programme que l'exemple précédent `DumpMachine` , mais dans un notebook Jupyter au lieu d'un `.qs` fichier.

1. Dans VS Code, appuyez sur **Ctrl + Maj + P** pour ouvrir la **palette de commandes**.
2. Entrez **Créer : Nouveau bloc-notes Jupyter** , puis appuyez sur **Entrée**. Un nouvel onglet Jupyter Notebook s'ouvre.
3. Dans la première cellule, copiez et exécutez le code suivant :

Python

```
from qdk import qsharp
```

4. Créez une cellule de code, puis copiez et exécutez le code suivant Q# :

Q#

```
%%qsharp

use qubits = Qubit[2];
X(qubits[0]);
H(qubits[1]);
```

5. Créez une nouvelle cellule de code. Copiez et exécutez le code Python suivant pour afficher l'état qubit à ce stade du programme :

Python

```
dump = qsharp.dump_machine()
dump
```

La `dump_machine` fonction affiche la sortie suivante :

Sortie

Basis State ( $ \psi_1 \dots \psi_n\rangle$ )	Amplitude	Measurement Probability	Phase
$ 10\rangle$	$0.7071+0.0000i$	50.0000%	$\uparrow$ 0.0000
$ 11\rangle$	$0.7071+0.0000i$	50.0000%	$\uparrow$ 0.0000

6. Créez une cellule de code, puis copiez et exécutez le code suivant Q# :

```
Q#

%%qsharp

R1Frac(1, 2, qubits[0]);
R1Frac(1, 3, qubits[1]);
```

7. Créez une nouvelle cellule de code. Copiez et exécutez le code Python suivant pour afficher l'état qubit à ce stade du programme :

```
Python

dump = qsharp.dump_machine()
dump
```

La `dump_machine` fonction affiche la sortie suivante :

Sortie												
<table> <thead> <tr> <th>Basis State (<math> \psi_1 \dots \psi_n\rangle</math>)</th> <th>Amplitude</th> <th>Measurement Probability</th> <th>Phase</th> </tr> </thead> <tbody> <tr> <td><math> 10\rangle</math></td> <td><math>0.5000+0.5000i</math></td> <td>50.0000%</td> <td><math>\nearrow</math> 0.7854</td> </tr> <tr> <td><math> 11\rangle</math></td> <td><math>0.2706+0.6533i</math></td> <td>50.0000%</td> <td><math>\nearrow</math> 1.1781</td> </tr> </tbody> </table>	Basis State ( $ \psi_1 \dots \psi_n\rangle$ )	Amplitude	Measurement Probability	Phase	$ 10\rangle$	$0.5000+0.5000i$	50.0000%	$\nearrow$ 0.7854	$ 11\rangle$	$0.2706+0.6533i$	50.0000%	$\nearrow$ 1.1781
Basis State ( $ \psi_1 \dots \psi_n\rangle$ )	Amplitude	Measurement Probability	Phase									
$ 10\rangle$	$0.5000+0.5000i$	50.0000%	$\nearrow$ 0.7854									
$ 11\rangle$	$0.2706+0.6533i$	50.0000%	$\nearrow$ 1.1781									

8. Pour imprimer une version `dump_machine` abrégée, créez une cellule et exécutez le code Python suivant :

```
Python

print(dump)
```

9. Pour obtenir le nombre total de qubits dans le système, créez une cellule de code et exécutez le code Python suivant :

```
Python

dump.qubit_count
```

10. Vous pouvez accéder aux amplitudes des états qubits individuels qui ont une amplitude différente de zéro. Par exemple, créez une cellule de code et exécutez le code Python suivant pour obtenir les amplitudes individuelles des états  $|10\rangle$  et  $|11\rangle$  :

#### Python

```
print(dump[2])
print(dump[3])
```

## La fonction `dump_operation`

La `dump_operation` fonction est disponible dans le `qdk.qsharp` module Python. Cette fonction prend deux entrées : une opération ou une définition d'opération Q# en tant que chaîne et le nombre de qubits utilisés dans l'opération. La sortie de `dump_operation` est une liste imbriquée qui représente la matrice carrée de nombres complexes correspondant à l'opération quantique donnée. Les valeurs de matrice sont dans la base de calcul, et chaque sous-liste représente une ligne de la matrice.

L'exemple suivant utilise `dump_operation` pour afficher des informations pour un système de 1 qubit et 2 qubits.

1. Dans VS Code, appuyez sur **Ctrl + Maj + P** pour ouvrir la **palette de commandes**.
2. Entrez **Créer : Nouveau bloc-notes Jupyter** , puis appuyez sur **Entrée**. Un nouvel onglet Jupyter Notebook s'ouvre.
3. Dans la première cellule, copiez et exécutez le code suivant :

#### Python

```
from qdk.qsharp import dump_operation
```

4. Pour afficher les éléments de matrice d'une porte à qubit unique, appelez `dump_operation` et passez 1 pour le nombre de qubits. Par exemple, copiez et exécutez le code Python suivant dans une nouvelle cellule de code pour obtenir les éléments de matrice d'une porte d'identité et une porte Hadamard :

#### Python

```
res = dump_operation("qs => ()", 1)
print("Single-qubit identity gate:\n", res)
print()
```

```
res = dump_operation("qs => H(qs[0])", 1)
print("Single-qubit Hadamard gate:\n", res)
```

5. Vous pouvez également appeler la fonction `qsharp.eval` et ensuite référencer l'opération Q# dans `dump_operation` pour obtenir le même résultat. Par exemple, créez une cellule de code, puis copiez et exécutez le code Python suivant pour imprimer les éléments de matrice pour une porte Hadamard à qubit unique :

#### Python

```
qsharp.eval("operation SingleH(qs : Qubit[]) : Unit { H(qs[0]) }")

res = dump_operation("SingleH", 1)
print("Single-qubit Hadamard gate:\n", res)
```

6. Pour afficher les éléments de matrice d'une porte à deux qubits, appelez `dump_operation` et passez 2 pour le nombre de qubits. Par exemple, copiez et exécutez le code Python suivant dans une nouvelle cellule de code pour obtenir les éléments de matrice d'une opération Ry contrôlée où le deuxième qubit est le target qubit :

#### Python

```
qsharp.eval ("operation ControlRy(qs : Qubit[]) : Unit { Controlled Ry([qs[0]],
(0.5, qs[1])); }")

res = dump_operation("ControlRy", 2)
print("Controlled Ry rotation gate:\n", res)
```

Pour plus d'exemples de test et de débogage de votre code, `dump_operation` consultez [les opérations de test](#) à partir des exemples QDK.

## Tester votre code quantique

Le QDK fournit plusieurs Q# fonctions et opérations que vous pouvez utiliser pour tester votre code lors de son exécution. Vous pouvez également écrire des tests unitaires pour les programmes Q#.

### Expression `fail`

L'expression `fail` termine immédiatement votre programme. Pour incorporer des tests dans votre code, utilisez les `fail` expressions à l'intérieur d'instructions conditionnelles.



L'exemple suivant utilise une `fail` instruction pour tester qu'un tableau qubit contient exactement 3 qubits. Le programme se termine par un message d'erreur lorsque le test ne passe pas.

1. Dans VS Code, créez et enregistrez un fichier `.qs` avec le code suivant :

Q#

```
operation Main() : Unit {
 use qs = Qubit[6];
 let n_qubits = Length(qs);

 if n_qubits != 3 {
 fail $"The system should have 3 qubits, not {n_qubits}.";
 }
}
```

2. Appuyez sur **Ctrl + F5** pour exécuter le programme. Votre programme échoue et la sortie suivante s'affiche dans la **console de débogage** :

Sortie

Error: program failed: The system should have 3 qubits, not 6.

3. Modifiez votre code de `Qubit[6]` à `Qubit[3]`, enregistrez votre fichier, puis appuyez sur **Ctrl + F5** pour réexécuter le programme. Le programme s'exécute sans erreur, car le test réussit.

## La fonction `Fact`

Vous pouvez également utiliser la Q# `Fact` fonction à partir de l'espace `Std.Diagnostics` de noms pour tester votre code. La `Fact` fonction prend une expression booléenne et une chaîne de message d'erreur. Si l'expression booléenne a la valeur true, le test réussit et votre programme continue à s'exécuter. Si l'expression booléenne est false, `Fact` met fin à votre programme et affiche le message d'erreur.

Pour effectuer le même test de longueur de tableau dans votre code précédent, mais avec la fonction `Fact`, procédez comme suit :

1. Dans VS Code, créez et enregistrez un fichier `.qs` avec le code suivant :

Q#

```
import Std.Diagnostics.Fact;
```

```
operation Main() : Unit {
 use qs = Qubit[6];
 let n_qubits = Length(qs);

 Fact(n_qubits == 3, $"The system should have 3 qubits, not {n_qubits}.")
}
```

2. Appuyez sur **Ctrl + F5** pour exécuter le programme. La condition de test dans `Fact` n'est pas passée et le message d'erreur s'affiche dans la **console de débogage**.
3. Modifiez votre code de `Qubit[6]` à `Qubit[3]`, enregistrez votre fichier, puis appuyez sur **Ctrl + F5** pour réexécuter le programme. La condition de test dans `Fact` réussit et votre programme s'exécute sans erreur.

## Écrire des Q# tests unitaires avec l'annotation `@Test()`

Dans Q# les programmes, vous pouvez appliquer l'annotation `@Test()` à un appelant (fonction ou opération) pour transformer l'appelant en test unitaire. Ces tests d'unités s'affichent dans le menu **Test** dans VS Code afin que vous puissiez tirer parti de cette fonctionnalité VS Code. Vous pouvez transformer un appelant en test unitaire uniquement lorsque l'appelant ne prend pas de paramètres d'entrée.

L'exemple suivant encapsule le code de test de longueur du tableau dans une opération et transforme cette opération en un test unitaire :

1. Dans VS Code, créez et enregistrez un fichier `.qs` avec le code suivant :

```
Q#

import Std.Diagnostics.Fact;

@Test()
operation TestCase() : Unit {
 use qs = Qubit[3];
 let n_qubits = Length(qs);

 Fact(n_qubits == 3, $"The system should have 3 qubits, not {n_qubits}.");
}
```

Annotation `@Test()` sur la ligne avant la définition de l'opération `TestCase` transforme l'opération en test unitaire VS Code. Une flèche verte apparaît sur la ligne de définition de l'opération.

2. Choisissez la flèche verte pour exécuter `TestCase` et rapporter les résultats des tests.

3. Pour interagir avec vos tests unitaires dans l'Explorateur de tests de VS Code, choisissez l'icône en forme de flacon **de test** dans la barre latérale principale.
4. Modifiez votre code à partir de `Qubit[3]` et `Qubit[6]` réexécutez le test unitaire pour voir comment les informations de test changent.

Vous pouvez écrire et exécuter Q# des tests unitaires dans VS Code sans opération de point d'entrée dans votre programme.

#### ⚠ Remarque

Les appelables du namespace `Std.Diagnostics` ne sont pas compatibles avec QIR génération. Par conséquent, incluez uniquement des tests unitaires dans le code Q# que vous exécutez sur des simulateurs. Si vous souhaitez générer QIR à partir de votre Q# code, n'incluez pas de tests unitaires dans votre code.

## Les opérations `CheckZero` et `CheckAllZero`

Les opérations `CheckZero` et `CheckAllZero` Q# vérifient si l'état actuel d'un qubit ou array de qubits est  $|0\rangle$ . L'opération `CheckZero` prend un qubit unique et retourne `true` uniquement lorsque le qubit est dans l'état  $|0\rangle$ . Les `CheckAllZero` opérations prennent un tableau qubit et retournent `true` uniquement lorsque tous les qubits du tableau sont dans l'état  $|0\rangle$ . Pour utiliser `CheckZero` et `CheckAllZero`, importez-les à partir de l'espace de noms `Std.Diagnostics`.

L'exemple suivant utilise les deux opérations. Les `CheckZero` teste que l'opération `X` retourne le premier qubit de l'état  $|0\rangle$  à l'état  $|1\rangle$  et l'opération `CheckAllZero` teste que les deux qubits sont réinitialisés à l'état  $|0\rangle$ .

Dans VS Code, créez et enregistrez un nouveau `.qs` fichier avec le code suivant, puis exécutez le programme et examinez la sortie dans la **console Debug**.

Q#

```
import Std.Diagnostics.*;

operation Main() : Unit {
 use qs = Qubit[2];
 X(qs[0]);

 if CheckZero(qs[0]) {
 Message("X operation failed");
 }
 else {
```

```
 Message("X operation succeeded");
 }

 ResetAll(qs);

 if CheckAllZero(qs) {
 Message("Reset operation succeeded");
 }
 else {
 Message("Reset operation failed");
 }
}
```

---

Last updated on 22/06/2026

# Comment visualiser des diagrammes de circuits quantiques avec le QDK

Les diagrammes de circuit quantique sont une représentation visuelle des algorithmes quantiques. Les diagrammes de circuit montrent le flux de qubits à travers un programme quantique, y compris les portes et les mesures que le programme applique aux qubits.

Dans cet article, vous allez apprendre à créer des diagrammes de circuit pour les programmes Q# et OpenQASM avec l'utilisation Microsoft Quantum Development KitQDK (Visual Studio CodeVS Code) et Jupyter Notebook.

Pour plus d'informations sur les diagrammes de circuits quantiques, consultez [les conventions de diagramme de circuit quantique](#).

## Prérequis

Pour créer des diagrammes de circuit à partir de fichiers VS CodeQ# et OpenQASM, installez les éléments suivants :

- Dernière version de [VS Code](#) [↗](#), ou ouvrir [VS Code pour le Web](#) [↗](#).
- La version la plus récente de l'extension [QDK](#) [↗](#) dans VS Code.

Pour créer des diagrammes de circuit à partir de programmes Python, Jupyter Notebookinstallez les éléments suivants :

- L'extension Python [Python](#) [↗](#) et l'extension [Jupyter](#) [↗](#) dans VS Code.
- La dernière version de la `qdk` bibliothèque Python avec l'option `jupyter` en supplément.

Bash

```
pip install --upgrade "qdk[jupyter]"
```

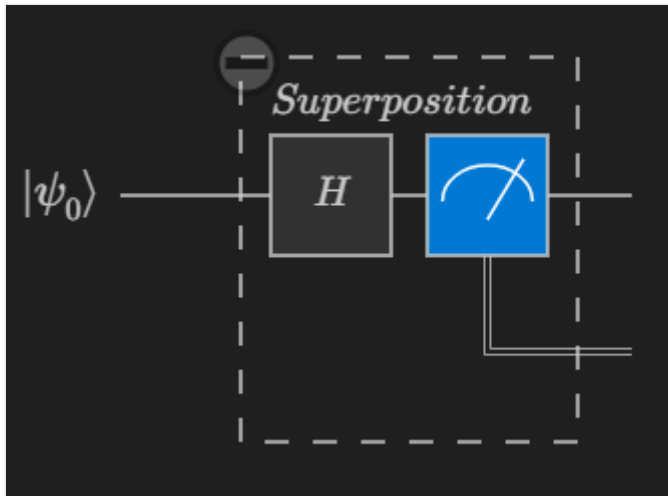
## Visualiser les circuits quantiques dans VS Code

Avec l'extension QDK dans VS Code, vous pouvez créer des diagrammes de circuit pour les fichiers Q# (`.qs`) et OpenQASM (`.qasm`).

Pour afficher un diagramme de circuit dans VS Code, procédez comme suit :

1. Ouvrez un fichier Q# ou OpenQASM dans VS Code, ou [chargez l'un des échantillons quantiques à partir du QDK](#).
2. Choisissez la commande **Circuit** dans le code Lens qui précède votre programme.

La **QDK fenêtre Circuit** s'ouvre et affiche le diagramme de circuit pour votre programme. Par exemple, le diagramme de circuit suivant correspond à un programme qui produit un bit aléatoire. Le circuit place le qubit dans un état de superposition, puis mesure le qubit.



#### 💡 Conseil

Sélectionnez un élément dans le diagramme de circuit pour mettre en surbrillance le code qui crée l'élément de circuit.

## Afficher les diagrammes de circuit pour les opérations Q# individuelles

Pour visualiser le circuit quantique d'une opération individuelle dans un fichier Q#, choisissez la commande **Circuit** à partir de l'objectif de code qui précède l'opération.

```

42 Circuit
43 operation PreparePhiPlus(register : Qubit[]) : Unit {
44 H(register[0]); // |+0>
45 CNOT(register[0], register[1]); // (|00> + |11>)/sqrt(2)
46 }
47
48
49 Circuit
50 operation PreparePhiMinus(register : Qubit[]) : Unit {
51 H(register[0]); // |+0>
52 Z(register[0]); // |-0>
53 CNOT(register[0], register[1]); // (|00> - |11>)/sqrt(2)
54 }

```

## Afficher les diagrammes de circuit lorsque vous déboguez

Lorsque vous utilisez le VS Code débogueur dans un programme Q#, vous pouvez visualiser le circuit quantique en fonction de l'état du programme au point d'arrêt actuel du débogueur.

1. Choisissez la commande **Debug** de CodeLens qui précède le fonctionnement de votre point d'entrée.
2. Dans le volet **Exécuter et déboguer**, développez la liste déroulante **Circuit quantique** dans le menu **VARIABLES**. Le QDK panneau **Circuit** s'ouvre, ce qui montre le circuit pendant que vous parcourez le programme.
3. Définissez des points d'arrêt et parcourez votre code pour voir comment le circuit est mis à jour au fur et à mesure que votre programme s'exécute.

## Visualiser les circuits quantiques dans Jupyter Notebook

Dans Jupyter Notebook, vous pouvez visualiser des circuits quantiques pour les programmes Q# et OpenQASM avec les modules `qdk.qsharp` et `qdk.widgets` Python. Le `widgets` module fournit un widget qui affiche un diagramme de circuit quantique en tant qu'image SVG.

Pour plus d'exemples de génération de diagrammes de circuits dans Jupyter Notebook, consultez l'[exemple de notebook Circuits](#) sur le dépôt QDK GitHub.

## Afficher les diagrammes de circuit pour les programmes Q#

Pour afficher un diagramme de circuit pour un programme Q#, suivez ces étapes :Jupyter Notebook

1. Dans VS Code, ouvrez le menu **Affichage** et choisissez **Palette de commandes**.
2. Entrez **Créer : Nouveau Jupyter Notebook**. Un fichier vide Jupyter Notebook s'ouvre dans un nouvel onglet.
3. Dans la première cellule du notebook, importez le `qdk.qsharp` module.

Python

```
from qdk import qsharp
```

4. Créez une cellule et entrez votre code Q#. Par exemple, le code suivant prépare un état Bell :

Q#

```
%%qsharp

// Prepare a Bell State.
operation BellState() : Unit {
 use register = Qubit[2];
 H(register[0]);
 CNOT(register[0], register[1]);
}
```

5. Pour afficher un diagramme de circuit quantique, passez l'opération Q# à la `qsharp.circuit` fonction. Exécutez le code suivant dans une nouvelle cellule :

Python

```
qsharp.circuit("BellState()")
```

Le résultat se présente ainsi :

Sortie

```
q_0 — H — • —
q_1 ————— X —
```

6. Pour afficher une image SVG du circuit quantique, utilisez le `widgets` module. Créez une cellule, puis exécutez le code suivant pour visualiser le même circuit que celui que vous avez créé dans la cellule précédente.

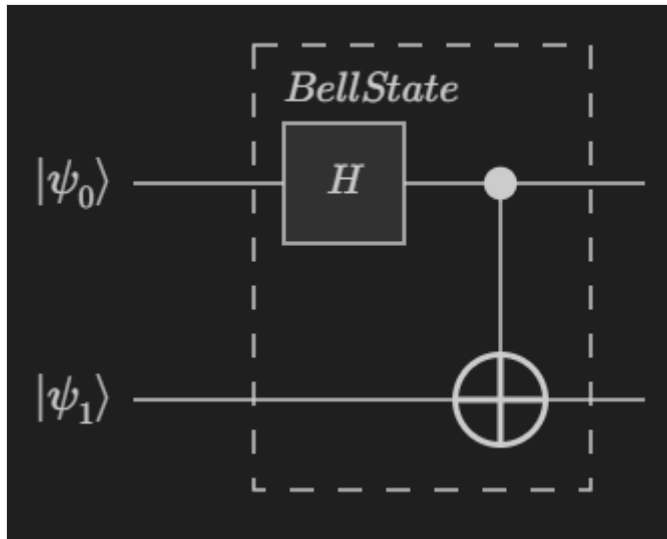


#### Python

```
from qdk.widgets import Circuit

Circuit(qsharp.circuit("BellState()))
```

Le diagramme de circuit ressemble à ceci :



## Afficher les diagrammes de circuit pour les opérations qui prennent des qubits comme entrée

Dans l'exemple d'état Bell précédent, l'opération `BellState` ne prend pas de qubits comme entrée. Si l'opération utilise des qubits ou des tableaux de qubits comme entrée, omettez les parenthèses lorsque vous dessinez des diagrammes de circuit.

Par exemple, procédez comme suit pour dessiner des diagrammes de circuit pour une opération qui prend des qubits :

1. Dans une nouvelle cellule, exécutez le code Q# suivant. Ce code prépare un état Cat.

#### Q#

```
%%qsharp

operation PrepareCatState(register : Qubit[]) : Unit {
 H(register[0]);
 ApplyToEach(CNOT(register[0], _), register[1...]);
}
```

2. Pour dessiner le circuit en tant que diagramme de texte, exécutez le code suivant :

#### Python

```
Circuit(qsharp.circuit(operation="PrepareCatState"))
```

3. Pour afficher le diagramme de circuit en tant qu'image SVG, exécutez le code suivant :

Python

```
Circuit(qsharp.circuit(operation="PrepareCatState"))
```

Lorsque l'opération prend un tableau de qubits (`Qubit[]`), le circuit affiche le tableau comme un registre de deux qubits.

## Afficher les diagrammes de circuit pour les programmes OpenQASM

Pour afficher un diagramme de circuit pour un programme OpenQASM dans Jupyter Notebook procédez comme suit :

1. Dans VS Code, ouvrez le menu **Affichage** et choisissez **Palette de commandes**.
2. Entrez **Créer : Nouveau Jupyter Notebook**. Un fichier vide Jupyter Notebook s'ouvre dans un nouvel onglet.
3. Dans la première cellule du notebook, exécutez le code suivant pour importer les objets nécessaires pour créer et appeler des fonctions OpenQASM avec la QDK bibliothèque Python :

Python

```
from qdk.openqasm import import_openqasm, ProgramType
```

4. Dans une nouvelle cellule, écrivez votre programme OpenQASM dans une chaîne Python et transmettez la chaîne à la `import_openqasm` fonction. Pour appeler le programme dans votre code Python, donnez un nom à la fonction, puis définissez `program_type` à `ProgramType.File`.

Python

```
source = """
 include "stdgates.inc";
 bit[2] c;
 qubit[2] q;
 h q[0];
 cx q[0], q[1];
```

```

c = measure q;
"""

import_openqasm(source, name="bell", program_type=ProgramType.File)

```

5. Dans une nouvelle cellule, importez le programme OpenQASM en tant que fonction Python et appelez le programme pour obtenir les résultats de mesure.

Python

```

from qdk.code.qasm_import import bell

bell()

```

6. Pour dessiner le circuit en tant que diagramme de texte, exécutez le code suivant dans une nouvelle cellule :

Python

```

from qdk.qsharp import circuit

circuit(bell)

```

Le résultat se présente ainsi :

Sortie

```

q_0 — H — • — M —
 | ══
q_1 ——— X — M —
 ══

```

7. Pour afficher le diagramme de circuit en tant qu'image SVG, exécutez le code suivant dans une nouvelle cellule :

Python

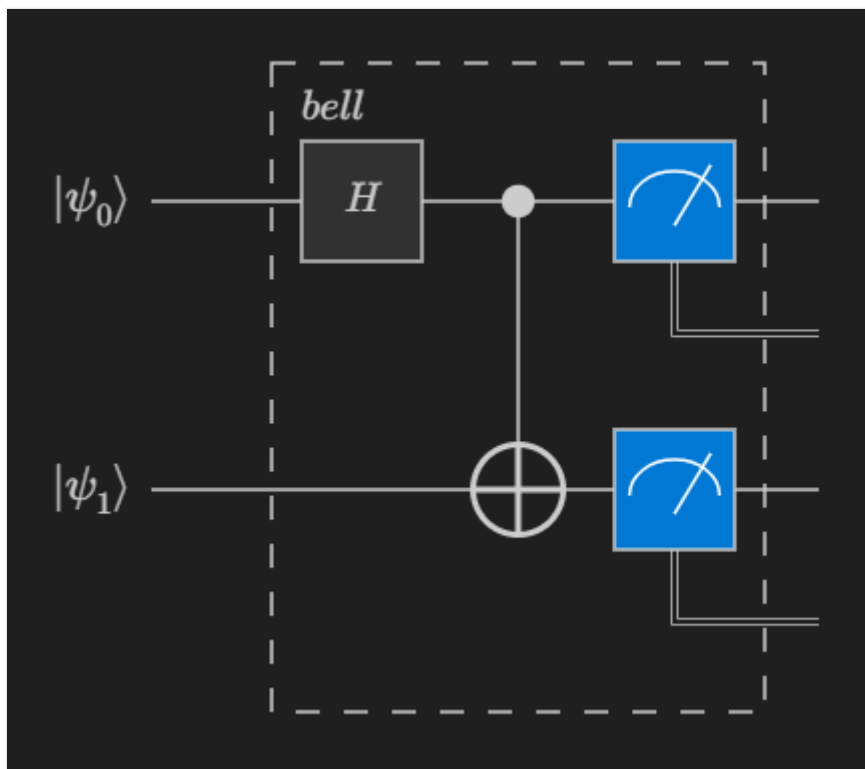
```

from qdk.widgets import Circuit

Circuit(qsharp.circuit(bell))

```

Le diagramme de circuit ressemble à ceci :



#### ⓘ Remarque

Pour les programmes OpenQASM, vous ne pouvez pas afficher les diagrammes de circuit pour des fonctions individuelles. Vous pouvez afficher des diagrammes de circuits uniquement pour l'ensemble du programme.

## Contenu connexe

- [Explorer les diagrammes de circuits dans le QDK](#)
- [Conventions de diagramme de circuit quantique](#)

Last updated on 22/06/2026

# Exécution du programme

Article • 21/03/2025

L'exemple suivant donne un premier aperçu de la façon dont un programme Q# est implémenté :

Q#

```
/// # Sample
/// Bell States
///
/// # Description
/// Bell states or EPR pairs are specific quantum states of two qubits
/// that represent the simplest (and maximal) examples of quantum
/// entanglement.
///
/// This Q# program implements the four different Bell states.

import Std.Diagnostics.*;
import Std.Measurement.*;

operation Main() : (Result, Result)[] {
 // Allocate the two qubits that will be used to create a Bell state.
 use register = Qubit[2];

 // This array contains a label and a preparation operation for each one
 // of the four Bell states.
 let bellStateTuples = [
 ("|Φ+>", PreparePhiPlus),
 ("|Φ->", PreparePhiMinus),
 ("|Ψ+>", PreparePsiPlus),
 ("|Ψ->", PreparePsiMinus)
];

 // Prepare all Bell states, show them using the `DumpMachine` operation
 // and measure the Bell state qubits.
 mutable measurements = [];
 for (label, prepare) in bellStateTuples {
 prepare(register);
 Message($"Bell state {label}:");
 DumpMachine();
 measurements += [(MResetZ(register[0]), MResetZ(register[1]))];
 }
 return measurements;
}

operation PreparePhiPlus(register : Qubit[]) : Unit {
 ResetAll(register); // |00>
 H(register[0]); // |+0>
 CNOT(register[0], register[1]); // 1/sqrt(2)(|00> + |11>)
}
```

```

operation PreparePhiMinus(register : Qubit[]) : Unit {
 ResetAll(register); // |00>
 H(register[0]); // |+0>
 Z(register[0]); // |-0>
 CNOT(register[0], register[1]); // 1/sqrt(2)(|00> - |11>)
}

operation PreparePsiPlus(register : Qubit[]) : Unit {
 ResetAll(register); // |00>
 H(register[0]); // |+0>
 X(register[1]); // |+1>
 CNOT(register[0], register[1]); // 1/sqrt(2)(|01> + |10>)
}

operation PreparePsiMinus(register : Qubit[]) : Unit {
 ResetAll(register); // |00>
 H(register[0]); // |+0>
 Z(register[0]); // |-0>
 X(register[1]); // |-1>
 CNOT(register[0], register[1]); // 1/sqrt(2)(|01> - |10>)
}

```

Ce programme implémente les quatre états Bell de base de l'intrication quantique, et est l'un des exemples de programmes inclus l'extension [Azure Quantum Visual Code](#).

Vous pouvez exécuter le programme à partir du simulateur intégré dans l'extension VS Code QDK et obtenir une sortie standard

#### Output

Message: Bell state  $|\Phi^+\rangle$ :

DumpMachine:

Basis	Amplitude	Probability	Phase
00>	0.7071+0.0000i	50.0000%	0.0000
11>	0.7071+0.0000i	50.0000%	0.0000

Message: Bell state  $|\Phi^-\rangle$ :

DumpMachine:

Basis	Amplitude	Probability	Phase
00>	0.7071+0.0000i	50.0000%	0.0000
11>	-0.7071+0.0000i	50.0000%	-3.1416

Message: Bell state  $|\Psi^+\rangle$ :

DumpMachine:

Basis	Amplitude	Probability	Phase
01⟩	0.7071+0.0000i	50.0000%	0.0000
10⟩	0.7071+0.0000i	50.0000%	0.0000

Message: Bell state  $|\Psi\rangle$ :

DumpMachine:

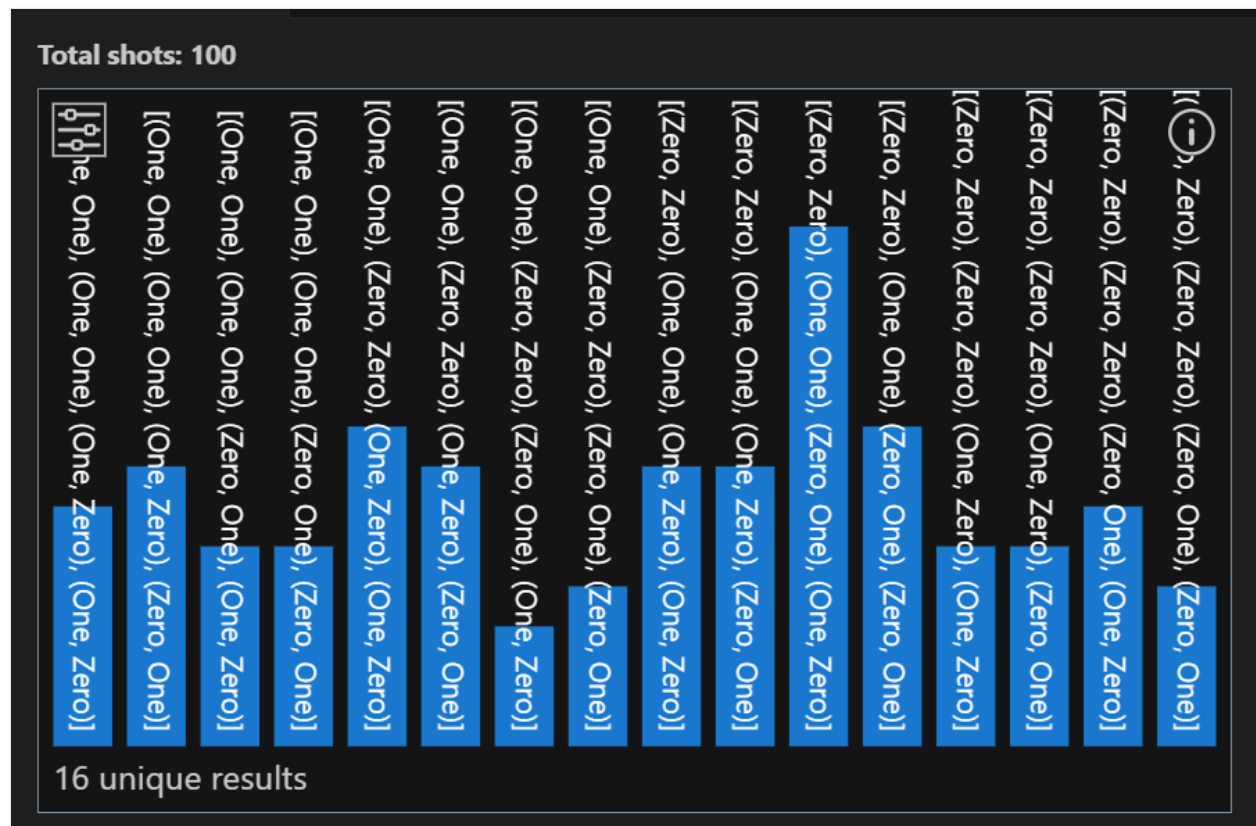
Basis	Amplitude	Probability	Phase
01⟩	0.7071+0.0000i	50.0000%	0.0000
10⟩	-0.7071+0.0000i	50.0000%	-3.1416

Result: "[ (One, One), (Zero, Zero), (One, Zero), (Zero, One)]"

Finished shot 1 of 1

Q# simulation completed.

ou exécutez le simulateur avec une sortie d'histogramme



Pour exécuter le programme sur du matériel quantique, le programme doit d'abord être compilé et soumis à Azure Quantum, qui peuvent tous être effectués à partir de VS Code. Pour connaître le processus complet de bout en bout, consultez [Bien démarrer avec les programmes Q# et Visual Studio Code](#).

# Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)



# Espaces de noms

Article • 18/02/2025

Au niveau supérieur, un programme Q# se compose d'un ensemble d'espaces de noms. Hormis les [commentaires](#), les espaces de noms sont les seuls éléments de niveau supérieur d'un programme Q# et tous les autres éléments doivent se trouver dans un espace de noms. Chaque fichier peut contenir zéro, un ou plusieurs espaces de noms, et chaque espace de noms peut s'étendre sur plusieurs fichiers. Q# ne prend pas en charge les espaces de travail imbriqués.

## ! Notes

Si vous ne déclarez pas explicitement d'espace de noms, le Q# compilateur utilise le nom de fichier comme nom d'espace de noms.

Un bloc d'espace de noms se compose du mot clé `namespace` suivi du nom de l'espace de noms et du contenu du bloc à l'intérieur des accolades `{ }`. Les noms d'espaces de noms sont constitués d'une séquence d'un ou de plusieurs [symboles légaux](#) séparés par un point (`.`). Tandis que les noms d'espaces de noms peuvent contenir des points pour une meilleure lisibilité, Q# ne prend pas en charge les références relatives aux espaces de noms. Par exemple, deux espaces de noms `Foo` et `Foo.Bar` ne sont pas liés, et il n'existe aucune notion de hiérarchie. En particulier, pour une fonction `Baz` définie dans `Foo.Bar`, il n'est *pas* possible d'ouvrir `Foo`, puis d'accéder à cette fonction via `Bar.Baz`.

Les blocs d'espace de noms peuvent contenir des [directives d'importation](#), des [opérations](#), des [fonctions](#) et des [déclarations de type](#). Ces éléments peuvent se produire dans n'importe quel ordre et sont récursifs par défaut, ce qui signifie qu'ils peuvent être déclarés et utilisés dans n'importe quel ordre et peuvent s'appeler eux-mêmes. Il n'est pas nécessaire que la déclaration d'un type ou d'un appel précède son utilisation.

## Directives Import

Par défaut, tous les éléments déclarés dans le même espace de noms sont accessibles sans qualification supplémentaire. Toutefois, les déclarations dans un autre espace de noms peuvent uniquement être utilisées en qualifiant leur nom avec le nom de l'espace de noms auquel elles appartiennent ou en ouvrant cet espace de noms avant son utilisation, comme illustré dans l'exemple suivant.

```
namespace Microsoft.Quantum.Samples {

 import Microsoft.Quantum.Arithmetic.*;
 import Microsoft.Quantum.Arrays.* as Array;

 // ...
}
```

### ⓘ Notes

Pour les espaces de noms dans la bibliothèque standard Q#, la racine de l'espace de noms peut être simplifiée. `Std` Par exemple, l'exemple précédent peut être réécrit comme suit :

```
Q#

import Std.Arithmetic.*;
import Std.Arrays.* as Array;
```

L'exemple utilise une `import` directive pour importer tous les types et appelants déclarés dans l'espace `Microsoft.Quantum.Arithmetic` de noms. Vous pouvez ensuite les référencer par leur nom non qualifié, sauf si ce nom est en conflit avec une déclaration dans le bloc d'espace de noms ou un autre espace de noms ouvert.

Pour éviter de taper le nom complet tout en faisant la distinction entre certains éléments, vous pouvez définir un autre nom ou *un alias* pour un espace de noms particulier. Dans ce cas, vous pouvez qualifier tous les types et appelants déclarés dans cet espace de noms par le nom court défini à la place. L'exemple précédent utilise un alias pour l'espace `Microsoft.Quantum.Arrays` de noms. Vous pouvez ensuite utiliser la fonction `IndexRange`, déclarée dans `Microsoft.Quantum.Arrays`, par exemple, via `Array.IndexRange` ce bloc d'espace de noms.

Que vous ouvrez un espace de noms ou que vous définissez un alias, `import` les directives sont valides dans l'élément d'espace de noms de ce fichier uniquement.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Déclarations de type

Article • 21/03/2025

Q# prend en charge les types de `struct` définis par l'utilisateur. `struct` types sont similaires aux types d'enregistrements en F# ; ils sont immuables, mais prennent en charge une construction [copie et mise à jour](#).

## Types de structs

`struct` types ne peuvent contenir que des éléments nommés et ne prennent pas en charge les éléments anonymes. Toute combinaison d'éléments nommés est prise en charge, même si les éléments ne peuvent pas être imbriqués.

La déclaration suivante, par exemple, définit un struct `Complex` qui a deux éléments nommés `Real` et `Imaginary`, tous deux de type `Double`:

Q#

```
struct Complex {
 Real : Double,
 Imaginary : Double,
}
```

Vous pouvez accéder aux éléments contenus par le biais de leur nom ou par de déconstruction (pour plus d'informations, consultez '[accès aux éléments](#)'). Vous pouvez également accéder à un tuple de tous les éléments où la forme correspond à celle définie dans la déclaration via l'opérateur [unwrap](#).

`struct` types sont utiles pour deux raisons. Tout d'abord, tant que les bibliothèques et les programmes qui utilisent les types définis accèdent aux éléments via leur nom plutôt qu'en déconstruction, le type peut être étendu pour contenir des éléments supplémentaires ultérieurement sans interrompre le code de bibliothèque. En raison de cela, l'accès aux éléments via la déconstruction est déconseillé.

Deuxièmement, Q# vous permet de transmettre l'intention et les attentes d'un type de données spécifique, car il n'existe aucune conversion automatique entre les valeurs de deux types `struct`, même si leurs types d'éléments sont identiques.

## Constructeurs de structs

Le compilateur génère automatiquement des constructeurs pour les nouveaux types `struct` lorsqu'il lit une définition de `struct`. Pour le struct `Complex` dans l'exemple précédent, vous pouvez créer une instance avec

Q#

```
let complexPair = Complex(1.4, 2.1);
```

Vous pouvez également définir des instances avec le mot clé `new`, par exemple

Q#

```
let complexPair = new Complex { Real = 1.4, Imaginary = 2.1 };
```

Vous pouvez copier un struct existant avec la syntaxe `...`

Q#

```
let copyPair = new Complex { ...complexPair };
```

Lors de la copie, vous pouvez spécifier des champs individuels à modifier

Q#

```
let updatedPair = new Complex { ...complexPair, Real = 3.5 };
```

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Déclarations appelables

Article • 21/03/2025

Les déclarations pouvant être appelées ou , déclarées dans une étendue globale sont visibles publiquement par défaut ; autrement dit, ils peuvent être utilisés n'importe où dans le même projet et dans un projet qui fait référence à l'assembly dans lequel ils sont déclarés. [modificateurs d'accès](#) vous permettent de limiter leur visibilité à l'assembly actuel uniquement, de sorte que les détails de l'implémentation peuvent être modifiés ultérieurement sans interrompre le code qui s'appuie sur une bibliothèque spécifique.

Q# prend en charge deux types d'appelants : les opérations et les fonctions. La rubrique [Opérations et Fonctions](#) décrit la distinction entre les deux. Q# prend également en charge la définition de modèles ; par exemple, les implémentations paramétrables de type pour un certain appelant. Pour plus d'informations, consultez [paramétrages de type](#).

## ! Notes

Ces implémentations paramétrables de type ne peuvent pas utiliser de constructions de langage qui s'appuient sur des propriétés particulières des arguments de type ; Il n'existe actuellement aucun moyen d'exprimer des contraintes de type dans Q#, ou de définir des implémentations spécialisées pour des arguments de type particuliers.

## Callables et functors

Q# permet des implémentations spécialisées à des fins spécifiques ; Par exemple, les opérations dans Q# peuvent implicitement ou explicitement définir la prise en charge de certains fonctors , ainsi que les implémentations spécialisées à appeler lorsqu'un fonctor spécifique est appliqué à cet appelant.

Un fonctor, dans un sens, est une fabrique qui définit une nouvelle implémentation pouvant être appelée qui a une relation spécifique avec l'appelant auquel elle a été appliquée. Les fonctors sont plus que les fonctions de niveau supérieur traditionnelles dans lesquelles ils nécessitent l'accès aux détails d'implémentation de l'appelant auquel ils ont été appliqués. Dans ce sens, ils sont similaires à d'autres usines, telles que des modèles. Elles peuvent également être appliquées aux callables paramétrables de type.

Tenez compte de l'opération suivante `ApplyQFT`:

qsharp

```
operation ApplyQFT(qs : Qubit[]) : Unit is Adj + Ctl {
 let length = Length(qs);
 Fact(length >= 1, "ApplyQFT: Length(qs) must be at least 1.");
 for i in length - 1..-1..0 {
 H(qs[i]);
 for j in 0..i - 1 {
 Controlled R1Frac([qs[i]], (1, j + 1, qs[i - j - 1]));
 }
 }
}
```

Cette opération prend un argument de type `Qubit[]` et retourne une valeur de type `Unit`. L'annotation `is Adj + Ctl` dans la déclaration de `ApplyQFT` indique que l'opération prend en charge les `Adjoint` et le fonctor `Controlled`. (Pour plus d'informations, consultez [caractéristiques de l'opération](#)). L'expression `Adjoint ApplyQFT` accède à la spécialisation qui implémente l'adjoint de `ApplyQFT` et `Controlled ApplyQFT` accède à la spécialisation qui implémente la version contrôlée de `ApplyQFT`. Outre l'argument de l'opération d'origine, la version contrôlée d'une opération prend un tableau de qubits de contrôle et applique l'opération d'origine à la condition que tous ces qubits de contrôle soient dans un état  $|1\rangle$ .

En théorie, une opération pour laquelle une version adjointe peut être définie doit également avoir une version contrôlée et vice versa. Toutefois, dans la pratique, il peut être difficile de développer une implémentation pour l'un ou l'autre, en particulier pour les implémentations probabilistes à la suite d'un modèle répétitif jusqu'à la réussite. Pour cette raison, Q# vous permet de déclarer la prise en charge de chaque fonctor individuellement. Toutefois, étant donné que les deux foncteurs commutent, une opération qui définit la prise en charge des deux doit également avoir une implémentation (généralement définie implicitement, c'est-à-dire générée par le compilateur) lorsque les deux foncteurs sont appliqués à l'opération.

Il n'y a pas de foncteurs qui peuvent être appliqués aux fonctions. Les fonctions ont actuellement exactement une implémentation de corps et aucune spécialisation supplémentaire. Par exemple, la déclaration

Q#

```
function Hello (name : String) : String {
 $"Hello, {name}!"
}
```

équivalent à

Q#

```
function Hello (name : String) : String {
 body ... {
 $"Hello, {name}!"
 }
}
```

Ici, `body` spécifie que l'implémentation donnée s'applique au corps par défaut de la fonction `Hello`, ce qui signifie que l'implémentation est appelée lorsqu'aucun foncteurs ou d'autres mécanismes de fabrique n'ont été appliqués avant l'appel. Les trois points de `body ...` correspondent à une directive du compilateur indiquant que les éléments d'argument de la déclaration de fonction doivent être copiés et collés dans cet emplacement.

Les raisons derrière lesquelles indiquer explicitement où les arguments de la déclaration pouvant être appelée parent doivent être copiés et collés sont deux dossiers : un, il est inutile de répéter la déclaration d'argument, et deux, il garantit que les foncteurs qui nécessitent des arguments supplémentaires, comme le foncteur `Controlled`, peuvent être introduits de manière cohérente.

Lorsqu'il existe exactement une spécialisation définissant l'implémentation du corps par défaut, l'habillage supplémentaire du formulaire `body ... { <implementation> }` peut être omis.

## Récurtivité

Q# les appelants peuvent être directement ou indirectement récursifs et peuvent être déclarés dans n'importe quel ordre ; une opération ou une fonction peut s'appeler elle-même, ou elle peut appeler une autre appelante qui appelle directement ou indirectement l'appelant.

L'espace de pile peut être limité lors de l'exécution sur du matériel quantique et les récursivités qui dépassent cette limite d'espace de pile entraînent une erreur d'exécution.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Déclarations de spécialisation

Article • 21/03/2025

Comme expliqué dans la section sur [déclarations pouvant être appelées](#), il n'existe actuellement aucune raison de déclarer explicitement des spécialisations pour les fonctions. Cette rubrique s'applique aux opérations et explique comment déclarer les spécialisations nécessaires pour prendre en charge certains functors.

C'est un problème courant dans l'informatique quantique pour exiger l'adjoint d'une transformation donnée. De nombreux algorithmes quantiques nécessitent à la fois une opération et son adjoint pour effectuer un calcul. Q# utilise des calculs symboliques qui peuvent générer automatiquement l'implémentation adjointe correspondante pour une implémentation de corps spécifique. Cette génération est possible même pour les implémentations qui mélangent librement les calculs classiques et quantiques. Toutefois, certaines restrictions s'appliquent dans ce cas. Par exemple, la génération automatique n'est pas prise en charge pour des raisons de performances si l'implémentation utilise des variables mutables. De plus, chaque opération appelée dans le corps génère les besoins correspondants pour prendre en charge le functor `Adjoint` lui-même.

Même si on ne peut pas facilement annuler les mesures dans le cas multi-qubit, il est possible de combiner des mesures afin que la transformation appliquée soit unitaire. Dans ce cas, cela signifie que, même si l'implémentation du corps contient des mesures qui ne prennent pas en charge le functor `Adjoint`, le corps dans son intégralité est adjoint. Néanmoins, la génération automatique de l'implémentation d'adjoint échoue dans ce cas. Pour cette raison, il est possible de spécifier manuellement l'implémentation. Le compilateur génère automatiquement des implémentations optimisées pour des modèles courants tels que des conjugations. Néanmoins, une spécialisation explicite peut être souhaitable pour définir une implémentation plus optimisée manuellement. Il est possible de spécifier une implémentation et un certain nombre d'implémentations explicitement.

## ! Notes

Le compilateur ne vérifie pas l'exactitude d'une telle implémentation spécifiée manuellement.

Dans l'exemple suivant, la déclaration d'une opération `SWAP`, qui échange l'état de deux qubits `q1` et `q2`, déclare une spécialisation explicite pour sa version adjointe et sa version contrôlée. Bien que les implémentations de `Adjoint SWAP` et de `Controlled SWAP` soient donc définies par l'utilisateur, le compilateur doit toujours générer



l'implémentation pour la combinaison des deux foncteurs (`Controlled Adjoint SWAP`, qui est identique à `Adjoint Controlled SWAP`).

Q#

```
operation SWAP (q1 : Qubit, q2 : Qubit) : Unit
is Adj + Ctl {

 body ... {
 CNOT(q1, q2);
 CNOT(q2, q1);
 CNOT(q1, q2);
 }

 adjoint ... {
 SWAP(q1, q2);
 }

 controlled (cs, ...) {
 CNOT(q1, q2);
 Controlled CNOT(cs, (q2, q1));
 CNOT(q1, q2);
 }
}
```

## Directives de génération automatique

Lorsque vous déterminez comment générer une spécialisation particulière, le compilateur hiérarchise les implémentations définies par l'utilisateur. Cela signifie que si une spécialisation adjointe est définie par l'utilisateur et qu'une spécialisation contrôlée est générée automatiquement, la spécialisation d'adjoint contrôlée est générée en fonction de l'adjoint défini par l'utilisateur et vice versa. Dans ce cas, les deux spécialisations sont définies par l'utilisateur. Étant donné que la génération automatique d'une implémentation d'adjoint est soumise à plus de limitations, la spécialisation adjointe contrôlée par défaut génère la spécialisation contrôlée de l'implémentation définie explicitement de la spécialisation adjointe.

Dans le cas de l'implémentation de `SWAP`, la meilleure option consiste à ajouter la spécialisation contrôlée pour éviter inutilement de conditionner inutilement l'exécution du premier et du dernier `CNOT` sur l'état des qubits de contrôle. L'ajout d'une déclaration explicite pour la version de l'adjoint contrôlé qui spécifie une directive de génération *appropriée* force le compilateur à générer la spécialisation adjointe contrôlée en fonction de l'implémentation spécifiée manuellement de la version contrôlée à la place.

Une telle déclaration explicite d'une spécialisation générée par le compilateur prend la forme

Q#

```
controlled adjoint invert;
```

et est inséré à l'intérieur de la déclaration de `SWAP`. En revanche, insérer la ligne

Q#

```
controlled adjoint distribute;
```

force le compilateur à générer la spécialisation en fonction de la spécialisation adjointe définie (ou générée). Pour plus d'informations, consultez cette [proposition d'inférence de spécialisation partielle](#) pour plus d'informations.

Pour l'opération `SWAP`, il existe une meilleure option. `SWAP` est *auto-adjoint*, c'est-à-dire son propre inverse ; la mise en œuvre définie de l'adjoint appelle simplement le corps de `SWAP` et est exprimé avec la directive

Q#

```
adjoint self;
```

La déclaration de la spécialisation adjointe de cette façon garantit que la spécialisation adjointe contrôlée que le compilateur insère automatiquement appelle simplement la spécialisation contrôlée.

Les directives de génération suivantes existent et sont valides :

[🔗](#) Agrandir le tableau

Spécialisation	Directives
spécialisation <code>body</code> :	-
spécialisation <code>adjoint</code> :	<code>self</code> , <code>invert</code>
spécialisation <code>controlled</code> :	<code>distribute</code>
spécialisation <code>controlled adjoint</code> :	<code>self</code> , <code>invert</code> , <code>distribute</code>

Que toutes les directives de génération sont valides pour une spécialisation adjointe contrôlée n'est pas une coïncidence ; tant que les foncteurs se déplacent, l'ensemble de

directives de génération valides pour l'implémentation de la spécialisation pour une combinaison de foncteurs est toujours l'union de l'ensemble de générateurs valides pour chacun d'eux.

Outre les directives répertoriées précédemment, la directive `auto` est valide pour toutes les spécialisations, sauf `body`; il indique que le compilateur doit choisir automatiquement une directive de génération appropriée. Déclaration

Q#

```
operation DoNothing() : Unit {
 body ... { }
 adjoint auto;
 controlled auto;
 controlled adjoint auto;
}
```

équivalent à

Q#

```
operation DoNothing() : Unit
is Adj + Ctl { }
```

L'annotation `is Adj + Ctl` dans cet exemple spécifie les caractéristiques d'opération, qui contiennent les informations sur les foncteurs pris en charge par une opération particulière.

Bien que pour la lisibilité, nous vous recommandons d'annoter chaque opération avec une description complète de ses caractéristiques, le compilateur insère ou termine automatiquement l'annotation en fonction de spécialisations déclarées explicitement. À l'inverse, le compilateur génère également des spécialisations qui ne sont pas déclarées explicitement, mais qui doivent exister en fonction des caractéristiques annotées. Nous disons que l'annotation donnée *déclare implicitement* ces spécialisations. Le compilateur génère automatiquement les spécialisations nécessaires si elle le peut, en choisissant une directive appropriée. Q# prend donc en charge l'inférence des caractéristiques d'opération et des spécialisations existantes basées sur des annotations (partielles) et des spécialisations définies explicitement.

Dans un sens, les spécialisations sont similaires aux surcharges individuelles pour le même appelant, avec la mise en garde que certaines restrictions s'appliquent aux surcharges que vous pouvez déclarer.

---

# Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Attributs

Les attributs attachent des informations supplémentaires à une déclaration que le Q# compilateur utilise pour modifier la façon dont la déclaration est traitée. Un attribut est écrit avec un symbole de début `@`, suivi du nom de l'attribut et d'une liste d'arguments entre parenthèses, et il est placé immédiatement avant la déclaration à laquelle elle s'applique.

Q#

```
@EntryPoint()
operation Main() : Result {
 use q = Qubit();
 H(q);
 MResetZ(q)
}
```

Vous pouvez appliquer plusieurs attributs à la même déclaration en plaçant chaque attribut sur sa propre ligne avant la déclaration.

Q# prend en charge les attributs intégrés suivants.

## @EntryPoint()

L'attribut `@EntryPoint()` marque une opération ou une fonction comme point de départ de l'exécution d'un programme. Si vous ne spécifiez pas de point d'entrée, le compilateur démarre l'exécution à partir de l'opération `Main()`, le cas échéant. Vous pouvez placer l'attribut `@EntryPoint()` sur n'importe quel appelant qui n'accepte aucun paramètre d'entrée pour le rendre au point d'entrée à la place.

Q#

```
@EntryPoint()
operation MeasureOneQubit() : Result {
 use q = Qubit();
 H(q);
 MResetZ(q)
}
```

Lorsque vous travaillez dans un fichier unique `.qs` qui ne fait pas partie d'un Q# projet, vous pouvez également passer un nom de profil cible à l'attribut, tel que `@EntryPoint(Base)`, , `@EntryPoint(Adaptive_RI)` `@EntryPoint(Adaptive_RIF)` ou, pour `@EntryPoint(Unrestricted)` définir

le profil cible QIR pour le programme. Pour plus d'informations, consultez [les profils cibles de l'informatique quantique](#).

## @Config(...)

L'attribut `@Config(...)` fournit des informations de prétraitement sur le moment où une déclaration doit être incluse dans la compilation, en fonction des fonctionnalités du [profil cible](#) actuel. Cela permet à une bibliothèque de fournir différentes implémentations d'un appelant pour les cibles avec différentes fonctionnalités.

Les arguments valides sont `Base`, `Adaptive IntegerComputations`, `FloatingPointComputations`, `BackwardsBranching`, `HigherLevelConstructs`, et `Unrestricted`. Vous pouvez négation d'un argument avec l'opérateur `not` pour inclure la déclaration uniquement lorsque la fonctionnalité *n'est pas* disponible.

Q#

```
@Config(Adaptive)
operation MeasureAndReset(q : Qubit) : Result {
 let result = M(q);
 if result == One {
 X(q);
 }
 result
}

@Config(not Adaptive)
operation MeasureAndReset(q : Qubit) : Result {
 MResetZ(q)
}
```

## @Test()

L'attribut `@Test()` transforme un appelant en test unitaire. Vous ne pouvez l'appliquer qu'à un appelant qui ne prend aucun paramètre d'entrée. Les tests marqués avec cet attribut apparaissent dans la vue **Test** dans Visual Studio Code, où vous pouvez les exécuter individuellement ou en tant que groupe. Pour plus d'informations, consultez [Tests et débogage Q# des programmes](#).

Q#

```
@Test()
operation TestCase() : Unit {
```

```
Fact(1 + 1 == 2, "Expected 1 + 1 to equal 2.");
}
```

## Attributs pour les appelants intrinsèques

Les attributs suivants décrivent comment un appelant intrinsèque est traité pendant la génération de code [QIR \(Quantum Intermediate Representation\)](#). Ils sont principalement utilisés par les auteurs de bibliothèque et de matériel qui définissent des opérations intrinsèques de bas niveau.

### @SimulatableIntrinsic()

L'attribut `@SimulatableIntrinsic()` indique qu'un appelant doit être traité comme un appelant intrinsèque pour la génération de code QIR, tandis que le corps que vous fournissez est utilisé uniquement pendant la simulation. Cela vous permet de fournir une implémentation de simulation pour une opération qui est sinon opaque pour le matériel cible.

### @Measurement()

L'attribut `@Measurement()` indique qu'un appelant intrinsèque est une mesure. L'opération est marquée comme *irréversible* dans le QIR généré et la valeur de sortie `Result` est déplacée vers les arguments de la fonction générée.

### @Reset()

L'attribut `@Reset()` indique qu'un appelant intrinsèque est une réinitialisation. L'opération est marquée comme *irréversible* dans le QIR généré.

# Commentaires

Article • 21/03/2025

Les commentaires commencent par deux barres obliques (`//`) et continuent jusqu'à la fin de la ligne. Ces commentaires de fin de ligne peuvent apparaître n'importe où dans le code source. Q# ne prend actuellement pas en charge les commentaires de bloc.

## Commentaires de documentation

Les commentaires commençant par trois barres obliques, `///`, sont traités spécialement par le compilateur lorsqu'ils apparaissent avant un type ou une déclaration pouvant être appelée. Dans ce cas, leur contenu est pris comme documentation pour le type défini ou pouvant être appelé, comme pour d'autres langages .NET.

Dans `///` commentaires, le texte à afficher dans le cadre de la documentation de l'API est mis en forme comme [Markdown](#), avec différentes parties de la documentation indiquées par des en-têtes nommés spécialement. En tant qu'extension à Markdown, les références croisées aux opérations, aux fonctions et aux types de struct dans Q# peuvent être incluses à l'aide de `@<ref target>`, où `<ref target>` est remplacée par le nom complet de l'objet de code référencé. Si vous le souhaitez, un moteur de documentation peut également prendre en charge d'autres extensions Markdown.

Par exemple:

Q#

```
/// # Summary
/// Given an operation and a target for that operation,
/// applies the given operation twice.
///
/// # Input
/// ## op
/// The operation to be applied.
/// ## target
/// The target to which the operation is to be applied.
///
/// # Type Parameters
/// ## 'T
/// The type expected by the given operation as its input.
///
/// # Example
/// ```Q#
/// // Should be equivalent to the identity.
/// ApplyTwice(H, qubit);
/// ```
```



```
///
/// # See Also
/// - Microsoft.Quantum.Intrinsic.H
operation ApplyTwice<'T>(op : ('T => Unit), target : 'T) : Unit {
 op(target);
 op(target);
}
```

Q# reconnaît les noms suivants en tant qu'en-têtes de commentaires de documentation.

- **Résumé:** résumé court du comportement d'une fonction ou d'une opération ou de l'objectif d'un type. Le premier paragraphe du résumé est utilisé pour les informations de pointage. Il doit s'agir d'un texte brut.
- **Description:** description du comportement d'une fonction ou d'une opération ou de l'objectif d'un type. Le résumé et la description sont concaténés pour former le fichier de documentation généré pour la fonction, l'opération ou le type. La description peut contenir des symboles et équations au format LaTeX en ligne.
- **d'entrée :** description du tuple d'entrée pour une opération ou une fonction. Peut contenir des sous-sections Markdown supplémentaires indiquant chaque élément du tuple d'entrée.
- **sortie:** description du tuple retourné par une opération ou une fonction.
- **Paramètres de type:** section vide qui contient une sous-section supplémentaire pour chaque paramètre de type générique.
- **éléments nommés:** description des éléments nommés dans un type de struct. Peut contenir des sous-sections Markdown supplémentaires avec la description de chaque élément nommé.
- **Exemple:** exemple court de l'opération, de la fonction ou du type utilisé.
- **Remarques:** prose divers décrivant un aspect de l'opération, de la fonction ou du type.
- **Voir également:** liste de noms complets indiquant les fonctions, les opérations ou les types de struct associés.
- **Références:** liste de références et de citations pour l'élément documenté.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Instructions

Article • 21/03/2025

Q# distingue les instructions et les expressions. Q# programmes se composent d'un mélange de calculs classiques et quantiques, et l'implémentation ressemble beaucoup à n'importe quel autre langage de programmation classique. Certaines instructions, telles que les liaisons `let` et `mutable`, sont connues des langages classiques, tandis que d'autres, telles que les allocations de qubits, sont uniques au domaine quantique.

Les instructions suivantes sont actuellement disponibles dans Q#:

- **instruction expression**  
Contient une expression Q# à exécuter, telle qu'un appel à une opération. Si la dernière instruction d'un bloc est une instruction d'expression, elle peut avoir son point-virgule de fin omis pour donner au bloc la valeur évaluée de l'expression contenue.
- **déclaration de variable**  
Définit une ou plusieurs variables locales valides pour le reste de l'étendue actuelle et les lie aux valeurs spécifiées. Les variables peuvent être définitivement liées ou déclarées comme réaffectables ultérieurement. Pour plus d'informations, consultez [déclarations et réaffectations de variables](#).
- **d'allocation Qubit**  
Instancie et initialise des qubits, ou des tableaux de qubits, et les lie aux variables déclarées. L'instruction peut éventuellement être utilisée avec un bloc de code spécifié, dans lequel les allocations qubits sont valides. Sinon, les allocations sont valides pour l'étendue englobante. Les qubits sont automatiquement libérés à la fin de l'étendue appropriée. Pour plus d'informations, consultez [gestion de la mémoire Quantum](#).

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Visibilité des variables locales

Article • 21/03/2025

En général, les liaisons de symboles dans Q# deviennent inopérantes à la fin du bloc d'instruction dans lequel elles se produisent. Toutefois, il existe certaines exceptions à cette règle.

## Visibilité des variables locales

 Agrandir le tableau

Étendue	Visibilité
Variables de boucle	Les liaisons de variables de boucle dans une boucle <code>for</code> sont définies uniquement pour le corps de la boucle. Elles sont inopérantes en dehors de la boucle.
Déclarations <code>use</code> et <code>borrow</code>	Les liaisons de qubits alloués dans <code>use</code> et les instructions <code>borrow</code> sont définies pour le corps de l'allocation et sont inopérantes une fois l'instruction terminée. Cette action s'applique uniquement aux instructions <code>use</code> et <code>borrow</code> qui ont un bloc d'instructions associé.
Expressions <code>repeat</code>	Pour les expressions <code>repeat</code> , les blocs et la condition sont traités comme une seule étendue, autrement dit, les symboles liés dans le corps sont accessibles à la fois dans la condition et dans le bloc <code>fixup</code> .
Boucles	Chaque itération d'une boucle s'exécute dans sa propre étendue, et toutes les variables définies sont liées à zéro pour chaque itération.

Les liaisons dans les blocs externes sont visibles et définies dans les blocs internes. Un symbole peut uniquement être lié plusieurs fois par bloc, auquel cas la dernière liaison dans l'étendue est celle utilisée pour la logique. Ce scénario est appelé « ombre ».

Les séquences suivantes sont valides :

```
Q#

if a == b {
 ...
 let n = 5;
 ... // n is 5
}
let n = 8;
... // n is 8
```

and

Q#

```
if a == b {
 ...
 let n = 5;
 ... // n is 5
} else {
 ...
 let n = 8;
 ... // n is 8
}
... // n is not bound to a value
```

Q#

```
let n = 5;
... // n is 5
let n = 8;
... // n is 8
```

and

Q#

```
let n = 8;
if a == b {
 ... // n is 8
 let n = 5;
 ... // n is 5
}
... // n is 8 again
```

Pour plus d'informations, consultez [déclarations et réaffectations de variables](#).

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Déclarations et réaffectations de variables

Article • 21/02/2025

Les valeurs peuvent être liées aux symboles à l'aide des instructions `let` et `mutable`. Ces types de liaisons offrent un moyen pratique d'accéder à une valeur via le handle défini. Malgré la terminologie trompeuse empruntée à d'autres langues, les handles déclarés sur une étendue locale et contenant des valeurs sont appelés *variables*. Ce nommage peut être trompeur, car `let` instructions définissent *handles à affectation unique*, qui sont des handles qui restent liés à la même valeur pendant la durée de leur validité. Les variables qui peuvent être re-liées à différentes valeurs à différents points du code doivent être déclarées explicitement, et sont spécifiées à l'aide de l'instruction `mutable`.

Q#

```
let var1 = 3;
mutable var2 = 3;
var2 = var2 + 1;
```

Dans cet exemple, l'instruction `let` déclare une variable nommée `var1` qui ne peut pas être réaffectée et contient toujours la valeur `3`. L'instruction `mutable` définit une variable `var2` liée temporairement à la valeur `3`, mais peut être réaffectée à une autre valeur ultérieurement à l'aide d'une expression d'affectation, comme indiqué dans la dernière ligne. Vous pouvez exprimer la même instruction avec la version plus courte `var2 += 1;`, comme c'est le cas dans d'autres langues. Pour plus d'informations, consultez [Évaluer et réaffecter des instructions](#).

Pour résumer :

- `let` est utilisé pour créer une liaison immuable.
- `mutable` est utilisé pour créer une liaison mutable.
- `=` sans `let` est utilisé pour modifier la valeur d'une liaison mutable.

Pour les trois instructions, le côté gauche se compose d'un symbole ou d'un tuple de symboles. Si le côté droit de la liaison est un tuple, ce tuple peut être entièrement ou partiellement déconstruit lors de l'affectation. La seule exigence de déconstruction est que la forme du tuple sur le côté droit correspond à la forme du tuple symbole sur le côté gauche. Le tuple de symbole peut contenir des tuples imbriqués ou des symboles omis, ou les deux, indiqués par un trait de soulignement. Par exemple:

Q#

```
let (a, (_, b)) = (1, (2, 3)); // a is bound to 1, b is bound to 3
mutable (x, y) = ((1, 2), [3, 4]); // x is bound to (1, 2), y is bound to
[3, 4]
(x, _, y) = ((5, 6), 7, [8]); // x is re-bound to (5,6), y is re-bound to
[8]
```

Pour plus d'informations sur la déconstruction à l'aide de l'opérateur unwrap (!), consultez [Accès Élément pour les types de struct](#).

Toutes les affectations dans Q# obéissent aux mêmes règles de déconstruction, notamment les allocations qubits et les affectations de variables de boucle.

Pour les deux types de liaisons, les types des variables sont déduits du côté droit de la liaison. Le type d'une variable reste toujours le même et une expression d'affectation ne peut pas la modifier. Les variables locales peuvent être déclarées comme mutables ou immuables. Il existe certaines exceptions, telles que les variables de boucle dans `for` boucles, où le comportement est prédéfini et ne peut pas être spécifié. Les arguments de fonction et d'opération sont toujours immuablement liés. En combinaison avec l'absence de types de référence, comme indiqué dans la rubrique [immuabilité](#), étant immuablement lié signifie qu'une fonction ou une opération appelée ne peut jamais modifier les valeurs côté appelant.

Étant donné que les états de `qubit` valeurs ne sont pas définis ou observables à partir de Q#, cela n'empêche pas l'accumulation d'effets secondaires quantiques, qui sont observables uniquement par le biais de mesures. Pour plus d'informations, consultez [types de données Quantum](#).

Indépendamment de la façon dont une valeur est liée, les valeurs elles-mêmes sont immuables. En particulier, cela est vrai pour les tableaux et les éléments de tableau. Contrairement aux langages classiques populaires où les tableaux sont souvent des types de référence, les tableaux de Q#, comme tous les types, sont des types valeur et toujours immuables ; autrement dit, vous ne pouvez pas les modifier après l'initialisation. La modification des valeurs accessibles par les variables de type tableau nécessite donc de construire explicitement un nouveau tableau et de le réaffecter au même symbole. Pour plus d'informations, consultez [d'immuabilité](#) et [copier et mettre à jour des expressions](#).

## Instructions Evaluate-and-reassign

Les instructions du formulaire `intValue += 1;` sont courantes dans de nombreuses autres langues. Ici, `intValue` doit être une variable de type `Int` de type mutable. Ces instructions fournissent un moyen pratique de concaténation si le côté droit consiste à appliquer un opérateur binaire et que le résultat est re-lié à l'argument gauche de l'opérateur. Par exemple, ce segment de code

Q#

```
mutable counter = 0;
for i in 1 .. 2 .. 10 {
 counter += 1;
 // ...
}
```

incrémente la valeur du compteur `counter` dans chaque itération de la boucle `for` et équivaut à

Q#

```
mutable counter = 0;
for i in 1 .. 2 .. 10 {
 counter = counter + 1;
 // ...
}
```

Des instructions similaires existent pour un large éventail d'opérateurs . Ces expressions d'évaluation et de réaffectation existent pour tous les opérateurs où le type de la sous-expression la plus à gauche correspond au type d'expression. Plus précisément, ils sont disponibles pour les opérateurs logiques et binaires binaires, notamment le décalage droit et gauche, les expressions arithmétiques, y compris l'exponentiation et les modulus, et les concaténations, ainsi que les expressions de copie et de mise à jour .

L'exemple de fonction suivant calcule la somme d'un tableau de nombres [Complex](#) :

Q#

```
function ComplexSum(values : Complex[]) : Complex {
 mutable res = Complex(0., 0.);
 for complex in values {
 res = new Complex { Re = res.Re + complex.Re, Im = res.Im +
complex.Im };
 }
 return res;
}
```

De même, la fonction suivante multiplie chaque élément d'un tableau avec le facteur donné :

Q#

```
function Multiplied(factor : Double, array : Double[]) : Double[] {
 mutable res = new Double[Length(array)];
 for i in IndexRange(res) {
 res w/= i <- factor * array[i];
 }
 return res;
}
```

Pour plus d'informations, consultez [expressions contextuelles](#), qui contient d'autres exemples où les expressions peuvent être omises dans un contexte spécifique lorsqu'une expression appropriée peut être déduite par le compilateur.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)



# Gestion de la mémoire quantique

Article • 21/03/2025

Un programme démarre toujours sans qubits, ce qui signifie que vous ne pouvez pas passer des valeurs de type `Qubit` en tant qu'arguments de point d'entrée. Cette restriction est intentionnelle puisque l'objectif de Q# est d'exprimer et de raisonner sur un programme dans son intégralité. Au lieu de cela, un programme alloue et libère des qubits, ou de la mémoire quantique, comme il se passe. À cet égard, Q# modélise l'ordinateur quantique en tant que tas qubit.

Au lieu de prendre en charge des instructions *de* et de *de mise en production* distinctes pour la mémoire quantique, Q# prend en charge l'allocation de mémoire quantique sous la forme d'instructions de bloc, où la mémoire est accessible uniquement dans l'étendue de cette instruction de bloc. Le bloc d'instructions peut être implicitement défini lors de l'allocation de qubits pour la durée de l'étendue actuelle, comme décrit plus en détail dans les sections relatives aux instructions `use` et `borrow`. Toute tentative d'accès aux qubits alloués après la fin de l'instruction entraîne une exception d'exécution.

Q# a deux instructions, `use` et `borrow`, quiinstancient des valeurs de qubits, des tableaux de qubits ou toute combinaison de ces derniers. Vous ne pouvez utiliser ces instructions que dans les opérations. Ils rassemblent les valeurs qubit instanciées, les lient aux variables spécifiées dans l'instruction, puis exécutent un bloc d'instructions. À la fin du bloc, les variables liées sortent de l'étendue et ne sont plus définies.

Q# distingue l'allocation de propre et qubits sales. Les qubits propres sont nonanglés et ne sont pas utilisés par une autre partie du calcul. Les qubits sales sont des qubits dont l'état est inconnu et qui peuvent même être enchevêtrés avec d'autres parties de la mémoire du processeur quantique.

## Instruction Use

Les qubits propres sont alloués par l'instruction `use`.

- L'instruction se compose du mot clé `use` suivi d'une liaison et d'un bloc d'instructions facultatif.
- Si un bloc d'instructions est présent, les qubits sont disponibles uniquement dans ce bloc. Sinon, les qubits sont disponibles jusqu'à la fin de l'étendue actuelle.
- La liaison suit le même modèle que les instructions `let` : un symbole unique ou un tuple de symboles, suivi d'un signe égal `=`, et d'un tuple unique ou d'un tuple

correspondant de *initialiseurs*.

Les initialiseurs sont disponibles pour un qubit unique, indiqué comme `Qubit()`, ou un tableau de qubits, `Qubit[n]`, où `n` est une expression `Int`. Par exemple

```
Q#

use qubit = Qubit();
// ...

use (aux, register) = (Qubit(), Qubit[5]);
// ...

use qubit = Qubit() {
 // ...
}

use (aux, register) = (Qubit(), Qubit[5]) {
 // ...
}
```

Les qubits sont garantis dans un état  $|0\rangle$  lors de l'allocation. Ils sont libérés à la fin de l'étendue et doivent être dans un état  $|0\rangle$  lors de la mise en production. Cette exigence n'est pas appliquée par le compilateur, car cela nécessiterait une évaluation symbolique qui devient rapidement coûteuse. Lors de l'exécution sur des simulateurs, l'exigence peut être appliquée au moment de l'exécution. Sur les processeurs quantiques, l'exigence ne peut pas être appliquée au moment de l'exécution ; un qubit non mesuré peut être réinitialisé à  $|0\rangle$  via une transformation unitaire. L'échec de cette opération entraîne un comportement incorrect.

L'instruction `use` alloue les qubits du tas qubit libre du processeur quantique et les retourne au tas pas plus tard que la fin de l'étendue dans laquelle les qubits sont liés.

## Instruction Borrow

L'instruction `borrow` accorde l'accès aux qubits déjà alloués, mais pas actuellement en cours d'utilisation. Ces qubits peuvent être dans un état arbitraire et doivent être à nouveau dans le même état lorsque l'instruction borrow se termine. Certains algorithmes quantiques peuvent utiliser des qubits sans compter sur leur état exact et sans exiger qu'ils soient inanglés avec le reste du système. Autrement dit, ils nécessitent temporairement des qubits supplémentaires, mais ils peuvent s'assurer que ces qubits sont retournés exactement à leur état d'origine, indépendamment de l'état qui était.

S'il existe des qubits qui sont utilisés mais qui ne sont pas touchés pendant des parties d'une sous-routine, ces qubits peuvent être empruntés pour être utilisés par un tel algorithme au lieu d'allouer de la mémoire quantique supplémentaire. L'emprunt au lieu d'allouer peut réduire considérablement les besoins globaux en mémoire quantique d'un algorithme et est un exemple quantique d'un compromis classique au moment de l'espace.

Une instruction `borrow` suit le même modèle décrit précédemment pour l'instruction `use`, avec les mêmes initialiseurs disponibles. Par exemple

```
Q#

borrow qubit = Qubit();
// ...

borrow (aux, register) = (Qubit(), Qubit[5]);
// ...

borrow qubit = Qubit() {
 // ...
}

borrow (aux, register) = (Qubit(), Qubit[5]) {
 // ...
}
```

Les qubits empruntés sont dans un état inconnu et sortent de l'étendue à la fin du bloc d'instructions. L'emprunteur s'engage à quitter les qubits dans le même état que lorsqu'ils ont été empruntés ; autrement dit, leur état au début et la fin du bloc d'instructions est censé être identique.

L'instruction `borrow` récupère des qubits en cours d'utilisation qui ne sont pas utilisés par le programme à partir du moment où le qubit est lié jusqu'à la dernière utilisation de ce qubit. S'il n'y a pas suffisamment de qubits disponibles pour emprunter, les qubits sont alloués et retournés au tas comme une instruction `use`.

### ❗ Notes

Parmi les cas d'usage connus des qubits sales, citons des implémentations de portes CNOT multicommandes qui nécessitent très peu de qubits et d'implémentations d'incrémentateurs. Ce [document sur la factoring avec des qubits](#) fournit un exemple d'algorithme qui utilise des qubits empruntés.

# Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Expressions

Article • 21/03/2025

Au cœur, Q# expressions sont des littéraux de valeur ou [identificateurs](#), où les identificateurs peuvent faire référence à des variables déclarées localement ou à des callables déclarés globalement (il n'existe actuellement aucune constante globale dans Q#). Les opérateurs, les combineurs et les modificateurs peuvent être utilisés pour combiner ces identificateurs en une plus grande variété d'expressions.

- *Opérateurs* dans un sens ne sont qu'une syntaxe dédiée pour des appelants particuliers.

Même si Q# n'est pas encore suffisamment expressif pour capturer formellement les capacités de chaque opérateur sous la forme d'une déclaration pouvant être appelée, qui devrait être corrigée à l'avenir.

- *modificateurs* ne peuvent être appliqués qu'à certaines expressions. Un ou plusieurs modificateurs peuvent être appliqués à des expressions qui sont soit
  - Identificateurs
  - expressions d'accès aux éléments de tableau
  - Expressions d'accès aux éléments nommés
  - expression entre parenthèses qui est identique à un tuple d'élément unique.Pour plus d'informations, consultez [équivalence de tuple singleton](#)). Ils peuvent précéder (préfixe) l'expression ou suivre (postfix) l'expression. Ils sont donc des opérateurs unaires spéciaux qui lient plus étroitement que les appels de fonction ou d'opération, mais moins serrés que tout type d'accès aux éléments. Concrètement, [foncteurs](#) sont des modificateurs de préfixe, tandis que l'opérateur [unwrap](#) est un modificateur postfix.
- Les appels d'accès de fonction, d'opération et d'élément peuvent être considérés comme un type spécial d'opérateur, similaire aux modificateurs. Toutes sont soumises aux mêmes restrictions concernant l'endroit où elles peuvent être appliquées ; nous les appelons *combineurs*.

La section sur [priorité et l'association](#) contient une liste complète [de tous les opérateurs](#) ainsi qu'une liste complète [de tous les modificateurs et combineurs](#).

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Priorité et associativité

Article • 21/03/2025

La priorité et l'associativité définissent l'ordre dans lequel les opérateurs sont appliqués. Les opérateurs ayant une priorité plus élevée sont liés d'abord à leurs arguments (opérandes), tandis que les opérateurs ayant la même liaison de précedence dans la direction de leur associativité. Par exemple, l'expression `1+2*3` en fonction de la priorité pour l'ajout et la multiplication équivaut à `1+(2*3)`, et `2^3^4` est égale à `2^(3^4)`, car l'exposant est associatif de droite.

## Opérateurs

Le tableau suivant répertorie les opérateurs disponibles dans Q#, ainsi que leur priorité et leur associativité. Des modificateurs et des combinateurs de [supplémentaires](#) sont également répertoriés et lient plus étroitement que l'un de ces opérateurs.

 Agrandir le tableau

Description	Syntaxe	Opérateur	Associativité	Préséance
<a href="#">opérateur copy-and-update</a>	<code>w/ &lt;-</code>	ternaire	Gauche	1
<a href="#">opérateur de plage</a>	<code>..</code>	Infixe	Gauche	2
d'opérateur conditionnel	<code>? \ </code>	ternaire	Droite	3
OR logique	<code>or</code>	Infixe	Gauche	4
<a href="#">logique AND</a>	<code>and</code>	Infixe	Gauche	5
OR au niveau du bit	<code>\ \  \ </code>	Infixe	Gauche	6
XOR au niveau du bit	<code>^^^</code>	Infixe	Gauche	7
AND au niveau du bit	<code>&amp;&amp;&amp;</code>	Infixe	Gauche	8
d'égalité	<code>==</code>	Infixe	Gauche	9
<a href="#">inégalité</a>	<code>!=</code>	Infixe	Gauche	9
inférieures ou égales	<code>&lt;=</code>	Infixe	Gauche	10
<a href="#">inférieur à</a>	<code>&lt;</code>	Infixe	Gauche	11
supérieure ou égale	<code>&gt;=</code>	Infixe	Gauche	11

Description	Syntaxe	Opérateur	Associativité	Préséance
supérieur à	>	Infixe	Gauche	11
maj droite	>>>	Infixe	Gauche	12
décalage gauche	<<<	Infixe	Gauche	12
ou concaténation	+	Infixe	Gauche	13
soustraction	-	Infixe	Gauche	13
de multiplication	*	Infixe	Gauche	14
division	/	Infixe	Gauche	14
modulus	%	Infixe	Gauche	14
l'exposant	^	Infixe	Droite	15
pas au niveau du bit	~~~	prefix	Droite	16
logique NOT	not	prefix	Droite	16
négative	-	prefix	Droite	16

Les expressions de copie et de mise à jour doivent nécessairement avoir la priorité la plus faible pour garantir un comportement cohérent de l'instruction [evaluate-and-reassignment correspondante](#). Des considérations similaires sont prises en compte pour l'opérateur de plage afin de garantir un comportement cohérent de l'expression contextuelle [correspondante](#).

## Modificateurs et combineurs

Les modificateurs peuvent être considérés comme des opérateurs spéciaux qui peuvent être appliqués à certaines expressions uniquement. Ils peuvent être affectés à une priorité artificielle pour capturer leur comportement.

Pour plus d'informations, consultez [expressions](#).

Cette priorité artificielle est répertoriée dans le tableau suivant, ainsi que la façon dont la priorité des opérateurs et modificateurs est liée à la liaison entre les combineurs d'accès à l'élément ([, ] et :: respectivement) et les combineurs d'appel ((, )).



Description	Syntaxe	Opérateur	Associativité	Préséance
de combineur d'appels	( )	n/a	Gauche	17
'fonctor adjoint	Adjoint	prefix	Droite	18
functor contrôlé	Controlled	prefix	Droite	18
application Unwrap	!	Postfix	Gauche	19
accès aux éléments nommés	.	n/a	Gauche	20
d'accès aux éléments de tableau	[ ]	n/a	Gauche	20
fonction lambda	->	n/a	Droite	21
Opération lambda	=>	n/a	Droite	21

Pour illustrer les implications des précédences affectées, supposons que vous disposez d'une opération unitaire `DoNothing` (comme défini dans [déclarations de spécialisation](#)), d'un `GetStatePrep` appelant qui retourne une opération unitaire et d'un tableau `algorithms` qui contient des éléments de type `Algorithm` définis comme suit :

Q#

```
struct Algorithm {
 Register : Qubit[],
 Initialize : Transformation,
 Apply : Transformation,
}
```

Les expressions suivantes sont toutes valides :

Q#

```
GetStatePrep()(arg)
Adjoint DoNothing()
Controlled Adjoint DoNothing(cs, ())
Controlled algorithms[0].Apply!(cs, _)
algorithms[0].Register![i]
```

En examinant les précédences définies dans le tableau ci-dessus, vous pouvez voir que les parenthèses autour de `(Transformation(GetStatePrep()))` sont nécessaires pour que l'opérateur unwrap suivant soit appliqué à la valeur `Transformation` plutôt qu'à l'opération retournée. Toutefois, les parenthèses ne sont pas requises dans `GetStatePrep()(arg)` ; les fonctions sont appliquées de gauche à droite. Cette expression équivaut donc à `(GetStatePrep())(arg)`. Les applications fonctorielles ne nécessitent

pas non plus de parenthèses pour appeler la spécialisation correspondante, ni les expressions d'accès aux tableaux ou aux éléments nommés. Ainsi, l'expression `arr2D[i][j]` est parfaitement valide, comme c'est `algorithms[0]::Register![i]`.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Expressions de copie et de mise à jour

Article • 21/12/2024

Pour réduire le besoin de liaisons mutables, Q# prend en charge les expressions de copie et de mise à jour pour les tableaux, ce qui vous permet d'accéder aux éléments via un index ou une plage d'index.

Les expressions copy-and-updateinstancient un nouveau tableau avec tous les éléments définis sur la valeur correspondante dans le tableau d'origine, à l'exception des éléments spécifiés, qui sont définis sur ceux définis sur le côté droit de l'expression. Ils sont construits à l'aide d'un opérateur ternaire `w/ <-`; la syntaxe `w/` doit être lue comme la notation abrégée couramment utilisée pour « with » :

Q#

```
original w/ itemAccess <- modification
```

où `original` est une expression de tableau, `itemAccess` est une expression valide pour le découpage de tableau et `modification` est la nouvelle valeur ou les nouvelles valeurs. Concrètement, l'expression `itemAccess` peut être de type `Int` ou `Range`. Si `itemAccess` est une valeur de type `Int`, le type de `modification` doit correspondre au type d'élément du tableau. Si `itemAccess` est une valeur de type `Range`, le type de `modification` doit être identique au type de tableau.

Par exemple, si `arr` contient un tableau `[0, 1, 2, 3]`, puis

- `arr w/ 0 <- 10` est le tableau `[10, 1, 2, 3]`.
- `arr w/ 2 <- 10` est le tableau `[0, 1, 10, 3]`.
- `arr w/ 0..2..3 <- [10, 12]` est le tableau `[10, 1, 12, 3]`.

En termes de précedence, l'opérateur copy-and-update est associatif de gauche et a la priorité la plus faible, et, en particulier, la priorité inférieure à l'opérateur de plage (`..`) ou l'opérateur conditionnel ternaire (`? |`). L'associativité gauche choisie permet un chaînage facile d'expressions de copie et de mise à jour :

Q#

```
let model = ArrayConstructor()
 w/ 1 <- alpha
 w/ 3 <- gamma
 w/ 5 <- epsilon;
```

Comme pour tout opérateur qui construit une expression du même type que l'expression la plus à gauche impliquée, l'instruction [evaluate-and-reassignment correspondante](#) est disponible. Les deux instructions suivantes, par exemple, obtiennent ce qui suit : la première instruction déclare une variable mutable `arr` et la lie à la valeur par défaut d'un tableau entier. La deuxième instruction génère ensuite un nouveau tableau avec le premier élément (avec l'index 0) défini sur 10 et le réaffecte à `arr`.

Q#

```
mutable arr = [0, size = 3]; // arr contains [0, 0, 0]
arr w/= 0 <- 10; // arr contains [10, 0, 0]
```

La deuxième instruction est simplement courte pour la syntaxe plus détaillée `arr = arr w/ 0 <- 10;`.

Les expressions de copie et de mise à jour permettent la création efficace de nouveaux tableaux basés sur des expressions existantes. L'implémentation des expressions de copie et de mise à jour évite de copier l'ensemble du tableau en dupliquant uniquement les parties nécessaires pour obtenir le comportement souhaité et effectuer une modification sur place si possible. Par conséquent, les initialisations de tableau n'entraînent pas de surcharge supplémentaire en raison de l'immuabilité.

L'espace de noms `Std.Arrays` fournit un arsenal d'outils pratiques pour la création et la manipulation de tableaux.

Les expressions de copie et de mise à jour sont un moyen pratique de construire de nouveaux tableaux à la volée ; l'expression suivante, par exemple, prend la valeur d'un tableau avec tous les éléments définis sur `PauliI`, à l'exception de l'élément à l'index `i`, qui est défini sur `PauliZ`:

Q#

```
[PauliI, size = n] w/ i <- PauliZ
```

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#) | [Obtenir de l'aide sur Microsoft Q&A](#)

# Expressions conditionnelles

Article • 21/03/2025

Les expressions conditionnelles se composent de trois sous-expressions, où la sous-expression la plus à gauche est de type `Bool` et détermine l'une des deux autres sous-expressions évaluées. Ils sont de la forme

Q#

```
cond ? ifTrue | ifFalse
```

Plus précisément, si `cond` prend la valeur `true`, l'expression conditionnelle prend la valeur de l'expression `ifTrue` ; sinon, elle prend la valeur de l'expression `ifFalse`. L'autre expression (l'expression `ifFalse` et `ifTrue`, respectivement) n'est jamais évaluée, comme les branches d'une instruction `if`. Par exemple, dans une expression `a == b ? C(qs) | D(qs)`, si `a` est égal à `b`, le `C` pouvant être appelé est appelé. Sinon, `D` est appelée.

Les types de l' `ifTrue` et de l'expression `ifFalse` doivent avoir un type de base commun. Indépendamment de celui qui génère finalement la valeur à laquelle l'expression évalue, son type correspond toujours au type de base déterminé.

Par exemple, si

- `Op1` de type `Qubit[] => Unit is Adj`
- `Op2` de type `Qubit[] => Unit is Ctl`
- `Op3` de type `Qubit[] => Unit is Adj + Ctl`

, puis

- `cond ? Op1 | Op2` de type `Qubit[] => Unit`
- `cond ? Op1 | Op3` de type `Qubit[] => Unit is Adj`
- `cond ? Op2 | Op3` de type `Qubit[] => Unit is Ctl`

Pour plus d'informations, consultez [sous-saisie](#).

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No



# Expressions de comparaison

Article • 21/03/2025

## Comparaisons d'égalité

*comparaisons d'égalité* (`==`) et *comparaisons d'inégalités* (`!=`) sont actuellement limitées aux types de données suivants : `Int`, `BigInt`, `Double`, `String`, `Bool`, `Result`, `Pauli` et `Qubit`. Les comparaisons d'égalité des types `struct` et des callables ne sont actuellement pas prises en charge.

Comparaison d'égalité pour les valeurs de type `Qubit` évalue si les deux expressions identifient le même qubit. Il n'y a aucune notion d'état quantique dans Q#; La comparaison d'égalité, en particulier, n'*pas* accès, de mesure ou de modification de l'état quantique des qubits.

Les comparaisons d'égalité pour les valeurs de `Double` peuvent être trompeuses en raison d'effets arrondis. Par exemple, la comparaison suivante prend la valeur `false` en raison d'erreurs d'arrondi : `49.0 * (1.0/49.0) == 1.0`.

La comparaison d'égalité des tableaux et des tuples est prise en charge par des comparaisons de leurs éléments et ne sont prises en charge que si tous leurs types imbriqués prennent en charge la comparaison d'égalité.

La comparaison d'égalité des plages fermées est prise en charge, et deux plages sont considérées comme égales si elles produisent la même séquence d'entiers. Par exemple, les deux plages suivantes

Q#

```
let r1 = 0..2..5; // generates the sequence 0,2,4
let r2 = 0..2..4; // generates the sequence 0,2,4
```

sont considérés comme égaux. La comparaison d'égalité des plages ouvertes n'est pas prise en charge.

## Comparaison quantitative

Les opérateurs *inférieurs à* (`<`), *inférieurs ou égaux* (`<=`), *supérieur à* (`>`) et *supérieur ou égal à* (`>=`) définissent des comparaisons quantitatives. Ils ne peuvent être appliqués qu'aux types de données qui prennent en charge ces comparaisons, c'est-à-dire les

mêmes types de données qui peuvent également prendre en charge [expressions arithmétiques](#).

---

## Commentaires

Cette page a-t-elle été utile ?

 **Yes**

 **No**

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)



# Expression logiques

Article • 21/03/2025

Les opérateurs logiques sont exprimés sous forme de mots clés. Q# prend en charge les opérateurs logiques standard *AND* (`and`), *OR* (`or`) et *NOT* (`not`). Actuellement, il n'existe pas d'opérateur pour un *XOR logique*. Tous ces opérateurs agissent sur les opérandes de type `Bool`, et entraînent une expression de type `Bool`. Comme dans la plupart des langages, l'évaluation de *AND* et *OR* court-circuits, ce qui signifie que si la première expression de *OR* est évaluée à `true`, la deuxième expression n'est pas évaluée et la même chose tient si la première expression de *AND* est évaluée à `false`. Le comportement des expressions conditionnelles dans un sens est similaire, car seule la condition et l'une des deux expressions est évaluée.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Expressions au niveau du bit

Article • 21/03/2025

Les opérateurs au niveau du bit sont exprimés sous la forme de trois caractères autres que des lettres. En plus des versions au niveau du bit pour *AND* (`&&&`), *OR* (`|||`) et *NOT* (`~~~`), un *XOR* (`^^^`) au niveau du bit existe également. Ils attendent des opérandes de type `Int` ou `BigInt`, et pour les opérateurs binaires, le type des deux opérandes doit correspondre. Le type de l'expression entière est égal au type des opérandes.

En outre, les opérateurs de gauche et de droite (`<<<` et `>>>` respectivement) existent, multiplient ou divisent l'expression de gauche (lhs) donnée par les pouvoirs de deux. L'expression `lhs <<< 3` déplace la représentation binaire de `lhs` par trois, ce qui signifie que `lhs` est multipliée par  $2^3$ , à condition qu'elle se trouve toujours dans la plage valide pour le type de données de `lhs`. Les lhs peuvent être de type `Int` ou `BigInt`. L'expression de droite doit toujours être de type `Int`. L'expression résultante est du même type que l'opérande lhs.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Expressions arithmétiques

Article • 21/03/2025

Les opérateurs arithmétiques sont addition (+), soustraction (-), multiplication (\*), division (/), négation (-) et exposant (^). Ils peuvent être appliqués aux opérandes de type `Int`, `BigInt` ou `Double`. En outre, pour les types intégraux (`Int` et `BigInt`), un opérateur calculant le module (%) est disponible.

Pour les opérateurs binaires, le type des deux opérandes doit correspondre, à l'exception de l'exposant ; un exposant pour une valeur de type `BigInt` doit être de type `Int`. Le type de l'expression entière correspond au type de l'opérande gauche. Pour l'exposant de `Int` et de `BigInt`, le comportement n'est pas défini si l'exposant est négatif ou nécessite plus de 32 bits à représenter (autrement dit, s'il est supérieur à 2147483647).

Division et module pour les valeurs de type `Int` et `BigInt` suivent le comportement suivant pour les nombres négatifs :

[Agrandir le tableau](#)

A	B	A / B	A % B
5	2	2	1
5	-2	-2	1
-5	2	-2	-1
-5	-2	2	-1

Autrement dit, `a % b` a toujours le même signe que `a`, et `b * (a / b) + a % b` est toujours égal `a`.

Q# ne prend pas en charge les conversions automatiques entre les types de données arithmétiques ou d'autres types de données. Cela est particulièrement important pour le type de données `Result` et facilite la propagation des informations d'exécution. Il a l'avantage d'éviter les erreurs accidentelles, telles que celles liées à la perte de précision.

## Commentaires

Cette page a-t-elle été utile ?



Yes



No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Concaténation

Article • 21/03/2025

Les concaténations sont prises en charge pour les valeurs de type `String` et de tableaux. Dans les deux cas, ils sont exprimés via l'opérateur `+`. Par exemple, `"Hello " + "world!"` prend la valeur `"Hello world!"` et `[1, 2, 3] + [4, 5, 6]` prend la valeur `[1, 2, 3, 4, 5, 6]`.

La concaténation de deux tableaux nécessite que les deux tableaux soient du même type. Cette exigence diffère de la construction d'un littéral de tableau où le compilateur détermine un type de base commun pour tous les éléments de tableau. Cette différence est due au fait que les tableaux sont traités comme invariants. Le type de l'expression entière correspond au type des opérandes.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Application de foncteur

Article • 21/03/2025

Les foncteurs sont des fabriques qui vous permettent d'accéder à des implémentations de spécialisation particulières d'un appelant. Q# prend actuellement en charge deux functors ; les `Adjoint` et les `Controlled`, dont les deux peuvent être appliqués aux opérations qui fournissent les spécialisations nécessaires.

Les `Controlled` et les functors `Adjoint` se déplacent ; si `ApplyUnitary` est une opération qui prend en charge les deux functors, il n'existe aucune différence entre `Controlled Adjoint ApplyUnitary` et `Adjoint Controlled ApplyUnitary`. Les deux ont le même type et, lors de l'appel, exécutez l'implémentation définie pour la spécialisation `controlled adjoint`.

## Functor adjoint

Si l'opération `ApplyUnitary` définit une transformation unitaire  $U$  de l'état quantique, `Adjoint ApplyUnitary` accède à l'implémentation de  $U^\dagger$ . Le functor `Adjoint` est son propre inverse, car  $(U^\dagger)^\dagger = U$  par définition. Par exemple, `Adjoint Adjoint ApplyUnitary` est identique à `ApplyUnitary`.

L'expression `Adjoint ApplyUnitary` est une opération du même type que `ApplyUnitary` ; il a le même argument et le même type de retour et prend en charge les mêmes functors. Comme toute opération, elle peut être appelée avec un argument de type approprié. L'expression suivante applique la spécialisation `adjoint` de `ApplyUnitary` à un argument `arg`:

```
Q#
```

```
Adjoint ApplyUnitary(arg)
```

## Fonctor contrôlé

Pour une opération `ApplyUnitary` qui définit une transformation unitaire  $U$  de l'état quantique, `Controlled ApplyUnitary` accède à l'implémentation qui applique  $U$  conditionnelle sur tous les qubits d'un tableau de qubits de contrôle étant dans l'état  $|1\rangle$ .

L'expression `Controlled ApplyUnitary` est une opération avec le même type de retour et [caractéristiques d'opération](#) comme `ApplyUnitary`, ce qui signifie qu'elle prend en charge les mêmes functors. Il prend un argument de type `(Qubit[], <TIn>)`, où `<TIn>` doit être remplacé par le type d'argument de `ApplyUnitary`, en prenant [équivalence de tuple singleton](#) en compte.

[Agrandir le tableau](#)

Opération	Type d'argument	Type d'argument contrôlé
X	<code>Qubit</code>	<code>(Qubit[], Qubit)</code>
ÉCHANGER	<code>(Qubit, Qubit)</code>	<code>(Qubit[], (Qubit, Qubit))</code>

Concrètement, si `cs` contient un tableau de qubits, `q1` et `q2` sont deux qubits, et que l'opération `SWAP` est définie [ici](#), l'expression suivante échange l'état de `q1` et `q2` si tous les qubits dans `cs` sont dans l'état  $|1\rangle$  :

Q#

```
Controlled SWAP(cs, (q1, q2))
```

#### ⓘ Notes

L'application conditionnelle d'une opération basée sur les qubits de contrôle dans un état autre que l'état  $|1\rangle$  peut être obtenue en appliquant la transformation pouvant être adjointe appropriée aux qubits de contrôle avant l'appel et en appliquant les inverses après. Le conditionnement de la transformation sur tous les qubits de contrôle étant dans l'état  $|0\rangle$ , par exemple, peut être obtenu en appliquant l'opération de `X` avant et après. Cela peut être exprimé de manière pratique à l'aide d'une [conjugation](#). Néanmoins, le verbe d'une telle construction peut mériter une prise en charge supplémentaire d'une syntaxe plus compacte à l'avenir.

## Commentaires

Cette page a-t-elle été utile ?

☐ Yes

☐ No

[Indiquer des commentaires sur le produit](#) | [Obtenir de l'aide sur Microsoft Q&A](#)

# Accès aux éléments

Article • 21/03/2025

Q# prend en charge l'accès aux éléments de tableau et aux types `struct`. Dans les deux cas, l'accès est en lecture seule ; la valeur ne peut pas être modifiée sans créer une nouvelle instance à l'aide d'une expression [copy-and-update](#).

## Accès aux éléments de tableau et découpage de tableau

Étant donné une expression de tableau et une expression de type `Int` ou `Range`, une nouvelle expression peut être formée à l'aide de l'opérateur d'accès aux éléments de tableau composé de `[` et de `]`.

Si l'expression entre crochets est de type `Int`, la nouvelle expression contient l'élément de tableau à cet index. Par exemple, si `arr` est de type `Double[]` et contient cinq éléments ou plus, `arr[4]` est une expression de type `Double`.

Si l'expression entre crochets est de type `Range`, la nouvelle expression contient un tableau de tous les éléments indexés par le `Range` spécifié. Si la `Range` est vide, le tableau résultant est vide. Par exemple

Q#

```
let arr = [10, 11, 36, 49];
let ten = arr[0]; // contains the value 10
let odds = arr[1..2..4]; // contains the value [11, 49]
let reverse = arr[...-1...]; // contains the value [49, 36, 11, 10]
```

Dans la dernière ligne de l'exemple, la valeur de début et de fin de la plage a été omise par commodité. Pour plus d'informations, consultez [expressions contextuelles](#).

Si l'expression de tableau n'est pas un identificateur simple, elle doit être placée entre parenthèses pour extraire un élément ou une tranche. Par exemple, si `arr1` et `arr2` sont les deux tableaux d'entiers, un élément de la concaténation est exprimé sous la forme `(arr1 + arr2)[13]`. Pour plus d'informations, consultez [priorité et l'associativité](#).

Tous les tableaux de Q# sont de base zéro, c'est-à-dire le premier élément d'un tableau `arr` est toujours `arr[0]`. Une exception est levée au moment de l'exécution si l'index ou l'un des index utilisés pour le découpage est en dehors des limites du tableau, par exemple s'il est inférieur à zéro ou supérieur ou égal à la longueur du tableau.



# Accès aux éléments pour les types de struct

Les éléments contenus dans un `struct` sont accessibles de deux façons :

- Par référence, à l'aide d'un point (`.`)
- Par déconstruction, à l'aide de l'opérateur `unwrap` (`!`).

Par exemple, étant donné le struct `Complex`

Q#

```
struct Complex {
 Real : Double,
 Imaginary : Double,
}

// create an instance of type Complex
let complex = Complex(1., 0.);

// item access via deconstruction
let (re, _) = complex!;

// item access via name reference
let im = complex.Imaginary;
```

L'accès via la déconstruction utilise l'opérateur `unwrap` (`!`). L'opérateur `unwrap` retourne un tuple de tous les éléments contenus, où la forme du tuple correspond à celle définie dans la déclaration, et un tuple d'élément unique équivaut à l'élément lui-même (voir [cette section](#)).

L'opérateur `!` a une priorité inférieure que les deux opérateurs d'accès aux éléments, mais une priorité plus élevée que n'importe quel autre opérateur. Pour obtenir la liste complète des précédences, consultez [priorité et l'associativité](#).

Pour plus d'informations sur la syntaxe de déconstruction, consultez [déclarations et réaffectations de variables](#).

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Expressions d'appel

Article • 21/03/2025

Les expressions d'appel constituent une partie importante de n'importe quel langage de programmation. Les appels d'opération et de fonction, comme [applications partielles](#), peuvent être utilisés en tant qu'expression n'importe où tant que la valeur retournée est d'un type approprié.

L'utilité des opérations d'appel dans ce formulaire réside principalement dans le débogage, et ces appels d'opération sont l'une des constructions les plus courantes dans n'importe quel programme Q#. En même temps, les opérations ne peuvent être appelées qu'à partir d'autres opérations et non à partir de fonctions. Pour plus d'informations, consultez également [Qubits](#).

Les appelants étant des valeurs de première classe, les expressions d'appel sont un moyen générique de prendre en charge les modèles qui ne sont pas assez courants pour mériter leur propre construction de langage dédié, ou la syntaxe dédiée n'a pas encore été introduite pour d'autres raisons. Voici quelques exemples de méthodes de bibliothèque qui servent cet objectif sont `ApplyIf`, qui appelle une opération conditionnelle sur un bit classique défini ; `ApplyToEach`, qui applique une opération donnée à chaque élément d'un tableau ; et `ApplyWithInputTransformation`, comme indiqué dans l'exemple suivant.

Q#

```
operation ApplyWithInputTransformation<'TArg, 'TIn>(
 fn : 'TIn -> 'TArg,
 op : 'TArg => Unit,
 input : 'TIn
) : Unit {

 op(fn(input));
}
```

`ApplyWithInputTransformation` prend une fonction `fn`, une opération `op` et une valeur `input` en tant qu'arguments, puis applique la fonction donnée à l'entrée avant d'appeler l'opération donnée avec la valeur retournée par la fonction.

Pour que le compilateur génère automatiquement les spécialisations pour prendre en charge des foncteurs [particuliers](#), il faut généralement que les opérations appelées prennent également en charge ces foncteurs. Les deux exceptions sont des appels dans des blocs externes de [conjugations](#), qui doivent toujours prendre en charge le foncteur

Adjoint, mais jamais besoin de prendre en charge les opérations Controlled, et les opérations auto-adjoints, qui prennent en charge le fonctor Adjoint sans imposer d'exigences supplémentaires sur les appels individuels.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Retours et terminaison

Article • 21/03/2025

Il existe deux expressions disponibles qui concluent l'exécution de la sous-routine actuelle ou du programme ; les expressions `return` et `fail`. En règle générale, les appelants peuvent mettre fin à leur exécution avant d'exécuter toutes leurs instructions avec une expression `return` ou `fail`. Une expression `return` termine simplement l'exécution de l'appelant actuel, tandis qu'une `fail` met fin à l'exécution de l'ensemble du programme et entraîne une erreur d'exécution.

## Expression de retour

L'expression `return` quitte l'appelant actuel et retourne le contrôle à l'appelé. Il modifie le contexte de l'exécution en faisant apparaître une trame de pile.

L'expression retourne toujours une valeur au contexte de l'appelé ; il se compose du mot clé `return`, suivi d'une expression du type approprié. La valeur de retour est évaluée avant que toutes les actions de fin soient effectuées et que le contrôle soit retourné. Les actions de fin incluent, par exemple, le nettoyage et la libération des qubits alloués dans le contexte de l'appelant. Lors de l'exécution sur un simulateur ou un validateur, les actions de fin incluent souvent des vérifications liées à l'état de ces qubits. Par exemple, ils peuvent vérifier s'ils sont correctement démêlés de tous les qubits qui restent en direct.

L'expression `return` à la fin d'un appelant qui retourne une valeur `Unit` peut être omise. Dans ce cas, le contrôle est retourné automatiquement lorsque toutes les instructions se terminent et que toutes les actions de fin sont effectuées. Les appelants peuvent contenir plusieurs expressions `return`, même si l'implémentation adjointe pour les opérations contenant plusieurs expressions `return` ne peut pas être générée automatiquement.

Par exemple

```
Q#
```

```
return 1;
```

ou

```
Q#
```

```
return ();
```

## Expression d'échec

L'expression `fail` termine entièrement le calcul. Il correspond à une erreur irrécupérable qui abandonne le programme.

Il se compose du mot clé `fail`, suivi d'une expression de type `String`. Le `String` doit fournir des informations sur l'échec rencontré.

Par exemple

Q#

```
fail "Impossible state reached";
```

ou, à l'aide d'une chaîne interpolée ,

Q#

```
fail $"Syndrome {syn} is incorrect";
```

En plus de la `String` donnée, une expression `fail` collecte idéalement et permet la récupération d'informations sur l'état du programme. Cela facilite le diagnostic et la correction de la source de l'erreur, et nécessite la prise en charge du runtime et du microprogramme en cours d'exécution, qui peuvent varier entre différentes cibles.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Itérations

Article • 21/03/2025

Les boucles qui itèrent sur une séquence de valeurs sont exprimées sous forme de boucles `for` dans Q#. Une boucle `for` dans Q# ne s'arrête pas en fonction d'une condition, mais correspond plutôt à une itération, ou ce qui est souvent exprimé en tant que `foreach` ou `iter` dans d'autres langues. Il existe actuellement deux types de données dans Q# qui prennent en charge l'itération : les tableaux et les plages.

L'expression se compose du mot clé `for`, suivi d'un symbole ou d'un tuple de symboles, du mot clé `in`, d'une expression de type tableau ou `Range` et d'un bloc d'instructions.

Le bloc d'instructions (le corps de la boucle) est exécuté à plusieurs reprises, avec une ou plusieurs variables de boucle liées à chaque valeur de la plage ou du tableau. Les mêmes règles de déconstruction s'appliquent aux variables de boucle définies que les autres affectations de variables, telles que les liaisons dans `let`, les `mutable`, les `use` et les instructions `borrow`. Les variables de boucle elles-mêmes sont immuablement liées, ne peuvent pas être réaffectées dans le corps de la boucle et sortir de portée lorsque la boucle se termine. L'expression sur laquelle la boucle itère est évaluée avant d'entrer dans la boucle et ne change pas pendant l'exécution de la boucle.

Ce scénario est illustré dans l'exemple suivant. Supposons `qubits` est une valeur de type `Qubit[]`, puis

Q#

```
for qubit in qubits {
 H(qubit);
}

mutable results : (Int, Result)[] = [];
for index in 0 .. Length(qubits) - 1 {
 results += [(index, M(qubits[index]))];
}

mutable accumulated = 0;
for (index, measured) in results {
 if measured == One {
 accumulated += 1 <<< index;
 }
}
```

## Restrictions spécifiques à la cible

Étant donné qu'il n'existe aucune `break` ou `continue` primitives dans Q#, la longueur de la boucle est connue dès que la valeur d'itération est connue. Par conséquent, `for` boucles peuvent être exécutées sur tout le matériel quantique.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Boucles conditionnelles

Article • 21/03/2025

Comme la plupart des langages de programmation classiques, Q# prend en charge les boucles qui s'arrêtent en fonction d'une condition : boucles pour lesquelles le nombre d'itérations est inconnu et peut varier d'une exécution à l'autre. Étant donné que la séquence d'instructions est inconnue au moment de la compilation, le compilateur gère ces boucles conditionnelles de manière particulière dans un runtime quantique.

## ❗ Important

### restrictions matérielles **Quantum**

Les boucles qui s'arrêtent en fonction d'une condition sont difficiles à traiter sur le matériel quantique *si la condition dépend des résultats de mesure*, car la longueur de la séquence d'instructions à exécuter n'est pas connue à l'avance.

Malgré leur présence courante dans des classes particulières d'algorithmes quantiques, le matériel actuel ne fournit pas encore de prise en charge native pour ces types de constructions de flux de contrôle. L'exécution de ces types de boucles sur du matériel quantique peut être prise en charge ultérieurement en imposant un nombre maximal d'itérations, ou lorsque d'autres prises en charge matérielles sont disponibles. Toutefois, les simulateurs quantiques exécutent toutes les boucles en fonction des mesures.

## Compilation de boucles

Tant que la condition ne dépend pas des mesures quantiques, les boucles conditionnelles sont traitées avec une compilation juste-à-temps avant d'envoyer la séquence d'instructions au processeur quantique. En particulier, l'utilisation de boucles conditionnelles dans des fonctions n'est pas emblématique, car le code dans les fonctions peut toujours s'exécuter sur du matériel conventionnel (non quantique). Q# prend donc en charge l'utilisation de boucles de `while` traditionnelles au sein des fonctions.

## Répéter l'expression

Lorsque vous exécutez des programmes sur des simulateurs quantiques, Q# vous permet d'exprimer le flux de contrôle qui dépend des résultats des mesures quantiques.



Cette fonctionnalité permet des implémentations probabilistes qui peuvent réduire considérablement les coûts de calcul. Un exemple courant est le *modèle de répétition jusqu'à la réussite*, qui répète un calcul jusqu'à une certaine condition, qui dépend généralement d'une mesure, est satisfait. Ces boucles `repeat` sont largement utilisées dans des classes particulières d'algorithmes quantiques. Q# a donc une construction de langage dédiée pour les exprimer, malgré qu'elles présentent toujours un défi pour l'exécution sur le matériel quantique.

L'expression `repeat` prend la forme suivante

```
Q#

repeat {
 // ...
}
until condition
fixup {
 // ...
}
```

ou alternativement

```
Q#

repeat {
 // ...
}
until condition;
```

où `condition` est une expression arbitraire de type `Bool`.

La boucle `repeat` exécute un bloc d'instructions avant d'évaluer une condition. Si la condition a la valeur `true`, la boucle se ferme. Si la condition prend la valeur `false`, un bloc d'instructions supplémentaire défini dans le cadre d'un bloc de `fixup` facultatif, le cas échéant, est exécuté avant d'entrer l'itération de la boucle suivante.

## Boucle While

Une boucle plus familière pour les calculs classiques est la boucle `while`, qui se compose du mot clé `while`, d'une expression de type `Bool` et d'un bloc d'instructions. Par exemple, si `arr` est un tableau d'entiers positifs,

```
Q#
```

```
mutable (item, index) = (-1, 0);
while index < Length(arr) && item < 0 {
 item = arr[index];
 index += 1;
}
```

Le bloc d'instructions est exécuté tant que la condition est évaluée à `true`.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Branchement conditionnel

Article • 21/03/2025

La branche conditionnelle est exprimée sous la forme d'expressions `if`. Une expression `if` se compose d'une clause `if`, suivie de zéro ou plusieurs clauses `elif` et éventuellement d'un autre bloc. Chaque clause suit le modèle

Q#

```
keyword condition {
 <statements>
}
```

où `keyword` est remplacé par `if` ou `elif` respectivement, `condition` est une expression de type `Bool` et `<statements>` doit être remplacé par zéro ou plusieurs instructions. Le `else`-block facultatif se compose du mot clé `else` suivi de zéro ou plusieurs instructions placées entre accolades, `{ }`.

Premier bloc pour lequel le `condition` est évalué à `true` s'exécutera. Le bloc `else`, le cas échéant, s'exécute si aucune des conditions n'est évaluée à `true`. Le bloc est exécuté dans sa propre étendue, ce qui signifie que toutes les liaisons effectuées dans le cadre du bloc ne sont pas visibles après la fin du bloc.

Par exemple, supposons que `qubits` est une valeur de type `Qubit[]` et `r1` et `r2` sont de type `Result`,

Q#

```
if r1 == One {
 let q = qubits[0];
 H(q);
}
elif r2 == One {
 let q = qubits[1];
 H(q);
}
else {
 H(qubits[2]);
}
```

Vous pouvez également exprimer une branche simple sous la forme d'une expression conditionnelle .

# Restrictions spécifiques à la cible

L'intégration étroite entre les constructions de flux de contrôle et les calculs quantiques pose un défi pour le matériel quantique actuel. Certains processeurs quantiques ne prennent pas en charge la branche en fonction des résultats de mesure. Par conséquent, la comparaison des valeurs de type `Result` entraîne toujours une erreur de compilation pour les programmes Q# qui sont ciblés pour s'exécuter sur ce matériel.

D'autres processeurs quantiques prennent en charge des types spécifiques de branchement basés sur les résultats de mesure. Les expressions de `if` plus générales prises en charge dans Q# sont compilées en instructions appropriées qui peuvent être exécutées sur de tels processeurs. Les restrictions imposées sont que les valeurs de type `Result` peuvent uniquement être comparées dans le cadre de la condition dans `if` expressions dans les opérations. En outre, les blocs d'exécution conditionnelle ne peuvent pas contenir d'expressions `return` ou mettre à jour des variables mutables déclarées en dehors de ce bloc.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Conjugaisons

Article • 21/03/2025

Les conjugaisons sont courantes dans les calculs quantiques. En termes mathématiques, ils sont des modèles de la forme  $U^\dagger V U$  pour deux transformations unitaires  $U$  et  $V$ . Ce modèle est pertinent en raison des particularités de la mémoire quantique : les calculs créent des corrélations quantiques, ou, pour utiliser les ressources uniques du quantum. Toutefois, cela signifie également qu'une sous-routine n'a plus besoin de ses qubits, ces qubits ne peuvent pas facilement être réinitialisés et libérés, car l'observation de leur état aurait un impact sur le reste du système. Pour cette raison, l'effet d'un calcul précédent doit généralement être inversé avant de libérer et de réutiliser la mémoire quantique.

Q# a donc une construction dédiée pour exprimer des calculs nécessitant un tel nettoyage. Les expressions se composent de deux blocs de code, l'un contenant l'implémentation de  $U$  et l'autre contenant l'implémentation de  $V$ . La de non-compatibilité (autrement dit, l'application de  $U^\dagger$ ) est effectuée automatiquement dans le cadre de l'expression.

L'expression prend la forme

Q#

```
within {
 <statements>
}
apply {
 <statements>
}
```

où `<statements>` est remplacé par un nombre quelconque d'instructions définissant l'implémentation de  $U$  et  $V$  respectivement. Les deux blocs peuvent contenir des calculs classiques arbitraires, à part les restrictions habituelles pour générer automatiquement des versions adjoints qui s'appliquent au bloc `within`. Les variables liées de manière mutable utilisées dans le cadre du bloc `within` peuvent ne pas être réaffectées dans le cadre du bloc `apply`.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No



# Fermetures

Article • 21/03/2025

Les fermetures sont des appelants qui capturent des variables à partir de l'environnement englobant. Les fermetures de fonction et d'opération peuvent être créées. Une fermeture d'opération peut être créée à l'intérieur d'une fonction, mais elle ne peut être appliquée qu'à une opération.

Q# a deux mécanismes pour créer des fermetures : les expressions lambda et l'application partielle.

## Expressions lambda

Une expression lambda crée une fonction ou une opération anonyme. La syntaxe de base est un tuple de symboles pour lier les paramètres, une flèche (`->` pour une fonction et `=>` pour une opération) et une expression à évaluer lorsqu'elle est appliquée.

Q#

```
// Function that captures 'x':
y -> x + y

// Operation that captures 'qubit':
deg => Rx(deg * PI() / 180.0, qubit)

// Function that captures nothing:
(x, y) -> x + y
```

## Paramètres

Les paramètres sont liés à l'aide d'un tuple de symboles identique au côté gauche d'une instruction de déclaration de variable. Le type du tuple de paramètre est implicite. Les annotations de type ne sont pas prises en charge ; si l'inférence de type échoue, vous devrez peut-être créer une déclaration appelante de niveau supérieur et utiliser une application partielle à la place.

## Variables de capture mutables

Les variables mutables ne peuvent pas être capturées. Si vous devez uniquement capturer la valeur d'une variable mutable à l'instant où l'expression lambda est créée, vous pouvez créer une copie immuable :

Q#

```
// ERROR: 'variable' cannot be captured.
mutable variable = 1;
let f = () -> variable;

// OK.
let value = variable;
let g = () -> value;
```

## Caractéristiques

Les caractéristiques d'une opération anonyme sont déduites en fonction des applications de l'expression lambda. Si l'expression lambda est utilisée avec une application fonctoreuse ou dans un contexte qui attend une caractéristique, l'expression lambda est alors déduite d'avoir cette caractéristique. Par exemple:

Q#

```
operation NoOp(q : Qubit) : Unit is Adj {}
operation Main() : Unit {
 use q = Qubit();
 let foo = () => NoOp(q);
 foo(); // Has type Unit => Unit with no characteristics

 let bar = () => NoOp(q);
 Adjoint bar(); // Has type Unit => Unit is Adj
}
```

Si vous avez besoin de caractéristiques différentes pour une opération lambda que ce qui a été déduit, vous devez créer une déclaration d'opération de niveau supérieur à la place.

## Application partielle

L'application partielle est un raccourci pratique pour appliquer certains arguments d'un appelant, mais pas tous. La syntaxe est identique à une expression d'appel, mais les arguments nonappliés sont remplacés par `_`. Conceptuellement, l'application partielle équivaut à une expression lambda qui capture les arguments appliqués et accepte les arguments non appliqués en tant que paramètres.

Par exemple, étant donné que `f` est une fonction et `o` est une opération, et que la variable capturée `x` est immuable :



Application partielle	Expression lambda
<code>f(x, _)</code>	<code>a -&gt; f(x, a)</code>
<code>o(x, _)</code>	<code>a =&gt; o(x, a)</code>
<code>f(_, (1, _))</code>	<code>(a, b) -&gt; f(a, (1, b))^[1]</code>
<code>f(_, _, x), (1, _)</code>	<code>((a, b), c) -&gt; f((a, b, x), (1, c))</code>

## Variables de capture mutables

Contrairement aux expressions lambda, l'application partielle peut capturer automatiquement une copie de la valeur d'une variable mutable :

Q#

```
mutable variable = 1;
let f = Foo(variable, _);
```

Cela équivaut à l'expression lambda suivante :

Q#

```
mutable variable = 1;
let value = variable;
let f = x -> Foo(value, x);
```

[^1] : le tuple de paramètre est strictement écrit `(a, (b))`, mais `(b)` équivaut à `b`.

## Commentaires

Cette page a-t-elle été utile ?

👍 Yes

👎 No

[Indiquer des commentaires sur le produit](#) 🔗 | [Obtenir de l'aide sur Microsoft Q&A](#)

# Expressions contextuelles et omises

Article • 21/03/2025

Les expressions contextuelles sont des expressions qui sont valides uniquement dans certains contextes, telles que l'utilisation de noms d'éléments dans [expressions de copie et de mise à jour](#) sans avoir à les qualifier.

Les expressions peuvent être *omises* lorsqu'elles peuvent être déduites et insérées automatiquement par le compilateur, telles que [instructions evaluate-and-reassignment](#).

Les plages ouvertes sont un autre exemple qui s'applique aux expressions contextuelles et omises. Ils sont valides uniquement dans un certain contexte, et le compilateur les traduit en expressions `Range` normales pendant la compilation en inférence des limites appropriées.

Une valeur de type `Range` génère une séquence d'entiers, spécifiée par une valeur de début, une valeur d'étape (facultative) et une valeur de fin. Par exemple, l'expression littérale `Range 1..3` génère la séquence 1,2,3. De même, l'expression `3..-1..1` génère la séquence 3,2,1. Vous pouvez également utiliser des plages pour créer un tableau à partir d'un tableau existant en découpage, par exemple :

Q#

```
let arr = [1,2,3,4];
let slice1 = arr[1..2..4]; // contains [2,4]
let slice2 = arr[2..-1..0]; // contains [3,2,1]
```

Vous ne pouvez pas définir une plage infinie dans Q#; Les valeurs de début et de fin doivent toujours être spécifiées. La seule exception est lorsque vous utilisez un `Range` pour découper un tableau. Dans ce cas, le compilateur peut raisonnablement déduire les valeurs de début ou de fin de la plage.

Dans les exemples précédents de découpage de tableau, il est raisonnable pour le compilateur de supposer que la fin de plage prévue doit être l'index du dernier élément du tableau si la taille de l'étape est positive. Si la taille de l'étape est négative, la fin de la plage doit probablement être l'index du premier élément du tableau, `0`. L'inverse contient le début de la plage.

Pour résumer, si vous omettez la valeur de début de plage, la valeur de début déduite

- est égal à zéro si aucune étape n'est spécifiée ou si l'étape spécifiée est positive.
- est la longueur du tableau moins un si l'étape spécifiée est négative.

Si vous omettez la valeur de fin de plage, la valeur de fin déduite

- est la longueur du tableau moins une si aucune étape n'est spécifiée ou si l'étape spécifiée est positive.
- est égal à zéro si l'étape spécifiée est négative.

Q# permet donc l'utilisation de plages ouvertes au sein d'expressions de découpage de tableau, par exemple :

Q#

```
let arr = [1,2,3,4,5,6];
let slice1 = arr[3...]; // slice1 is [4,5,6];
let slice2 = arr[0..2...]; // slice2 is [1,3,5];
let slice3 = arr[...2]; // slice3 is [1,2,3];
let slice4 = arr[...2..3]; // slice4 is [1,3];
let slice5 = arr[...2...]; // slice5 is [1,3,5];
let slice7 = arr[4..-2...]; // slice7 is [5,3,1];
let slice8 = arr[...-1..3]; // slice8 is [6,5,4];
let slice9 = arr[...-1...]; // slice9 is [6,5,4,3,2,1];
let slice10 = arr[...]; // slice10 is [1,2,3,4,5,6];
```

Étant donné que la détermination de l'étape de plage est positive ou négative se produit au moment de l'exécution, le compilateur insère une expression appropriée évaluée au moment de l'exécution. Pour les valeurs de fin omises, l'expression insérée est `step < 0 ? 0 | Length(arr)-1` et, pour les valeurs de début omises, elle est `step < 0 ? Length(arr)-1 | 0`, où `step` est l'expression donnée pour l'étape de plage, ou `1` si aucune étape n'est spécifiée.

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

## Littéral d'unité

Le seul littéral existant pour le type `Unit` est la valeur `()`.

La valeur `Unit` est couramment utilisée comme argument pour les appelants, soit parce qu'aucun autre argument n'a besoin d'être passé ou de retarder l'exécution. Il est également utilisé comme valeur de retour lorsqu'aucune autre valeur ne doit être retournée, c'est-à-dire pour les opérations unitaires, c'est-à-dire les opérations qui prennent en charge le `Adjoint` et/ou le fonctor `Controlled`.

## Littéraux int

Les littéraux de valeur pour le type `Int` peuvent être exprimés en représentation binaire, octal, décimale ou hexadécimale. Les littéraux exprimés en binaire sont précédés de `0b`, avec `0o` pour octal et avec `0x` pour hexadécimal. Il n'existe aucun préfixe pour la représentation décimale couramment utilisée.

[🔍 Agrandir le tableau](#)

Représentation	Littéral de valeur
Binaire	<code>0b101010</code>
Octal	<code>0o52</code>
Décimal	<code>42</code>
Format hexadécimal	<code>0x2a</code>

## Littéraux BigInt

Les littéraux de valeur pour le type `BigInt` sont toujours postfixés avec `L` et peuvent être exprimés en représentation binaire, octale, décimale ou hexadécimale. Les littéraux exprimés en binaire sont précédés de `0b`, avec `0o` pour octal et avec `0x` pour hexadécimal. Il n'existe aucun préfixe pour la représentation décimale couramment utilisée.

Représentation	Littéral de valeur
Binaire	<code>0b101010L</code>
Octal	<code>0o52L</code>
Décimal	<code>42L</code>
Format hexadécimal	<code>0x2aL</code>

## Littéraux doubles

Les littéraux de valeur pour le type `Double` peuvent être exprimés en notation standard ou scientifique.

Représentation	Littéral de valeur
Norme	<code>0.1973269804</code>
Scientifique	<code>1.973269804e-1</code>

Si rien ne suit après la virgule décimale, le chiffre après le point décimal peut être omis. Par exemple, `1.` est un littéral `Double` valide et identique à `1.0`.

## Littéraux Bool

Les littéraux existants pour le type `Bool` sont `true` et `false`.

## Littéraux de chaîne

Un littéral de valeur pour le type `String` est une séquence de longueur arbitraire de caractères Unicode entre guillemets doubles. À l'intérieur d'une chaîne, le caractère de barre oblique inverse `\` peut être utilisé pour échapper un caractère de guillemet double et insérer une nouvelle ligne en tant que `\n`, un retour chariot en tant que `\r` et un onglet comme `\t`.

Voici des exemples pour les littéraux de chaîne valides :

```
Q#
```

```
"This is a simple string."
"\\"This is a more complex string.\", she said.\n"
```

Q# prend également en charge les chaînes interpolées . Une chaîne interpolée est un littéral de chaîne qui peut contenir un nombre quelconque d'expressions d'interpolation. Ces expressions peuvent être de types arbitraires. Lors de la construction, les expressions sont évaluées et leur représentation `String` est insérée à l'emplacement correspondant dans le littéral défini. L'interpolation est activée en préinitialisant le caractère spécial `$` directement avant la citation initiale, sans espace blanc entre eux.

Par exemple, si `res` est une expression qui prend la valeur `1`, la deuxième phrase du littéral `String` suivant affiche « Le résultat était 1 ».

```
Q#
```

```
$"This is an interpolated string. The result was {res}."
```

## Littéraux Qubit

Aucun littéral pour le type `Qubit` n'existe, car la mémoire quantique est gérée par le runtime. Les valeurs de type `Qubit` ne peuvent donc être obtenues qu'via d'allocation.

Les valeurs de type `Qubit` représentent un identificateur opaque par lequel un bit quantique, ou qubit, peut être traité. Le seul opérateur qu'ils prennent en charge est [comparaison d'égalité](#). Pour plus d'informations sur le type de données `Qubit`, consultez [Qubits](#).

## Littéraux de résultats

Les littéraux existants pour le type `Result` sont `Zero` et `One`.

Les valeurs de type `Result` représentent le résultat d'une mesure quantique binaire. `Zero` indique une projection sur l'espace propre +1, `One` indique une projection sur l'espace propre -1.

## Littéraux Pauli

Les littéraux existants pour le type `Pauli` sont `PauliI`, `PauliX`, `PauliY` et `PauliZ`.

Les valeurs de type `Pauli` représentent l'une des quatre matrices à qubit unique [Pauli](#), avec `PauliI` représentant l'identité. Les valeurs de type `Pauli` sont couramment utilisées pour désigner l'axe des rotations et spécifier la base à mesurer.

## Littéraux de plage

Les littéraux de valeur du type `Range` sont des expressions du formulaire `start..step..stop`, où `start`, `step` et `end` sont des expressions de type `Int`. Si la taille de l'étape est une, elle peut être omise. Par exemple, `start..stop` est un littéral `Range` valide et identique à `start..1..stop`.

Les valeurs de type `Range` représentent une séquence d'entiers, où le premier élément de la séquence est `start`, et les éléments suivants sont obtenus en ajoutant `step` à la précédente, jusqu'à ce que `stop` soit passé. `Range` valeurs sont inclusives aux deux extrémités, autrement dit, le dernier élément de la plage est `stop` si la différence entre `start` et `stop` est un multiple de `step`. Une plage peut être vide si, par exemple, `step` est positive et `stop < start`.

Voici des exemples pour les littéraux `Range` valides :

- `1..3` correspond à la plage 1, 2, 3.
- `2..2..5` correspond à la plage 2, 4.
- `2..2..6` correspond à la plage 2, 4, 6.
- `6..-2..2` correspond à la plage 6, 4, 2.
- `2..-2..1` correspond à la plage 2.
- `2..1` est la plage vide.

Pour plus d'informations, consultez [expressions contextuelles](#).

## Littéraux de tableau

Un tableau [littéral](#) est une séquence de zéro ou plusieurs expressions, séparées par des virgules et placées entre crochets `[` et `]`; par exemple, `[1,2,3]`. Toutes les expressions doivent avoir un type de base [commun](#), qui est le type d'élément du tableau. Si un tableau vide est spécifié avec `[]`, une annotation de type peut être nécessaire au compilateur pour déterminer le type approprié de l'expression.

Des tableaux de longueur arbitraire peuvent être créés à l'aide d'une expression de tableau de taille. Une telle expression est de la forme `[expr, size = s]`, où `s` peut être n'importe quelle expression de type `Int` et `expr` est évaluée à une valeur qui sera les

éléments du tableau répété `s` fois. Par exemple, `[1.2, size = 3]` crée le même tableau que `[1.2, 1.2, 1.2]`.

## Littéraux tuples

Un tuple littéral est une séquence d'une ou plusieurs expressions de tout type, séparées par des virgules et placées entre parenthèses `(` et `)`. Le type du tuple inclut les informations relatives à chaque type d'élément.

 Agrandir le tableau

Littéral de valeur	Catégorie
<code>("Id", 0, 1.)</code>	<code>(String, Int, Double)</code>
<code>(PauliX,(3,1))</code>	<code>(Pauli, (Int, Int))</code>

Les tuples contenant un seul élément sont traités comme identiques à l'élément lui-même, à la fois dans le type et la valeur, qui est appelé [équivalence de tuple singleton](#).

Les tuples sont utilisés pour regrouper des valeurs en une seule valeur, ce qui facilite leur passage. Cela permet à chaque appelant de prendre exactement une entrée et de retourner exactement une sortie.

## Littéraux pour les types de structs

Les valeurs d'un type de struct sont attribuées lorsque vous créez une instance, à l'aide de l'instruction `new` ou du constructeur créé par le compilateur.

Par exemple, si `IntPair` est une `struct` définie comme

Q#

```
struct IntPair {
 num1 : Int,
 num2 : Int,
}
```

puis vous créez une instance et attribuez des valeurs à l'aide de

Q#

```
let MyPair = new IntPair { num1 = 5, num2 = 7 };
```



ou

Q#

```
let MyPair = IntPair(5, 7);
```

## Littéraux d'opération et de fonction

Les opérations et fonctions anonymes peuvent être créées à l'aide d'une expression lambda .

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Système de types

Article • 21/03/2025

Avec le focus pour l'algorithme quantique étant plus vers ce qui doit être réalisé plutôt que sur une représentation problématique en termes de structures de données, prendre une perspective plus fonctionnelle sur la conception du langage est un choix naturel. En même temps, le système de type est un mécanisme puissant qui peut être utilisé pour l'analyse du programme et d'autres vérifications au moment de la compilation qui facilitent la simulation de code robuste.

Dans l'ensemble, le système de type Q# est assez minimaliste, dans le sens où il n'y a pas de notion explicite de classes ou d'interfaces, car il peut être utilisé à partir de langages classiques comme C# ou Java. Nous prenons également une approche quelque peu pragmatique qui fait des progrès incrémentiels, de sorte que certaines constructions ne sont pas encore entièrement intégrées au système de type. Voici un exemple de foncteurs, qui peuvent être utilisés dans des expressions, mais qui n'ont pas encore de représentation dans le système de type. En conséquence, ils ne peuvent pas être affectés ou passés en tant qu'arguments, comme c'est le cas pour les callables paramétrables de type. Nous prévoyons d'effectuer des progrès incrémentiels dans l'extension du système de type afin qu'il soit plus complet et équilibrer les besoins immédiats avec les plans à long terme.

## Types disponibles

Tous les types de Q# sont immuables .

[🔍 Agrandir le tableau](#)

Catégorie	Description
Unit	Représente un type singleton dont la seule valeur est <code>()</code> .
Int	Représente un entier signé 64 bits. Valeurs comprise entre -9 223 372 036 854 775 808 et 9 223 372 036 854 775 807.
BigInt	Représente les valeurs d'entier signé de n'importe quelle taille.
Double	Représente un nombre à virgule flottante double précision 64 bits. Valeurs plage comprise entre -1.79769313486232e308 à 1,79769313486232e308, ainsi que NaN (pas un nombre).
Bool	Représente les valeurs booléennes. Les valeurs possibles sont <code>true</code> ou <code>false</code> .

Catégorie	Description
String	Représente du texte sous forme de valeurs qui se composent d'une séquence d'unités de code UTF-16.
Qubit	Représente un identificateur opaque par lequel la mémoire quantique virtuelle peut être traitée. Valeurs de type Qubit sont instanciées via l'allocation.
Result	Représente le résultat d'une mesure projective sur les espaces propres d'un opérateur quantique avec des valeurs propres $\pm 1$ . Les valeurs possibles sont Zero ou One.
Pauli	Représente une matrice Pauli à qubit unique. Les valeurs possibles sont PauliI, PauliX, PauliY ou PauliZ.
Range	Représente une séquence ordonnée de valeurs de Int égales. Valeurs peuvent représenter des séquences dans l'ordre croissant ou décroissant.
Tableau	Représente valeurs qui contiennent chacune une séquence de valeurs du même type.
Tuple	Représente valeurs qui contiennent chacun un nombre fixe d'éléments de différents types. Les tuples contenant un seul élément sont équivalents à l'élément qu'ils contiennent.
struct	Représente un type défini par l'utilisateur composé d'éléments nommés de différents types. Valeurs sont instanciées lors de la déclaration d'une nouvelle instance.
Opération	Représente un non déterministe qui accepte un argument d'entrée (éventuellement tuple) et retourne une sortie (éventuellement tuple-valued). Les appels à l'opération valeurs peuvent avoir des effets secondaires et la sortie peut varier pour chaque appel, même lorsqu'elle est appelée avec le même argument.
Fonction	Représente un déterministe qui accepte un argument d'entrée (éventuellement tuple) et retourne une sortie (éventuellement tuple-valued). Les appels à la fonction valeurs n'ont pas d'effets secondaires et la sortie sera toujours la même en fonction de la même entrée.

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#) | [Obtenir de l'aide sur Microsoft Q&A](#)

# Types de données spécifiques au quantum

Article • 21/03/2025

Cette rubrique décrit le type `Qubit`, ainsi que deux autres types qui sont un peu spécifiques au domaine quantique : `Pauli` et `Result`.

## Qubit

Q# traite les qubits comme des éléments opaques qui peuvent être passés à la fois aux fonctions et aux opérations, mais ne peuvent être interagissant qu'en les transmettant à des instructions natives au processeur quantique ciblé. Ces instructions sont toujours définies sous la forme d'opérations, car leur intention est de modifier l'état quantique. La restriction que les fonctions ne peuvent pas modifier l'état quantique, malgré le fait que les qubits peuvent être passés en tant qu'arguments d'entrée, est appliquée en exigeant que les fonctions ne puissent appeler que d'autres fonctions et ne peuvent pas appeler d'opérations.

Les bibliothèques Q# sont compilées par rapport à un ensemble standard d'opérations intrinsèques, ce qui signifie que les opérations qui n'ont aucune définition pour leur implémentation dans le langage. Lors du ciblage, les implémentations qui les expriment en termes d'instructions natives de la cible d'exécution sont liées par le compilateur. Un programme Q# combine donc ces opérations comme défini par une machine cible pour créer de nouvelles opérations de niveau supérieur pour exprimer le calcul quantique. De cette façon, Q# facilite l'expression de la logique des algorithmes quantiques et quantiques hybrides, tout en étant très général par rapport à la structure d'une machine cible et à sa réalisation de l'état quantique.

Dans Q# lui-même, il n'existe aucun type ou construction dans Q# qui représente l'état quantique. Au lieu de cela, un qubit représente la plus petite unité physique adressable dans un ordinateur quantique. Par conséquent, un qubit est un élément de longue durée, donc Q# n'a pas besoin de types linéaires. Par conséquent, nous ne faisons pas explicitement référence à l'état dans Q#, mais nous décrivons plutôt comment l'état est transformé par le programme, par exemple via l'application d'opérations telles que `X` et `H`. Similaire à la façon dont un programme de nuanceur graphique accumule une description des transformations de chaque vertex, un programme quantique dans Q# accumule des transformations en états quantiques, représentées sous forme de références entièrement opaques à la structure interne d'une machine cible.

Un programme Q# n'a aucune capacité à introspecter dans l'état d'un qubit, et est donc entièrement indépendant de ce qu'est un état quantique ou sur la façon dont il est réalisé. Au lieu de cela, un programme peut appeler des opérations telles que `Measure` pour en savoir plus sur l'état quantique du calcul.

## Pauli

Les valeurs de type `Pauli` spécifier un opérateur Pauli à qubit unique ; les possibilités sont `PauliI`, `PauliX`, `PauliY` et `PauliZ`. `Pauli` valeurs sont utilisées principalement pour spécifier la base d'une mesure quantique.

## Résultat

Le type `Result` spécifie le résultat d'une mesure quantique. Q# reflète la plupart du matériel quantique en fournissant des mesures dans les produits d'opérateurs Pauli à qubit unique ; une `Result` de `Zero` indique que la valeur propre +1 a été mesurée et qu'une `Result` de `One` indique que la valeur propre -1 a été mesurée. Autrement dit, Q# représente les valeurs propres par la puissance à laquelle -1 est levée. Cette convention est plus courante dans la communauté des algorithmes quantiques, car elle correspond plus étroitement aux bits classiques.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Immuabilité

Article • 21/02/2025

Tous les types de Q# sont *types valeur*. Q# n'a pas de concept de référence ou de pointeur. Au lieu de cela, il vous permet de réaffecter une nouvelle valeur à une variable précédemment déclarée via une expression d'affectation. Par exemple, il n'existe aucune distinction dans le comportement entre les réaffectations pour les variables de type `Int` ou les variables de type `Int[]`. Considérez la séquence d'instructions suivante :

Q#

```
mutable arr1 = new Int[3];
let arr2 = arr1;
arr1 w/= 0 <- 3;
```

La première instruction instancie un nouveau tableau d'entiers `[0,0,0]` et l'affecte à `arr1`. L'instruction suivante affecte cette valeur à une variable portant le nom `arr2`. La dernière instruction crée une instance de tableau basée sur `arr1` avec les mêmes valeurs, à l'exception de la valeur à l'index 0, qui est définie sur 3. Le tableau nouvellement créé est ensuite affecté à la variable `arr1`. La dernière ligne utilise la syntaxe abrégée pour [instructions evaluate-and-reassignment](#), et pourrait avoir été écrite de manière équivalente en tant que `arr1 = arr1 w/ 0 <- 1;`.

Après avoir exécuté les trois instructions, `arr1` contiendra la valeur `[3,0,0]` tandis que `arr2` reste inchangée et contient la valeur `[0,0,0]`.

Q# distingue clairement la mutabilité d'un handle et le comportement d'un type. La mutabilité dans Q# est un concept qui s'applique à un symbole plutôt qu'à un type ou une valeur ; elle s'applique au handle qui vous permet d'accéder à une valeur plutôt qu'à la valeur elle-même. Il n'est *pas* représenté dans le système de type, implicitement ou explicitement.

Bien sûr, il s'agit simplement d'une description du comportement formellement défini ; sous le capot, l'implémentation réelle utilise un schéma de comptage de référence pour éviter de copier la mémoire autant que possible. La modification est spécifiquement effectuée tant qu'il n'existe qu'un seul handle actuellement valide qui accède à une certaine valeur.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Opérations et fonctions

Article • 21/03/2025

Comme indiqué plus en détail dans la description du type de données qubit , les calculs quantiques sont exécutés sous la forme d'effets secondaires d'opérations pris en charge en mode natif sur le processeur quantique ciblé. Il s'agit en fait des seuls effets secondaires dans Q#. Étant donné que tous les types sont immuables, aucun effet secondaire n'impacte une valeur qui est explicitement représentée dans Q#. Par conséquent, tant qu'une implémentation d'un certain appelant n'appelle pas directement ou indirectement l'une de ces opérations implémentées en mode natif, son exécution produit toujours la même sortie, en fonction de la même entrée.

Q# vous permet de fractionner explicitement ces calculs purement déterministes en fonctions . Étant donné que l'ensemble d'instructions prises en charge en mode natif n'est pas fixe et intégré au langage lui-même, mais plutôt entièrement configurable et exprimé en tant que bibliothèque Q#, le déterminisme est garanti en exigeant que les fonctions puissent appeler uniquement d'autres fonctions et ne peuvent pas appeler d'opérations. En outre, les instructions natives qui ne sont pas déterministes, c'est-à-dire parce qu'elles ont un impact sur l'état quantique, sont représentées en tant qu'opérations. Avec ces deux restrictions, les fonctions peuvent être évaluées dès que leur valeur d'entrée est connue et, en principe, ne doivent jamais être évaluées plusieurs fois pour la même entrée.

Q# distingue donc deux types de [pouvant être appelées](#): opérations et fonctions. Tous les appelants acceptent un seul argument (potentiellement tuple-valued) comme entrée et produisent une valeur unique (tuple) comme sortie. Syntactiquement, le type d'opération est exprimé en tant que `<TIn> => <TOut> is <Char>`, où `<TIn>` doit être remplacé par le type d'argument, `<TOut>` doit être remplacé par le type de retour, et `<Char>` doit être remplacé par les caractéristiques de l'opération . Si aucune caractéristique n'a besoin d'être spécifiée, la syntaxe simplifie la `<TIn> => <TOut>`. De même, les types de fonctions sont exprimés en tant que `<TIn> -> <TOut>`.

En dehors de cette garantie déterminisme, il y a peu de différence entre les opérations et les fonctions. Les deux sont des valeurs de première classe qui peuvent être transmises librement ; ils peuvent être utilisés comme valeurs de retour ou arguments pour d'autres appelants, comme illustré dans l'exemple suivant :

Q#

```
function Pow<'T>(op : 'T => Unit, pow : Int) : 'T => Unit {
 return PowImpl(op, pow, _);
}
```



```
}
```

Les deux peuvent être instanciés en fonction d'une définition paramétrable par type, par exemple, la fonction `paramétrable` `Pow` précédemment, et elles peuvent être `partiellement appliquées` comme fait dans l'instruction `return` dans l'exemple.

## Caractéristiques de l'opération

En plus des informations sur le type d'entrée et de sortie, le type d'opération contient des informations sur les caractéristiques d'une opération. Ces informations, par exemple, décrivent les foncteurs pris en charge par l'opération. En outre, la représentation interne contient également des informations pertinentes sur l'optimisation qui sont déduites par le compilateur.

Les caractéristiques d'une opération sont un ensemble d'étiquettes prédéfinies et intégrées. Elles sont exprimées sous la forme d'une expression spéciale qui fait partie de la signature de type. L'expression se compose de l'un des ensembles prédéfinis d'étiquettes ou d'une combinaison d'expressions de caractéristiques via un opérateur binaire pris en charge.

Il existe deux ensembles prédéfinis, `Adj` et `Ctl`.

- `Adj` est l'ensemble qui contient une étiquette unique indiquant qu'une opération est adjoint, ce qui signifie qu'elle prend en charge le foncteur `Adjoint` et que la transformation quantique appliquée peut être « annulée », c'est-à-dire qu'elle peut être inversée.
- `ctl` est l'ensemble qui contient une étiquette unique indiquant qu'une opération est contrôlable, ce qui signifie qu'elle prend en charge le foncteur `Controlled` et son exécution peut être conditionnée sur l'état d'autres qubits.

Les deux opérateurs pris en charge dans le cadre d'expressions de caractéristiques sont l'union définie `+` et l'intersection définie `*`. Dans EBNF (extended Backus–Naur form),

```
predefined = "Adj" | "Ctl";
characteristics = predefined
 | "(" , characteristics , ")"
 | characteristics ("+"|"*") characteristics;
```

Comme prévu, `*` a une priorité plus élevée que `+` et les deux sont associatifs de gauche. Le type d'une opération unitaire, par exemple, est exprimé en tant que `<TIn> => <TOut>`

is Adj + Ct1, où <TIn> doit être remplacé par le type de l'argument d'opération, et <TOut> remplacé par le type de la valeur retournée.

### 📌 Notes

L'indication des caractéristiques d'une opération de cette forme présente deux avantages majeurs ; pour une, de nouvelles étiquettes peuvent être introduites sans avoir de mots clés de langage exponentiellement pour toutes les combinaisons d'étiquettes. Plus important encore, l'utilisation d'expressions pour indiquer les caractéristiques d'une opération prend également en charge les paramétrages sur les caractéristiques d'opération à l'avenir.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Équivalence de tuple singleton

Article • 21/03/2025

Pour éviter toute ambiguïté entre les tuples et les parenthèses qui regroupent les sous-expressions, un tuple avec un seul élément est considéré comme équivalent à l'élément contenu, y compris son type. Par exemple, les types `Int`, `(Int)` et `((Int))` sont traités comme étant identiques. Il en va de même pour les valeurs `5`, `(5)` et `((5))`, ou pour `(5, (6))` et `(5, 6)`. Cette équivalence s'applique à toutes fins, y compris l'affectation. Étant donné qu'il n'existe aucune répartition dynamique ou réflexion dans Q# et que tous les types de Q# sont résolubles au moment de la compilation, l'équivalence de tuple singleton peut être facilement implémentée pendant la compilation.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Sous-typage et variance

Article • 21/03/2025

Q# ne prend en charge que quelques mécanismes de conversion. Les conversions implicites peuvent se produire uniquement lors de l'application d'opérateurs binaires, de l'évaluation des expressions conditionnelles ou de la construction d'un littéral de tableau. Dans ces cas, un supertype commun est déterminé et les conversions nécessaires sont effectuées automatiquement. Outre ces conversions implicites, les conversions explicites via les appels de fonction sont possibles et souvent nécessaires.

Actuellement, la seule relation de sous-saisie qui existe s'applique aux opérations. Intuitivement, il est logique que l'on puisse remplacer une opération qui prend en charge plus que l'ensemble requis de functors. Concrètement, pour deux types concrets `TIn` et `TOut`, la relation de sous-frappe est

```
(TIn => TOut) :>
(TIn => TOut is Adj), (TIn => TOut is Ctl) :>
(TIn => TOut is Adj + Ctl)
```

où `A :> B` indique que `B` est un sous-type de `A`. Autrement dit, `B` est plus restrictif que `A` afin qu'une valeur de type `B` puisse être utilisée partout où une valeur de type `A` est requise. Si un appelant s'appuie sur un argument (élément) de type `A`, un argument de type `B` peut être substitué en toute sécurité, car s'il fournit toutes les fonctionnalités nécessaires.

Ce type de polymorphisme s'étend aux tuples dans lequel un tuple de type `B` est un sous-type d'un type tuple `A` s'il contient le même nombre d'éléments et que le type de chaque élément est un sous-type du type d'élément correspondant dans `A`. C'est ce qu'on appelle *sous-frappe de profondeur*. Il n'existe actuellement aucune prise en charge des sous-types de largeur, autrement dit, il n'existe aucune relation de sous-type entre deux types `struct` ou un type `struct` et n'importe quel type intégré. L'existence de l'opérateur `unwrap`, qui vous permet d'extraire un tuple contenant tous les éléments nommés, empêche cela.

## 📌 Notes

En ce qui concerne les appelants, si un appelant traite un argument de type `A`, il est également capable de traiter un argument de type `B`. Si un appelant est passé en

tant qu'argument à un autre appelant, il doit être capable de traiter tout ce dont la signature de type peut nécessiter. Cela signifie que si l'appelant doit être en mesure de traiter un argument de type  $B$ , tout appelant capable de traiter un argument plus général de type  $A$  peut être passé en toute sécurité. À l'inverse, nous nous attendons à ce que si nous exigeons que l'appelant transmise retourne une valeur de type  $A$ , la promesse de retourner une valeur de type  $B$  est suffisante, car cette valeur fournira toutes les fonctionnalités nécessaires.

L'opération où le type de fonction est contravariant dans son type d'argument et covariant dans son type de retour.  $A \rightarrow B$  implique donc que pour tout type concret  $T1$ ,

$$(B \rightarrow T1) \rightarrow (A \rightarrow T1), \text{ and}$$
$$(T1 \rightarrow A) \rightarrow (T1 \rightarrow B)$$

où  $\rightarrow$  ici peut signifier une fonction ou une opération, et nous omettez toutes les annotations pour les caractéristiques. Le remplacement de  $A$  par  $(B \rightarrow T2)$  et de  $(T2 \rightarrow A)$  respectivement, et le remplacement de  $B$  par  $(A \rightarrow T2)$  et de  $(T2 \rightarrow B)$  respectivement conduit à la conclusion que, pour tout type concret  $T2$ ,

$$((A \rightarrow T2) \rightarrow T1) \rightarrow ((B \rightarrow T2) \rightarrow T1), \text{ and}$$
$$((T2 \rightarrow B) \rightarrow T1) \rightarrow ((T2 \rightarrow A) \rightarrow T1), \text{ and}$$
$$(T1 \rightarrow (B \rightarrow T2)) \rightarrow (T1 \rightarrow (A \rightarrow T2)), \text{ and}$$
$$(T1 \rightarrow (T2 \rightarrow A)) \rightarrow (T1 \rightarrow (T2 \rightarrow B))$$

Par induction, il suit que chaque indirection supplémentaire inverse la variance du type d'argument et laisse la variance du type de retour inchangée.

### ⓘ Notes

Cela permet également de clarifier le comportement de variance des tableaux ; la récupération d'éléments via un opérateur d'accès aux éléments correspond à l'appel d'une fonction de type  $(Int \rightarrow TItem)$ , où  $TItem$  est le type des éléments dans le tableau. Étant donné que cette fonction est implicitement passée lors du passage d'un tableau, elle suit que les tableaux doivent être covariants dans leur type d'élément. Les mêmes considérations tiennent également pour les tuples, qui sont immuables et donc covariants par rapport à chaque type d'élément. Si les

tableaux n'étaient pas immuables, l'existence d'une construction qui vous permettrait de définir des éléments dans un tableau, et donc de prendre un argument de type `TItem`, implique que les tableaux doivent également être contravariants. La seule option pour les types de données qui prennent en charge l'obtention et la définition d'éléments est donc d'être *invariant*, ce qui signifie qu'il n'y a aucune relation de sous-saisie ; `B[]` n'est *pas* un sous-type de `A[]` même si `B` est un sous-type de `A`. Malgré le fait que les tableaux dans Q# sont immuables, ils sont invariants plutôt que covariants. Cela signifie, par exemple, qu'une valeur de type `(Qubit => Unit is Adj)[]` ne peut pas être passée à un appelant qui nécessite un argument de type `(Qubit => Unit)[]`. La conservation des tableaux invariant permet une plus grande flexibilité liée à la façon dont les tableaux sont gérés et optimisés dans le runtime, mais il peut être possible de le réviser à l'avenir.

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Paramétrages de types

Article • 21/03/2025

Q# prend en charge les opérations et fonctions paramétrables de type. Les bibliothèques standard Q# utilisent fortement les callables paramétrables de type pour fournir une multitude d'abstractions utiles, notamment des fonctions telles que `Mapped` et `Fold` familières à partir de langages fonctionnels.

Pour motiver le concept de paramétrage de type, considérez l'exemple de la fonction `Mapped`, qui applique une fonction donnée à chaque valeur d'un tableau et retourne un nouveau tableau avec les valeurs calculées. Cette fonctionnalité peut être parfaitement décrite sans spécifier les types d'éléments des tableaux d'entrée et de sortie. Étant donné que les types exacts ne modifient pas l'implémentation de la fonction `Mapped`, il est logique qu'il soit possible de définir cette implémentation pour les types d'éléments arbitraires ; Nous voulons définir une *fabrique* ou *modèle* qui, compte tenu des types concrets des éléments dans le tableau d'entrée et de sortie, retourne l'implémentation de fonction correspondante. Cette notion est formalisée sous la forme de paramètres de type.

## Concrétisation

Toute opération ou déclaration de fonction peut spécifier un ou plusieurs paramètres de type qui peuvent être utilisés comme types, ou partie des types, de l'entrée ou de la sortie de l'appelant, ou les deux. Les exceptions sont des points d'entrée, qui doivent être concrets et ne peuvent pas être paramétrés par type. Les noms de paramètres de type commencent par une graduation (') et peuvent apparaître plusieurs fois dans les types d'entrée et de sortie. Tous les arguments qui correspondent au même paramètre de type dans la signature pouvant être appelée doivent être du même type.

Un callable paramétrable de type doit être concret, c'est-à-dire qu'il doit être fourni avec les arguments de type nécessaires avant qu'il puisse être affecté ou passé en tant qu'argument, de sorte que tous les paramètres de type peuvent être remplacés par des types concrets. Un type est considéré comme concret s'il s'agit de l'un des types intégrés, d'un type `struct` ou s'il est concret dans l'étendue actuelle. L'exemple suivant illustre ce qu'il signifie pour qu'un type soit concret dans l'étendue actuelle, et est expliqué plus en détail ci-dessous :

Q#

```
function Mapped<'T1, 'T2> (
 mapper : 'T1 -> 'T2,
```

```

 array : 'T1[]
) : 'T2[] {

 mutable mapped = new 'T2[Length(array)];
 for (i in IndexRange(array)) {
 mapped w/= i <- mapper(array[i]);
 }
 return mapped;
 }

 function AllCControlled<'T3> (
 ops : ('T3 => Unit)[]
) : ((Bool, 'T3) => Unit)[] {

 return Mapped(CControlled<'T3>, ops);
 }

```

La fonction `CControlled` est définie dans l'espace de noms `Microsoft.Quantum.Canon`. Il faut une opération `op` de type `'TIn => Unit` comme argument et retourne une nouvelle opération de type `(Bool, 'TIn) => Unit` qui applique l'opération d'origine, à condition qu'un bit classique (de type `Bool`) soit défini sur *true*; il s'agit souvent de la version contrôlée classiquement de `op`.

La fonction `Mapped` prend un tableau d'un type d'élément arbitraire `'T1` en tant qu'argument, applique la fonction `mapper` donnée à chaque élément et retourne un nouveau tableau de type `'T2[]` contenant les éléments mappés. Il est défini dans l'espace de noms `Microsoft.Quantum.Array`. Dans le cadre de l'exemple, les paramètres de type sont numérotés pour éviter de rendre la discussion plus déroutante en donnant aux paramètres de type dans les deux fonctions le même nom. Cela n'est pas nécessaire ; Les paramètres de type pour différents appelants peuvent avoir le même nom, et le nom choisi est visible et pertinent uniquement dans la définition de cet appelant.

La fonction `AllCControlled` prend un tableau d'opérations et retourne un nouveau tableau contenant les versions contrôlées classiquement de ces opérations. L'appel de `Mapped` résout son paramètre de type `'T1` en `'T3 => Unit`, et son paramètre de type `'T2` à `(Bool, 'T3) => Unit`. Les arguments de type de résolution sont déduits par le compilateur en fonction du type de l'argument donné. Nous disons qu'elles sont *implicitement* définies par l'argument de l'expression d'appel. Les arguments de type peuvent également être spécifiés explicitement, comme c'est le cas pour `CControlled` dans la même ligne. La `CControlled<'T3>` de concédation explicite est nécessaire lorsque les arguments de type ne peuvent pas être déduits.



Le type `'T3` est concret dans le contexte de `AllCControlled`, car il est connu pour chaque appel de `AllCControlled`. Cela signifie que dès que le point d'entrée du programme, qui ne peut pas être paramétré par type, est connu, il s'agit du type concret `'T3` pour chaque appel à `AllCControlled`, de sorte qu'une implémentation appropriée pour cette résolution de type particulière peut être générée. Une fois que le point d'entrée d'un programme est connu, toutes les utilisations des paramètres de type peuvent être éliminées au moment de la compilation. Nous appelons ce processus comme *monomorphisation*.

Certaines restrictions sont nécessaires pour s'assurer que cela peut effectivement être effectué au moment de la compilation plutôt qu'au moment de l'exécution.

## Restrictions

Prenons l'exemple suivant :

Q#

```
operation Foo<'TArg> (
 op : 'TArg => Unit,
 arg : 'TArg
) : Unit {

 let cbit = RandomInt(2) == 0;
 Foo(CControlled(op), (cbit, arg));
}
```

En ignorant qu'un appel de `Foo` entraîne une boucle infinie, il sert à l'illustration. `Foo` s'appelle avec la version contrôlée classiquement de l'opération d'origine `op` qui a été passée, ainsi qu'un tuple contenant un bit classique aléatoire en plus de l'argument d'origine.

Pour chaque itération dans la récursivité, le paramètre de type `'TArg` de l'appel suivant est résolu en `(Bool, 'TArg)`, où `'TArg` est le paramètre de type de l'appel actuel. Concrètement, supposons que `Foo` est appelée avec l'opération `H` et un argument `arg` de type `Qubit`. `Foo` appelle ensuite lui-même un argument de type `(Bool, Qubit)`, qui appelle ensuite `Foo` avec un argument de type `(Bool, (Bool, Qubit))`, et ainsi de suite. Clairement, dans ce cas, `Foo` ne peut pas être monomorphisé au moment de la compilation.

Des restrictions supplémentaires s'appliquent aux cycles du graphique d'appel qui impliquent uniquement des callables paramétrables de type. Chaque appelant doit être appelé avec le même jeu d'arguments de type après avoir parcouru le cycle.

### ⓘ Notes

Il serait possible d'être moins restrictif et exiger que pour chaque appelant dans le cycle, il existe un nombre fini de cycles après lequel il est appelé avec l'ensemble d'arguments de type d'origine, tel que le cas de la fonction suivante :

Q#

```
function Bar<'T1,'T2,'T3>(a1:'T1, a2:'T2, a3:'T3) : Unit{
 Bar<'T2,'T3,'T1>(a2, a3, a1);
}
```

Par souci de simplicité, l'exigence la plus restrictive est appliquée. Notez que pour les cycles qui impliquent au moins un appel concret sans paramètre de type, un tel callable garantit que les callables paramétrables de type dans ce cycle sont toujours appelées avec un ensemble fixe d'arguments de type.

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Inférence de type

Article • 21/03/2025

Q#'algorithmme d'inférence de type est basé sur des algorithmes d'inférence conçus pour le système de type Hindley-Milner. Bien que les appelants de niveau supérieur doivent être déclarés avec des annotations de type explicites, la plupart des types utilisés dans un appelant peuvent être déduits. Par exemple, étant donné ces callables :

Q#

```
function Length<'a>(xs : 'a[]) : Int
function Mapped<'a, 'b>(f : 'a -> 'b, xs : 'a[]) : 'b[]
```

et cette expression :

Q#

```
Mapped(Length, [], ["a"], ["b", "c"])
```

puis l'argument de type à `Length` est déduit comme `Length<String[]>`, et les arguments de type à `Mapped` sont déduits comme `Mapped<String[], Int>`. Il n'est pas nécessaire d'écrire ces types explicitement.

## Types ambigus

Il n'existe parfois pas de type principal unique qui peut être déduit pour une variable de type. Dans ces cas, l'inférence de type échoue avec une erreur faisant référence à un type ambigu. Par exemple, modifiez légèrement l'exemple précédent :

Q#

```
Mapped(Length, [[]])
```

Quel est le type de `[[]]` ? Dans certains systèmes de type, il est possible de lui donner le type `∀a. a[][]`, mais cela n'est pas pris en charge dans Q#. Un type concret est requis, mais il existe un nombre infini de types qui fonctionnent : `String[][]`, `(Int, Int)[][]`, `Double[][][]`, et ainsi de suite. Vous devez explicitement indiquer le type que vous avez voulu.

Il existe plusieurs façons de le faire, en fonction de la situation. Par exemple:

1. Appelez `Length` avec un argument de type.

Q#

```
Mapped(Length<String>, [[]])
```

2. Appelez `Mapped` avec son premier argument de type. (Le `_` pour son deuxième argument de type signifie qu'elle doit toujours être déduite.)

Q#

```
Mapped<String[], _>(Length, [[]])
```

3. Remplacez le littéral de tableau vide par un appel explicitement typé à une fonction de bibliothèque.

Q#

```
Mapped<String[], _>(Length, [EmptyArray<String>()])
```

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Tutoriel : Implémenter un générateur de nombres aléatoires quantique en Q#

Dans ce tutoriel, vous allez apprendre à écrire un programme quantique de base qui Q# tire parti de la nature de la mécanique quantique pour produire un nombre aléatoire.

Dans ce didacticiel, vous allez :

- ✓ Créez un programme Q#.
- ✓ Passez en revue les principaux composants d'un Q# programme.
- ✓ Définir la logique d'un problème.
- ✓ Combiner des opérations classiques et quantiques pour résoudre un problème.
- ✓ Utiliser des qubits et la superposition pour créer un générateur de nombres aléatoires quantiques

## Prérequis

Pour développer et exécuter l'exemple de code dans Visual Studio Code (VS Code) et Jupyter Notebook, installez les éléments suivants :

- Dernière version de [VS Code](#) ou ouvrir [VS Code pour le web](#).
- La plus récente version de l'extension QDK. Pour plus d'informations sur l'installation, consultez [Configurer l'extension QDK](#).
- Extensions [Python](#) et [Jupyter](#) pour VS Code.
- La dernière `qdk` bibliothèque Python avec la fonctionnalité supplémentaire `jupyter`. Pour les installer, ouvrez un terminal et exécutez la commande suivante :

Bash

```
pip install --upgrade "qdk[jupyter]"
```

## Définir le problème

Les ordinateurs classiques ne produisent pas de nombres aléatoires, mais plutôt des nombres pseudorandom. Un générateur de nombres pseudo-random génère une séquence déterministe

de nombres en fonction d'une valeur initiale, appelée valeur initiale. Pour obtenir la meilleure approximation possible des valeurs aléatoires, cette valeur seed est souvent l'heure actuelle de l'horloge du processeur.

Les ordinateurs quantiques, d'autre part, peuvent générer des nombres vraiment aléatoires. Cela est dû au fait que la mesure d'un qubit en superposition est un processus probabiliste. Le résultat de la mesure est aléatoire, il n'existe aucun moyen de prédire le résultat. C'est le principe de base des générateurs de nombres aléatoires quantiques.

Un qubit est une unité d'informations quantiques qui peuvent être en superposition. Lorsqu'il est mesuré, un qubit ne peut être qu'à l'état 0 ou dans l'état 1. Toutefois, avant la mesure, l'état du qubit représente la probabilité de lire un 0 ou un 1 avec une mesure.

Pour commencer, prenez un qubit dans un état basique, par exemple zéro. La première étape du générateur de nombres aléatoires consiste à utiliser une opération Hadamard pour placer le qubit dans une superposition égale. La mesure de cet état entraîne un zéro ou un avec une probabilité de 50 % de chaque résultat, un bit vraiment aléatoire.

Il n'existe aucun moyen de savoir ce que vous obtiendrez après la mesure du qubit en superposition, et le résultat est une valeur différente chaque fois que le code est appelé. Mais comment pouvez-vous utiliser ce comportement pour générer des nombres aléatoires plus grands ?

Supposons que vous répétiez le processus quatre fois, ce qui générerait cette suite de chiffres binaires :

0, 1, 1, 0

Si vous concaténez (ou combinez) ces bits dans une chaîne binaire, vous pouvez obtenir un nombre plus grand. Dans cet exemple, la séquence de bits 0110 est équivalente à 6 en décimal.

$$0110_{binary} \equiv 6_{decimal}$$

Si vous répétez ce processus plusieurs fois, vous pouvez combiner plusieurs bits pour former un grand nombre. À l'aide de cette méthode, vous pouvez créer un nombre à utiliser comme mot de passe sécurisé, car vous pouvez être sûr qu'aucun pirate n'a pu déterminer les résultats de la séquence de mesures.

## Définir la logique du générateur de nombres aléatoires

Décrivons la logique d'un générateur de nombres aléatoires :

1. Définissez `max` comme le nombre maximal à générer.
2. Définissez le nombre de bits aléatoires à générer. Pour ce faire, calculez le nombre de bits, `nBits` vous devez exprimer des entiers jusqu'à `max`.
3. Générez une chaîne de bits aléatoires dont la longueur est `nBits`.
4. Si la chaîne de bits représente un nombre supérieur à `max`, revenez à l'étape 3.
5. Sinon, l'opération est terminée. Retournez le nombre généré sous la forme d'un entier.

À titre d'exemple, nous allons définir `max` sur 12. Autrement dit, 12 est le plus grand nombre que vous souhaitez utiliser comme mot de passe.

Vous avez besoin de  $\lfloor \ln(12)/\ln(2) + 1 \rfloor$ , ou de 4 bits pour représenter un nombre compris entre 0 et 12. Nous pouvons utiliser la fonction `BitSizeI` intégrée, qui prend n'importe quel entier et retourne le nombre de bits requis pour le représenter.

Mettons que vous générez la chaîne de bits `1101binary`, qui est équivalente à `{13_\ decimal}}`\$. Comme 13 est supérieur à 12, vous devez répéter le processus.

Ensuite, vous générez la chaîne de bits `0110binary`, qui est équivalente à `6decimal`. Étant donné que 6 est inférieur à 12, le processus est terminé.

Le générateur de nombres aléatoires quantiques retourne le numéro 6 comme mot de passe. Dans la pratique, définissez un nombre plus grand comme valeur maximale, car les nombres inférieurs sont faciles à craquer en essayant juste tous les mots de passe possibles. En fait, pour augmenter la difficulté de deviner ou de craquer votre mot de passe, vous pouvez utiliser le code ASCII pour convertir le binaire en texte et générer un mot de passe en utilisant des chiffres, des symboles et un mélange de lettres majuscules et minuscules.

## Écrire un générateur de bits aléatoire

La première étape consiste à écrire une Q# opération qui génère un bit aléatoire. Cette opération sera l'un des blocs de construction du générateur de nombres aléatoires.

Q#

```
operation GenerateRandomBit() : Result {
 // Allocate a qubit.
 use q = Qubit();

 // Set the qubit into superposition of 0 and 1 using the Hadamard
 H(q);

 // At this point the qubit `q` has 50% chance of being measured in the
 // |0> state and 50% chance of being measured in the |1> state.
```

```
// Measure the qubit value using the `M` operation, and store the
// measurement value in the `result` variable.
let result = M(q);

// Reset qubit to the $|0\rangle$ state.
// Qubits must be in the $|0\rangle$ state by the time they are released.
Reset(q);

// Return the result of the measurement.
return result;
}
```

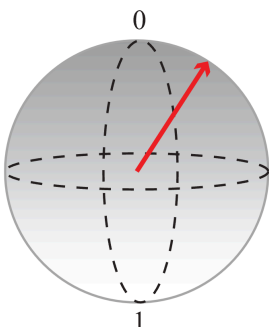
Examinons à présent le nouveau code.

- Vous définissez l'opération `GenerateRandomBit`, qui ne prend aucune entrée et produit une valeur de type `Result`. Le type `Result` représente le résultat d'une mesure et peut avoir deux valeurs : `Zero` ou `One`.
- Vous allouez un qubit unique avec le `use` mot clé. Lorsqu'il est alloué, un qubit est toujours dans l'état  $|0\rangle$ .
- Vous utilisez l'opération `H` pour placer le qubit dans une superposition égale.
- Vous utilisez l'opération `M` pour mesurer le qubit, renvoyer la valeur mesurée (`Zero` ou `One`).
- Vous utilisez l'opération `Reset` pour réinitialiser le qubit à l'état  $|0\rangle$ .

En plaçant le qubit en superposition avec l'opération `H` et en le mesurant avec l'opération `M`, le résultat est une valeur différente chaque fois que le code est appelé.

## Visualiser le Q# code avec la sphère Bloch

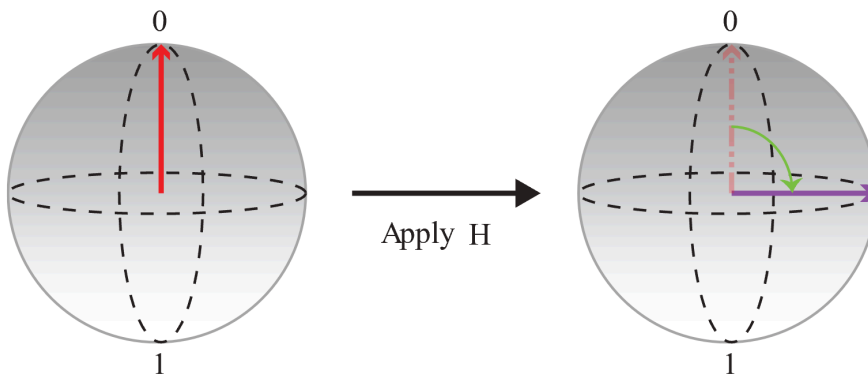
Dans la sphère Bloch, le pôle nord représente la valeur classique 0 et le pôle sud représente la valeur classique 1. Toute superposition peut être représentée par un point sur la sphère (représentée par une flèche). Plus l'extrémité de la flèche est proche d'un pôle, plus la probabilité est élevée que le qubit soit réduit à la valeur classique attribuée à ce pôle lors de la mesure. Par exemple, l'état qubit représenté par la flèche de la figure suivante a une probabilité plus élevée de donner la valeur 0 si vous la mesurez.



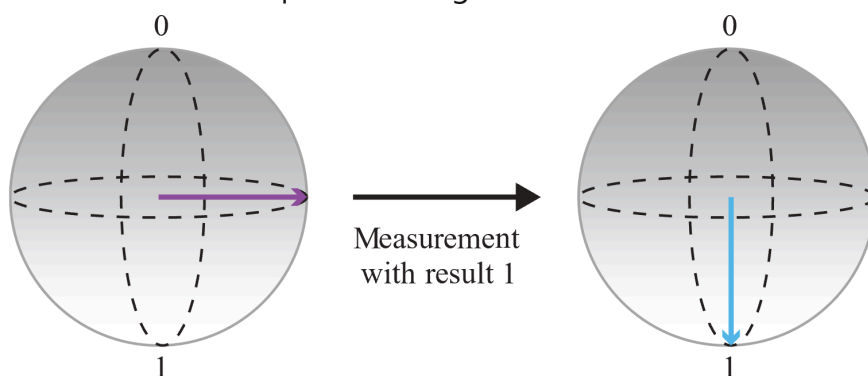
Vous pouvez utiliser cette représentation pour visualiser ce que fait le code :



1. Tout d'abord, commencez par un qubit initialisé dans l'état  $|0\rangle$  et appliquez une **H** opération pour créer une superposition égale dans laquelle les probabilités pour 0 et 1 sont identiques.



2. Ensuite, mesurez le qubit et enregistrez la sortie :



Étant donné que le résultat de la mesure est aléatoire et que les probabilités de mesurer 0 et 1 sont identiques, vous avez obtenu un bit complètement aléatoire. Nous pouvons appeler cette opération plusieurs fois pour créer des entiers. Par exemple, si vous appelez l'opération trois fois pour obtenir trois bits aléatoires, vous pouvez générer des nombres de 3 bits aléatoires (c'est-à-dire, un nombre aléatoire compris entre 0 et 7).

## Écrire un générateur de nombres aléatoires complet

1. Tout d'abord, vous devez importer les espaces de noms requis de la Q# bibliothèque standard dans le programme. Le Q# compilateur charge automatiquement de nombreuses fonctions et opérations courantes, mais pour le générateur de nombres aléatoires complet, vous avez besoin de certaines fonctions et opérations supplémentaires à partir de deux Q# espaces de noms : **Std.Math** et **Std.Convert**.

Q#

```
import Std.Convert.*;
import Std.Math.*;
```

2. Ensuite, vous définissez l'opération `GenerateRandomNumberInRange`. Cette opération appelle à plusieurs reprises l'opération `GenerateRandomBit` pour générer une chaîne de bits.

Q#

```
/// Generates a random number between 0 and `max`.
operation GenerateRandomNumberInRange(max : Int) : Int {
 // Determine the number of bits needed to represent `max` and store it
 // in the `nBits` variable. Then generate `nBits` random bits which will
 // represent the generated random number.
 mutable bits = [];
 let nBits = BitSizeI(max);
 for idxBit in 1..nBits {
 bits += [GenerateRandomBit()];
 }
 let sample = ResultArrayAsInt(bits);

 // Return random number if it is within the requested range.
 // Generate it again if it is outside the range.
 return sample > max ? GenerateRandomNumberInRange(max) | sample;
}
```

Prenons un moment pour examiner ce nouveau code.

- Vous devez calculer le nombre de bits nécessaires pour exprimer des entiers jusqu'à `max`. La `BitSizeI` fonction du `Std.Math` namespace convertit un entier en nombre de bits nécessaires pour le représenter.
- L'opération `SampleRandomNumberInRange` utilise une boucle `for` pour générer des nombres aléatoires jusqu'à générer une valeur inférieure ou égale à `max`. La boucle `for` fonctionne exactement de la même façon qu'une boucle `for` dans d'autres langages de programmation.
- La variable `bits` est une variable mutable. Une variable mutable peut changer pendant le calcul. Vous devez utiliser la directive `set` pour modifier la valeur d'une variable mutable.
- La `ResultArrayAsInt` fonction, à partir de l'espace de noms par défaut `Std.Convert`, convertit la chaîne de bits en entier positif.

3. Enfin, vous ajoutez un point d'entrée au programme. Par défaut, le Q# compilateur recherche une `Main` opération et commence à y traiter. Elle appelle l'opération `GenerateRandomNumberInRange` pour générer un nombre aléatoire compris entre 0 et 100.

Q#

```
operation Main() : Int {
 let max = 100;
 Message($"Sampling a random number between 0 and {max}: ");

 // Generate random number in the 0..max range.
 return GenerateRandomNumberInRange(max);
}
```

La directive `let` déclare des variables qui ne changent pas pendant le calcul. Ici, vous définissez la valeur maximale sur 100.

Pour plus d'informations sur l'opération `Main`, consultez [Points d'entrée](#).

4. Le code complet du générateur de nombres aléatoires est le suivant :

Q#

```
import Std.Convert.*;
import Std.Math.*;

operation Main() : Int {
 let max = 100;
 Message($"Sampling a random number between 0 and {max}: ");

 // Generate random number in the 0..max range.
 return GenerateRandomNumberInRange(max);
}

/// Generates a random number between 0 and `max`.
operation GenerateRandomNumberInRange(max : Int) : Int {
 // Determine the number of bits needed to represent `max` and store it
 // in the `nBits` variable. Then generate `nBits` random bits which will
 // represent the generated random number.
 mutable bits = [];
 let nBits = BitSizeI(max);
 for idxBit in 1..nBits {
 bits += [GenerateRandomBit()];
 }
 let sample = ResultArrayAsInt(bits);

 // Return random number if it is within the requested range.
 // Generate it again if it is outside the range.
 return sample > max ? GenerateRandomNumberInRange(max) | sample;
}

operation GenerateRandomBit() : Result {
 // Allocate a qubit.
 use q = Qubit();

 // Set the qubit into superposition of 0 and 1 using a Hadamard operation
 H(q);
}
```

```

// At this point the qubit `q` has 50% chance of being measured in the
// $|0\rangle$ state and 50% chance of being measured in the $|1\rangle$ state.
// Measure the qubit value using the `M` operation, and store the
// measurement value in the `result` variable.
let result = M(q);

// Reset qubit to the $|0\rangle$ state.
// Qubits must be in the $|0\rangle$ state by the time they are released.
Reset(q);

// Return the result of the measurement.
return result;
}

```

## Exécuter le programme de génération de nombres aléatoires

Choisissez votre environnement de développement préféré, soit l'extension QDK dans VS Code, soit la bibliothèque de Python QDK dans Jupyter Notebook.

Q# fichier dans VS Code

1. Dans VS Code, ouvrez le menu **Fichier** et choisissez **Nouveau fichier texte** pour créer un fichier.
2. Enregistrez le fichier sous le nom `RandomNumberGenerator.qs`. Ce fichier contient le Q# code de votre programme.
3. Copiez le code suivant dans le `RandomNumberGenerator.qs` fichier.

Q#

```

import Std.Convert.*;
import Std.Math.*;

operation Main() : Int {
 let max = 100;
 Message($"Sampling a random number between 0 and {max}: ");

 // Generate random number in the 0..max range.
 return GenerateRandomNumberInRange(max);
}

/// # Summary
/// Generates a random number between 0 and `max`.
operation GenerateRandomNumberInRange(max : Int) : Int {

```

```

 // Determine the number of bits needed to represent `max` and store it
 // in the `nBits` variable. Then generate `nBits` random bits which will
 // represent the generated random number.
 mutable bits = [];
 let nBits = BitSizeI(max);
 for idxBit in 1..nBits {
 bits += [GenerateRandomBit()];
 }
 let sample = ResultArrayAsInt(bits);

 // Return random number if it is within the requested range.
 // Generate it again if it is outside the range.
 return sample > max ? GenerateRandomNumberInRange(max) | sample;
}

/// # Summary
/// Generates a random bit.
operation GenerateRandomBit() : Result {
 // Allocate a qubit.
 use q = Qubit();

 // Set the qubit into superposition of 0 and 1 using the Hadamard
 // operation `H`.
 H(q);

 // At this point the qubit `q` has 50% chance of being measured in the
 // $|0\rangle$ state and 50% chance of being measured in the $|1\rangle$ state.
 // Measure the qubit value using the `M` operation, and store the
 // measurement value in the `result` variable.
 let result = M(q);

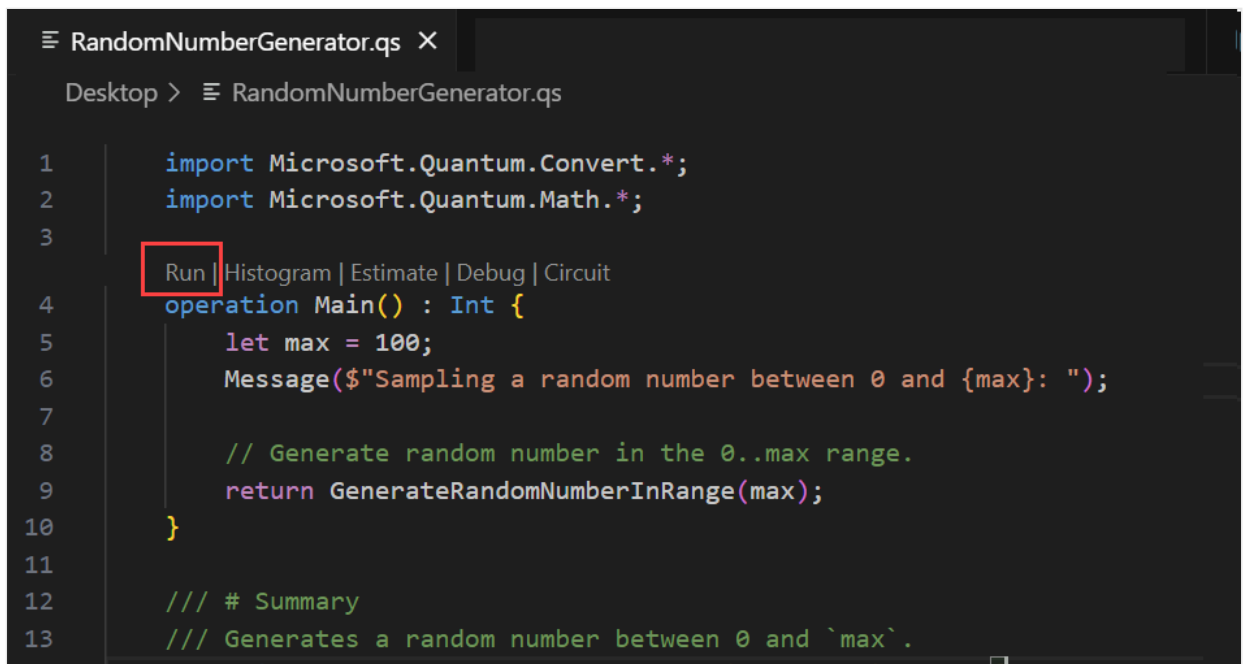
 // Reset qubit to the $|0\rangle$ state.
 // Qubits must be in the $|0\rangle$ state by the time they are released.
 Reset(q);

 // Return the result of the measurement.
 return result;

 // Note that Qubit `q` is automatically released at the end of the
 block.
}

```

4. Pour exécuter votre programme, choisissez l'option **Exécuter** dans la liste des commandes précédentes `Main()`, ou appuyez sur **Ctrl + F5**. Le programme exécute l'opération `Main()` sur le simulateur par défaut.



```
1 import Microsoft.Quantum.Convert.*;
2 import Microsoft.Quantum.Math.*;
3
4 Run | Histogram | Estimate | Debug | Circuit
 operation Main() : Int {
5 let max = 100;
6 Message($"Sampling a random number between 0 and {max}: ");
7
8 // Generate random number in the 0..max range.
9 return GenerateRandomNumberInRange(max);
10 }
11
12 /// # Summary
13 /// Generates a random number between 0 and `max`.
```

5. Votre résultat s’affichera sur la console de débogage.

6. Réexécutez le programme pour voir un résultat différent.

#### ❗ Remarque

Si le profil QIR target n’est pas défini sur **Non restreint**, vous obtenez une erreur lorsque vous exécutez le programme. Pour ce programme, le compilateur définit automatiquement le target profil sur **Non restreint** , sauf si vous définissez le profil vous-même.

## Tracer l’histogramme de fréquence

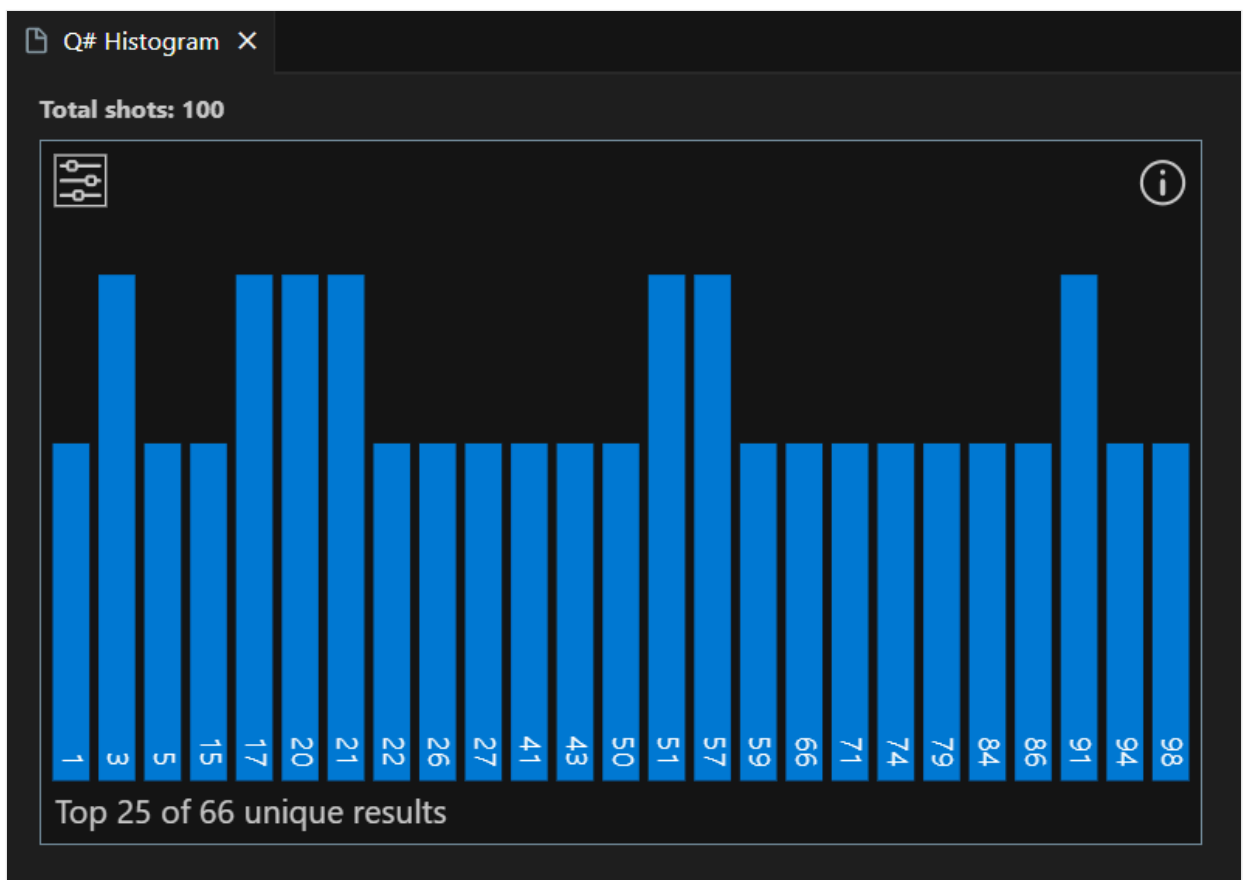
Nous allons visualiser la distribution des résultats obtenus à partir de l’exécution du programme quantique plusieurs fois. L’histogramme de fréquence permet de visualiser la distribution de probabilité de ces résultats.

1. Sélectionnez **Affichage -> Palette de commandes** et tapez **histogramme**, ce qui affiche la **commande QDK : Exécuter le fichier et afficher l’histogramme**. Vous pouvez également sélectionner **histogramme** dans la liste des commandes précédentes `Main()`. Sélectionnez cette option pour ouvrir la fenêtre d’histogramme Q# .

```
RandomNumberGenerator.qs X
Desktop > RandomNumberGenerator.qs

1 import Microsoft.Quantum.Convert.*;
2 import Microsoft.Quantum.Math.*;
3
4 Run Histogram | Estimate | Debug | Circuit
5 operation Main() : Int {
6 let max = 100;
7 Message($"Sampling a random number between 0 and {max}: ");
8
9 // Generate random number in the 0..max range.
10 return GenerateRandomNumberInRange(max);
11 }
12
13 /// # Summary
14 /// Generates a random number between 0 and `max`.
```

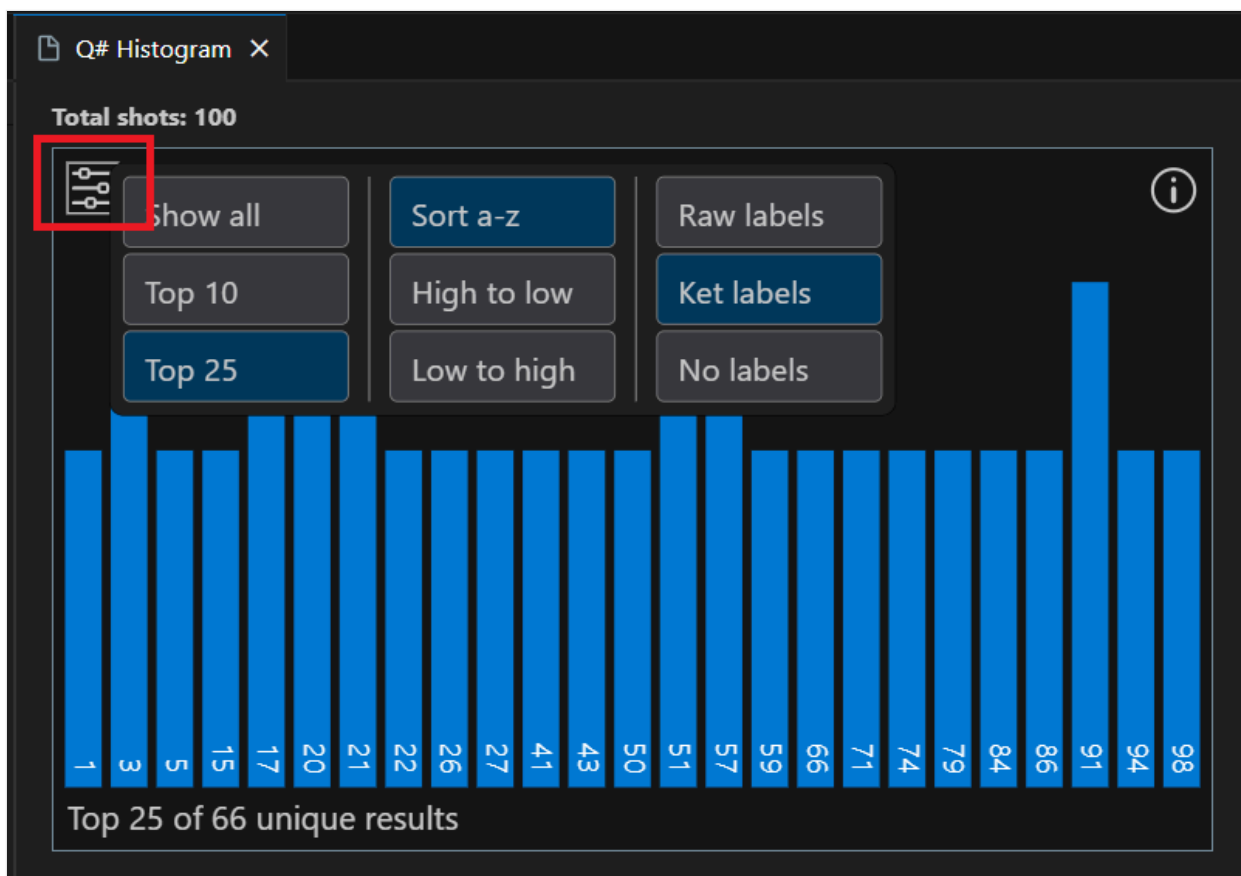
2. Entrez un certain nombre de **captures** pour exécuter le programme, par exemple 100 captures, puis appuyez sur **Entrée**. L'histogramme s'affiche dans la fenêtre d'histogramme Q#.
3. Chaque barre de l'histogramme correspond à un résultat possible, et sa hauteur représente le nombre de fois où le résultat est observé. Le nombre de résultats différents peut différer chaque fois que vous exécutez l'histogramme.



### 💡 Conseil

Vous pouvez effectuer un zoom avant sur l'histogramme à l'aide de la roulette de défilement de la souris ou d'un mouvement de trackpad. Lorsque vous effectuez un zoom avant, vous pouvez parcourir le graphique en appuyant sur « Alt » lors du défilement.

4. Sélectionnez une barre pour afficher le **pourcentage** de ce résultat.
5. Sélectionnez l'icône **des paramètres en haut à gauche** pour afficher les options. Vous pouvez afficher les 10 premiers résultats, les 25 premiers résultats ou tous les résultats. Vous pouvez également trier les résultats d'un niveau élevé à faible ou faible à élevé.



### ⓘ Remarque

Cet extrait de code ne s'exécute actuellement sur aucun matériel targetsAzure Quantum disponible, car l'appelant `ResultArrayAsInt` nécessite un QPU avec un profil de calcul complet.



# Contenu connexe

Explorez les autres tutoriels Q# :

- [L'intrication quantique](#) montre comment écrire un Q# programme qui manipule et mesure des qubits et illustre les effets de la superposition et de l'intrication.
- [L'algorithme](#) de recherche de Grover montre comment écrire un Q# programme qui utilise l'algorithme de recherche de Grover.
- [Quantum Fourier Transforms](#) explore comment écrire un Q# programme qui traite directement des qubits spécifiques.
- Les [Katas Quantiques](#) [↗](#) sont des tutoriels auto-dirigés et des exercices de programmation visant à enseigner les éléments de l'informatique quantique et de la programmation simultanément.

---

Last updated on 19/05/2026

# Tutoriel : explorer l'intrication quantique avec Q#

Dans ce tutoriel, vous écrivez un Q# programme qui prépare deux qubits dans un état quantique spécifique, opère sur les qubits pour les entasser les uns avec les autres, et effectue des mesures pour illustrer les effets de la superposition et de l'enchevêtrement. Vous créez votre Q# programme fragment par pièce pour introduire des états qubit, des opérations quantiques et des mesures.

Avant de commencer, passez en revue les concepts d'informatique quantique suivants :

- Les bits classiques contiennent une valeur binaire unique telle qu'un 0 ou 1, mais les qubits peuvent être dans une superposition des deux états, 0 et 1. Chaque état qubit possible est décrit par un ensemble d'amplitudes de probabilité.
- Lorsque vous mesurez l'état d'un qubit, vous obtenez toujours 0 ou 1. La probabilité de chaque résultat est déterminée par les amplitudes de probabilité qui définissent l'état de superposition lorsque vous effectuez une mesure.
- Plusieurs qubits peuvent être enchevêtrés de sorte que vous ne pouvez pas les décrire indépendamment les uns des autres. Lorsque vous mesurez un qubit dans une paire enchevêtrée, vous obtenez également des informations sur l'autre qubit sans la mesurer.

Dans ce tutoriel, vous allez apprendre à :

- ✓ Créez des opérations Q# pour initialiser des états de qubit.
- ✓ Placez un qubit dans un état de superposition.
- ✓ Intriquer une paire de qubits.
- ✓ Mesurez un qubit et affichez les résultats.

## Prérequis

Pour développer et exécuter l'exemple de code dans votre environnement de développement local, installez les outils suivants :

- La dernière version de [Visual Studio Code \(VS Code\)](#) ou ouvrez [VS Code pour le web](#).
- Dernière version de [l'extension QDK \(Microsoft Quantum Development Kit\)](#). Pour plus d'informations sur [l'installation](#), consultez [Configurer le QDK](#).

# Créer un nouveau Q# fichier

1. Dans VS Code, ouvrez le menu **Fichier** et choisissez **Nouveau fichier texte** pour créer un fichier.
2. Enregistrez le fichier sous le nom `CreateBellStates.qs`. Ce fichier est l'endroit où vous écrivez le Q# code de votre programme.

## Initialiser un qubit à un état connu

La première étape consiste à définir une Q# opération qui initialise un qubit à un état classique souhaité, soit 0 ou 1. Cette opération mesure un qubit dans un état quantique général, ce qui retourne une valeur de type Q# `Result`, soit `Zero` ou `One`. Si le résultat de la mesure est différent de l'état souhaité, l'opération inverse l'état pour que l'opération donne l'état souhaité 100% du temps.

Ouvrez `CreateBellStates.qs` et copiez le code suivant :

Q#

```
operation SetQubitState(desired : Result, target : Qubit) : Unit {
 if desired != M(target) {
 X(target);
 }
}
```

L'exemple de code introduit deux opérations standard Q# et `M X`, qui transforment l'état d'un qubit.

Voici une description détaillée du fonctionnement de l'opération `SetQubitState` :

1. Prend deux paramètres : un `Result` paramètre de type nommé `desired` qui représente l'état souhaité pour que le qubit soit dans (`Zero` ou `One`) et un `Qubit` paramètre de type.
2. Effectue une opération de mesure, `M` qui mesure l'état du qubit (`Zero` ou `One`) et compare le résultat à la valeur que vous passez `desired`.
3. Si le résultat de la mesure ne correspond pas à la valeur pour `desired`, alors une opération `X` est appliquée au qubit. Cette opération retourne l'état du qubit afin que les probabilités de mesure pour `Zero` et `One` soient inversées.

## Écrire une opération de test pour tester l'état Bell

Pour appeler l'opération `SetQubitState` dans votre Q# programme, créez une autre opération nommée `Main`. Cette opération alloue deux qubits, des appels `SetQubitState` pour définir le premier qubit à un état connu, puis mesure les qubits pour voir les résultats.

Ajoutez l'opération suivante à votre fichier `CreateBellStates.qs` après l'opération `SetQubitState` :

Q#

```
operation Main() : (Int, Int, Int, Int) {
 mutable numOnesQ1 = 0;
 mutable numOnesQ2 = 0;
 let count = 1000;
 let initial = One;

 // allocate the qubits
 use (q1, q2) = (Qubit(), Qubit());
 for test in 1..count {
 SetQubitState(initial, q1);
 SetQubitState(Zero, q2);

 // measure each qubit
 let resultQ1 = M(q1);
 let resultQ2 = M(q2);

 // Count the number of 'Ones' returned:
 if resultQ1 == One {
 numOnesQ1 += 1;
 }
 if resultQ2 == One {
 numOnesQ2 += 1;
 }
 }

 // reset the qubits
 SetQubitState(Zero, q1);
 SetQubitState(Zero, q2);

 // Display the times that |0> is returned, and times that |1> is returned
 Message($"Q1 - Zeros: {count - numOnesQ1}");
 Message($"Q1 - Ones: {numOnesQ1}");
 Message($"Q2 - Zeros: {count - numOnesQ2}");
 Message($"Q2 - Ones: {numOnesQ2}");
 return (count - numOnesQ1, numOnesQ1, count - numOnesQ2, numOnesQ2);
}
```

Dans le code, les variables `count` et `initial` sont définies à `1000` et `One` respectivement. Cela initialise le premier qubit à `One` et mesure chaque qubit 1 000 fois.

L'opération `Main` effectue les opérations suivantes :

1. Définit des variables pour le nombre de captures (`count`) et l'état initial du qubit (`One`).
2. Appelle l'instruction `use` pour initialiser deux qubits.
3. Répète l'expérience `count` fois.
4. Dans la boucle, appelle `SetQubitState` pour définir la valeur spécifiée `initial` sur le premier qubit, puis appelle à nouveau `SetQubitState` pour mettre le deuxième qubit dans l'état `Zero`.
5. Dans la boucle, applique l'opération `M` pour mesurer chaque qubit, puis stocke le nombre de mesures pour chaque qubit qui retourne `One`.
6. Une fois la boucle terminée, appelle à nouveau `SetQubitState` pour réinitialiser les qubits à un état connu (`Zero`). Vous devez réinitialiser les qubits que vous allouez avec l'instruction `use`.
7. Appelle la `Message` fonction pour imprimer vos résultats dans la console.

## Exécuter le code

Avant d'écrire du code pour la superposition et l'enchevêtrement, testez votre programme en cours pour voir l'initialisation et la mesure des qubits.

Pour exécuter le code en tant que programme autonome, le Q# compilateur doit savoir où démarrer le programme. Comme vous n'avez pas spécifié d'espace de noms, le compilateur reconnaît le point d'entrée par défaut comme opération `Main`. Pour plus d'informations, consultez [Projets et espaces de noms implicites](#).

Votre `CreateBellStates.qs` fichier ressemble maintenant à ceci :

Q#

```
operation SetQubitState(desired : Result, target : Qubit) : Unit {
 if desired != M(target) {
 X(target);
 }
}

operation Main() : (Int, Int, Int, Int) {
 mutable numOnesQ1 = 0;
 mutable numOnesQ2 = 0;
 let count = 1000;
 let initial = One;

 // allocate the qubits
 use (q1, q2) = (Qubit(), Qubit());
 for test in 1..count {
 SetQubitState(initial, q1);
 SetQubitState(Zero, q2);
 }
}
```

```

 // measure each qubit
 let resultQ1 = M(q1);
 let resultQ2 = M(q2);

 // Count the number of 'Ones' returned:
 if resultQ1 == One {
 numOnesQ1 += 1;
 }
 if resultQ2 == One {
 numOnesQ2 += 1;
 }
}

// reset the qubits
SetQubitState(Zero, q1);
SetQubitState(Zero, q2);

// Display the times that |0> is returned, and times that |1> is returned
Message($"Q1 - Zeros: {count - numOnesQ1}");
Message($"Q1 - Ones: {numOnesQ1}");
Message($"Q2 - Zeros: {count - numOnesQ2}");
Message($"Q2 - Ones: {numOnesQ2}");
return (count - numOnesQ1, numOnesQ1, count - numOnesQ2, numOnesQ2);
}

```

Pour exécuter le programme, choisissez la commande **Exécuter** à partir de l'objectif de code qui précède l'opération **Main** , ou **entrez Ctrl + F5**. Le programme exécute l'opération **Main** sur le simulateur par défaut.

Votre sortie s'affiche dans la console de débogage.

#### Sortie

```

Q1 - Zeros: 0
Q1 - Ones: 1000
Q2 - Zeros: 1000
Q2 - Ones: 0

```

Votre programme ne modifie pas encore les états qubit, donc la mesure du premier qubit retourne **One** toujours, et le second qubit retourne **Zero** toujours .

Si vous modifiez la valeur de **initial** à **Zero** et réexécutez le programme, alors le premier qubit retourne toujours **Zero** .

#### Sortie

```

Q1 - Zeros: 1000
Q1 - Ones: 0

```

Q2 - Zeros: 1000  
Q2 - Ones: 0

## Placer un qubit dans un état de superposition

Actuellement, les qubits de votre programme sont dans un état classique, soit 1 ou 0, tout comme les bits sur un ordinateur normal. Pour entangler les qubits, vous devez d'abord placer l'un des qubits dans un état de superposition égal. La mesure d'un qubit dans un état de superposition égal a 50% à retourner `Zero` et une chance de 50% de retourner `One`.

Pour placer un qubit dans un état de superposition, utilisez l'opération `Q#H`, ou Hadamard. L'opération `H` convertit un qubit qui se trouve dans un état pur `Zero` ou `One` dans un état de demi-chemin entre `Zero` et `One`.

Modifiez votre code dans l'opération `Main`. Réinitialisez la valeur initiale à `One` et insérez une ligne pour l'opération `H`:

Q#

```
for test in 1..count {
 use (q1, q2) = (Qubit(), Qubit());
 for test in 1..count {
 SetQubitState(initial, q1);
 SetQubitState(Zero, q2);

 H(q1); // Add the H operation after initialization and before measurement

 // measure each qubit
 let resultQ1 = M(q1);
 let resultQ2 = M(q2);
 ...
 }
}
```

Réexécutez votre programme. Étant donné que le premier qubit est dans une superposition égale lorsque vous le mesurez, vous obtenez près d'un résultat de 50/50 pour `Zero` et `One`. Par exemple, votre sortie ressemble à ceci :

Sortie

Q1 - Zeros: 523  
Q1 - Ones: 477  
Q2 - Zeros: 1000  
Q2 - Ones: 0

Chaque fois que vous exécutez le programme, les résultats du premier qubit varient légèrement, mais sont proches de 50% `One` et 50% `Zero`, tandis que les résultats du deuxième qubit sont

toujours toujours `Zero`.

Initialisez le premier qubit à `Zero` au lieu de `One` et réexécutez le programme. Vous obtenez des résultats similaires, car l'opération transforme à la fois un état pur `H` et un état pur `Zero` en un état de superposition équilibré.

## Intriquer deux qubits

Les qubits enchevêtrés sont corrélés de sorte qu'ils ne peuvent pas être décrits indépendamment les uns des autres. Lorsque vous mesurez l'état d'un qubit enchevêtré, vous connaissez également l'état de l'autre qubit sans le mesurer. Ce tutoriel utilise un exemple avec deux qubits enchevêtrés, mais vous pouvez également entangler trois qubits ou plus.

Pour créer un état enchevêtré, utilisez l'opération `Q# CNOT`, ou Controlled-NOT. Lorsque vous appliquez `CNOT` à deux qubits, un qubit est le qubit de contrôle et l'autre est le qubit cible. Si l'état du qubit de contrôle est `One`, l'opération `CNOT` retourne l'état du qubit cible. Sinon, `CNOT` ne fait rien aux qubits.

Ajoutez l'opération `CNOT` à votre programme immédiatement après l'opération `H`. Votre programme complet ressemble à ceci :

Q#

```
operation SetQubitState(desired : Result, target : Qubit) : Unit {
 if desired != M(target) {
 X(target);
 }
}

operation Main() : (Int, Int, Int, Int) {
 mutable numOnesQ1 = 0;
 mutable numOnesQ2 = 0;
 let count = 1000;
 let initial = Zero;

 // allocate the qubits
 use (q1, q2) = (Qubit(), Qubit());
 for test in 1..count {
 SetQubitState(initial, q1);
 SetQubitState(Zero, q2);

 H(q1);
 CNOT(q1, q2); // Add the CNOT operation after the H operation

 // measure each qubit
 let resultQ1 = M(q1);
 let resultQ2 = M(q2);
```



```

 // Count the number of 'Ones' returned:
 if resultQ1 == One {
 numOnesQ1 += 1;
 }
 if resultQ2 == One {
 numOnesQ2 += 1;
 }
}

// reset the qubits
SetQubitState(Zero, q1);
SetQubitState(Zero, q2);

// Display the times that |0> is returned, and times that |1> is returned
Message($"Q1 - Zeros: {count - numOnesQ1}");
Message($"Q1 - Ones: {numOnesQ1}");
Message($"Q2 - Zeros: {count - numOnesQ2}");
Message($"Q2 - Ones: {numOnesQ2}");
return (count - numOnesQ1, numOnesQ1, count - numOnesQ2, numOnesQ2);
}

```

Exécutez le programme et affichez la sortie. Les résultats varient très légèrement chaque fois que vous exécutez le programme.

#### Sortie

```

Q1 - Zeros: 502
Q1 - Ones: 498
Q2 - Zeros: 502
Q2 - Ones: 498

```

Les statistiques pour le premier qubit montrent toujours environ 50% chance de mesurer les deux **One** et **Zero**, mais les résultats de mesure pour le deuxième qubit ne sont pas toujours maintenant **Zero**. Chaque qubit a le même nombre de résultats **Zero** et **One**. Le résultat de mesure du deuxième qubit est toujours le même que le résultat du premier qubit, car les deux qubits sont enchevêtrés. Si le premier qubit est mesuré comme étant **Zero**, alors le qubit enchevêtré doit être **Zero** également. Si le premier qubit est mesuré comme étant **One**, alors le qubit enchevêtré doit être **One** également.

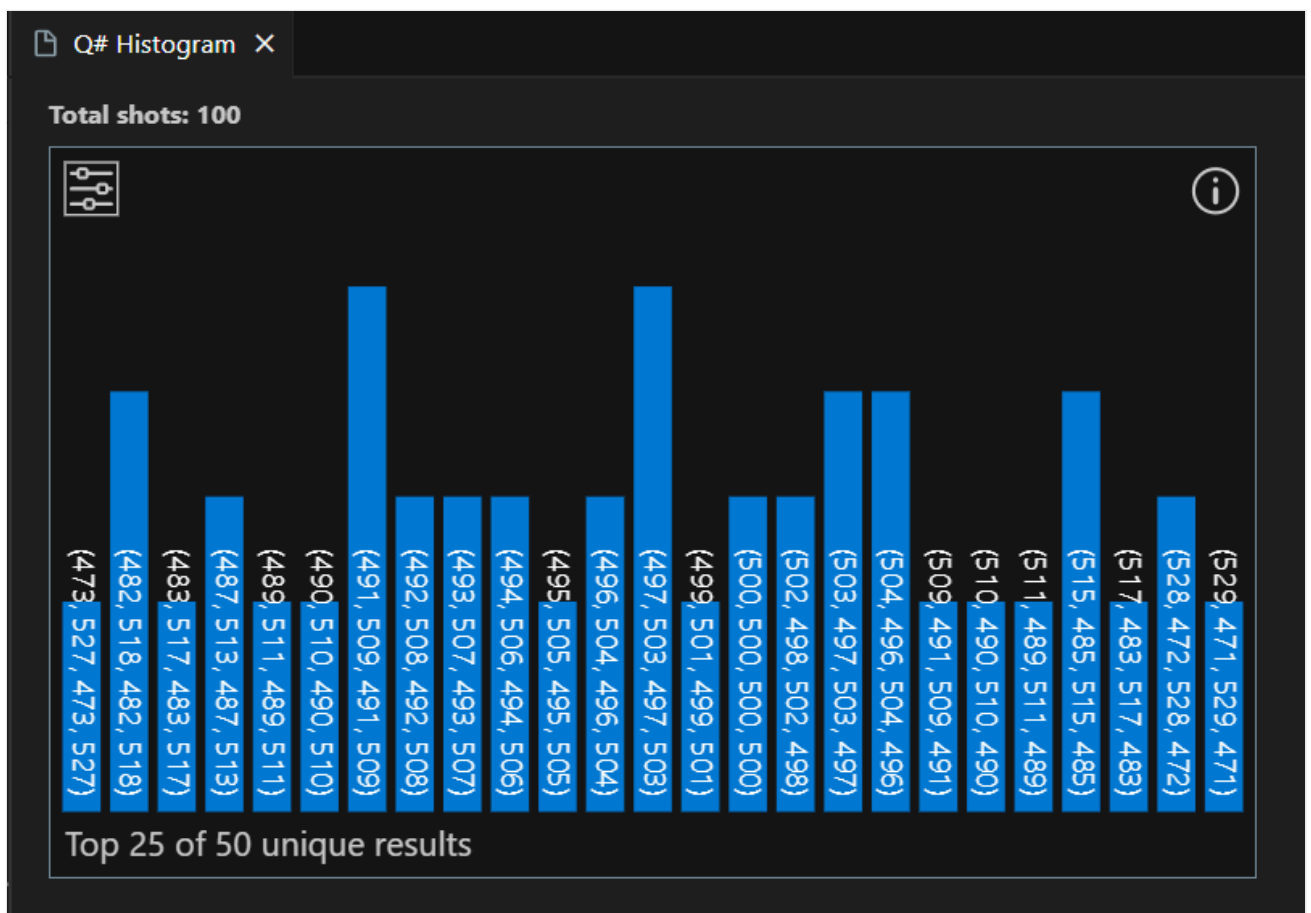
## Tracer l'histogramme de fréquence

Pour tracer un histogramme de fréquence qui affiche la distribution des résultats lorsque vous exécutez votre programme plusieurs fois, effectuez les étapes suivantes :

1. Ouvrez votre `CreateBellStates.qs` fichier dans VS Code.

2. Ouvrez le menu **Affichage** et choisissez **PaLETTE de commandes**.
3. Entrez **histogramme** pour afficher l'option **QDK : Exécuter le fichier et afficher l'histogramme**. Vous pouvez également choisir la commande **Histogramme** dans l'option de l'objectif de code qui précède l'opération **Main**. Ensuite, saisissez un nombre de prises de vue (par exemple, 100). L'histogramme Q# s'ouvre dans un nouvel onglet.

Chaque barre de l'histogramme correspond à un résultat possible lorsque le circuit d'entanglement s'exécute 1 000 fois. La hauteur d'une barre représente le nombre de fois où le résultat se produit. Par exemple, l'histogramme suivant montre une distribution avec 50 résultats uniques. Notez que pour chaque résultat, les résultats de mesure pour le premier et le deuxième qubit sont toujours identiques.

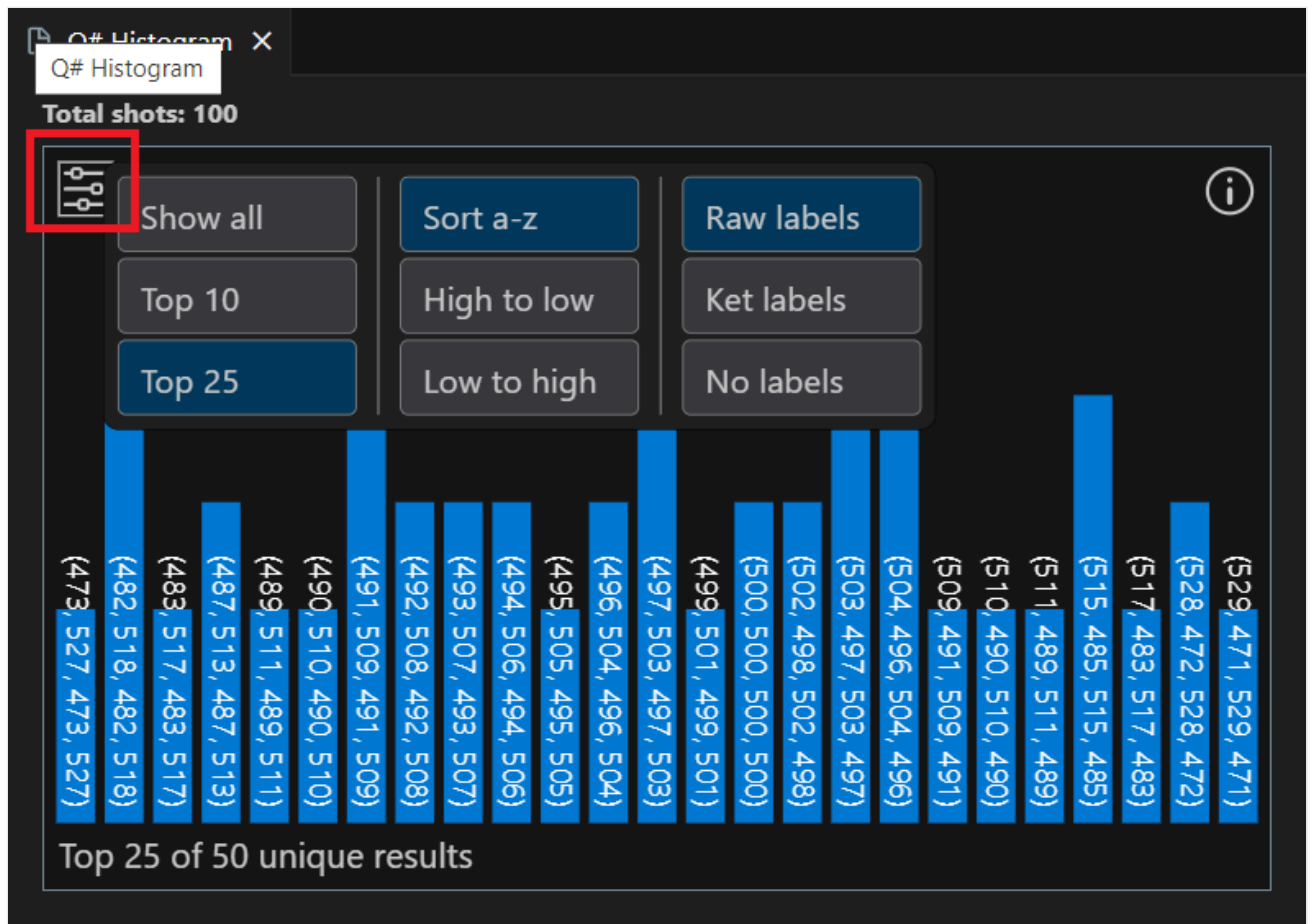


#### 💡 Conseil

Pour effectuer un zoom avant sur l'histogramme, utilisez la roulette de défilement de la souris ou un mouvement de trackpad. Pour faire défiler le graphique lorsque vous effectuez un zoom avant, maintenez la **touche Alt** enfoncée pendant le défilement.

4. Choisissez une barre pour afficher le pourcentage de tirs totaux qui ont produit ce résultat.

5. Choisissez l'icône des paramètres en haut à gauche pour afficher les options de visualisation.



6. Réexécutez le code, mais cette fois avec 1 000 coups. À mesure que le nombre de tirs augmente, la distribution des résultats approche d'une distribution normale.

## Contenu connexe

Explorez les autres tutoriels Q# :

- [L'algorithme](#) de recherche de Grover montre comment écrire un Q# programme qui utilise l'algorithme de recherche de Grover.
- [Quantum Fourier Transform](#) explore comment écrire un Q# programme qui traite directement des qubits spécifiques.
- Les [Katas Quantiques](#) sont des tutoriels auto-rythmés et des exercices de programmation qui enseignent les éléments de l'informatique quantique et Q# de la programmation à la fois.

# Tutoriel : Implémenter l'algorithme de recherche de Grover dans Q#

Dans ce tutoriel, vous implémentez l'algorithme Q# de Grover pour résoudre les problèmes basés sur la recherche. Pour une explication détaillée de la théorie sous-jacente de l'algorithme de Grover, consultez la [théorie de l'algorithme de Grover](#).

Dans ce tutoriel, vous allez :

- ✓ Définir l'algorithme de Grover pour un problème de recherche
- ✓ Implémenter l'algorithme de Grover dans Q#

## Prérequis

Pour développer et exécuter l'exemple de code dans Visual Studio Code (VS Code), vous devez installer les outils suivants :

- Dernière version de [VS Code](#) ou ouvrir [VS Code pour le web](#).
- La plus récente version de l'extension QDK. Pour plus d'informations sur [l'installation](#), consultez [Configurer le QDK](#).

## Définir le problème

L'algorithme de Grover est l'un des algorithmes les plus connus en calcul quantique. Le type de problème qu'il résout est souvent appelé « recherche dans une base de données », mais il est plus précis de le considérer en termes de problème de recherche.

Tout problème de recherche peut être formulé de façon mathématique à l'aide d'une fonction abstraite  $f(x)$  qui accepte des éléments de recherche  $x$ . Si l'élément  $x$  est une solution du problème de recherche, alors  $f(x) = 1$ . Si l'élément  $x$  n'est pas une solution, alors  $f(x) = 0$ . Le problème de recherche consiste à trouver tout élément  $x_0$  tel que  $f(x_0) = 1$ .

Ainsi, vous pouvez formuler n'importe quel problème de recherche comme suit : étant donné une fonction classique  $f(x) : \{0, 1\}^n \rightarrow \{0, 1\}$ , où  $n$  est la taille de bits de l'espace de recherche, recherchez une entrée  $x_0$  pour laquelle  $f(x_0) = 1$ .

Pour implémenter l'algorithme de Grover pour résoudre un problème de recherche, vous devez :

1. Transformer le problème sous la forme d'une **tâche de Grover**. Par exemple, supposons que vous souhaitiez trouver les facteurs d'un entier  $M$  à l'aide de l'algorithme de Grover. Vous pouvez transformer le problème de factorisation des entiers en une tâche de Grover en créant une fonction

$$f_M(x) = 1[r],$$

où  $1[r] = 1$  si  $r = 0$  et  $1[r] = 0$  si  $r \neq 0$  et  $r$  est le reste de  $M/x$ . De cette façon, les entiers  $x_i$  qui figurent dans  $f_M(x_i) = 1$  sont les facteurs de  $M$  et vous avez transformé le problème en une tâche de Grover.

2. Implémentez la fonction de la tâche de Grover en tant qu'oracle quantique. Pour implémenter l'algorithme de Grover, vous devez implémenter la fonction  $f(x)$  de votre tâche de Grover en tant qu'**oracle quantique**.
3. Utilisez l'algorithme de Grover avec votre oracle pour résoudre la tâche. Une fois que vous avez un oracle quantique, vous pouvez le connecter à l'implémentation de votre algorithme de Grover pour résoudre le problème et interpréter la sortie.

## Algorithme de Grover

Supposons que nous avons  $N = 2^n$  éléments éligibles pour le problème de recherche et que nous les indexons en affectant à chaque élément un entier compris entre 0 et  $N - 1$ . Les étapes de l'algorithme sont les suivantes :

1. Commencer avec un registre de  $n$  qubits initialisés à l'état  $|0\rangle$ .
2. Préparer le registre dans une superposition uniforme en appliquant  $H$  à chaque qubit dans le registre :

$$|\psi\rangle = \frac{1}{N^{1/2}} \sum_{x=0}^{N-1} |x\rangle$$

3. Appliquez les opérations suivantes au registre  $N_{\text{optimal}}$  fois :
  - a. L'oracle de phase  $O_f$  qui applique un décalage de phase conditionnel de  $-1$  pour les éléments de la solution.
  - b. Appliquer  $H$  à chaque qubit dans le registre.
  - c. Appliquer  $-O_0$ , un décalage de phase conditionnel de  $-1$  à chaque état de base de calcul, sauf  $|0\rangle$ .
  - d. Appliquer  $H$  à chaque qubit dans le registre.
4. Mesurer le registre pour obtenir l'index d'un élément qui est une solution avec une très grande probabilité.

5. Vérifiez l'élément pour voir s'il s'agit d'une solution valide. Si ce n'est pas le cas, recommencer.

## Écrire le code de l'algorithme de Grover dans Q#

Cette section explique comment implémenter l'algorithme dans Q#. Il existe quelques points à prendre en compte lors de l'implémentation de l'algorithme de Grover. Vous devez définir l'état marqué, le mode de réflexion et le nombre d'itérations pour lesquelles exécuter l'algorithme. Vous devez également définir l'oracle qui implémente la fonction de la tâche de Grover.

### Définir l'état marqué

Tout d'abord, vous définissez l'entrée que vous essayez de trouver dans la recherche. Pour ce faire, écrivez une opération qui applique les étapes **b**, **c** et **d** à partir de l'algorithme de Grover.

Ensemble, ces étapes sont également appelées **opérateur** de diffusion de Grover  $-H^{\otimes n}O_0H^{\otimes n}$ .

Q#

```
operation ReflectAboutMarked(inputQubits : Qubit[]) : Unit {
 Message("Reflecting about marked state...");
 use outputQubit = Qubit();
 within {
 // We initialize the outputQubit to $(|0\rangle - |1\rangle) / \sqrt{2}$, so that
 // toggling it results in a (-1) phase.
 X(outputQubit);
 H(outputQubit);
 // Flip the outputQubit for marked states.
 // Here, we get the state with alternating 0s and 1s by using the X
 // operation on every other qubit.
 for q in inputQubits[...2...] {
 X(q);
 }
 } apply {
 Controlled X(inputQubits, outputQubit);
 }
}
```

L'opération `ReflectAboutMarked` reflète l'état de base marqué par une alternance de zéros et de uns. Il le fait en appliquant l'opérateur de diffusion de Grover aux qubits d'entrée. L'opération utilise un qubit auxiliaire, `outputQubit`, qui est initialisé dans l'état  $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$  en appliquant les portes  $X$  et  $H$ . L'opération applique ensuite la porte  $X$  à un qubit sur deux dans le registre, retournant ainsi l'état de chaque qubit concerné. Enfin, il applique la porte  $X$

contrôlée au qubit auxiliaire et aux qubits d'entrée. Cette opération retourne le qubit auxiliaire si et uniquement si tous les qubits d'entrée sont dans l'état  $|1\rangle$ , qui est l'état marqué.

## Définir le nombre d'itérations optimales

La recherche de Grover a un nombre optimal d'itérations qui offre la probabilité la plus élevée de mesurer une sortie valide. Si le problème comporte  $N = 2^n$  éléments éligibles possibles, dont  $M$  d'entre eux constituent des solutions au problème, le nombre optimal d'itérations est :

$$N_{\text{optimal}} \approx \frac{\pi}{4} \sqrt{\frac{N}{M}}$$

Continuer à itérer au-delà du nombre optimal d'itérations commence à réduire cette probabilité jusqu'à ce que vous atteignez une probabilité de réussite presque nulle sur l'itération  $2N_{\text{optimal}}$ . Après cela, la probabilité augmente à nouveau jusqu'à  $3N_{\text{optimal}}$ , et ainsi de suite.

Dans les applications pratiques, on ne connaît généralement pas le nombre de solutions au problème avant de le résoudre. Une stratégie efficace pour gérer ce problème consiste à « deviner » le nombre de solutions  $M$  en augmentant progressivement l'estimation selon des puissances de deux (p. ex. 1, 2, 4, 8, 16, ...,  $2^n$ ). L'une de ces estimations sera suffisamment proche pour que l'algorithme trouve encore la solution avec un nombre moyen d'itérations proche de  $\sqrt{\frac{N}{M}}$ .

La fonction suivante Q# calcule le nombre optimal d'itérations pour un nombre donné de qubits dans un registre.

Q#

```
function CalculateOptimalIterations(nQubits : Int) : Int {
 if nQubits > 63 {
 fail "This sample supports at most 63 qubits.";
 }
 let nItems = 1 <<< nQubits; // 2^nQubits
 let angle = ArcSin(1. / Sqrt(IntAsDouble(nItems)));
 let iterations = Round(0.25 * PI() / angle - 0.5);
 return iterations;
}
```

La `CalculateOptimalIterations` fonction utilise la formule indiquée précédemment pour calculer le nombre d'itérations, puis l'arrondit à l'entier le plus proche.

## Définir l'opération de Grover

L'opération Q# pour l'algorithme de recherche de Grover a trois entrées :

- Nombre de qubits, `nQubits : Int` dans le registre qubit. Cet enregistrement encode la solution provisoire au problème de recherche et est mesuré après l'opération.
- Nombre d'itérations optimales, `iterations : Int`.
- Opération, `phaseOracle : Qubit[] => Unit) : Result[]` qui représente l'oracle de phase pour la tâche de Grover. Cette opération applique une transformation unitaire sur un registre qubit générique.

Q#

```
operation GroverSearch(nQubits : Int, iterations : Int, phaseOracle : Qubit[] =>
Unit) : Result[] {

 use qubits = Qubit[nQubits];
 PrepareUniform(qubits);

 for _ in 1..iterations {
 phaseOracle(qubits);
 ReflectAboutUniform(qubits);
 }

 // Measure and return the answer.
 return MResetEachZ(qubits);
}
```

L'opération `GroverSearch` initialise un registre de qubits  $n$  dans l'état  $|0\rangle$ , prépare le registre dans une superposition uniforme, puis applique l'algorithme de Grover pour le nombre spécifié d'itérations. La recherche elle-même consiste à appliquer à plusieurs reprises des réflexions sur l'état marqué et l'état de départ, que vous pouvez exprimer dans Q# comme une boucle for. Enfin, il mesure le registre et retourne le résultat.

Le code utilise trois opérations d'assistance : `PrepareUniform`, `ReflectAboutUniform` et `ReflectAboutAllOnes`.

Étant donné un registre dans l'état tout-zéros, l'opération `PrepareUniform` prépare une superposition égale sur tous les états de base.

Q#

```
operation PrepareUniform(inputQubits : Qubit[]) : Unit is Adj + Ctl {
 for q in inputQubits {
 H(q);
 }
}
```



L'opération `ReflectAboutAllOnes` reflète l'état de tous les éléments.

Q#

```
operation ReflectAboutAllOnes(inputQubits : Qubit[]) : Unit {
 Controlled Z(Most(inputQubits), Tail(inputQubits));
}
```

L'opération `ReflectAboutUniform` reflète l'état de superposition uniforme. Tout d'abord, elle transforme la superposition uniforme en zéro. Ensuite, il transforme l'état tout-zéro en tout-uns. Enfin, il reflète l'état de tous les éléments. L'opération est appelée `ReflectAboutUniform`, car elle peut être interprétée géométriquement comme une réflexion dans l'espace vectoriel au sujet de l'état de superposition uniforme.

Q#

```
operation ReflectAboutUniform(inputQubits : Qubit[]) : Unit {
 within {
 Adjoint PrepareUniform(inputQubits);
 // Transform the all-zero state to all-ones
 for q in inputQubits {
 X(q);
 }
 } apply {
 ReflectAboutAllOnes(inputQubits);
 }
}
```

## Exécuter le code final

Vous disposez à présent de tous les ingrédients requis pour implémenter une instance particulière de l'algorithme de recherche de Grover et résoudre le problème de factorisation. Pour terminer, l'opération `Main` configure le problème en spécifiant le nombre de qubits et le nombre d'itérations

Q#

```
operation Main() : Result[] {
 let nQubits = 5;
 let iterations = CalculateOptimalIterations(nQubits);
 Message($"Number of iterations: {iterations}");

 // Use Grover's algorithm to find a particular marked state.
 let results = GroverSearch(nQubits, iterations, ReflectAboutMarked);
}
```

```
 return results;
}
```

## Exécuter le programme

1. Dans VS Code, ouvrez le menu **Fichier** et choisissez **Nouveau fichier texte** pour créer un fichier.
2. Enregistrez le fichier sous le nom `GroversAlgorithm.qs`. Ce fichier contient le code Q# de votre programme.
3. Copiez le code suivant dans le fichier `GroversAlgorithm.qs`.

Q#

```
import Std.Convert.*;
import Std.Math.*;
import Std.Arrays.*;
import Std.Measurement.*;
import Std.Diagnostics.*;

operation Main() : Result[] {
 let nQubits = 5;
 let iterations = CalculateOptimalIterations(nQubits);
 Message($"Number of iterations: {iterations}");

 // Use Grover's algorithm to find a particular marked state.
 let results = GroverSearch(nQubits, iterations, ReflectAboutMarked);
 return results;
}

operation GroverSearch(
 nQubits : Int,
 iterations : Int,
 phaseOracle : Qubit[] => Unit) : Result[] {

 use qubits = Qubit[nQubits];

 PrepareUniform(qubits);

 for _ in 1..iterations {
 phaseOracle(qubits);
 ReflectAboutUniform(qubits);
 }

 // Measure and return the answer.
 return MResetEachZ(qubits);
}
```

```

function CalculateOptimalIterations(nQubits : Int) : Int {
 if nQubits > 63 {
 fail "This sample supports at most 63 qubits.";
 }
 let nItems = 1 <<< nQubits; // 2^nQubits
 let angle = ArcSin(1. / Sqrt(IntAsDouble(nItems)));
 let iterations = Round(0.25 * PI() / angle - 0.5);
 return iterations;
}

operation ReflectAboutMarked(inputQubits : Qubit[]) : Unit {
 Message("Reflecting about marked state...");
 use outputQubit = Qubit();
 within {
 // We initialize the outputQubit to $(|0\rangle - |1\rangle) / \sqrt{2}$, so that
 // toggling it results in a (-1) phase.
 X(outputQubit);
 H(outputQubit);
 // Flip the outputQubit for marked states.
 // Here, we get the state with alternating 0s and 1s by using the X
 // operation on every other qubit.
 for q in inputQubits[...2...] {
 X(q);
 }
 } apply {
 Controlled X(inputQubits, outputQubit);
 }
}

operation PrepareUniform(inputQubits : Qubit[]) : Unit is Adj + Ctl {
 for q in inputQubits {
 H(q);
 }
}

operation ReflectAboutAllOnes(inputQubits : Qubit[]) : Unit {
 Controlled Z(Most(inputQubits), Tail(inputQubits));
}

operation ReflectAboutUniform(inputQubits : Qubit[]) : Unit {
 within {
 // Transform the uniform superposition to all-zero.
 Adjoint PrepareUniform(inputQubits);
 // Transform the all-zero state to all-ones
 for q in inputQubits {
 X(q);
 }
 } apply {
 // Now that we've transformed the uniform superposition to the
 // all-ones state, reflect about the all-ones state, then let the
 // within/apply block transform us back.
 ReflectAboutAllOnes(inputQubits);
 }
}

```

4. Pour exécuter votre programme, choisissez **Exécuter** dans le menu de l'opération **Main** , ou appuyez sur **Ctrl + F5**. Par défaut, le compilateur exécute l'opération ou la **Main** fonction sur le simulateur par défaut.
5. Votre sortie s'affiche dans la console de débogage dans le terminal.

#### ⓘ Remarque

Si le profil QIR target n'est pas défini sur **Non restreint**, vous obtenez une erreur lorsque vous exécutez le programme. Pour ce programme, le compilateur définit automatiquement le target profil sur **Non restreint** , sauf si vous définissez le profil vous-même.

## Contenu connexe

Explorez les autres tutoriels Q# :

- [L'intrication](#) quantique montre comment écrire un Q# programme qui manipule et mesure des qubits et illustre les effets de la superposition et de l'intrication.
- [Le générateur quantique de nombres aléatoires](#) montre comment écrire un Q# programme qui génère des nombres aléatoires à partir de qubits en superposition.
- [Quantum Fourier Transform](#) explore comment écrire un Q# programme qui traite directement des qubits spécifiques.
- Les [katas quantiques](#) [↗](#) sont des tutoriels auto-rythmés et des exercices de programmation visant à enseigner les éléments de l'informatique quantique et de la programmation simultanément.

Last updated on 19/05/2026

# Tutoriel : Implémenter la transformation Quantum Fourier dans Q#

Ce tutoriel montre comment écrire et simuler un programme quantique de base qui fonctionne sur des qubits individuels.

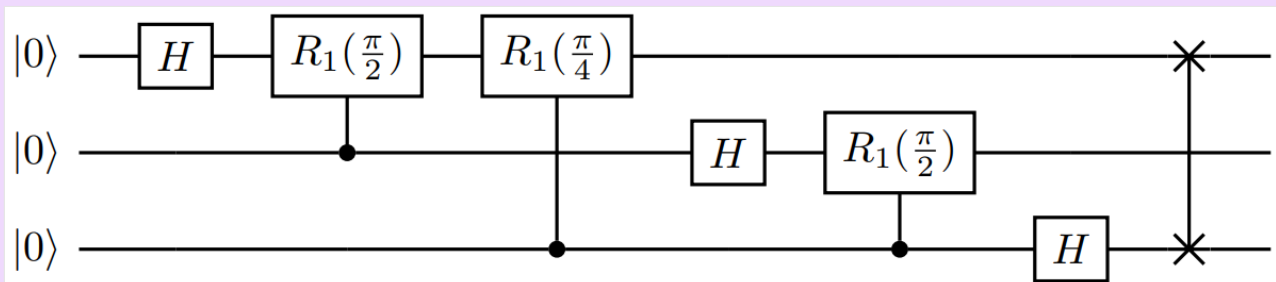
Même si Q# a principalement été créé comme un langage de programmation élémentaire destiné à des programmes quantiques à grande échelle, il peut aussi s'utiliser pour explorer la programmation quantique de manière plus approfondie, à savoir en traitant directement des qubits spécifiques. Plus précisément, ce tutoriel étudie plus en détail la [transformation de Fourier quantique](#) (QFT), une sous-routine qui fait partie intégrante de nombreux algorithmes quantiques plus importants.

Dans ce tutoriel, vous allez apprendre à :

- ✓ Définir des opérations quantiques dans Q#.
- ✓ Écrivez le circuit de transformation de Fourier quantique.
- ✓ Simuler une opération quantique depuis l'allocation du qubit jusqu'à l'obtention de la mesure.
- ✓ Observez comment la fonction d'onde simulée du système quantique évolue tout au long de l'opération.

## ⚠ Remarque

Cette vue plus approfondie du traitement des informations quantiques est souvent décrite par le terme de circuits quantiques, qui représentent l'application séquentielle de portes, ou *opérations*, à des qubits spécifiques d'un système. Ainsi, les opérations mono- et multi-qubit que vous appliquez séquentiellement peuvent être facilement représentées dans des schémas de circuit. Par exemple, la transformation fourier quantique à trois qubits complète utilisée dans ce didacticiel présente la représentation suivante en tant que circuit :



## 💡 Conseil

Si vous souhaitez accélérer votre parcours d'informatique quantique, consultez [Code avec Microsoft Quantum](#), une fonctionnalité unique du [site web Microsoft Quantum](#). Ici, vous pouvez exécuter des exemples intégrés Q# ou vos propres Q# programmes, générer du nouveau Q# code à partir de vos invites, ouvrir et exécuter votre code dans [VS Code pour le web](#) en un clic et poser des questions à Copilot sur l'informatique quantique.

## Prérequis

- La dernière version de [Visual Studio Code \(VS Code\)](#) ou ouvrir [VS Code pour le web](#).
- La plus récente version de l'extension QDK. Pour plus d'informations sur l'installation, consultez [Configurer l'extension QDK](#).
- Pour utiliser jupyter Notebooks, installez les extensions [Python](#) et [Jupyter](#) dans VS Code.
- Le dernier `qdk` paquet Python avec l'extra `jupyter`. Pour les installer, ouvrez un terminal et exécutez la commande suivante :

Bash

```
pip install --upgrade "qdk[jupyter]"
```

## Créer un nouveau fichier Q#

1. Dans VS Code, ouvrez le menu **Fichier** et choisissez **Nouveau fichier texte**.
2. Enregistrez le fichier en tant que **QFTcircuit.qs**. Ce fichier contient le code Q# de votre programme.
3. Ouvrez **QFTcircuit.qs**.

## Écrire un circuit QFT dans Q#

La première partie de ce tutoriel consiste à définir l'opération Q# `Main`, qui exécute la transformation de Fourier quantique sur trois qubits. La fonction `DumpMachine` est utilisée pour observer la façon dont la fonction d'onde simulée du système à trois qubits évolue au fil de l'opération. Dans la deuxième partie du tutoriel, vous ajoutez des fonctionnalités de mesure et comparez les états de pré-et post-mesure des qubits.

Vous construisez l'opération étape par étape. Copiez et collez le code dans les sections suivantes dans le fichier **QFTcircuit.qs**.

Vous pouvez afficher le code complet pour cette section comme référence.

## Importer des bibliothèques requises Q#

Dans votre fichier `Q#`, importez les espaces de noms `Std.\*` appropriés.

Q#

```
import Std.Diagnostics.*;
import Std.Math.*;
import Std.Arrays.*;

// operations go here
```

## Définir des opérations avec des arguments et des retours

Ensuite, définissez l'opération `Main` :

Q#

```
operation Main() : Unit {
 // do stuff
}
```

L'opération `Main` ne prend jamais d'arguments, et pour le moment retourne un `Unit` objet, qui est analogue au retour `void` en C# ou à un tuple vide, `Tuple[()]` en Python. Plus tard, vous modifiez l'opération pour retourner un tableau de résultats de mesure.

## Allouer des qubits

Dans l'opération Q# , allouez un registre de trois qubits avec le `use` mot clé. Avec `use`, les qubits sont automatiquement alloués dans l'état  $|0\rangle$ .

Q#

```
use qs = Qubit[3]; // allocate three qubits

Message("Initial state |000>:");
DumpMachine();
```

Comme dans les calculs quantiques réels, Q# ne vous permet pas d'accéder directement aux états qubit. Toutefois, l'opération de `DumpMachine` imprime l'état actuel de la machine target,

afin qu'elle puisse fournir des informations précieuses pour le débogage et l'apprentissage lorsqu'elle est utilisée avec le simulateur d'état complet.

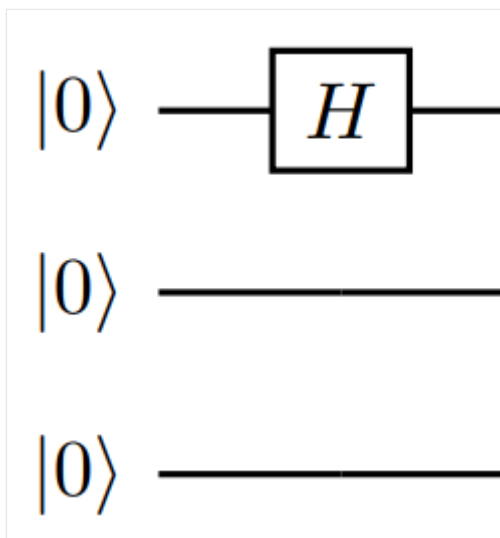
## Appliquer des opérations à qubit unique et contrôlées

Ensuite, appliquez les opérations qui composent l'opération `Main` elle-même. Q# contient déjà un grand nombre de ces opérations quantiques, ainsi que d'autres opérations quantiques de base, dans l'espace `Std.Intrinsic` de noms.

### ❗ Remarque

`Std.Intrinsic` n'a pas été importé dans l'extrait de code précédent avec les autres espaces de noms, car il est chargé automatiquement par le compilateur pour tous les Q# programmes.

L'opération `H` (de Hadamard) est la première opération appliquée au premier qubit :



Pour appliquer une opération à un qubit spécifique à partir d'un registre (par exemple, un seul `qubit` issu d'un tableau `qubit[]`), utilisez la notation d'index standard. Donc, l'application de l'opération `H` au premier qubit du registre `qs` s'écrit ainsi :

Q#

```
H(qs[0]);
```

En plus d'appliquer l'opération `H` à des qubits individuels, le circuit QFT consiste principalement en des rotations `R1` contrôlées. Une `R1( $\theta$ , <qubit>)` opération en général laisse le composant  $|0\rangle$  du qubit inchangé tout en appliquant une rotation de  $e^{i\theta}$  au composant  $|1\rangle$ .

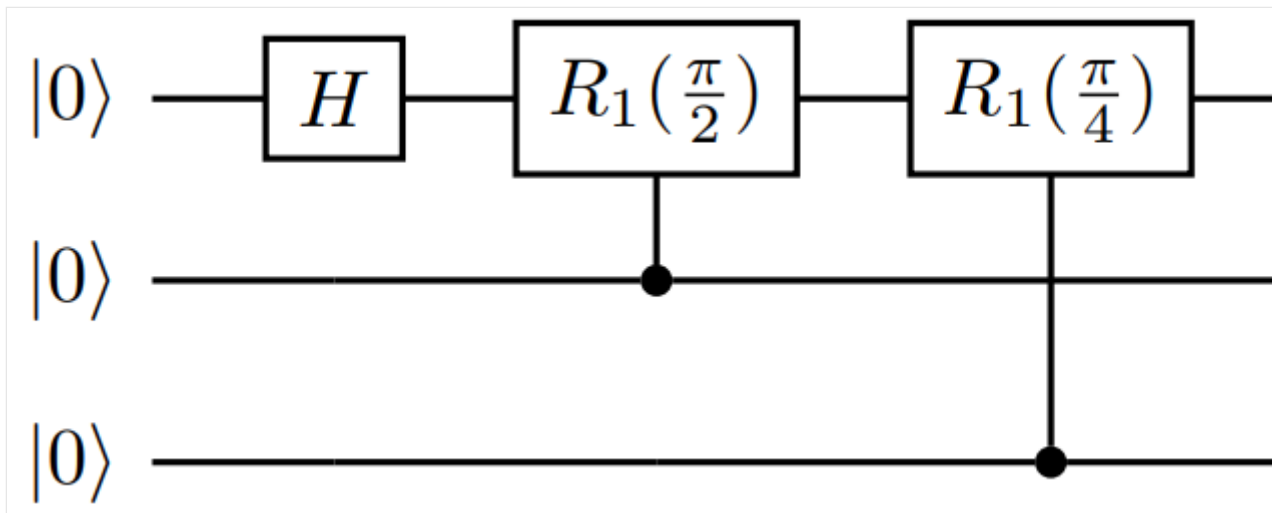


Q# permet de conditionner facilement l'exécution d'une opération sur un ou plusieurs qubits de contrôle. En général, il suffit de faire précéder l'appel par `Controlled` pour que les arguments d'opération changent comme suit :

`Op(<normal args>) → Controlled Op([<control qubits>], (<normal args>))`

Notez que l'argument de qubit de contrôle doit être un tableau, même s'il référence un seul qubit.

Les opérations contrôlées dans le QFT sont les `R1` opérations qui agissent sur le premier qubit (et contrôlées par les deuxième et troisième qubits) :



Dans votre fichier Q#, appelez ces opérations avec ces instructions :

Q#

```
Controlled R1([qs[1]], (PI())/2.0, qs[0]));
Controlled R1([qs[2]], (PI())/4.0, qs[0]));
```

La fonction `PI()` est utilisée pour définir les rotations en termes de pi radians.

## Appliquer l'opération SWAP

Après avoir appliqué les opérations pertinentes `H` et les rotations contrôlées aux deuxième et troisième qubits, le circuit ressemble à ceci :

Q#

```
//second qubit:
H(qs[1]);
Controlled R1([qs[2]], (PI())/2.0, qs[1]));
```

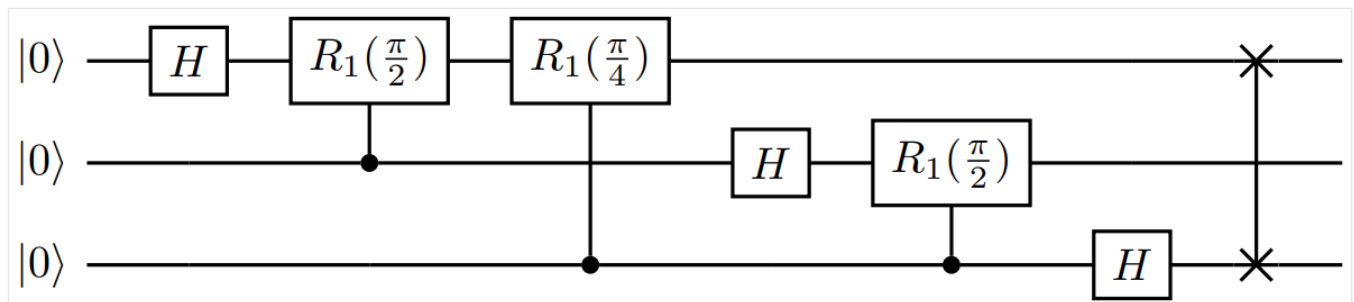
```
//third qubit:
H(qs[2]);
```

Enfin, vous appliquez une **SWAP** opération aux premier et troisième qubits pour terminer le circuit. Cette opération est nécessaire, car la transformation de Fourier quantique génère les qubits dans l'ordre inverse, de sorte que les permutations permettent une intégration transparente de la sous-routine dans des algorithmes plus volumineux.

Q#

```
SWAP(qs[2], qs[0]);
```

L'opération Q# inclut désormais les opérations au niveau qubit de la transformation de Fourier quantique :



## Libérer des qubits

La dernière étape consiste à appeler **DumpMachine()** une nouvelle fois pour voir l'état après l'opération, puis à libérer les qubits. Les qubits étaient dans l'état  $|0\rangle$  lorsque vous les avez alloués. Vous devez maintenant les rétablir à leur état initial en utilisant l'opération **ResetAll**.

Exiger que tous les qubits soient explicitement réinitialisés à  $|0\rangle$  est une fonctionnalité de base de Q#, car il permet à d'autres opérations de connaître leur état précisément quand ils commencent à utiliser ces mêmes qubits (une ressource rare). De plus, leur réinitialisation garantit qu'ils ne sont pas intriqués avec aucun autre qubit du système. Si la réinitialisation n'est pas effectuée à la fin d'un bloc d'allocation **use**, une erreur d'exécution peut être générée.

Ajoutez les lignes suivantes à votre fichier Q# :

Q#

```
Message("After:");
DumpMachine();

ResetAll(qs); // deallocate qubits
```

# Opération QFT complète

Le Q# programme est terminé. Votre fichier **QFTcircuit.qs** doit maintenant ressembler à ceci :

Q#

```
import Std.Diagnostics.*;
import Std.Math.*;
import Std.Arrays.*;

operation Main() : Unit {

 use qs = Qubit[3]; // allocate three qubits

 Message("Initial state |000>:");
 DumpMachine();

 //QFT:
 //first qubit:
 H(qs[0]);
 Controlled R1([qs[1]], (PI()/2.0, qs[0]));
 Controlled R1([qs[2]], (PI()/4.0, qs[0]));

 //second qubit:
 H(qs[1]);
 Controlled R1([qs[2]], (PI()/2.0, qs[1]));

 //third qubit:
 H(qs[2]);

 SWAP(qs[2], qs[0]);

 Message("After:");
 DumpMachine();

 ResetAll(qs); // deallocate qubits

}
```

## Exécuter le circuit QFT

Pour l'instant, l'opération `Main` ne retourne aucune valeur : l'opération retourne `Unit` la valeur. Plus tard, vous modifiez l'opération pour retourner un tableau de résultats de mesure (`Result[]`).

Q# programme autonome

1. Pour exécuter votre programme, choisissez **Exécuter Q# un fichier** dans le menu qui précède `Main`, ou appuyez sur **Ctrl + F5**. Le programme exécute l'opération `Main` sur le simulateur par défaut.
2. Les sorties `Message` et `DumpMachine` s'affichent dans la console de débogage.

Si vous êtes curieux de savoir comment d'autres états d'entrée sont affectés, essayez d'appliquer d'autres opérations qubit avant la transformation.

## Ajouter des mesures au circuit QFT

L'affichage de la fonction `DumpMachine` a montré les résultats de l'opération, mais malheureusement, une pierre angulaire de la mécanique quantique indique qu'un système quantique réel ne peut pas avoir une telle fonction `DumpMachine`. Au lieu de cela, les informations sont extraites par le biais de mesures, ce qui, en général, ne permet pas de fournir des informations sur l'état quantique complet, mais peut également altérer radicalement le système en lui-même.

Il existe de nombreux types de mesures quantiques, mais l'exemple étudié ici se concentre sur les plus basiques, à savoir les mesures projectives sur des qubits uniques. En cas de mesure dans une base donnée (par exemple, la base de calcul  $|0\rangle, |1\rangle$ ), l'état du qubit est projeté sur tout état de base qui a été mesuré, détruisant ainsi toute superposition entre les deux.

## Modifier l'opération QFT

Pour implémenter des mesures dans un programme Q#, utilisez l'opération `M`, qui retourne un type `Result`.

Tout d'abord, modifiez l'opération `Main` pour retourner un tableau de résultats de mesure, `Result[]`, au lieu de `Unit`.

Q#

```
operation Main() : Result[] {
```

## Définir et initialiser un tableau `Result[]`

Avant d'allouer des qubits, déclarez et liez un tableau à trois éléments (un `Result` pour chaque qubit) :

Q#

```
mutable resultArray = [Zero, size = 3];
```

La `mutable` préfacation `resultArray` du mot clé permet de modifier la variable ultérieurement dans le code, par exemple lors de l'ajout de résultats de mesure.

## Effectuer des mesures dans une boucle `for` et ajouter des résultats au tableau

Après les opérations de transformation QFT, insérez le code suivant :

Q#

```
for i in IndexRange(qs) {
 resultArray w/= i <- M(qs[i]);
}
```

La fonction `IndexRange` appelée sur un tableau (par exemple, le tableau de qubits, `qs`) retourne une plage sur les index du tableau. Ici, il est utilisé dans la boucle `for` pour mesurer séquentiellement chaque qubit à l'aide de l'instruction `M(qs[i])`. Chaque type `Result` mesuré (soit `Zero`, soit `One`) est ensuite ajouté à la position d'index correspondante dans `resultArray` avec une instruction de mise à jour et réaffectation.

### ⚠ Remarque

La syntaxe de cette instruction est propre à Q#, mais elle correspond à la réaffectation de variables similaires `resultArray[i] <- M(qs[i])`, vue dans d'autres langages comme F# et R.

Le mot clé `set` est toujours utilisé pour réaffecter des variables liées à l'aide de `mutable`.

## Retourner `resultArray`

Une fois les trois qubits mesurés et les résultats ajoutés à `resultArray`, vous pouvez réinitialiser et désallouer les qubits comme auparavant. Pour retourner les mesures, insérez :

Q#

```
return resultArray;
```

# Exécuter le circuit QFT avec les mesures

Changez maintenant le placement des fonctions `DumpMachine` pour afficher l'état avant et après les mesures. Votre code Q# final doit ressembler à ceci :

Q#

```
import Std.Diagnostics.*;
import Std.Math.*;
import Std.Arrays.*;

operation Main() : Result[] {

 mutable resultArray = [Zero, size = 3];

 use qs = Qubit[3];

 //QFT:
 //first qubit:
 H(qs[0]);
 Controlled R1([qs[1]], (PI()/2.0, qs[0]));
 Controlled R1([qs[2]], (PI()/4.0, qs[0]));

 //second qubit:
 H(qs[1]);
 Controlled R1([qs[2]], (PI()/2.0, qs[1]));

 //third qubit:
 H(qs[2]);

 SWAP(qs[2], qs[0]);

 Message("Before measurement: ");
 DumpMachine();

 for i in IndexRange(qs) {
 resultArray w/= i <- M(qs[i]);
 }

 Message("After measurement: ");
 DumpMachine();

 ResetAll(qs);
 Message("Post-QFT measurement results [qubit0, qubit1, qubit2]: ");
 return resultArray;
}
```

 Conseil

N'oubliez pas d'enregistrer votre fichier chaque fois que vous introduisez une modification du code avant de l'exécuter à nouveau.

#### Q# programme autonome

1. Pour exécuter votre programme, choisissez **Exécuter Q#** dans le menu qui précède **Main**, ou appuyez sur **Ctrl + F5**. Le programme exécute l'opération **Main** sur le simulateur par défaut.
2. Les sorties **Message** et **DumpMachine** s'affichent dans la console de débogage.

La sortie doit ressembler à ceci :

#### Output

Before measurement:

Basis	Amplitude	Probability	Phase
-----			
000>	0.3536+0.0000i	12.5000%	0.0000
001>	0.3536+0.0000i	12.5000%	0.0000
010>	0.3536+0.0000i	12.5000%	0.0000
011>	0.3536+0.0000i	12.5000%	0.0000
100>	0.3536+0.0000i	12.5000%	0.0000
101>	0.3536+0.0000i	12.5000%	0.0000
110>	0.3536+0.0000i	12.5000%	0.0000
111>	0.3536+0.0000i	12.5000%	0.0000

After measurement:

Basis	Amplitude	Probability	Phase
-----			
010>	1.0000+0.0000i	100.0000%	0.0000

Post-QFT measurement results [qubit0, qubit1, qubit2]:

[Zero, One, Zero]

Cette sortie illustre plusieurs choses différentes :

1. Lors de la comparaison du résultat retourné à la pré-mesure **DumpMachine**, il n'illustre *clairement pas* la superposition post-QFT sur les états de base. Une mesure ne retourne qu'un seul état de base, avec une probabilité déterminée par l'amplitude de cet état dans la fonction d'onde du système.
2. À partir de la post-mesure, la mesure **DumpMachine** *change* l'état lui-même, en le projetant de la superposition initiale sur les états de base à l'état de base unique qui correspond à

la valeur mesurée.

Si vous répétez cette opération plusieurs fois, les statistiques des résultats commencent à illustrer la superposition pondérée égale de l'état post-QFT, qui entraîne un résultat aléatoire à chaque tir. *Toutefois*, en plus d'être inefficace et toujours imparfait, cela ne ferait que reproduire les amplitudes relatives des états de base, et non les phases relatives entre elles. Ce dernier n'est pas un problème dans cet exemple, mais les phases relatives s'affichent si une entrée plus complexe est donnée au QFT que  $|000\rangle$ .

## Utiliser les Q# opérations pour simplifier le circuit QFT

Comme mentionné en introduction, la puissance de Q# repose en grande partie sur sa capacité à vous libérer des soucis liés à la gestion des qubits individuels. En effet, si vous voulez développer des programmes quantiques applicables à grande échelle, déterminer si une opération **H** doit être effectuée avant ou après une rotation particulière ne fait que vous ralentir. Azure Quantum fournit l'opération `ApplyQFT`, que vous pouvez utiliser et appliquer pour n'importe quel nombre de qubits.

1. Remplacez tout, de la première **H** opération à l'opération `SWAP`, inclusive, par :

Q#

```
ApplyQFT(qs);
```

2. Votre code doit maintenant ressembler à ceci

Q#

```
import Std.Diagnostics.*;
import Std.Math.*;
import Std.Arrays.*;

operation Main() : Result[] {

 mutable resultArray = [Zero, size = 3];

 use qs = Qubit[3];

 //QFT:
 //first qubit:

 ApplyQFT(qs);

 Message("Before measurement: ");
```



```

DumpMachine();

for i in IndexRange(qs) {
 resultArray w/= i <- M(qs[i]);
}

Message("After measurement: ");
DumpMachine();

ResetAll(qs);
Message("Post-QFT measurement results [qubit0, qubit1, qubit2]: ");
return resultArray;
}

```

3. Réexécutez le Q# programme et notez que la sortie est la même que précédemment.
4. Pour voir l'avantage réel de l'utilisation Q# des opérations, remplacez le nombre de qubits par quelque chose d'autre que 3 :

Q#

```

mutable resultArray = [Zero, size = 4];

use qs = Qubit[4];
//...

```

Vous pouvez donc appliquer le QFT approprié pour un nombre donné de qubits, sans avoir à vous soucier de l'ajout de nouvelles H opérations et rotations sur chaque qubit.

## Contenu connexe

Explorez les autres tutoriels Q# :

- [Le générateur quantique de nombres aléatoires](#) montre comment écrire un Q# programme qui génère des nombres aléatoires à partir de qubits en superposition.
- [L'algorithme](#) de recherche de Grover montre comment écrire un Q# programme qui utilise l'algorithme de recherche de Grover.
- [L'intrication](#) quantique montre comment écrire un Q# programme qui manipule et mesure des qubits et illustre les effets de la superposition et de l'intrication.
- Les [katas quantiques](#) [↗](#) sont des tutoriels auto-rythmés et des exercices de programmation visant à enseigner les éléments de l'informatique quantique et de la programmation simultanément.



# Vecteurs et matrices dans l'informatique quantique

Article • 19/01/2025

Une certaine connaissance de l'algèbre linéaire est essentielle pour comprendre l'informatique quantique. Cet article présente les concepts de base de l'algèbre linéaire et explique comment utiliser des vecteurs et des matrices dans l'informatique quantique.

## Vecteurs

Un vecteur de colonne, ou vecteur pour un format court,  $v$  de dimension (ou taille)  $n$  est une collection de *nombres* complexes  $(v_1, v_2, \dots, v_n)$  organisés en tant que colonne :

$$v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

La norme d'un vecteur  $v$  est définie sous la forme  $\sqrt{\sum_i |v_i|^2}$ . Un vecteur est appelé vecteur *d'unité* si sa norme est 1.

L'adjoint d'un vecteur de colonne  $v$  est un vecteur de ligne indiqué comme  $v^\dagger$  et est défini comme la transpose de conjugué de  $v$ . Pour un vecteur de colonne  $v$  de dimension  $n$ , l'adjoint est un vecteur de ligne de dimension  $1 \times n$  :

$$\begin{bmatrix} v_1 & \vdots & v_n \end{bmatrix}^\dagger = [v_1^* \quad \cdots \quad v_n^*]$$

où  $v_i^*$  désigne la conjugaison complexe de  $v_i$ .

À l'aide de l'algèbre linéaire, l'état d'un qubit  $\psi = a|0\rangle + b|1\rangle$  est décrit comme un vecteur d'état quantique  $\begin{bmatrix} a & b \end{bmatrix}$ , où  $|a|^2 + |b|^2 = 1$ . Pour plus d'informations, consultez [le qubit](#).

## Produit scalaire

Deux vecteurs peuvent être multipliés par le *produit scalaire*, également appelé *produit dot* ou *produit interne*. Comme le nom l'indique, le résultat du produit scalaire de deux vecteurs est un scalaire. Le produit scalaire donne la projection d'un vecteur sur un autre et est utilisé pour exprimer un vecteur sous la forme d'une somme d'autres vecteurs

plus simples. Le produit scalaire entre deux vecteurs  $u$  et  $v$  est indiqué comme  $\langle u, v \rangle = u^\dagger v$  et est défini comme étant

Missing or unrecognized delimiter for \left

Avec le produit scalaire, la norme d'un vecteur  $v$  peut être écrite en tant que  $\sqrt{\langle v, v \rangle}$ .

Vous pouvez multiplier un vecteur par un nombre  $a$  pour former un nouveau vecteur dont les entrées sont multipliées par  $a$ . Vous pouvez également ajouter deux vecteurs  $u$  et  $v$  pour former un nouveau vecteur dont les entrées sont la somme des entrées de  $u$  et  $v$ . Ces opérations sont les suivantes :

$$au + bv = \begin{bmatrix} au_1 + bv_1 \\ au_2 + bv_2 \\ \vdots \\ au_n + bv_n \end{bmatrix}$$

## Matrices

Une *matrice* de taille  $m \times n$  est une collection de *nombres complexes*  $m \cdot n$  organisés en *lignes*  $m$  et  $n$  colonnes, comme indiqué ci-dessous :

$$M = \begin{bmatrix} M_{11} & M_{12} & \cdots & M_{1n} \\ M_{21} & M_{22} & \cdots & M_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ M_{m1} & M_{m2} & \cdots & M_{mn} \end{bmatrix}$$

### Notes

Notez qu'un vecteur de dimension  $n$  est simplement une matrice de taille  $n \times 1$ .

Les opérations quantiques sont représentées par *des matrices carrées*, autrement dit, le nombre de lignes et de colonnes est égal. Par exemple, les opérations à qubit unique sont représentées par  $2 \times 2$  matrices, comme l'opération Pauli  $X$

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

### Conseil

Dans Q#, l'opération Pauli  $X$  est représentée par l'opération `X`.

Comme pour les vecteurs, vous pouvez multiplier une matrice avec un nombre  $c$  pour obtenir une nouvelle matrice où chaque entrée est multipliée par  $c$ , et deux

matrices de même taille peuvent être ajoutées pour produire une nouvelle matrice dont les entrées sont la somme des entrées respectives des deux matrices.

## Multiplication de matrices

Vous pouvez également multiplier une matrice  $M$  de dimension  $m \times n$  et une matrice  $N$  de dimension  $n \times p$  pour obtenir une nouvelle matrice  $P$  de dimension  $m \times p$  comme suit :

$$\begin{aligned} & \begin{bmatrix} M_{11} & M_{12} & \cdots & M_{1n} \\ M_{21} & M_{22} & \cdots & M_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ M_{m1} & M_{m2} & \cdots & M_{mn} \end{bmatrix} \begin{bmatrix} N_{11} & N_{12} & \cdots & N_{1p} \\ N_{21} & N_{22} & \cdots & N_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ N_{n1} & N_{n2} & \cdots & N_{np} \end{bmatrix} = \begin{bmatrix} P_{11} & P_{12} & \cdots & P_{1p} \\ P_{21} & P_{22} & \cdots & P_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ P_{m1} & P_{m2} & \cdots & P_{mp} \end{bmatrix} \end{aligned}$$

où les entrées de  $P$  sont  $P_{ik} = \sum_j M_{ij}N_{jk}$ . Par exemple, l'entrée  $P_{11}$  est le produit scalaire de la première ligne de  $M$  avec la première colonne de  $N$ . Notez que comme un vecteur est simplement un cas particulier de matrice, cette définition s'applique à la multiplication matrice-vecteur.

## Types spéciaux de matrices

La *matrice d'identité*, notée  $\mathbb{I}$ , est une matrice carrée spéciale. Tous ses éléments diagonaux sont égaux à 1, et les éléments restants sont égaux à 0 :

$$\mathbb{I} =$$

$$\begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

.

Pour une matrice carrée  $A$ , une matrice  $B$  est son *inverse* si  $AB = BA = \mathbb{I}$ . Si une matrice  $A$  a une inverse, la matrice inverse est unique et est écrite en tant que  $A^{-1}$ .

Pour n'importe quelle matrice  $M$ , la matrice adjointe ou transposée conjuguée de  $M$  est une matrice  $N$  telle que  $N_{ij} = M_{ji}^*$ . L'adjoint de  $M$  est indiqué  $M^\dagger$ .

Une matrice  $U$  est *unitaire* si  $UU^\dagger = U^\dagger U = \mathbb{I}$ , soit  $U^{-1} = U^\dagger$ . La conservation de la norme d'un vecteur est l'une des propriétés les plus importantes des matrices unitaires. Elle s'explique ainsi :

$$\langle v, v \rangle = v^\dagger v = v^\dagger U^{-1} U v = v^\dagger U^\dagger U v = \langle U v, U v \rangle.$$

### ⓘ Notes

Les opérations quantiques sont représentées par des matrices unitaires, qui sont des matrices carrées dont l'adjoint est égal à leur inverse.

Une matrice  $M$  est appelée *Hermitienne* si  $M = M^\dagger$ .

En informatique quantique, vous ne rencontrerez que les matrices hermitiennes et les matrices unitaires.

## Produit tensoriel

Une autre opération importante est le *produit* tensoriel, également appelé *produit* direct matriciel ou *produit* Kronecker.

Considérez les deux vecteurs  $v = \begin{bmatrix} a \\ b \end{bmatrix}$  et  $u = \begin{bmatrix} c \\ d \end{bmatrix}$ . Leur produit tensoriel est noté  $v \otimes u$  et génère une matrice par blocs.

$$\begin{bmatrix} a \\ b \end{bmatrix} \otimes \begin{bmatrix} c \\ d \end{bmatrix} = \begin{bmatrix} a \begin{bmatrix} c \\ d \end{bmatrix} & b \begin{bmatrix} c \\ d \end{bmatrix} \end{bmatrix} = \begin{bmatrix} ac \\ ad \\ bc \\ bd \end{bmatrix}$$

### ⓘ Notes

Notez que le produit tensoriel se distingue de la multiplication de matrices, qui est une opération entièrement différente.

Le produit tensoriel est utilisé pour représenter l'état combiné de plusieurs qubits. La puissance réelle de l'informatique quantique provient de tirer parti de plusieurs qubits pour effectuer des calculs. Pour plus d'informations, consultez [Opérations sur plusieurs qubits](#).

## Contenu connexe

- [Qubit](#)
- [Qubits multiples](#)
- [Notation de Dirac](#)
- [Mesures Pauli](#)

---

## Commentaires

Cette page a-t-elle été utile ?

 **Yes**

 **No**

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Le qubit en informatique quantique

Article • 20/01/2025

Tout comme les bits sont l'objet d'information fondamental en informatique classique, les [qubits](#) (bits quantiques) sont l'objet d'information fondamental en informatique quantique. Pour vous permettre de comprendre cette correspondance, cet article aborde l'exemple le plus simple : un qubit unique.

## Représentation d'un qubit

Si un bit, ou chiffre binaire, peut avoir la valeur 0 ou 1, un qubit, peut avoir la valeur 0, 1 ou une superposition quantique de 0 et 1.

L'état d'un qubit unique peut être décrit par un vecteur colonne à deux dimensions de norme unitaire, c'est-à-dire que la somme des carrés des amplitudes de ses entrées doit être égale à 1. Ce vecteur, appelé vecteur d'état quantique, contient toutes les informations nécessaires pour décrire le système quantique à un qubit tout comme un bit unique contient toutes les informations nécessaires pour décrire l'état d'une variable binaire. Pour connaître les principes de base des vecteurs et des matrices dans l'informatique quantique, consultez [Vector et matrices](#).

Tout vecteur colonne à deux dimensions de nombres réels ou complexes de norme 1 représente un état quantique possible contenu par un qubit. Ainsi,  $\begin{bmatrix} \alpha \\ \beta \end{bmatrix}$  représente l'état d'un qubit si  $\alpha$  et  $\beta$  sont des nombres complexes satisfaisant  $|\alpha|^2 + |\beta|^2 = 1$ .

Certains exemples de vecteurs d'état quantique valides représentant des qubits sont  $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$  et  $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ . Ces deux vecteurs forment une base pour l'espace vectoriel qui décrit l'état du qubit. Ceci signifie que tout vecteur d'état quantique peut être écrit comme somme de ces vecteurs de base. Plus précisément, le vecteur  $\begin{bmatrix} x \\ y \end{bmatrix}$  peut être écrit sous la forme  $x \begin{bmatrix} 1 \\ 0 \end{bmatrix} + y \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ . Alors qu'une rotation de ces vecteurs offrirait une base parfaitement valide pour le qubit, nous choisissons celle-ci en particulier et l'appelons *base de calcul*.

Nous prenons ces deux états quantiques pour correspondre avec les deux états d'un bit classique, à savoir 0 et 1. La convention standard consiste à choisir

$$0 \equiv \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$



$$1 \equiv \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

même si le choix opposé pourrait également être retenu. Ainsi, sur l'infinité de vecteurs d'état quantique à un qubit possibles, seuls deux correspondent aux états des bits classiques. Ce n'est pas le cas de tous les autres états quantiques.

## Mesure d'un qubit

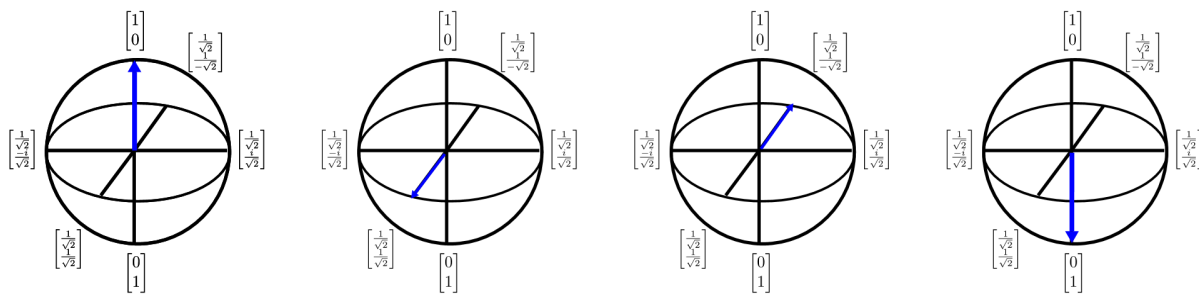
Sachant désormais comment représenter un qubit, nous pouvons déterminer intuitivement ce que ces états représentent en abordant le concept de [mesure](#). Une mesure correspond à l'idée informelle d'« observation » du qubit, ce qui ramène immédiatement l'état quantique à l'un des deux états classiques  $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$  ou  $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ . Quand un qubit donné par le vecteur d'état quantique  $\begin{bmatrix} \alpha \\ \beta \end{bmatrix}$  est mesuré, nous obtenons le résultat 0 avec la probabilité  $|\alpha|^2$  et le résultat 1 avec la probabilité  $|\beta|^2$ . Si le résultat est 0, le nouvel état du qubit est  $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$  ; si le résultat est 1, son état est  $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ . Notez que la somme de ces probabilités est égale à 1 en raison de la condition de normalisation  $|\alpha|^2 + |\beta|^2 = 1$ .

Les propriétés de mesure signifient également que le signe global du vecteur d'état quantique n'a pas d'importance. Rendre un vecteur négatif équivaut à  $\alpha \rightarrow -\alpha$  et  $\beta \rightarrow -\beta$ . Comme la probabilité de mesure de 0 et 1 dépend du carré des amplitudes des termes, l'insertion de ces signes ne change en rien les probabilités. Ces phases sont généralement appelées &« phases globales » et, plus généralement, peuvent être de forme  $e^{i\phi}$  plutôt que simplement  $\pm 1$ .

Une dernière propriété importante de la mesure est qu'elle ne détruit pas nécessairement tous les vecteurs d'état quantique. Si nous commençons par un qubit dans l'état  $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$  correspondant à l'état classique 0, la mesure de cet état produira toujours le résultat 0 et laissera l'état quantique inchangé. Dans ce sens, s'il y a uniquement des bits classiques (par exemple, des qubits qui sont  $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$  ou  $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ ), la mesure n'endommage pas le système. Ceci signifie que nous pouvons répliquer des données classiques et les manipuler sur un ordinateur quantique comme on pourrait le faire avec un ordinateur classique. Toutefois, la capacité à stocker des informations dans les deux états à la fois est ce qui élève l'informatique quantique au-delà de ce que permet l'approche classique et empêche les ordinateurs quantiques de copier des données quantiques de manière non discriminatoire. Consultez également cette page sur le [théorème de non-clonage](#).

# Visualisation des qubits et transformations à l'aide de la sphère Bloch

Les qubits peuvent également être représentés en 3D à l'aide de la [sphère de Bloch](#). La sphère de Bloch offre un moyen de décrire un état quantique à un qubit (un vecteur à deux dimensions complexe) sous la forme d'un vecteur à valeurs réelles en trois dimensions. Cet aspect est important parce que nous pouvons alors visualiser les états à un qubit et ainsi de développer un raisonnement précieux pour comprendre les états multiqubits (dont la représentation ne fonctionne malheureusement pas avec la sphère de Bloch). La sphère de Bloch peut être représentée comme suit :



Les flèches de ce schéma indiquent la direction du vecteur d'état quantique et chaque transformation de la flèche peut être considérée comme une rotation autour de l'un des axes cardinaux. Envisager un calcul quantique comme séquence de rotations permet un excellent raisonnement intuitif, qu'il est cependant difficile d'utiliser pour concevoir et décrire des algorithmes. Q# atténue ce problème en fournissant un langage qui permet de décrire ces rotations.

## Opérations à un qubit

Les ordinateurs quantiques traitent les données en appliquant un ensemble universel de portes quantiques qui peuvent émuler n'importe quelle rotation du vecteur d'état quantique. Cette notion d'universalité est comparable à celle de l'informatique classique où un ensemble de portes est considéré comme universel si chaque transformation des bits d'entrée peut être effectuée à l'aide d'un circuit de longueur finie. En informatique quantique, les transformations valides que nous sommes autorisés à effectuer sur un qubit sont les transformations unitaires et la mesure. L'*opération adjointe*, ou transposée conjuguée complexe, revêt une importance capitale en informatique quantique, car elle est nécessaire à l'inversion des transformations quantiques.

Les opérations à qubit unique ou les portes quantiques à qubit unique peuvent être classées en deux catégories : les portes Clifford et les portes non-Clifford. Les portes non-Clifford se composent uniquement de *laporteT* (également appelée *porte* $\pi/8$ ).

$$T = \begin{bmatrix} 1 & 0 & 0 & e^{i\pi/4} \end{bmatrix}.$$

L'ensemble standard de portes à qubit Clifford, inclus par défaut dans Q#, include

$$H = \frac{1}{\sqrt{2}}$$

$$\begin{bmatrix} 1 & 1 & 1 & -1 \end{bmatrix}$$

, \quad S =

$$\begin{bmatrix} 1 & 0 & 0 & i \end{bmatrix}$$

= T^2, \quad X =

$$\begin{bmatrix} 0 & 1 & 1 & 0 \end{bmatrix}$$

= HT^4H, \$

$$Y = \begin{bmatrix} 0 & -i & i & 0 \end{bmatrix} = T^2HT^4HT^6, \quad Z = \begin{bmatrix} 1 & 0 & 0 & -1 \end{bmatrix} = T^4.$$

Ici, les opérations  $X$ ,  $Y$  et  $Z$  sont utilisées particulièrement fréquemment et sont appelées [opérateurs de Pauli](#), du nom de leur créateur Wolfgang Pauli. Avec la porte non Clifford (porte  $T$ ), ces opérations peuvent être composées pour approximer toute transformation unitaire sur un qubit unique.

Si la précédente représente les portes primitives les plus populaires pour décrire les opérations au niveau logique de la pile (considérez le niveau logique comme le niveau de l'algorithme quantique), il est souvent pratique de considérer les opérations moins basiques au niveau algorithmique, par exemple les opérations plus proches d'un niveau descriptif des fonctions. Heureusement, Q# offre également des méthodes permettant d'implémenter des unitaires de plus haut niveau qui, à leur tour, permettent d'implémenter des algorithmes de haut niveau sans décomposer explicitement tous les éléments en portes Clifford et  $T$ .

La rotation à un qubit représente la primitive de ce type la plus simple. On considère généralement trois rotations à un qubit comme :  $R_x$ ,  $R_y$  et  $R_z$ . Pour visualiser l'action de la rotation  $R_x(\theta)$ , par exemple, imaginez que vous pointez votre pouce droit dans la direction de l'axe  $x$  de la sphère de Bloch et que vous faites pivoter le vecteur avec votre main sur un angle de  $\theta/2$  radians. Ce facteur de confusion de 2 est dû à la séparation des vecteurs orthogonaux avec un angle de  $180^\circ$  quand ils sont tracés sur la sphère de Bloch, mais de  $90^\circ$  géométriquement. Les matrices unitaires correspondantes sont les suivantes :

$$R_z(\theta) = e^{-i\theta Z/2} = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix},$$

$$R_x(\theta) = e^{-i\theta X/2} = HR_z(\theta)H = \begin{bmatrix} \cos(\theta/2) & -i \sin(\theta/2) \\ -i \sin(\theta/2) & \cos(\theta/2) \end{bmatrix},$$

$$R_y(\theta) = e^{-i\theta Y/2} = SHR_z(\theta)HS^\dagger = \begin{bmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{bmatrix}.$$

Tout comme trois rotations quelconques peuvent être combinées pour effectuer une rotation arbitraire dans trois dimensions, la représentation sur la sphère de Bloch montre que toute matrice unitaire peut également être écrite sous la forme d'une séquence de trois rotations. Plus précisément, pour chaque matrice unitaire  $U$ , il existe  $\alpha, \beta, \gamma, \delta$ , de sorte que  $U = e^{i\alpha}R_x(\beta)R_z(\gamma)R_x(\delta)$ . Ainsi,  $R_z(\theta)$  et  $H$  forment également un ensemble de portes universel bien qu'il ne s'agisse pas d'un ensemble discret, car  $\theta$  peut prendre n'importe quelle valeur. Pour cette raison, et de par les applications en simulation quantique, ces portes continues sont cruciales pour le calcul quantique, en particulier au niveau conceptuel des algorithmes quantiques. Pour parvenir à une implémentation matérielle tolérante aux pannes, elles seront finalement compilées en séquences de portes discrètes approximant ces rotations de façon très proche.

## Contenu connexe

- [Qubits multiples](#)
- [Notation de Dirac](#)
- [Mesures Pauli](#)
- [Portes T et usines T](#)
- [Circuits quantiques](#)

## Commentaires

Cette page a-t-elle été utile ?

[Indiquer des commentaires sur le produit](#) [↗](#) | [Obtenir de l'aide sur Microsoft Q&A](#)

# Opérations sur plusieurs qubits

Cet article passe en revue les règles utilisées pour générer des états multiqubits à partir d'états à qubit unique. En outre, il présente les opérations de porte qui doivent être incluses dans un ensemble de portes pour créer un ordinateur quantique multiqubit universel. Ces outils sont nécessaires pour comprendre les ensembles de portes couramment utilisés dans le Q# code. Ils sont également importants pour obtenir une intuition sur la raison pour laquelle les effets quantiques tels que l'intrication ou l'interférence rendent l'informatique quantique plus puissante que l'informatique classique.

## Portes à qubit unique et multi-qubit

La véritable puissance de l'informatique quantique devient évidente à mesure que vous augmentez le nombre de qubits. Les qubits uniques possèdent certaines fonctionnalités contre-intuitives, telles que la possibilité d'être dans plusieurs états à un moment donné. Cependant, si vous ne disposez que de portes à qubit unique dans un ordinateur quantique, alors une calculatrice et certainement un superordinateur classique éclipseraient sa capacité de calcul.

La puissance de l'informatique quantique réside en partie dans le fait que la dimension de l'espace vectoriel des vecteurs d'état quantique augmente de façon exponentielle avec le nombre de qubits. Cela signifie que si un seul qubit peut être modélisé très simplement, la simulation d'un calcul quantique à 50 qubits dépassera sans doute les limites des superordinateurs existants. L'augmentation de la taille du calcul par un seul qubit supplémentaire double la mémoire nécessaire pour stocker l'état et double approximativement le temps de calcul. Ce doublement rapide de la puissance de calcul est la raison pour laquelle un ordinateur quantique avec un nombre relativement faible de qubits peut largement dépasser les plus puissants des superordinateurs d'aujourd'hui, de demain, et même au-delà, pour certaines tâches de calcul.

## États à deux qubits

Supposons que vous avez deux qubits distincts dans les états suivants :

$$\psi = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$
$$\phi = \begin{bmatrix} \gamma \\ \delta \end{bmatrix}$$

L'état à deux qubits correspondant est le résultat du produit tensoriel, ou [du produit Kronecker](#), de vecteurs, qui est défini comme suit :

$$\psi \otimes \phi = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \otimes \begin{bmatrix} \gamma \\ \delta \end{bmatrix} = \begin{bmatrix} \alpha \begin{bmatrix} \gamma \\ \delta \end{bmatrix} \\ \beta \begin{bmatrix} \gamma \\ \delta \end{bmatrix} \end{bmatrix} = \begin{bmatrix} \alpha\gamma \\ \alpha\delta \\ \beta\gamma \\ \beta\delta \end{bmatrix}.$$

Par conséquent, étant donné deux états  $\psi$  à qubit unique et  $\phi$ , chacun de la dimension 2, l'état  $\psi \otimes \phi$  à deux qubits correspondant est de 4 dimensions. Le vecteur

$$\begin{bmatrix} \alpha_{00} \\ \alpha_{01} \\ \alpha_{10} \\ \alpha_{11} \end{bmatrix}$$

représente un état quantique sur deux qubits si

$$|\alpha_{00}|^2 + |\alpha_{01}|^2 + |\alpha_{10}|^2 + |\alpha_{11}|^2 = 1.$$

Plus généralement, vous pouvez voir que le vecteur  $d'unit\acute{e} v_1 v_2 \otimes v_n \otimes \dots \otimes$  de dimension  $2 \cdot 2 \cdot 2 \dots = 2^n$  représente l'état quantique de  $n$  qubits à l'aide de cette construction. Comme pour les qubits uniques, le vecteur d'état quantique de plusieurs qubits contient toutes les informations nécessaires pour décrire le comportement du système. Pour plus d'informations sur les vecteurs et les produits tensoriels, consultez [Vecteurs et matrices en informatique quantique](#).

La base de calcul pour les états à deux qubits est formée par les produits tensoriels d'états de base d'un qubit. Pour un système à deux qubits, il existe les quatre états suivants :

$$00 \equiv \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$01 \equiv \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$10 \equiv \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$$11 \equiv \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Bien que vous puissiez toujours prendre le produit tensoriel de deux états à qubit unique pour former un état à deux qubits, tous les états quantiques à deux qubits ne peuvent pas être écrits comme le produit tensoriel de deux états à qubit unique. Par exemple, il n'existe aucun état  $\psi$  et  $\phi$  dont le produit tensoriel est l'état.

$$\psi \otimes \phi = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Un tel état à deux qubits, qui ne peut pas être écrit en tant que produit tensoriel d'états de qubit unique, est appelé état intriqué ; les deux qubits sont dits [enchevêtrés](#). Faiblement parlant, parce que l'état quantique ne peut pas être considéré comme un produit tensoriel d'états qubits uniques, les informations que l'état contient ne sont pas limitées individuellement à l'un des qubits. Au lieu de cela, les informations sont stockées non localement, dans les corrélations qui existent entre les deux états. Cette non-localité de l'information est l'une des principales caractéristiques distinctives de

l'informatique quantique sur l'informatique classique, et est essentielle pour de nombreux protocoles quantiques, y compris la correction des erreurs quantiques.

## Mesure des états à deux qubits

La mesure des états à deux qubits est similaire aux mesures à qubit unique. Mesure de l'état

$$\begin{bmatrix} \alpha_{00} \\ \alpha_{01} \\ \alpha_{10} \\ \alpha_{11} \end{bmatrix}$$

génère 00 avec probabilité  $|\alpha_{00}|^2$ , 01 avec probabilité  $|\alpha_{01}|^2$ , 10 avec probabilité  $|\alpha_{10}|^2$  et 11 avec probabilité  $|\alpha_{11}|^2$ . Les variables  $\alpha_{00}$ ,  $\alpha_{01}$ ,  $\alpha_{10}$  et  $\alpha_{11}$

ont été délibérément nommées pour rendre cette connexion claire. Après la mesure, si le résultat est 00, l'état quantique du système à deux qubits s'effondre, devenant ainsi.

$$00 \equiv \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

Il est également possible de mesurer un qubit d'un état quantique à deux qubits. Lorsque vous mesurez un seul qubit d'un état à deux qubits, l'impact de la mesure est subtilement différent de la mesure de deux qubits. Il est différent, car l'état entier n'est pas réduit à un état de base de calcul, plutôt qu'à un seul sous-système. En d'autres termes, la mesure d'un qubit d'un état à deux qubits réduit uniquement le sous-système associé à un état de base de calcul.

Pour ce faire, considérez mesurer le premier qubit de l'état suivant, qui est formé en appliquant la transformation de Hadamard  $H$  sur deux qubits initialement définis à l'état 0 :

$$H^{\otimes 2} \left( \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right) = \frac{1}{2}$$

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$\otimes$

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\right) = \frac{1}{2}$$

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$=\frac{1}{2}$$

$$\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\mapsto \begin{cases} \text{Résultat} = 0 & \frac{1}{\sqrt{2}} \end{cases}$$

$$\begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{Résultat} = 1 & \frac{1}{\sqrt{2}}$$

$$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

$$\end{cases}.$$

Les deux résultats ont une probabilité de 50% de se produire. Ce résultat peut être intuitif du fait que l'état quantique avant la mesure ne change pas si 0 est échangé avec 1 sur le premier qubit.

La règle mathématique pour mesurer le premier ou le deuxième qubit est simple. Supposons  $e_k$  être le vecteur de base de calcul  $k^{\text{ième}}$  et  $S$  être l'ensemble de toutes les  $e_k$  de sorte que le qubit en question accepte la valeur 1 pour cette valeur de  $k$ . Par exemple, si vous souhaitez mesurer le premier qubit,  $S$  se compose de  $e_1 \equiv 10$  et  $e_3 \equiv 11$ . De même, si vous êtes intéressé par le deuxième qubit  $S$  se compose de  $e_2 \equiv 01$  et  $e_3 \equiv 11$ . Ensuite, la probabilité de mesurer le qubit choisi à 1 est pour le vecteur d'état  $\psi$

$$P(\text{résultat} = 1) = \sum_{e_k \text{ dans l'ensemble } S} \psi^\dagger e_k e_k^\dagger \psi.$$

⚠ Remarque

Cet article utilise le format little-endian pour étiqueter la base de calcul. Dans un format little-endian, les bits les moins significatifs viennent en premier. Par exemple, au format little-endian, la chaîne de bits 001 représente le nombre quatre.

Étant donné que chaque mesure qubit ne peut générer que 0 ou 1, la probabilité de mesurer 0 est  $1 - P(\text{résultat} = 1)$ , c'est pourquoi vous avez seulement besoin d'une formule pour la probabilité de



mesurer 1.

L'effet qu'une telle mesure a sur l'état peut être exprimé de façon mathématique sous la forme suivante

$$\psi \mapsto \frac{\sum_{e_k \text{ dans l'ensemble } S} e_k e_k^\dagger \psi}{\sqrt{P(\text{résultat} = 1)}}.$$

Le lecteur prudent peut s'inquiéter de ce qui se passe si le dénominateur est égal à zéro. Bien que cet état ne soit pas défini, vous n'avez pas besoin de vous soucier de ces éventualités, car la probabilité est égale à zéro.

Si vous prenez  $\psi$  pour être le vecteur d'état uniforme donné précédemment et êtes intéressé par mesurer le premier qubit, puis

$$P(\text{mesure du premier qubit} = 1) = (\psi^\dagger e_1)(e_1^\dagger \psi) + (\psi^\dagger e_3)(e_3^\dagger \psi) = |e_1^\dagger \psi|^2 + |e_3^\dagger \psi|^2.$$

Il s'agit simplement de la somme des deux probabilités qui seraient attendues pour mesurer les résultats 10 et 11. Pour notre exemple, cela revient à

$$\frac{1}{4} \left| \right.$$

$$\begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\left|^2 + \frac{1}{4} \left| \right.$$

$$\begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\left|^2 = \frac{1}{2}.$$

qui correspond parfaitement à notre intuition. De même, l'état après le premier qubit est mesuré comme 1 peut être écrit en tant que

$$\frac{\frac{e_1}{2} + \frac{e_3}{2}}{\sqrt{\frac{1}{2}}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

là encore, conformément à notre intuition.

## Opérations à deux qubits

Comme avec les qubits uniques, toute transformation unitaire est une opération valide sur les qubits. En général, une transformation unitaire sur  $n$  qubits est une matrice  $U$  de taille  $2^n \times 2^n$  (elle agit donc sur les vecteurs de taille  $2^n$ ), de sorte que  $U^{-1} = U^\dagger$ . Par exemple, la porte CNOT (controlled-NOT) est une porte à deux qubits couramment utilisée, qui est représentée par la matrice unitaire suivante :

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Nous pouvons également former des portes à deux qubits en appliquant des portes à qubit unique sur les deux qubits. Par exemple, si vous appliquez les portes

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

et

$$\begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

aux premier et deuxième qubits, respectivement, cela revient à appliquer l'opérateur unitaire à deux qubits donné par leur produit de tenseur :

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \otimes \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae & af & be & bf \\ ag & ah & bg & bh \\ ce & cf & de & df \\ cg & ch & dg & dh \end{bmatrix}$$

Ainsi, vous pouvez former des portes à deux qubits en prenant le produit tensoriel de certaines portes à qubit connues. Voici quelques exemples de portes à deux qubits :  $HH \otimes I$ ,  $X \otimes I$  et  $X \otimes Z$ .

Notez que si deux portes à qubit unique définissent une porte à deux qubits en prenant leur produit tensoriel, l'inverse n'est pas vrai. Les portes à deux qubits ne peuvent pas être écrites en tant que produit tensoriel de portes à qubit unique. Les portes de ce type se nomment *portes d'intrication*. Un exemple de porte d'intrication est la porte CNOT.

L'intuition associée à une porte controlled-not peut être généralisée aux portes arbitraires. Une porte contrôlée en général est une porte qui agit comme une identité, sauf si un qubit spécifique est 1. Vous désignez une unité contrôlée, contrôlée dans ce cas sur le qubit étiqueté  $x$ , avec un  $\Lambda_x(U)$ . Par exemple  $\Lambda_{x,0}(U)e_1 \otimes \psi = e_1 \otimes U\psi$  et  $\Lambda_{x,1}(U)e_1 \otimes \psi = e_1 \otimes U\psi + e_0 \otimes \psi$ , où  $e_0$  et  $e_1$  sont les vecteurs de base d'un qubit unique correspondant aux valeurs 0 et 1.

Par exemple, considérez la porte contrôlée —  $Z$  suivante, puis vous pouvez l'exprimer comme

$$\Lambda_{0,Z} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} = (\mathbf{H} \otimes I) \text{CNOT} (H \otimes I)$$

La création d'unités contrôlées de manière efficace est un défi majeur. La façon la plus simple d'implémenter des unités contrôlées nécessite de former une base de données de versions contrôlées de portes fondamentales et de remplacer chaque porte fondamentale dans l'opération unitaire d'origine par son équivalent contrôlé. Cette méthode est souvent gaspillée, et l'intuition peut souvent être utilisée pour remplacer simplement quelques portes par des versions contrôlées, permettant d'atteindre le même résultat. Pour cette raison, l'infrastructure offre la possibilité d'effectuer la méthode

naïve de contrôle ou d'autoriser l'utilisateur à définir une version contrôlée de l'unitaire si une version optimisée paramétrée à la main est connue.

Les portes peuvent également être contrôlées à l'aide d'informations classiques. Une porte NOT contrôlée classiquement, par exemple, est simplement une porte NOT ordinaire qui n'est appliquée que si un bit classique est 1, par opposition à un bit quantique. Dans ce sens, une porte contrôlée classiquement peut être considérée comme une instruction if dans le code quantique, où la porte est appliquée uniquement dans une branche du code.

Comme dans le cas d'un qubit unique, un ensemble de portes à deux qubits est universel si une matrice unitaire  $4 \times 4$  peut être approchée par un produit de portes issues de cet ensemble avec une précision arbitraire. Un exemple d'ensemble de portes universel est constitué de la porte Hadarmard, de la porte T et de la porte CNOT. En prenant des produits de ces portes, vous pouvez estimer n'importe quelle matrice unitaire sur deux qubits.

## Systèmes multiqubits

Nous suivons exactement les mêmes modèles que ceux explorés dans le scénario avec deux qubits pour créer des états quantiques multiqubits à partir de systèmes plus petits. De tels états sont créés en formant les produits tensoriels d'états plus petits. Par exemple, vous pouvez encoder la chaîne de bits 1011001 sur un ordinateur quantique. Vous pouvez encoder cela en tant que

$$1011001 \equiv \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Les portes quantiques fonctionnent exactement de la même façon. Par exemple, si vous souhaitez appliquer la *porte* $X$  au premier qubit, puis effectuer un CNOT entre les deuxième et troisième qubits, vous pouvez exprimer cette transformation comme

$$\begin{aligned} &X \otimes \operatorname{CNOT}_{12} \otimes \mathbf{1} \otimes \\ &\mathbf{1} \otimes \mathbf{1} \otimes \mathbf{1} \end{aligned}$$

$$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$\otimes$

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$\otimes$

$$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$\otimes$

$$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

\otimes

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

\otimes

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

\otimes

$$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

\equiv 0011001. \end{align} \\$\\$

Dans de nombreux systèmes qubits, il est souvent nécessaire d'allouer et de libérer des qubits qui servent de mémoire temporaire pour l'ordinateur quantique. Un tel qubit est dit être auxiliaire. Par défaut, vous pouvez supposer que l'état qubit est initialisé pour  $e_0$  lors de l'allocation. Vous pouvez également supposer que la porte revient à  $e_0$  avant la désallocation. Cette hypothèse est importante car si un qubit auxiliaire devient enchevêtré avec un autre registre qubit lorsqu'il est désalloué, le processus de désallocation endommage le qubit auxiliaire. Pour cette raison, vous supposez toujours que ces qubits sont rétablis à leur état initial avant d'être libérés.

Enfin, même si de nouvelles portes ont dû être ajoutées à notre ensemble de portes afin d'obtenir un calcul quantique universel pour deux ordinateurs quantiques à deux qubits, il n'est pas nécessaire d'ajouter des portes dans le scénario multiqubit. Les portes H, T et CNOT forment un ensemble universel de portes sur de nombreux qubits, car toute transformation unitaire générale peut être divisée en une série de deux rotations de qubits. Vous pouvez ensuite utiliser la théorie développée pour le cas à deux qubits et l'utiliser à nouveau ici lorsque vous avez de nombreux qubits.

#### ⚠ Remarque

Bien que la notation algébrique linéaire utilisée à ce stade puisse certainement être utilisée pour décrire les états multi-qubits, elle devient de plus en plus fastidieuse à mesure que vous augmentez la taille des états. Le vecteur de colonne obtenu pour une chaîne 7 bits, par exemple, est 128 dimensions, ce qui rend son expression fastidieuse à l'aide de la notation décrite précédemment. Au lieu de cela, la notation Dirac, un raccourci symbolique qui simplifie la représentation des états quantiques, est utilisé. Pour plus d'informations, consultez [la notation Dirac](#).

## Contenu connexe

- [Notation de Dirac](#)

- [Mesures Pauli](#)
  - [Portes T et usines T](#)
  - [Circuits quantiques](#)
  - [Oracles quantiques](#)
- 

Last updated on 05/01/2026

# Notation Dirac en informatique quantique

Article • 20/01/2025

[notation Dirac](#) est un moyen concis et puissant de décrire les états et opérations quantiques. Il est nommé après le physicien Paul Dirac, qui a développé la notation dans les années 1930. La notation Dirac est utilisée dans l'informatique quantique pour décrire les états quantiques, les opérations quantiques et les mesures quantiques.

Cet article vous présente la notation Dirac et vous montre comment l'utiliser pour décrire les états et opérations quantiques.

## Vecteurs en notation Dirac

Il existe deux types de vecteurs en notation Dirac : le vecteur *bra*, correspondant à un vecteur de ligne, et le vecteur *ket*, correspondant à un vecteur de colonne. C'est pourquoi la notation (ou formalisme) de Dirac est également appelée notation *bra-ket*.

Si  $\psi$  est un vecteur de colonne, vous pouvez l'écrire en notation Dirac comme  $|\psi\rangle$ , où le  $\cdot$  indique qu'il s'agit d'un vecteur.

De même, le vecteur ligne  $\psi^\dagger$  est exprimé sous la forme  $\langle\psi|$ , qui est un vecteur *bra*. En d'autres termes,  $\psi^\dagger$  s'obtient en appliquant le conjugué complexe des entrées aux éléments de la transposée de  $\psi$ . La notation bra-ket implique directement que  $\langle\psi|\psi\rangle$  est le produit intérieur du vecteur  $\psi$  avec lui-même, qui est par définition 1.

Plus généralement, si  $\psi$  et  $\phi$  sont des vecteurs d'état quantique, leur produit intérieur est  $\langle\phi|\psi\rangle$ . Ce produit intérieur implique que la probabilité de mesurer l'état  $|\psi\rangle$  comme étant  $|\phi\rangle$  est  $|\langle\phi|\psi\rangle|^2$ .

Les états de base de calcul 0 et 1 sont représentés comme \$

$$\begin{bmatrix} 1 & 0 \end{bmatrix}$$

$$=|\text{ket}\{0\}\rangle$$

$$\begin{bmatrix} 0 & 1 \end{bmatrix}$$

$$=|\text{ket}\{1\}\rangle, \text{ respectivement.}$$

## Exemple : Représenter l'opération Hadamard avec la notation de Dirac

Nous allons appliquer la porte Hadamard  $H$  aux états quantiques  $|0\rangle$  et  $|1\rangle$  à l'aide de la notation Dirac :

$$\frac{1}{\sqrt{2}}$$

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$=H|0\rangle=|+\rangle$$

$$\frac{1}{\sqrt{2}}$$

$$\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$=H|1\rangle=|-\rangle$$

Les états résultants correspondent aux vecteurs unitaires dans les directions  $+x$  et  $-x$  de la sphère de Bloch. Ces états peuvent également être développés à l'aide de la notation de Dirac comme sommes de  $|0\rangle$  et  $|1\rangle$  :

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

## Vecteurs de la base de calcul

Chaque état quantique peut toujours être exprimé sous forme de somme de vecteurs de base de calcul et de telles sommes sont facilement exprimées à l'aide de la notation Dirac. L'inverse est également vrai dans le sens où les états  $|+\rangle$  et  $|-\rangle$  constituent aussi une base pour les états quantiques. Vous pouvez le déduire ainsi :

$$|0\rangle = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle)$$

$$|1\rangle = \frac{1}{\sqrt{2}}(|+\rangle - |-\rangle)$$

Prenons par exemple le bra-ket  $\langle 0|1\rangle$ , qui est le produit intérieur entre 0 et 1 en notation de Dirac. Il peut s'écrire ainsi :

$$\langle 0|1\rangle = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0.$$

Cet exemple indique que  $|0\rangle$  et  $|1\rangle$

sont des vecteurs orthogonaux, ce qui signifie que  $\langle 0|1\rangle = \langle 1|0\rangle = 0$

. Par ailleurs, par définition,  $\langle 0 | 0 \rangle = \langle 1 | 1 \rangle = 1$ , ce qui signifie que les deux vecteurs de la base de calcul peuvent également être appelés *orthonormaux*.

Ces propriétés orthonormales sont utilisées dans l'exemple suivant. Si vous avez un état  $|\psi\rangle = \frac{3}{5}|\ket{1}\rangle + \frac{4}{5}|\ket{0}\rangle$ , alors, comme  $\langle 1 | 0 \rangle = 0$ , la probabilité de mesure de 1 est

$$|\langle 1 | \psi \rangle|^2 = \left| \frac{3}{5} \langle 1 | 1 \rangle + \frac{4}{5} \langle 1 | 0 \rangle \right|^2 = \frac{9}{25}.$$

## Notation du produit tensoriel

La notation Dirac est très utile pour exprimer le produit tensoriel. Le produit Tensor est important dans l'informatique quantique, car le vecteur d'état décrit par deux registres quantiques non corrélés est les produits tensoriels des deux vecteurs d'état.

Le produit tensoriel  $\psi \otimes \phi$  pour deux vecteurs d'état quantique  $\phi$  et  $\psi$  est écrit en notation Dirac comme  $|\psi\rangle \otimes |\phi\rangle$ . Par convention, vous pouvez également écrire le produit tensoriel comme  $|\psi\rangle |\phi\rangle = |\psi\phi\rangle$ .

Par exemple, l'état avec deux qubits initialisés à l'état zéro est  $|\ket{0}\rangle \otimes |\ket{0}\rangle = |\ket{00}\rangle$ .

## Exemple : Décrire la superposition avec la notation de Dirac

Comme autre exemple de la façon dont vous pouvez utiliser la notation Dirac pour décrire un état quantique, considérez les méthodes équivalentes suivantes d'écriture d'un état quantique qui est une superposition **égale** sur chaque chaîne de bits de longueur  $n$  possible

$$H^{\otimes n} |0\rangle = \frac{1}{2^{n/2}} \sum_{j=0}^{2^n-1} |j\rangle = |+\rangle^{\otimes n}.$$

Vous vous demandez peut-être pourquoi la somme porte sur une plage de 0 à  $2^n - 1$  pour  $n$  bits. Notez tout d'abord que  $n$  bits peuvent prendre  $2^n$  configurations différentes. Vous pouvez le constater en notant qu'un bit peut prendre 2 valeurs, mais que deux bits peuvent prendre 4 valeurs et ainsi de suite. En général, ceci signifie qu'il y a  $2^n$  chaînes de bits différentes possibles. Cependant, la plus grande valeur codée parmi ces chaînes  $1 \cdots 1 = 2^n - 1$ . C'est donc la limite supérieure de la somme. Remarquez que dans cet exemple, vous n'avez pas utilisé  $|+\rangle^{\otimes n} = |+\rangle$  par analogie à  $|\ket{0}^{\otimes n}\rangle = |\ket{0}\rangle$ . Cette convention notationnelle est réservée à l'état de



base de calcul avec chaque qubit initialisé à zéro. Bien qu'une telle convention soit sensible dans ce cas, elle n'est pas employée dans la littérature informatique quantique.

## Exprimer la linéarité avec la notation de Dirac

Une autre caractéristique de la notation de Dirac est sa linéarité. Par exemple, pour deux nombres complexes  $\alpha$  et  $\beta$ , vous pouvez écrire

$$|\psi\rangle \otimes (\alpha |\phi\rangle + \beta |\chi\rangle) = \alpha |\psi\rangle |\phi\rangle + \beta |\psi\rangle |\chi\rangle.$$

Autrement dit, vous pouvez distribuer la notation du produit tensoriel sous forme de notation de Dirac. Ainsi, quand vous prenez les produits tensoriels entre les vecteurs d'état, cela se présente finalement comme une multiplication classique.

Les vecteurs bra suivent une convention similaire à celle des vecteurs ket. Par exemple, le vecteur  $\langle\psi| \langle\phi|$  est équivalent au vecteur d'état  $\psi^\dagger \otimes \phi^\dagger = (\psi \otimes \phi)^\dagger$ . Si le vecteur ket  $|\psi\rangle$  est  $\alpha |0\rangle + \beta |1\rangle$ , alors la version bra du vecteur est  $\langle\psi| = \langle\psi|^\dagger = \langle\alpha|0\rangle + \langle\beta|1\rangle$ .

Par exemple, imaginons que vous souhaitiez calculer la probabilité de mesurer l'état  $|\psi\rangle = \frac{3}{5} |1\rangle + \frac{4}{5} |0\rangle$

*à l'aide d'un programme quantique pour mesurer des états pouvant être  $|+\rangle$  ou  $|-\rangle$ . La probabilité que l'appareil indique comme sorti l'état  $|-\rangle$  est*

$$\begin{aligned} \langle\psi| \langle\psi| = \left( \frac{1}{\sqrt{2}} \langle\alpha| - \frac{1}{\sqrt{2}} \langle\beta| \right) \left( \frac{3}{5} |1\rangle + \frac{4}{5} |0\rangle \right) \\ \left( \frac{3}{5\sqrt{2}} - \frac{4}{5\sqrt{2}} \right) = \frac{1}{50} \end{aligned}$$

La présence du signe négatif dans le calcul de probabilité est une manifestation de l'interférence quantique, qui est l'un des mécanismes par lequel l'informatique quantique prend l'avantage sur l'informatique classique.

## Produit ket-bra ou extérieur

Le dernier élément qu'il convient d'aborder concernant la notation de Dirac est le produit *ket-bra* ou extérieur. Le produit externe est représenté dans les notations Dirac comme  $|\psi\rangle \langle\phi|$ . Le produit extérieur est défini par la multiplication des matrices sous la forme  $|\psi\rangle \langle\phi| = \psi \phi^\dagger$  pour les vecteurs d'état quantique  $\psi$  et  $\phi$ . L'exemple le plus simple et certainement le plus courant de cette notation est :

$$|0\rangle \langle 0| =$$

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & ; 0 \end{bmatrix}$$

=

$$\begin{bmatrix} 1 & ; 0 \\ 0 & ; 0 \end{bmatrix}$$

$$\quad \ket{1}\bra{1} =$$

$$\begin{bmatrix} \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & ; 1 \end{bmatrix}$$

=

$$\begin{bmatrix} 0 & ; 0 \\ 0 & ; 1 \end{bmatrix}$$

. \$\$

Les ket-bra sont souvent appelés projecteurs, car ils projettent un état quantique sur une valeur fixe. Comme ces opérations ne sont pas unitaires (et ne préservent même pas la norme d'un vecteur), un ordinateur quantique ne peut pas appliquer un projecteur de façon déterministe. Toutefois, les projecteurs décrivent de façon très efficace l'action de la mesure sur un état quantique. Par exemple, si vous mesurez un état  $|\psi\rangle$  comme 0, la transformation de l'état qui en résulte est

$$\ket{\psi} \rightarrow \frac{(\ket{0}\bra{0})\ket{\psi}}{\langle 0 | \psi \rangle} = \ket{0},$$

comme vous l'attendiez si vous avez mesuré l'état et trouvé qu'il était  $|0\rangle$ . Rappelons qu'un ordinateur quantique ne peut pas appliquer ces projecteurs sur un état de manière déterministe. En fait, ils peuvent, au mieux, être appliqués de façon aléatoire avec le résultat  $|0\rangle$  apparaissant avec une probabilité fixe. La probabilité de réussite d'une telle mesure peut être écrite comme valeur de prédiction du projecteur quantique dans l'état

$$\langle \psi | (|0\rangle \langle 0|) | \psi \rangle = |\langle \psi | 0 \rangle|^2,$$

ce qui montre que les projecteurs offrent un nouveau moyen d'exprimer le processus de mesure.

Si vous envisagez plutôt de mesurer le premier qubit d'un état à plusieurs qubits comme 1, vous pouvez également décrire ce processus facilement à l'aide de

projecteurs et de la notation de Dirac :

$$P(\text{first qubit} = 1) = |\psi\rangle\langle\psi| \otimes \mathbb{I} \\ \text{where } \mathbb{I} = \sum_{i=0}^1 |i\rangle\langle i|$$

Ici, la matrice d'identité peut être écrite facilement en notation de Dirac ainsi :

$$\mathbb{I} = |0\rangle\langle 0| + |1\rangle\langle 1| =$$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

.

En présence de deux qubits, le projecteur peut être développé sous la forme suivante :

$$|\psi\rangle\langle\psi| \otimes \mathbb{I} = |\psi\rangle\langle\psi| \otimes (|0\rangle\langle 0| + |1\rangle\langle 1|) = |\psi 0\rangle\langle\psi 0| + |\psi 1\rangle\langle\psi 1|$$

Vous constatez ici la cohérence avec la discussion sur les probabilités de mesure pour les états à plusieurs qubits avec la notation en vecteur colonne :

$$P(\text{premier qubit} = 1) = \langle\psi| (|0\rangle\langle 0| + |1\rangle\langle 1|) |\psi\rangle = \langle\psi| |0\rangle\langle 0| |\psi\rangle + \langle\psi| |1\rangle\langle 1| |\psi\rangle = |e_0|^2 + |e_1|^2$$

ce qui correspond à la discussion sur la mesure avec plusieurs qubits. Cependant, la généralisation de ce résultat au cas impliquant plusieurs qubits est légèrement plus simple à exprimer avec la notation de Dirac qu'avec la notation en vecteur colonne et est totalement équivalente au traitement précédent.

## Opérateurs de densité

La notation de Dirac permet d'exprimer un autre opérateur utile : l'*opérateur de densité*, parfois appelé *opérateur d'état*. En tant que vecteur d'état quantique, l'opérateur de densité décrit l'état quantique d'un système. Bien que les vecteurs d'état quantique ne puissent représenter *que des états purs*, les opérateurs de densité peuvent également représenter *des états mixtes*.

Plus généralement, une matrice  $\rho$  donnée est un opérateur de densité valide si les conditions suivantes sont remplies :

- $\rho$  est une matrice de nombres complexes
- $\rho = \rho^\dagger$  (autrement dit,  $\rho$  est hermitien)
- Chaque valeur propre de  $\rho$  n'est pas négative
- La somme de toutes les valeurs propres de  $\rho$  est égale à 1

Ensemble, ces conditions garantissent que  $\rho$  peut être considéré comme un ensemble.

Un opérateur de densité pour un vecteur d'état quantique  $|\psi\rangle$  prend la forme  $\rho = \sum_i p_i |\psi_i\rangle \langle \psi_i|$  est une décomposition de valeur propre de  $\rho$ , puis  $\rho$  décrit l'ensemble  $\rho = |\psi_i\rangle$  avec la probabilité  $p_i$ .

Les états quantiques purs sont ceux qui sont caractérisés par un seul vecteur ket ou wavefunction et ne peuvent pas être écrits en tant que mélange statistique (ou *combinaison convexe*) d'autres états quantiques. Un état quantique mixte est un ensemble statistique d'états purs.

Sur une sphère Bloch, les états purs sont représentés par un point sur la surface de la sphère, tandis que les états mixtes sont représentés par un point intérieur. L'état mixte d'un qubit unique est représenté par le centre de la sphère, par symétrie. La pureté d'un état peut être visualisé comme le degré dans lequel il est proche de la surface de la sphère.

Ce concept de représentation de l'état comme matrice, plutôt qu'un vecteur, est souvent pratique, car il donne un moyen pratique de représenter les calculs de probabilité, et vous permet également de décrire à la fois l'incertitude statistique et l'incertitude quantique dans le même formalisme.

Un opérateur de densité  $\rho$  représente un état pur si et seulement si :

- $\rho$  peut être écrit comme un produit externe d'un vecteur d'état,  $\rho = |\psi\rangle \langle \psi|$
- $\rho = \rho^2$
- $\text{tr}(\rho^2) = 1$

Pour déterminer la façon dont la fermeture d'un opérateur de densité  $\rho$  donné est pure, vous pouvez examiner la trace (autrement dit, la somme des éléments diagonales) de  $\rho^2$ . Un opérateur de densité représente un état pur si et seulement si  $\text{tr}(\rho^2) = 1$ .

## Séquences de portes Q# équivalentes aux états quantiques

Abordons un dernier point important concernant la notation quantique et le langage de programmation Q# : au début de ce document, nous avons mentionné que l'état quantique est l'objet d'information fondamental en informatique quantique. L'absence de notion d'état quantique dans le langage Q# peut donc sembler étonnante. En fait, tous les états sont décrits uniquement par les opérations utilisées pour les préparer. L'exemple précédent en offre une excellente illustration. Au lieu d'exprimer une superposition uniforme sur chaque chaîne de bits quantiques dans un registre, vous pouvez représenter le résultat sous la forme  $H^{\otimes n} |0\rangle$ . Grâce à cette description

exponentiellement plus courte de l'état, vous pouvez en avoir un raisonnement classique. De plus, elle définit de manière concise les opérations qui doivent être propagées par l'intermédiaire de la pile logicielle pour implémenter l'algorithme. Pour cette raison, Q# est conçu pour émettre des séquences de portes plutôt que des états quantiques. Toutefois, à un niveau théorique, les deux perspectives sont équivalentes.

## Rubriques connexes

- [Mesures Pauli](#)
- [Portes T et usines T](#)
- [Circuits quantiques](#)
- [Oracles quantiques](#)

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Mesures Pauli à qubit unique et multi-qubit

Article • 16/09/2024

À mesure que vous travaillez avec Q#, vous constatez que *les mesures Pauli* sont un type commun de mesure. Les mesures Pauli généralisent les mesures de base de calcul afin d'inclure des mesures dans d'autres bases et de parité entre différents qubits. Dans de tels cas, il est courant de discuter de la mesure d'un opérateur Pauli, qui est un opérateur tel que  $X$ ,  $Y$ ,  $Z$  ou  $Z \otimes X$ ,  $X \otimes X$ ,  $X \otimes Y$ , etc. Pour connaître les principes de base de la mesure quantique, consultez [les qubits](#) et [les qubits multiples](#).

La discussion de la mesure en termes d'opérateurs Pauli est courante dans le sous-champ de correction d'erreur quantique.

Le guide Q# suit une convention similaire. Cet article explique cette autre vision des mesures.

## 💡 Conseil

Dans Q#, les opérateurs Pauli multi-qubit sont généralement représentés par des tableaux de type `Pauli[]`. Par exemple, pour représenter  $X \otimes Z \otimes Y$ , vous pouvez utiliser le tableau `[PauliX, PauliZ, PauliY]`.

Avant d'examiner en détail comment considérer une mesure de Pauli, il est utile de réfléchir à l'effet que produit sur l'état quantique le fait de mesurer un qubit unique dans un ordinateur quantique. Prenons un état quantique à  $n$  qubits. Mesurer un qubit exclut immédiatement la moitié des  $2^n$  possibilités de l'état. En d'autres termes, la mesure projette l'état quantique sur un demi-espace. Vous pouvez généraliser la façon dont vous réfléchissez à la mesure pour refléter cette intuition.

Pour identifier ces sous-espaces de manière concise, il nous faut un langage permettant de les décrire. Nous pouvons par exemple spécifier les deux sous-espaces à l'aide d'une matrice possédant seulement deux valeurs propres uniques, par convention  $\pm 1$ .

Prenons un exemple simple pour décrire les sous-espaces de cette manière avec  $Z$  :

$$Z =$$

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$\end{bmatrix}$$

En lisant les éléments en diagonale de la matrice  $Z$  de Pauli, nous voyons que  $Z$  a deux vecteurs propres,  $|0\rangle$  et  $|1\rangle$ , avec les valeurs propres  $\pm 1$  correspondantes. Ainsi, si une mesure du qubit aboutit `Zero` (correspondant à l'état  $|0\rangle$ ), il est connu que l'état du qubit est un *état propre*  $+1$  de l'opérateur  $Z$ . De même, si le résultat est `One`, il est connu que l'état du qubit est un *état propre*  $-1$  de  $Z$ . Ce processus, appelé dans le langage des mesures de Pauli "mesure de Pauli  $Z$ ", est parfaitement équivalent à une mesure de base computationnelle.

Toute matrice  $2 \times 2$  qui est une transformation unitaire de  $Z$  respecte également ce critère. Autrement dit, nous pourrions aussi utiliser une matrice  $A = U^\dagger Z U$ , où  $U$  est une autre matrice unitaire, pour fournir une matrice qui définit les deux résultats d'une mesure dans ses vecteurs propres  $\pm 1$ . La notation des mesures de Pauli fait référence à cette équivalence unitaire en identifiant les mesures  $X, Y, Z$  comme des mesures équivalentes qu'il est possible d'effectuer pour obtenir des informations à partir d'un qubit. Ces mesures sont fournies ici pour des raisons pratiques.

[🔍 Agrandir le tableau](#)

Mesures de Pauli	Transformation unitaire
$Z$	$I$
$X$	$H$
$Y$	$HS^\dagger$

Autrement dit, en utilisant cette langue, `&entre guillemets ; mesure Y` ; équivaut à appliquer  $HS^\dagger$  puis à mesurer dans la base de calcul, où `S` est une opération quantique intrinsèque parfois appelée `quot & ; porte de phase,& quot` ; et peut être simulé à l'aide de la matrice unitaire

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}.$$

Cela équivaut également à appliquer  $HS^\dagger$  au vecteur d'état quantique, puis à mesurer  $Z$ . Ainsi, l'opération suivante est équivalente à `Measure([PauliY], [q])` :

```
Q#
operation MeasureY(qubit : Qubit) : Result {
 mutable result = Zero;
 within {
 Adjoint S(q);
 H(q);
 } apply {
 set result = M(q);
 }
}
```

```

 }
 return result;
}

```

L'état correct est ensuite trouvé en revenant à la base de calcul, qui équivaut à appliquer  $sh$  au vecteur d'état quantique ; dans l'extrait de code, la transformation vers la base de calcul est gérée automatiquement avec l'utilisation du `within ... apply` bloc.

Dans Q#, le résultat---that est que les informations classiques extraites de l'interaction avec l'état---is donnés à l'aide d'une `Result` valeur  $j \in \{\text{Zero}, \text{One}\}$  indiquant si le résultat se trouve dans l'espace  $proprie(-1)^j$  propre de l'opérateur Pauli mesuré.

## Mesures à plusieurs qubits

Les mesures des opérateurs Pauli à plusieurs qubits sont définies de la même manière, comme le montre l'exemple suivant :

$$ZZ \otimes = \begin{bmatrix} 1 & ; 0 & ; 0 & ; 00 \\ & amp; -1 & ; 0 & ; 0 \\ 0 & ; 0 & ; -1 & ; 0 \\ 0 & ; 0 & ; 0 & ; 1 \end{bmatrix}.$$

Ainsi, le produit tensoriel de deux opérateurs Pauli  $Z$  forme une matrice composée de deux espaces contenant des valeurs propres  $+1$  et  $-1$ . Comme dans le cas à un qubit, les deux constituent un demi-espace, ce qui signifie que la moitié de l'espace vectoriel accessible appartient à l'espace propre  $+1$  et l'autre à l'espace propre  $-1$ . En général, il est facile de voir, à partir de la définition du produit tensoriel, que tout produit tensoriel d'opérateurs Pauli  $Z$  et l'identité obéissent également à cette règle. Par exemple,

$$\begin{aligned} Z \otimes I &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \\ I \otimes Z &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \end{aligned}$$

Comme précédemment, toute transformation unitaire de ces matrices décrit également deux demi-espaces étiquetés avec *des valeurs propres*  $\pm 1$ . par exemple,  $X \otimes X = H \otimes H (Z \otimes Z) H \otimes H$  à partir de l'identité  $Z = HXH$ . Comme pour le cas un qubit, toutes les mesures pauli-à-deux qubits peuvent être écrites sous la forme  $U^\dagger (Z \otimes 1) U$  pour  $4 \times 4$  matrices unitaires  $U$ . Les transformations sont énumérées dans le tableau suivant.



Dans cette table,  $\begin{matrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{matrix}$  utilisée pour simuler l'opération SWAP intrinsèque.

[Agrandir le tableau](#)

Mesures de Pauli	Transformation unitaire
$Z \otimes \mathbf{1}$	$\mathbf{1} \otimes \mathbf{1}$
$X \otimes \mathbf{1}$	$H \otimes \mathbf{1}$
$Y \otimes \mathbf{1}$	$HS^\dagger \otimes \mathbf{1}$
$\mathbf{1} \otimes Z$	SWAP
$\mathbf{1} \otimes X$	$(H \otimes \mathbf{1})$ SWAP
$\mathbf{1} \otimes Y$	$(HS^\dagger \otimes \mathbf{1})$ SWAP
$Z \otimes Z$	CNOT <sub>10</sub>
$X \otimes Z$	CNOT <sub>10</sub> ( $H \otimes \mathbf{1}$ )
$Y \otimes Z$	CNOT <sub>10</sub> ( $HS^\dagger \otimes \mathbf{1}$ )
$Z \otimes X$	CNOT <sub>10</sub> ( $\mathbf{1} \otimes H$ )
$X \otimes X$	CNOT <sub>10</sub> ( $H \otimes H$ )
$Y \otimes X$	CNOT <sub>10</sub> ( $HS^\dagger \otimes H$ )
$Z \otimes Y$	CNOT <sub>10</sub> ( $\mathbf{1} \otimes HS^\dagger$ )
$X \otimes Y$	CNOT <sub>10</sub> ( $H \otimes HS^\dagger$ )
$Y \otimes Y$	CNOT <sub>10</sub> ( $HS^\dagger \otimes HS^\dagger$ )

Ici, l'opération CNOT apparaît pour la raison suivante. Chaque mesure Pauli qui n'inclut pas la  $\mathbf{1}$  matrice équivaut à une unité à  $Z \otimes Z$  par le raisonnement précédent. Les valeurs propres de  $Z \otimes Z$  dépendent uniquement de la parité des qubits qui composent chaque vecteur de base computationnelle. Les opérations CNOT servent à calculer cette parité et à la stocker dans le premier bit. Une fois le premier bit mesuré, nous pouvons récupérer l'identité du demi-espace résultant, ce qui équivaut à mesurer l'opérateur Pauli.

En outre, même s'il peut être tentant de supposer que la mesure  $de Z \otimes Z$  est identique à la mesure *séquentielle*  $de Z \otimes 1$ , puis  $1 \otimes Z$ , cette hypothèse serait fausse. En effet, mesurer  $Z \otimes Z$  projette l'état quantique dans l'état propre  $+1$  ou  $-1$  de ces opérateurs. Mesurer  $Z \otimes 1$  puis  $1 \otimes Z$  projette d'abord le vecteur d'état quantique sur un demi-espace de  $Z \otimes \mathbb{C}^1$  puis sur un demi-espace de  $\mathbb{C}^1 \otimes Z$ . Étant donné qu'il y a quatre vecteurs de base computationnelle, les deux mesures permettent de réduire l'état à un quart d'espace et, par conséquent, à un seul vecteur de base computationnelle.

## Corrélations entre qubits

Une autre façon d'examiner la mesure des produits tensoriels de matrices de Pauli (par exemple  $X \otimes X$  ou  $Z \otimes Z$ ) consiste à considérer que ces mesures permettent de consulter les informations stockées dans les corrélations entre les deux qubits. La mesure  $de X \otimes 1$  vous permet d'examiner les informations stockées localement dans le premier qubit. Si les deux types de mesures sont tout aussi utiles en informatique quantique, c'est le premier qui met en évidence toute sa puissance. Il révèle que, dans ce domaine, les informations que l'on souhaite connaître ne sont pas stockées dans un qubit unique, mais plutôt stockées non localement dans toutes les qubits en même temps. Par conséquent, elles ne se manifestent que si on les examine par une mesure conjointe (par exemple  $Z \otimes Z$ ).

Des opérateurs Pauli arbitraires tels que  $X \otimes Y \otimes Z \otimes 1$  peuvent également être mesurés. Les produits tensoriels des opérateurs Pauli n'ont que deux valeurs propres  $\pm 1$ . Les deux espaces propres constituent des demi-espaces de l'ensemble de l'espace vectoriel. Ainsi, ils coïncident avec les exigences indiquées précédemment.

En  $Q^\#$ , ces mesures retournent  $j$  si la mesure produit un résultat dans l'espace propre du signe  $(-1)^j$ . L'utilisation de mesures Pauli en tant que caractéristique  $Q^\#$  intégrée est utile, car la mesure de ces opérateurs nécessite de longues chaînes de portes contrôlées et de transformations de base pour décrire la porte *Udiagonal* nécessaire pour exprimer l'opération en tant que produit tensoriel de  $Z$  et  $1$ . Vous pouvez ainsi spécifier que vous souhaitez effectuer l'une de ces mesures prédéfinies, ce qui vous évite d'avoir à chercher comment transformer votre base de sorte qu'une mesure de base computationnelle fournisse les informations nécessaires.  $Q^\#$  gère automatiquement toutes les transformations de la base nécessaires.

## Le théorème d'impossibilité du clonage quantique

Les informations quantiques sont puissantes. Il vous permet d'effectuer des choses étonnantes telles que les nombres de facteurs exponentiellement plus rapidement que les algorithmes classiques les plus connus, ou de simuler efficacement des systèmes d'électrons corrélés qui nécessitent classiquement un coût exponentiel pour simuler avec précision. Toutefois, la puissance de l'informatique quantique présente quelques limitations, parmi lesquelles figure le *théorème d'impossibilité du clonage quantique*.

Le théorème d'impossibilité du clonage quantique porte bien son nom. Il interdit le clonage d'états quantiques génériques par un ordinateur quantique. La preuve du théorème est remarquablement simple. Bien qu'une preuve complète du théorème sans clonage soit trop technique pour cet article, la preuve dans le cas d'aucun qubits auxiliaire supplémentaire n'est dans la portée.

Pour un tel ordinateur quantique, l'opération de clonage doit être décrite avec une matrice unitaire. La mesure quantique n'est pas autorisée, car elle entraînerait la corruption de l'état quantique à cloner. Pour simuler l'opération de clonage, la matrice unitaire utilisée doit avoir la propriété selon laquelle  $U$

$U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$  pour n'importe quel état  $|\psi\rangle$ . La propriété de linéarité de la multiplication de matrices implique alors que, pour n'importe quel autre état quantique  $|\phi\rangle$ ,

$$\begin{aligned} U \left[ \frac{1}{\sqrt{2}} (\ket{\phi} + \ket{\psi}) \right] |0\rangle &= \frac{1}{\sqrt{2}} U \ket{\phi} |0\rangle + \frac{1}{\sqrt{2}} U \ket{\psi} |0\rangle \\ &= \frac{1}{\sqrt{2}} (\ket{\phi} \ket{\phi} + \ket{\psi} \ket{\psi}) \end{aligned}$$

Cela nous donne l'intuition fondamentale qui sous-tend le principe d'impossibilité du clonage : tout appareil qui copie un état quantique inconnu doit provoquer des erreurs dans au moins une partie des états qu'il copie. L'hypothèse clé selon laquelle le cloneur agit de manière linéaire sur l'état d'entrée peut être violée par l'ajout et la mesure des qubits auxiliaires. Cependant, ces interactions révèlent également des informations sur le système dans les statistiques de mesure et empêchent le clonage exact dans ces cas aussi.

Le théorème d'impossibilité du clonage quantique est important pour une compréhension qualitative de l'informatique quantique. Si l'on pouvait cloner des états quantiques sans coût, cela donnerait le pouvoir quasiment magique d'apprendre des états quantiques. Il serait en effet possible de violer le fameux principe d'incertitude de Heisenberg. Vous pourriez également utiliser un cloneur optimal pour découvrir un maximum de choses sur une distribution quantique complexe à partir d'un seul exemple. C'est comme si vous retournez une pièce et observez les têtes, puis en disant à

un ami le résultat de leur réponse &entre guillemets ; Ah la distribution de cette pièce doit être Bernoulli avec  $p = 0,512643 \dots$  !&Quot; Une telle déclaration ne serait pas absurde, car un peu d'informations (résultat des têtes) ne peut simplement pas fournir les nombreux bits d'informations nécessaires pour encoder la distribution sans informations préalables substantielles. De même que nous ne pouvons pas préparer un ensemble de ces pièces sans connaître  $p$ , nous ne pouvons pas cloner parfaitement un état quantique sans informations préalables.

En informatique quantique, les informations ne sont pas gratuites. Chaque qubit mesuré fournit une seule information. Le principe d'impossibilité du clonage indique qu'il n'existe pas de porte dérobée permettant de contourner le compromis fondamental entre les informations acquises sur le système et la perturbation provoquée.

## Contenu connexe

- [Portes T et usines T](#)
- [Circuits quantiques](#)
- [Oracles quantiques](#)
- [Algorithme de Grover](#)

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Théorie de l'algorithme de recherche de Grover

Article • 19/01/2025

Dans cet article, vous trouverez une explication théorique détaillée des principes mathématiques qui décrivent le fonctionnement de l'algorithme de Grover.

Pour une implémentation pratique de l'algorithme de Grover pour résoudre les problèmes mathématiques, consultez [l'algorithme](#) de recherche de Grover.

## Énoncé du problème

L'algorithme de Grover accélère la solution aux recherches de données non structurées (ou *problème* de recherche), en exécutant la recherche en moins d'étapes que n'importe quel algorithme classique. Toute tâche de recherche peut être exprimée à l'aide d'une fonction abstraite  $f(x)$  qui accepte des éléments de recherche  $x$ . Si l'élément  $x$  est une solution à la tâche de recherche, alors  $f(x) = 1$ . Si l'élément  $x$  n'est pas une solution, alors  $f(x) = 0$ . Le *problème de recherche* consiste à trouver tout élément  $x_0$  tel que  $f(x_0) = 1$ .

Tout problème qui vous permet de vérifier si une valeur donnée  $x$  est une solution valide (un &problème oui ou non&) peut être formulé en termes de problème de recherche. Voici quelques exemples :

- Problème de satisfiabilité booléenne : Étant donné un ensemble de valeurs booléennes, le jeu satisfait-il à une formule booléenne donnée ?
- Problème de vendeur itinérant : Quelle est la boucle la plus courte possible qui connecte toutes les villes ?
- Problème de recherche de base de données : une table de base de données contient-elle l'enregistrement  $x$  ?
- Problème de factorisation des entiers : le nombre fixe  $N$  est-il divisible par le nombre  $x$  ?

La tâche que l'algorithme de Grover vise à résoudre peut être exprimée comme suit : étant donné une fonction classique  $f(x) : 0, 1^n \rightarrow \{0, 1\}$ , où  $n$  est la taille de l'espace de recherche, recherchez une entrée  $x_0$  pour laquelle  $f(x_0) = 1$ . La complexité de l'algorithme est mesurée par le nombre d'utilisations de la fonction  $f(x)$ .

Traditionnellement, dans le pire des cas, nous devons évaluer  $f(x)$  un total de  $N - 1$  fois, où  $N = 2^n$ , en essayant toutes les possibilités. Après  $N - 1$  éléments, il doit s'agir du dernier élément. L'algorithme quantique de Grover peut résoudre ce problème

beaucoup plus rapidement, en offrant une accélération quadratique. Quadratique ici implique que seules  $\sqrt{\text{évaluations}N}$  environ sont requises, par rapport à  $N$ .

## Structure de l'algorithme

Supposons que nous avons  $N = 2^n$  éléments éligibles pour la tâche de recherche et que nous les indexons en affectant à chaque élément un entier compris entre 0 et  $N - 1$ . Par ailleurs, supposons qu'il y a  $M$  entrées valides différentes, ce qui signifie qu'il y a  $M$  entrées pour lesquelles  $f(x) = 1$ . Les étapes de l'algorithme sont les suivantes :

1. Commencez avec un registre de  $n$  qubits initié dans l'état  $|0\rangle$ .
2. Préparez le registre en une superposition uniforme en appliquant  $H$  à chaque qubit du registre :

$$|\text{register}\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

3. Appliquez les opérations suivantes au registre  $N_{\text{optimal}}$  un certain nombre de fois :
  - a. La phase Oracle  $O_f$  qui applique un décalage de phase conditionnel de  $-1$  pour les éléments de la solution.
  - b. Appliquez  $H$  à chaque qubit dans le registre.
  - c. Un décalage de phase conditionnel de  $-1$  à chaque état de base de calcul, sauf  $|0\rangle$ . Cela peut être représenté par l'opération unitaire  $-O_0$ , car  $O_0$  représente le décalage de phase conditionnel sur  $|0\rangle$  uniquement.
  - d. Appliquez  $H$  à chaque qubit dans le registre.
4. Mesurez le registre pour obtenir l'index d'un élément qui est une solution avec une très grande probabilité.
5. Vérifier s'il s'agit d'une solution valide. Si ce n'est pas le cas, recommencer.

$N_{\text{optimal}} = \left\lceil \left\lfloor \frac{\pi}{4} \sqrt{\frac{N}{M}} - \frac{1}{2} \right\rfloor \right\rceil$   
est le nombre optimal d'itérations qui optimise la probabilité d'obtention de l'élément correct en mesurant le registre.

### 📌 Notes

L'application conjointe des étapes 3.b, 3.c et 3.d est généralement appelée opérateur de diffusion *Grover*.

L'opération d'unité globale appliquée au registre est la suivante :

$$(-H^{\otimes n} O_0 H^{\otimes n} O_f)^{N_{\text{optimal}}} H^{\otimes n}$$

# Suivre l'état du registre, étape par étape

Pour illustrer le processus, nous allons suivre les transformations mathématiques de l'état du registre pour un cas simple dans lequel il y a uniquement deux qubits et où l'élément valide est  $|01\rangle$ .

1. Le registre démarre dans l'état :

$$|\text{registre}\rangle = |00\rangle$$

2. Après avoir appliqué  $H$  à chaque qubit, les transformations d'état du registre sont les suivantes :

$$|\text{registre}\rangle = \frac{1}{\sqrt{4}} \sum_{i \in \{0,1\}} |i\rangle = \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

3. Ensuite, l'oracle de phase est appliqué pour obtenir :

$$|\text{registre}\rangle = \frac{1}{2} (|00\rangle - |01\rangle + |10\rangle + |11\rangle)$$

4. Ensuite,  $H$  agit à nouveau sur chaque qubit pour donner :

$$|\text{registre}\rangle = \frac{1}{2} (|00\rangle + |01\rangle - |10\rangle + |11\rangle)$$

5. À présent, le décalage de phase conditionnelle est appliqué à chaque état, sauf  $|00\rangle$ :

$$|\text{registre}\rangle = \frac{1}{2} (|00\rangle - |01\rangle + |10\rangle - |11\rangle)$$

6. Enfin, la première itération Grover se termine par l'application de  $H$  à nouveau pour obtenir :

$$|\text{registre}\rangle = |01\rangle$$

En suivant les étapes ci-dessus, l'élément valide est trouvé en une seule itération.

Comme vous le verrez plus tard, c'est parce que pour  $N=4$  et un seul élément valide, le nombre optimal de fois est  $N_{\text{optimal}} = 1$ .

## Explication géométrique

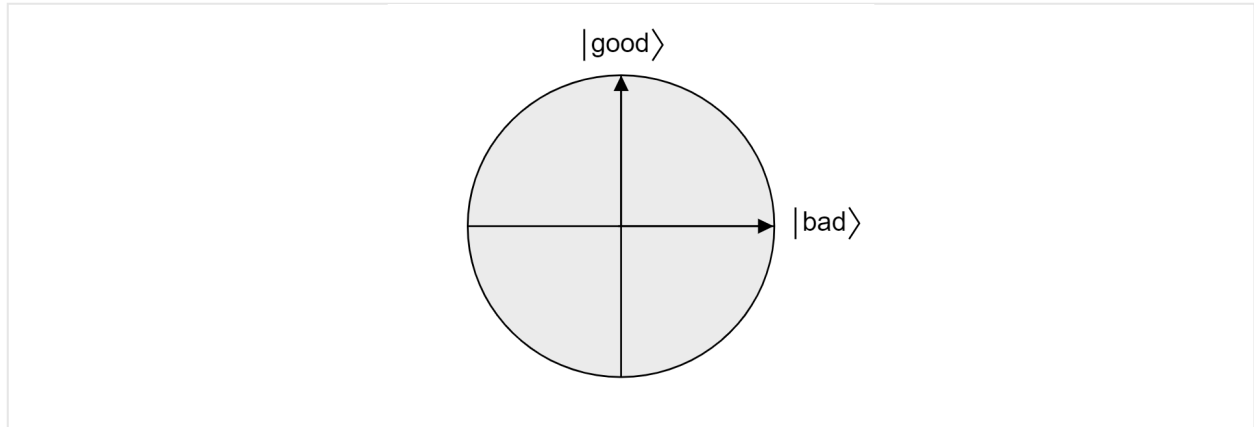
Pour comprendre le fonctionnement de l'algorithme de Grover, nous allons étudier l'algorithme d'un point de vue géométrique. En supposant qu'il y ait  $M$  solutions valides, la superposition de tous les états quantiques qui ne sont *pas* une solution au problème de recherche :

$$|\text{bad}\rangle = \frac{1}{\sqrt{N-M}} \sum_{x:f(x)=0} |x\rangle$$

La superposition de tous les états qui *sont* une solution au problème de recherche :

$$|\text{good}\rangle = \frac{1}{\sqrt{M}} \sum_{x:f(x)=1} |x\rangle$$

Étant donné que *bon* et *mauvais* sont des ensembles mutuellement exclusifs, puisqu'un élément ne peut être à la fois valide et non valide, les états sont orthogonaux. Les deux états forment la base orthogonale d'un plan dans l'espace de vectoriel. Nous pouvons utiliser ce plan pour visualiser l'algorithme.



Prenons maintenant  $|\psi\rangle$ , un état arbitraire qui réside dans le plan couvert par  $|\text{good}\rangle$  et  $|\text{bad}\rangle$ . Tout état vivant dans ce plan peut être exprimé comme suit :

$$|\psi\rangle = \alpha |\text{good}\rangle + \beta |\text{bad}\rangle$$

où  $\alpha$  et  $\beta$  sont des nombres réels. À présent, nous allons introduire l'opérateur de réflexion  $R_{|\psi\rangle}$ , où  $|\psi\rangle$  est tout état qubit vivant dans le plan. L'opérateur est défini ainsi :

$$R_{|\psi\rangle} = 2 |\psi\rangle \langle\psi| - \mathcal{I}$$

On parle d'opérateur de réflexion sur  $|\psi\rangle$ , car il peut être interprété géométriquement comme une réflexion sur la direction de  $|\psi\rangle$ . Pour le voir, prenez la base orthogonale du plan formé par  $|\psi\rangle$  et son complément orthogonal  $|\psi^\perp\rangle$ . Tous état  $|\xi\rangle$  du plan peut être décomposé de la manière suivante :

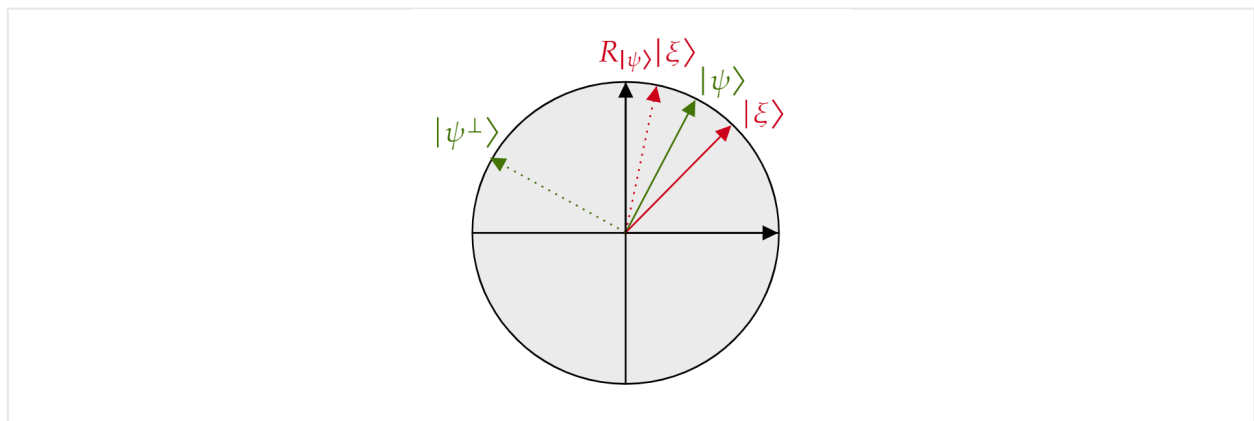
$$|\xi\rangle = \mu |\psi\rangle + \nu |\psi^\perp\rangle$$

Si l'opérateur  $R_{|\psi\rangle}$  est appliqué à  $|\xi\rangle$  :

$$R_{|\psi\rangle} |\xi\rangle = \mu |\psi\rangle - \nu |\psi^\perp\rangle$$

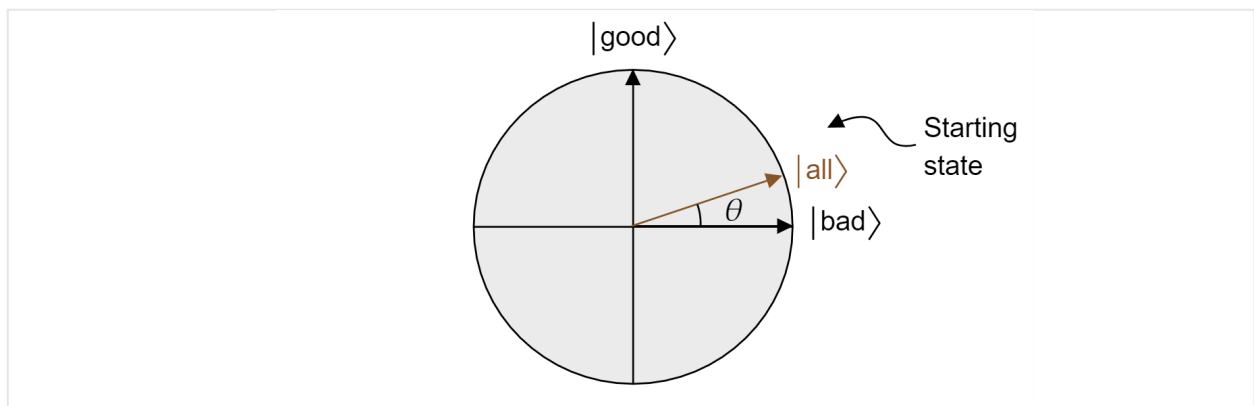
L'opérateur  $R_{|\psi\rangle}$  inverse le composant orthogonal à  $|\psi\rangle$ , mais laisse le composant  $|\psi\rangle$  inchangé. Par conséquent,  $R_{|\psi\rangle}$  est une réflexion sur  $|\psi\rangle$ .





L'algorithme de Grover, après la première application de  $H$  à chaque qubit, commence par une superposition uniforme de tous les états. Cela peut être écrit comme suit :

$$|\text{all}\rangle = \sqrt{\frac{M}{N}} |\text{good}\rangle + \sqrt{\frac{N-M}{N}} |\text{bad}\rangle$$



Par conséquent, l'état vit dans le plan. Notez que la probabilité d'obtenir un résultat correct lors de la mesure à partir de la superposition égale est juste bonne  $|\langle \text{toutes} | \hat{\cdot} \rangle|^2 = M/N$ , c'est-à-dire ce que vous attendez d'une estimation aléatoire.

L'oracle  $O_f$  ajoute une phase négative à toute solution au problème de recherche. Par conséquent, il peut être écrit comme une réflexion sur l'axe  $|\text{bad}\rangle$  :

$$O_f = R_{|\text{bad}\rangle} = 2|\text{bad}\rangle\langle\text{bad}| - \mathbb{I}$$

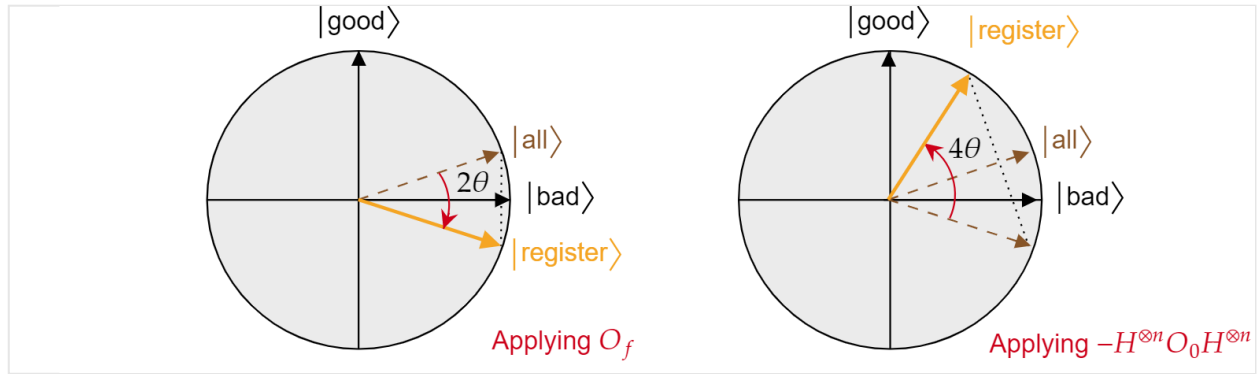
De même, le changement de phase conditionnelle  $O_0$  est simplement une réflexion inversée sur l'état  $|0\rangle$  :

$$O_{|0\rangle} = R_{|\text{ket}\{0\}\rangle} = -2|\text{ket}\{0\}\rangle\langle\text{bra}\{0\}| + \mathbb{I}$$

En connaissant ce fait, il est facile de vérifier que l'opération de diffusion de Grover  $-H^{\otimes n} O_0 H^{\otimes n}$  est également une réflexion sur l'état  $|\text{all}\rangle$ . Il suffit d'effectuer :

$$-H^{\otimes n} O_{|0\rangle} H^{\otimes n} = 2H^{\otimes n} |\text{ket}\{0\}\rangle\langle\text{bra}\{0\}| H^{\otimes n} - H^{\otimes n} \mathbb{I} H^{\otimes n} = 2|\text{ket}\{\text{all}\}\rangle\langle\text{bra}\{\text{all}\}| - \mathbb{I} = R_{|\text{ket}\{\text{all}\}\rangle}$$

Nous venons de démontrer que chaque itération de l'algorithme de Grover est une composition de deux réflexions,  $R_{|\text{bad}\rangle}$  et  $R_{|\text{all}\rangle}$ .



L'effet combiné de chaque itération de Grover est une rotation dans le sens horaire inverse d'un angle  $2\theta$ . Heureusement, l'angle  $\theta$  est facile à trouver. Étant donné que  $\theta$  est juste l'angle entre  $|\text{all}\rangle$  et  $|\text{bad}\rangle$ , nous pouvons utiliser le produit scalaire pour trouver l'angle. Nous savons que  $\cos \theta = \langle \text{all} | \text{bad} \rangle$ , donc nous devons calculer  $\langle \text{all} | \text{bad} \rangle$ . De la décomposition de  $|\text{all}\rangle$  en  $|\text{bad}\rangle$  et  $|\text{good}\rangle$ , nous obtenons ceci :

$$\theta = \arccos(\langle \text{all} | \text{bad} \rangle) = \arccos\left(\sqrt{\frac{N-M}{N}}\right)$$

L'angle entre l'état du registre et le  $|\text{bon}\rangle$  état diminue avec chaque itération, ce qui entraîne une probabilité plus élevée de mesurer un résultat valide. Pour calculer cette probabilité, vous devez simplement calculer  $|\langle \text{le bon} | \text{registre} \rangle|^2$ . L'angle entre le  $|\text{bon}\rangle$  et le  $|\text{registre}\rangle$  est noté  $\gamma(k)$ , où  $k$  est le nombre d'itérations :

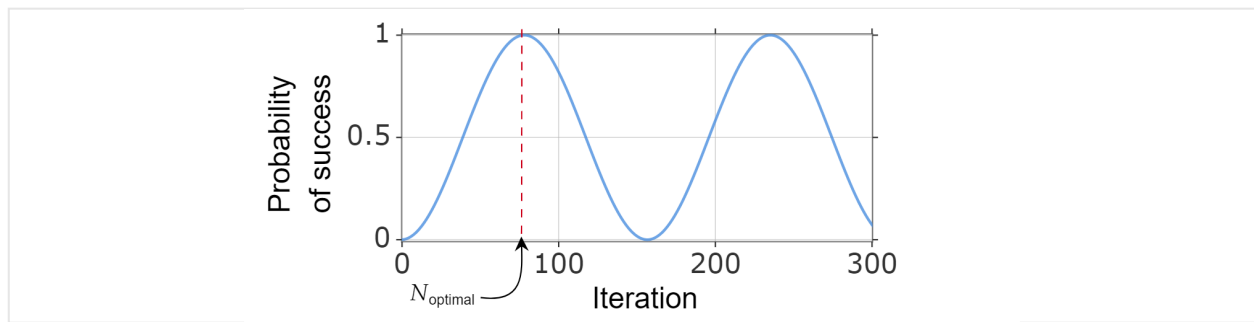
$$\gamma = \frac{\pi}{2} - \theta - 2k\theta = \frac{\pi}{2} - (2k+1)\theta$$

Par conséquent, la probabilité de réussite est :

$$P(\text{success}) = \cos^2(\gamma(k)) = \sin^2\left[(2k+1) \arccos\left(\sqrt{\frac{N-M}{N}}\right)\right]$$

## Nombre optimal d'itérations

Étant donné que la probabilité de réussite peut être écrite en fonction du nombre d'itérations, nous pouvons trouver le nombre optimal d'itérations  $N_{\text{optimal}}$  en calculant le plus petit entier positif (approximativement) qui maximise la fonction de probabilité de réussite.



Nous savons que  $\sin^2 x$  atteint sa première valeur maximale pour  $x = \frac{\pi}{2}$ , donc :

$$\frac{\pi}{2} = (2k_{\text{optimal}} + 1) \arccos \left( \sqrt{\frac{N-M}{N}} \right)$$

Cela donne :

$$k_{\text{optimal}} = \frac{\pi}{4 \arccos \left( \sqrt{1 - M/N} \right)} - 1/2 = \frac{\pi}{4} \sqrt{\frac{N}{M}} - \frac{1}{2} - O \left( \sqrt{\frac{M}{N}} \right)$$

Où, dans la dernière étape,  $\arccos \sqrt{1-x} = \sqrt{x} + O(x^{3/2})$ .

Par conséquent, vous pouvez choisir  $N_{\text{optimal}} = \left\lceil \left\lfloor \frac{\pi}{4} \sqrt{\frac{N}{M}} - \frac{1}{2} \right\rfloor \right\rceil$ .

## Analyse de complexité

À partir de l'analyse précédente,  $O \left( \sqrt{\frac{N}{M}} \right)$  requêtes de l'oracle  $O_f$  sont nécessaires pour trouver un élément valide. Toutefois, l'algorithme peut-il être implémenté efficacement en termes de complexité temporelle ?  $O_0$  est basé sur le calcul d'opérations booléennes sur  $n$  bits et est connu pour être implémenté à l'aide de  $O(n)$  portes. Nous avons également deux couches de  $n$  portes de Hadarmard. Ces deux composants nécessitent donc uniquement des  $O(n)$  portes par itération. Comme  $N = 2^n$ , nous avons  $O(n) = O(\log(N))$ . Par conséquent, si  $O \left( \sqrt{\frac{N}{M}} \right)$  itérations et de  $O(\log(N))$  portes par itération sont nécessaires, la complexité temporelle totale (sans tenir compte de l'implémentation de l'oracle) est  $O \left( \sqrt{\frac{N}{M}} \log(N) \right)$ .

La complexité globale de l'algorithme dépend finalement de la complexité de l'implémentation de l'oracle  $O_f$ . Si une évaluation de fonction est beaucoup plus compliquée sur un ordinateur quantique que sur un ordinateur classique, le runtime d'algorithme global sera plus long dans le cas quantique, même si techniquement, il utilise moins de requêtes.

# Références

Si vous souhaitez poursuivre l'apprentissage de l'algorithme de Grover, vous pouvez consulter les sources suivantes :

- [Papier d'origine par Lov K. Grover](#) ↗
- Section Algorithmes de recherche quantique de Nielsen, M. A. & Chuang, I. L. (2010). Informations sur le calcul quantique et le quantum.
- [Algorithme de Grover sur Arxiv.org](#) ↗

## Contenu connexe

- [Inanglement dans l'informatique quantique](#)
- [Vecteurs et matrices](#)
- [Portes T et usines T](#)
- [Qubits multiples](#)

---

## Commentaires

Cette page a-t-elle été utile ?

 **Yes**

 **No**

[Indiquer des commentaires sur le produit](#) ↗ | [Obtenir de l'aide sur Microsoft Q&A](#)

# Conventions encadrant les schémas de circuit quantique

Article • 16/01/2025

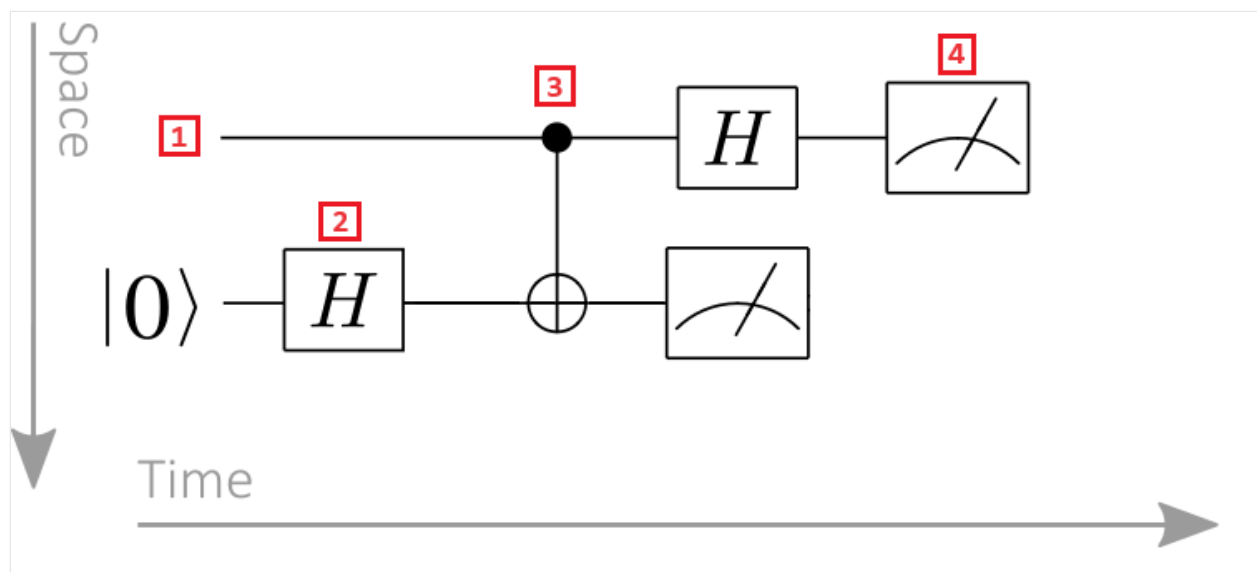
Parfois, les algorithmes quantiques sont plus faciles à comprendre dans un diagramme de circuit que dans la représentation de matrice écrite équivalente. Cet article explique comment lire les diagrammes de circuits quantiques et leurs conventions.

Pour plus d'informations, consultez [Comment visualiser les diagrammes de circuits quantiques](#).

## Lecture de diagrammes de circuits quantiques

Dans un circuit quantique, le temps s'écoule de gauche à droite. Les portes quantiques sont représentées dans l'ordre chronologique, la porte la plus à gauche étant la première appliquée aux qubits.

Prenez le diagramme de circuit quantique suivant comme exemple :

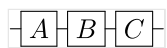


1. **Registre Qubit** : les registres Qubit sont affichés sous forme de lignes horizontales, chaque ligne représentant un qubit. La ligne supérieure est le registre qubit étiqueté 0, la deuxième ligne est le registre qubit étiqueté 1, et ainsi de suite.
2. **Porte quantique** : les opérations quantiques sont représentées par *des portes quantiques*. L'expression « porte quantique » est utilisée par analogie avec les portes logiques classiques. Les portes agissant sur un ou plusieurs registres de qubits sont représentées sous forme de cadres. Dans cet exemple, le symbole représente une opération Hadamard.

3. **Porte contrôlée** : les portes contrôlées agissent sur deux qubits ou plus. Dans cet exemple, le symbole représente une porte CNOT. Le cercle noir représente le qubit de contrôle, et la croix dans un cercle représente le target qubit.
4. **Opération de mesure** : le symbole du compteur représente une opération de mesure. L'opération de mesure prend un registre qubit en tant qu'entrée et génère des informations classiques.

## Application de portes quantiques

Étant donné que le temps passe de gauche à droite, la porte la plus à gauche est appliquée en premier, par exemple, l'action du circuit quantique suivant est la matrice *unitaire*  $CBA$ .



### ! Notes

La multiplication des matrices obéit à la convention opposée : la matrice la plus à droite est appliquée en premier. Toutefois, dans les schémas de circuit quantique, la porte la plus à gauche est appliquée en premier. Cette différence peut parfois prêter à confusion. Il est donc important de noter cette différence majeure entre la notation algébrique linéaire et les schémas de circuit quantique.

## Entrées et sorties de circuits quantiques

Dans un diagramme de circuit quantique, les fils entrant dans une porte quantique représentent les qubits qui sont entrés dans la porte quantique, et les fils sortant de la porte quantique représentent les qubits qui sont sorties de la porte quantique.

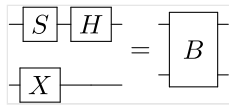
Le nombre d'entrées d'une porte quantique est égal au nombre de sorties d'une porte quantique. Cela est dû au fait que les opérations quantiques sont unitaires et donc réversibles. Si une porte quantique avait plus de sorties que d'entrées, elle ne serait pas réversible et donc pas unitaire, ce qui est une contradiction.

Pour cette raison, pour tout cadre tracé dans un schéma de circuit, le nombre de fils qui entrent et sortent doit être exactement le même.

## Opérations multi-qubits

Les schémas de circuit à plusieurs qubits suivent des conventions similaires à ceux basés sur un qubit. Par exemple, une opération *unitaire à deux qubits*  $B$  peut être définie

comme étant  $(HS \otimes X)$ , le circuit quantique équivalent est le suivant :



Vous pouvez également voir  $B$  comme ayant une action sur un registre de deux qubits plutôt que sur deux registres d'un qubit en fonction du contexte d'utilisation du circuit.

La propriété la plus utile de schémas de circuit aussi abstraits est peut-être qu'ils permettent de décrire des algorithmes quantiques compliqués à un niveau détaillé sans qu'il soit nécessaire de les compiler en portes fondamentales. Ainsi, vous pouvez avoir une idée intuitive du flux de données d'un grand algorithme quantique sans avoir à comprendre en détail le fonctionnement de chacune des sous-routines de l'algorithme.

## Portes contrôlées

Les portes contrôlées par quantum sont des portes à deux qubits qui appliquent une porte à qubit unique à un target qubit si un qubit de contrôle est dans un état spécifique.

Par exemple, considérez une porte contrôlée quantique, indiquée  $\Lambda(G)$ , où une valeur de qubit unique contrôle l'application de l'opération  $G$ . La porte  $\Lambda(G)$  peut être comprise en examinant l'exemple suivant d'une entrée d'état de produit :

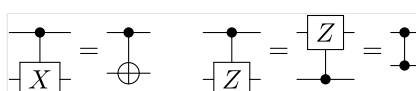
$$\Lambda(G) (\alpha \ket{0} + \beta \ket{1}) \ket{\psi} = \alpha \ket{0} \ket{\psi} + \beta \ket{1} G \ket{\psi}$$

Autrement dit, la porte contrôlée applique  $G$  au registre qui contient  $\psi$  si et seulement si le qubit de contrôle prend la valeur 1. En général, ces opérations contrôlées sont décrites dans les diagrammes de circuits par le symbole suivant :



Ici, le cercle noir représente le bit quantique sur lequel la porte est contrôlée, et un fil vertical représente l'unitaire qui est appliqué quand le qubit de contrôle prend la valeur 1.

Pour les cas spéciaux où  $G = X$  et  $G = Z$ , la notation suivante est utilisée pour décrire la version contrôlée des portes (notez que la porte contrôlée-X est la porte CNOT) :



Q# offre des méthodes pour générer automatiquement la version contrôlée des opérations, évitant au programmeur d'avoir à les coder manuellement. En voici un exemple :

Q#

```
operation PrepareSuperposition(qubit : Qubit) : Unit
is Ctl { // Auto-generate the controlled specialization of the operation
 H(qubit);
}
```

## Portes contrôlées classiquement

Les portes quantiques peuvent également être appliquées après une mesure, où le résultat de la mesure agit comme un bit de contrôle classique.

Le symbole suivant représente une porte contrôlée classiquement, où  $G$  est appliqué conditionné sur la valeur 1 du bit de contrôle classique :



## Opérateur de mesure

Les opérations de mesure prennent un registre qubit, la mesure et génère le résultat sous forme d'informations classiques.

Une opération de mesure est représentée par un symbole de compteur. Elle prend toujours comme entrée un registre de qubits (représenté par une ligne pleine) et retourne des informations classiques (représentées par une ligne double). Plus précisément, le symbole de l'opération de mesure ressemble à ceci :



Dans Q#, l'opérateur `Measure` implémente l'opération de mesure.

## Exemple : Transformation unitaire

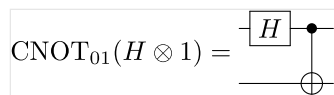
Considérez la transformation unitaire  $\text{CNOT}_{01}(H \otimes 1)$ . Cette séquence de portes revêt une importance fondamentale en informatique quantique, car elle crée un état à deux qubits intriqué au maximum :

$$\text{CNOT}_{01}(H \otimes 1) \ket{00} = \frac{1}{\sqrt{2}} (\ket{00} + \ket{11})$$



Les opérations de cette complexité, voire d'une complexité supérieure, sont omniprésentes dans les algorithmes quantiques et la correction d'erreur quantique.

Le schéma de circuit pour la préparation de cet état quantique intriqué au maximum est le suivant :



Le symbole derrière la porte Hadamard représente une porte CNOT, où le cercle noir indique le qubit de contrôle et la croix dans un cercle indique le target qubit. Ce circuit quantique est représenté comme agissant sur deux qubits (ou de manière équivalente, deux registres d'un qubit).

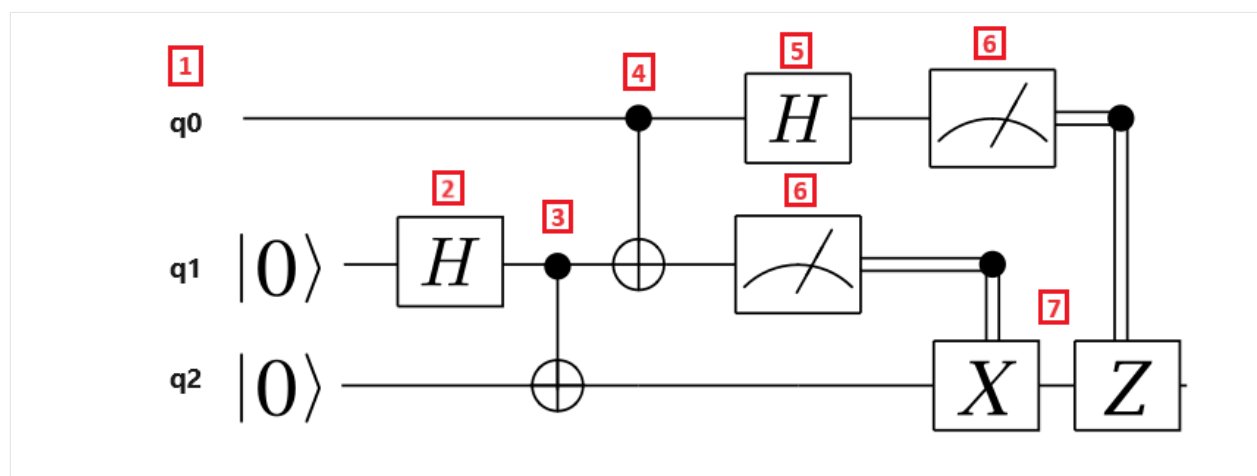
## Exemple : Diagramme de circuit de téléportation

La téléportation quantique est l'un des meilleurs algorithmes quantiques pour illustrer les composants de circuit.

La téléportation quantique est un protocole qui permet à un état quantique d'être transmis d'un qubit à un autre, avec l'aide d'un état enchevêtré partagé entre l'expéditeur et le récepteur, et la communication classique entre eux.

À des fins d'apprentissage, l'expéditeur est appelé Alice, le récepteur est appelé Bob, et le qubit à téléporter est appelé qubit du message. Alice et Bob contiennent chacun un qubit, et Alice a un qubit supplémentaire qui est le qubit de message.

Le diagramme de circuit suivant illustre le protocole de téléportation :



Nous allons décomposer les étapes du protocole de téléportation :

1. Le registre **qubit q0** est le qubit de message, le registre qubit **q1** est le qubit d'Alice et le registre **qubit q2** est le qubit de Bob. Le qubit de message est dans un état inconnu, et Alice et les qubits de Bob sont dans l'état  $|0\rangle$ .
2. Une **porte Hadamard** est appliquée au **qubit** d'Alice. Étant donné que le qubit est dans l'état  $|0\rangle$ , l'état résultant est  $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ .
3. Une **porte CNOT** est appliquée aux **qubits** d'Alice et de Bob. Le qubit d'Alice est le qubit de contrôle, et le qubit de Bob est le target qubit. L'état résultant est  $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ . Alice et Bob partagent maintenant un état enchevêtré.
4. Une **porte CNOT** est appliquée au qubit de **message** et au **qubit** d'Alice. Étant donné que le qubit d'Alice est également enchevêtré avec le qubit de Bob, l'état résultant est un état enchevêtré de trois qubits.
5. Une **porte Hadamard** est appliquée au qubit **de** message.
6. Alice **mesure** ses deux qubits et indique les résultats de mesure à Bob, ce qui n'est pas reflété dans le circuit. Les résultats de mesure sont deux bits classiques, qui peuvent prendre les valeurs 00, 01, 10 ou 11.
7. Deux portes **Pauli contrôlées classiquement** X et Z sont appliquées au **qubit** de Bob, selon la valeur 1 du bit de résultat. L'état résultant est l'état qubit du message d'origine.

## Contenu connexe

- [Enchevêtrement quantique](#)
- [Algorithme de Grover](#)
- [Représentation intermédiaire quantique](#)
- [Vecteurs et matrices](#)

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Inanglement et corrélations

Article • 16/09/2024

Entanglement est un concept fondamental dans la mécanique quantique qui décrit une corrélation quantique entre les systèmes quantiques. Lorsque deux qubits ou plus sont enchevêtrés, l'état d'un qubit dépend de l'état de l'autre qubit, même s'ils sont éloignés. Cette corrélation quantique est une caractéristique unique des systèmes quantiques qui n'ont pas d'équivalent classique.

Cet article fournit une vue d'ensemble de l'inanglement, des corrélations et explique comment créer un entanglement à l'aide de portes quantiques.

## Qu'est-ce que l'intrication ?

Imaginez que vous avez deux qubits,  $A$  et  $B$ . Les qubits sont indépendants les uns des autres, ce qui signifie que les informations sur l'état du qubit  $A$ , quel qu'il soit, appartiennent uniquement au qubit  $A$ . De même, les informations sur l'état de qubit  $B$  appartiennent à qubit  $B$ . Dans ce cas, les qubits ne sont pas enchevêtrés, car ils ne partagent pas d'informations sur leurs états.

Imaginez maintenant que vous entassez les qubits. Si les qubits  $A$  et  $B$  sont enchevêtrés, les informations sur l'état du qubit  $A$  ne sont pas indépendantes *de l'état du qubit*  $B$ . Lorsque elles sont enchevêtrées, les informations sont partagées entre les deux qubits et il n'existe aucun moyen de connaître l'état du qubit  $A$  ou du qubit  $B$ . Vous pouvez uniquement décrire l'état du système global, et non celui de chaque qubit.

L'intrication est une **corrélation quantique** entre deux particules ou plus. Si deux particules sont enchevêtrées, elles ne peuvent pas être décrites indépendamment, mais seulement en tant que système entier.

Deux particules ou plus peuvent être enchevêtrées même si elles sont séparées par de grandes distances. Cette corrélation est plus forte que n'importe quelle corrélation classique, et il s'agit d'une ressource clé pour les tâches de traitement des informations quantiques telles que la téléportation quantique, le chiffrement quantique et l'informatique quantique. Si vous souhaitez apprendre à téléporter un qubit à l'aide d'un entanglement, consultez [ce module dans le chemin](#) d'apprentissage Azure Quantum.

### 📌 Notes

Entanglement est une propriété de systèmes multi-qubits, pas de qubits uniques. Autrement dit, un qubit unique ne peut pas être enchevêtré.

## Définition de l'enchevêtrement dans les systèmes quantiques

Imaginez deux qubits, soit  $A$  et  $B$ , de sorte que l'état du système global  $|\phi\rangle$  soit :

$$|\phi\rangle = \frac{1}{\sqrt{2}}(|0_A 0_B\rangle + |1_A 1_B\rangle)$$

### Notes

En notation Dirac,  $|0_A 0_B\rangle = |0\rangle_A |0\rangle_B$ . La première position correspond au premier qubit et la seconde position au deuxième qubit.

Le système global  $|\phi\rangle$  est dans une superposition des états  $|00\rangle$  et  $|11\rangle$ . Quel est toutefois l'état individuel du qubit  $A$  ? Et celui du qubit  $B$  ? Si vous essayez de décrire l'état du qubit  $A$  sans tenir compte de l'état du qubit  $B$ , vous échouez. Les sous-systèmes  $A$  et  $B$  sont enchevêtrés et ne peuvent pas être décrits indépendamment.

Si vous mesurez les deux qubits, seuls deux résultats sont possibles :  $|\ket{00}$  et  $|\ket{11}$ , chacun avec la même probabilité de  $\frac{1}{2}$ . La probabilité d'obtenir les états  $|01\rangle$  et  $|10\rangle$  est égale à zéro.

Mais que se passe-t-il si vous ne mesurez qu'un seul qubit ? Lorsque deux particules sont enchevêtrées, les résultats de mesure sont également corrélés. Autrement dit, quelle que soit l'opération qui arrive à l'état d'un qubit dans une paire enchevêtrée, affecte également l'état de l'autre qubit.

Si vous mesurez uniquement le qubit  $A$  et que vous obtenez l'état  $|0\rangle$ , cela signifie que le système global se réduit à l'état  $|00\rangle$ . Il s'agit du seul résultat possible, car la probabilité de mesurer  $|01\rangle$  est nulle. Par conséquent, sans mesurer le qubit  $B$ , vous pouvez être sûr que le deuxième qubit est également dans  $|0\rangle$  état. Les résultats de la mesure sont corrélés, car les qubits sont intriqués.

L'état  $|\phi\rangle$  quantique est appelé état **Bell**. Il y a quatre états Bell :

$$|\ket{\phi^+}\rangle = \frac{1}{\sqrt{2}}|\ket{00}\rangle + \frac{1}{\sqrt{2}}|\ket{11}\rangle$$

$$|\ket{\phi^-}\rangle = \frac{1}{\sqrt{2}}|\ket{00}\rangle - \frac{1}{\sqrt{2}}|\ket{11}\rangle$$

$$|\psi^+\rangle = \frac{1}{\sqrt{2}}|01\rangle + \frac{1}{\sqrt{2}}|10\rangle$$

$$|\psi^-\rangle = \frac{1}{\sqrt{2}}|01\rangle - \frac{1}{\sqrt{2}}|10\rangle$$

### ⓘ Notes

Cet exemple utilise deux qubits, mais l'intrication quantique n'est pas limitée à deux qubits. En général, il est possible que les systèmes à plusieurs qubits partagent l'intrication.

## Création d'un enchevêtrement avec des opérations quantiques

Vous pouvez utiliser des opérations quantiques pour créer un enchevêtrement quantique. L'une des façons les plus courantes de créer l'entanglement à deux qubits dans l'état  $|00\rangle$  consiste à appliquer l'opération *Hadamard*  $H$  et l'opération *CNOT* contrôlée-NOT pour les transformer en état  $|\text{phiBell}^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ .

L'opération *CNOT* prend deux qubits comme entrée, l'un agit comme qubit de contrôle et l'autre est le qubit cible. L'opération `CNOT` retourne l'état du qubit cible si, et seulement si, l'état du qubit de contrôle est  $|1\rangle$ .

[🔍 Agrandir le tableau](#)

Entrée	Sortie
$ 00\rangle$	$ 00\rangle$
$ 01\rangle$	$ 01\rangle$
$ 10\rangle$	$ 11\rangle$
$ 11\rangle$	$ 10\rangle$

Voici son fonctionnement :

1. Prenez deux qubits dans l'état  $|00\rangle$ . Le premier qubit est le qubit de contrôle et le deuxième qubit est le qubit cible.
2. Préparez un état de superposition dans le qubit de contrôle en appliquant  $H$ .

$$\frac{1}{\sqrt{2}}(|0_c\rangle + |1_c\rangle)|0_t\rangle$$

### ⚠ Notes

Les indices  $c$  et  $t$  spécifier le contrôle et les qubits cibles.

3. Appliquez l'opérateur  $CNOT$  au qubit de contrôle, qui se trouve dans un état de superposition et le qubit cible, qui se trouve dans l'état  $|0_t\rangle$ .

$$CNOT \frac{1}{\sqrt{2}}(|0_c\rangle + |1_c\rangle)|0_t\rangle = CNOT \frac{1}{\sqrt{2}}(|0_c0_t\rangle + |1_c0_t\rangle) =$$

$$= \frac{1}{\sqrt{2}}(CNOT |0_c0_t\rangle + CNOT |1_c0_t\rangle) =$$

$$= \frac{1}{\sqrt{2}}(|0_c0_t\rangle + |1_c1_t\rangle)$$

### 💡 Conseil

Pour savoir comment entangler deux qubits avec Q#, consultez [Démarrage rapide : Créer votre premier Q# programme](#).

## Séparabilité et enchevêtrement quantique

L'inanglement peut être considéré comme l'absence de séparabilité : un état est enchevêtré lorsqu'il n'est pas séparable.

Un état quantique est séparable s'il peut être écrit en tant qu'état produit des sous-systèmes. Autrement dit, un état  $|\psi\rangle_{AB}$  est séparable s'il peut être écrit en tant que combinaison d'états produits des sous-systèmes, c'est-à-dire

**Extra close brace or missing open brace**

## Entanglement dans des états purs

Un état quantique pur est un vecteur de ket unique, tel que l'état  $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ .

Les états purs ne peuvent pas être écrits en tant que mélange statistique (ou *combinaison* convexe) d'autres états quantiques.

Sur la [sphère](#) Bloch, les états purs sont représentés par un point sur la surface de la sphère, tandis que les états mixtes sont représentés par un point intérieur.

Un état  $|\phi\rangle$  pur  $AB$  est enchevêtré s'il ne peut pas être écrit en tant que combinaison d'états produits des sous-systèmes, c'est-à-dire  $|\phi\rangle \neq |a\rangle_A |b\rangle_B$ .

Par exemple, considérez l'état  $|\psi\rangle_{AB} = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$ .

Au début, l'état  $|\psi\rangle_{AB}$  ne ressemble pas à un état de produit, mais si nous réécrivons l'état comme

$$|\psi\rangle_{AB} = \frac{1}{\sqrt{2}}(|0\rangle_A + |1\rangle_A) \otimes \frac{1}{\sqrt{2}}(|0\rangle_B + |1\rangle_B) = \frac{1}{2}(|0\rangle_A + |1\rangle_A)(|0\rangle_B + |1\rangle_B)$$

l'état  $|\psi\rangle_{AB}$  est un état de produit, par conséquent, il n'est pas enchevêtré.

## Inanglement dans des états mixtes

Les états quantiques mixtes sont un ensemble statistique d'états purs. Pour décrire les états mixtes, il est plus facile d'utiliser leur matrice de densité  $\rho$  plutôt que la notation de ket.

Un état *mixte* est séparable s'il peut être écrit en tant que combinaison convexe d'états produits des sous-systèmes, par exemple

$$\rho = \sum_j p_j \rho_j^A \otimes \rho_j^B$$

où  $p_j \geq 0$ ,  $\sum p_j = 1$  et  $\rho_j^A \geq 0$ ,  $\rho_j^B \geq 0$ .

Pour plus d'informations, consultez [Matrices de densité](#).

Un état *mixte* est enchevêtré s'il n'est pas séparable, autrement dit, il ne peut pas être écrit en tant que combinaison convexe d'états de produit.

### Notes

- Si un état *inanglé* est pur, il contient uniquement des corrélations quantiques.
- Si un état *enchevêtré* est mixte, il contient à la fois des corrélations classiques et quantiques.

## Présentation des corrélations classiques

Les corrélations classiques sont dues au manque de connaissances de l'état du système. Autrement dit, il existe une certaine aléatoire associée à la corrélation classique, mais elle peut être éliminée en obtenant des connaissances.

Par exemple, considérez deux boîtes, chacune contenant une boule. Vous savez que les deux boules sont de la même couleur, bleu ou rouge. Si vous ouvrez une boîte et découvrez que la balle à l'intérieur est bleue, alors nous savons que l'autre boule est bleue aussi. Par conséquent, ils sont corrélés. Cependant, l'incertitude que nous avons lors de l'ouverture de la boîte est due à notre manque de connaissances, ce n'est pas fondamental. La boule était bleue avant d'ouvrir la boîte. Par conséquent, il s'agit d'une corrélation classique, et non d'une corrélation quantique.

L'état quantique mixte du système formé par les deux zones  $\rho_{boxes}$  peut être écrit en tant que

$$\rho_{boxes} = \frac{1}{2} (\ket{rouge}_A \bra{rouge}_B + \ket{bleu}_A \bra{bleu}_B)$$

Notez que l'état  $\rho_{boxes}$  est séparable, où  $p_1 p_2 = \frac{1}{2}$  alors il contient uniquement des corrélations classiques. Un autre exemple d'état séparable mixte est

$$\rho = \frac{1}{2} (\ket{0}_A \bra{0}_B + \ket{1}_A \bra{1}_B)$$

À présent, tenez compte de l'état suivant :

$$\rho = \frac{1}{4} (\ket{00} + \ket{11}) (\bra{00} + \bra{11})$$

Dans ce cas, notre connaissance de l'état est parfaite, nous savons avec une certitude maximale que le système  $AB$  est dans l'état  $|\text{Bell}^+\rangle$  et  $\rho$  est un état pur. Par conséquent, il n'existe pas de corrélations classiques. Mais si nous mesurons une observable sur le sous-système  $A$ , nous obtenons un résultat aléatoire qui nous donne des informations sur l'état du sous-système  $B$ . Ce caractère aléatoire est fondamental, c'est-à-dire les corrélations quantiques.

Un exemple d'état quantique qui contient à la fois des corrélations classiques et quantiques est

$$\rho = \frac{1}{2} (\ket{\phi^+} \bra{\phi^+} + \ket{\phi^-} \bra{\phi^-})$$

### Notes

Un état séparable contient uniquement des corrélations classiques.



# Contenu connexe

---

## Commentaires

Cette page a-t-elle été utile ?



Yes



No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Introduction à la correction des erreurs quantiques

Article • 28/03/2025

Cet article explique les principes de base de la correction des erreurs quantiques, les types d'erreurs quantiques et certains codes courants de correction des erreurs quantiques. Il fournit également un exemple de correction des erreurs à l'aide du code à trois qubits.

## Qu'est-ce que la correction des erreurs quantiques ?

La correction des erreurs quantiques (QEC) est une technique qui nous permet de protéger les informations quantiques contre les erreurs. La correction des erreurs est particulièrement importante dans les ordinateurs quantiques, car les algorithmes quantiques efficaces utilisent des ordinateurs quantiques à grande échelle, qui sont sensibles au bruit.

Le principe de base de la correction des erreurs quantiques est que le nombre de bits utilisés pour encoder une quantité donnée d'informations est augmenté. Cette redondance permet au code de détecter et de corriger les erreurs.

Les taux d'erreur des ordinateurs quantiques sont généralement plus élevés que les erreurs de l'ordinateur classique en raison des défis liés à la création et à l'exploitation de systèmes quantiques. Le bruit, la décohérence et les imperfections dans les portes quantiques peuvent provoquer des erreurs dans les calculs quantiques. Les ordinateurs quantiques actuels ont des taux d'erreur compris entre 1 % et 0,1 %. En d'autres termes, cela signifie qu'en moyenne, une opération de porte quantique sur 100 à 1 000 entraîne une erreur.

## Types d'erreurs quantiques

Il existe deux types fondamentaux d'erreurs quantiques : les retournements de bits et les retournements de phase.

Les erreurs de retournement de bits se produisent lorsqu'un qubit passe de  $|0\rangle$  à  $|1\rangle$  ou vice versa. Les erreurs d'inversion de bits sont également appelées  $\sigma_x$ , car elles mappent les états de qubits  $\sigma_x$   $|\ket{0}\rangle = |\ket{1}\rangle$  et  $|\ket{1}\rangle = |\ket{0}\rangle$ . Cette erreur est analogue à une erreur classique de retournement de bits.

Les erreurs de retourne de phase se produisent lorsqu'un qubit change sa phase. Ils sont également appelés erreurs  $\sigma_z$ , car ils correspondent aux états des qubits  $\sigma_z$   $|\ket{0}\rangle = |\ket{0}\rangle$  et  $|\ket{1}\rangle = -|\ket{1}\rangle$ . Ce type d'erreur n'a pas d'analogie classique.

Dans l'informatique quantique, les erreurs quantiques peuvent se manifester sous forme de retournements de bits, de retournes de phase ou d'une combinaison des deux.

## Comment fonctionne la correction des erreurs quantiques ?

Les codes de correction d'erreur quantique fonctionnent en encodant les informations quantiques dans un plus grand ensemble de qubits, appelés *qubits physiques*. L'état conjoint des qubits physiques représente un *qubit* logique.

Les qubits physiques sont soumis à des erreurs en raison de la décohérence et des imperfections dans les portes quantiques. Le code est conçu afin que les erreurs puissent être détectées et corrigées en mesurant certains qubits du code.

Par exemple, imaginez que vous souhaitez envoyer le message  $|0\rangle$  à qubit unique. Vous pouvez utiliser trois qubits physiques pour encoder le message, en envoyant  $|000\rangle$ , ce qui est appelé codeword. Ce code de correction d'erreur est un *code* de répétition, car le message est répété trois fois.

À présent, imaginez qu'une erreur de basculement de bit unique se produit pendant la transmission, de sorte que l'état reçu par le destinataire est  $|010\rangle$ . Dans ce scénario, le destinataire peut être en mesure de déduire que le message prévu est  $|000\rangle$ . Toutefois, si le message est soumis à deux erreurs de basculement de bits, le destinataire peut déduire un message incorrect. Enfin, si les trois bits sont retournés afin que le message  $|000\rangle$  d'origine devienne  $|111\rangle$ , le destinataire n'a aucun moyen de connaître une erreur s'est produite.

La distance de code d'un code QEC est le nombre minimal d'erreurs qui modifient un mot de code en un autre, autrement dit le nombre d'erreurs qui ne peuvent pas être détectées. La distance de code  $d$  peut être définie en tant que

$$d = 2t + 1$$

où  $t$  est le nombre d'erreurs que le code peut corriger. Par exemple, le code à trois bits peut détecter et corriger une erreur de retournement de bits, donc  $t = 1$ , et donc la distance de code est  $d = 3$ .

Notez que les codes de répétition, tels que le code trois bits utilisé dans cet exemple, ne peuvent corriger que les erreurs de retournement de bits, et non les erreurs de retour de phase. Pour corriger les deux types d'erreurs, des codes de correction d'erreurs quantiques plus sophistiqués sont nécessaires.

## Types de codes QEC

Il existe de nombreux types de codes QEC différents, chacun avec ses propres propriétés et avantages. Voici quelques codes QEC courants :

- **Code de répétition** : code de correction d'erreur quantique le plus simple, où un qubit unique est encodé en plusieurs qubits en le répétant plusieurs fois. Le code de répétition peut corriger les erreurs de retournement de bits, mais pas les erreurs de retournement de phase.
- **Code Shor** : le premier code de correction d'erreur quantique, développé par Peter Shor. Il encode un qubit logique en neuf qubits physiques. Le code de Shor peut corriger une erreur de retournement d'un bit ou une erreur d'inversion de phase, mais il ne peut pas corriger les deux types d'erreurs en même temps.
- **Code Steane** : il s'agit d'un code de sept qubits qui peut corriger les erreurs de retournement de bits et de phase. Il présente l'avantage d'être tolérant aux erreurs, ce qui signifie que le processus de correction des erreurs lui-même n'introduit pas d'erreurs supplémentaires.

- **Code de surface** : il s'agit d'un code de correction d'erreur topologique qui utilise une maille bidimensionnelle de qubits pour encoder des qubits logiques. Il a un seuil de correction d'erreur élevé et est considéré comme l'une des techniques les plus prometteuses pour l'informatique quantique à tolérance de panne à grande échelle. Le code de surface est utilisé par le [Estimateur de Ressources Azure Quantum](#).
- **Code Hastings-Haah** : ce code de correction d'erreur quantique offre de meilleurs coûts d'espace-temps que les codes de surface sur les qubits Majorana dans de nombreux régimes. Pour les jeux d'instructions basés sur des portes, le surcoût est plus important, ce qui rend cette approche moins efficace que le code de surface.

## Exemple : code à trois qubits

Le code de correction d'erreur à trois qubits est un code de répétition simple qui peut détecter et corriger une erreur de retournement de bit. Il encode un qubit logique unique en trois qubits physiques en répétant le qubit trois fois.

### Conseil

Consultez l'exemple de code pour le code à trois qubits.

Imaginez que vous souhaitez envoyer un qubit  $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$  unique arbitraire. Pour éviter les erreurs, vous encoderez les états  $|0\rangle$  de base et  $|1\rangle$  dans un état conjoint de trois qubits. Les deux états de base logiques sont  $|0_L\rangle = |000\rangle$  et  $|1_L\rangle = |111\rangle$ .

Par conséquent, le qubit  $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$  est encodé comme suit :

$$|\psi_L\rangle = \alpha|000\rangle + \beta|111\rangle = \alpha|0_L\rangle + \beta|1_L\rangle$$

Nous allons décomposer les étapes du code à trois qubits.

## Préparation des qubits

Tout d'abord, vous encodez votre qubit  $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$  dans un état commun de trois qubits.

Ensuite, vous préparez deux qubits additionnels dans l'état  $|0\rangle$ . Ainsi, l'état global des trois qubits est  $(\alpha|0\rangle + \beta|1\rangle)|0\rangle|0\rangle = \alpha|000\rangle + \beta|100\rangle$ .

Enfin, vous encodez le qubit unique dans un état conjoint de trois qubits, en appliquant deux opérations CNOT. Le premier CNOT utilise le premier qubit comme contrôle et agit sur le deuxième qubit, produisant  $\alpha|000\rangle + \beta|110\rangle$ . Le deuxième CNOT utilise le premier qubit comme contrôle et agit sur le troisième qubit. L'état des trois qubits est maintenant  $\alpha|000\rangle + \beta|111\rangle$ .

## Envoi des qubits

Vous envoyez les trois qubits. En supposant que seules des erreurs de retournement d'un bit peuvent se produire, les qubits reçus se trouvent dans l'un des états suivants :

[Agrandir le tableau](#)

État	Erreur
$\alpha  000\rangle + \beta  111\rangle$	Aucune erreur
$\alpha  100\rangle + \beta  011\rangle$	Qubit 1
$\alpha  010\rangle + \beta  101\rangle$	Qubit 2
$\alpha  001\rangle + \beta  110\rangle$	Qubit 3

## Ajout de qubits auxiliaires

Tout d'abord, vous introduisez deux qubits supplémentaires, préparés dans l'état  $|00\rangle$ . Cette paire auxiliaire de qubits est utilisée pour extraire des informations de l'erreur sans mesurer ou obtenir directement des informations sur l'état logique.

Ensuite, vous effectuez quatre opérations CNOT : les deux premières opérations utilisent les qubits du premier et du deuxième reçu comme contrôle et agissent sur le premier qubit auxiliaire, et les deux dernières opérations utilisent les qubits du premier et du troisième reçu comme contrôle et agissent sur le deuxième bit auxiliaire. L'état total des cinq qubits est maintenant :

[Agrandir le tableau](#)

État	Erreur
$(\alpha  000\rangle + \beta  111\rangle)  00\rangle$	Aucune erreur
$(\alpha  100\rangle + \beta  011\rangle)  11\rangle$	Qubit 1
$(\alpha  010\rangle + \beta  101\rangle)  10\rangle$	Qubit 2
$(\alpha  001\rangle + \beta  110\rangle)  01\rangle$	Qubit 3

## Récupération du syndrome d'erreur

Pour récupérer les informations d'erreur, vous mesurez les deux qubits auxiliaires dans les états  $|0\rangle$  de base de calcul et  $|1\rangle$ . En procédant ainsi, vous récupérez l'état de l'articulation, qui est appelé syndrome d'erreur, car il aide à diagnostiquer les erreurs dans les qubits reçus.

Maintenant, vous savez dans lequel des quatre états possibles se trouvent les trois qubits reçus. Vous pouvez corriger l'erreur en appliquant l'opération de correction. Dans ce cas, vous faites face à des erreurs d'inversion de bits. Par conséquent, la correction est une opération  $\sigma_x$  appliquée à un (ou à aucun) des qubits.

Par exemple, si le syndrome d'erreur est  $|00\rangle$ , alors les qubits reçus sont dans l'état  $\alpha |000\rangle + \beta |111\rangle$ , qui est l'état que vous avez envoyé à l'origine. Si le syndrome d'erreur est  $|11\rangle$ , les qubits reçus sont dans l'état  $\alpha |100\rangle + \beta |011\rangle$ .

*Ilya une erreur de retournement de bits sur le premier qubit, que vous pouvez corriger en appliquant une opération  $\sigma_x$  au premier qubit.*

 Agrandir le tableau

Syndrome d'erreur	État réduit	Correction
$ 00\rangle$	$\alpha  000\rangle + \beta  111\rangle$	Ne rien faire
$ 01\rangle$	$\alpha  100\rangle + \beta  011\rangle$	Appliquer $\sigma_x$ à qubit 3
$ 10\rangle$	$\alpha  010\rangle + \beta  101\rangle$	Appliquer $\sigma_x$ à qubit 2
$ 11\rangle$	$\alpha  001\rangle + \beta  110\rangle$	Appliquer $\sigma_x$ à qubit 1

## Extraction du qubit d'origine

Enfin, pour extraire le qubit unique que vous souhaitez transmettre à l'origine, vous appliquez deux opérations CNOT : l'une utilise le premier qubit comme contrôle et agit sur le deuxième qubit, et d'autres utilisent le premier qubit comme contrôle et agit sur le troisième.

L'état du premier qubit est maintenant  $\alpha |0\rangle + \beta |1\rangle$ , qui est le qubit d'origine que vous souhaitez transmettre.

 Important

Le code QEC n'a pas d'informations sur les coefficients  $\alpha$  et  $\beta$ , par conséquent, les superpositions de l'état de calcul restent intactes pendant le processus de correction.

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Représentation intermédiaire quantique

14/02/2025

Quantum Intermediate Representation (QIR) est une représentation intermédiaire qui sert d'interface commune entre les langages/frameworks de programmation quantique et les plateformes de calcul quantique ciblées. QIR spécifie un ensemble de règles pour représenter des programmes quantiques à l'aide d'un format indépendant du langage et du matériel dans LLVM IR. QIR est un projet développé par l'Alliance QIR dont Microsoft est l'un des membres.

## Qu'est-ce qu'une représentation intermédiaire ?

Un modèle courant dans les compilateurs classiques consiste à commencer par compiler le langage source dans une représentation intermédiaire. Une représentation intermédiaire est, comme son nom l'indique, une étape intermédiaire dans la conversion d'instructions du code source en langage machine.

Une représentation intermédiaire agit comme une représentation abstraite d'un programme. Tous les programmes, quel que soit le langage dans lequel ils sont écrits, sont traduits dans cette représentation intermédiaire par un compilateur frontal, tandis qu'un composant back-end est chargé de traduire cette représentation intermédiaire en une représentation automatique. La représentation intermédiaire permet donc de dissocier les langages sources des plateformes matérielles et de créer un compilateur de manière modulaire, où chaque nouveau langage nécessite uniquement un nouveau serveur front-end pour être pris en charge sur toutes les plateformes pour lesquelles un back end est disponible.

La représentation intermédiaire est généralement conçue pour permettre à de nombreux langages sources différents d'être représentés. De plus, à ce niveau intermédiaire, il est également possible d'effectuer une optimisation et une réorganisation de circuit qui rend l'implémentation finale plus efficace. Une fois la plateforme d'exécution finale target connue, la représentation intermédiaire peut être compilée en code exécutable réel.

Cette approche permet à de nombreux langages sources de partager un ensemble commun d'optimiseurs et de générateurs exécutables. Il facilite également la compilation d'un langage source unique pour de nombreux autres targets. La représentation intermédiaire fournit une plateforme commune qui peut être partagée entre de nombreuses sources et targets permet une grande réutilisation dans les machines du compilateur.

## Qu'est-ce que Quantum Intermediate Representation ?

QIR est une représentation intermédiaire pour les programmes quantiques développés par [l'Alliance QIR](#), à laquelle Microsoft appartient. Il fournit une interface commune qui prend en charge de nombreux langages et target plateformes pour le calcul quantique. Vous pouvez considérer QIR comme un langage universel de couche intermédiaire qui permet la communication entre les langages de haut niveau et les machines. Alors que Q# se compile en QIR, QIR n'est pas spécifique à Q#. Toute infrastructure de programmation quantique peut tirer parti du QIR pour représenter un programme quantique. Il est indépendant du matériel, ce qui signifie qu'il ne spécifie pas d'instruction quantique ou de jeu de portes, en laissant cela à l'environnement target informatique.

QIR est basée sur le compilateur classique [LLVM](#) open source populaire. LLVM est une collection de technologies de compilateur et de chaîne d'outils modulaires et réutilisables qui ont été adaptées par un large éventail de langages. QIR spécifie un ensemble de règles pour représenter des constructions quantiques dans LLVM, mais il ne nécessite aucune extension ni modification de LLVM.

Le fait que LLVM est la chaîne d'outils sous-jacente signifie que QIR est naturellement capable de traiter à la fois la logique classique et quantique. Cette fonctionnalité est essentielle pour les algorithmes quantiques hybrides classiques, qui sont devenus de plus en plus importants pour les applications de l'informatique quantique. En outre, elle vous permet de tirer parti des outils de compilation et d'optimisation du secteur de l'informatique classique et, par conséquent, de réduire le coût de l'écriture de traductions.

De nombreux secteurs de pointe de l'informatique quantique ont déjà adopté QIR. Par exemple, NVIDIA, Oak Ridge National Laboratory, Quantinuum, Quantum Circuits Inc. et Rigetti Computing créent des chaînes d'outils qui tirent parti de QIR.

Pour plus d'informations, consultez la [spécification](#) QIR. Si vous êtes intéressé par les outils et projets du compilateur qui utilisent QIR, consultez ces [dépôts QIR](#).

## Qu'est-ce que l'Alliance QIR ?

L'Alliance [QIR](#) est un effort conjoint visant à développer une représentation intermédiaire quantique prospective avec l'objectif de permettre une interopérabilité totale au sein de l'écosystème quantique, de réduire les efforts de développement de toutes les parties et de fournir une représentation adaptée aux processeurs quantiques hétérogènes actuels et futurs.

Les SDK et les langages quantiques apparaissent et évoluent à un rythme rapide, ainsi que de nouveaux processeurs quantiques avec des fonctionnalités uniques et distinctes les unes des autres. Pour fournir une interopérabilité entre de nouveaux langages et de nouvelles fonctionnalités matérielles, il est impératif que l'écosystème développe et partage une représentation intermédiaire qui fonctionne avec le matériel quantique actuel et futur.



Avec leur travail collectif et leur partenariat, l'Alliance QIR vise à :

- Réduire les efforts de développement requis pour toutes les parties en favorisant l'interopérabilité entre différents frameworks et langages.
- Activer le développement de bibliothèques partagées à la fois pour le développement d'applications quantiques et pour le développement de compilateur quantique.
- Tirer parti de la technologie de compilateur de pointe et des outils, bibliothèques et apprentissages existants du calcul hautes performances.
- Autoriser l'évolution incrémentielle et progressive dans la façon dont les calculs classiques et quantiques peuvent interagir au niveau matériel.
- Fournir la flexibilité nécessaire pour connecter facilement les technologies émergentes d'une manière qui permet l'expérimentation avec des fonctionnalités matérielles distinctes et différenciées.

L'Alliance QIR fait partie du travail de la [Linux Foundation's Joint Development Foundation](#) sur les normes ouvertes. Parmi les membres fondateurs figurent Microsoft, ainsi que Quantinuum, Oak Ridge National Laboratory, Quantum Circuits Inc. et Rigetti Computing.

## À quoi ressemble-t-il Quantum Intermediate Representation ?

Étant donné que QIR est basé sur LLVM, QIR ressemble à LLVM.

Prenons comme exemple le code Q# suivant pour générer une paire Bell :

Q#

```
operation CreateBellPair(q1 : Qubit, q2 : Qubit) : Unit {
 H(q1);
 CNOT(q1, q2);
}
```

Lorsqu'il est compilé en QIR, cela devient :

```
define void @CreateBellPair__body(%Qubit* %q1, %Qubit* %q2) {
entry:
 call void @__quantum__qis__h(%Qubit* %q1)
 call void @__quantum__qis__cnot(%Qubit* %q1, %Qubit* %q2)
 ret void
}
```

Dans cet extrait de code, vous pouvez voir quelques fonctionnalités QIR :

- Les opérations dans Q# (ou tout autre langage de programmation quantique) sont représentées par des fonctions LLVM.
- Les qubits sont représentés en tant que pointeurs vers un type de structure opaque nommé appelé `%Qubit`.

Bien que la QIR pour l'opération `CreateBellPair` soit très simple, QIR hérite de toutes les fonctionnalités de LLVM pour exprimer des boucles, des conditions et d'autres flux de contrôle complexes. QIR hérite également de la capacité de LLVM à exprimer un calcul classique arbitraire.

Pour plus d'informations, regardez la session développeur de Microsoft qui a été présentée pendant [l'événement Q2B 2021](#).

## Pourquoi est-ce Quantum Intermediate Representation important ?

QIR est un outil essentiel lors de l'exécution d'algorithmes quantiques sur du matériel réel. Mais les représentations intermédiaires peuvent jouer un rôle important même si vous souhaitez simplement développer des algorithmes à un niveau plus théorique.

Par exemple, une application activée par QIR consiste à utiliser [le compilateur](#) Clang, un front-end de langage C pour LLVM, pour compiler QIR en code d'ordinateur exécutable pour un langage classique target. Cela permet de créer facilement un simulateur en C ou C++ en implémentant les instructions quantiques, ce qui peut simplifier la création de simulateurs quantiques.

En outre, vous pouvez utiliser la représentation intermédiaire pour générer du code fourni ultérieurement en tant qu'entrée dans un simulateur quantique (au lieu d'un appareil réel) qui pourrait potentiellement utiliser un langage différent du code source. De cette façon, vous pouvez facilement comparer et évaluer différents langages ou simulateurs à l'aide d'une infrastructure commune.

En termes d'optimisation du code, il existe des étapes d'optimisation qui peuvent être effectuées au niveau intermédiaire qui peuvent rendre l'implémentation globale de l'algorithme plus efficace. L'examen de cette optimisation de votre code d'entrée peut vous aider à mieux comprendre où rendre les algorithmes plus efficaces et comment améliorer les langages de programmation quantique.

Une autre application consiste à utiliser l'infrastructure LLVM standard « pass » pour créer des optimiseurs de code quantique qui fonctionnent sur QIR. L'approche indépendante du langage et du matériel de QIR permet de réutiliser ces optimiseurs pour différents langages de calcul et plateformes informatiques sans effort.

# Contenu connexe

- [Présentation de Q#](#)
- [Présentation de l'estimation des ressources](#)
- [Fournisseur Rigetti](#)

# Présentation des oracles quantiques

Article • 16/10/2024

Un oracle,  $O$ , est une opération non exposée utilisée comme entrée d'un autre algorithme. Souvent, ces opérations sont définies à l'aide d'une fonction classique  $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ , qui prend une *entrée binaire*  $n$  bits et produit une *sortie binaire*  $m$  bits. Pour ce faire, considérons une entrée binaire particulière  $x = (x_0, x_1, \dots, x_{n-1})$ . Vous pouvez étiqueter les états qubit en tant que  $|\vec{x}\rangle = |x_0\rangle \otimes |x_1\rangle \otimes \dots \otimes |x_{n-1}\rangle$

Vous pouvez d'abord tenter de définir  $O$  afin que  $O|x\rangle = |f(x)\rangle$ , mais cette méthode présente quelques problèmes. Tout d'abord,  $f$  peut avoir une taille différente d'entrée et de sortie ( $n \neq m$ ), de sorte que l'application  $d'O$  modifie le nombre de qubits dans le registre. Ensuite, même si  $n = m$ , la fonction peut ne pas être inversée : si  $f(x) = f(y)$  pour un  $x \neq y$ ,  $O|x\rangle \otimes |y\rangle$  mais  $\text{Extra close brace or missing open brace}$ . Cela signifie que vous ne pouvez pas construire l'opération  $d'adjoint O^\dagger$ , et les oracles doivent avoir un adjoint défini pour eux.

## Définir un oracle par son effet sur les états de base de calcul

Vous pouvez traiter ces deux problèmes en introduisant un deuxième registre de *qubits*  $m$  pour contenir la réponse. Ensuite, définissez l'effet de l'oracle sur tous les états de calcul : pour tous les  $x \in \{0, 1\}^n$  et  $y \in \{0, 1\}^m$ ,

$$O(|x\rangle \otimes |y\rangle) = |x\rangle \otimes |y \oplus f(x)\rangle.$$

Maintenant  $O = O^\dagger$  en construction et vous avez résolu les deux problèmes précédents.

### 💡 Conseil

Pour voir si  $O = O^\dagger$ , notez que  $O^2 = \mathbf{1}$  depuis  $a \oplus b \oplus b = a$  pour tout  $a, b \in \{0, 1\}$ . Par conséquent,  $O|x\rangle \otimes |y \oplus f(x)\rangle = |x\rangle \otimes |y \oplus f(x) \oplus f(x)\rangle = |x\rangle \otimes |y\rangle$ .

Plus important encore, la définition d'un oracle de cette façon pour chaque état de base de calcul  $|x\rangle \otimes |y\rangle$  définit également la manière dont  $O$  agit pour tout autre état. Ce comportement suit immédiatement le fait que  $O$ , comme toutes les opérations quantiques, est linéaire dans l'état sur lequel il agit. Considérez l'opération Hadamard,

par exemple, qui est définie  $H|0\rangle = |+\rangle$  et  $H|1\rangle = |-\rangle$ . Si vous souhaitez savoir comment  $H$  agit sur  $|+\rangle$ , vous pouvez utiliser que  $H$  est linéaire,

$$\begin{aligned} H|\text{ket}{+}\rangle &= \frac{1}{\sqrt{2}} H(|\text{ket}{0}\rangle + |\text{ket}{1}\rangle) = \frac{1}{\sqrt{2}} (H|\text{ket}{0}\rangle + H|\text{ket}{1}\rangle) \\ &= \frac{1}{\sqrt{2}} (|\text{ket}{+}\rangle + |\text{ket}{-}\rangle) = \frac{1}{2} (|\text{ket}{+}\rangle + |\text{ket}{-}\rangle + |\text{ket}{+}\rangle + |\text{ket}{-}\rangle) = |\text{ket}{0}\rangle \end{aligned}$$

Lors de la définition de l'oracle  $O$ , vous pouvez utiliser de la même façon que  $n$  qubits peuvent être écrits en tant que

$$|\text{ket}{\psi}\rangle = \sum_{x \in \{0, 1\}^n, y \in \{0, 1\}^m} \alpha(x, y) |\text{ket}{x}\rangle |\text{ket}{y}\rangle$$

Ici,  $\alpha : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \mathbb{C}$  représente les coefficients de l'état  $|\psi\rangle$ . Donc

$$\begin{aligned} O|\text{ket}{\psi}\rangle &= O \sum_{x \in \{0, 1\}^n, y \in \{0, 1\}^m} \alpha(x, y) |\text{ket}{x}\rangle |\text{ket}{y}\rangle \\ &= \sum_{x \in \{0, 1\}^n, y \in \{0, 1\}^m} \alpha(x, y) O|\text{ket}{x}\rangle |\text{ket}{y}\rangle \\ &= \sum_{x \in \{0, 1\}^n, y \in \{0, 1\}^m} \alpha(x, y) |\text{ket}{x}\rangle |\text{ket}{y \oplus f(x)}\rangle. \end{aligned}$$

## Oracles de phase

Vous pouvez également encoder  $f$  dans un oracle  $O$  en appliquant une *phase* basée sur l'entrée à  $O$ . Par exemple, vous pouvez définir  $O$  de sorte que

$$O|\text{ket}{x}\rangle = (-1)^{f(x)} |\text{ket}{x}\rangle$$

Si un oracle de phase agit sur un registre initialement dans un état de base de calcul  $|x\rangle$ , cette phase est une phase globale qui n'est donc pas observable. Toutefois, un tel oracle peut être une ressource puissante si elle est appliquée à une superposition ou en tant qu'opération contrôlée. Par exemple, considérez un oracle de phase  $O_f$  pour une fonction unique  $f$ . Ensuite,

$$\begin{aligned} O_f|\text{ket}{+}\rangle &= O_f(|\text{ket}{0}\rangle + |\text{ket}{1}\rangle) \\ &= \frac{1}{\sqrt{2}} (O_f|\text{ket}{0}\rangle + O_f|\text{ket}{1}\rangle) \\ &= \frac{1}{\sqrt{2}} ((-1)^{f(0)} |\text{ket}{0}\rangle + (-1)^{f(1)} |\text{ket}{1}\rangle) \\ &= \frac{1}{\sqrt{2}} ((-1)^{f(0)} |\text{ket}{0}\rangle + (-1)^{f(1) - f(0)} |\text{ket}{1}\rangle) \\ &= (-1)^{f(0)} \frac{1}{\sqrt{2}} (|\text{ket}{0}\rangle + Z^{f(1) - f(0)} |\text{ket}{1}\rangle) \end{aligned}$$

### Notes

Notez que  $Z^{-1} = Z^\dagger = Z$  et donc  $Z^{f(0) - f(1)} = Z^{f(1) - f(0)}$ .

Plus généralement, les deux vues d'oracles peuvent être élargies pour représenter des fonctions classiques, qui retournent des nombres réels au lieu d'un seul bit.

Le choix de la meilleure façon d'implémenter un oracle dépend fortement de la façon dont cet oracle doit être utilisé dans un algorithme donné. Par exemple, l'[algorithme Deutsch-Jozsa](#) s'appuie sur l'oracle implémenté de la même façon, tandis que l'[algorithme de Grover](#) s'appuie sur l'oracle implémenté de la seconde façon.

Pour plus d'informations, consultez la discussion dans [Gilyén 1711.00465](#).

## Contenu connexe

- [Algorithme de Grover](#)
- [Représentation intermédiaire quantique](#)
- [Vecteurs et matrices](#)
- [Portes T et usines T](#)

---

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#) | [Obtenir de l'aide sur Microsoft Q&A](#)

# Le rôle des portes T et des usines T dans l'informatique quantique

Article • 16/09/2024

Cet article décrit le rôle des portes T et des usines T dans l'informatique quantique tolérante aux pannes. Donner un algorithme quantique, l'estimation des ressources requises pour l'exécution des portes T et des usines T devient cruciale pour déterminer la faisabilité de l'algorithme. L'estimateur de ressources Azure Quantum calcule le nombre d'états T nécessaires pour exécuter l'algorithme, le nombre de qubits physiques pour une fabrique T unique et le runtime de la fabrique T.

## Ensemble universel de portes quantiques

Selon les [critères](#) de DiVincenzo, un ordinateur quantique évolutif doit être en mesure d'implémenter un **ensemble universel de portes quantiques**. Un *ensemble* universel contient toutes les portes nécessaires pour effectuer tout calcul quantique, autrement dit, tout calcul doit se décomposer en une séquence finie de portes universelles. Au minimum, un ordinateur quantique doit être en mesure de déplacer des qubits uniques vers n'importe quelle position sur la sphère Bloch (à l'aide de portes à qubit unique), ainsi que d'introduire l'enchevêtrement dans le système, ce qui nécessite une porte multi-qubit.

Quatre fonctions seulement mappent un bit à un bit sur un ordinateur classique. En revanche, sur un ordinateur quantique, il existe un nombre infini de transformations unitaires sur un seul qubit. Par conséquent, aucun ensemble fini d'opérations quantiques primitives ou de portes, ne peut répliquer exactement l'ensemble infini de transformations unitaires autorisées dans l'informatique quantique. À la différence d'un ordinateur classique, un ordinateur quantique ne peut donc pas implémenter exactement chaque programme quantique possible avec un nombre fini de portes. Ainsi, les ordinateurs quantiques ne peuvent pas être universels au même titre que les ordinateurs classiques. Quand nous disons qu'un ensemble de portes est *universel* en informatique quantique, cela n'a donc pas précisément la même portée qu'en informatique classique.

Pour l'universalité, il est nécessaire qu'un ordinateur quantique se rapproche *uniquement* de chaque matrice unitaire dans une erreur finie à l'aide d'une séquence de porte de longueur finie.

En d'autres termes, un ensemble de portes est universel si une transformation unitaire quelconque peut être approximativement écrite comme produit des portes de cet

ensemble. Il est nécessaire que pour toute erreur prescrite liée, il existe des portes  $G_1, G_2, \dots, G_N$  de l'ensemble de portes  $\{1\}$  de telle sorte que

$$G_N G_{N-1} \cdots G_2 G_1 \approx U.$$

Étant donné que la convention de multiplication de matrice consiste à multiplier de droite à gauche la première opération de porte dans cette séquence,  $G_N$ , est en fait la dernière appliquée au vecteur d'état quantique. Plus formellement, cet ensemble de portes est dit « universel » si, pour chaque tolérance d'erreur  $\epsilon > 0$ , il existe  $G_1, \dots, G_N$  de sorte que la distance entre  $G_N \dots G_1$  et  $U$  ne dépasse pas  $\epsilon$ . Dans l'idéal, la valeur de  $N$  nécessaire pour atteindre cette distance d' $\epsilon$  doit croître de façon poly-logarithmique avec  $1/\epsilon$ .

Par exemple, l'ensemble formé par hadamard, CNOT et T gates est un ensemble universel, à partir duquel tout calcul quantique (sur n'importe quel nombre de qubits) peut être généré. Le Hadamard et le jeu de portes T génèrent n'importe quelle porte à qubit unique :

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$$

Dans un ordinateur quantique, les portes quantiques peuvent être classées en deux catégories : les portes clifford et les portes non-Clifford, dans ce cas, la porte T. Les programmes quantiques fabriqués à partir de seules portes Clifford peuvent être simulés efficacement à l'aide d'un ordinateur classique, et par conséquent, les portes non-Clifford sont requises pour obtenir un avantage quantique. Dans de nombreux schémas de correction d'erreurs quantiques (QEC), les portes de Clifford sont faciles à implémenter, c'est-à-dire qu'elles nécessitent très peu de ressources en termes d'opérations et de qubits pour implémenter la tolérance de panne, alors que les portes non-Clifford sont assez coûteuses lors de la nécessité d'une tolérance de panne. Dans un ensemble de portes quantiques universelles, la porte T est couramment utilisée comme porte non-Clifford.

L'ensemble standard de portes à qubit Clifford, inclus par défaut dans  $Q^\#$ , include

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad S =$$

$$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$$

$$= T^2, \quad X =$$

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$



$$= HT^4H,\$ \$$$

$$Y = \begin{bmatrix} 0 & ; -i \\ amp & ; 0 \end{bmatrix} = T^2HT^4HT^6, \quad Z = \begin{bmatrix} 1 & ; 0 \\ 0 & ; -1 \end{bmatrix} = T^4.$$

Avec la porte non-Clifford (la porte T), ces opérations peuvent être composées de manière à estimer toute transformation unitaire sur un qubit unique.

## Fabriques T dans l'estimateur de ressources Azure Quantum

La préparation de la porte non-Clifford T est cruciale car les autres portes quantiques ne sont pas suffisantes pour le calcul quantique universel. Pour implémenter des opérations non-Clifford pour des algorithmes à l'échelle pratique, des portes T à faible taux d'erreur (ou états T) sont requises. Toutefois, ils peuvent être difficiles à implémenter directement sur des qubits logiques et peuvent également être difficiles pour certains qubits physiques.

Dans un ordinateur quantique tolérant aux pannes, les états T de faible taux d'erreur requis sont produits à l'aide d'une fabrique de distillation d'état T ou D'usine T pour un court terme. Ces fabriques T impliquent généralement une séquence d'arrondis de distillation, où chaque tour prend de nombreux états T bruyants encodés dans un code de distance plus petit, les traite à l'aide d'une unité de distillation et génère moins d'états T bruyants encodés dans un code de distance plus grand, avec le nombre d'arrondis, d'unités de distillation et de distances qui peuvent être variées. Cette procédure est itérée, où les états T de sortie d'un tour sont alimentés dans le tour suivant en tant qu'entrées.

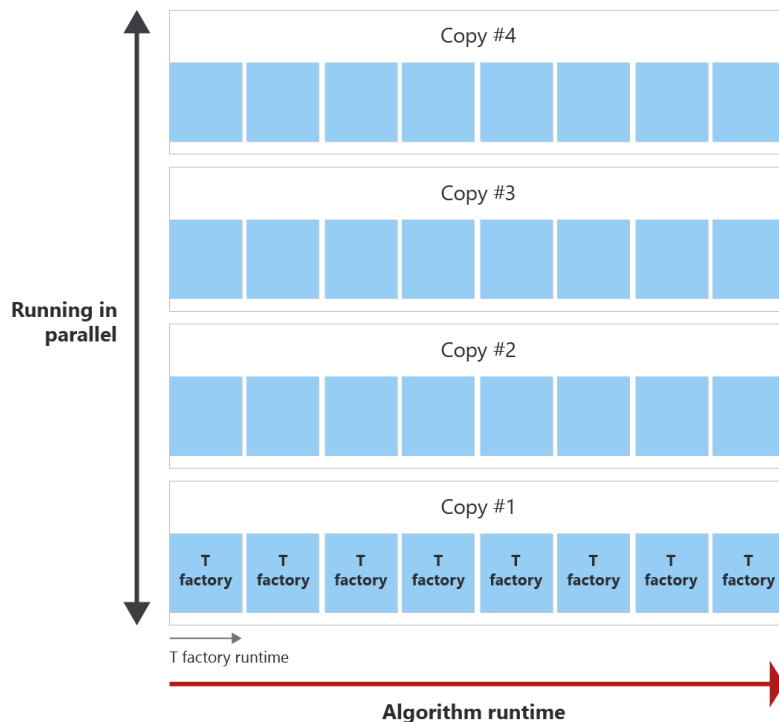
En fonction de la durée de la fabrique T, l'estimateur de ressources Azure Quantum détermine la fréquence à laquelle une fabrique T peut être appelée avant qu'elle dépasse le runtime total de l'algorithme, et ainsi le nombre d'états T pouvant être générés pendant le runtime de l'algorithme. En règle générale, plus d'états T sont requis que ce qui peut être produit dans les appels d'une fabrique T unique pendant le runtime de l'algorithme. Pour produire plus d'états T, l'estimateur de ressources utilise des copies des usines T.

## Estimation physique de la fabrique T

L'estimateur de ressources calcule le nombre total d'états T nécessaires pour exécuter l'algorithme, ainsi que le nombre de qubits physiques pour une fabrique T unique et son runtime.

L'objectif est de produire tous les états T au sein du runtime d'algorithme avec le plus peu de copies de fabrique T que possible. Le diagramme suivant illustre un exemple du runtime de l'algorithme et du runtime d'une fabrique T. Vous pouvez voir que le runtime de la fabrique T est plus court que le runtime de l'algorithme. Dans cet exemple, une fabrique T peut distiller un état T. Deux questions se posent :

- À quelle fréquence la fabrique T peut-elle être appelée avant la fin de l'algorithme ?
- Combien de copies du cycle de distillation de la fabrique T sont nécessaires pour créer le nombre d'états T requis pendant le runtime de l'algorithme ?



Avant la fin de l'algorithme, la fabrique T peut être appelée huit fois, ce qui est appelé round de distillation. Par exemple, si vous avez besoin de 30 états T, une fabrique T unique est appelée huit fois pendant l'exécution de l'algorithme et crée donc huit états T. Ensuite, vous avez besoin de quatre copies du cycle de distillation de la fabrique T en parallèle pour distiller les 30 états T nécessaires.

#### 📌 Notes

Notez que les copies T factory et les appels de fabrique T ne sont pas les mêmes.

Les usines de distillation de l'état T sont implémentées dans une séquence d'arrondis, où chaque tour se compose d'un ensemble de copies d'unités de distillation s'exécutant en parallèle. L'estimateur de ressources calcule le nombre de qubits physiques nécessaires pour exécuter une fabrique T et la durée pendant laquelle la fabrique T s'exécute, entre autres paramètres requis.

Vous ne pouvez effectuer que des appels complets d'une fabrique T. Par conséquent, il peut arriver que le runtime cumulé de tous les appels de fabrique T soit inférieur au runtime d'algorithme. Étant donné que les qubits sont réutilisés par des arrondis différents, le nombre de qubits physiques pour une fabrique T est le nombre maximal de qubits physiques utilisés pour un tour. Le runtime de la fabrique T est la somme du runtime dans toutes les rondes.

#### ⓘ Notes

Si le taux d'erreur de porte T physique est inférieur au taux d'erreur d'état T logique requis, l'estimateur de ressources ne peut pas effectuer une bonne estimation des ressources. Lorsque vous envoyez un travail d'estimation des ressources, vous pouvez rencontrer que la fabrique T est introuvable, car le taux d'erreur d'état T logique requis est trop faible ou trop élevé.

Pour plus d'informations, consultez l'annexe C de l'évaluation des [exigences pour effectuer une mise à l'échelle vers un avantage](#)  quantique pratique.

## Contenu connexe

- [Circuits quantiques](#)
- [Oracles quantiques](#)
- [Algorithme de Grover](#)
- [Représentation intermédiaire quantique](#)

## Commentaires

Cette page a-t-elle été utile ?

 Yes

 No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Concepts de l'informatique quantique hybride

Article • 16/09/2024

L'informatique quantique est hybride en quelque sorte. Les premiers systèmes quantiques utilisaient des processus classiques pour envoyer des instructions à l'ordinateur quantique, et recevoir et traiter les données obtenues. Les architectures hybrides les plus avancées apportent une intégration plus étroite et plus riche entre le calcul classique et quantique, accélérant le temps d'exécution et ouvrant la porte à une nouvelle génération d'algorithmes. Par exemple, l'informatique classique est souvent plus rapide et plus efficace que l'informatique quantique pour certaines tâches, telles que le traitement et l'analyse des données. Toutefois, l'informatique quantique fonctionne mieux pour certains types d'optimisation et de problèmes de simulation.

## Registres classiques et quantiques

Dans l'informatique quantique, même si les processeurs classiques et quantiques sont étroitement intégrés, ils sont toujours des entités physiques distinctes, et les programmes de calcul quantique hybride peuvent tirer parti de chacune de leurs fonctionnalités.

Un registre classique utilise l'architecture familière basée sur la puce de silicium et convient le mieux aux opérations telles que l'envoi d'instructions au processeur quantique, la capture des résultats de mesure et l'utilisation de ces résultats pour déterminer le prochain ensemble d'instructions.

Un registre quantique est un système composé de plusieurs qubits. Les ordinateurs quantiques excellent pour effectuer des calculs complexes en manipulant leurs qubits au sein d'un registre quantique.

## Mesure du circuit moyen

La mesure du circuit intermédiaire est le processus d'exécution de mesures d'état quantique à différents points pendant l'exécution du programme, plutôt qu'à la fin. Ces mesures sont utilisées pour obtenir des informations sur les états intermédiaires du système et sont utilisées par le code classique pour prendre des décisions en temps réel sur le flux du programme. Les mesures intermédiaires agissent également comme des « vérifications d'intégrité » de correction d'erreurs en vérifiant l'état du circuit à un certain point avant de se déplacer, et sont étroitement liées à la réutilisation du qubit.

# Réutilisation du Qubit

Les ordinateurs quantiques d'aujourd'hui peuvent prendre en charge un nombre croissant de qubits. Toutefois, nous sommes encore loin d'être en mesure de prendre en charge les millions de qubits nécessaires pour exécuter des calculs totalement tolérants aux pannes. Pour plus d'informations sur l'échelle des ordinateurs quantiques nécessaires, consultez la rubrique Présentation de l'estimation des ressources. En outre, il est souhaitable d'utiliser autant de qubits que possible.

La réutilisation de Qubit est la pratique de la conception de circuits pour utiliser le même qubit plusieurs fois dans un calcul quantique pour réduire le nombre total de qubits nécessaires pour exécuter votre programme. Par exemple, après avoir effectué une [mesure](#) de circuit intermédiaire et traité le résultat, ce qubit peut être réinitialisé et réutilisé pour un autre calcul au lieu d'allouer un nouveau qubit. Il existe différentes techniques pour réutiliser des qubits dans l'informatique quantique, telles que la téléportation quantique, la correction des erreurs quantiques et l'informatique quantique basée sur la mesure.

## Atténuation des erreurs

Pour rendre le matériel quantique actuel plus robuste par rapport aux erreurs et au bruit, *les qubits logiques* peuvent être utilisés. Les qubits logiques sont créés à l'aide de plusieurs qubits physiques pour encoder et protéger les informations quantiques. Toutefois, étant donné que plusieurs qubits doivent être utilisés pour créer un qubit logique, le nombre global de qubits pouvant être utilisés pour les calculs est réduit. À mesure que la capacité du matériel à prendre en charge davantage de qubits physiques augmente, les fonctionnalités de tolérance de panne seront donc disponibles.

Outre l'utilisation de qubits logiques, les erreurs dans les calculs quantiques peuvent être atténuées à l'aide de techniques telles que plusieurs mesures, la conception d'algorithmes qui réduisent le nombre d'opérations nécessaires ou l'ajustement des paramètres sur les portes quantiques pour réduire l'impact du bruit.

## Correction des erreurs et tolérance de panne

La correction des erreurs et la tolérance de panne sont des aspects critiques de l'informatique quantique, car les qubits sont plus sujets aux erreurs que les bits classiques en raison de la nature délicate des états quantiques, et des systèmes à tolérance de panne sont nécessaires pour obtenir les avantages complets de [l'informatique](#) quantique hybride distribuée. Dans l'informatique classique, les erreurs peuvent être corrigées en ajoutant la redondance au calcul et en utilisant des codes de

correction d'erreur. Toutefois, les techniques traditionnelles de correction des erreurs ne s'appliquent pas directement à l'informatique quantique, car elles s'appuient sur la possibilité de répéter le calcul plusieurs fois, ce qui n'est pas possible dans l'informatique quantique en raison du [théorème](#) sans clonage.

Les codes Floquet sont une nouvelle classe de codes de correction d'erreur qui répondent dynamiquement au bruit et aux erreurs, au lieu de codes de correction traditionnels qui protègent contre les erreurs statiques. Pour plus d'informations, consultez [Correction des erreurs avec des codes](#) [↗](#) Floquet.

## Algorithmes hybrides

- **Variational Quantum Eigensolver (VQE)** - Un algorithme quantique pour la chimie quantique, les simulations quantiques et les problèmes d'optimisation et est utilisé pour trouver l'état terrestre d'un système physique donné. L'ordinateur classique est utilisé pour définir des circuits quantiques avec certains paramètres. Une fois l'état quantique mesuré, l'ordinateur classique évalue comment améliorer les paramètres, puis soumettre à nouveau les circuits. Les VQE sont généralement des programmes de longue durée qui peuvent tirer parti de l'intégration plus étroite de l'informatique quantique hybride.
- **Quantum Approximate Optimization Algorithm (QAOA)** - également un algorithme quantique variationnel, il est utilisé pour trouver des solutions approximatives aux problèmes d'optimisation combinatoire - problèmes où le nombre de solutions possibles augmente extrêmement grande avec la taille du problème. Il s'agit d'un domaine de recherche actif pour identifier son applicabilité aux applications telles que le contrôle de la circulation aérienne, l'expédition ou les itinéraires de livraison ou les optimisations financières.
- **Estimation de phase itérative** : est une autre méthode d'exécution *de l'estimation* de phase quantique, qui estime la phase d'un opérateur unitaire et est utilisée dans de nombreux autres algorithmes quantiques. Les deux méthodes utilisent une série de portes de rotation pour déterminer la phase, mais l'estimation de phase itérative utilise le registre classique pour stocker des informations et des calculs sur les mesures de porte. Cela réduit le nombre de qubits requis et réduit le bruit et les erreurs.

Pour plus d'informations sur les algorithmes quantiques, visitez le zoo [d'algorithmes](#) [↗](#) quantiques.

---

## Commentaires

Cette page a-t-elle été utile ?



Yes



No

[Indiquer des commentaires sur le produit](#)  | [Obtenir de l'aide sur Microsoft Q&A](#)

# Contribution au Kit de développement Microsoft Quantum

Merci d'être une partie de la communauté Microsoft Quantum, nous sommes heureux de vos contributions !

- Si vous souhaitez contribuer au code au Kit de développement Microsoft Quantum ou signaler un bogue, consultez [Contribution à Q#](#) .
- Si vous souhaitez contribuer à la documentation du Kit de développement Microsoft Quantum, consultez [Comment contribuer à la documentation](#).

---

Last updated on 29/01/2026



# Lecture supplémentaire : Ressources d'apprentissage sur l'informatique quantique

Cet article compile certaines des ressources les plus populaires qui pourraient vous être utiles dans votre apprentissage de l'informatique quantique.

## Ressources d'informatique quantique Microsoft

Découvrez comment développer et appliquer des solutions d'informatique quantique avec les services Microsoft Quantum Development Kit (QDK) et Azure Quantum.

- [Parcours](#) d'apprentissage Azure Quantum : parcours d'apprentissage interactif, gratuit et pratique. Dans ces modules, vous allez découvrir l'informatique quantique et comment développer des solutions quantiques à l'aide Q# et le QDK.
- [Katas quantiques](#)  : collection de tutoriels de programmation quantique Q# auto-rythmés.
- [Vidéos Microsoft Quantum](#)  : une playlist avec des vidéos d'annonces Microsoft Quantum, des démonstrations et des discussions à partir de *Quantum Innovator Series*.
- [Exemples de codeQ#](#)  : commencez à créer votre première solution quantique avec cette collection d'exemples de code prêts à l'emploi.
- [Blog Q#](#)  : blog écrit par les développeurs pour les développeurs. Vous pouvez en savoir plus sur les derniers QDK et Q# insights, et en savoir plus sur les défis quantiques et les annonces de hacks.
- [Publications de recherche](#)  : découvrez les dernières avancées dans le domaine du matériel quantique et des algorithmes développés par les chercheurs Microsoft.

Ces ressources sur l'informatique quantique sont disponibles dans la [page d'apprentissage quantique de Microsoft](#) .

## Contenu réalisé par la communauté Q#

Les ressources suivantes sont créées et développées par la communauté quantique qui est excitée par la programmation quantique.

### Livres créés par la communauté

- [Introduction à Quantum Computing avec Q# et QDK](#) , Filip Wojcieszyn, Springer, 2022.

- [Q# Guide](#) de poche, Mariia Mykhailova, O'Reilly, 2021. Q# les exemples mis à jour sont [ici](#).
- [Présentation de Microsoft Quantum Computing for Developers](#), Johnny Hooyberghs, Springer, 2021.
- [Découvrez l'informatique quantique avec Python et Q# - Une approche](#) pratique, S. Kaiser et C. Granade. Publications de Manning, 2021.

## Blogs créés par la communauté

- [Awesome qsharp](#) : liste open source de ressources et de codes Q#.
- [Q# Communauté](#) : espace GitHub pour les projets pilotés par la communauté.

## Forums et communautés pour les développeurs quantiques

[Quantum computing StackExchange](#) : communauté en ligne destinée aux développeurs quantiques pour apprendre et partager leurs connaissances.

## Cours d'informatique quantique

Consultez les cours suivants consacrés à l'apprentissage de l'informatique quantique.

- [Quantum Computing avec Microsoft QDK](#) : série de liveProjects qui vous aident à apprendre le développement de logiciels quantiques en créant des projets de bout en bout. Vous explorez le plein potentiel du chiffrement, de la transmission des données, de la reconstruction des données, etc.
- [Quantum Computing - Cours brillant](#) : apprendre à créer des algorithmes quantiques à partir du terrain avec un ordinateur quantique simulé dans votre navigateur dans ce cours, créé en collaboration avec des chercheurs quantiques et des praticiens de Microsoft, X et Caltech's Institute for Quantum Information and Matter (IQIM).
- [Quantum Computing through Comics - HackadayU classes](#) : découvrez les concepts de l'informatique quantique et la programmation d'algorithmes au cours d'une discussion en classe complétée par des bandes dessinées intuitives.

## Bibliographie

La bibliographie suivante est une collection de publications qui couvrent un large éventail de sujets d'informatique quantique.

## Informatique quantique pour les débutants

Si vous êtes un passionné quantique et souhaitez commencer à apprendre la théorie derrière l'informatique quantique, les ressources suivantes proposent des sujets tels que la physique quantique, la science informatique et l'algèbre linéaire.

- Nielsen, M. A. & Chuang, I. L. *Calcul quantique et informations quantiques*. Calcul quantique et informations quantiques. Royaume-Uni : Cambridge University Press, 2010.
- Kaye, P., Laflamme, R., & Mosca, M. *Introduction à l'informatique quantique*. Oxford University Press, 2007.
- Rieffel, E. G., & Polak, W. H. *Informatique quantique : une introduction douce*. MIT Press, 2011.

## Différents types de qubits

- Sergey Bravyi, Oliver Dial, Jay M. Gambetta, Dario Gil et Zaira Nazario. *L'avenir de l'informatique quantique avec des qubits superconducteurs*, 2022.
- Microsoft Quantum. *InAs-Al Hybrid Devices Passing the Topological Gap Protocol*, arXiv :2207.02472 [cond-mat.mes-hall], (2022).
- M Saffman. *Informatique quantique avec interactions atomiques et rydberg : progression et défis*, Journal of Physics B : Atomic, Molecular, and Optical Physics, 49(20) : 202001, (2016).
- J. I. Cirac et P. Zoller. *Calculs quantiques avec des ions piégés à froid*, Phys. Rev. Lett., 74:4091-4094 (1995).

## Correction des erreurs quantiques

- Michael Beverland, Vadym Kliuchnikov et Eddie Schoute. *Compilation de code Surface via des chemins d'accès disjoints*, PRX Quantum, 3:020342, (2022) .
- Adam Paetznick, Christina Knapp, Nicolas Delfosse, Bela Bauer, Jeongwan Haah, Matthew B. Hastings et Marcus P. da Silva. *Performances des codes de floquet planar avec des qubits basés sur majorana*, 2022.
- Austin G. Fowler, Matteo Mariantoni, John M. Martinis et Andrew N. Cleland. *Codes de surface : Vers un calcul quantique à grande échelle pratique*, Phys. Rev. A, 86:032324, (2012).
- Daniel Gottesman. *Introduction à la correction des erreurs quantiques et au calcul quantique à tolérance de panne*. Dans la science de l'information quantique et ses contributions aux mathématiques, Proceedings of Symposia in Applied Mathematics, volume 68, pages 13 à 58, (2010).

## Estimation des ressources

- M. E. Beverland, P. Murali, M. Troyer, K. M. Svore, T. Hoefler, V. Kliuchnikov, G. H. Low, M. Soeken, A. Sundaram et A. Vashchillo. *Évaluation des exigences de mise à l'échelle vers un*

*avantage* quantique pratique, arXiv :2211.07629v1, 2022.

- Isaac H. Kim, Ye-Hua Liu, Sam Pallister, William Pol, Sam Roberts et Eunseok Lee. Estimation des ressources à tolérance de panne pour les simulations chimiques quantiques : étude de cas sur les molécules d'électrolyse de batterie li-ion. *Phys. Rev. Research*, 4:023019, Avril 2022.
- Julia Meuli, Mathias Soeken, Martin Roetteler et Thomas Häner. *Activation des compilateurs quantiques prenant en charge la précision à l'aide de l'estimation des ressources symboliques*, Proc. Programme ACM. Lang., 4(OOPSLA), 2020.

## Informatique quantique tolérante aux pannes

- Hector Bombin, Chris Dawson, Ryan V. Mishmash, Naomi Nickerson, Fernando Pastawski et Sam Roberts. *Blocs logiques pour le calcul quantique topologique à tolérance de panne*, 2021.
- Antonio D. Córcoles, Abhinav Kandala, Ali Javadi-Abhari, Douglas T. McClure, Andrew W. Cross, Kristan Temme, Paul D. Nation, Matthias Steffen et Jay M. Gambetta. *Défis et opportunités des systèmes de calcul quantique à court terme*, Procédures de l'IEEE, 108(8) : 1338-1352 (2020).
- Michael Edward Beverland. *Vers des ordinateurs quantiques réalistes*, thèse de doctorat, California Institute of Technology, 2016.
- Peter W Shor. *Calcul quantique tolérant aux pannes*. Dans Les procédures de la 37e conférence sur les fondements de la science informatique, pages 56 à 65. IEEE (1996).

## Chimie quantique

- J. Tilly, Hong milliseconde Chen, Shu huang Cao, D. Picozzi, K. Setia, Ying Li, E. Grant, L. Wossnig, I. Rungger, G. Booth, J. Tennyson. *The Variational Quantum Eigensolver : a review of methods and best practices*, arXiv :2111.05176v3 [quant-ph], 2022.
- V. von Burg, Guang Hao Low, T. Häner, D.S. Steiger, M. Reiher, M. Roetteler et M. Troyer. *L'informatique quantique a amélioré la catalyse de calcul*. *Phys. Rev. Research* 3, 033055 (2021).
- Bela Bauer, Sergey Bravyi, Mario Motta et Garnet Kin-Lic Chan. *Algorithmes quantiques pour la chimie quantique et la science des matériaux quantiques*, Examens chimiques, 120(22) : 12685-12717 (2020).